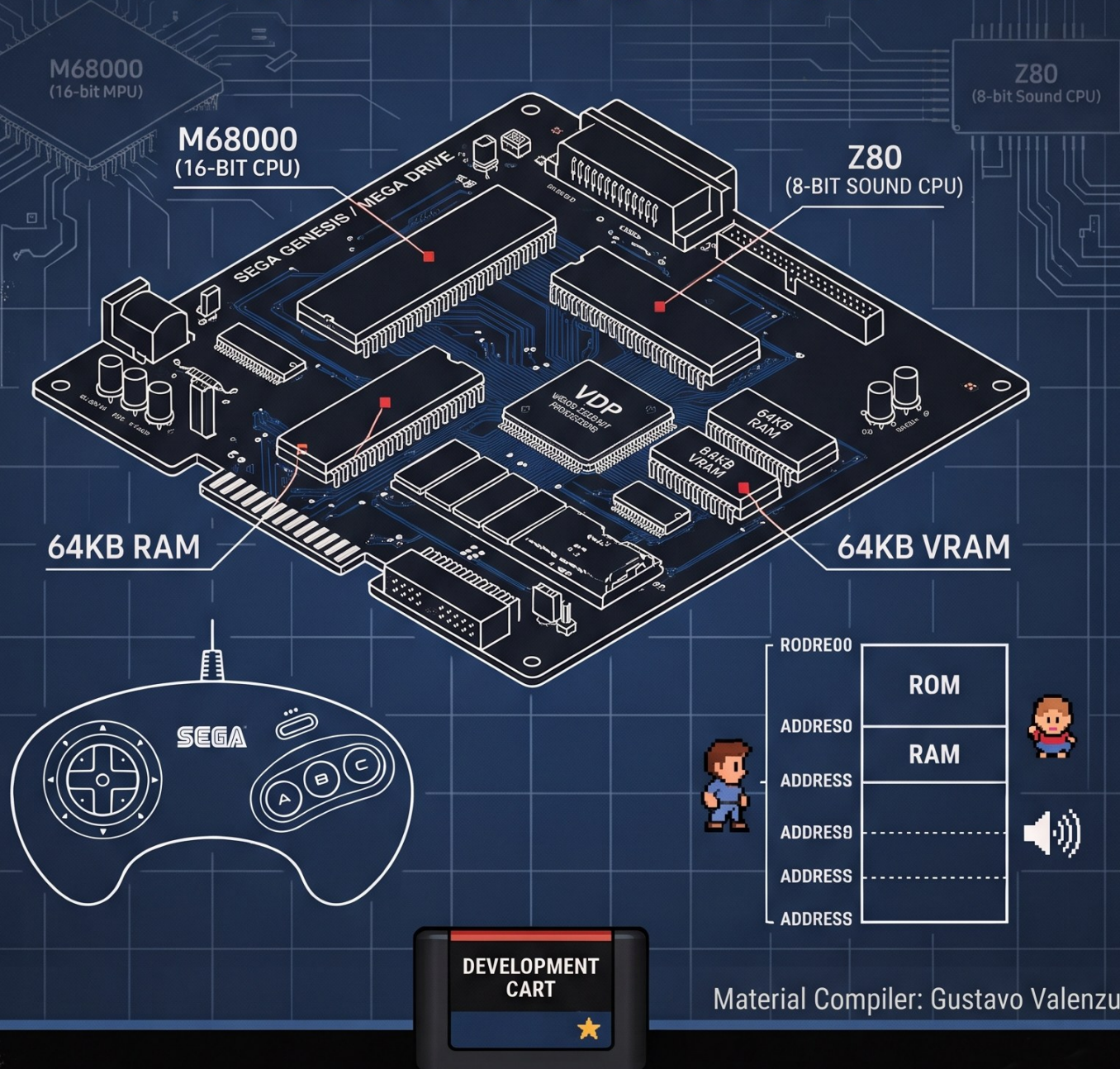


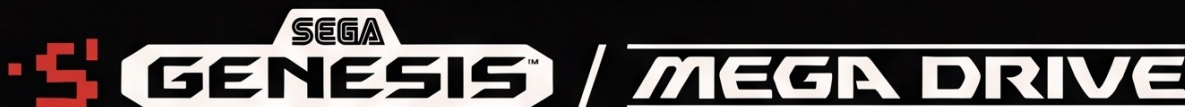
SOFTWARE DEVELOPMENT MANUAL

SEGA GENESIS / MEGA DRIVE

16-BIT PROGRAMMING GUIDE



Material Compiler: Gustavo Valenzuela



SEGA ENTERPRISES, LTD. / © 1989-1994

Author's Note

I was born in 1984. I don't quite remember how I discovered video games, but I do know that my first console was a Famicom clone made by a brand that sold them in my country. My aunt and uncle gave it to me, and it became one of the centers of my life for several years. But one afternoon, at a friend's house, surrounded by other friends (we all went to the same school), I watched Mortal Kombat running on a Sega Mega Drive clone. At that moment I knew nothing about consoles, clones, or companies... but I was fascinated. It was a before and after. I knew I had to have that at home, no matter what.

Unfortunately, it wasn't that simple. After wearing her down, my mom bought me a Sega Genesis (which I'm sure cost her a fortune, being a single mother; my dad passed away when I was about 5 years old). I wore that console out. It literally broke. After that, my interests kept shifting: I'm a person with many hobbies, and they've spent my whole life competing for my time.

A few years ago, though, one of them turned into a goal I hope to accomplish someday: porting one of the games I played — first on the NES, and later in arcades — that I always thought was a shame never got a Genesis release: the arcade TMNT, the first one Konami put out. While looking for a way to take on this monumental task, I found SGDK as the tool and Pigsy's video tutorials as a companion. I'll get there someday — I know it's possible. This manual was born from that ongoing drive.

I want to thank my mom, Rita, for sacrificing so much to buy me that console back then, and my wife, Aldana, for putting up with my hobbies. I hope you enjoy this book.

Gustavo Valenzuela

Contents

Author's Note	iii
Acknowledgments	1
Chapter 1 — History, Specifications, and Architecture of the Mega Drive	3
Chapter 2 — The VDP in Depth - VRAM, CRAM, VSRAM, Planes, Tiles, and Palettes	6
Chapter 3 — Installing and Configuring the SGDK 2.11 Environment	9
Chapter 4 — Editor, Extension, and Emulators	11
Chapter 5 — First Program - SGDK Project Structure and Compilation	13
Chapter 6 — C Applied to Retro Development - Data Types, Limited Memory, and Pointers	16
Chapter 7 — How to Display Backgrounds on the Mega Drive	20
Chapter 8 — Creating Graphics with Aseprite	29
Chapter 9 — Scroll and Parallax Scrolling	37
Chapter 10 — H40 and H32 Resolution Modes	47
Chapter 11 — Large Level Maps with MAP_create	52
Chapter 12 — Level Maps with Tiled and TMX Files	57
Chapter 13 — Creating Maps with 16Tile	61
Chapter 14 — Multiple Palettes on the Same Background with 16Tile	67
Chapter 15 — Depth via Tile Priority	77
Chapter 16 — Manually Assigning Tile Priority Without a Script	83
Chapter 17 — Tile Scrolling	95
Chapter 18 — Background Tile Animation	102
Chapter 19 — Complex Level Maps with Aseprite	113
Chapter 20 — Converting Many-Color Images to the Mega Drive Palette	122

Chapter 21 — How to Display Sprites	132
Chapter 22 — Creating Sprites and Animation Sheets	138
Chapter 23 — Sprite Animation	150
Chapter 24 — 2D Camera	155
Chapter 25 — Parallax Scrolling Over a Camera	164
Chapter 26 — Transparency with Shadow and Highlight	169
Chapter 27 — Color Transparency Without Shadow and Highlight	175
Chapter 28 — Ranger X-Style Dynamic Lighting	183
Chapter 29 — Palette Cycling Animations	189
Chapter 30 — Palette Fades	197
Chapter 31 — D-Pad Controls	204
Chapter 32 — Button Controls and Event-Based Reading	212
Chapter 33 — 2-Player Mode	221
Chapter 34 — The Sound Chips - YM2612 and PSG	230
Chapter 35 — Music Playback - XGM and XGM2 Drivers	234
Chapter 36 — Creating Music for the Mega Drive	238
Chapter 37 — Programming Sound Effects	245
Chapter 38 — Creating Sound Effects	250
Chapter 39 — The Resource Compiler in Depth	258
Chapter 40 — Level Collision for a Top-Down 2D Game	263
Chapter 41 — Jump and Gravity for a 2D Platformer	272
Chapter 42 — Level Collision for Larger Sprites	279
Chapter 43 — Semi-Solid Platforms	287
Chapter 44 — DMA - Direct Memory Transfers	294
Chapter 45 — Managing VRAM Limits	300
Chapter 46 — The New SGDK 2.x VRAM Configuration	305
Chapter 47 — Reducing Tiles and Saving VRAM	311
Chapter 48 — Saving VRAM Through Manual Sprite Loading	320
Chapter 49 — Profiling and CPU Load	332
Chapter 50 — VDP Planes and Enemies in a Beat 'em Up	337

Chapter 51 — Organizing the Project with Header Files	346
Chapter 52 — Adding Enemies to the Game	354
Chapter 53 — C Structs Applied to Game Entities	362
Chapter 54 — C Enumerations (enums)	369
Chapter 55 — Permanent Enemy Types	374
Chapter 56 — Function Pointers for Enemy AI	381
Chapter 57 — Game States	390
Chapter 58 — Loading Different Enemies per Level	398
Chapter 59 — Preparing Level Graphics	407
Chapter 60 — Transitioning Between Levels in the Game	414
Chapter 61 — Loading a New Level's Graphics	421
Chapter 62 — Wrapping Up and Extending the Project	430
Chapter 63 — HInterrupts and Raster Effects	433
Chapter 64 — Saving Games - SRAM and Flash Save	440
Chapter 65 — Multitasking and Advanced SGDK 2.x Features	446
Chapter 66 — Raylib as an Alternative for Porting Games to PC	450
Chapter 67 — Scope Note - Sega CD and 32X	456
Chapter 68 — Final Project - BOINGMD, a Pong Clone from Start to Finish	458

Acknowledgments

No manual about a console more than three decades old is written from scratch. This one in particular exists thanks to the work — often unpaid and sustained over years — of a handful of people who decided to document, teach, and keep homebrew development for the Sega Mega Drive alive. Before beginning the journey through its 68 chapters, it’s worth naming the people who made it possible.

The first thanks go to **Stéphane Dallongeville**, creator and maintainer of SGDK (Sega Genesis Development Kit), the tool that every line of code in this manual rests on. SGDK turned what once required programming directly in Motorola 68000 assembly into an accessible C workflow, without sacrificing the level of hardware control that a Mega Drive project needs. Maintaining a complete development kit — VDP, DMA, sound, sprites, the resource compiler, and everything else — for so many years, and for free, is work on a scale that is rarely recognized enough. Everything explained in Part VI about `rescomp`, in Part VIII about DMA and VRAM limits, and in general any `VDP_`, `SPR_`, `SND_`, or `DMA_` function mentioned throughout this manual exists thanks to that work.

Equally important thanks go to **Pigsy**, of *Pigsy’s Retro Game Dev Tutorials*, whose extensive video series on SGDK development served as the source material from which much of this manual was compiled and adapted, especially the parts covering graphics, sprites, sound, collision physics, and the capstone project of an action game with enemies. The screenshots illustrating the various chapters of this manual are taken from that video series, and are included here with explicit credit to that authorship. Anyone who wants to dig deeper into any of the topics covered by this manual, watch the development process on video, or simply support that work can find the channel and its social media at the following links:

- YouTube: [Pigsy’s Retro Game Dev Tutorials](#)
- X / Twitter: [@PigsyRetro](#)
- Patreon: [Pigsy’s Retro Game Dev Tutorials](#)

Thanks are also due to **Andrej Preradovic**, of **Ohsat Games**, whose series of SGDK tutorials — Megapong, Megatiler, Megarunner, and Megalaga, among others — served as an entry point into Mega Drive programming for a large part of the Spanish- and English-speaking community for years, at a time when in-depth material of that kind barely existed. The fact that those tutorials have become outdated as SGDK versions have moved on takes nothing away from the educational effort they represented, nor from the number of developers who took their first steps thanks to them; this manual, in fact, adopts some of that same didactic spirit in trying to stay relevant against future versions of SGDK.

Finally, a more general but no less sincere thanks to the entire ecosystem of the SGDK and Mega Drive homebrew development community: to those who maintain forums, Discord servers, and wikis where specific questions about VDP behavior or DMA timing — questions that appear in no official documentation — get answered; to those who develop and maintain the tools this manual recommends in Part I — emulators with debugging capabilities like Gens KMod and BlastEm, Tiled as a map editor, Aseprite for creating graphics, DefleMask for

music composition, and the “Genesis Code” extension for Visual Studio Code; and to those who, with flash carts like the ones in the EverDrive family, made it possible to test one’s own work on real hardware and not just in an emulator. Each of these pieces, small on its own, is part of what together makes it possible today to program for a console from 1988 with all the comforts of modern development.

This manual would not exist without that prior work. What follows is, in large part, a way of honoring it: bringing together knowledge scattered across videos, articles, and years of community experimentation into a single unified work, updated for SGDK 2.11, and designed to guide anyone starting from scratch all the way through.

On Using and Distributing This Manual

This manual is free to use, copy, and distribute, in the broadest sense of the word: no permission is needed, there are no restrictions on personal, educational, or community use, and you can share it with anyone you like at no cost. It is free, and it will stay that way.

That said, if you found it useful and would like to give something back for the work of gathering, updating, and organizing all this material into a single unified work, you can make a voluntary donation to **gustavo221b@gmail.com** (PayPal). This is absolutely not required — to be clear: the manual is, and will remain, free either way — but the option is there for anyone who wants to use it.

Chapter 1 — History, Specifications, and Architecture of the Mega Drive

What hardware lies beneath the code: processors, memory, and the division of labor that will shape every programming decision in this manual

Before writing a single line of C, it's worth understanding, even if only in broad strokes, the machine you're programming for. This isn't a trivia chapter: almost every design decision you'll make throughout this manual —why 8- or 16-bit data types are used instead of int out of habit, why there's no floating point, why sound is programmed separately from the rest of the game, why you have to manually keep track of how much video memory is being used— has a concrete reason rooted in the architecture described here. Come back to this chapter whenever a limitation feels arbitrary: it almost certainly isn't.

1. A bit of history

Sega launched the Mega Drive in Japan in 1988, as the successor to the 8-bit Master System. The console arrived in the United States in 1989 under the name Genesis (a name it kept in that market for its entire commercial life) and in Europe in 1990, already under the Mega Drive name. It was the main competitor to the Super Nintendo during the so-called “16-bit war” of the early 1990s, and it went on to sell more than 30 million units worldwide, with a catalog that includes series like Sonic the Hedgehog, Streets of Rage, Phantasy Star, Shining Force, and Golden Axe, plus a huge catalog of arcade conversions.

Over the course of its commercial life, three hardware revisions were manufactured (Mega Drive 1 in 1988, Mega Drive 2 in 1993, and Mega Drive 3 in 1998), with external and manufacturing-cost differences but full compatibility with the same game catalog. The console also had a series of expansions and peripherals: the Mega-CD (a CD reader with additional graphics and sound hardware) and the 32X (a cartridge with extra processors aimed at 3D graphics). Neither is within the scope of this manual, which focuses on the original cartridge-based Mega Drive/Genesis; if you ever move on to developing for those platforms, much of what you'll learn here —the VDP, memory organization, the SGDK workflow— remains the necessary foundation, with additional hardware layered on top.

Before the appearance of modern, free development kits like SGDK, programming for the Mega Drive required direct knowledge of the main processor's assembly language and access to official, Sega-licensed development kits, which were expensive and restricted. SGDK —created by Stéphane Dallongeville and still actively maintained today— changed that completely: it's free, open source, lets you program in C, and includes both the function library you'll use throughout this manual and the tools needed to convert graphics, maps, music, and

sound effects into data the console can use.

2. The three protagonists of the hardware

The Mega Drive splits its work among three main pieces of silicon, each with a clearly defined job. That division explains much of the design decisions you'll encounter throughout the rest of the manual.

The main processor: Motorola 68000. This is the console's brain, a 16/32-bit microprocessor running at approximately 7.67 MHz on NTSC consoles (Japan/America) and 7.6 MHz on PAL (Europe). All the C code you write with SGDK ends up compiled into instructions for this processor. The 68000 has no floating-point unit (FPU): there's no native, hardware-level way to work with decimal numbers, which is why SGDK offers fixed-point types (`fix16`, `fix32`) instead of `float` or `double` — a topic revisited in detail later in this same foundational block.

The sound coprocessor: Zilog Z80. The Z80 is a second processor, simpler than the 68000, dedicated almost entirely to handling sound: it controls the FM synthesizer and the programmable sound generator (both described below). It's also the same processor used by the Master System, which lets the Mega Drive run games from that earlier console in a special compatibility mode. For the 68000 to talk to the sound chips, it first has to "ask permission" from the Z80 by temporarily handing it control of the data bus; that bus-handoff mechanism is why, in SGDK, playing music and sound effects goes through specific functions (covered in the part of this manual dedicated to sound) instead of being written directly as if it were ordinary memory.

The video processor: VDP (Video Display Processor). The VDP draws everything that appears on screen: backgrounds, sprites, any graphic visible on the television. It has its own video memory, separate from main RAM, and is programmed through a set of control registers. Given how central it is to everything visual in a Mega Drive game, the next chapter is devoted entirely to explaining it in depth.

3. The sound chips

Within the audio subsystem, controlled by the Z80, two sound chips coexist with very different characteristics, and their combination is one of the console's most recognizable sonic trademarks:

- **Yamaha YM2612:** a 6-channel stereo FM (frequency modulation) synthesizer. It's responsible for the more "metallic" and orchestral sound characteristic of Mega Drive music, and it can also play back digitized audio samples (PCM), sacrificing one of its six channels to do so.
- **PSG (Programmable Sound Generator, an SN76489 or equivalent):** a simpler, 4-channel sound generator (three tone channels and one noise channel), inherited from the previous generation of Sega hardware. It's typically used for sound effects and to reinforce the YM2612's music with additional lines.

The part of this manual dedicated to sound explains in detail how SGDK exposes both chips through its XGM and XGM2 drivers, as well as the workflow for creating and playing music and sound effects.

4. Memory: a map of scarce resources

One of the most important habits you'll develop throughout this manual is thinking of memory as a finite resource that has to be actively managed, not something that's simply "there." The

Mega Drive splits its memory into several independent blocks, each with its own purpose and fixed size:

Memory	Size	Use
Main RAM	64 KB	Variables, data stacks, and structures for the C program
VRAM (video memory)	64 KB	Graphic tiles, plane name tables, sprite table
CRAM (palette memory)	128 bytes	4 palettes of 16 colors each
VSRAM (vertical scroll memory)	80 bytes	Per-column vertical scroll offset values
Z80 RAM	8 KB	Internal working memory for the sound subsystem
Cartridge ROM	Variable (up to several MB)	The compiled program and all its assets (graphics, maps, music)

The first four rows of this table are internal console memory: they exist in a fixed amount regardless of how large the inserted cartridge is, and they run out easily if not managed carefully. The last row, ROM, is the cartridge's own memory: that's where the compiled code lives, along with all the game's assets (images, maps, music, sound effects) converted into the format the console can read. ROM is much larger than RAM, but it has a different limitation: everything in ROM is read-only during execution, so any data the game needs to modify at runtime first has to be copied into one of the working memories (RAM, VRAM, CRAM).

This scarcity of internal memory —64 KB of main RAM is, to give a point of reference, several thousand times less than the memory of a modern phone— is behind much of the programming practices you'll see throughout the manual: choosing the smallest data type that will do for each variable, reusing graphic tiles instead of duplicating them, manually keeping track of which portion of VRAM is occupied at any given time, and preferring constant data in ROM over unnecessary copies in RAM.

Review exercises

1. Why doesn't SGDK allow you to declare a float-type variable to represent, for example, a character's position with decimals?
2. Explain, in your own words, why playing sound on the Mega Drive isn't as straightforward as writing data to a memory location: which processor is involved, and why?
3. What's the difference between the console's main RAM and the cartridge's ROM, in terms of what you can do with each during a game's execution?
4. A 4 MB cartridge has, in that sense, much more capacity than the console's 64 KB of main RAM. Why doesn't this eliminate the need to be careful with memory while programming?

Chapter 2 — The VDP in Depth - VRAM, CRAM, VSRAM, Planes, Tiles, and Palettes

The graphics vocabulary that will be used, without re-explaining it, throughout the rest of the manual

This chapter has no code: it is purely conceptual, and its sole purpose is to establish the VDP (Video Display Processor) vocabulary that every following chapter will take for granted. Everything visual in a Mega Drive game (backgrounds, sprites, text, scrolling, transparencies) passes through this chip, and it's worth understanding how it organizes its work before you start asking it to do things from C.

1. Tiles: the smallest unit of graphics

The VDP doesn't draw loose images of arbitrary size: everything that appears on screen, whether background or sprite, is built out of **tiles**, small 8×8 pixel square blocks. Each tile takes up 32 bytes of video memory, because each of its 64 pixels is represented with 4 bits (half a byte): those 4 bits aren't a color directly, but rather an index from 0 to 15 into a palette of 16 colors. This indexed-color scheme —color index in the tile, actual color in the palette— is the reason why, later on, you'll see that changing the palette assigned to a graphic completely changes its appearance without touching a single bit of the tile itself.

VRAM has 64 KB in total, which would allow up to 2048 distinct tiles to be stored (64 KB ÷ 32 bytes per tile) if all of VRAM were used exclusively for that, which doesn't happen in practice: part of that memory is reserved for the planes' name tables and for the sprite table, both explained below.

2. Palettes and CRAM: from index to actual color

The 128 bytes of CRAM (Color RAM) hold four palettes of 16 colors each (identified in SGDK as PAL0 through PAL3), and each color takes up 2 bytes with a resolution of 9 bits: 3 bits for the red component, 3 for green, and 3 for blue, giving 8 intensity levels per channel and up to 512 possible colors in total. At any given instant, however, only the 64 colors contained in the four active palettes can be on screen (4 palettes × 16 colors), and of those 64, the first color of each palette acts as the transparent color for sprites and for tiles with normal priority: in practice, this leaves 60 usable colors on screen plus the background color (index 0 of palette 0), a total traditionally cited as 61 simultaneous colors.

Changing a palette's values at runtime, without touching any tile, is a common technique

for achieving fade-ins and fade-outs or color-cycling animations; both are revisited in later chapters.

3. The planes: A, B, and Window

The VDP organizes the screen background into independent **planes**, each formed by a grid of tiles called a name table, stored in VRAM. There are three planes:

- **Plane B (BG_B)**: by default, the one furthest back in the drawing order.
- **Plane A (BG_A)**: by default, drawn in front of plane B.
- **Window**: a special plane, usually used for user interface elements (score displays, fixed text) that don't need to scroll along with the rest of the background; it's typically overlaid on top of plane A.

Each position in a plane's name table doesn't store the graphic itself, but rather a compact reference to a tile already loaded into VRAM, together with a handful of additional attributes: which palette to use to paint it (one of the four, PAL0 through PAL3), whether it has high or low priority relative to other graphical elements, and whether it should be displayed flipped horizontally or vertically. This bundle of information —palette, priority, flip, tile index— is exactly what the `TILE_ATTR_FULL()` macro assembles in SGDK, which you'll use for the first time in the chapter devoted to displaying a background on screen.

By default, the drawing order from back to front is: plane B, plane A, sprites. That order can be altered by marking a particular plane or sprite with high priority, which brings it in front of the rest of the elements that don't have that flag; this tool is the basis for depth effects such as making a character pass behind a foreground element.

Each plane also has a configurable size in tiles (for example, 64×32 or 128×64 tiles) that usually exceeds the size of the visible screen: the non-visible part of the plane remains stored in VRAM, ready to be shown as the screen scrolls over it. That movement, scrolling, is controlled independently for each plane and is the mechanism behind parallax scrolling, covered later on.

4. Sprites: graphics independent of the background plane

Unlike the tiles of a plane, a **sprite** is an independent graphical element that can be positioned at any pixel coordinate on the screen, without being tied to a plane's grid. Sprites have their own control table in VRAM (the Sprite Attribute Table, or SAT) and are assembled by combining one or more tiles: a sprite can be as large as 32×32 pixels (4×4 tiles) as a single hardware unit, although for larger characters it's common to combine several hardware sprites to form what looks like a single one from the game's point of view.

The VDP hardware imposes strict limits on the number of sprites, which depend on the horizontal resolution mode (see below): in H40 mode (320 pixels wide), up to 80 simultaneous sprites are allowed on screen, with a maximum of 20 per horizontal line; in H32 mode (256 pixels), the limits drop to 64 sprites total and 16 per line. Exceeding the per-line limit doesn't produce an error, but rather a visible flicker of the excess sprites, a very characteristic (and often unavoidable) effect of games from this console generation.

5. VSRAM and scroll modes

The 80 bytes of VSRAM (Vertical Scroll RAM) store the vertical scroll offset values for the planes. The VDP supports different scroll modes, which determine the granularity at which

each plane can be scrolled: by whole plane (the entire plane moves as a unit), by pixel line (each horizontal line of the screen can scroll independently, the basis of many parallax effects), or by 16-pixel column. This combination of modes is what allows, for example, a per-line horizontal scroll that creates a sense of depth while the vertical scroll moves by whole plane. The chapter devoted to scrolling and parallax scrolling returns to this with concrete code.

6. Resolution modes: H40 and H32

The VDP can operate in two horizontal resolution modes: H40, with 320 pixels of width (40 tiles), and H32, with 256 pixels (32 tiles), largely inherited from compatibility with the Master System. The standard vertical resolution is 224 visible lines in V28 mode (the most common, available on both NTSC and PAL consoles), although PAL consoles also support a V30 mode of 240 lines, not available on NTSC. The choice of horizontal mode isn't purely cosmetic: as mentioned above, it also changes the limits on the number of available sprites.

Review exercises

1. Explain what it means for a tile's color to be "indexed," and why that makes it possible to completely change a graphic's appearance without modifying its tiles.
2. What is the maximum number of colors that can exist across the VDP's four active palettes at any given instant? How many of those colors are actually visible on screen, and why?
3. What's the difference between a tile in a background plane and a sprite, in terms of how they're positioned on screen?
4. Why isn't switching from H40 resolution mode to H32 a purely cosmetic decision?

Chapter 3 — Installing and Configuring the SGDK 2.11 Environment

How to get your machine ready to build your first project, on Windows, Linux, and macOS

1. What SGDK Is

SGDK (Sega Genesis Development Kit) is a free, open-source set of tools and libraries that lets you program Mega Drive/Genesis games in C. It consists of three pieces: a C function library (the one used throughout this manual, included via `#include <genesis.h>`), a set of resource-conversion tools (for images, maps, music, and sound) known as `rescomp`, and a version of the GCC compiler adapted to generate code for the Motorola 68000 processor. The library and the tools are MIT-licensed; the GCC compiler itself, being a third-party project, is distributed under the GPL3 license. The project is actively maintained by Stéphane Dallongeville, and this manual is based on its latest stable release, **SGDK 2.11** (April 2025).

2. Installing on Windows

Windows is the platform with the best direct support, because the project itself distributes precompiled toolchain binaries (including GCC 13.2) ready to use.

1. **Java:** `rescomp`, the resource-conversion tool, needs Java to run. Install a recent version of a Java distribution (Oracle JDK or OpenJDK) and make sure it's available on the system `PATH`.
2. **Download:** grab the latest SGDK release from the official repository's releases page (github.com/Stephane-D/SGDK/releases) and unzip it into a folder whose path contains no spaces, for example `C:\sgdk`.
3. **Environment variable (optional but recommended):** create an environment variable named `GDK` pointing to that folder. Several tools and editor extensions (like the one covered in the next chapter) use it to automatically locate the toolchain.
4. **Verification:** open a terminal in that folder and run:

```
bin\make -f makelib.gen
```

If everything is installed correctly, the command finishes by generating the file `lib\libmd.a`, SGDK's already-compiled library. If that file appears, the environment is ready.

3. Installing on Linux and macOS

Unlike Windows, SGDK does not distribute precompiled binaries for Linux or macOS, and as of version 2.10 the project **stopped maintaining its own Docker image**, recommending instead two alternative paths:

- **MarsDev** (github.com/andwn/marsdev): a cross-platform (Windows, Linux, macOS) development environment that installs and keeps up to date both SGDK and the rest of the required toolchain, including support for the 32X if you ever need it. It's generally the recommended option for anyone not on Windows: you download the project, set an environment variable pointing to the desired installation folder, and run `make sgdk` to complete the installation.
- **Third-party Docker images**: the community maintains updated, tested Docker images, the most recommended one currently being Doragasu's (gitlab.com/doragasu/docker-sgdk), designed specifically to replace the discontinued official image. This alternative is useful if you already work with containers in your usual workflow, or if you'd rather not install the toolchain directly on your operating system.

In either case, Java remains a requirement for `rescomp`, just as on Windows.

A detail that's changed since older tutorials |
It's common to find, in tutorials and documentation from a few | years back, references to an official Docker image maintained by | the SGDK project itself. That image is no longer maintained: if | you're following a guide that mentions it, replace it with MarsDev | or with the Doragasu image mentioned above. |

4. Verifying the Installation

On Windows, Linux, or macOS, the environment is ready once the GCC compiler for the 68000 responds (natively, via MarsDev, or inside a Docker container), Java is available for `rescomp`, and SGDK's `libmd.a` library finishes compiling without errors. The next chapter installs the editor and extension used throughout the rest of the manual to create projects, build them, and test the result in an emulator, without having to type out every build command by hand.

Review exercises

1. What is Java needed for in an SGDK installation, given that the programming language is C?
2. What concrete difference is there between installing SGDK on Windows versus on Linux or macOS, in terms of the availability of precompiled binaries?
3. If you're following a tutorial that mentions an official SGDK Docker image, what should you do, and why?

Chapter 4 — Editor, Extension, and Emulators

The everyday working tools: a code editor, assisted compilation, and how to test the result

With the SGDK toolchain installed (previous chapter), two more things need to be sorted out before writing the first program: a comfortable editor for working with the code and resource files, and a way to run the result without having to pull a real console out of the closet every time you compile.

1. Visual Studio Code and the Genesis Code extension

This manual uses Visual Studio Code as the reference editor, together with the **Genesis Code** extension, available on the VS Code extension marketplace. The extension doesn't replace SGDK: it automates tasks that would otherwise have to be done by hand from a terminal (creating the structure of a new project, compiling, running it on the configured emulator) and adds conveniences like autocompletion for `.res` resource files and a built-in image viewer.

To configure it, you need to tell it where the toolchain is installed. From File → Preferences → Settings (or Ctrl+,), searching for the extension's section, you can choose between several toolchain types depending on which one was installed in the previous chapter:

- **Native SGDK**, pointing to the GDK environment variable.
- **MarsDev**, pointing to the environment variable under which it was installed (for example `GENDEV` or `MARS_INSTALL_DIR`, depending on the chosen installation).
- **Docker**, specifying the image to use (for example, the Doragasu image mentioned in the previous chapter).

The same settings section also lets you define which emulator to use for testing the compiled game (see below), and you can enable auxiliary buttons in the editor's status bar to build, run, and clean the project with a single click.

2. Emulators

An emulator lets you test the result of your code in seconds, without going through a physical cartridge or a real console. None of them reproduces the original hardware one hundred percent, so it's worth knowing more than one and being aware of their differences:

- **Gens (or its modified variant, Gens KMod, Windows only)**: includes tools designed specifically for development, such as a VDP memory viewer, a sprite viewer, and 68000 debugging functions. It's a convenient option for day-to-day work if you're on Windows.

- **BlastEm:** generally considered the emulator that most faithfully reproduces the behavior of the real hardware, with its own VDP memory viewer and 68000 debugging. It's cross-platform.
- **Kega Fusion:** a broadly compatible emulator, useful as an additional reference for spotting behaviors that depend on the quirks of one particular emulator and aren't replicated in another.

None of the three completely replaces the others: it's common practice, especially when debugging strange behavior, to test the same code on more than one emulator before assuming the problem lies in the game itself rather than in the particularities of one specific emulator.

3. Testing on real hardware

The project compiled with SGDK generates, inside the project folder (usually in an out subfolder), a `rom.bin` file. That file can be loaded onto an original Mega Drive or Genesis console using a **flash cart**: a cartridge with a memory card (SD or similar) that lets you run homebrew ROMs without having to burn a new physical cartridge every time. The device most widely used in the community is the EverDrive. Testing your own code on original hardware isn't a mandatory step for following this manual, but it's the only way to confirm with certainty that something works exactly the same as on a real console, and not only on the emulator used during development.

Review exercises

1. What does the Genesis Code extension automate that you would otherwise have to do by hand from a terminal?
2. Why can it be useful to test the same code on more than one emulator, instead of always relying on just one?
3. What is a flash cart, and what is it used for in the context of developing a Mega Drive game?

Chapter 5 — First Program - SGDK Project Structure and Compilation

From an empty folder to a “Hello World” running in the emulator

With the environment installed and the editor configured, this chapter walks through the complete path to seeing something on screen: creating a new project, understanding what it contains, writing the first program, and compiling it.

1. Creating a new project

From Visual Studio Code, with the Genesis Code extension installed, the command palette (Ctrl+Shift+P) command **Genesis Code: Create Project** creates the minimal structure of an SGDK project in the specified folder. That structure consists of three main folders:

- **src/**: the project’s C source code. This is where the main file, `main.c`, lives.
- **inc/**: the project’s own header files (`.h`), distinct from the ones SGDK generates automatically from resources (a topic revisited in the part of this manual dedicated to project organization).
- **res/**: the resource files — images, maps, sounds, music — along with the `resources.res` file that tells SGDK how to convert them.

2. The minimal skeleton of a program

Every SGDK program starts from a minimal structure like this:

```
#include <genesis.h>

int main()
{
    while (1)
    {
        SYS_doVBlankProcess();
    }

    return (0);
}
```

`#include <genesis.h>` brings in the entire SGDK library: it’s the only inclusion needed to

access the functions used throughout this manual (later on, once the project has its own resources, `#include <resources.h>` is added as well — the file SGDK generates automatically at compile time).

The body of `main()` is an infinite loop (`while (1)`) that never ends as long as the console is powered on: unlike a desktop program, a game doesn't "finish" execution in the traditional sense — instead, it keeps running indefinitely, drawing frame after frame. Inside that loop, `SYS_doVBlankProcess()` tells SGDK to synchronize pending work (updating sprites, processing the queued DMA, waiting for the next video frame) with the VDP's vertical blanking interval, the brief span between one drawn frame and the next during which it's safe to modify certain graphical data. Practically every main loop you'll write from here on contains, at minimum, a call to this function on every pass.

In older code or documentation it's common to find `VDP_waitVSync()` fulfilling a similar role. That function was replaced by `SYS_doVBlankProcess()` as of SGDK 1.60; if you come across `VDP_waitVSync()` in an old tutorial, it's the equivalent from an earlier version of the library.

3. A first visible result: text on screen

To confirm that the project compiles and runs, it's enough to draw a string of text on one of the background planes, using `VDP_drawText()`:

```
#include <genesis.h>

int main()
{
    VDP_drawText("Hola Mundo", 10, 12);

    while (1)
    {
        SYS_doVBlankProcess();
    }

    return (0);
}
```

`VDP_drawText()` takes three parameters: the text string to display, and the horizontal and vertical position in tiles (not pixels) where it should appear, relative to the default text font that comes with SGDK. There's no need to declare any resource in `resources.res` for this particular example: the default text font is already included in the library.

4. Compiling and running

With the **Genesis Code: Compile & Run** command (or the equivalent button in the status bar, if enabled in the extension's settings) the project is compiled and, if an emulator is configured, it's run automatically to show the result. Manually, without going through the extension, compilation is triggered with `make` from the project folder, using the Makefile that comes with the template.

The result of a successful compilation is a `rom.bin` file inside an `out/` subfolder of the project: that binary file is, ultimately, the complete program, ready to run both in an emulator and, via a flash cart, on real hardware.

Review exercises

1. Why is the body of `main()` in an SGDK program an infinite loop, unlike many desktop or traditional console programs?
2. What does `SYS_doVBlankProcess()` do, and why does practically every main loop in this manual call it at least once per pass?
3. Are the position parameters of `VDP_drawText()` expressed in pixels or in tiles?
4. Where does the `rom.bin` file end up after compiling, and what can be done with it besides running it in an emulator?

Chapter 6 — C Applied to Retro Development - Data Types, Limited Memory, and Pointers

When to use u8, u16, int, and the fixed-point types fix16/fix32; how to think about 64 KB of memory; and what changes about pointers in this environment

This chapter assumes basic knowledge of C (variable declaration, functions, introductory-level pointers) and focuses on the part that is **not** the same as programming for a modern computer: which data types are worth choosing and why, how to think about a memory of barely 64 KB, and what nuances pointers have when much of the data you work with lives in the cartridge's read-only ROM.

1. Integer types: u8, s8, u16, s16, and int

In addition to C's standard types, SGDK defines a set of aliases with names that make their size and sign explicit: u8/s8 (8 bits), u16/s16 (16 bits), and, for 32 bits, simply int (signed) without needing a special alias. The initial letter indicates whether the type is signed (s, *signed*) or not (u, *unsigned*); the number indicates how many bits it occupies in memory.

- **u8**: 8 bits (1 byte), unsigned. Since 8 bits allow for $2^8 = 256$ distinct combinations, a u8 represents integers from 0 to 255.
- **s8**: the same 8 bits, the same 256 combinations, the same size in memory, but split between negative and positive values: from -128 to 127.
- **u16**: 16 bits (2 bytes), unsigned, with $2^{16} = 65,536$ possible combinations: from 0 to 65,535. It's no coincidence that this figure matches the size in bytes of both the console's main RAM and its VRAM.
- **s16**: the same 16 bits, signed, from -32,768 to 32,767.
- **int**: 32 bits, signed, from -2,147,483,648 to 2,147,483,647. This range is so large that, in practice, it's almost never overflowed by accident within the reasonable duration of a play session.

When any of these types exceeds its upper limit (or lower limit, for signed types), no error occurs: the value **overflows** and keeps counting from the other end of its range. This behavior is predictable, and once understood in advance, it can be used to your advantage (for example, for cyclic counters); without understanding it, it's a frequent source of hard-to-track bugs.

A practical detail worth keeping in mind when displaying these values as text (with `sprintf()` and then `VDP_drawText()` or `VDP_drawTextBG()`, as is done in several chapters of this manual): the negative sign takes up a character of its own within the destination string. A string declared with just enough size for a u8 (`char texto[3]`, for "255") falls short as soon as that same variable becomes an s8 and takes on a negative value like "-128" (4 characters);

the result, if the string and the `sprintf()` format aren't enlarged accordingly, gets visibly corrupted. The same precaution — one extra character for the sign — applies to any signed type displayed on screen.

When to use each type

The general rule is to always use the smallest type that suffices for the intended purpose: each `int` takes up 4 bytes out of a total memory of 64 KB, each `u16/s16` takes up 2, and each `u8/s8` just 1. For values with a naturally bounded range — number of lives, number of continues, the current level, or a frame counter for invincibility after taking damage (255 frames equal more than four seconds at 60 frames per second, more than enough in most cases) — a `u8` is usually more than sufficient; in fact, this is a common pattern in commercial games from that era: the rupee counter in *The Legend of Zelda*, for example, caps at exactly 255.

2. Numbers with decimals: there is no float

As explained in the architecture chapter, the Motorola 68000 processor has no floating-point unit. Using `float` or `double` in SGDK is possible at the language level, but every operation with those types ends up resolved in software, at a very high performance cost for hardware of this era; in practice, they're virtually never used. Instead, SGDK offers **fixed-point** types: numbers that reserve a fixed number of bits for the integer part and another fixed number for the decimal part, and that are represented internally as ordinary integers (and are therefore just as fast to operate on as any `int`).

- **fix16**: 16 bits total, split into 10 bits for the integer part and 6 for the decimal part. It's declared and initialized with the `FIX16()` macro:

```
fix16 posicion = FIX16(0);  
posicion += FIX16(1);      // add 1 whole integer
```

With 6 decimal bits, the smallest representable increment is $1/64 \approx 0.0156$, and with only 10 integer bits, a `fix16` overflows far sooner than an ordinary `s16` (a little past 500). That combination — small integer range, but very fine decimal steps — makes it ideal for slow, bounded movement, such as a background layer in a parallax scroll.

- **fix32**: 32 bits total, with 22 bits for the integer part and 10 for the decimal part, initialized with `FIX32()`. Its integer range is enormous compared to `fix16`, at the cost of taking up twice the memory. It's the recommended type for representing the position of a character or an enemy on a map several thousand pixels wide, where a `fix16`'s integer range would fall short.

To convert a fixed-point value to a regular integer (for example, to display it with `sprintf()`, or to use it as a pixel coordinate in a function that expects an integer), SGDK provides dedicated conversion functions: `F16_toInt()` for `fix16` and `F32_toInt()` for `fix32`.

Code or tutorials written before April 2025 (SGDK 2.10) use the names `fix16ToInt()` and `fix32ToInt()` instead of `F16_toInt()` and `F32_toInt()`. Starting with that version, the entire fixed-point math library switched to using the `F16_/F32_/FF16_/FF32_` prefix consistently; the functions' behavior didn't change, only the name.

Adding a value smaller than 1 with an ordinary integer type doesn't make sense (`entero += 0.1` simply doesn't compile to represent the intended meaning), but it does with fixed point: it's the standard technique for movement slower than one whole pixel per frame. That said, it's worth keeping the screen resolution in mind when choosing an increment: in H40 mode, an element can only visually move by one whole pixel at a time, so a very small fractional

increment takes several frames to translate into visible movement, which can feel choppy if the chosen step is too small for the desired speed.

3. Thinking in 64 KB: memory as an active resource

The console's main RAM has 64 KB in total, a fixed limit regardless of the cartridge's size. That total has to be enough for all of the program's global variables, the function call stack, any dynamic data structures, and the internal work that the SGDK library itself needs in order to function. This leads to a few habits that may not be necessary when programming for a modern computer:

- **Prefer constant data in ROM over copies in RAM.** An array declared as `const` (for example, a table of fixed values, or the data of a resource already converted by `rescomp`) is stored in the cartridge's ROM and read directly from there, without taking up any of the 64 KB of RAM. Reserving that memory for what actually needs to be able to change at runtime is one of the most important design decisions in this whole manual.
- **Be wary of dynamic memory allocation.** SGDK provides `MEM_alloc()` and `MEM_free()` (and, for compatibility, `malloc()/free()` redirect to the same functions) to allocate memory at runtime, but careless use — allocating and freeing blocks of very different sizes throughout an entire playthrough — can quickly fragment such a small memory space, leaving free blocks too small and scattered to satisfy a new allocation, even when the total amount of free space would be sufficient. Declaring fixed-size arrays whose size is known in advance (for example, an array for up to 16 active enemies instead of a variable-sized list) is usually preferable whenever the maximum limit can be estimated with some margin.
- **Choose the smallest type that suffices**, as already covered in section 1: the difference between a 50-element array declared as `int` (200 bytes) and the same array as `u8` (50 bytes) may seem small in the abstract, but it becomes significant once it's repeated across dozens of structures throughout a complete game.

4. Pointers in an environment with read-only memory

Pointers in SGDK work exactly the same as in any other C program: a variable that stores a memory address instead of a direct value. What's particular about this environment is that a good deal of the data a pointer can point to — resources generated by `rescomp` (images, maps, sound tables), any array declared `const` — lives in the cartridge's ROM, which is **read-only** at runtime. A pointer can perfectly well read that data, but any attempt to write through it to a ROM address has no useful effect (in the best case) or corrupts the program's behavior unpredictably (in the worst case). For this reason, it's good practice to declare as `const` any pointer intended purely for reading a resource's data, letting the compiler itself flag it if some part of the code mistakenly tries to write to that memory.

For cartridges exceeding 4 MB of ROM, SGDK also includes a *bank switching* mechanism that allows addressing more memory than the 68000 can access directly at any given instant; this topic is beyond the scope of this introductory chapter and is revisited, along with the `FAR()/FAR_SAFE()` functions that manage it, in the part of this manual dedicated to optimization.

Review exercises

1. Why can an `s8` represent the same 256 values as a `u8`, but with a different numeric range?

2. A teammate changes a variable from `u16` to `s16` without adjusting the size of the text string used to display it with `sprintf()`. What problem can arise, and why?
3. Why doesn't SGDK offer a practically usable `float` type, and what does it use instead to represent numbers with decimals?
4. Explain in your own words why `fix32` is more appropriate than `fix16` for the position of a character on a map several thousand pixels wide.
5. Why is it a good idea to declare as `const` a pointer that's only used to read the data of an image generated by `rescomp`?

Chapter 7 — How to Display Backgrounds on the Mega Drive

The BG_A and BG_B planes, the SGDK resource pipeline, and the VDP_drawImageEx function

Every Mega Drive game needs, at the very least, to put an image on screen. This chapter covers that first step in the most direct way possible: how to take a PNG image made on a computer and turn it into visible data on one of the two background planes offered by the VDP. The journey passes through four stages that will reappear, with variations, in practically every graphic displayed on screen from here on: declaring the resource so the compiler includes it in the ROM, reserving a spot for it in VRAM, assigning it a color palette, and finally asking the VDP to draw it on a plane.

Chapter 2 developed the hardware behind this operation in depth: the VDP, the VRAM, the BG_A/BG_B/Window planes, tiles, and the PAL0-PAL3 palettes. That vocabulary is assumed to be familiar here; this chapter's focus is on the code: which function to use, with which parameters, and why.

Prerequisites

Before continuing, it helps to have:

- A working SGDK 2.11 environment installed, capable of compiling and running a project in an emulator, as covered in chapters 3 and 4.
- Basic notions of C: declaring variables, types, functions, and structures (revisited in more detail in chapter 6).
- A general understanding of what VRAM, a tile, and a color palette are on the Mega Drive's VDP (chapter 2).
- An SGDK project with a minimal `main.c`, following the structure shown in chapter 5.

The central functions in this chapter (`PAL_setPalette()`, `VDP_drawImageEx()`, the `TILE_ATTR_FULL()` macro, the `TILE_USERINDEX` constant, and the syntax of the `IMAGE` resource in a `.res` file) keep, in SGDK 2.11, exactly the same name, the same signature, and the same behavior as in fairly old versions of the kit. All the code in this chapter can be written and compiled exactly as it appears here, with no further adaptation. If you come across older code or forum posts, one naming detail may catch your eye: before SGDK 1.50, the planes were called `PLAN_A` and `PLAN_B` instead of `BG_A` and `BG_B` — the same concept under a different name.

1. The resource pipeline: from a PNG file to the ROM

An image created on a computer is not automatically available inside a Mega Drive program. Before it can be drawn, SGDK needs to be told to include it in the ROM (the final binary file

that runs on the console or in an emulator) and under what name the C code will be able to refer to it. That step is handled with a resource file.

Inside the project's `res` folder, a file called `resources.res` is created. Each line in this file describes a distinct resource to be included in the ROM: later on there will be lines for sound effects, music, and sprites, but for now a single image is enough:

```
IMAGE bg1 "background1.png" BEST ALL
```

This line declares an image-type resource called `bg1`, taken from the file `background1.png`, with `BEST` compression and `ALL` map optimization. Each field plays a specific role, and SGDK is strict about uppercase, lowercase, and spacing:

- **IMAGE** (uppercase) indicates the resource type: in this case, a full image (tileset plus tile map plus palette), meant for a simple static background.
- **bg1** is the name the C code will use to refer to this resource. It can be called whatever you like, as long as it's a single word with no spaces and reasonably short.
- **"background1.png"** is the path to the image file in quotes, relative to the `res` folder.
- **BEST** (uppercase) indicates the compression mode. The most common alternatives are `NONE` (uncompressed), `FAST` (an algorithm that's quicker to decompress but less compact), and `BEST` (which produces the smallest file, though it takes a bit longer to decompress at runtime). For a background image loaded only once, `BEST` is usually the most sensible choice.
- **ALL**, at the end, is the tile map optimization: it tells the compiler to reuse identical tiles (including tiles flipped horizontally or vertically) instead of repeating them, saving space in the ROM and, later, in VRAM.

The name `bg1` isn't available for autocomplete or for the `main.c` compiler until the project is saved and compiled at least once: only then does SGDK generate the corresponding `resources.h` file, from which the rest of the code can recognize and include that resource. If you type `bg1` in `main.c` immediately after creating the line in `resources.res`, it's normal for the editor not to recognize it yet.

The `IMAGE` resource type bundles the tileset, the tile map, and the palette of a full image into a single declaration, which is convenient for a static background like the one in this chapter. For backgrounds larger than one screen, for levels with scrolling, or for sharing the same set of tiles across several images, SGDK lets you declare the `TILESET`, `MAP` (or `TILEMAP`), and `PALETTE` resources separately, giving more control over how they're organized in the ROM and in VRAM. This chapter doesn't need that yet; the topic comes back in the chapter devoted to scroll and parallax scrolling, later in this same part of the manual.

With the resource declared and the project compiled at least once, `main.c` can now include the file generated by SGDK and start setting the stage for drawing the image. Since no text is going to be shown yet, it's worth removing any sample call to `VDP_drawText()` that the project template might include:

```
#include <genesis.h>
#include <resources.h>

int main()
{
    while(1)
    {
        SYS_doVBlankProcess();
    }

    return (0);
}
```

The file included from `main.c` is `resources.h` (with a `.h` extension), generated automatically when compiling from `resources.res`. The `.res` file is the source you edit by hand; the `.h` file is the C interface that results from processing it, and it's the one the rest of the program consumes. There's no need to include the `.res` file directly, and in fact it isn't possible.

2. Reserving a spot in VRAM: `TILE_USERINDEX`

The VDP's VRAM isn't free space from start to finish: its first positions are reserved for the system and, if the project uses SGDK's default text font, also for that font's tiles. Before uploading the tiles of your own image, you need to start from a position that doesn't overwrite any of that. SGDK exposes exactly that position through a constant:

```
u16 ind = TILE_USERINDEX;
```

`TILE_USERINDEX` is the first VRAM tile index left free for the programmer after subtracting the tiles reserved by the system. The variable `ind`, declared as `u16` (an unsigned 16-bit integer, more than enough since the VDP's VRAM has 2048 tile positions in total), keeps track of the VRAM position at which each new graphic is uploaded as the program needs it. A variable that advances as chunks of VRAM are reserved — the same idea reappears, with much more detail and with the complications it can bring, in the chapter on saving VRAM through manual sprite loading, in the part of this manual devoted to optimization.

3. Assigning a palette: `PAL_setPalette`

The Mega Drive can display up to 64 colors on screen, split across four palettes of 16 colors each (`PAL0` through `PAL3`). Before drawing an image, the VDP needs to be told which colors to paint it with, and those colors have to come from one of those four palettes. The function that does this is `PAL_setPalette()`:

```
void PAL_setPalette(u16 numPal, const u16 *pal, TransferMethod tm);
```

The three parameters are: `numPal`, the palette number this color information should be assigned to (`PAL0`, `PAL1`, `PAL2`, or `PAL3`); `pal`, a pointer to the color data itself; and `tm`, the transfer method. The full call, in this case, is:

```
PAL_setPalette(PAL0, bg1.palette->data, DMA);
```

`bg1.palette->data` takes the color information directly from the `Image` structure generated from `background1.png`: there's no need to write the colors by hand, since they're already packaged together with the image from the moment the resource was declared in `resources.res`. The last parameter, `DMA`, indicates that the transfer of that color data is done via direct memory access (*Direct Memory Access*), a copying mechanism that doesn't use up CPU time; the alternative is `CPU`, which copies the data using the processor. For a one-off operation like this, either one works fine; `DMA` is, generally speaking, the recommended option since it costs less CPU time.

If one of the background or foreground images isn't assigned to `PAL0`, it's common for it to lose its first color (the color at index 0 of the palette, which on the Mega Drive's VDP is always the background/transparency color). For that reason, as a general rule, it's a good idea to reserve `PAL0` for the farthest-back background graphic — usually the background itself on `BG_B` — and use `PAL1`, `PAL2`, or `PAL3` for the rest.

4. Drawing the image: VDP_drawImageEx and TILE_ATTR_FULL

With the resource declared, the VRAM space reserved, and the palette assigned, the last step is to ask the VDP to actually draw the image on one of the two background planes. The function for this is `VDP_drawImageEx()`:

```
bool VDP_drawImageEx(VDPPlane plane, const Image *image, u16 basetile,
                    u16 x, u16 y, bool loadpal, bool dma);
```

The parameters, in order, are: `plane`, the background plane to draw on (`BG_A` or `BG_B`; there's also `WINDOW`, which isn't used in this chapter); `image`, a pointer to the `Image` structure generated from the resource; `basetile`, the VRAM position from which this image's tiles will be written; `x` and `y`, the position on screen; `loadpal`, whether the palette should also be loaded as part of this same call; and `dma`, whether the tile transfer should be done via DMA.

The `basetile` parameter isn't passed as a plain number, but through a macro that packs several pieces of information into a single value: `TILE_ATTR_FULL(pal, prio, flipV, flipH, index)`.

```
VDP_drawImageEx(BG_B, &bg1, TILE_ATTR_FULL(PAL0, FALSE, FALSE, FALSE, ind), 0, 0,
↪ FALSE, TRUE);
```

Each tile in the Mega Drive's VRAM doesn't just store its pixel pattern: it also carries, in the plane's name table, attribute bits that indicate which palette to paint it with, whether it has high or low priority, and whether it should be shown flipped vertically or horizontally. `TILE_ATTR_FULL()` packs those attributes into a single value:

- **pal** — which of the four palettes (`PAL0` through `PAL3`) to use for this tile.
- **prio** (*priority*) — whether this plane is drawn in front of (`TRUE`) or behind (`FALSE`, the default value) the other graphic elements it shares the screen with.
- **flipV** — whether the tile is shown flipped vertically.
- **flipH** — whether the tile is shown flipped horizontally.
- **index** — the VRAM position (the same value tracked in the `ind` variable) from which this image's tiles start.

With all this in place, saving, compiling, and running the project already shows the first background on screen.

First result in the emulator: the nighttime cafe scene with the neon sign, now drawn on BG_B.

5. Experimenting with the parameters

Before adding a second plane, it's worth pausing to modify, one at a time, the parameters just used. This isn't an idle exercise: clearly understanding what each one does saves a lot of debugging time later on, when these same parameters show up combined with scrolling, sprites, and several layers at once.

5.1 Switching palettes without having loaded one

If you change the `pal` in `TILE_ATTR_FULL()` from `PAL0` to `PAL1` without having first called `PAL_setPalette(PAL1, ...)`, the result is not subtle: the image comes out drawn almost entirely in red.

The same background, specifying PAL1 without having initialized it: the VDP draws it just the same, but with the default color of an empty palette.



Figure 1: The background1.png background drawn on the BG_B plane

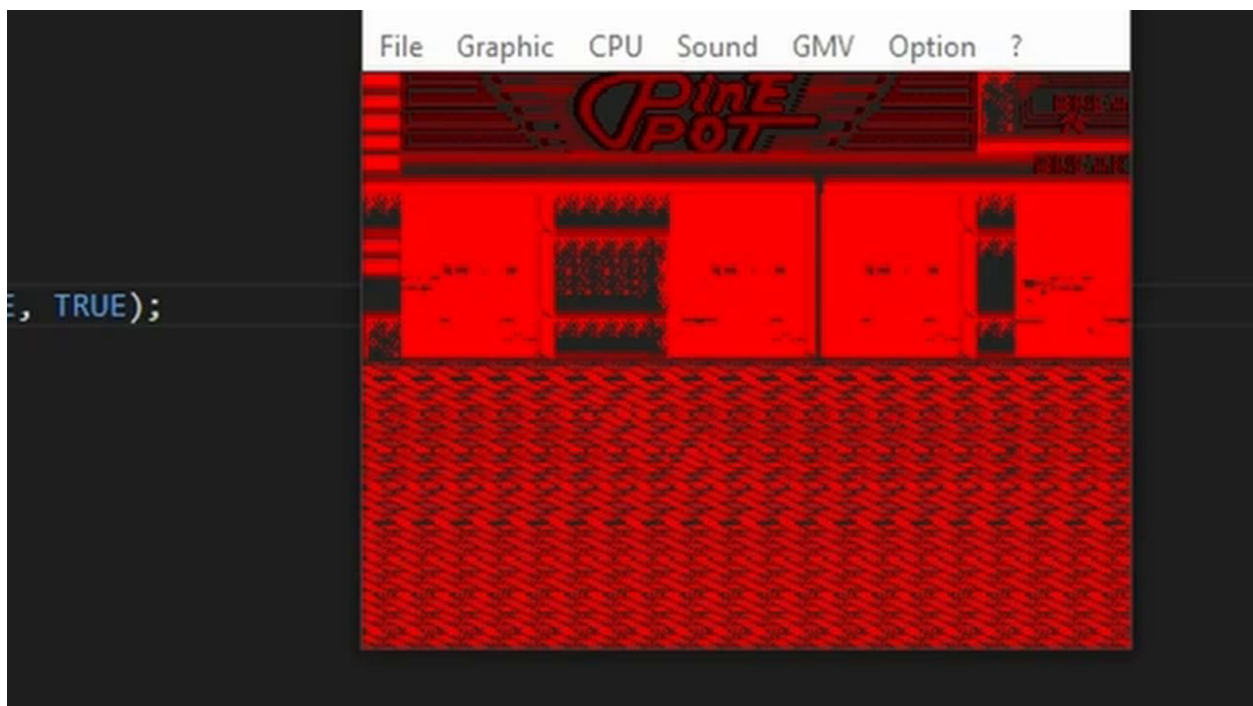


Figure 2: The background drawn with PAL1 without having loaded color data into that palette

The red you see isn't an effect or an error in the image data: it's simply the default content of a palette that was never loaded with color information. The VDP draws the tiles just the same, with the correct geometry, but looks up colors in an empty palette. Switching back to PAL0, the palette that was actually initialized with `PAL_setPalette()`, the background looks normal again.

5.2 Flipping tiles: `flipV` and `flipH`

Changing `flipV` to `TRUE` makes every tile on the plane show up flipped vertically: the neon sign and the building's facade appear upside down. The result with `flipH` is analogous, flipping horizontally.

In this specific context — a static background — flipping the whole plane doesn't have much practical use, but the same mechanism is the basis for a significant memory saving: if a tile and its flipped version are visually identical, there's no need to store both in VRAM or in the ROM. It's enough to keep just one and, when drawing it, indicate that you want it shown flipped. In practice there's no need to manage this by hand tile by tile: when converting graphics with the right tools, SGDK automatically recognizes which tiles are flips of others and avoids duplicating them in VRAM, something that's revisited in more detail in the chapter devoted to creating graphics.

Although flipping an entire plane rarely serves a purpose for a static background, the same parameter is very useful for sprites: it's the standard mechanism for making a character face left or right without needing double the graphics or double the VRAM space, by flipping the same sprite horizontally depending on which direction it's moving.

5.3 Priority: what gets drawn in front of what

By default, the Mega Drive's drawing order runs from the background plane (`BG_B`) at the back, the foreground plane (`BG_A`) in the middle, and the sprites in front of everything. The `prio` parameter lets you alter that order for a particular plane. Changing `BG_B`'s priority to `TRUE`, for example, makes that plane get drawn in front of `BG_A`.

With `BG_B` set to high priority, the checkered floor of the background covers almost the entire car and lamppost in the foreground, which are left barely peeking through.

In this particular example the result isn't especially useful (the foreground ends up practically hidden), but the tool itself is very useful in other situations. *Streets of Rage* is a frequently cited case: in several scenes, the character sprites need to appear behind certain foreground elements (a streetlamp or a sign, for example), even though sprites are drawn in front of everything by default. The solution is the same tool just shown: give the foreground (`BG_A`) high priority and leave both the background and the sprites at normal priority, so the final order ends up being background, then sprites, and finally the foreground with high priority.

5.4 Coordinates: `x` and `y` are measured in tiles, not pixels

The `x` and `y` parameters of `VDP_drawImageEx()` aren't pixel coordinates but tile coordinates. Since each Mega Drive tile measures 8×8 pixels, a value of `x = 10` shifts the image 80 pixels to the right (10 tiles × 8 pixels), not 10 pixels.

It's a common mistake, especially when starting out, to pass a value of `x` or `y` thinking in pixels and end up with a shift eight times larger than expected. It's worth keeping this in mind especially later on, when calculating positions from other elements' coordinates (a sprite's position in pixels, for instance) which, if combined without converting, can produce results very different from what was intended.

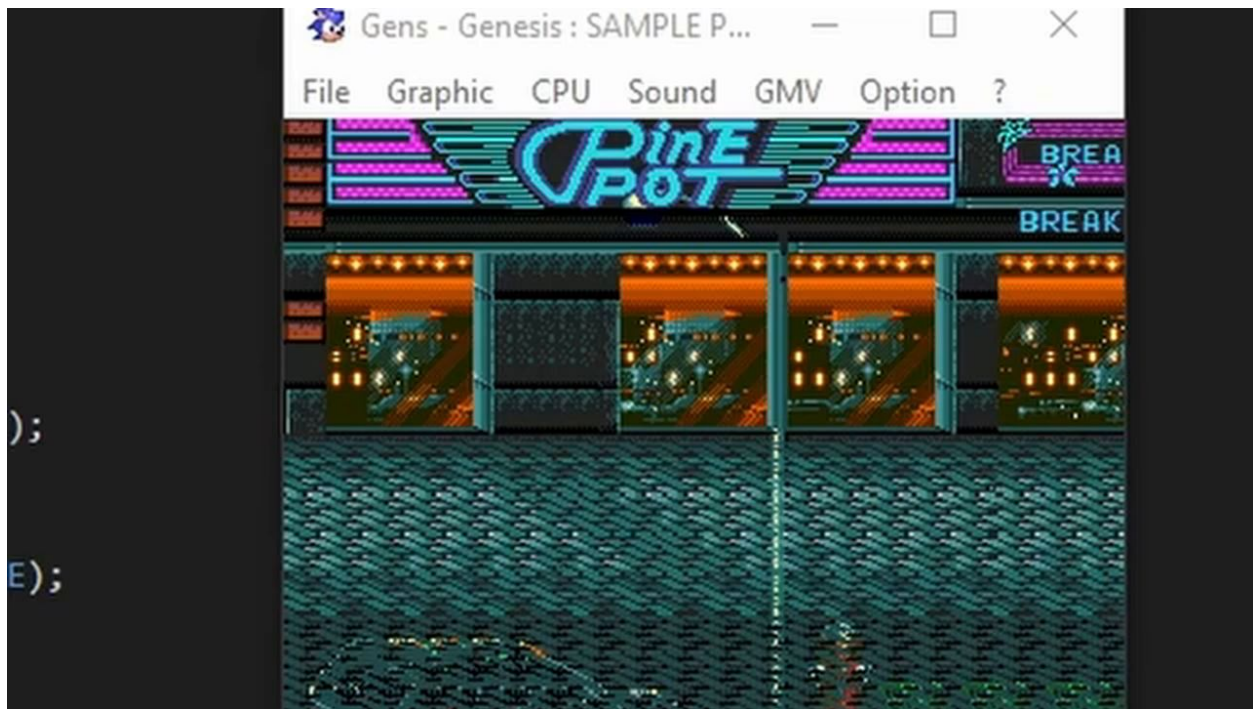


Figure 3: BG_B with high priority covering the foreground

These same position parameters, combined with an offset that changes frame by frame instead of staying fixed, are the basis of the scrolling and parallax scrolling covered later in this part of the manual.

6. Adding a second plane: the foreground (BG_A)

With one plane working, adding the second one is, for the most part, a matter of repeating the same recipe: declare the resource, assign a palette, and draw the image — this time on BG_A. First, the new line is added to `resources.res`:

```
IMAGE bg1 "background1.png" BEST ALL
IMAGE fg1 "foreground1.png" BEST ALL
```

After saving and compiling once so the new name `fg1` becomes available, the full code looks like this:

```
u16 ind = TILE_USERINDEX;

PAL_setPalette(PAL0, bg1.palette->data, DMA);
VDP_drawImageEx(BG_B, &bg1, TILE_ATTR_FULL(PAL0, FALSE, FALSE, FALSE, ind), 0, 0,
    ↪ FALSE, TRUE);

ind += bg1.tileset->numTile;

PAL_setPalette(PAL1, fg1.palette->data, DMA);
VDP_drawImageEx(BG_A, &fg1, TILE_ATTR_FULL(PAL1, FALSE, FALSE, FALSE, ind), 0, 0,
    ↪ FALSE, TRUE);
```

Two details are not optional. The first is the line `ind += bg1.tileset->numTile;`, which adds

to find the number of tiles taken up by the first background before uploading the second one. The second is using PAL1 (not PAL0) for the foreground, since it's an image with its own colors that have no reason to match the background's.

If the advance of `ind` is skipped before uploading the second graphic, its tiles get written over the same ones already occupied by the first background, corrupting both. It's exactly the same kind of error, with the same cause (a mismanaged VRAM index), explained in more detail in the chapter on saving VRAM through manual sprite loading. It's worth deliberately testing what happens if this line is removed, to learn to recognize the symptom: it doesn't break anything permanently or damage any hardware, and it helps identify the problem if it shows up by accident in another context.

The result with both planes active combines the background with the foreground drawn on top, each with its own palette and its own stretch of VRAM.

7. One last experiment: swapping the palettes

To wrap up, let's see what happens if the palettes are swapped between the two planes: assigning PAL1 to the background and PAL0 to the foreground, the reverse of how they're set up.

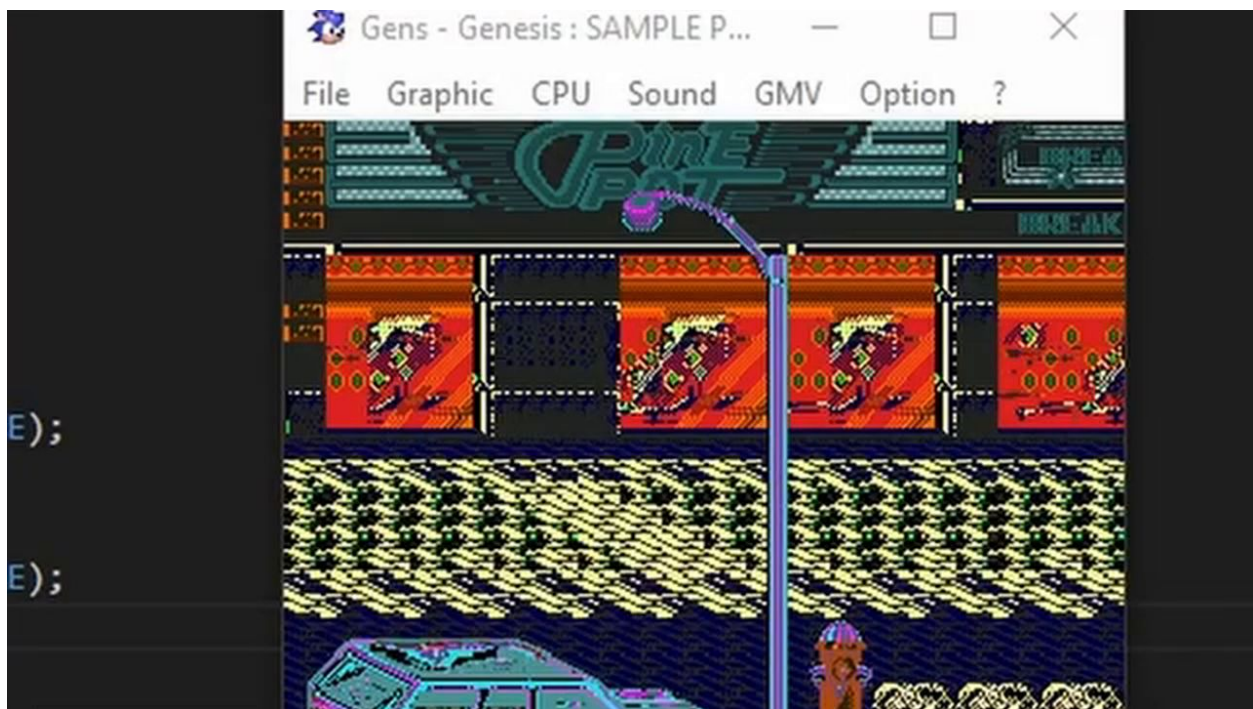


Figure 4: Palettes swapped between the two planes

Palettes swapped between the two planes: the result isn't simply "all red" or "all blue," but a confusing mix of colors that doesn't match either of the two original images.

Unlike the experiment in section 5.1, where the palette was simply empty, here both palettes do have color data, just not the data each plane expects: each one ends up painted with the other's color palette, and on top of that each image was converted assuming its own color indexes would match its own palette. The result combines both mismatches at once. This chapter doesn't go into why the combination looks exactly like this; the topic is revisited with

the detail it deserves in the chapter devoted to creating graphics, when it's explained how an image is converted to Mega Drive format and exactly what relationship its color indexes have with the palette that accompanies it.

8. Testing on real hardware

The compiled project doesn't only run in the emulator: inside the project folder, in the out subfolder, there's a `rom.bin` file. By copying that file to a flash cartridge (the devices that let you load homebrew ROMs onto an original Mega Drive or Genesis), you can run it on a real console connected to a period television. This isn't a required step for following this manual, but trying your own code on original hardware is often, for someone doing it for the first time, one of the most rewarding moments of the whole process.

Review exercises

1. Explain, in your own words, what each of the four fields of an `IMAGE` line in a `.res` file is for (type, name, file, compression, and map optimization).
2. Why does the project need to be recompiled after adding a new line to `resources.res` before the new resource name is available in `main.c`?
3. A teammate loads a background's palette with `PAL_setPalette(PAL2, bg1.palette->data, DMA)` but, when drawing the image, writes `TILE_ATTR_FULL(PAL1, FALSE, FALSE, FALSE, ind)`. What will show up on screen, and why?
4. Suppose you add a third plane of graphic information after `bg1` and `fg1`, but forget to add `fg1.tileset->numTile` to `ind` before uploading it. Describe the expected symptom.
5. Why does a value of `x = 5` in `VDP_drawImageEx()` shift the image 40 pixels instead of 5? What would you need to do if you needed to shift it by exactly 5 pixels?

Chapter 8 — Creating Graphics with Aseprite

The previous chapter showed how to bring an already-existing image onto the Mega Drive screen: declaring it as an `IMAGE` resource in `resources.res`, converting it into a tileset hosted starting at `TILE_USERINDEX`, loading its palette with `PAL_setPalette()`, and drawing it with `VDP_drawImageEx()`. Throughout that whole process the image arrived ready-made: the PNG file was already sitting in the `res` folder, ready to be declared as a resource. This chapter fills that gap backward and explains where that file comes from: how to draw a graphic from scratch in a way the Mega Drive can display it, what format rules the hardware imposes before it will accept an image, and how to go from a blank canvas to a PNG compatible with SGDK's resource pipeline.

This is not a pixel-art drawing chapter: there are resources dedicated entirely to that discipline, and that's not the goal here. The focus is on the layer that almost never gets explained in those general materials: what turns an arbitrary drawing into a graphic fit for the Mega Drive. That includes canvas size, color mode, palette discipline, and the final file format. The editor used throughout the chapter is Aseprite, but nothing explained here is specific to that program: any pixel-art editor that supports indexed palettes and PNG export works just as well.

Prerequisites

Before continuing, it helps to have on hand:

- Having completed Chapter 7, “Displaying Backgrounds on the Mega Drive”: the concepts of tile, VRAM, palette, and plane (`BG_A/BG_B`) are assumed knowledge, as is the `IMAGE` directive in `resources.res`.
- A pixel-art graphics editor installed. This chapter uses Aseprite as its reference; any alternative capable of working in indexed mode and exporting PNG is equivalent.
- The SGDK sample project used in the previous chapter, with the `res` folder and the `resources.res` file accessible.

1. The right canvas: dimensions and the multiple-of-8 rule

Every graphic for the Mega Drive starts with choosing the right canvas size, and that choice depends on what the image will be used for. If the goal is to fill a single screen with no scrolling, or just to test a result, 320×224 pixels is enough: the visible resolution in the console's standard mode. But if the plane is going to scroll later on — something revisited further along in this manual — it's better to draw directly at the actual size the VDP reserves for each plane: 512×256 pixels. Drawing at 320×224 and then running into empty gaps once

scrolling kicks in isn't a coding mistake: it's simply not having planned for the real size of the tile board from the start.

The dialog for creating a new canvas asks, besides the size, for a color mode. It's best to choose RGB at this first step, even though the final destination is an indexed image; the reason is explained in section 5, and it has to do with a very concrete problem that only shows up once you switch to indexed mode with an overly large palette. The initial background color (white, black, or transparent), on the other hand, is just a matter of drawing convenience with no effect on the final result, except in the particular case of a graphic with transparency, which is revisited in section 7.

Whatever size is chosen, there's one restriction that isn't negotiable: both the width and the height of any Mega Drive graphic must be multiples of 8. The reason is the same one already covered when discussing the VDP: VRAM doesn't store loose pixels, but 8×8 blocks — tiles — and there's no way to represent a fraction of a tile. A width of 320 is valid (an exact 40 tiles); a width of 321 is not, and SGDK's resource compiler rejects it as soon as it tries to convert the image into a tileset, halting the build with an explicit error.

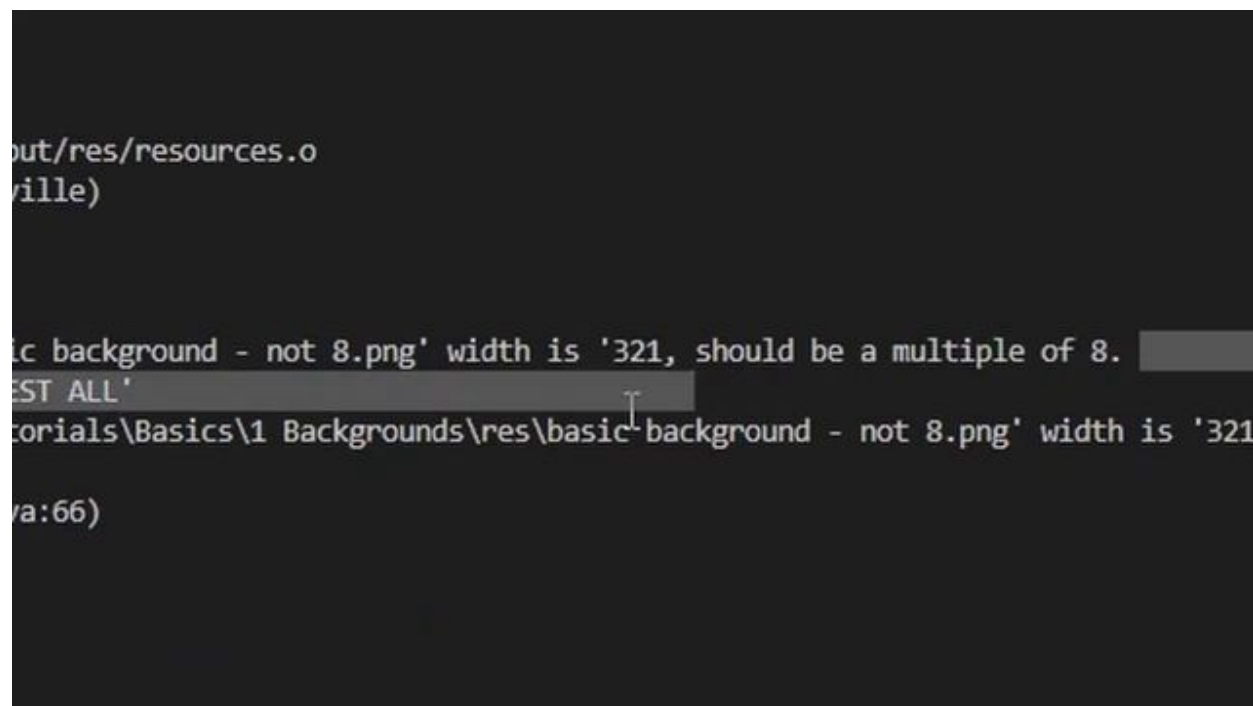


Figure 5: Compile error due to a width that isn't a multiple of 8

SGDK's resource compiler refuses to generate the tileset and halts compilation when the declared width isn't a multiple of 8.

This kind of error tends to show up when an image is cropped or resized by hand and its exact dimensions get lost track of; before exporting any graphic, it's worth double-checking its dimensions in the editor itself. The rule applies equally to backgrounds and sprites: 8, 16, 24, 32, 320, 512... any multiple of 8 is valid; any other number is not.

2. The Mega Drive palette in the editor

The Mega Drive can display up to 64 simultaneous colors on screen, organized into four palettes of 16 colors each (PAL0 through PAL3, already covered when describing the VDP).

That limit isn't a retro-style suggestion: it's just as structural as the multiple-of-8 rule, and any graphic being drawn has to respect it from the outset, not adjust to it afterward.

Aseprite — like most pixel-art editors aimed at retro consoles — ships with predefined palettes for several classic systems: Atari 2600, Commodore 64, Game Boy, Game Boy Color, Master System, among others. The Mega Drive, however, usually isn't among them by default. That means you need to import a Mega Drive palette from an external file: several circulate on homebrew community forums and repositories, and any one that faithfully reproduces the hardware's 512 possible colors — 9 bits of color, split across 3 bits per channel — will do the job. This manual doesn't ship any particular palette file; it's enough to obtain or generate one that's faithful to that bit distribution.

Once imported and applied to the canvas, the editor's palette panel is ready to work with. Since that palette will be used across every graphic in the project, it's worth avoiding having to re-import it every time a new file is created: Aseprite lets you set the current palette as the default (the "Save Palette as Default" option in the palette menu), so that every new canvas includes it automatically.

3. The 8×8 grid and the discipline of drawing tile by tile

Since the VDP organizes everything into 8×8-pixel blocks, it's worth making that organization visible while drawing, not just at export time. Most pixel-art editors let you overlay a configurable grid on the canvas; for Mega Drive graphics it makes sense to set it exactly to 8-pixel steps, so each cell corresponds to an actual VRAM tile.

This isn't an aesthetic preference. Seeing each tile's boundaries while drawing helps anticipate how well SGDK's automatic tile deduplication (covered in section 8.3) will be able to kick in, and it avoids surprises when a shape ends up split awkwardly between two neighboring tiles. The grid can be adjusted to other sizes depending on what's being drawn — for example, 16×16 to visually group sets of four tiles, as seen in section 7 — but the underlying reference should always be thinkable in multiples of 8.

4. Keeping the 16-color limit in check: the tally column

Each individual graphic — each image that ends up converted into an SGDK IMAGE resource — is limited to a single 16-color palette, even though the hardware has four available in total. While drawing a scene with several elements (sky, clouds, ground, and eventually sprites), it's easy to lose track of how many distinct colors have been added, especially if the editor doesn't display that count prominently.

A practical trick for keeping count is to set aside a strip of tiles outside the area that will later be exported, and use it as a visual log: every time a new color is introduced into the scene, a swatch of that color gets added to the strip too. A glance at that column is enough to know, at any moment, how many distinct colors the graphic has used so far, without opening any additional panel. This column is purely a working tool: it gets cropped out before export and isn't part of the final graphic.

The habit of counting colors matters much more in complex graphics than in a simple sky-clouds-ground example: in a more detailed scene it's easy to go over 16 without noticing, and fixing it after everything has already been drawn is considerably more work than keeping count from the start.

5. From RGB to indexed: trimming the palette and exporting the PNG

With the drawing finished, there are three steps left before the file is ready for SGDK: reducing the canvas palette down to only the colors actually used, switching the color mode from RGB to indexed, and saving as PNG. All three are mechanical, but each has a reason behind it that's worth understanding, not just executing.

The first step removes from the file's palette any color that wasn't actually used in the drawing: what's left, for example, are the three or four colors actually painted, instead of the hundreds carried by the full palette imported in section 2. Most editors — Aseprite included — offer a function to automatically generate the canvas palette from the colors present in the image. This step also doubles, conveniently, as an extra check on the 16-color limit: if, after generating it, you end up with 17, 18, or more, that's an unmistakable sign that at some point in the drawing an extra color slipped in without notice — perhaps a barely-different shade that went unnoticed — and the graphic is worth reviewing before moving on.

The second step changes the file's color mode from RGB to indexed, via the canvas properties menu. Why do this at the end and not at the beginning? Many editors let you work directly in indexed mode from the very start, but doing so before trimming the palette can produce unexpected results if the starting palette is very large. The palette imported in section 2 can cover all 512 possible hardware colors, and pixel-art editors usually impose a practical limit well below that on the number of entries an indexed palette can hold — in Aseprite a figure around 256 slots is often cited, though it's worth checking the exact value for the version you're using, since it can vary. If you switch to indexed mode with a palette that exceeds that limit, the entries above the cutoff aren't silently discarded: they get reassigned to already-occupied slots, and the visible color at those slots changes abruptly and unpredictably.

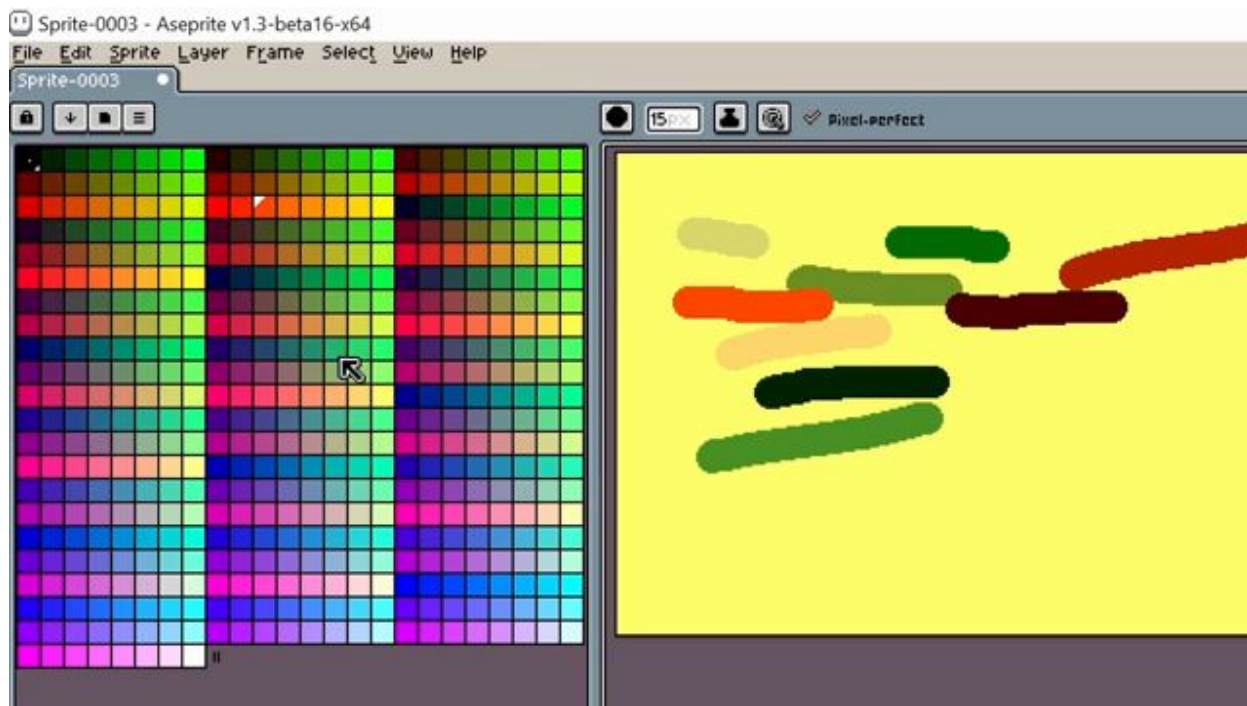


Figure 6: Colors shifted when indexing with a 512-color palette

When switching to indexed mode with all 512 colors still loaded, the upper half of the palette is preserved, but the lower half shifts: whites that turn yellow, blues that turn purple.

Working first in RGB, where each color is identified by its actual value rather than a palette slot, and only switching to indexed once the canvas palette has been trimmed down to the few colors actually used, avoids this problem entirely: with a palette of 16 entries or fewer, that limit is never reached. That limit (somewhere around 256 slots) is a quirk of the editor, not of the Mega Drive hardware: it shouldn't be confused with the real, much stricter limit of 16 colors per individual graphic already covered in section 4.

With the palette trimmed and the color mode already set to indexed, the last step is to save the file. The name can be anything, but the format isn't negotiable: it has to be saved as PNG, the only image format that SGDK's resource pipeline — covered in Chapter 7 with the `IMAGE` directive in `resources.res` — knows how to interpret.

6. Bringing the graphic into SGDK and seeing it on screen

With the PNG already exported, bringing it into the project requires no new steps beyond what was covered in Chapter 7: it's enough to replace, inside `resources.res`, the name of the image file that the already-declared resource references. There's no need to change the resource's own name (`bg1`, in this example) or touch a single line of `main.c`, because the code still refers to the same identifier; only the file behind it changes.

```
IMAGE bg1 "background - 512 256.png" BEST ALL
```

Saving the resource file and rebuilding the project is all that's left to see the newly created graphic running in the emulator. This is, in short, the full loop: draw respecting size and palette, export as an indexed PNG, declare (or update) the resource in `resources.res`, and rebuild. It's the same loop for any future graphic, whether it's a background, a tileset, or a sprite.

7. A second graphic: transparency and depth

The same procedure repeats identically for a second graphic meant for a different plane — for example, one closer to the viewer than the background already drawn, with trees or other elements that will later in this manual be combined via independent per-plane scrolling — with two differences from the first one.

The first shows up right in the canvas creation dialog: instead of a white or black background, it's best to choose the transparent background option, so that the areas of the graphic left undrawn show, in the editor's preview, whatever is behind them.

The actual mechanism behind that transparency has a quirk specific to indexed color worth spelling out: there's no special transparency color reserved by the format. What actually happens is that, by convention, both the editor and later SGDK treat the palette's first entry — index 0 — as transparent, whatever the actual color assigned to it happens to be. It could be pink, green, or anything else; since it's never drawn on screen, the color itself is irrelevant. What does matter is that no other element of the drawing uses that first palette slot, even by accident, because it would end up invisible without the editor warning about the problem. The transparent-background option in the creation dialog is simply an editor convenience for previewing how that convention will look in practice.

The second difference is one of scale: if this second graphic contains larger, more spaced-out elements than a continuous texture like a sky, it can help to set the grid to a larger multiple of 8 — 16×16 , for example — to visually group sets of four tiles without losing track of the real unit. The 16-color limit and the tally-column discipline from section 4 still apply just the same. The rest of the process — generating the canvas palette, switching to indexed mode,

saving as PNG, updating `resources.res` — is identical to what was covered in the previous sections.

8. The Mega Drive's color model: palettes, VRAM, and their real limits

With at least one graphic already working, this section pauses to look at how the indexed color used since section 5 actually works under the hood, and at two hardware aspects left out of the previous examples.

8.1 Indexed color as a paint-by-numbers book

Indexed color mode can be thought of with the same logic as one of those paint-by-numbers books that put a number inside each area to indicate which color to use: the image data doesn't directly store "this pixel is light blue," but rather "this pixel uses color number 6," and it's the palette, kept separately, that defines which actual color corresponds to number 6 at any given moment. The Mega Drive numbers its 64 possible on-screen colors from 0 to 63, and works under exactly that same numbering logic.

The practical consequence is that changing which actual color occupies a palette slot instantly changes the color of every pixel using that slot, without needing to touch a single pixel of the image. Swapping two entries in an already-indexed graphic's palette — light blue and white, say — makes everything that used to be light blue turn white and vice versa, across the entire image at once.

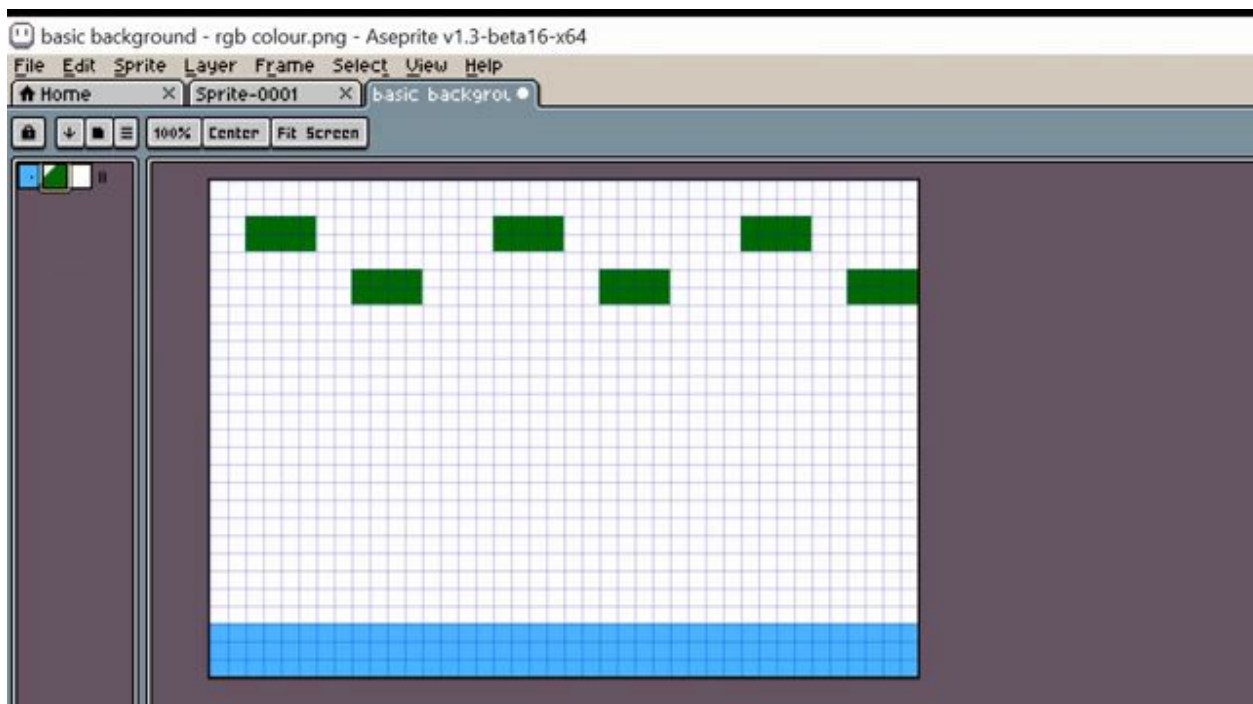


Figure 7: Background with two palette entries swapped

The same background from section 4 with two entries of its already-indexed palette swapped: the light blue sky with green clouds appears inverted without a single pixel of the drawing having been touched.

Far from being a mere curiosity, this property is the basis of a technique widely used in commercial games of the era: palette cycling, which consists of reassigning, frame by frame in real time, which actual colors occupy certain palette slots, without redrawing or moving a single tile. The visual result is a smooth animation — flowing water, blinking lights, pulsing logos — achieved at a fraction of the processing cost that redrawing the scene would take. The opening logo and the neon sign in the introduction to *Streets of Rage*, for example, use exactly this mechanism.

8.2 A limitation of the tool used so far, not of the hardware: per-tile palettes

All the code seen up to this point assigns a single palette to an entire background plane, via the `pal` parameter of `TILE_ATTR_FULL()` in the call to `VDP_drawImageEx()` from Chapter 7. It's worth keeping in mind that this is a limitation of how the function has been used so far, not a limitation of the hardware: the Mega Drive's VDP actually allows a different palette to be assigned to each individual tile within the same plane, not just to the plane as a whole. A single background could combine tiles using `PAL0`, `PAL1`, `PAL2`, and `PAL3` simultaneously, which enables, for instance, scenes with far more than 16 colors visible at once, sensibly distributed across different areas.

Taking advantage of that tile-by-tile flexibility with the tools used up to this point — a pixel-art editor plus `VDP_drawImageEx()` — is cumbersome, since it would require manually chopping up the image and assigning each fragment to a base tile with its own `pal` parameter. There are third-party tools, outside Aseprite itself, aimed specifically at assigning palette and tile priority visually over an already-drawn graphic; it's worth looking into them if a project needs to take advantage of this flexibility systematically, though they aren't part of the basic workflow covered in this chapter.

8.3 VRAM and automatic tile deduplication

The Mega Drive's VRAM stores tiles, not a continuous grid of pixels, and its capacity — although considerably larger than that of 8-bit systems like the NES or the Master System — isn't infinite: it has to be split between the background, the foreground, and the sprites for everything on screen. The most effective strategy for not running it dry is, simply, reusing tiles: a background made of a handful of tiles repeated in a pattern (a *tileset*) takes up much less space than the same scene drawn as a single image with no repetition.

SGDK helps with this automatically: when converting an image into a *tileset*, it detects which tiles are, in fact, the same tile flipped horizontally or vertically relative to one already present, and instead of duplicating the data in VRAM it stores a single tile and marks the rest with the corresponding flip flag (the same `flipH/flipV` from `TILE_ATTR_FULL()` already used in Chapter 7). An emulator with VRAM debugging tools lets you inspect this savings directly.

The emulator's VRAM debugging tool shows the CRAM with all four palettes loaded and, below it, the individual tiles: a single marked tile reappears flipped in several positions without having been duplicated in memory.

The practical takeaway is that, when designing a graphic — especially a background meant for a full level and not just a test screen — it's worth thinking of it from the start as a *tileset*: a relatively small set of distinct tiles, combined and repeated (flips included) to build a scene much larger than the unique-tile count would suggest. There's no need to manage tile flipping by hand: SGDK detects it on its own. What does depend on the artist is drawing in a way that gives it tiles to detect in the first place.

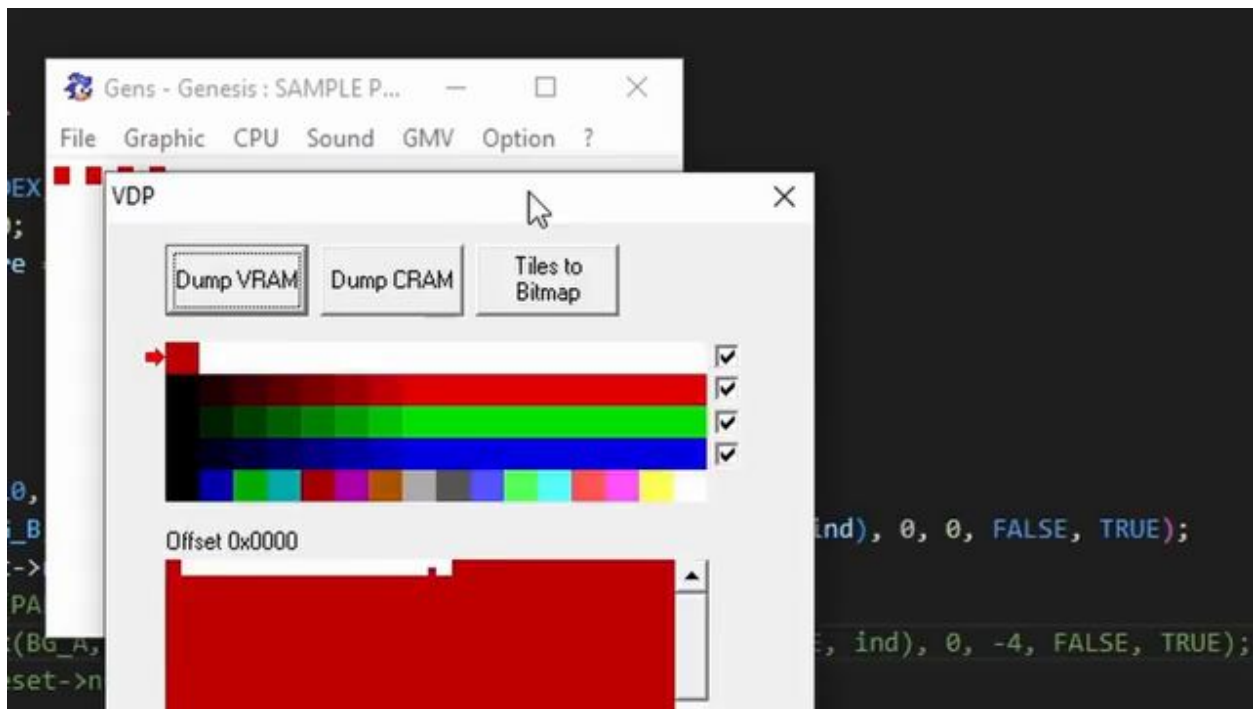


Figure 8: VRAM dump with one tile flipped in several positions

Review exercises

1. A graphic is drawn at 300×200 pixels and an attempt is made to declare it as an IMAGE resource in resources.res. Explain why SGDK will reject it and what minimal change would fix it.
2. A teammate switches their canvas's color mode from RGB to indexed as soon as they start drawing, without first trimming the imported 512-color palette. After drawing a light-blue sky, they notice part of the image shows shades they never painted. What's going on, and at which step of the workflow should they have intervened to avoid it?
3. Why is a single Mega Drive graphic limited to 16 colors and not the 64 the console can display in total? Relate your answer to the concept of palettes (PAL0 through PAL3) already covered when describing the VDP.
4. Explain, in your own words, why the transparency of a graphic in indexed mode doesn't depend on any particular color but on a palette slot, and what visual glitch results if something is accidentally drawn using that slot.
5. What's the difference between flipping a tile (flipH/flipV) and duplicating it, in terms of VRAM usage? Why is it in a Mega Drive artist's interest to design backgrounds with reusable tiles in mind from the start, rather than fixing the graphic after noticing it takes up too much VRAM?

Chapter 9 — Scroll and Parallax Scrolling

Chapter 7 left a static image on each of the VDP’s two background planes. This chapter sets those same images in motion: it’s the first step toward scrolling and, in particular, toward *parallax scrolling*, the technique so characteristic of 16-bit games in which the background and foreground move at different speeds to suggest depth. Without parallax, a game world looks oddly flat; with it, even in its simplest form, it starts to feel three-dimensional.

Chapter 2 already introduced, at a conceptual level, the three scroll granularities offered by the VDP: whole-plane, per-tile (or per-column, on the vertical axis), and per-line. Here they get code for the first time, starting with the simplest of the three: whole-plane scrolling, which moves the entire screen at once. The other two variants — per-tile/column and per-line, which allow moving strips of the screen at different speeds within the same plane, the classic effect used in racing games or certain boss fights — are mentioned but not developed in depth yet: they require more code and a deeper understanding of how VRAM is organized, and they are territory for a dedicated chapter on levels with scrolling that follows the player, later in this manual.

Implementing plane scrolling requires only a couple more lines of code beyond what was already written in Chapter 7. What it does demand, and in depth, is understanding several underlying ideas: what the difference is between specifying a position in tiles versus pixels, what three distinct sections a typical SGDK program has and what happens if code is written in the wrong section, and why the VDP stores each plane at a fixed size of 512×256 pixels even though the visible screen is much smaller.

Prerequisites

Before continuing, it helps to have:

- Completed Chapter 7, “Displaying Backgrounds: BG_A/BG_B and VDP_drawImageEx”: this chapter builds on the same project, with the same two planes (bg1 on BG_B and fg1 on BG_A) already declared, palettized, and drawn on screen.
- An understanding of the concepts of tile, VRAM, and palette covered in Chapter 2, as well as the distinction between BG_A (foreground) and BG_B (distant background).
- The fix16/fix32 fixed-point types introduced in Chapter 6, which reappear toward the end of this chapter.
- Basic knowledge of C: global versus local variables, and the structure of a while loop.

1. Configuring the scroll mode: `VDP_setScrollingMode`

The VDP doesn't move a plane simply because the programmer changes an x or y coordinate every frame: first it must be explicitly told what mode that plane's scroll will operate in. That configuration is done once, with the function `VDP_setScrollingMode()`:

```
void VDP_setScrollingMode(u16 hscroll, u16 vscroll);
```

The two parameters configure, separately, the horizontal and vertical scroll modes. As previewed in Chapter 2, the horizontal mode offers three possible granularities:

- `HSCROLL_PLANE` — the offset is applied to the whole plane: the entire screen moves uniformly. It's the simplest mode and the only one used in this chapter.
- `HSCROLL_TILE` — the offset can be defined independently for each 8-pixel-tall block (a tile). This allows, for example, different horizontal strips of the screen to move at different speeds.
- `HSCROLL_LINE` — the offset can be defined line by line (each 1-pixel-tall line), the finest granularity possible and the one used by the more elaborate scroll effects, such as the curves of a racetrack.

The vertical mode offers two granularities: `VSCROLL_PLANE`, analogous to `HSCROLL_PLANE` (the whole plane moves as a unit), and `VSCROLL_COLUMN`, which allows an independent offset per 16-pixel-wide column. Unlike horizontal scrolling, the Mega Drive does not offer a line-by-line vertical scroll mode; the hardware itself gives preference to fine horizontal scrolling, on the logic that the vast majority of games scroll mainly in that direction.

If old sample code ever turns up using a constant called `VSCROLL_2TILE` for vertical scrolling, keep in mind that it's deprecated: SGDK replaced it with `VSCROLL_COLUMN`, which fills the same role (independent offset per vertical strip) with a more precise name. Using `VSCROLL_2TILE` directly in SGDK 2.11 no longer compiles.

Since this chapter deals with the simplest case, it uses whole-plane scrolling in both directions:

```
VDP_setScrollingMode(HSCROLL_PLANE, VSCROLL_PLANE);
```

Don't forget the semicolon. A common typo when writing this line for the first time is forgetting the trailing semicolon, which produces a compilation error. This isn't specific to this function — it applies to any statement in C — but it's worth keeping in mind because the compiler's error message in these cases doesn't always point to the exact line of the omission, but to the following line.

2. Moving the plane: `VDP_setHorizontalScroll` and `VDP_setVerticalScroll`

With the scroll mode configured, what's still missing is the part that actually moves the plane frame by frame: telling the VDP, on every pass of the main loop, what the current offset is. Two functions handle that, one per axis:

```
void VDP_setHorizontalScroll(VDPPlane plane, s16 value);  
void VDP_setVerticalScroll(VDPPlane plane, s16 value);
```

The first parameter is, again, the plane the offset is applied to (`BG_A` or `BG_B`). The second, `value`, is the offset value, with one important detail that sets these two functions apart from `VDP_drawImageEx()`: here `value` is measured in pixels, not tiles. A value of 8 shifts the plane 8 pixels — that is, exactly one tile — not 8 tiles (64 pixels).

VDP_drawImageEx() uses tiles; VDP_setHorizontalScroll() uses pixels. This is a common source of confusion: the x and y coordinates of VDP_drawImageEx(), already used in Chapter 7 to position an image, are measured in tiles (multiples of 8 pixels). The values passed to VDP_setHorizontalScroll() and VDP_setVerticalScroll(), on the other hand, are measured directly in pixels, and although they often appear together in the same program, they don't share a unit of measurement.

For the vertical axis, a positive value moves the plane up and a negative one moves it down (it helps to think of the value as how much “the window” looking at the plane is shifted, rather than which way the content visually moves). It's better to verify the exact sign experimentally for each case rather than memorize it: what matters is that value is of type `s16`, a signed integer, so both directions are available without any additional function.

3. Where to put each line of code: the three sections of main()

Before writing the frame-by-frame offset code, it's worth pausing on a point that isn't exclusive to scrolling but applies to any program built with SGDK, and which becomes especially visible in this chapter because a misplaced line produces a spectacularly wrong result. In practice, every SGDK `main()` is organized into three clearly distinct sections:

1. **Global section:** everything declared before `int main()`, outside any function. These are variables that need to exist for the entire life of the program and be visible from different functions (or, as in this chapter, to keep their value from one frame to the next).
2. **Initial setup section:** the stretch of code between the opening brace of `main()` and the `while(1)` that starts the main loop. Everything here runs exactly once, when the program starts: loading palettes, drawing the backgrounds, configuring the scroll mode.
3. **Main loop section:** the contents of `while(1) { ... }`. Everything here runs over and over, once per frame, for as long as the program is running.

The chapter's full code, now with the offset variable declared globally and scrolling working, looks like this:

```
u16 ind = TILE_USERINDEX;
int hscroll_offset = 0;

int main()
{
    PAL_setPalette(PAL0, bg1.palette->data, DMA);
    VDP_drawImageEx(BG_B, &bg1, TILE_ATTR_FULL(PAL0, FALSE, FALSE, FALSE, ind), 0,
    ↪ 0, FALSE, TRUE);
    ind += bg1.tileset->numTile;

    PAL_setPalette(PAL1, fg1.palette->data, DMA);
    VDP_drawImageEx(BG_A, &fg1, TILE_ATTR_FULL(PAL1, FALSE, FALSE, FALSE, ind), 0,
    ↪ 0, FALSE, TRUE);
    ind += fg1.tileset->numTile;

    VDP_setScrollingMode(HSCROLL_PLANE, VSCROLL_PLANE);

    while(1)
    {
        VDP_setHorizontalScroll(BG_B, hscroll_offset);
        hscroll_offset += 1;
    }
}
```

```

        SYS_doVBlankProcess();
    }

    return (0);
}

```

The variable `hscroll_offset` is declared globally, together with `ind`, because it needs to preserve its value from one frame to the next: if it were declared inside the while loop, it would reset to 0 on every pass and the plane would never advance. Inside the loop, each pass does three things: it tells the VDP the current scroll position with `VDP_setHorizontalScroll()`, increments that position by 1 for the next pass, and calls `SYS_doVBlankProcess()`.

`SYS_doVBlankProcess()` synchronizes the program with the VDP's vertical blanking interval (VBlank), the brief span between frames during which the screen isn't being drawn and it's safe to update VRAM without producing visual artifacts. For application-level programming, it's enough to know that this call must be present once per loop pass, typically at the end. In SGDK versions prior to 1.60 the equivalent function was called `VDP_waitVSync()`; if that name shows up in old projects, it fills the same role and should be replaced with `SYS_doVBlankProcess()` when porting to SGDK 2.11.

With this code, the intended effect is already achieved: the background scrolls continuously to the right, at a rate of 1 pixel per frame (which on the Mega Drive amounts to roughly 60 pixels per second, given that the system runs at 60 frames per second in the NTSC variant).

Reversing the scroll direction or changing its speed doesn't require any new function: it's enough to change the increment added to `hscroll_offset` on each pass. Subtracting instead of adding reverses the direction; adding or subtracting a number greater than 1 speeds up the movement.

```
hscroll_offset -= 2; // moves to the left, twice as fast
```

The scroll can look a bit choppy in an emulator than on a physical console connected to a period-accurate TV: different emulators handle timing and video filtering differently. If it's possible to test the same `rom.bin` on real hardware, the final result looks better there.

4. A deliberate mistake: what happens if code goes in the wrong section

The distinction between the initial setup section and the main loop, presented in the previous section as a simple organizational convention, has very concrete consequences if it isn't respected. To see this clearly, it's worth deliberately reproducing a common mistake: moving the palette-loading and image-drawing calls, which should run only once, inside the while loop, so that they run every frame.

```

while(1)
{
    PAL_setPalette(PAL0, bg1.palette->data, DMA);
    VDP_drawImageEx(BG_B, &bg1, TILE_ATTR_FULL(PAL0, FALSE, FALSE, FALSE, ind), 0,
    ↪ 0, FALSE, TRUE);
    ind += bg1.tileset->numTile;

    PAL_setPalette(PAL1, fg1.palette->data, DMA);
}

```



```

VDP_drawImageEx(BG_A, &fg1, TILE_ATTR_FULL(PAL1, FALSE, FALSE, FALSE, ind), 0,
↪ 0, FALSE, TRUE);
ind += fg1.tileset->numTile;

VDP_setHorizontalScroll(BG_B, hscroll_offset);
hscroll_offset += 1;

SYS_doVBlankProcess();
}

```

The result, far from being a simple flicker or slowdown, is obvious visual corruption almost immediately: blocks of tiles with colors and patterns that don't match either of the two original images.

The cause is exactly the VRAM bookkeeping discipline mentioned in Chapter 7. The variable `ind` is global and is never reset: each pass of the loop adds `bg1.tileset->numTile` and `fg1.tileset->numTile` again to its already-incremented value, so on every frame the tiles are being written to a VRAM position that's different — and increasingly further away — from where they should be. Sixty times per second, the program keeps writing new tiles to VRAM addresses that advance out of control, until they end up overwriting positions already used by something else (or, simply, until the available VRAM space runs out and the count wraps around). The visual result is the mix of incorrect tile data described above.

The general rule: configure once, update every frame. This experiment serves to fix a rule that's worth applying from here on to any SGDK program: everything that prepares an initial state — loading a palette, uploading tiles to VRAM, configuring the scroll mode — belongs in the setup section, before the `while`. Everything that needs to be reevaluated every frame — reading the controller, moving a sprite, updating the scroll position — belongs in the main loop. Mixing the two doesn't always produce a result as visually dramatic as this experiment, but it almost always produces some kind of error that's hard to diagnose if you don't know what to look for.

Reverting the change — moving those lines back to the initial setup section, before the `while` — restores the correct behavior.

5. The real size of a plane: 512×256 pixels

With horizontal scrolling working, a problem shows up that no code error explains: once the scroll moves far enough, the plane starts showing areas with no graphics, like black gaps.

The cause is a hardware characteristic, not a program defect. The VDP doesn't store each plane at exactly the size of the visible screen (320×224 pixels in standard mode); instead it reserves a much larger tile board in VRAM: 512×256 pixels, that is, 64×32 tiles. The reason is that this extra space lets the VDP always have tiles already loaded above, below, to the left, and to the right of the visible area, so that scrolling can show new content instantly at the edges of the screen instead of having to load it on the fly, frame by frame, which would be too slow to sustain 60 frames per second.

The problem arises when the image drawn on the plane is smaller than that 512×256 board: the VDP has extra tiles reserved, but those tiles were never filled with content, so they appear empty (black gaps) as soon as the scroll advances far enough to reveal them. To make the problem more visible, it's enough to also enable vertical scrolling on the same plane, using the same offset variable:

```
VDP_setHorizontalScroll(BG_B, hscroll_offset);
VDP_setVerticalScroll(BG_B, hscroll_offset);
hscroll_offset -= 2;
```

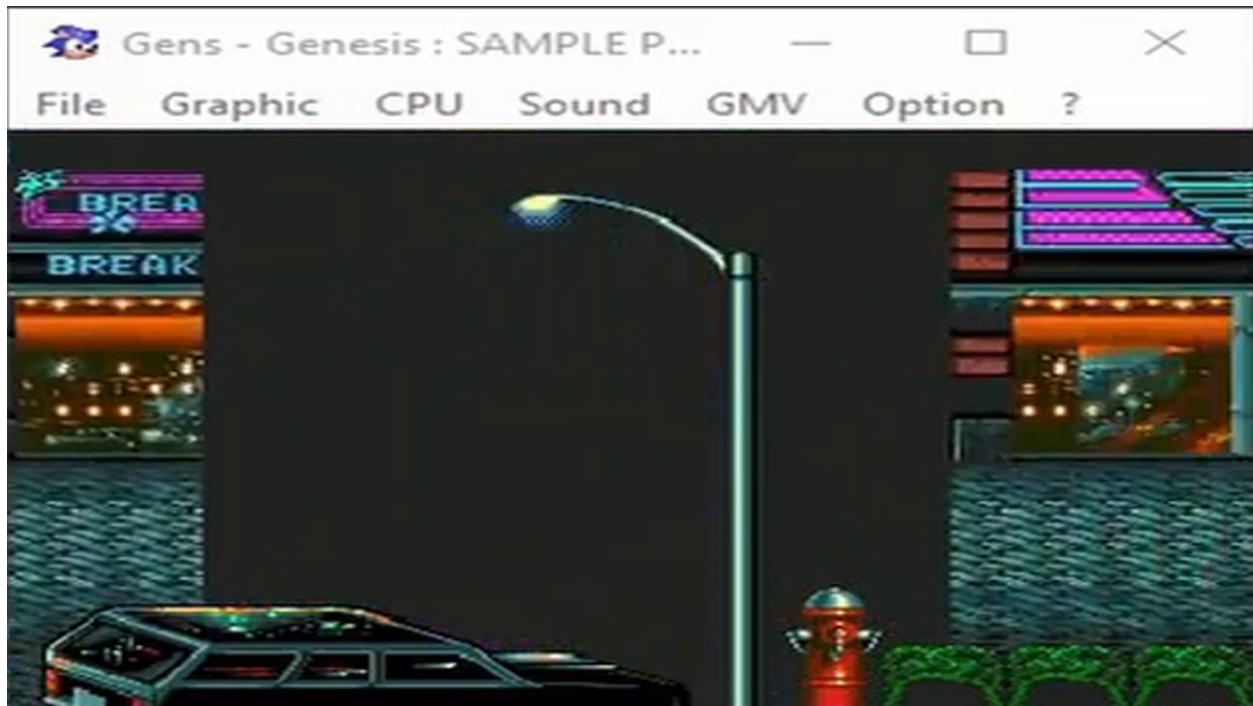


Figure 9: Empty gaps when scrolling outside the area occupied by the original image

With horizontal and vertical scrolling active on a 320×224 image, the rest of the plane’s 512×256 board becomes visible: outside the area occupied by the original image there is nothing but empty gaps.

The VDP also doesn’t reserve the same amount of extra space on both axes: it keeps proportionally less vertical margin than horizontal, which is no accident, since the vast majority of Mega Drive games scroll mainly along the horizontal axis, and the hardware is designed with that priority in mind.

The value of 512×256 pixels (64×32 tiles) is the default configuration and the most common one in practice, and it’s the one used in this chapter. SGDK does, however, allow configuring planes of other sizes (32, 64, or 128 tiles per axis, in any combination) via the function `VDP_setPlaneSize()`, as long as the available VRAM is enough for the resulting board. This option isn’t needed yet for what’s covered in this chapter, but it’s worth knowing that plane size is configurable and not a fixed hardware constant.

6. The solution: images at the plane’s full size

The solution to the gap problem is, at bottom, very straightforward: if the plane reserves 512×256 pixels of tiles, it needs to be given an image of that same size so that no areas are left unfilled.

The original 320×224-pixel image, next to one of the new 512×256 replacement images; the checkered background marks the transparent areas.



Figure 10: Comparison between the original 320×224 image and the new 512×256 one

An important practical detail when replacing an image with a larger one: there's no need to change the resource names (bg1, fg1) already used by the code in `main.c`. It's enough to replace, inside `resources.res`, the name of the image file each resource references:

```
IMAGE bg1 "background - 512 256.png" BEST ALL
IMAGE fg1 "foreground - 512 256.png" BEST ALL
```

A trick to force resource recompilation. Sometimes, after editing only `resources.res`, SGDK doesn't reliably detect the change and keeps using the previously compiled image data. A practical and widely used trick to force recompilation is: type any character into `main.c` (for example, a stray letter) and save; then delete that character and save again. That double save of `main.c` is enough for the build system to notice that something changed and reprocess `resources.res` from scratch. It's not an elegant solution, but it's effective and avoids having to manually delete intermediate files.

When recompiling with an image taller than the previous one, an additional vertical misalignment can appear: the result looks shifted from what was expected. The solution is the same positioning tool already used in Chapter 7, the `y` parameter of `VDP_drawImageEx()`, this time with a negative value to shift the image upward:

```
VDP_drawImageEx(BG_B, &bg1, TILE_ATTR_FULL(PAL0, FALSE, FALSE, FALSE, ind), 0, -4,
    ↪ FALSE, TRUE);
// ...
VDP_drawImageEx(BG_A, &fg1, TILE_ATTR_FULL(PAL1, FALSE, FALSE, FALSE, ind), 0, -4,
    ↪ FALSE, TRUE);
```

This parameter is measured in tiles, as already explained in Chapter 7: a value of -4 shifts the image 4 tiles (32 pixels) upward from its original position. With this adjustment, both planes — now at the full 512×256 size — end up correctly aligned, with no visible gaps at any point

along the scroll's range.

7. Two speeds: parallax proper

Up to now, both the background (BG_B) and the foreground (BG_A) moved — when they moved at all — at the same speed, controlled by the same variable `hscroll_offset`. That's not parallax scrolling in the strict sense: it's just a single plane in motion. The depth effect that gives the technique its name only appears once each plane moves at a different speed, typically with the foreground moving faster than the distant background, mimicking how nearby objects appear to move faster than distant ones when you move in real life.

Implementing it requires no new function: it's enough to declare a second, independent global offset variable and use it to move BG_A:

```
u16 ind = TILE_USERINDEX;
int hscroll_offset = 0;
int hscroll_offset_fore = 0;

int main()
{
    // ... palette loading and drawing of bg1 and fg1, as before ...

    VDP_setScrollingMode(HSCROLL_PLANE, VSCROLL_PLANE);

    while(1)
    {
        VDP_setHorizontalScroll(BG_B, hscroll_offset);
        hscroll_offset -= 1;

        VDP_setHorizontalScroll(BG_A, hscroll_offset_fore);
        hscroll_offset_fore -= 2;

        SYS_doVBlankProcess();
    }

    return (0);
}
```

With the background advancing at 1 pixel per frame and the foreground at 2 pixels per frame — twice the speed — the result is already recognizable parallax scrolling: the foreground visibly glides faster than the background, suggesting it's closer to the camera.

A visible limitation: the repeat seam. When a plane's image is narrower than the full scroll range it's meant to cover, at some point the pattern starts repeating and a "seam" or edge can be noticed where one copy of the image ends and the next begins. This is a natural limitation of working with a fixed-width image rather than a procedurally generated level or one of arbitrary width, and it becomes less noticeable the wider and more varied the source image is. Long levels that follow the player, with sections much wider than a single screen, are revisited later in this manual.

8. The limit of integers: why int isn't enough

With parallax working at speeds of 1 and 2 pixels per frame, a natural question comes up: what happens if a slower speed is needed, for example half a screen's worth of scroll every two frames, that is, 0.5 pixels per frame? Trying that value directly doesn't work:

```
hscroll_offset -= 0.5; // produces no visible movement
```

The plane stays completely still. The reason has nothing to do with the VDP or any of the scroll functions: it's a limitation of the very data type used for `hscroll_offset`. It was declared as `int`, an integer, which by definition cannot represent fractions. When 0.5 is subtracted from an `int`, the C compiler truncates that result to the nearest integer before storing it; the practical effect is that the stored integer value never changes from one frame to the next, and therefore neither does the scroll position. The same problem shows up with any increment smaller than 1 in absolute value, not just 0.5: 0.8 subtracted repeatedly from an `int` still rounds to 0 at every step, so the visual result is identical to not moving the plane at all.

The solution, already familiar from Chapter 6, is to use a fixed-point type instead of an integer. `fix32` (or its lighter variant `fix16`) reserves part of its bits to represent a fraction, instead of using all its bits for the integer part. Declaring the offset as `fix32` and using the `FIX32()` macro to express the fractional increment, subtracting half a pixel per frame does produce a real accumulated offset:

```
fix32 hscroll_offset_f = FIX32(0);

// inside the loop:
hscroll_offset_f -= FIX32(0.5);
VDP_setHorizontalScroll(BG_B, F32_toInt(hscroll_offset_f));
```

The key is in the last line: `VDP_setHorizontalScroll()` still expects an `s16` in whole pixels, so the fixed-point value accumulates with its fraction intact in `hscroll_offset_f`, and is only converted to an integer with `F32_toInt()` at the moment it's passed to the VDP. Even though you'll never see less-than-one-pixel movement in a single frame (the pixel itself is the smallest unit that can be shown on screen), the accumulated fraction means that, every two frames, the resulting integer value does advance by one pixel, achieving the targeted average speed of 0.5 pixels per frame. This same technique — accumulating in fixed point and converting to an integer only at the end — is what `SGDK` uses internally for sprite coordinates, and it comes up again in any situation that requires a fractional speed, such as a character's jump.

Integers (`int`, `u16`, `s16`, etc.) are enough for speeds of 1 pixel per frame or more, but any speed slower than that requires a fixed-point type such as `fix32` or `fix16`.

Review exercises

1. Explain why the variable that accumulates the scroll offset (for example, `hscroll_offset`) has to be declared outside `main()`, and what would happen if it were declared inside the while loop instead of outside it.
2. A teammate mistakenly moves the call to `VDP_setScrollingMode()` inside the while loop, along with the rest of the per-frame update code. Unlike moving the `PAL_setPalette()` and `VDP_drawImageEx()` calls, this particular call doesn't re-upload data to VRAM every frame. Even so, why is it still bad practice to leave it there, even though the program doesn't get visually corrupted the same way as in section 4?
3. You want a plane to shift exactly 3 tiles to the right using `VDP_setHorizontalScroll()`. What value should be passed to the function? Justify the answer based on this function's

unit of measurement.

4. A 320×224-pixel image is drawn on a plane, and horizontal and vertical scrolling are enabled on that plane. Describe what will be seen when scrolling beyond the edge of the original image, and propose the correct solution without resorting yet to fixed-point types or per-tile scrolling.
5. Why does the statement `hscroll_offset -= 0.5;` produce no movement if `hscroll_offset` was declared as `int`? Rewrite it using `fix32`, `FIX32()`, and `F32_toInt()` so that the plane shifts, on average, 0.5 pixels per frame.

Chapter 10 — H40 and H32 Resolution Modes

How to switch horizontal resolution with a single line of code, and what practical criteria come into play when choosing between H40 and H32

The chapter on the VDP already introduced, at a conceptual level, the Mega Drive's two horizontal resolution modes and their associated sprite limits. This chapter picks that idea back up from the practical side: what code is needed to activate each mode, what side effects that change brings, and why much of the industry ended up using one mode or the other depending on the situation.

1. Quick recap: H40 and H32

As a reminder, the Mega Drive supports two horizontal resolutions:

- **H40**: 320×224 pixels (40 tiles, 8 pixels wide each). This is the default mode and the one used by the vast majority of commercial games. It allows up to 80 simultaneous sprites on screen, with a maximum of 20 per horizontal line.
- **H32**: 256×224 pixels (32 tiles wide). Sprite limits drop to 64 on screen and 16 per line.

The reduction in sprites under H32 isn't arbitrary: by displaying fewer pixels of width, the VDP also lowers its internal clock speed to match, and that limits how many sprites it can process in the time available per line. Vertical resolution, on the other hand, doesn't depend on this horizontal mode: the standard is 224 visible lines (V28 mode), available on both NTSC and PAL consoles. PAL consoles additionally support a V30 mode of 240 lines, exclusive to that region; this chapter doesn't go into it in depth, so it's enough to know it exists and adds 16 extra vertical lines only on PAL systems.

2. How to activate it in SGDK

If a program doesn't explicitly specify which mode to use, SGDK starts up in H40 by default. Switching to H32 (256 pixels wide) takes a single line of code, placed anywhere before the screen is drawn:

```
VDP_setScreenWidth256();
```

This function takes no parameters: it changes the horizontal resolution to 256 pixels the moment it's called. Symmetrically, there's `VDP_setScreenWidth320()` to go back to (or stay in) the 320-pixel H40 mode:

```
VDP_setScreenWidth320();
```

Neither function takes any arguments: each one directly sets its own width. This also means a program can switch modes at different points during execution — for example, using H32 on a title screen and H40 during gameplay — as long as the corresponding call is made before each screen is displayed.

3. The visual result: a speed effect that isn't really one

If the same project is compiled in both modes — say, a wide level map that scrolls automatically at a rate of one pixel per frame, using `MAP_scrollTo()` as covered in the previous chapter — and the two results are run side by side, the width difference is obvious: one window shows 320 pixels and the other 256. What's most striking, however, isn't the width itself, but that the map in H32 gives the impression of scrolling *faster* than the one in H40, even though the scrolling code is exactly the same in both cases.

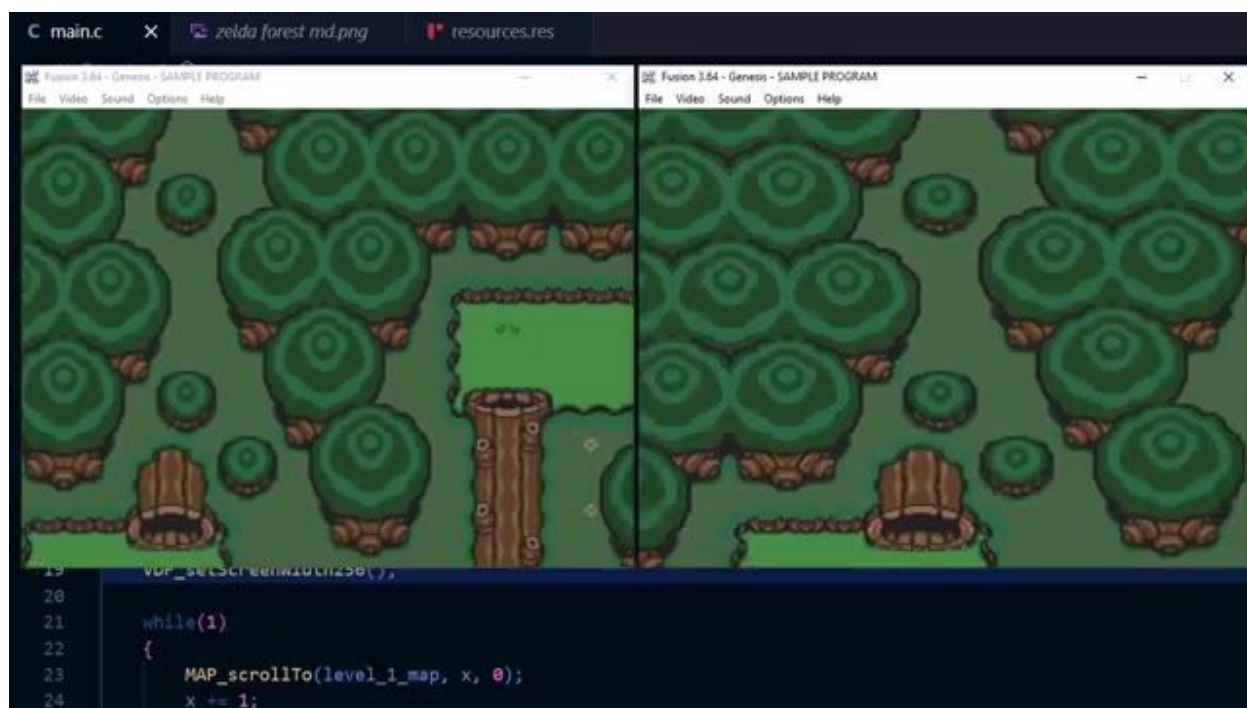


Figure 11: The same map in H40 and H32, a few seconds after the scroll starts

The H32 window (right) appears to have advanced further than the H40 one (left), even though both scroll at the same rate of one pixel per frame.

That sense of different speed is a perceptual effect, not an actual difference in speed: since the H32 screen has fewer pixels of width to cross, the same number of pixels scrolled per second represents a larger fraction of that screen's total width, which is why it's perceived as faster movement. It's worth keeping this in mind when tuning a game's scroll speed: a value that feels right in H40 may feel too fast if the same project is switched to H32 without revisiting that constant.

4. Why choose H32: examples from the industry

Although most Mega Drive games use H40, a fair number of commercial titles choose H32 under various circumstances. A community researcher known as FirebrandX compiled, years

ago, an extensive and fairly well-cited list of games that use the 256×224 mode during different parts of the game (including both title screens and gameplay itself), and reviewing the patterns in that list helps clarify what criteria might come into play when making this decision in a project of your own.

A common pattern is low-resolution title screens followed by a game that runs in H40, as happens in *Golden Axe*. A reasonable explanation is that it reduces the number of unique tiles needed to fill the title screen, and therefore the ROM space those tiles take up; this isn't a detail confirmed by its original developers, so it should be taken as a plausible guess rather than a verified fact.

Ports are another common case: games that already ran at 256 pixels wide on their original system, such as several Konami titles (*Sunset Riders*, *Lethal Enforcers*) or EA Sports sports games (the NHL series), use H32 to avoid having to redesign the art at a resolution different from that of the source system. But not every case is a conversion: *Shining Force 1* and *2*, both originally developed by Sega for the Mega Drive, also opt for H32, possibly to save space given the game's numerous character portraits. As with *Golden Axe*, there's no direct confirmation from the developers about this particular decision either.

5. H32 to reduce CPU load: software-rendered 3D engines

Another reason to choose H32 shows up in games that render pseudo-3D graphics in software instead of relying on the hardware's tile system: in those cases, the entire image is computed pixel by pixel on the CPU (or on an additional coprocessor, such as the SVP chip in *Virtua Racing*). With fewer pixels of width to compute, H32 mode can significantly ease that rendering load.

Not every game with an ambitious pseudo-3D engine resorts to H32, though: *Zero Tolerance*, with an engine of this kind that was fairly demanding for its time, uses H40 and instead shrinks the game viewport itself, leaving the rest of the screen occupied by the weapon panel and character portrait, in order to keep the frame rate up without sacrificing the system's full resolution. This is an option worth considering when trying to ease the load of expensive rendering without dropping out of H40: reduce the effective computation area and leave the rest of the screen for the HUD.

6. H40 for more simultaneous sprites

The decision can also go the other way, prioritizing H40 over H32 even when porting content from another system. In fan projects that aim to adapt games from other consoles to the Mega Drive, this dilemma comes up often; in an unofficial conversion of *Castlevania: Symphony of the Night* (originally from PlayStation, whose resolution doesn't exactly match either of the Mega Drive's two modes), the standard H40 mode was chosen instead of trying to replicate the original resolution. The usual reasons for such a choice are, on one hand, being able to show more simultaneous sprites on screen, and on the other, an aesthetic preference for how the final result looks at that resolution.

When working with art originally designed for a higher-resolution console, H40 also has an added advantage when displaying the result on a widescreen display (on an emulator or a widescreen CRT television): when the final image is stretched to fill that width, the result comes closer to the look of the original game than if it were based on an image generated in H32 mode.

7. The challenge of porting from other consoles

Choosing a resolution gets even trickier when the source system doesn't match either of the Mega Drive's two modes. The Game Boy Advance, for example, has a resolution of 240×160 pixels, quite different from the Mega Drive's on both axes. There are fan projects that explore exactly this scenario, such as a technical demo of *Aria of Sorrow* (originally from Game Boy Advance) running on the Mega Drive.

The difference in vertical resolution tends to be the most visible problem in this kind of conversion: the Game Boy Advance's original 160 pixels of height are considerably smaller than the Mega Drive's 224 lines in V28 mode, which forces a choice between showing black bars above and below, stretching the image, or redesigning the level outright to make use of the extra vertical space (the most labor-intensive of the three options, but also the one that tends to give the best results). On top of that resolution difference, there's also the need to adapt the original color palette to the Mega Drive's limit of colors per palette.

8. Why stretching sprites or levels is a bad idea

Stretching the image to compensate for a resolution difference is rarely a good idea. When a sprite is stretched vertically without first correcting the horizontal proportion, the result looks distorted and no longer aligns with the 8×8 pixel tile grid that all of the Mega Drive's hardware works on.

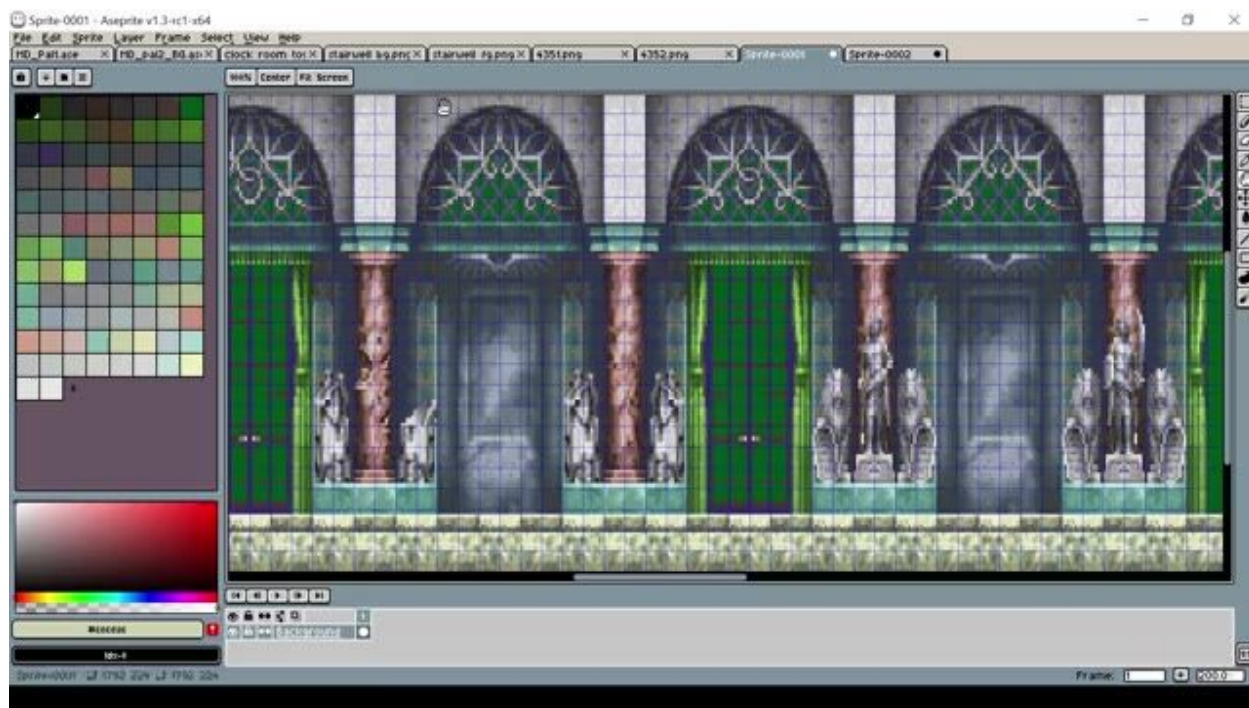


Figure 12: The same sprite, stretched vertically

Stretching distorts the art and throws it out of alignment with the 8×8 pixel tile grid.

That misalignment with the tile grid is the more serious problem, beyond the purely aesthetic one: the same 8×8 grid is used both to load graphics into VRAM and to calculate collisions at the map level, so once the art stops matching it, both the VRAM loading and the level and collision design become considerably harder to get right. In cases like *Aria of Sorrow*, it's

usually more practical to redesign a level on a case-by-case basis or add extra tiles than to accept that kind of distortion.

Review exercises

1. Explain the difference between H40 and H32 modes in terms of horizontal resolution and sprite limits per frame and per line.
2. What line of code needs to be added to switch from H40 to H32, and what happens if a program doesn't call either of the two resolution-switching functions?
3. In a side-by-side comparison of the same scrolling map, the H32 version appears to move faster than the H40 one even though both use the same scrolling code. Explain why this happens.
4. Name at least two distinct reasons why a commercial game might choose H32 mode instead of H40.
5. Explain why stretching a sprite or a level to fit a different resolution tends to cause problems beyond the purely aesthetic.

Chapter 11 — Large Level Maps with MAP_create

Chapter 7 introduced the IMAGE resource: a single graphic, up to 512×256 pixels, loaded in full onto one of the two background planes with `VDP_drawImageEx()`. That method is enough for many screens, such as a menu, a static background, or a title screen, but not for a large platformer level, in the style of a zone from *Sonic the Hedgehog*, which extends well beyond those dimensions. This chapter presents the solution SGDK offers for that case: splitting the map into two separate resources, a TILESET with the unique tiles that make it up and a MAP with the arrangement of those tiles across the whole level, and scrolling the visible portion of a map that, as a whole, is much larger than the screen.

The example used is a simplified version of the Green Hill Zone from *Sonic the Hedgehog*, reduced to 16 colors to respect the limit of a Mega Drive palette (the original uses more than one palette in the foreground, a resource revisited in Chapter 14, dedicated to combining several palettes in the same background). Before writing any code, it's worth taking a close look at the tiles that make up that map: the checkered pattern of the ground, the repeated foliage of the trees, and much of the landscape are built from very few distinct tiles, repeated over and over. That repetition is precisely what makes it possible to represent such a large map with a reasonable amount of memory.

The full Green Hill Zone map open in the editor: the checkered pattern of the ground and much of the landscape are built by repeating very few distinct tiles.

Prerequisites

Before continuing, it helps to have fresh in mind:

- Chapters 7 (“Displaying backgrounds: BG_A/BG_B and VDP_drawImageEx”) and 9 (“Scroll and parallax scrolling”): the idea of an 8×8-pixel tile, of background planes (BG_A and BG_B), of PAL_setPalette() to load a palette into CRAM, and the technique of scrolling a background by modifying a variable inside the main loop.
- The data types covered in Chapter 6, in particular u16, since they appear in several of this chapter’s functions.

1. Two resources from a single file: TILESET and MAP

The first step is, as always, to copy the graphic file into the res folder and declare the corresponding resources in `resources.res`. The first line declares a TILESET resource, which tells rescomp to examine the image and extract from it the set of unique tiles that make it up:

```
TILESET our_tileset "large green hill map.png" BEST ALL
```

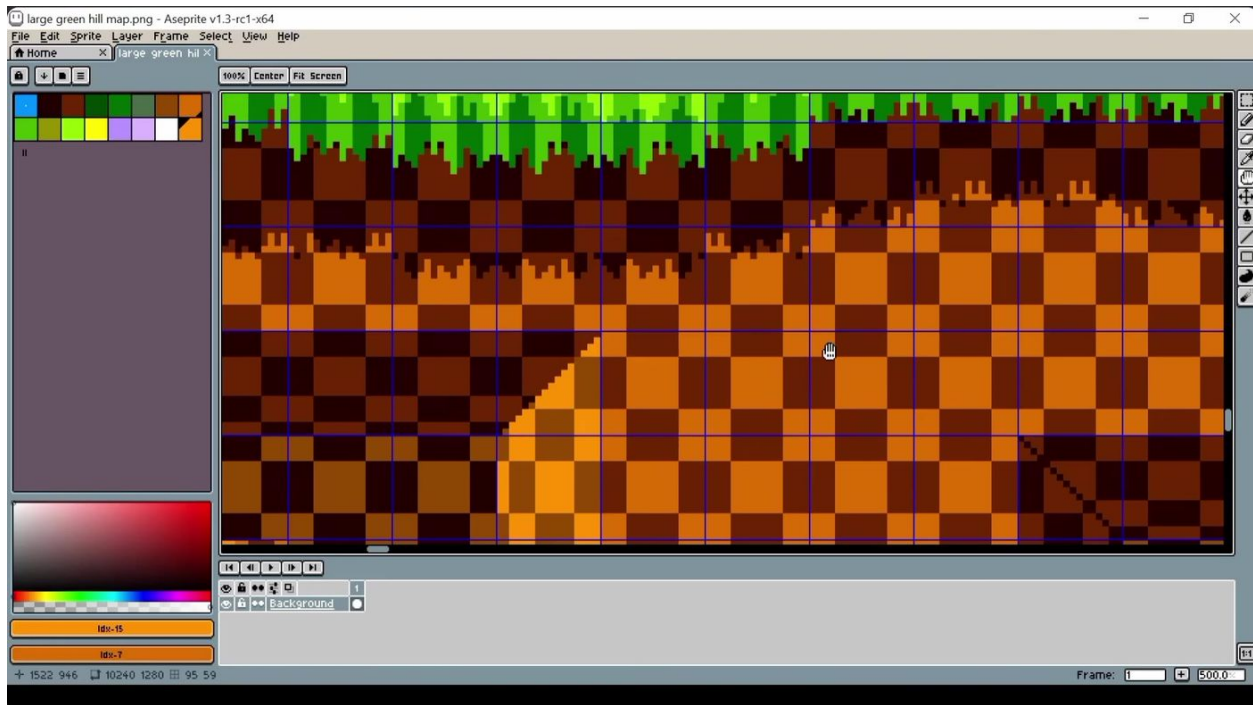


Figure 13: The full Green Hill Zone map in the editor

The first two parameters are already familiar from other graphic resources: the source file and the compression level (BEST, which lets rescomp automatically choose the most convenient method). The last parameter, ALL, is specific to a TILESET: it tells rescomp to discard both duplicate tiles and their mirrored variants when building the unique tile set. It's the default mode and the most convenient for a map like this one, since a platformer level tends to reuse the same ground, foliage, or sky tile many, many times, sometimes mirrored horizontally or vertically to vary the landscape without drawing a new tile.

The second line declares the MAP resource itself, which describes which tile goes in each position of the full map:

```
MAP our_level_map "large green hill map.png" our_tileset BEST 0
```

An important detail is that the MAP resource references the TILESET just declared by name (our_tileset), since a map on its own is just a grid of indices: it needs an associated tilesheet to know which tile corresponds to each index.

Both resources use the same image file as their source, and that's neither a mistake nor a coincidence. rescomp scans that single image from left to right and top to bottom, identifies each 8×8-pixel tile, and builds the TILESET by adding only the tiles it hasn't seen yet (thanks to the ALL mode, both an exact repeat and a mirrored version of a tile already seen are recognized and discarded). The MAP, on the other hand, preserves the complete layout of the level, but expressed as a reference to each tile's position within the already-reduced TILESET, instead of repeating the full graphic of each tile everywhere it appears.

This separation is exactly what allows a visually enormous map, like the one in this chapter, to take up relatively little space in ROM and VRAM: repeated information is stored only once, and the map only records "tile number 12, mirrored, goes here" instead of repeating the full graphic of tile 12 every time it appears.

2. Loading the tileset into VRAM and creating the map

With the resources already compiled and `#include "resources.h"` added to `main.c`, two variables are needed before `main()`: an index indicating where in VRAM the map's tiles will be loaded, and a pointer to the map itself, analogous to the `Sprite*` pointer already used for sprites.

For the index, SGDK provides a constant marking the first VRAM slot free for programmer-defined tiles (the tiles before that point are reserved for SGDK's internal use, such as the default text font):

```
u16 ind = TILE_USER_INDEX;
```

With `ind` already declared, the first step inside `main()` is to load the TILESET's tiles into VRAM with `VDP_loadTileSet()`, giving it the tileset to load, the VRAM index to start loading it at, and the transfer method (DMA, the same direct memory access already used to load sprites):

```
VDP_loadTileSet(&our_tileset, ind, DMA);
```

With the tiles already in VRAM, the next step is to create the map structure itself with `MAP_create()`, which needs three pieces of data: the MAP resource defined in `resources.res`, the background plane where it will be displayed (`BG_A` or `BG_B`), and the base attributes with which its tiles will be drawn:

```
Map* level_1_map;
```

```
level_1_map = MAP_create(&our_level_map, BG_B,  
    TILE_ATTR_FULL(PAL0, FALSE, FALSE, FALSE, ind));
```

That third argument is built with `TILE_ATTR_FULL(pal, prio, flipV, flipH, index)`, the same macro already used to combine palette, priority, flips, and VRAM index into a single value, only here it's applied to an entire map instead of a single sprite. The exact signature of `MAP_create()` is:

```
Map* MAP_create(const MapDefinition *mapDef, VDPPlane plane, u16 baseTile);
```

In this example, the map is displayed on `BG_B` instead of `BG_A` simply because the chapter works with a single plane and doesn't need to use both at once yet; it's a practical decision for this specific example, not a restriction of SGDK. In a real game, a platformer map like this one would normally go on the foreground plane (`BG_A`), reserving `BG_B` for a distant background with parallax, like the one seen in Chapter 9. `PAL0` is used because the map was prepared to fill the full 16 colors of a single palette, and no flip or special priority is applied (the three `FALSE` parameters), closing the call with the `ind` index already used when loading the tileset, so the map knows where, within VRAM, the tiles it references are located.

3. A separate PALETTE resource

One last step remains before compiling: loading the map's colors into the `PAL0` hardware palette, already indicated in `MAP_create()`. In previous chapters, an `IMAGE` or `SPRITE` resource includes a pointer to its own palette as part of its structure (`image.palette->data`), which allows the colors to be extracted directly from the same already-loaded graphic resource, without declaring anything additional.

A `MAP`, however, doesn't work the same way: its structure includes no palette field of its own, so there's never an `our_level_map.palette` to fall back on. Trying to write something like

`PAL_setPalette(PAL0, our_level_map.palette->data, DMA);`, copying the pattern used with an `IMAGE`, doesn't compile, because that field simply doesn't exist in the `Map` structure.

A `MAP` resource describes only the level's tile layout; it doesn't carry its own palette. The colors have to be obtained from elsewhere.

The solution is to declare an independent `PALETTE` resource in `resources.res`, telling it to extract the colors from that same image file already used for the tileset and the map. A `PALETTE` resource is, in itself, very simple: a name and a source file, nothing more.

```
PALETTE our_palette "large green hill map.png"
```

With this resource already compiled, loading the palette uses the `.data` field of the new `PALETTE` resource, instead of the `.palette->data` chain that isn't available for a map:

```
PAL_setPalette(PAL0, our_palette.data, DMA);
```

With all three pieces now in place (tileset loaded into `VRAM`, map created on `BG_B`, and palette loaded into `PAL0`), the program should now compile without errors.

4. The first result: an entirely sky-blue screen

Compiling and running at this point, however, the result isn't the expected Green Hill Zone landscape, but an entirely sky-blue screen, with no landscape element visible at all.

There's no error in the code: the map loaded and is being displayed correctly, it's just that `MAP_create()` starts showing the map from exactly its top-left corner, coordinate (0, 0). Since the full map is much taller than the screen's 224-pixel vertical resolution (see Chapter 10 on the `H40` and `H32` resolution modes), that top-left corner corresponds solely to the sky, well above where the ground, trees, and the rest of the recognizable landscape begin. The sky-blue screen is, in that sense, the correct result of displaying exactly that portion of the map: it simply isn't the portion of interest to show first.

5. Finding where to scroll the map, and `MAP_scrollTo()`

To display a more useful portion of the map, it needs to be scrolled vertically, and to know by how much, it helps to go back to the original image file in a pixel-art editor and locate, with the cursor, the `Y` coordinate where the recognizable landscape begins (just above the tree line, for instance). In this particular map, that coordinate turns out to be 768 pixels from the top. That figure isn't universal in any way: it's specific to this particular file, an artistic decision about where the landscape was drawn within the full image. Any other map requires repeating the same visual-inspection process to find its own starting coordinate.

With that coordinate already identified, `MAP_scrollTo()` takes care of scrolling the map. The function needs the map to scroll and the destination `X` and `Y` coordinates; in a first attempt, with no horizontal scrolling yet, it's enough to scroll vertically to the coordinate already found:

```
MAP_scrollTo(level_1_map, 0, 768);
```

Compiling again with this adjustment, the sky-blue screen gives way to the expected landscape: the trees, the flowers, the ground, and Green Hill Zone's characteristic checkered pattern, exactly as they appear in the original file, with no sign of tile corruption or visual artifacts.

6. Continuous scrolling: reusing the parallax scrolling idea

With the map now showing the correct portion, it's natural to extend the example so it scrolls continuously to the right, reusing exactly the same idea already explained in Chapter 9: replacing the fixed X coordinate value with a variable, and incrementing it on every pass through the main loop.

The variable is declared alongside the rest of the global variables, before `main()`:

```
int x = 0;
```

And the call to `MAP_scrollTo()` moves inside `while (TRUE)`, since it now depends on a variable that changes over time and it makes no sense to call it only once, followed by incrementing `x` on every pass:

```
while (TRUE)
{
    MAP_scrollTo(level_1_map, x, 768);
    x += 1;

    SYS_doVBlankProcess();
}
```

The result is an automatic, continuous scroll to the right, traversing the entire map from end to end with no player input at all. The landscape looks the same as in the original file, with no artifacts, only now in constant motion.

This automatic scrolling is merely a demonstration device, not the usual way to display a level of this kind in a finished game. A platformer instead needs a camera that follows the playable character's position, scrolling the map according to where the player is moving instead of doing so automatically and independently. Building that camera logic is left for Chapter 24; for now, it's enough to understand how a map larger than the screen is assembled and scrolled, which is the foundation that camera will build on.

Review exercises

1. Explain why this chapter declares two resources (TILESET and MAP) from the same image file, instead of using a single IMAGE resource as in Chapter 7.
2. What does the ALL optimization parameter do when declaring a TILESET resource, and why is it the most convenient choice for a map like the one in this chapter?
3. A teammate tries to write `PAL_setPalette(PAL0, our_level_map.palette->data, DMA);`, copying the pattern used with an IMAGE in an earlier chapter, and the code doesn't compile because that field doesn't exist. Explain why this happens and what alternative should be used instead.
4. When compiling for the first time, the screen shows up entirely sky-blue instead of the expected landscape, with no error message. Explain why this happens and which function fixes the problem.
5. Describe, in general terms, what would need to change in this chapter's code so that, instead of scrolling automatically, the map would follow the position of a playable character controlled with the D-pad.

Chapter 12 — Level Maps with Tiled and TMX Files

Designing levels with an external graphical tool instead of drawing the map by hand, and bringing that design into the SGDK project

The previous chapter showed how to declare the `TILESET`, `PALETTE`, and `MAP` resources separately, load them with `VDP_loadTileSet`, and use them to build a `Map` structure via `MAP_create`, capable of scrolling with `MAP_scrollTo`. That workflow assumes the map already exists in some format `rescomp` knows how to read, but it says nothing about how that map is designed in the first place. Drawing an entire level tile by tile, by hand, on top of an image or directly as a list of numbers, is workable for a small screen, but it quickly becomes impractical once the level grows to several screens wide, has several overlapping layers (background, platforms, decoration), or the design needs to be iterated on many times during development.

This is exactly what dedicated map editors are for. This chapter covers the workflow with **Tiled**, a general-purpose external editor for tile-based 2D games, and its export format, **TMX**. The next chapter presents an alternative built specifically for SGDK, the **16Tile** tool, in more depth; the goal here is to understand the Tiled workflow well enough to use it comfortably, and to be able to compare it critically once you’ve read chapter 13.

Prerequisites

- Have SGDK 2.11 installed and a working base project, as explained in chapter 3.
- Know the difference between the `TILESET`, `PALETTE`, and `MAP` resources, and the use of `MAP_create`/`MAP_scrollTo` covered in chapter 11.
- A tileset of your own in image format (PNG), respecting the same palette restrictions already covered in previous chapters: at most 16 colors per palette and 8×8-pixel tiles, the smallest unit the VDP can handle.

1. What a tileset is and why an external tool helps

As mentioned in chapter 11, generating a large background by drawing it entirely as a single image can eat up an unreasonable amount of ROM. The alternative is a **tileset**: a small set of 8×8-pixel graphic pieces that are combined and repeated to build maps much larger than the sum of their parts. A handful of “grass,” “dirt,” and “rock” tiles is enough to build an entire level, as long as there’s also a **map**: the list of which tile goes in each position of the grid.

Drawing that list by hand — a matrix of numbers, each one the index of a tile within the tileset — is tedious and error-prone even for a modest level. A map-editing tool solves exactly that problem: it presents the tileset as a palette of brushes and lets you “paint” the map visually, tile by tile, instead of writing numeric indices by hand.

2. Tiled (mapeditor.org)

Tiled is an open-source, cross-platform (Windows, Linux, and macOS) tile map editor released under the GPL license, available at mapeditor.org. It's not a Mega Drive- or SGDK-specific tool: it's a generic editor used in 2D game development in general, which brings a concrete advantage — abundant documentation and a community of its own, independent of this manual — and an equally concrete limitation: it knows nothing about the Mega Drive's particular hardware restrictions (16-color palettes, a maximum of four palettes, 8×8 tiles), so those restrictions remain the responsibility of whoever designs the source tileset, before bringing it into Tiled.

The typical workflow within Tiled is as follows:

1. Create a new map, specifying the grid size in tiles and the size of each tile in pixels (8×8, so that each cell of the map corresponds to exactly one VDP tile).
2. Import, as a *tileset*, the image already prepared with the level's graphic pieces.
3. Create one or more tile-map-type **layers**, and paint on each one using the tileset as a palette of brushes. It's common to use one layer for the walkable background and a separate one for elements that belong on a different plane (for example, decoration that appears in front of the sprites), since on the Mega Drive each Tiled layer can later be mapped to a different hardware plane (BG_A or BG_B).
4. Save the finished map, exporting it in **TMX** format (an XML format native to Tiled) or, alternatively, in **JSON**. SGDK, through *rescomp*, works with the TMX format.

Tiled's interface while editing a map typically shows the layers panel on the right — with the list of layers created so far and controls to show or hide them — and, at the bottom, the tile palette of the imported tileset, ready for picking the tile to paint onto the map grid.

When saving the tile layer, it's worth checking that the layer's encoding format is **CSV** (a list of comma-separated numbers) rather than a compressed binary encoding. With layers in CSV, the resulting TMX file is readable plain text, which also acts as a safety net: if something goes wrong during automatic import, it's always possible to open the TMX with a text editor and check at a glance what numbers were saved.

3. Importing the TMX into the project with *rescomp*

Since SGDK version 1.80, *rescomp* — the resource compiler that already converts images and palettes in previous chapters — can read a TMX file exported by Tiled directly, with no manual intermediate step. The corresponding resource in the `resources.res` file generally has the form:

```
TILEMAP resource_name "level1.tmx" [options]
```

This resource generates, in `resources.h/resources.c`, a structure of type `TileMap` (different from the `Map` structure that `MAP_create` builds) which essentially contains the width and height of the map in tiles and the list of tile indices from one of its layers, exactly as they were saved in the TMX's `<data>` tag. That structure can be used in two ways, depending on what the game needs:

- **Combined with what was already covered in chapter 11:** by also separately declaring the `TILESET` and `PALETTE` resources corresponding to the same tileset used in Tiled, and using `MAP_create` to build the final `Map` structure that is then scrolled with `MAP_scrollTo`. This is the most direct path when the goal is a large level with scrolling — exactly the same scenario that motivated the previous chapter.

- **More directly**, when the map is small or static and the scrolling machinery of the MAP unit isn't needed: by reading the index list from the TileMap structure and dumping it, tile by tile, onto a plane already set up with the corresponding tileset loaded into VRAM.

The .tmx file itself doesn't need to be copied by hand anywhere else: it's enough to leave it inside the project's resources folder (alongside the images and other files already referenced by resources.res) and declare the TILEMAP resource line pointing to that file, the same way it's done with IMAGE, TILESET, or PALETTE.

The manual technique, prior to direct import

Before rescomp could read a TMX directly, the only way to bring a map designed in Tiled into an SGDK project was a more artisanal technique. Today it's unnecessary in the normal case, but it's worth knowing: it helps explain what the TILEMAP resource does behind the scenes, and it's useful for specific debugging cases.

1. Save the map in Tiled with the tile layer encoded as **CSV**, and confirm that each tile is 8×8 pixels.
2. Open the resulting .tmx file with a plain text editor. Being XML, inside the <data> tag of each layer appears the full list of the map's tile indices, comma-separated, in the same order the grid is traversed (row by row).
3. Copy that list of numbers and paste it into an array in a custom header file (for example, map.h, inside the project's inc folder).
4. Include that map.h from the source code, declare the tileset as a resource the usual way (TILESET and PALETTE in the .res file), load it with VDP_loadTileSet and PAL_setPalette, and finally traverse the screen drawing each tile from the copied array, one by one, using the VDP's low-level functions.

This is, in essence, the same result the TILEMAP resource achieves today in a single step, but done by hand and without the validation or compression that rescomp applies. Understanding this manual path helps clarify what the <data> tag of a TMX actually contains, and what the automatic import is doing behind the scenes.

4. A complete example workflow

A level designed in Tiled and brought into an SGDK 2.11 project follows these steps, end to end:

1. Prepare the tileset as a PNG image, respecting the limit of 16 colors per palette.
2. Design the map in Tiled: create the grid with 8×8-pixel tiles, import the tileset, paint the level's layer (or layers), and save the layer in CSV format.
3. Export the file as .tmx and copy it into the project's resources folder.
4. Declare the resource in resources.res:


```
TILESET tileset_level1 "tileset.png" NONE
PALETTE palette_level1 "tileset.png"
TILEMAP map_level1 "level1.tmx"
```
5. In the code, load the tileset and palette with the functions already covered in chapter 11 (VDP_loadTileSet, PAL_setPalette), and use the data from map_level1 to build the Map structure with MAP_create, or to draw the map directly if the level doesn't need scrolling.
6. If the level uses scrolling, scroll it with MAP_scrollTo exactly as with a map declared without going through Tiled.

The important point is that, regardless of the map's origin — drawn tile by tile in a custom array, as in chapter 11, or visually designed in Tiled and brought in as TMX — the rest of the loading and scrolling code doesn't change: `MAP_create` and `MAP_scrollTo` work the same regardless of where the map data comes from. Tiled solves exclusively the problem of **designing** the level, not the problem of how the Mega Drive ultimately displays it on screen.

5. Tiled versus 16Tile

Chapter 13 introduces **16Tile**, a tool built specifically for the SGDK workflow, with more direct integration with the Mega Drive's own restrictions (palettes, unique-tile limits, export tailored to this particular hardware). Without getting into how it works just yet, it's worth outlining how the two compare as alternatives:

- **Tiled** is a generic tool, far more widely used and documented outside the Mega Drive scene, with active support and features useful for any tile-based game (multiple layers, objects, custom properties). In exchange, it knows nothing about this hardware's particulars: you have to manually account for palette and unique-tile-count restrictions when preparing the source tileset.
- **16Tile**, being built specifically for SGDK, can tailor its export to those restrictions more directly, at the cost of being a niche tool, with a community and documentation much smaller than Tiled's.

Neither one fully replaces the other: they're two valid paths to solving the same problem, and the choice depends largely on how comfortable each person is with a general-purpose interface versus one built to measure. Chapter 13 returns to this topic with the detail needed to decide with better judgment.

Review exercises

1. What specific problem does a tool like Tiled solve that drawing the map as a single large image doesn't?
2. Why is it worth saving the tile layer in CSV format when exporting a map from Tiled for an SGDK project, instead of leaving the default binary encoding?
3. Explain in your own words the difference between the `TileMap` structure generated by the `TILEMAP` resource and the `Map` structure generated by `MAP_create`.
4. What manual steps does the `TILEMAP` resource in `rescomp` replace, in practice, starting from SGDK 1.80?
5. If two layers of the same Tiled map represent, one, the walkable terrain, and the other, decoration that should appear in front of the characters, how would that separation translate when bringing the map to the Mega Drive?

Chapter 13 — Creating Maps with 16Tile

A tool built specifically for SGDK: VDP palette and priority assigned tile by tile, rather than uniformly across the whole background

The previous chapter showed how to design a level map in Tiled, a generic tile map editor, and bring it into an SGDK project through the TILEMAP resource from rescomp. Before that, we had already seen the even more basic possibility of building a background with a conventional image editor—a plain pixel canvas, with no knowledge whatsoever of the console’s hardware—and importing the result as an IMAGE or MAP resource. Both approaches work, and both share the same limitation: the entire image ends up tied to a single color palette and a single VDP priority value for the whole background, with no per-tile distinction.

This chapter introduces **16Tile**, a free external tool created specifically for Mega Drive development with SGDK, distributed by its author under the username **Allone-Works** on itch.io. Unlike Tiled, 16Tile is not a general-purpose map editor: it comes with built-in knowledge of the VDP’s constraints and capabilities, and it lets you assign palette and priority independently to each individual tile of a map, while also exporting the files SGDK needs directly. Together with Tiled from chapter 12, it is one of the two usual alternatives for solving the same level-design problem: one generic and well documented, the other purpose-built for this particular hardware.

Prerequisites

Before continuing, it helps to have:

- Completed chapter 11 (“Large level maps with MAP_create”), in particular the TILESET and MAP resources, and the MAP_create() function.
- A clear understanding of the concept of a palette for a Mega Drive graphics resource (PAL0–PAL3, 16 colors each, with the first color of any palette other than PAL0 always transparent).
- An image editor capable of working with indexed palettes—such as Aseprite, covered in chapter 8—to prepare the source tileset before importing it into 16Tile.

1. The Mega Drive palette: why not just any RGB value will do

The Mega Drive palette works in 9-bit RGB: 3 bits for each of the three channels (red, green, and blue). With 3 bits it’s possible to represent 8 distinct combinations of zeros and ones, so each channel supports 8 possible tones, and combining all three channels ($8 \times 8 \times 8$) yields

a total of 512 distinct colors available. For comparison, other machines from the same era—the 32X and the Super Nintendo—use 15-bit RGB: 5 bits per channel, 32 possible tones per channel, and $32 \times 32 \times 32 = 32,768$ colors in total, sixty-four times more than the Mega Drive.

The practical problem is that a conventional image editor doesn't work with 8 tones per channel, but with 256 (0 to 255, the standard 8 bits per channel). For a color defined in an image editor to translate faithfully into one of the 8 actual tones the Mega Drive can reproduce, its red, green, and blue values must fall exactly on the 8 values the hardware supports between 0 and 255; any intermediate value gets rounded to the nearest tone when displayed on real hardware or in an accurate emulator, which can subtly alter the colors of a tileset that wasn't prepared with those exact values in mind.

2. A practical reference for verifying valid colors

The question that remains open is exactly which 8 values per channel, between 0 and 255, faithfully reproduce the Mega Drive's hardware. More than one reference table circulates in the homebrew development community for this console for that purpose: some advance in approximate steps of 36 (0, 36, 72, 108, 144, 180, 216, 255), others follow a more irregular curve intended to approximate real measurements of the VDP's video output (0, 52, 87, 116, 144, 172, 206, 255), and others use regular steps of 32 (0, 32, 64, 96, 128, 160, 192, 224, 255, with a small adjustment to the last value). There is no publicly available official documentation from Sega confirming which of these tables most accurately reflects the hardware's real behavior; it's best to treat any of them as a practical community convention, useful for checking at a glance whether a color falls within valid tones, rather than as an officially verified fact.

For a background meant to be displayed through a generic IMAGE resource, any of these references works reasonably well. When working specifically with 16Tile, however, it's best to adopt the steps-of-32 table, since that's the one the tool itself expects and with which it produces the most consistent results: the rest of this chapter assumes that convention.

3. Getting 16Tile

16Tile is a free tool, distributed on itch.io under the username **Allone-Works**, that requires no installation: just download the compressed file, unzip it into any folder, and run the .exe directly. It's best to always use the most recent version available on the project's page.

The itch.io page itself confirms that the tool was designed specifically to work alongside SGDK, and it includes a sample project with the source code needed to import its resources.

The 16Tile interface just after opening, with the Layer and Tileset panels on the right and the main canvas still empty.

4. Preparing the source tileset

Before opening 16Tile you need a tileset already prepared in a conventional image editor. Two rules matter when preparing it:

1. The image's height must always be a multiple of 8, the base size of a Mega Drive tile.
2. It's a good idea to reserve the first 16×16 tile as blank, using the background color as its only color, and the second 16×16 tile for drawing the full palette in order, one pixel per color. This ensures that, when the image is imported into 16Tile, the order of the

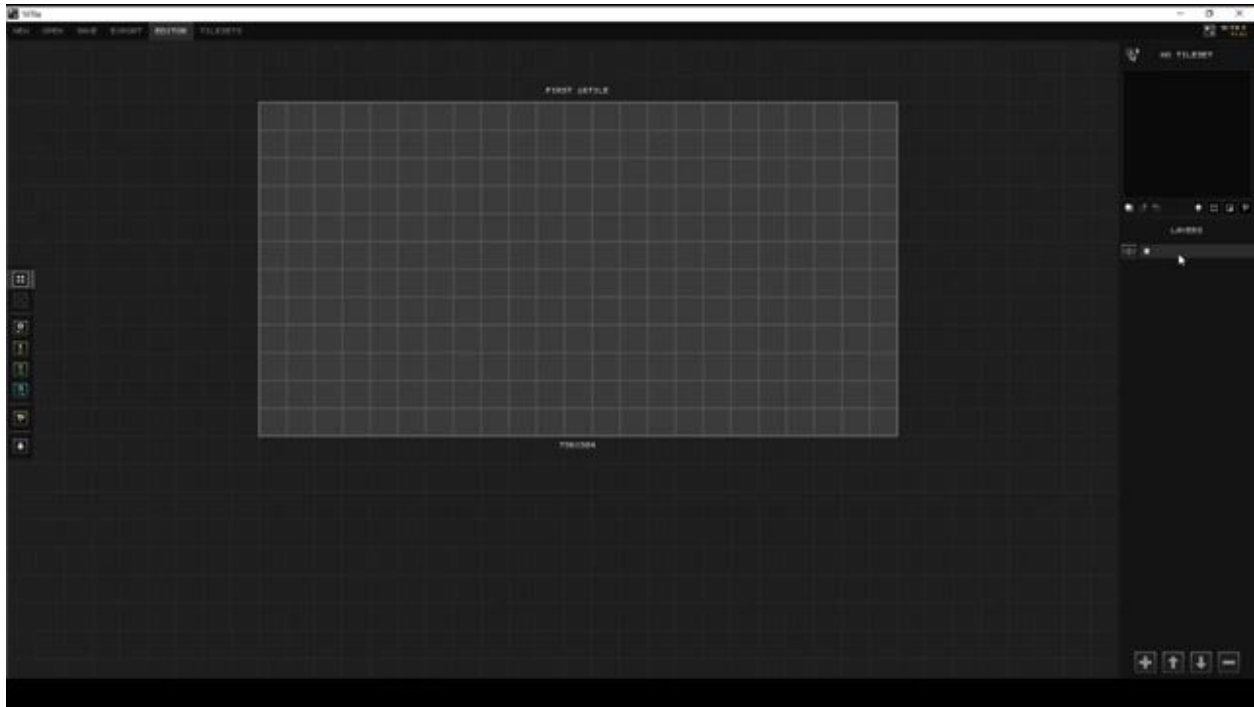


Figure 14: 16Tile interface freshly opened

palette indices is preserved exactly as defined, without the tool reordering them on its own.

5. First steps in 16Tile: project, layer, and tileset

The first step within 16Tile is to save the project (*Save*, at the top), giving it any name. Then, in the right-hand panel, the *Layer* section lets you name the background layer you're about to build. Once the layer has been created, you need to assign it a tileset: in the *Tileset* section of the same panel, the **+** button creates a new tileset, and the arrow icon lets you load the image prepared in the previous step.

If the source tileset comes from graphics extracted with an emulator—a common source for experimenting—it's common for the colors to look wrong the first time they're loaded into 16Tile: the emulator may export the RGB values on its own scale, which doesn't necessarily match the steps-of-32 scale. The fix is to go back to the image editor, manually adjust the RGB values of each color so they fall on those steps, save the corrected image, delete the previous tileset within 16Tile, and load the corrected version instead.

6. Painting the map: tile size, pasting, and flipping

With the tileset already loaded correctly, the right-hand panel lets you choose the tile size to paste onto the main map: 8×8 , 16×16 , or 32×32 pixels, depending on the level of detail needed for that particular tileset. Painting the map consists of left-clicking a tile in the tileset panel to select it, and then left-clicking on the main canvas to paste it; holding the button down while dragging the cursor lets you paste several tiles in a row in a single motion, without repeating the process tile by tile.

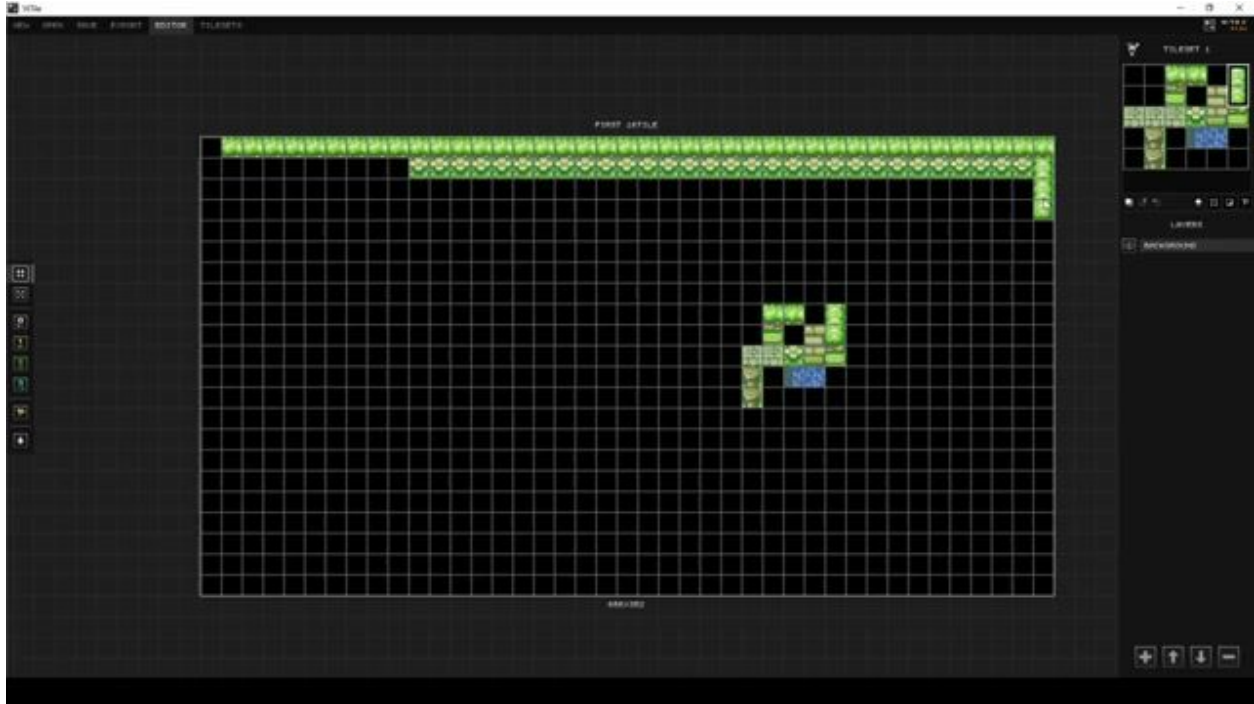


Figure 15: First tiles pasted onto the canvas

Several tiles already pasted onto the main canvas, forming the first elements of the map.

Two keyboard shortcuts let you flip the selected tile before pasting it: the **X** key flips it horizontally, and the **Z** key flips it vertically; an indicator in the right-hand panel confirms whether flipping is active.

16Tile has no undo function whatsoever: there is no shortcut equivalent to Ctrl+Z to revert a mistake. It's best to save the project frequently and pay close attention while painting, since any error has to be fixed by hand rather than undone with a single command.

7. Exporting the finished map

Once the map is finished, the last step within the tool is to save the project and use the **Export** button at the top. This generates three separate files —the palette, the tileset, and the tile map itself— each as a separate PNG image, ready to be imported into SGDK.

8. Importing the exported resources into SGDK

With the three files already exported, the tileset and the map are copied directly into the `res` folder of the SGDK project. There's no need to use the palette exported separately: it can be extracted from the map file itself, since the latter already contains all the necessary color information. In `resources.res` you need to declare the `TILESET`, pointing to the tileset file you just copied, and the `MAP`, pointing to the map file; the `PALETTE` line, instead of pointing to the separately exported palette file, also points to the map file:

```
TILESET tileset_village "background.png" NONE
PALETTE palette_village "background.png"
```



```
MAP_village_map "background.png" tileset_village NONE
```

9. Updating the constants in main.c

With the resources already in place, a few adjustments are needed in `main.c`. First, `MAP_WIDTH` and `MAP_HEIGHT` must be updated to reflect the actual width and height of the new map, values that can be checked directly in 16Tile before exporting. Second, if the base project had its horizontal resolution set to 256 pixels (H32 mode, covered in chapter 10), it's worth reviewing whether to switch back to 320 pixels (H40 mode, the usual horizontal resolution), both in the `HORIZONTAL_RESOLUTION` macro and in any related call involving screen width.

This same type of project —a large scrolling background— is also the foundation on which, later in this manual (chapter 24), a 2D camera is added; nothing shown here depends on that, but it's worth keeping in mind if you reuse the same base code later on.

10. The key change: replacing `TILE_ATTR_FULL` with the tileset index

The most important change in this chapter happens in the call to `MAP_create()`. Up through chapter 11, the last parameter of that function was always a call to `TILE_ATTR_FULL(palette, priority, vertical_flip, horizontal_flip, index)`, manually specifying a single set of attributes to be applied uniformly across the whole map:

```
MAP_create(&village_map, BG_B, TILE_ATTR_FULL(PAL0, FALSE, FALSE, FALSE, ind));
```

A map exported from 16Tile, by contrast, already includes the palette, priority, and flip information for each tile embedded directly within the map's own PNG file, in a format that rescomp recognizes automatically at compile time. This lets you replace the entire `TILE_ATTR_FULL(...)` call with simply the tileset's index variable (`ind` in this example), letting the resource itself —rather than a single function call— determine each tile's attributes individually:

```
MAP_create(&village_map, BG_B, ind);
```

When you compile and run the result in an emulator, the map created in 16Tile appears on screen as expected, with palette and priority already resolved tile by tile straight from the resource itself.

11. Why use 16Tile instead of a generic image editor

For a simple map, building it directly with a generic image editor —as in previous chapters— may even be faster. The reason to prefer 16Tile isn't speed in a simple case, but three capabilities that a generic image editor doesn't offer, or offers only with much more difficulty:

1. **Independent palette per tile.** Instead of forcing a single palette for the whole background, each tile can use any of the active palettes. Since the Mega Drive can have up to 4 palettes active simultaneously (with a combined total of 61 distinct colors visible at once, discounting the transparent color of each palette beyond `PAL0`), this allows for backgrounds far richer in color than what a single 16-color palette permits. Chapter 14 picks this technique back up in detail.
2. **Independent VDP priority per tile.** This allows for depth effects where the character can pass in front of some parts of the landscape and behind others, a subject developed in chapter 15.

3. **Generating a collision array** from the painted map itself, instead of having to define it separately by hand, a technique revisited when covering level collision in chapter 40.

Review exercises

1. Explain the difference between the Mega Drive's 9-bit RGB and the 15-bit RGB of the 32X or the Super Nintendo, and calculate how many total colors each supports.
2. Why is it necessary for the RGB values of a source tileset to match the tones per channel that the Mega Drive can reproduce, rather than using any value between 0 and 255?
3. Explain what it's for to reserve the first tile of a tileset as blank and the second one with the full palette drawn in order, before importing it into 16Tile.
4. What exactly changes in the call to `MAP_create()` when using a map exported from 16Tile, compared to a map created by hand or imported from Tiled, and why is that change possible?
5. Compare 16Tile with Tiled (chapter 12) in terms of the three advanced capabilities mentioned in section 11: which of them are more straightforward in 16Tile, and why?

Chapter 14 — Multiple Palettes on the Same Background with 16Tile

Overcoming the one-palette-per-background-layer limit, tile by tile

Chapter 13 introduced 16Tile as an alternative to a generic image editor for building Mega Drive backgrounds, and mentioned in passing one of its advantages: the ability to assign the palette independently for each tile of the map. This chapter develops that idea in detail: how to prepare a tileset that combines graphics intended for more than one palette, how to import it into 16Tile together with the additional palette it needs, and how to assign each palette to the corresponding tiles while painting the map.

Prerequisites

- Having completed Chapter 13 (“Creating maps with 16Tile”), in particular the tool’s full workflow and the use of a simple tile index in `MAP_create()` instead of `TILE_ATTR_FULL(...)`.
- Remembering, from Chapter 2, that the Mega Drive supports 4 simultaneous hardware palettes (PAL0–PAL3): PAL0 has 16 usable colors, and the other three have 15 each, since the first color of any palette other than PAL0 is always transparent. This gives a combined total of 61 colors visible at once.
- Knowing `PAL_setPalette()`, covered in Chapter 7, for loading a palette into a given hardware register.
- Having an image editor capable of working with indexed palettes, to prepare and combine the source tilesets.

1. The problem: a background with more colors than fit in a single palette

The example for this chapter is the village background from **Beyond Oasis**, a 1994 Mega Drive game, which uses a total of 29 distinct colors. Since any individual Mega Drive palette supports at most 16 colors (or 15 usable ones if it isn’t PAL0), a 29-color background cannot be displayed correctly with a single palette: its graphics need to be split across two different palettes, deciding which background elements use each one.

The original Beyond Oasis village background, with a total of 29 distinct colors.

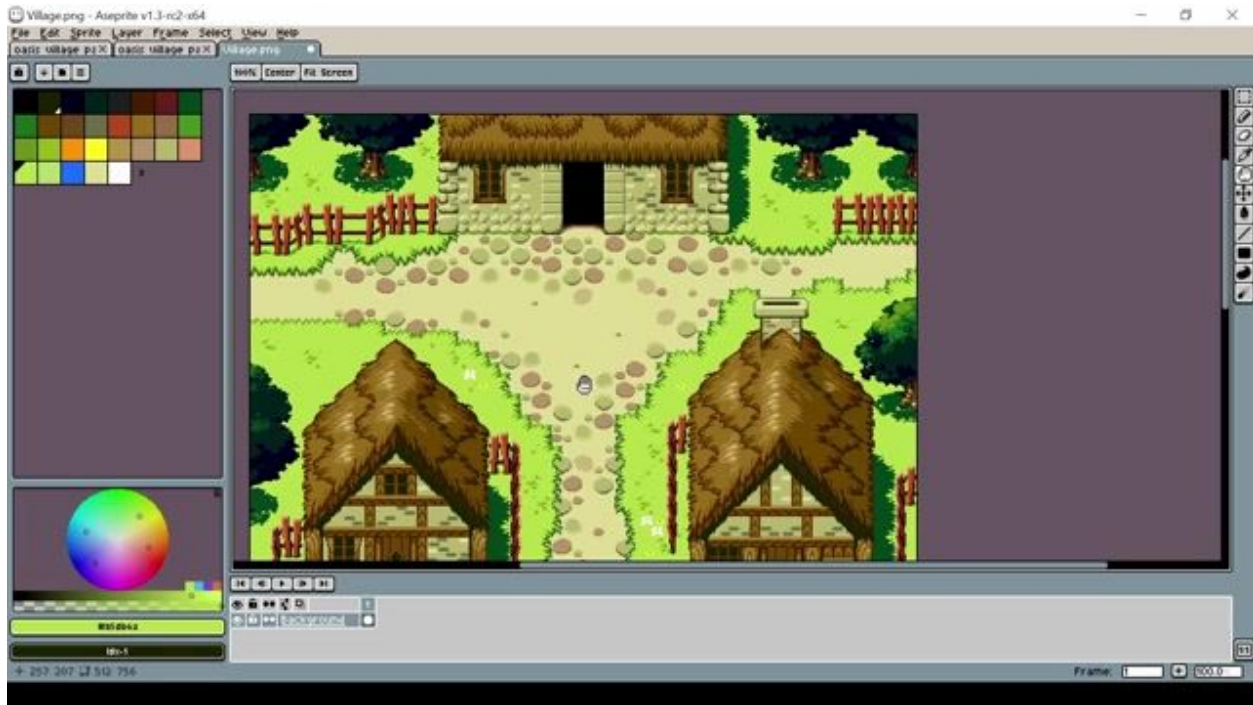


Figure 16: The original Beyond Oasis village background, with 29 distinct colors

2. Preparing two independent tilesets, each with its own palette

The first step is to create two separate tilesets, each limited to the colors of a single palette: one for the elements that will use PAL0 and another for the ones that will use PAL1. For this example there's no need to split every single element of the original background between the two palettes with pixel-perfect precision; it's enough to put together a representative selection for each one. It's also not essential to convert the resulting palette to the 32-step palette recommended for 16Tile: in this particular case, almost all the colors (except for a couple of flowers) carry over just as well without that conversion, though it's worth keeping in mind as a general best practice.

The rules already familiar from Chapter 13 still apply unchanged: height and width that are multiples of 8, indexed color mode, file saved as PNG, and the second 16×16 tile of each tileset with its corresponding palette drawn in order, so that 16Tile can read it on import.

Adjusting the canvas size to make room to combine both tilesets into a single file.

3. Combining both tilesets into a single file, taking care with copy order

16Tile imports a single tileset per layer, so the two tilesets prepared separately need to be combined into one file before continuing. The simplest approach — copying the PAL1 tileset's elements directly onto the already-open PAL0 tileset — produces an unexpected result: the colors of those elements change to something other than expected, because the image editor reinterprets those pixels using the indices of the currently active palette (PAL0's) instead of preserving their original colors.

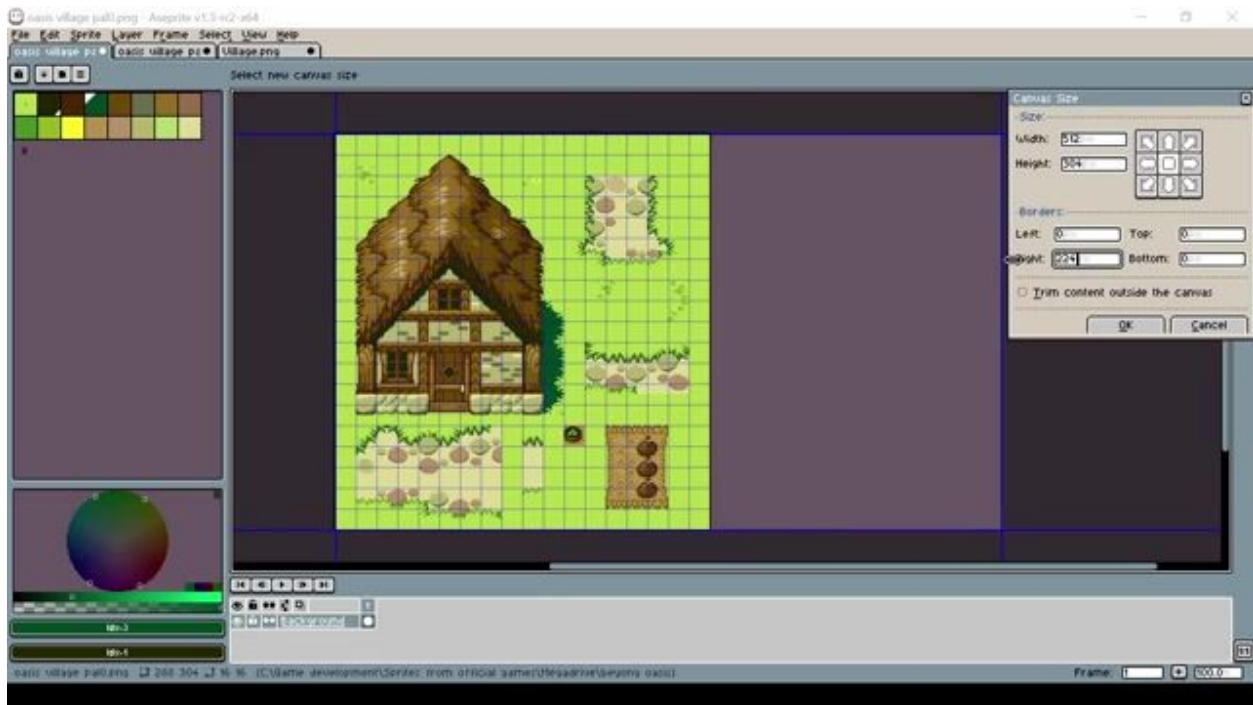


Figure 17: Adjusting the canvas size to combine both tilesets

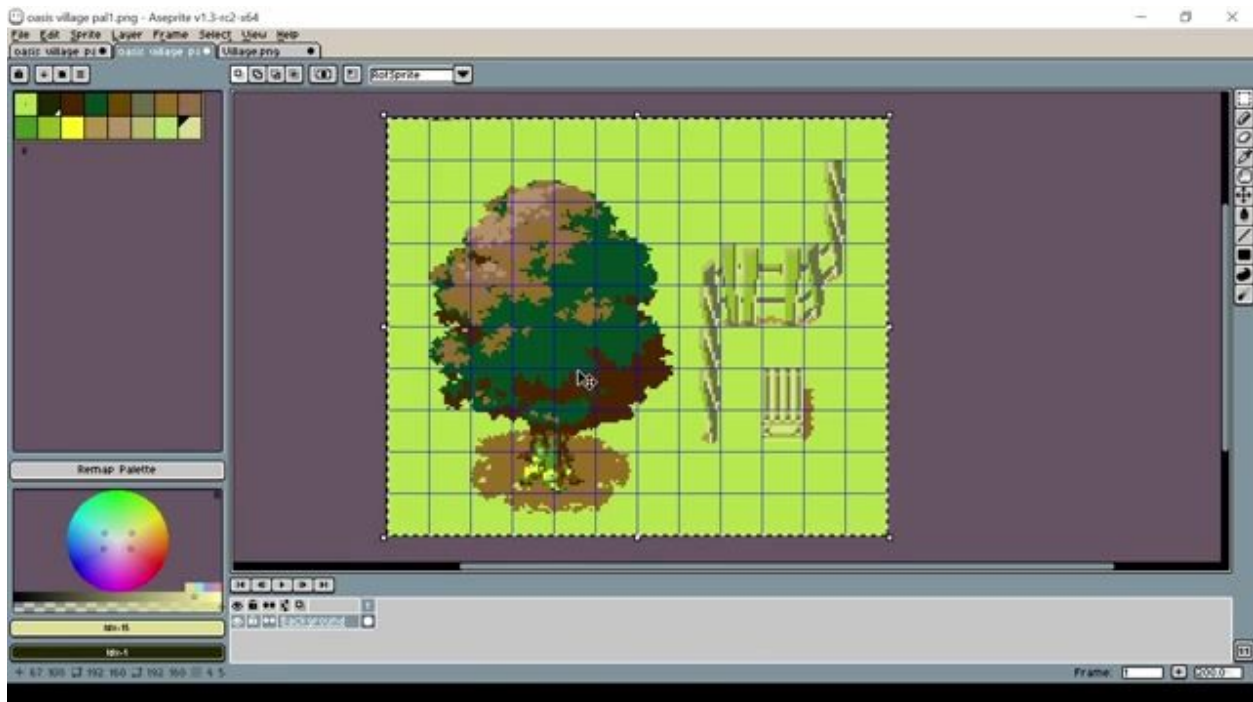


Figure 18: PAL1 elements with incorrect colors when pasted onto PAL0

The elements copied from the PAL1 tileset (the tree and the fence) take on incorrect colors when pasted directly onto the PAL0 tileset.

The solution is to reverse the order of the operation:

1. Paste the PAL0 palette into the PAL1 tileset itself, so that both momentarily share the same active palette.
2. Only then copy the elements, now correctly converted, into the final tileset.

When combining tilesets intended for different palettes into a single file, it's always best to first match the active palette of both and only then copy the graphics. Copying the graphics before matching the palette is the most common cause of an element changing color “for no apparent reason” when combined with another tileset.

Doing it in this order, the color indices are preserved correctly and the elements keep their original appearance when combined into the same file.

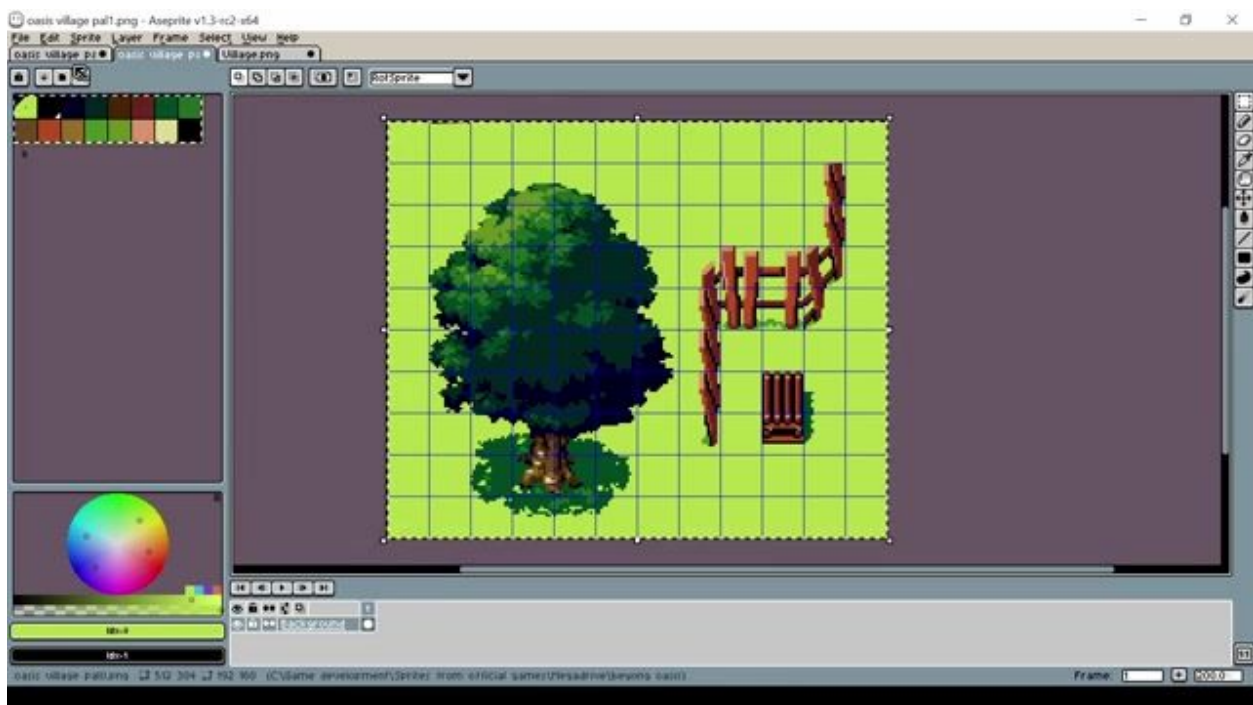


Figure 19: Elements combined correctly after matching the palette first

The same elements now combined correctly, after pasting the PAL0 palette into the PAL1 tileset before copying the graphics.

With the combined tileset already saved as PNG — also removing, along the way, any leftover from the palette reference row that’s no longer needed, since the combined tileset now uses a single shared palette row — the last preparation step is to export the PAL1 palette separately as a .pal file, since 16Tile needs to load it independently later on.

4. Importing the combined tileset and the additional palette into 16Tile

Inside 16Tile, the combined tileset is imported the same way already familiar from Chapter 13 (Tileset > + > arrow icon). The PAL0 palette is taken automatically from the colors of the imported tileset itself, but the PAL1 palette must be loaded separately: in the top-right corner, the icon corresponding to PAL1 lets you select the .pal file exported in the previous step.

Switching between PAL0 and PAL1 inside 16Tile should reproduce, for each half of the tileset, a result equivalent to what the original tilesets looked like in the image editor. If either half shows incorrect colors at this point, it's worth reviewing the copy order from the previous step before moving on.

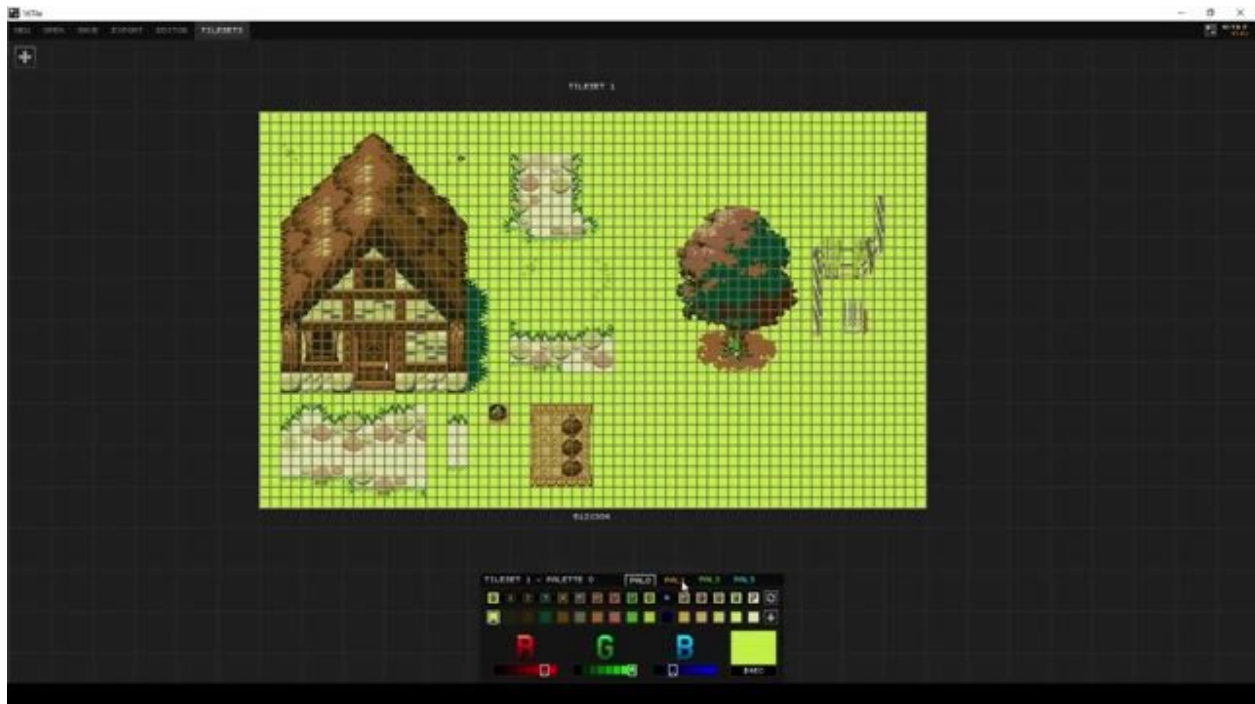


Figure 20: Combined tileset imported into 16Tile with the palette selector

The combined tileset already imported into 16Tile, with the PAL0/PAL1/PAL2/PAL3 selector visible at the bottom.

5. Painting the map with both palettes

With the tileset and both palettes already loaded, painting the map follows the same procedure already familiar from Chapter 13. It's best to first paste the elements intended for PAL0 (houses, path, grass) and then the ones intended for PAL1 (trees, fences, and a few additional decorative elements), without worrying yet about assigning them the correct palette: in 16Tile, every new tile is automatically assigned to PAL0 by default, regardless of which half of the combined tileset it came from.

The map with several houses already pasted using the PAL0 tileset.



Figure 21: Map with several houses pasted using the PAL0 tileset

6. Assigning the palette for each tile

The left-hand panel in 16Tile includes four buttons numbered 0 to 3, one for each Mega Drive palette. Since every new tile is already automatically assigned to PAL0, all that's needed is to manually select PAL1 (or PAL2/PAL3, in a background that uses them) and then paint over the tiles that should use that particular palette, the same way you'd paint normal tiles. A small numeric indicator appears over each tile already marked, making it possible to check at a glance which ones use each palette.

Tree tiles already marked with the "1" indicator, assigning them the PAL1 palette.

Transparency deserves attention when combining palettes this way: since the first color of any palette other than PAL0 is always transparent, it's worth deliberately choosing which color occupies that first slot in PAL1 (in this example, the same green as the grass, which already occupies the first slot in PAL0). That way, wherever a PAL1 tile shows transparency, what shows through underneath matches the expected background color instead of displaying an unexpected one.

As an additional demonstration — not as part of the intended final result for a real background — it's also possible to assign PAL1 to one of the houses, to visually check how much its appearance changes when using a palette intended for a different kind of element.

A house with the PAL1 palette applied for demonstration purposes, showing a visually different result from PAL0.

7. Exporting and integrating the result into SGDK

The export process is the same one already familiar from Chapter 13 (Save, then Export), again generating the tileset, the map, and the palette. The difference from the previous



Figure 22: Tree tiles marked with the “1” indicator for PAL1



Figure 23: House with PAL1 applied for demonstration purposes

chapter shows up when integrating these files into the SGDK project: since two palettes are now needed instead of one, `resources.res` needs two independent `PALETTE` lines, each pointing to its own palette file — unlike Chapter 13, where a single `PALETTE` line could be extracted directly from the map file:

```
TILESET village_tileset "village_tileset.png" NONE
PALETTE village_pal0    "village_pal0.pal"
PALETTE village_pal1    "village_pal1.pal"
MAP      village_map    "village_map.bin" village_tileset NONE
```

In `main.c`, correspondingly, two calls to `PAL_setPalette()` are needed: one for `PAL0` with the data from `village_pal0`, and another for `PAL1` with the data from `village_pal1`.

```
PAL_setPalette(PAL0, village_pal0.palette->data, DMA);
PAL_setPalette(PAL1, village_pal1.palette->data, DMA);
```

When writing the second call by copying and pasting the first, be careful to check that **two** values have been corrected, not just one: both the destination register (from `PAL0` to `PAL1`) and the source resource (from `village_pal0` to `village_pal1`). It's an easy mistake to make when editing a copied line, and the result — if you forget to change the source resource, for example — is that both calls end up loading the same palette into different registers, without the compiler flagging it as an error.

8. Compiling and verifying the result

With both palettes now loaded, the tileset and map updated, and the `MAP_WIDTH/MAP_HEIGHT` constants adjusted to the dimensions of the new map (different, again, from those in Chapter 13), compiling and running the result lets you directly compare, within the same emulator, a house with `PAL0` next to another with `PAL1` applied for demonstration purposes.

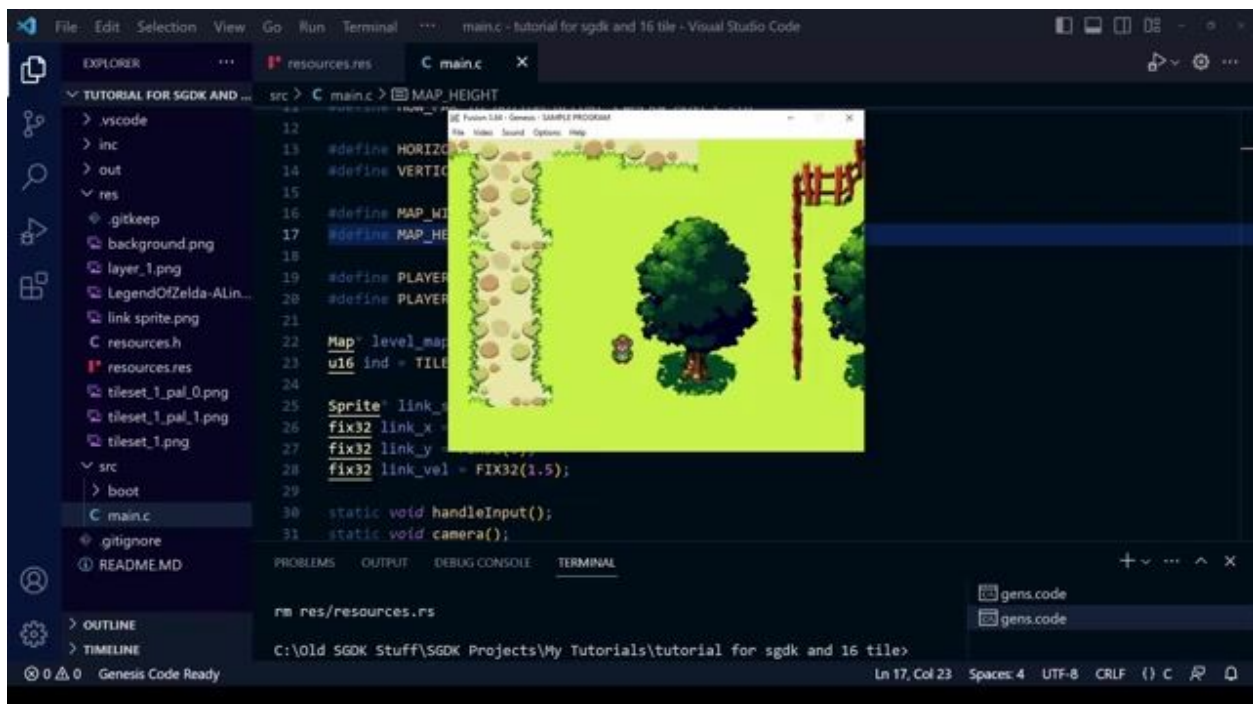


Figure 24: Compiled result: a house with `PAL0` next to another with `PAL1`

The compiled result, with a house with PAL0 (original colors) next to another with PAL1 applied for demonstration purposes (altered colors).

It's common, when comparing the compiled result against the map as it appears in the editor, to find some isolated discrepancy: for example, a tree tile that keeps PAL0's colors instead of the expected PAL1 ones. The most frequent cause of this kind of discrepancy is simply a tile that was left unmarked while painting the map in 16Tile: since this is a manual, tile-by-tile assignment, it's easy for one to get overlooked, especially in areas with many similar tiles next to each other (like a tree's foliage).

The method for diagnosing this kind of error is straightforward: go back to 16Tile and check the numeric palette indicators in the area of the map where the discrepancy appears, visually comparing that area against the compiled result in the emulator. Generally it's enough to locate the tile in question, confirm that its indicator shows "0" instead of "1," and paint over it again with the correct palette already selected. It's worth making this comparison between the editor and the compiled ROM a routine review step whenever working with more than one palette on the same background: a mis-assigned tile doesn't produce any compilation error, it simply looks wrong, and can only be caught by looking at the result.



Figure 25: Checking in 16Tile for a tree tile not assigned to PAL1

Checking in 16Tile, comparing the map against the emulator, to find which tree tile was left unassigned to PAL1.

9. How to distribute the 4 palettes across a full game

Beyond this specific two-palette example, it's worth thinking about how to distribute the 4 available palettes across a complete game. A common distribution — arising from practice rather than from any rule of the hardware or of SGDK — is to use PAL0 for the farthest background, a combination of PAL0 and PAL1 for the playable foreground, PAL2 for the main

character (along with sub-weapons or other elements that can share that same palette, and occasionally some enemy), and reserve PAL3 specifically for enemies.

This particular distribution isn't a fixed rule: it's just one practical convention among several possible ones. How to distribute palettes in any given project is ultimately up to each developer's judgment, depending on how many distinct elements need to be shown on screen at once and how much it makes sense to share a single palette among them.

Review exercises

1. Explain why a 29-color background cannot be displayed correctly using a single Mega Drive palette, and how many palettes at minimum would be needed to display it in full.
2. When combining two tilesets from different palettes into a single file, why does copying the graphics directly produce incorrect colors, and in what order must the operation be done to avoid that problem?
3. Where does 16Tile take the PAL0 palette from when importing a tileset, and where must any additional palette such as PAL1 be taken from?
4. Explain why it's a good idea to choose the same color for the first slot of PAL0 and of PAL1 when building a background with more than one palette.
5. A tile in a background with two palettes shows up with the wrong colors in the emulator, but the project compiles without any errors. What is the most likely cause, and how is it diagnosed within 16Tile?

Chapter 15 — Depth via Tile Priority

Chapters 13 and 14 showed how to use 16Tile to build a level map and how to split its tiles across several palettes. This chapter closes out the trilogy by covering the other major benefit of assigning attributes tile by tile: VDP priority applied selectively within a single plane, rather than to an entire plane as in the example from Chapter 7. With this technique, a background can be shown in front of the sprites in some areas and behind them in others within the same layer, giving the impression that a character is walking in the middle of the scenery rather than simply in front of it.

A classic example of this effect is the scene in *World of Illusion Starring Mickey Mouse* where Mickey and Donald appear now in front of, now behind, the tree trunks of a forest. That particular scene combines tile priority with tile scrolling, a technique revisited later in this manual (Chapter 17). The practical example in this chapter uses only priority, without that additional scrolling, but the result is already convincing on its own.



Figure 26: The reference scene in *World of Illusion Starring Mickey Mouse*

Mickey and Donald appear in front of some forest elements and behind others.

Prerequisites

Before continuing, you should have:

- Practiced the 16Tile workflow from Chapter 13 (creating a tileset and a map) and Chapter 14 (assigning more than one palette tile by tile).
- A clear grasp of the VDP priority concept introduced in Chapters 2 and 7: by default, the drawing order goes from BG_B (furthest back) to BG_A (in the middle) to the sprites (frontmost), and the priority bit of `TILE_ATTR_FULL()` allows that order to be reversed.
- A tileset and a map already prepared in 16Tile, following the same workflow as the previous two chapters.

1. From plane priority to tile priority

In Chapter 7, priority was applied to an entire plane: changing `prio` to `TRUE` in `TILE_ATTR_FULL()` made all of BG_B draw in front of BG_A. That granularity is enough for certain effects, but not for something as specific as “the character passes behind this trunk but in front of that bush,” because the trunk and the bush live on the same plane.

The solution is to assign the priority bit tile by tile, using exactly the same mechanism Chapter 14 used to assign palettes tile by tile: instead of setting a single `TILE_ATTR_FULL()` value for the whole map inside `MAP_create()`, 16Tile stores, for each individual tile, whether that particular tile should be drawn with high priority or not, and exports that information together with the map.

2. The starting tileset and map

The example in this chapter starts from a simplified tileset for a *World of Illusion* level. Just as in Chapter 14, that tileset uses two palettes: one for most of the trees and the ground, and a separate one for the mushrooms. The process of combining both palettes into a single tileset and assigning them tile by tile was already explained in the previous chapter, so this chapter takes that as known and focuses directly on priority.

The tileset already combines the two palettes mentioned above.

The mushroom tiles are marked with the second palette’s indicator.

3. Assigning high priority with the P button

16Tile assigns priority the same way it assigns palette: you first pick an option in the left-hand panel, then paint it onto the tiles that should receive it. The **P** button activates priority-assignment mode; any tile painted while that mode is active gets marked as high priority, which is reflected by a brighter tint on those tiles within the editor itself.

In this example, the tree trunks in the foreground are the ones given high priority, since they are the elements the character should not be able to walk in front of.

The column of tree trunk tiles is selected.

Within the editor, those tiles appear with a brighter tint.

The concrete effect of this marking: normally all sprites are drawn in front of any background tile, regardless of that background’s priority. By marking a specific tile as high priority, that tile is drawn in front of the sprites instead of behind them, reversing the usual order only for those tiles. That selective reversal is what produces the depth effect: the character can walk

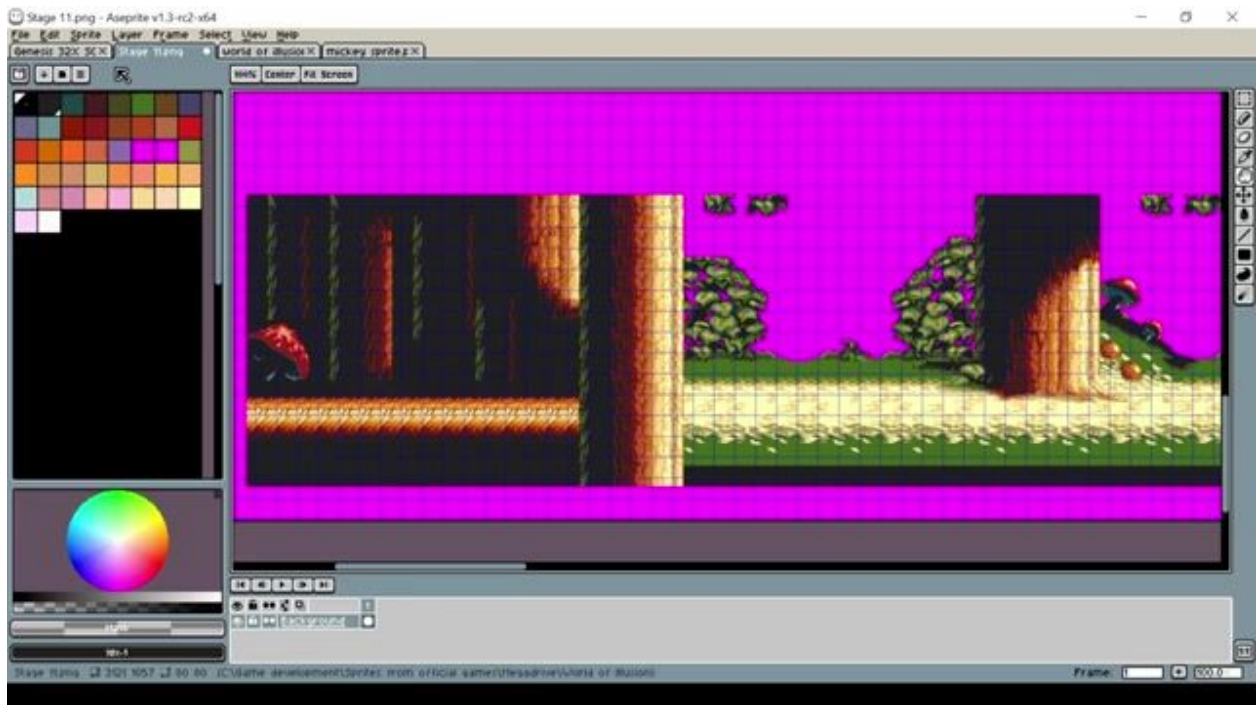


Figure 27: The simplified tileset for the World of Illusion level



Figure 28: The map assembled in 16Tile

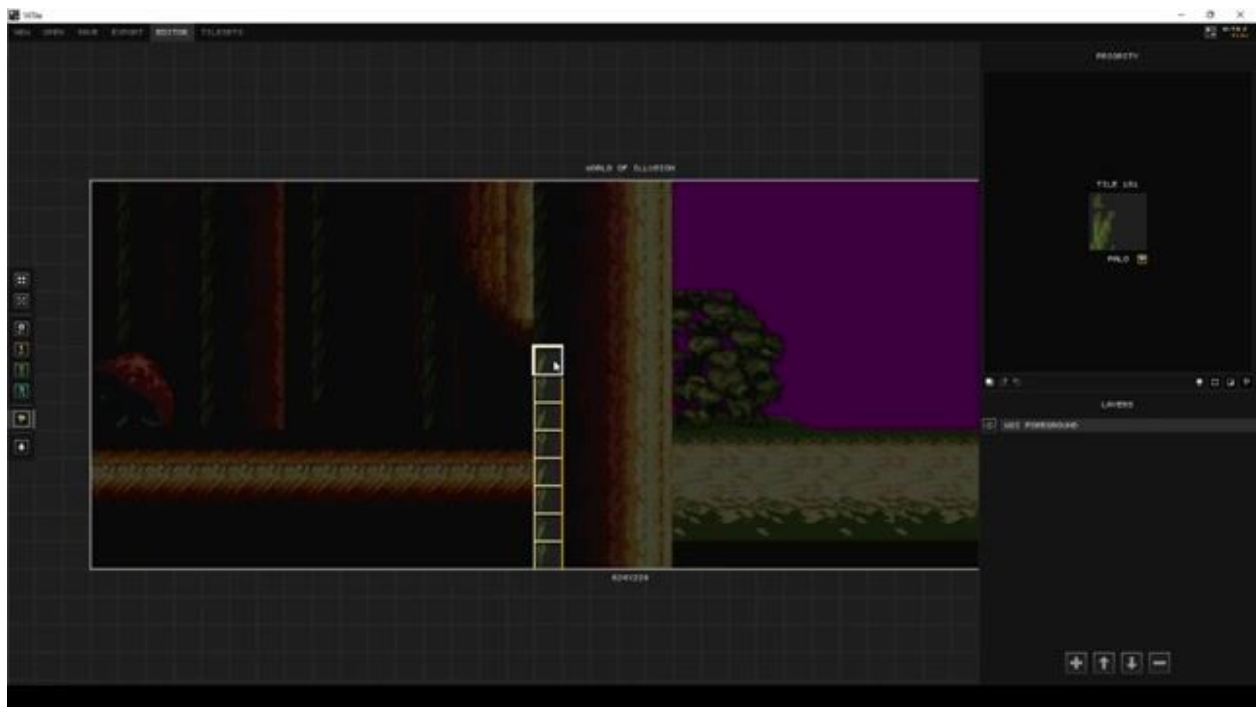


Figure 29: Priority mode activated in 16Tile

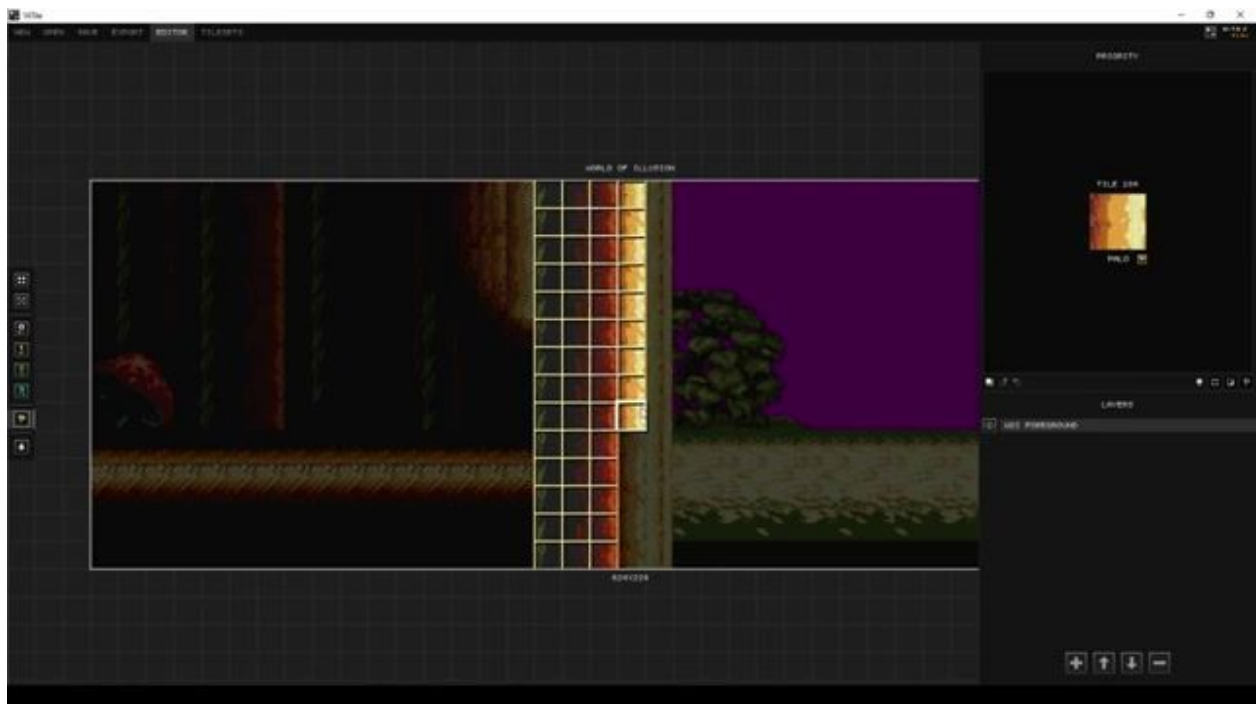


Figure 30: The trunk tiles marked with high priority

behind the trunk marked as high priority, while still appearing normally in front of any other tile in the same background that does not have that mark.

4. Exporting and integrating into SGDK

Exporting and integrating into the SGDK project follows the same procedure as Chapter 14: exporting from 16Tile again generates the palette, the tileset, and the map, ready to be copied into the res folder. Since this example reuses the project from the previous chapter, the tileset and palette remain unchanged, and only the map file needs to be replaced with the new one (woi_foreground.png in this example).

In resources.res, the MAP line must be updated to point to the new file:

```
MAP    woi_foreground_map    "woi_foreground.png"    woi_tileset    FAST
```

This chapter also replaces the playable character's sprite with one of Mickey Mouse, both to vary the example and because the new sprite is larger: 4 tiles wide by 6 tiles tall, instead of the smaller dimensions used so far. That is also reflected in the SPRITE line of resources.res:

```
SPRITE    mickey_sprite    "mickey.png"    4    6    FAST
```

5. Updating the constants in main.c

With Mickey's sprite being larger, PLAYER_WIDTH and PLAYER_HEIGHT must be updated to the sprite's actual dimensions in pixels: 32×48, corresponding to the 4×6 tiles declared in resources.res.

Likewise, MAP_WIDTH and MAP_HEIGHT — the constants that, as explained in Chapter 13, must reflect the actual width and height of the map exported from 16Tile, values the tool itself displays before exporting — need to be updated to the size of the new *World of Illusion* map and not left with the values from a previous map. This is an easy detail to overlook when only the map file is replaced without reviewing these constants, and a map loaded with the wrong dimensions produces distorted backgrounds or areas showing the default fill color (magenta) when scrolling beyond its actual bounds.

```
#define PLAYER_WIDTH    32
#define PLAYER_HEIGHT    48

// Actual width and height of the map exported from 16Tile, in tiles
// (the specific values are read from the tool itself before exporting)
#define MAP_WIDTH    64
#define MAP_HEIGHT    32
```

6. The priority parameter in SPR_addSprite()

The only code change needed for the character to respect tile priority happens in the call to SPR_addSprite(), in its TILE_ATTR(palette, priority, vflip, hflip) argument. The second parameter, the sprite's own priority, determines whether that sprite behaves in the usual way (low priority, FALSE) or takes precedence even over background tiles marked as high priority (high priority, TRUE).

```
SPR_addSprite(&mickeySprite, x, y,
    TILE_ATTR(PAL2, FALSE, FALSE, FALSE));
```

Leaving the sprite's priority at `FALSE`, the character passes behind any background tile that does have high priority, which is the effect sought in this example: Mickey ends up hidden behind the trunk marked in `16Tile`.

7. What happens if the sprite also has high priority

To see the other side of this relationship, it's enough to also change the sprite's priority to `TRUE`:

```
SPR_addSprite(&mickeySprite, x, y,  
             TILE_ATTR(PAL2, TRUE, FALSE, FALSE));
```

With both — the trunk tile and the character sprite — set to high priority, Mickey once again appears in front of the entire background, including the tiles that previously covered him. The general rule, then, is as follows:

A sprite only ends up hidden behind a background tile when the sprite has low priority **and**, in addition, that particular background tile has high priority. If the sprite has high priority, it is shown in front of any background tile regardless of that tile's priority.

Setting the sprite's priority back to `FALSE` restores the intended behavior: Mickey once again passes behind the trunk marked as high priority.

8. Possible improvements: a second background layer

The priority effect is already convincing on its own, but it can be intensified by adding a second background layer behind the foreground map already built: using `BG_A` for the map with tile priority (the front layer) and `BG_B` for a more distant background with simple parallax scrolling, using the technique already covered in Chapter 9. No tile scrolling is needed to achieve this improvement — that technique is revisited in Chapter 17 — and combining both resources (tile priority plus an additional parallax layer) is exactly what produces the multi-layered sense of depth in the original *World of Illusion* scene.

If `16Tile` or a similar tool is ever unavailable, the same per-tile priority can be assigned by hand in the code, without relying on an external script; that alternative is covered in Chapter 16.

Review exercises

1. Explain, in terms of the usual ordering between sprites and background, exactly what changes when a tile is marked as high priority in `16Tile`.
2. What is the difference between the full-plane priority used in Chapter 7 and the per-tile priority covered in this chapter?
3. Which parameter of `TILE_ATTR()`, used in `SPR_addSprite()`, determines the priority of the sprite itself, and what happens if that parameter is left at `TRUE` instead of `FALSE`?
4. If a sprite has high priority and a background tile also has high priority, which of the two is shown in front of the other?
5. Briefly describe how the tile priority from this chapter could be combined with a second background layer, to get closer to the visual effect of the reference scene from *World of Illusion*.

Chapter 16 — Manually Assigning Tile Priority Without a Script

This chapter presents a manual method for assigning tile priority to a background, as an alternative to the automated Lua script workflow that is picked up later, in Chapter 19. That automated method works well, but it has a cost: on a large map, with many tiles marked as priority, processing can take several minutes. The technique in this chapter aims at exactly the same result —high- and low-priority tiles coexisting on the same background— but done by hand, tile by tile, with more direct control over each step.

This is a brief addendum, not a new topic: it assumes familiarity with the row-based palette structure already used to encode tile priority, so it's worth having Chapters 14 and 15 fresh in mind before continuing.

Prerequisites

Before continuing, it helps to have:

- Read Chapters 14 and 15, on multiple palettes within a single background and on depth via tile priority, in order to understand what problem tile priority solves in terms of the Mega Drive hardware, and what the row-based palette structure is —normal rows, filler rows, and priority rows— that this chapter completes by hand instead of with a script.
- Basic familiarity with Aseprite: layers, selection, copy and paste, and the difference between RGB and Indexed color mode.
- SGDK installed; if you're going to update it to version 2.11 following the last section of this chapter, it helps to have the project's code editor on hand, for its search-and-replace features.

1. Analyzing the palette layout of a real-world example

A good exercise before applying the technique is to analyze how a commercial game distributes its colors across palettes. The first level of *Alien 3* serves as a good example of tile priority: there's a single scrolling background and, on top of it, within the same foreground layer, some columns let the character pass in front of them and others behind, which gives the scene a sense of depth.

The first level of Alien 3: different columns in the foreground use tile priority so the character appears in front of or behind them.

The foreground of this level uses two palettes, PAL0 and PAL1, and although in principle all four palettes could be used within a single background —as long as each tile uses only one of them—, a careful analysis reveals that the actual layout in this example is fairly inefficient:



Figure 31: The first level of Alien 3

there's quite a bit of color duplication between palettes, with black and several shades of gray repeating in almost all of them. Counting the truly distinct colors used by the level, you arrive at 44 unique colors, out of a possible maximum of 61.

A close-up of the level's palette: black and several shades of gray repeat across more than one of the four rows.

Going deeper: where does that maximum of 61 come from, if four 16-color palettes add up to 64? The color at index 0 of each palette is always the transparent color for tiles using that palette, and of the four times that index 0 appears (one per palette), only one actually ends up shown on screen, as the general background color. The other three are, in practice, palette space that can no longer be used to draw anything visible. Four 16-color palettes minus those three redundant entries give exactly 61 colors available for simultaneous use. Going beyond that barrier of 61 simultaneous colors is possible with more advanced palette-management techniques, which remain an interesting topic to explore further on your own.

2. Preparing a working copy

To avoid risking the original file, the first step is to work on a copy. On the original graphic, select all (Ctrl+A) and copy (Ctrl+C); then create a new file (Ctrl+N) of the exact same size as the original, and paste the copy into it (Ctrl+V). With that copy ready, copy the first four rows of the original palette —the 64 entries representing the four hardware palettes— and paste them into the new file's palette, also switching its color mode to Indexed.

The freshly prepared working file: the top rows retain the real colors copied from the original, the middle band is painted a flat color to set it apart, and the bottom band doesn't yet have the high-priority palette defined.

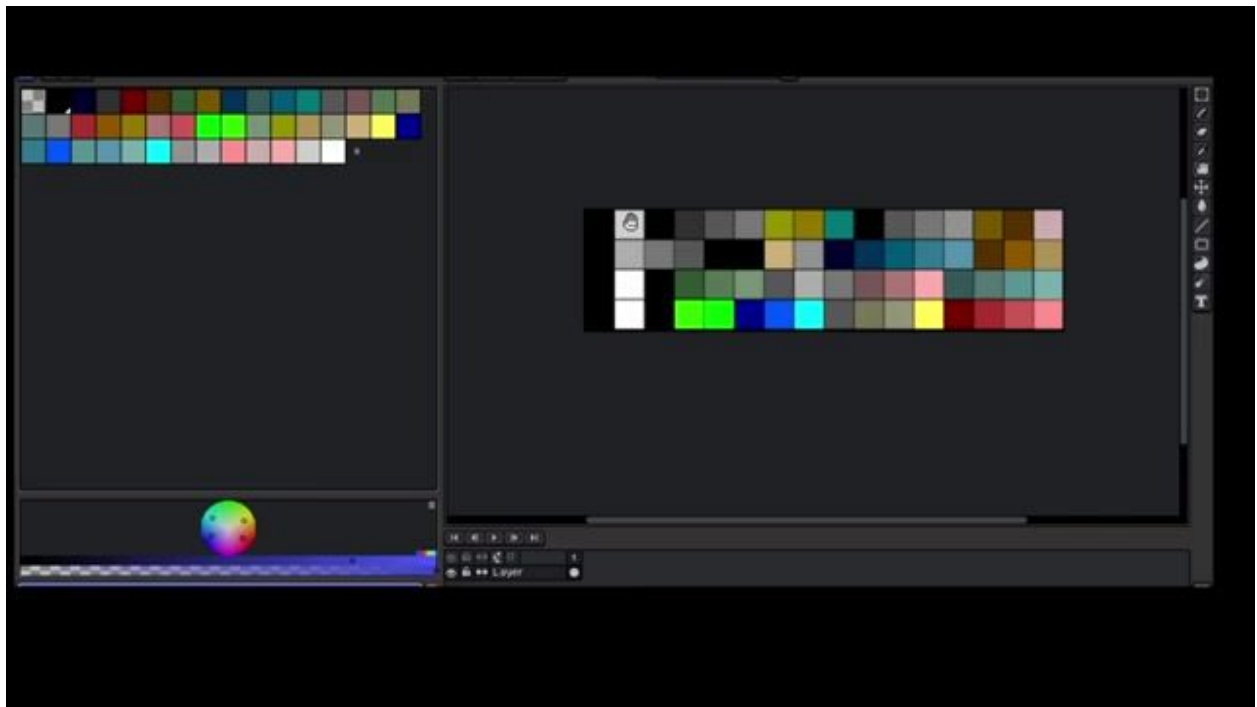


Figure 32: Close-up of the Alien 3 palette

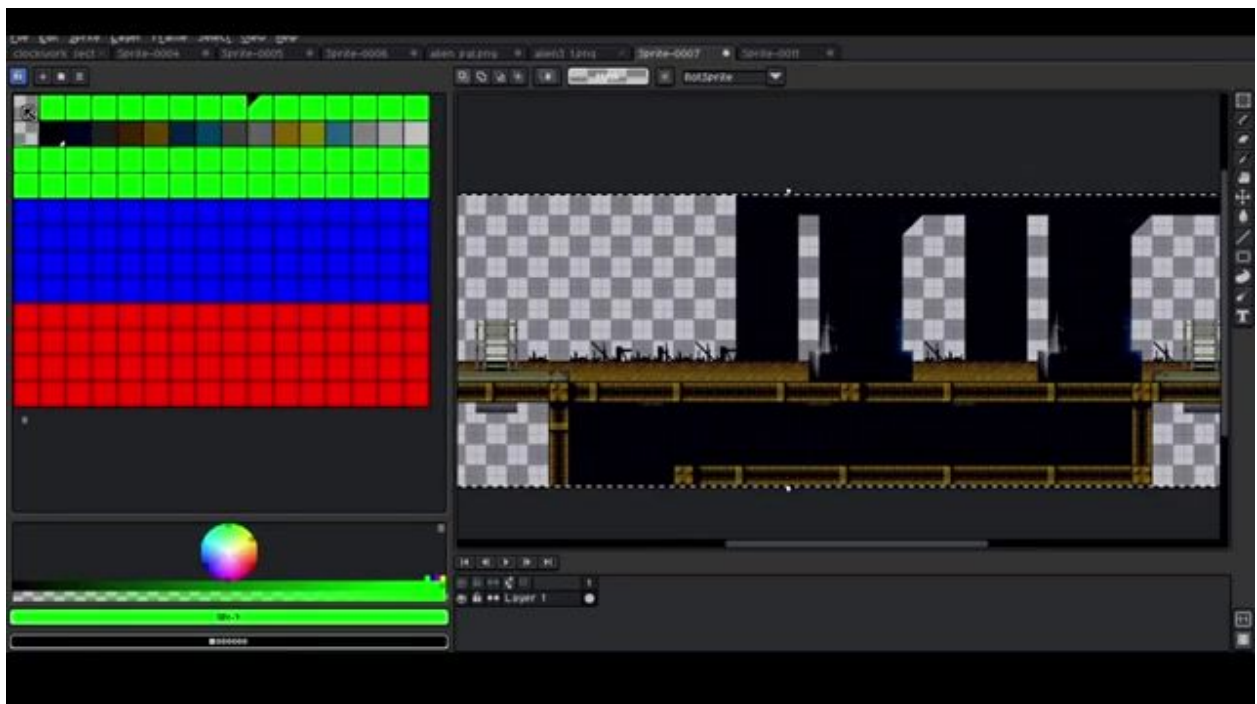


Figure 33: Freshly prepared working file

This layout is the same twelve-row palette structure used to encode tile priority: four rows with the real palettes, four filler rows with no use at all, and four final rows reserved for the high-priority version. The difference here is that it's filled in by hand, tile by tile, instead of letting an automated process do it. To keep the example simple, it helps to concentrate all the work on a single palette —PAL1, for instance— even though the actual level also uses PAL0 in other areas; that doesn't change the procedure, it just needs to be repeated.

3. Isolating the high-priority tiles

With the working copy ready, the next step is to keep only the tiles that will be high priority, deleting all the rest. Priority, just like palette, is assigned tile by tile, so you need to be careful not to accidentally erase only part of an 8×8 pixel tile. The safest way to avoid that is to first convert the layer into a tileset layer, so that any selection or deletion is automatically constrained within the boundaries of each tile.

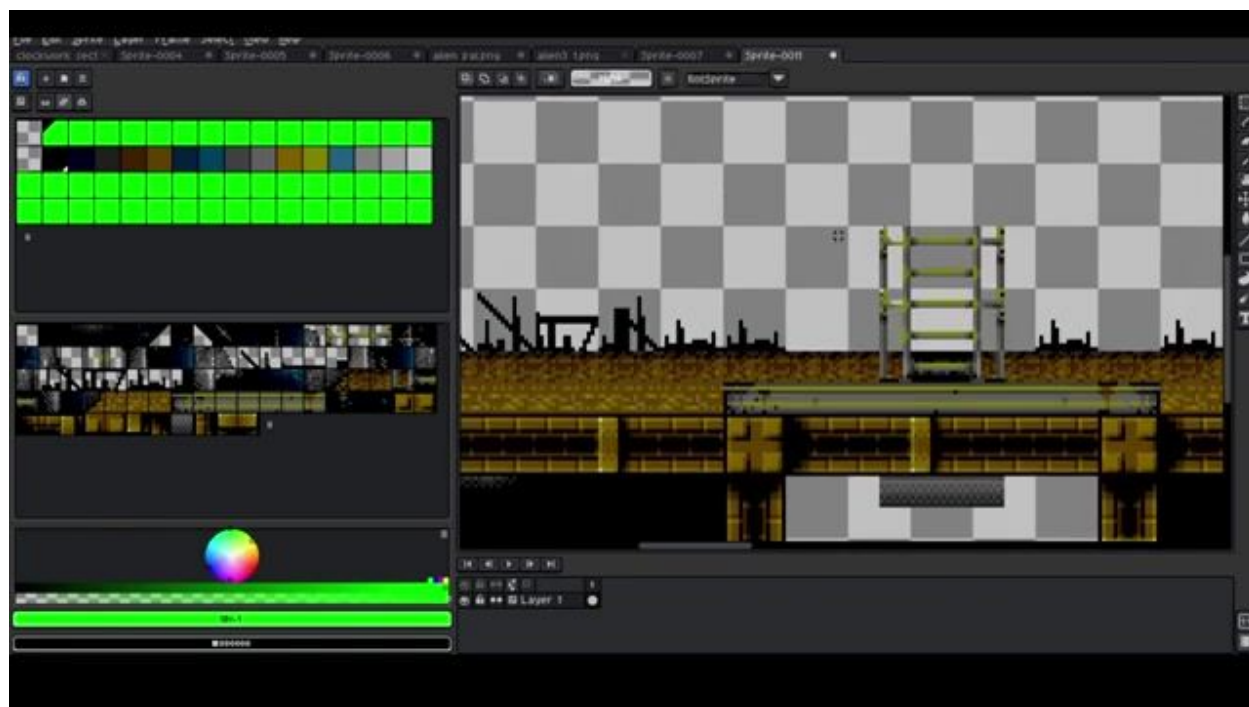


Figure 34: Layer converted to a tileset

The layer already converted into a tileset, with the strip of unique detected tiles at the bottom.

With that safeguard in place, delete all the tiles that should end up as low priority, leaving visible only the ones that should end up as high priority: in the Alien 3 example, the part of the staircase the character passes behind, and the front of the larger columns, which the character also passes behind. The rest of the floor and the smaller columns, which remain low priority, are deleted from this copy.

The working copy after deleting the low-priority tiles: only the priority tiles remain visible, with the rest of the canvas transparent.

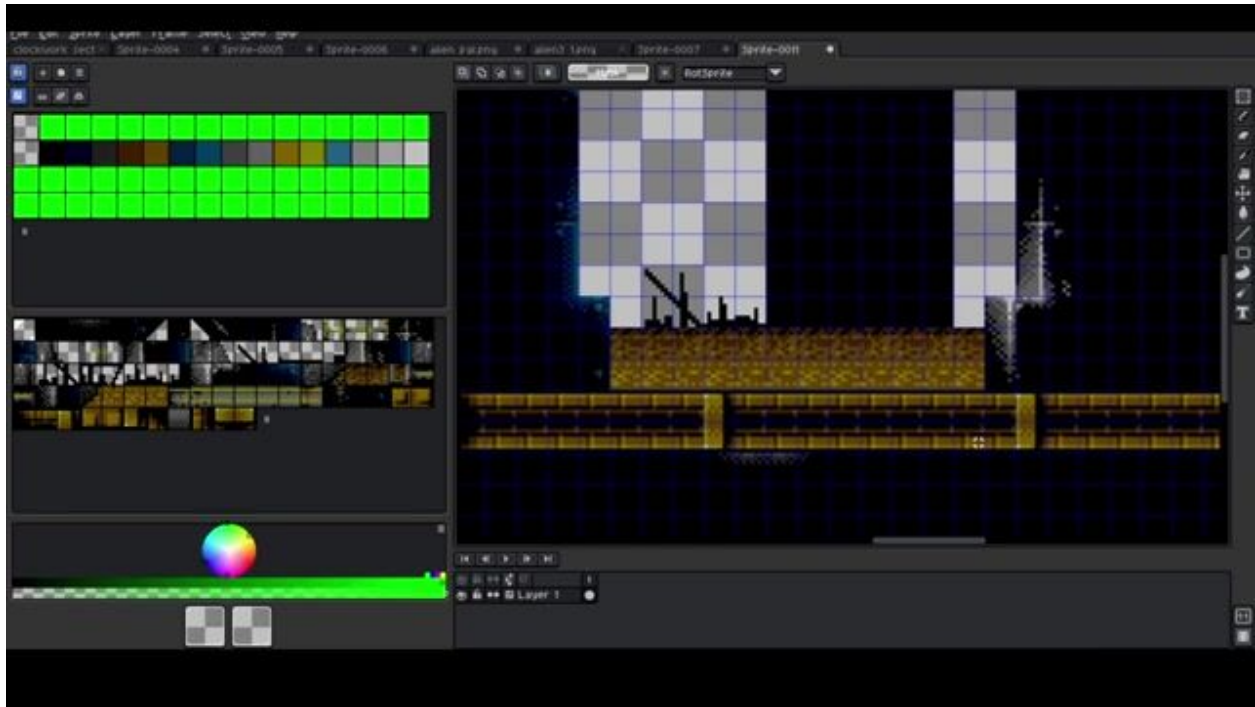


Figure 35: Working copy after deleting low-priority tiles

4. Giving them an unmistakable temporary palette: the Master System palette

The next step is to assign these surviving tiles a palette completely different from the four low-priority palettes, so they can be identified without any ambiguity later on. For that, it helps to use one of Aseprite’s preset palettes: the Master System palette, from Sega’s 8-bit console that preceded the Mega Drive.

The list of Aseprite preset palettes, with the Master System palette highlighted.

This palette turns out to be especially convenient for this task, regardless of any interest in Master System graphics, because it has exactly 64 colors that are all completely distinct from one another—the same number of entries occupied by the four low-priority rows. Once loaded, any tile using it stands out immediately from the low-priority tiles, with no possibility of confusing them at a glance.

The Master System palette already loaded, with its 64 colors, all different from each other.

Attention: there’s a single exception to keep in mind. The first color in the Master System palette is a pure black (RGB 0, 0, 0), exactly the same value probably already being used as black in the Mega Drive palette. If it isn’t modified, that color stops being unmistakable. The simple solution is to slightly alter that black—for example, nudging up one of its three values—to get a shade that still reads as black, but that is numerically distinct from any black already present in the low-priority palette.

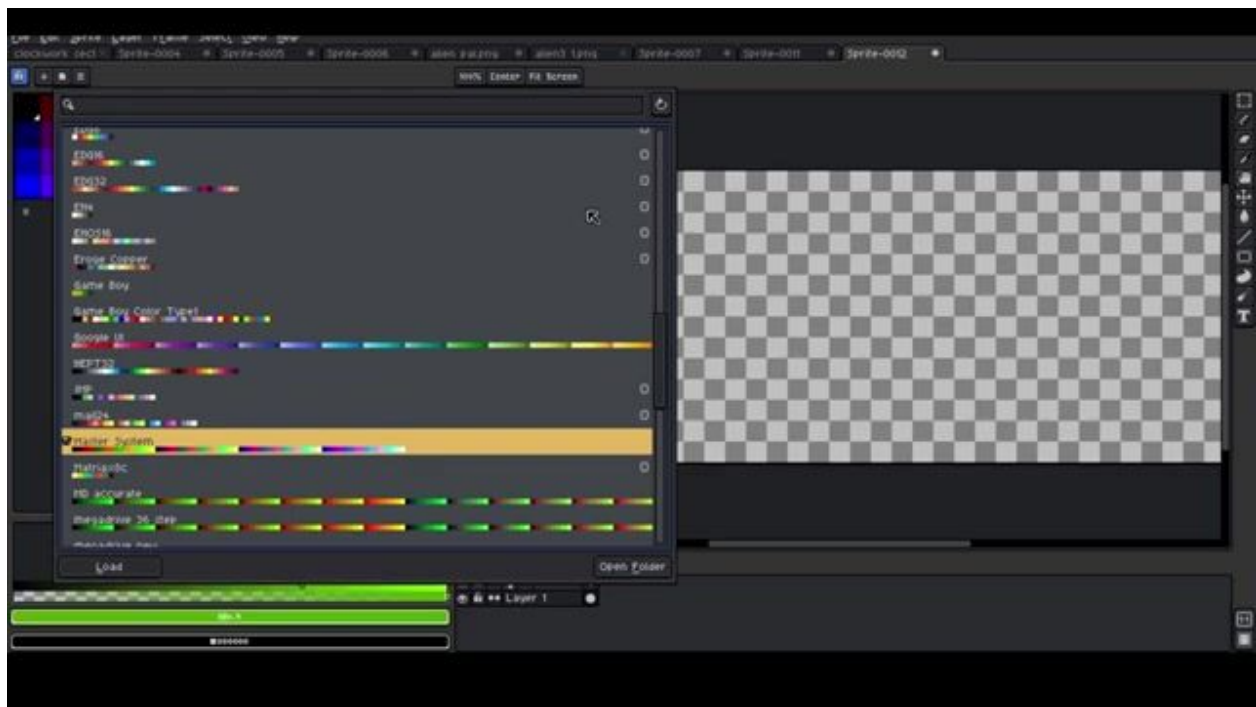


Figure 36: List of Aseprite preset palettes

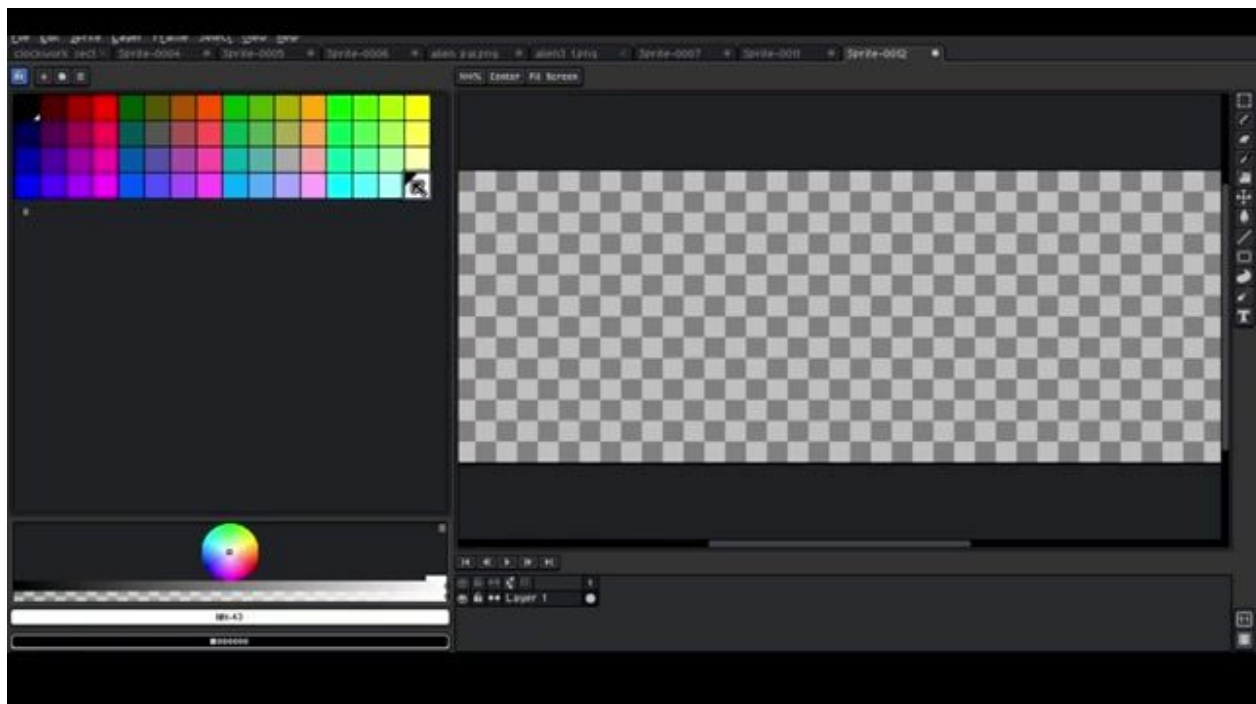


Figure 37: Master System palette loaded

5. Transplanting the high-priority tiles into the original file

With the Master System palette already adjusted, copy it in full —its 64 colors— and paste it over the palette of the working file (the one from section 3, with only the high-priority tiles). Since that file is already in Indexed mode, the palette swap immediately recolors all the tiles: the ones that used the PAL1 row, for example, now show the second row of the Master System palette instead.

Tip: if pasting the Master System palette doesn't change the tiles' colors, the file is most likely still in RGB mode rather than Indexed mode. Without that mode change, each pixel keeps its literal color value instead of referencing a palette position, so replacing the palette has no visible effect on the artwork.

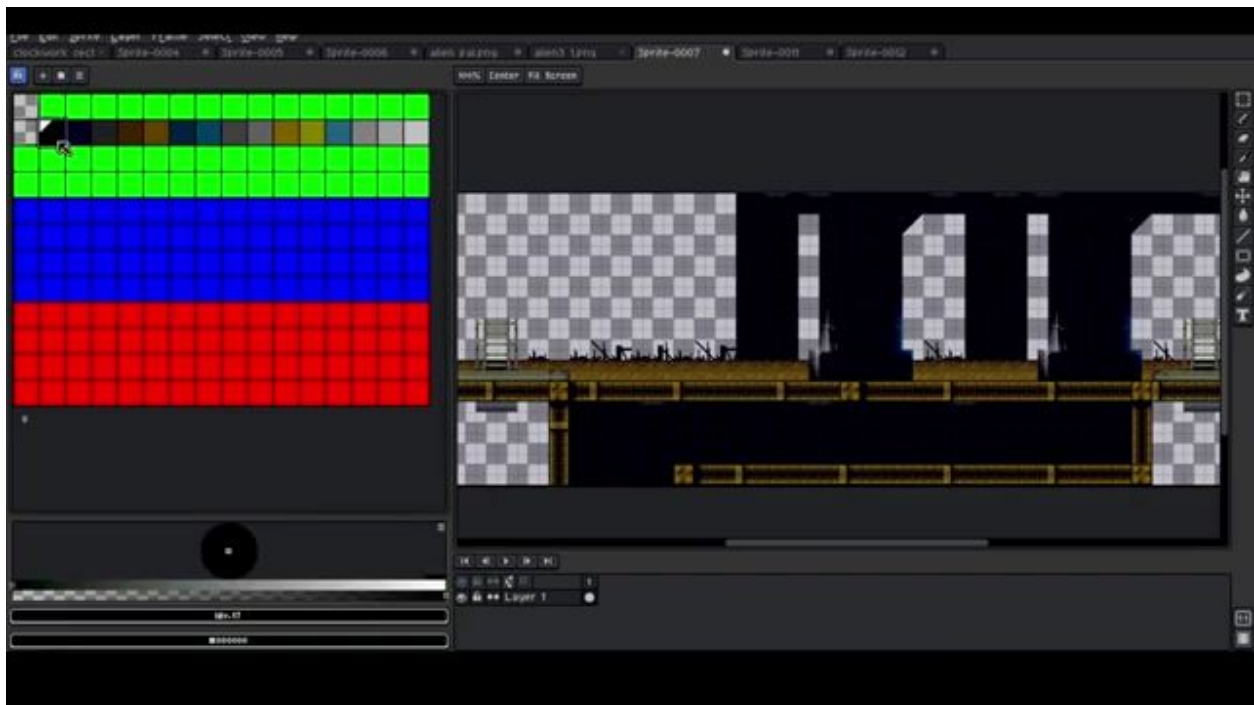


Figure 38: High-priority tiles recolored with Master System

The high-priority tiles now recolored with the Master System palette: where PAL1 used to be, there's now a completely different, unmistakable color.

The same procedure is repeated on the original file: copy the already-adjusted Master System palette and paste it, this time, over the final four rows of that file —the ones which, following the structure already covered, are reserved for the high-priority version and still have no color defined.

The final four rows of the original file, still undefined, right before receiving the Master System palette.

The last step in this section is to copy the high-priority tiles themselves —not just their palette, but their actual graphic content— from the working file into the original file. To do that, they need to be selected by clicking inside the color-index panel, in the top-left corner, and not within the canvas area: selecting there, Ctrl+A and Ctrl+C copy the tiles together with their new palette, and they get pasted directly onto the original file, in the same position they occupied before the low-priority tiles were deleted.

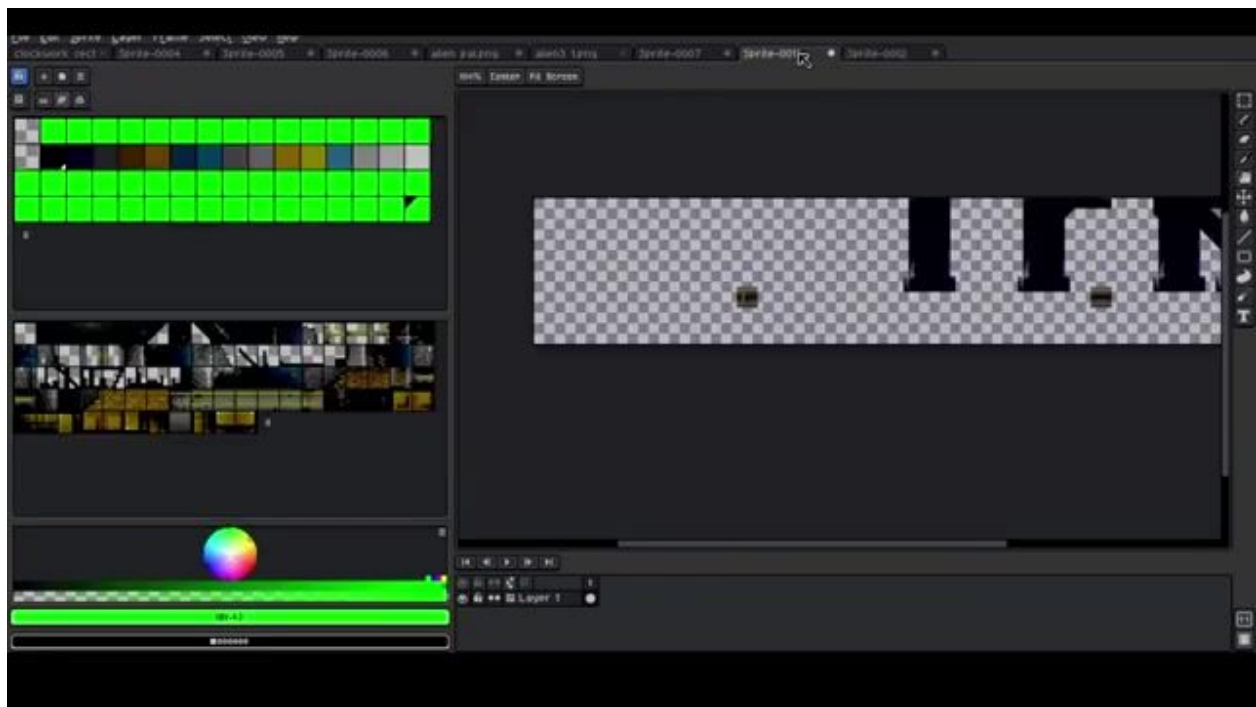


Figure 39: Final four rows still undefined

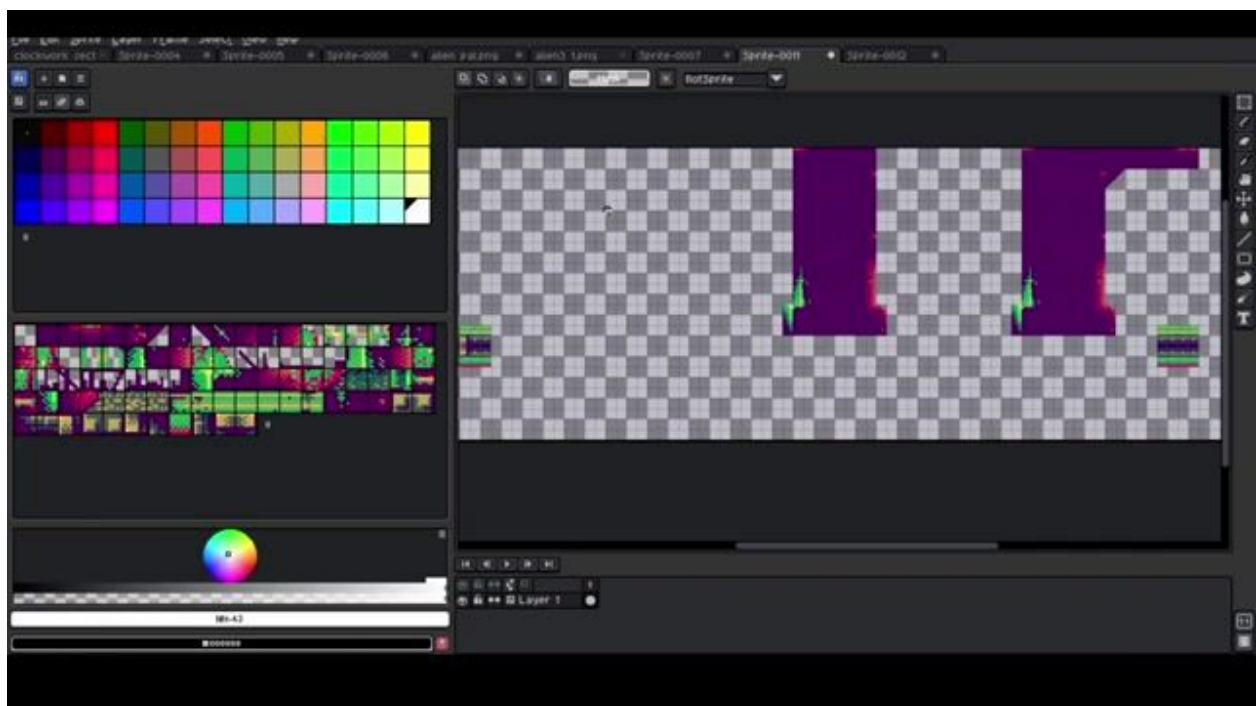


Figure 40: Immediate result after pasting

The immediate result after pasting: the high-priority tiles, still with the eye-catching Master System colors, overlaid in their original position within the complete file.

At this point, the original file has the complete twelve-row palette structure: the first four rows with the real low-priority colors, the next four as filler, and the last four with the high-priority tiles, still colored with the Master System palette instead of their real colors.

6. Checking for palette mixing, and restoring the real colors

Before saving and considering the file finished, it's worth taking advantage of how eye-catching the Master System colors are to do a full visual check: fill each of the eight relevant rows —the four low-priority ones and the four high-priority ones— with a single flat color, each one clearly different from the others, and go through the entire file looking for any pixel that doesn't match the color that should correspond to its row.

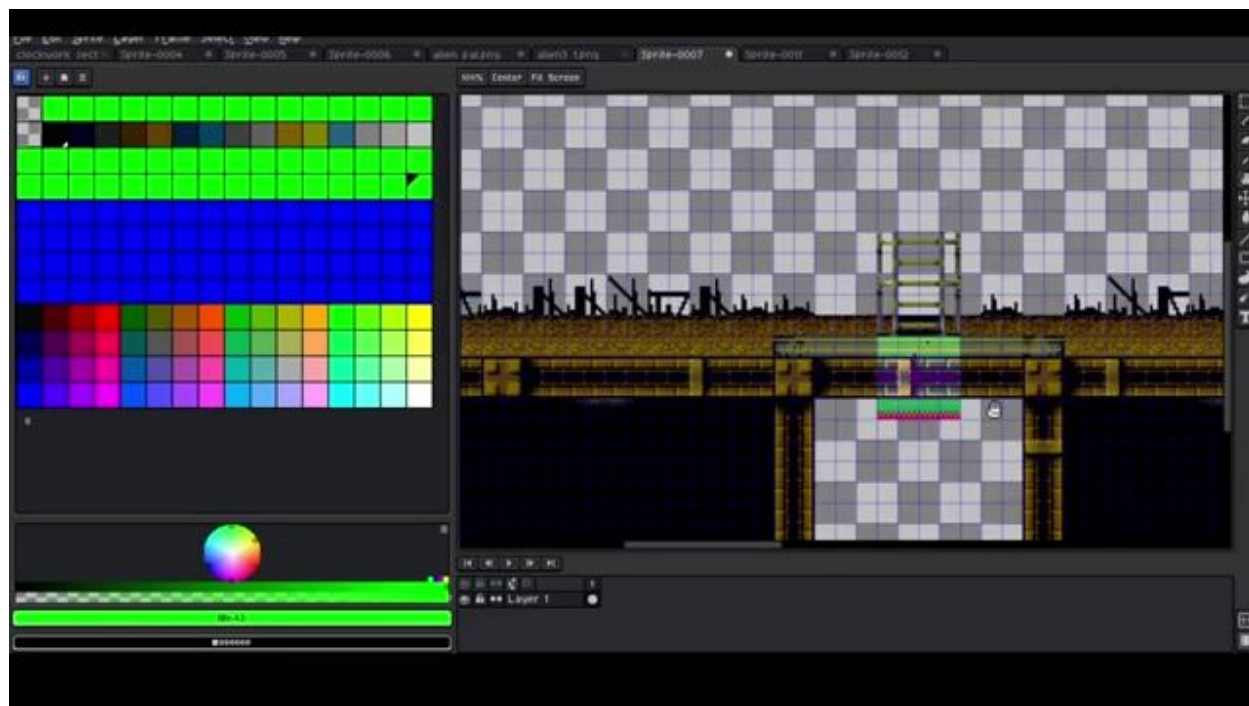


Figure 41: Flat-color test taken to an extreme case

The flat-color test taken to an extreme case: each half of the palette painted a clearly different color, so that any assignment error is impossible to miss.

The same test applied to the level's actual tiles: any pixel that wasn't exactly one of the two control colors would immediately reveal a palette error.

Once it's confirmed that there's no mixing between palettes —except for the transparent color, which can legitimately appear in any row without being an error—, the final step is to undo this test and restore each row's real color: both the four low-priority rows and the four high-priority rows get back the true colors from the game's art, and only then is the file saved for importing into SGDK.

Going deeper: this manual method makes something clear that's also worth keeping in mind when thinking about the script-based workflow in Chapter 19: the high-

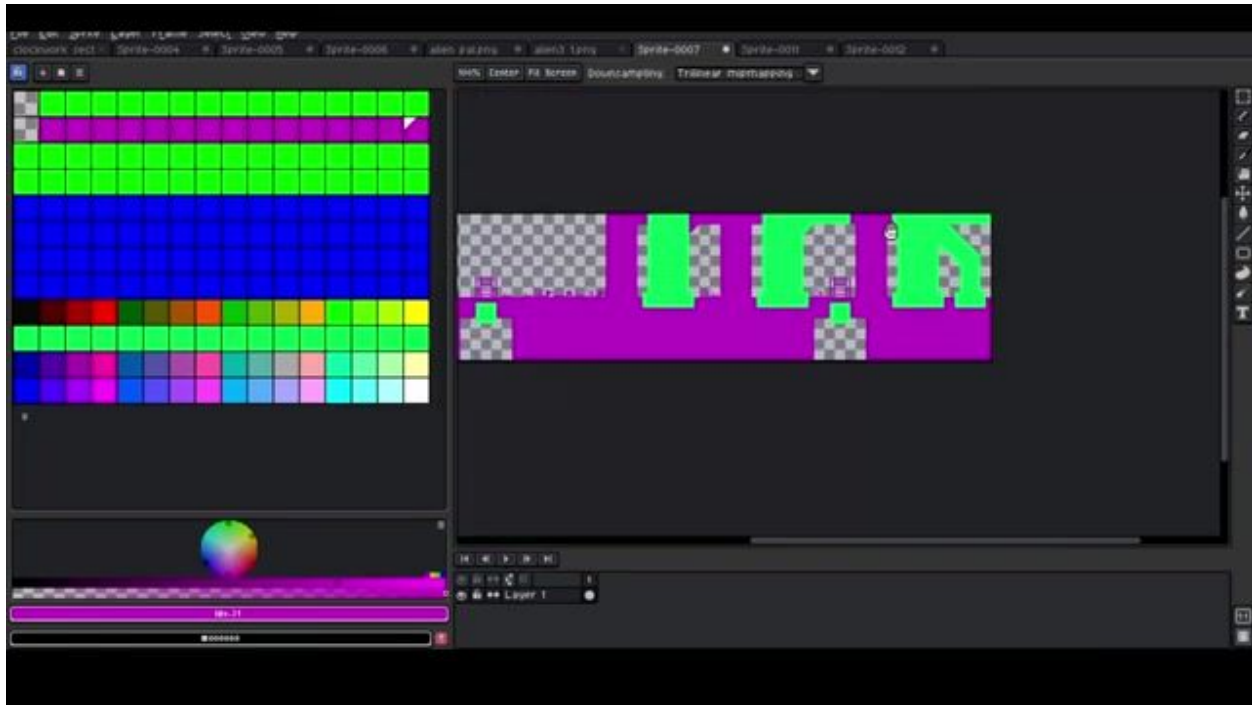


Figure 42: Flat-color test applied to the actual tiles

priority rows end up containing exactly the same real colors as their corresponding low-priority rows, and the only thing that distinguishes them, as far as SGDK is concerned, is the position of those rows within the palette —rows 8 through 11 instead of 0 through 3— not any special color or visible marker in the final image. It's reasonable to think that the script mentioned in Chapter 19 achieves, in effect, an equivalent result internally, although that particular part of its inner workings was not directly confirmed. What can be stated with certainty is that, in their final effect on the compiled game, both methods are interchangeable: they produce the same tile-priority behavior on screen.

The final result combines the parallax scrolling background, a foreground with columns of different priority, and test sprites overlaid on top, as already worked on in previous chapters. Deciding which tiles should be marked as priority is, to a large extent, a matter of visual trial and adjustment and not just a mechanical procedure: reviewing the result on the Alien 3 example, for instance, a couple of smaller columns that had been left as low priority would also have looked better marked as priority.

7. Updating to SGDK 2.11

One change to keep in mind when updating a project to SGDK 2.11 is that the names of the functions for converting fixed-point values to integers changed: the function previously called `fix32ToInt` is now called `F32_toInt` (the same change applies to the 16-bit variant, `fix16ToInt` → `F16_toInt`, already covered in Chapter 6). A typical example of this conversion, updated to its current form, looks like this:

```
// SGDK 2.11 onward
fix32 velocidadY = FIX32(2.5);
posY += velocidadY;
```



```
// Conversion to integer, for example, to compare against a tile position
s16 posYEntera = F32_toInt(posY);
```

Instead of changing every occurrence of the function by hand, it's best to use the code editor's search-and-replace feature: search for the old name, type the new name into the replacement field, and apply the option to replace all matches in the project at once, rather than one at a time.

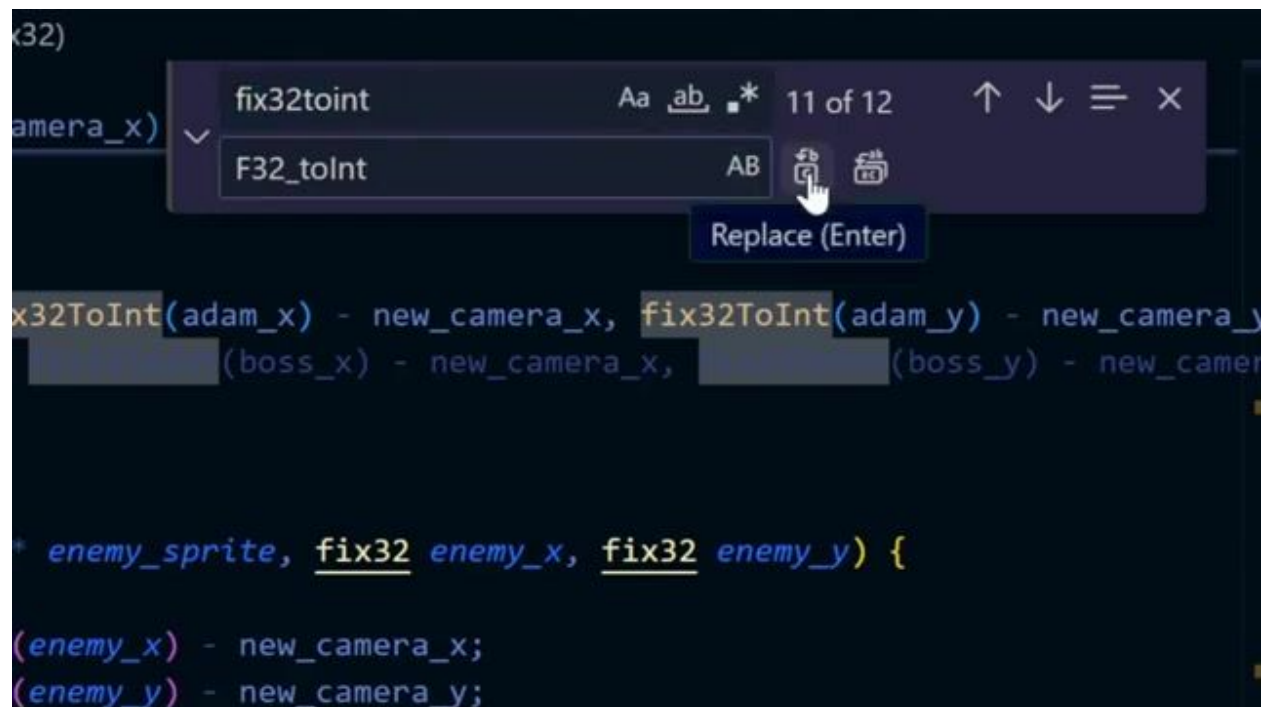


Figure 43: Search-and-replace dialog

The editor's search-and-replace dialog, with the old function name and its replacement already typed in, ready to be applied to every match in the project.

A general recommendation, valid for any SGDK version update and not just this one in particular: it's worth deleting the boot folder, located inside the project's src folder. SGDK automatically generates a new boot folder, corresponding to the installed version, the next time the project is compiled.

Review exercises

1. Explain why four 16-color palettes allow, at most, 61 truly visible colors on screen and not 64.
2. Why is it a good idea to convert the layer into a tilesheet before deleting the low-priority tiles, rather than deleting them directly on a regular layer?
3. Explain specifically what the Master System palette is used for in this procedure, and what particular precaution needs to be taken with its first color before using it.
4. Based on what's explained in this chapter, what actually distinguishes a high-priority tile from a low-priority tile in the final file that gets imported into SGDK?

5. What specific change needs to be taken into account when updating an existing project to SGDk 2.11 regarding fixed-point-to-integer conversions, and which code editor tool is best used to apply it safely across the whole project?

Chapter 17 — Tile Scrolling

The Mega Drive has only two scrollable background planes, BG_A and BG_B. In principle, that limits a game to just two independent parallax “layers” on screen at once. Tile scrolling (*tile scrolling*) is the technique that lets you stretch that limit quite a bit further: instead of shifting an entire plane as a single block — as was done in chapter 9 with HSCROLL_PLANE — each 8-pixel-tall row of that same plane is shifted independently, with its own speed and its own direction. Used well, the effect gives the impression that the console is handling four or five parallax layers at once, even though the hardware still only has two.

Chapter 2 already introduced, at a conceptual level, that the VDP supports three levels of granularity for horizontal scrolling — per plane, per tile, and per line — and two for vertical scrolling — per plane and per column. Chapter 9 put code to the first of those variants, the simplest one. This chapter picks up that foundation and extends it toward the intermediate granularity: horizontal scrolling per tile. Along the way, it introduces two C tools that will accompany the reader for the rest of the manual: arrays, mentioned in passing back in chapter 6 but not explained in detail until now, and for loops, which appear for the first time in this manual with a dedicated explanation.

Prerequisites

Before continuing, it helps to have:

- Completed chapter 9, “Scrolling and Parallax Scrolling,” since this chapter builds on the same type of project (a background plane drawn with `VDP_drawImageEx`) and directly reuses `VDP_setScrollingMode`, the three-section organization of `main()`, and the pattern of accumulating a displacement in `fix16/fix32` and converting it to an integer before passing it to the VDP.
- The three scrolling granularities introduced conceptually in chapter 2 (per plane, per tile/column, and per line).
- The `S16`, `U8`, and `FIX16` types from chapter 6.
- Basic notions of C: variable declaration, the structure of a while loop, and the use of square brackets to index a position within a collection of values (explained in detail in section 4 of this chapter).

1. What tile scrolling is and what it’s for

To see the effect clearly, it helps to have a test background where each 8-pixel-tall row is numbered or easily distinguishable from the next, so you can immediately tell which row is moving at any given moment.

The test background used in this chapter: each row of tiles is numbered, so you can tell at a glance which one is scrolling at any moment and in which direction.

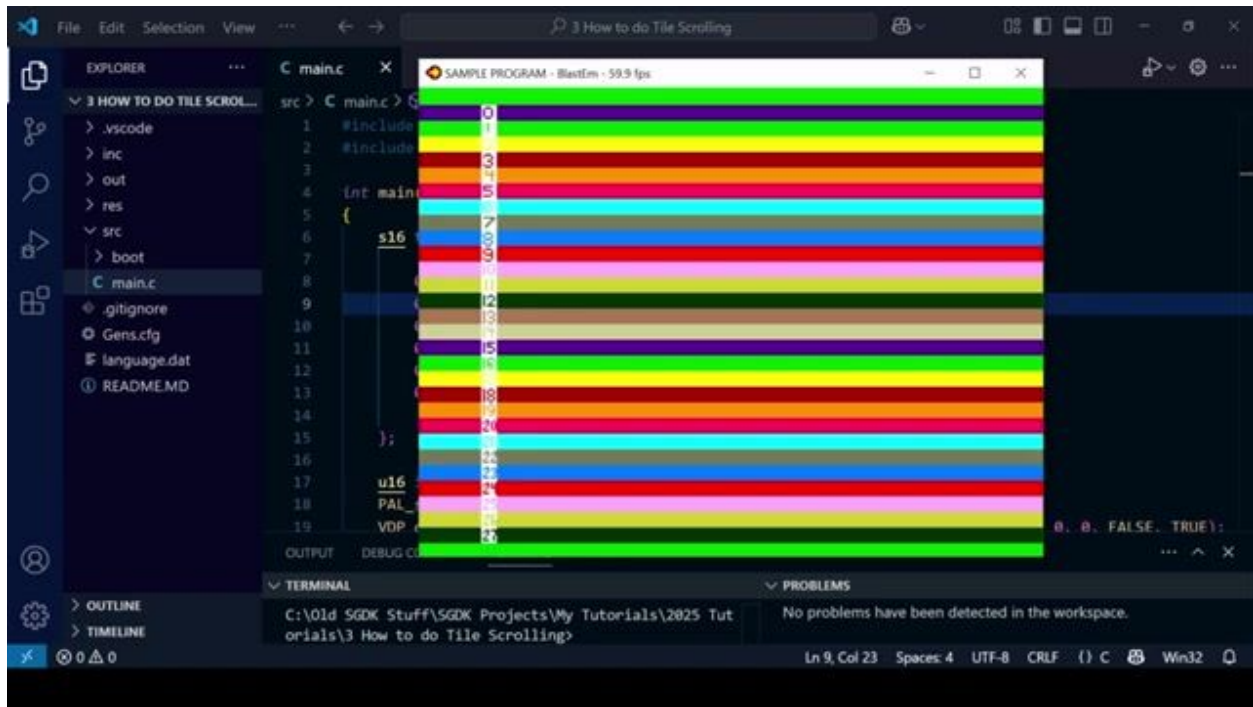


Figure 44: Test background with numbered tile rows

The core idea is simple to state, though it has more moving parts than plane scrolling: instead of a single displacement variable for the whole plane, you need one displacement value per visible row on the screen. With the standard vertical resolution of 224 pixels (see chapter 10) and 8-pixel-tall tiles, that comes out to $224 \div 8 = 28$ rows. The rest of this chapter deals with declaring those 28 values, updating them frame by frame, and doing so without repeating code 28 times.

2. Turning on tile scrolling mode

The first step is to tell SGDK that, for the plane you're going to use, horizontal scrolling should be handled row by row instead of moving as a single block. This is configured with the same function already used in chapter 9, `VDP_setScrollingMode()`, but with a different value for the horizontal mode:

```
VDP_setScrollingMode(HSCROLL_TILE, VSCROLL_PLANE);
```

As a recap from chapter 2, the horizontal scrolling options are `HSCROLL_PLANE` (the whole plane as a single block, the variant from chapter 9), `HSCROLL_TILE` (each 8-pixel row independently, the one covered here), and `HSCROLL_LINE` (each 1-pixel line independently, the finest granularity, revisited at the end of this chapter). For the vertical axis, the options are `VSCROLL_PLANE` and `VSCROLL_COLUMN` — the latter replacing the old `VSCROLL_2TILE`, as already noted in chapter 9. This chapter uses `HSCROLL_TILE` for the horizontal axis and `VSCROLL_PLANE` for the vertical one, since only the horizontal displacement of each row is being worked with.

Combining horizontal tile scroll with vertical tile scroll: unconfirmed.

There's no practical verification, within the scope of this chapter, of whether it's possible to combine `HSCROLL_TILE` with an independent column scroll (`VSCROLL_COLUMN`) on the same plane at the same time. There's reason to suspect it isn't — the VDP

might not offer that particular combination — but that’s a reasonable suspicion, not a confirmed fact checked against SGDK’s official documentation. Before designing an effect that depends on that combination, it’s worth testing it first in a small project and checking the result on screen.

3. Updating the scroll every frame: `VDP_setHorizontalScrollTile`

Turning on tile scrolling mode only tells SGDK what type of scrolling is going to be used; you still need the function that actually moves each row, which has to be called inside the main loop (while), since the displacement needs to be updated every frame. That function is `VDP_setHorizontalScrollTile()`, and it takes five parameters:

```
void VDP_setHorizontalScrollTile(VDPPlane plane, s16 tileY, s16 *values, s16 num,  
    ↪ u16 useDMA);
```

- `plane`: the background layer the tile scroll is applied to, `BG_A` or `BG_B`.
- `tileY`: the tile row, counted from the top, at which the effect starts being applied. With 0, the effect is applied starting from the first visible row, which is useful when there are elements — clouds, for example — that need to move starting from the very top of the screen.
- `values`: the array that holds the displacement value for each row, discussed in the next section.
- `num`: the number of tile rows that need to be updated. With the standard vertical resolution of 224 pixels and tiles 8 pixels tall, that number is 28.
- `useDMA` (or the equivalent name for the transfer method): SGDK recommends using `DMA_QUEUE` for this parameter, since several rows need to be updated every frame, and queuing the transfer via DMA avoids blocking the CPU while the data is copied.

4. Arrays: declaring the position array

In chapter 9, a single variable was enough to store how much a plane had shifted along the horizontal axis. That’s not enough here: since each of the 28 rows can move independently, you need to store one displacement value per row, and for that you need an **array** instead of a single variable.

An array is, essentially, a collection of variables of the same type, all grouped under a single name and accessed via a numeric index. It’s declared similarly to an ordinary variable, but with square brackets added containing the number of elements it will hold:

```
S16 tile_scroll_array[28];
```

This reserves space for 28 values of type `S16` (a signed 16-bit integer, which is more than enough for pixel displacements, whether positive or negative), all accessible through the same name, `tile_scroll_array`. To access a specific element of the array, you use the name followed by the index in square brackets, always starting from 0: `tile_scroll_array[0]` is the first element, `tile_scroll_array[27]` is the last of the 28.

Indexes start at 0. Index 0 of an array corresponds to the first element, not the second. In this chapter, that means `tile_scroll_array[0]` is the first visible row on screen, not the second; this is a common off-by-one mistake when you’re starting out with arrays.

With all values at zero, which is the initial state before any movement code is written, no row moves yet. To check that the array and `VDP_setHorizontalScrollTile()` are properly

connected, it's enough to manually assign a value to a specific index: positive to move that row to the right, negative to move it to the left. For example, `tile_scroll_array[0] = 10;` shifts the first row 10 pixels to the right, while `tile_scroll_array[1] = -5;` shifts the next row 5 pixels to the left. Once compiled, each row ends up shifted according to the sign and magnitude of the value you assigned it.

5. Dynamic scrolling with += and -=

Assigning a fixed value, as in the previous section, produces a static, one-time displacement. To achieve continuous movement, each row needs to change its displacement value frame by frame, inside the while loop. The simplest way is to take the current value of an array index and add or subtract a fixed amount on each pass, using the += and -= operators:

```
while(1)
{
    tile_scroll_array[0] += 1;    // row 0 advances 1 pixel per frame to the right
    tile_scroll_array[20] -= 2;   // row 20 advances 2 pixels per frame to the left

    VDP_setHorizontalScrollTile(BG_B, 0, tile_scroll_array, 28, DMA_QUEUE);
    SYS_doVBlankProcess();
}
```

Each row can thus have its own operator and its own speed, moving completely independently of the rest.

6. Fractional displacement with FIX16

One pixel per frame is a good pace for many elements, but for backgrounds that need to move much more slowly — distant clouds, for example — it's useful to be able to add an amount smaller than a full pixel, such as 0.5 or 0.1. The problem is that `VDP_setHorizontalScrollTile()` only accepts integer values through its S16 array, so there's no direct way to add a fraction of a pixel to `tile_scroll_array`.

The solution is the same one already used in chapter 9 to solve the same problem for plane scrolling, now applied row by row: a second array, with the same number of elements, but of type FIX16 instead of S16. FIX16 is the 16-bit variant of the fixed-point types already covered in chapter 6 (FIX32 was used in chapter 9 for the fractional displacement of an entire plane); here the lighter version is enough, because background scrolling doesn't need that much precision. To distinguish it from the original array, it helps to add a suffix to the name:

```
FIX16 tile_scroll_array_as_f16[28];

// inside the while loop:
tile_scroll_array_as_f16[1] += FIX16(0.5); // accumulates half a pixel per frame
```

Adding a value to this array doesn't move anything by itself, because `VDP_setHorizontalScrollTile()` still works with the original integer array. What's still needed is to convert the accumulated FIX16 value into an integer and copy that result into the same position of the S16 array, before it's used in the scroll function. For that conversion, SGDk 2.11 uses `F16_toInt()` — the FIX16 equivalent of the `F32_toInt()` function already used in chapter 9 for FIX32:

```
tile_scroll_array_as_f16[1] += FIX16(0.5);
tile_scroll_array[1] = F16_toInt(tile_scroll_array_as_f16[1]);
```

7. For loops to avoid repeating code

So far, every row you want to animate with fractional speed needs its own two lines of code: one to add the speed to the FIX16 array, and another to convert that value to an integer and copy it into the S16 array. Repeating that same pair of lines 28 times — once per row — would be a sure source of copy-paste errors and pretty messy code. This is the point to introduce, with a dedicated explanation for the first time in this manual, the for loop.

The syntax of a for loop has three parts, separated by semicolons, inside the parentheses that follow the word for:

```
for (U8 i = 0; i < 28; i++)
{
    // loop body: runs once for each value of i
}
```

- The first part, `U8 i = 0`, declares a control variable and gives it an initial value. `U8` is used because that type goes up to 255 and the array only has 28 elements; the name `i` is a very widespread convention in C, though it could be called anything else.
- The second part, `i < 28`, is the condition evaluated before each pass: as long as it's true, the loop keeps repeating. With this condition, the loop runs through indexes 0 to 27, which are exactly the 28 valid elements of the array.
- The third part, `i++`, is the instruction executed at the end of each pass: it adds one to `i`, so that the condition will eventually stop being true and the loop will end.

The control variable only exists inside the loop. A variable declared inside the parentheses of a for, like `i` in the example, only exists within the braces that delimit that loop's body. Outside of that, the compiler doesn't recognize it. That's why it's safe to reuse the same name, `i`, in several different for loops within the same file: each one has its own copy, isolated from the others.

This chapter uses two for loops for different purposes.

The first is used inside `main()`, and therefore runs only once, when the program starts: instead of writing 28 zeros by hand to initialize `tile_scroll_array`, and another 28 for `tile_scroll_array_as_f16`, a single loop runs the index from 0 to 27 and assigns zero to both arrays on each pass:

```
S16 tile_scroll_array[28];
FIX16 tile_scroll_array_as_f16[28];

int main()
{
    // ... palette loading and background drawing, as in chapter 9 ...

    for (U8 i = 0; i < 28; i++)
    {
        tile_scroll_array[i] = 0;
        tile_scroll_array_as_f16[i] = FIX16(0);
    }

    VDP_setScrollingMode(HSCROLL_TILE, VSCROLL_PLANE);

    while(1)
    {
        // ...
    }
}
```

```

    return (0);
}

```

It's best to declare the arrays first, with empty braces (that is, without initializing their elements yet), and write the for loop afterward: if the loop appeared before the arrays exist, the compiler flags an error.

The second for loop appears inside the while, and therefore runs every frame of the game. This one isn't for zeroing everything out — the previous loop already did that, just once — but rather to replace the two lines that previously had to be repeated by hand for each row: inside the loop's body, the corresponding speed is added to the FIX16 array and the result is converted to an integer with `F16_toInt()`, using the loop's index `i` instead of a fixed number, so that a single instance of those two lines is enough to update all 28 rows.

8. A third array for speeds, and named variables

The loop from the previous section needs a third array: one that stores the scroll speed of each row, in FIX16. This array can't be initialized with a for loop like the previous two, because its values aren't all the same: some rows might represent clouds, others mountains, others a forest, and each moves at a different pace. It helps to call it `scroll_speed_array_as_f16`, and manually assign the speed value for each row:

```

FIX16 scroll_speed_array_as_f16[28] = {
    FIX16(0.1), FIX16(0.1), FIX16(0.1), // rows 0-2: distant clouds
    FIX16(0.5), FIX16(0.5),           // rows 3-4: mountains
    // ... the rest of the rows, each with its own speed
};

// inside the while loop, replacing the two repeated lines:
for (U8 i = 0; i < 28; i++)
{
    tile_scroll_array_as_f16[i] += scroll_speed_array_as_f16[i];
    tile_scroll_array[i] = F16_toInt(tile_scroll_array_as_f16[i]);
}

VDP_setHorizontalScrollTile(BG_B, 0, tile_scroll_array, 28, DMA_QUEUE);

```

Grouping repeated speeds under a named variable. When several consecutive rows should always move at the same speed — for example, several rows that together make up a layer of clouds — instead of writing the same numeric value at each of those positions, it's better to declare a named FIX16 variable, such as `cloud_speed`, assign it the value once, and use it at every relevant position in the array:

```

FIX16 cloud_speed = FIX16(0.1);
FIX16 scroll_speed_array_as_f16[28] = {
    cloud_speed, cloud_speed, cloud_speed,
    FIX16(0.5), FIX16(0.5),
    // ...
};

```

That way, if the cloud speed needs adjusting later on, it's enough to change a single value in a single place, instead of hunting down and fixing that same figure

repeated across three, four, or five different positions in the array. If those same rows needed to represent clouds moving in the opposite direction from the rest of the background, it's enough to give that variable a negative value, with no need to create a new variable.

9. Beyond this chapter

With the three arrays and the two for loops in place, the final code ends up considerably shorter than it would have been writing each row by hand, even though fractional displacement and per-row speed management are, at bottom, repetitive tasks.

Combining this technique with tools already covered in earlier chapters — using both planes, BG_A and BG_B, each with its own tile scroll; adjusting tile priority between both planes, as in chapter 16; and adding background sprites above or below certain rows — the number of parallax layers you can simulate in practice grows well beyond the console's two physical layers.

The horizontal tile scrolling covered in this chapter has an even finer equivalent, line scrolling (HSCROLL_LINE, mentioned in section 2), which, instead of moving 8-pixel rows, moves individual 1-pixel lines. The code approach is equivalent to this chapter's — an array of values, a per-frame update function, the same for loops — just with many more positions to iterate over (192 lines instead of 28 rows, at the standard resolution). As an exercise to practice that finer variant, you can try recreating a *Thunder Force IV*-style parallax scene using HSCROLL_LINE instead of HSCROLL_TILE.

Review exercises

1. Explain the difference between HSCROLL_PLANE, HSCROLL_TILE, and HSCROLL_LINE, and in what situation each one is the right choice.
2. Why does the array passed to VDP_setHorizontalScrollTile() have 28 elements, and exactly where does that number come from? How many elements would the equivalent array have if HSCROLL_LINE were used instead of HSCROLL_TILE?
3. Explain why a second array in FIX16 is needed to achieve speeds lower than one pixel per frame, and which function is used to convert those values before passing them to VDP_setHorizontalScrollTile().
4. Why can't the speed array (scroll_speed_array_as_f16) be initialized with a for loop the same way as the other two arrays in this chapter?
5. A teammate writes the following code and wonders why it doesn't compile:

```
int main()
{
    for (U8 i = 0; i < 28; i++)
    {
        tile_scroll_array[i] = 0;
    }

    S16 tile_scroll_array[28];
    // ...
}
```

Identify the error and explain why the order of the lines matters in this case.

Chapter 18 — Background Tile Animation

Up to this point in the manual, VRAM has been treated as something that gets filled once, when loading a level, and then stays fixed for the rest of the game’s execution. This chapter breaks with that idea: it shows how to replace, at any moment and while the game keeps running, one or several specific tiles within VRAM that has already been loaded. That capability is the foundation of background tile animation, the technique used by games like *Sonic the Hedgehog* to give certain scenery elements, such as torches, water, or flowers, their own animation cycle independent of the rest of the background, without needing to touch sprites or reload anything.

The example used throughout the chapter is the yellow flowers in the background of Green Hill Zone, the first level of *Sonic the Hedgehog*, which spin continuously in place. The chapter starts from a demo with that level’s background and foreground already loaded and with parallax scrolling between the two layers, and builds the complete animation of those flowers step by step.

Prerequisites

- Having read chapter 7 (how to display backgrounds) and chapter 9 (scrolling and parallax scrolling), since this chapter starts from a demo with those same elements already working.
- Having read chapter 2 (the VDP in depth), to have a clear picture of how VRAM is organized, and chapter 16 (multiple palettes and tile priority), to be familiar with the way SGDK organizes a background’s tiles in VRAM when loading a level.
- Basic knowledge of C: arrays, conditional structures (`if / else if`), and increment operators.
- Basic handling of Aseprite, in particular converting a layer into a tileset, already used in chapter 12.

1. The starting point: `VDP_loadTileData`

This chapter’s initial demo is simple: an urban background with parallax scrolling behind it, and a Green Hill Zone foreground with its characteristic grass, palm trees, and yellow flowers.

The chapter’s initial demo: the Green Hill Zone foreground, with its yellow flowers, over an urban background with parallax scrolling.

This chapter’s central function is `VDP_loadTileData`, which lets you write new tile data directly into VRAM, at any time, without needing to reload the entire level. It takes four parameters: the tile data to load (normally the pointer to a tileset from an image already imported into

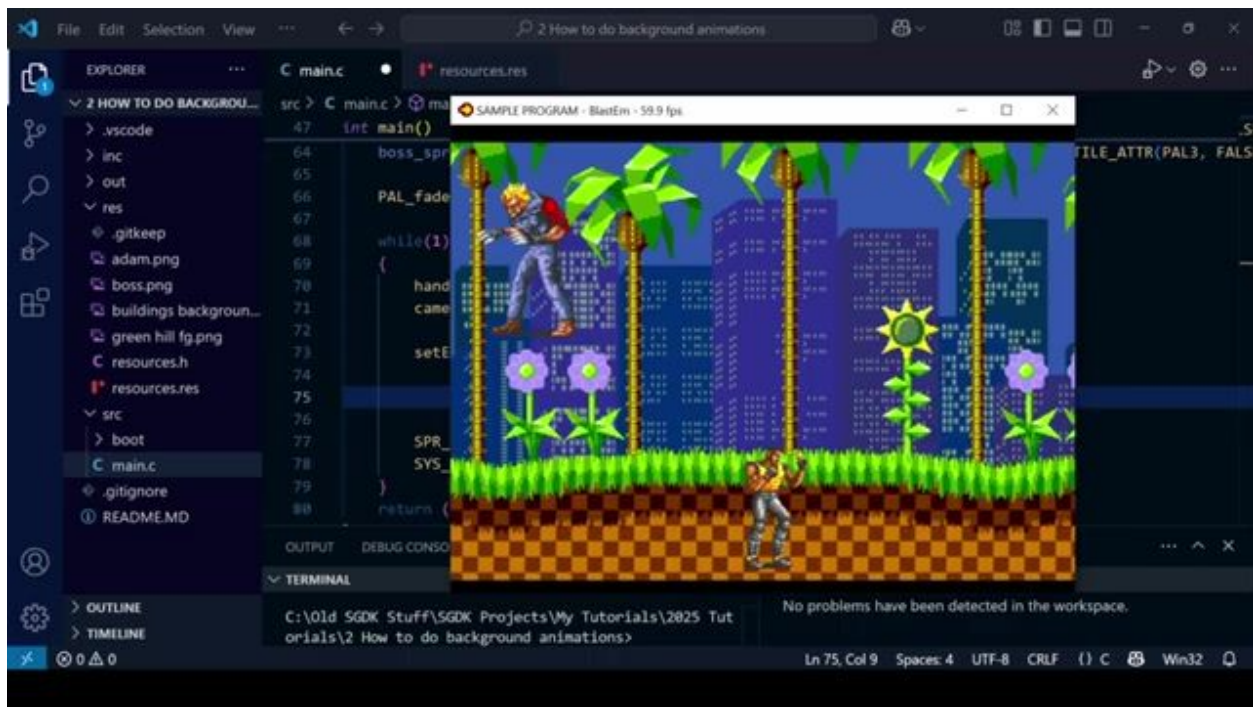


Figure 45: The chapter’s initial demo, with the Green Hill Zone foreground over the urban background with parallax scrolling

resources.res), the VRAM index at which to start writing, the number of tiles to write, and the transfer method, for which this chapter uses DMA_QUEUE.

```
VDP_loadTileData(datos_de_tiles, indice_vram_destino, cantidad_de_tiles,
    ↪ DMA_QUEUE);
```

There is another function with a very similar name, VDP_loadTileSet, which was already used in earlier chapters to load the tileset of a full level map. Its parameters are different, so even if one function is mistaken for the other due to a typo, the compiler flags the problem right away. VDP_loadTileSet is for loading a complete tileset when starting a level; VDP_loadTileData is for replacing specific tiles at any time, even while the game is running.

2. Understanding the VRAM map with the ind variable

Before you can change specific tiles in VRAM, you need to understand how it’s organized. When loading a level’s graphics, SGDK already uses a variable, usually called ind, to know at which VRAM address it should write the background tiles, and then the foreground tiles, without either one overlapping the other. That variable starts out with the value of TILE_USER_INDEX, an SGDK constant marking the first VRAM position free for the program to use (see chapter 2 for details on how VRAM is organized overall).

BlastEm’s VRAM debugger lets you see, tile by tile, what’s loaded at each position. In this chapter’s demo, right before where the background tiles start, there are 16 tiles that at first glance look a bit confusing: they’re actually the first 16 colors of the palette, represented as solid-color blocks, rather than actual graphical tiles.

BlastEm’s VRAM debugger: before the background and foreground tiles, 16 solid-color blocks

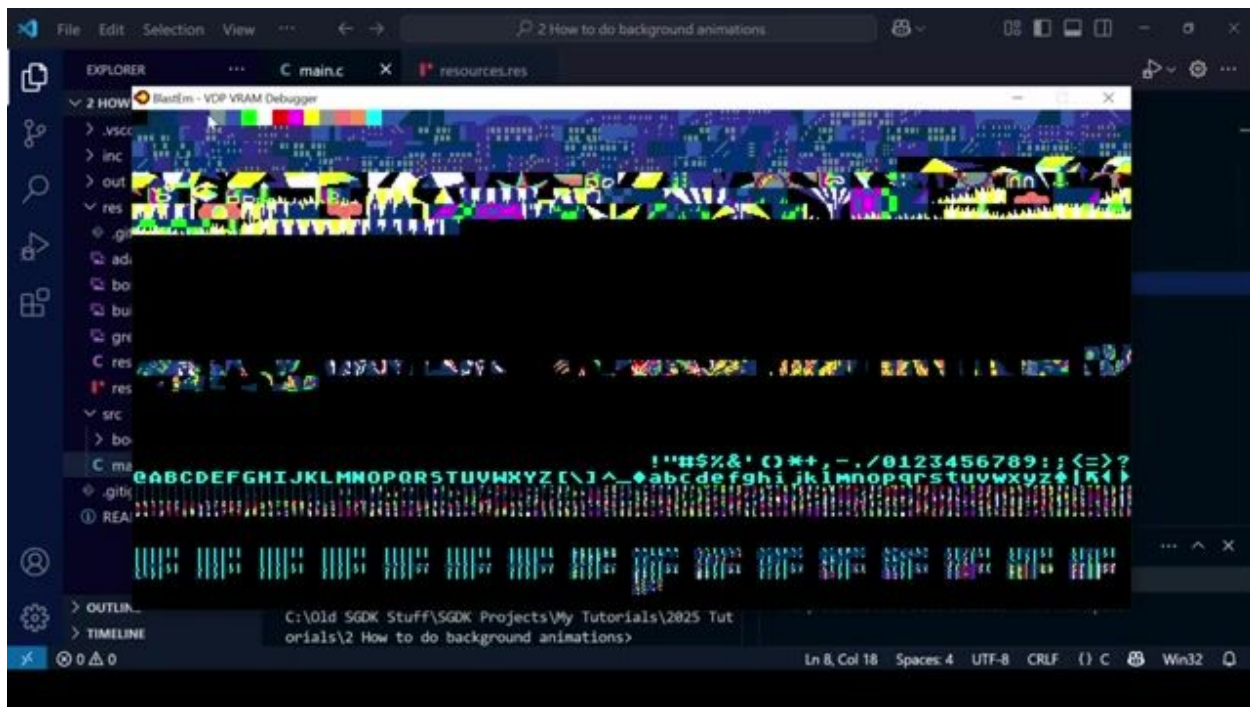


Figure 46: BlastEm’s VRAM debugger, with 16 solid-color blocks before the background and foreground tiles

appear, corresponding to the first 16 colors of the palette.

3. Experimenting with the base index

Those 16 tiles don’t have to be kept as they are: in practice, it’s usually safe to overwrite them with the background’s own tiles, gaining 16 tiles of VRAM that would otherwise go unused.

In general, it’s best not to touch SGDK’s default settings without a good reason, since they’re usually set up that way for a reason. In this particular case, however, overwriting those 16 tiles is safe.

To verify this, 16 is subtracted from the initial value of `ind`, so that the background starts being written 16 positions earlier than usual, right where those color blocks used to be:

```
u16 ind = TILE_USER_INDEX - 16;
```

After compiling, those 16 solid-color tiles disappear from the VRAM debugger: the background now starts exactly where those blocks used to be, without leaving any gap.

It’s also worth trying the opposite case, as a sanity check: instead of subtracting 16, add 100 to the initial value (`u16 ind = TILE_USER_INDEX + 100;`). The result is a gap of 100 unused tiles between the color blocks and the start of the background tiles, immediately visible in the VRAM debugger.

Finally, it’s worth understanding what happens if the adjustment is too aggressive: subtracting 17 instead of 16. In that case, the background starts being written one position earlier than it should, overwriting VRAM data that doesn’t belong to it, related to scrolling and palettes, and the result on screen ends up visibly broken. In this particular case, then, 16 is the exact margin available: that figure shouldn’t be exceeded.

4. Storing the background and foreground addresses in an array

To be able to animate tiles later on, you need to know at which VRAM address the background tiles start and at which address the foreground tiles start, and to have both values available anywhere in the code, not just at the moment the level was loaded. The solution is to store those two addresses in a two-element array, instead of letting the `ind` variable overwrite them as it advances:

```
u16 background_tile_index[2];
```

Within the code that already loads the level, it's enough to add one line right after writing the background tiles, and another after writing the foreground tiles, to copy the current value of `ind` at that moment into the corresponding array position: `background_tile_index[0]` for the background and `background_tile_index[1]` for the foreground.

```
// after loading the background tiles
background_tile_index[0] = ind;

// ... more loading code ...

// after loading the foreground tiles
background_tile_index[1] = ind;
```

5. Testing the VRAM change with a test tile

With those two addresses stored, `VDP_loadTileData` can now be tested with a simple case: a single 8x8-pixel test tile, with an easily recognizable design (concentric squares of different colors) so it's easy to spot where it appears once loaded into VRAM. This tile is imported into `resources.res` as an image, not as a sprite, with the last two import parameters set to `NONE` instead of `BEST` and `ALL` as was done with background images; the reason for this detail is explained later, in section 10, on compression.

The 8x8-pixel test tile, with an easily recognizable concentric-squares design once loaded into VRAM.

Inside the game's main loop (the same while loop already used for the per-tile scrolling from chapter 17), `VDP_loadTileData` is called using the tiles from that test image as the first parameter, `background_tile_index[0]` as the destination VRAM index (the address where the background tiles start), 1 as the number of tiles to copy, and `DMA_QUEUE` as the transfer method:

```
VDP_loadTileData(square_image.tileset->tiles, background_tile_index[0], 1,
↳ DMA_QUEUE);
```

The result replaces the background's first tile (in this case, a sky tile) with the test square, and that change is reflected in every instance of that tile within the background, not just one. Adding 1 to the index affects the second tile in the sequence (a building edge); adding 8, the eighth (part of a roof). The same procedure, but using `background_tile_index[1]` instead of `background_tile_index[0]`, lets you change foreground tiles instead of background ones: adding 0 replaces the fully transparent tile that this layer's tiles start with, and adding 1, the corner of a tree leaf.

The general pattern for replacing any specific tile in a background or foreground is always the same: take the starting address of that layer stored in the array, add

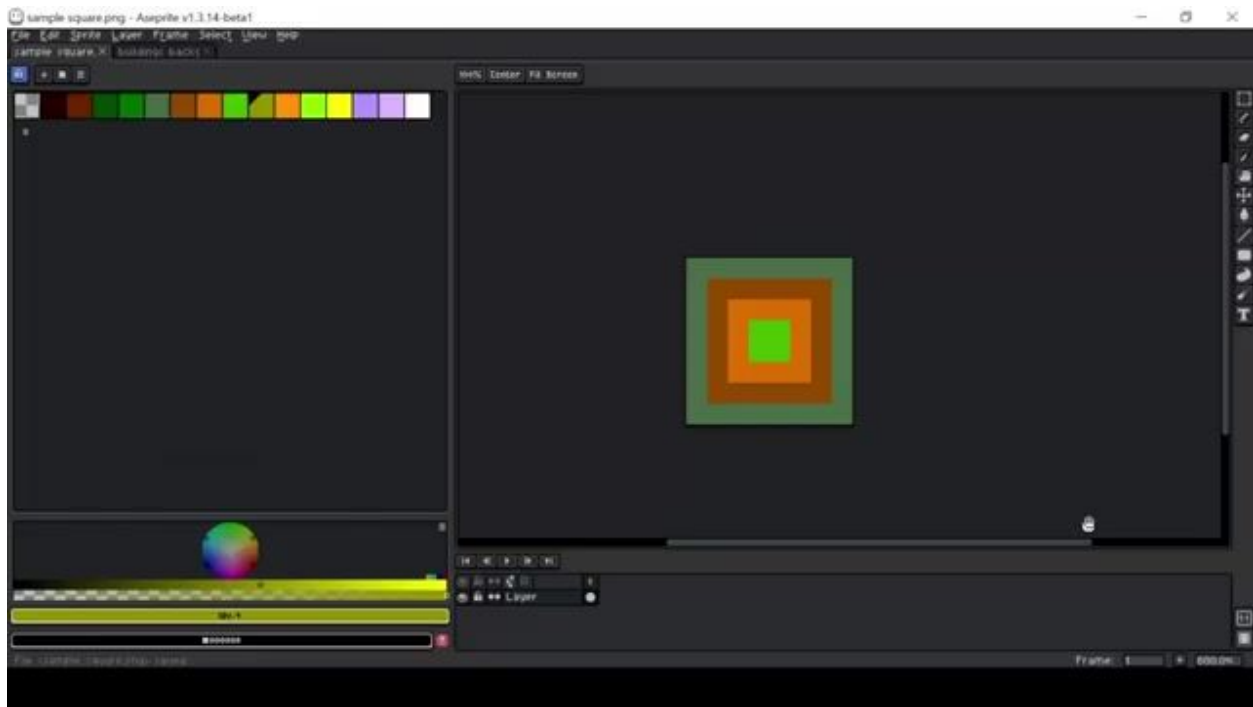


Figure 47: The 8x8 test tile with a concentric-squares design, seen in Aseprite

the position of the tile you want to change within that layer, and pass that sum as the second parameter to `VDP_loadTileData`.

6. Locating the flower tile

The ultimate goal is to animate the flower, not a test square, so you need to find at which VRAM position the flower’s first tile sits within the foreground. Counting tile by tile directly on the VRAM debugger is possible, but slow and error-prone, especially if the flower spans several rows of tiles and each one has to be located separately.

The practical way to solve this is to open the foreground file in Aseprite and turn on *tileset* mode (“Tiled”), the same tool already used in chapter 12 to work with complex level maps: Aseprite scans the image from top to bottom and left to right, in exactly the same order SGDK uses, and assigns an index number to each unique tile it finds.

Aseprite’s tileset configuration dialog, with the base index set to 1 because index 0 is reserved.

The flower is made up of 16 distinct tiles, arranged in four blocks of four tiles each within the tileset, without those blocks sitting next to one another. The flower’s first tile, the one in the top-left corner, turns out to be index number 38 within the foreground’s tileset:

```
VDP_loadTileData(square_image.tileset->tiles, background_tile_index[1] + 38, 1,
    DMA_QUEUE);
```

7. Reorganizing the tileset so the flower is contiguous

Changing the whole flower, tile by tile, using four separate calls to `VDP_loadTileData` (one for each block of four non-contiguous tiles) would work, but it would be awkward to maintain. A

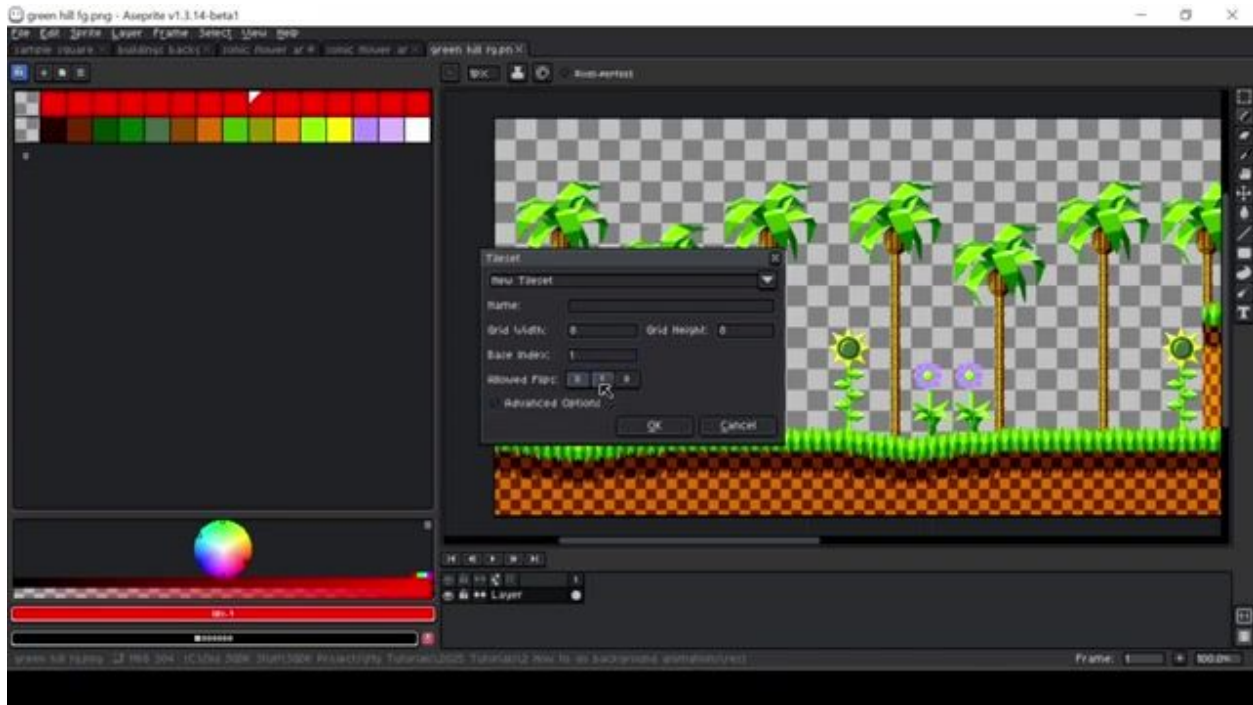


Figure 48: Aseprite’s tileset configuration dialog, with the base index set to 1

much more practical solution is to reorganize the tileset itself in Aseprite, moving the flower’s 16 tiles so they all sit together, one after another, starting at index 1 (index 0 can’t be used because Aseprite reserves it). That way, a single call to the function, with one base index and 16 tiles at once, is enough to change the whole flower.

The flower’s 16 tiles already moved within the Aseprite tileset, grouped and contiguous instead of being spread across four separate blocks.

When moving tiles within the tileset in Aseprite, it’s best to move several at once, as long as they’re contiguous with each other, rather than moving them one at a time. Each time a move is made, a prompt appears asking to confirm the remap (“remap”), which must be accepted so that Aseprite updates the level map’s references to each tile’s new position within the tileset, without altering the map image itself.

Reorganizing the tileset within Aseprite has no effect on SGDK by itself. Up to this point, the project had been letting SGDK generate its own tileset automatically from the foreground image, the same way Aseprite does: scanning the image and discarding duplicate tiles. For the new order to carry over into the game, the already-reorganized tileset must be exported from Aseprite and SGDK made to use it directly, instead of generating its own.

8. Exporting the reorganized tileset and importing it into SGDK

To export the tileset, Aseprite’s “Export Sprite Sheet” option is used, choosing the *Packed* sheet type. As far as SGDK is concerned, there’s no functional difference between exporting it as one long row or as a more compact grid; *Packed* is chosen simply because it’s more convenient to check visually.

Aseprite’s sprite sheet export dialog, with the sheet type set to Packed.

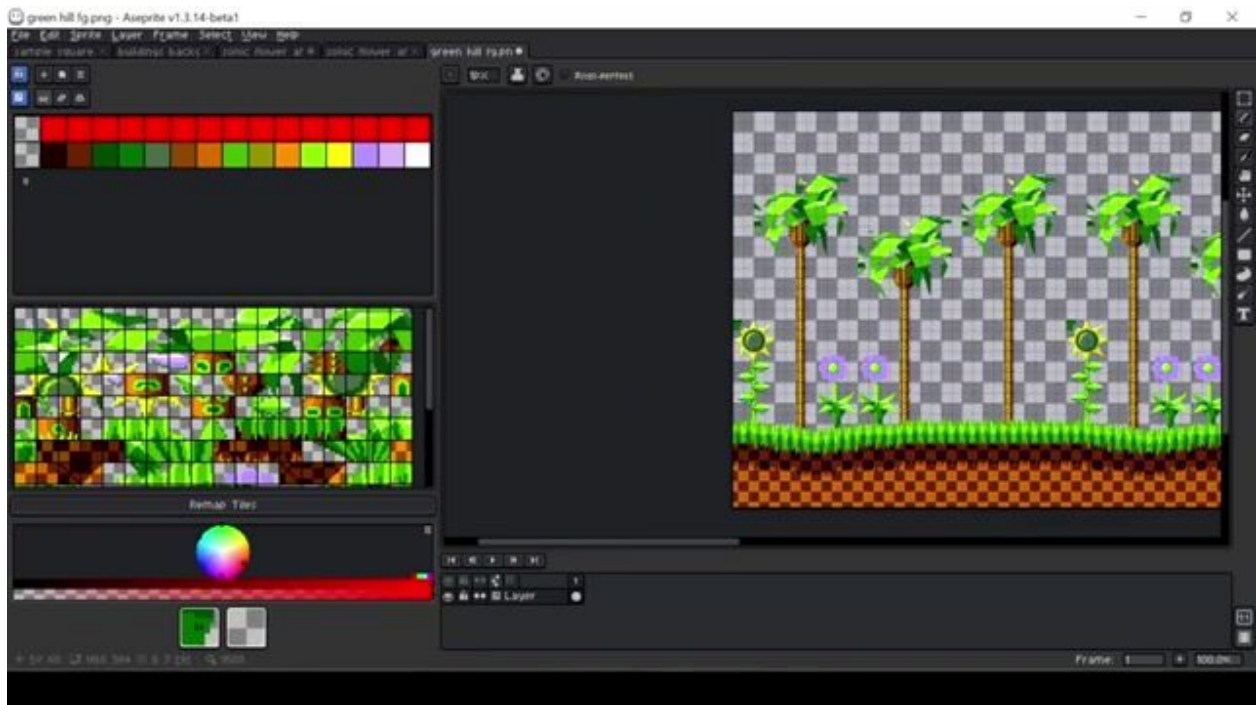


Figure 49: The flower's 16 tiles already moved and grouped within the Aseprite tilesheet

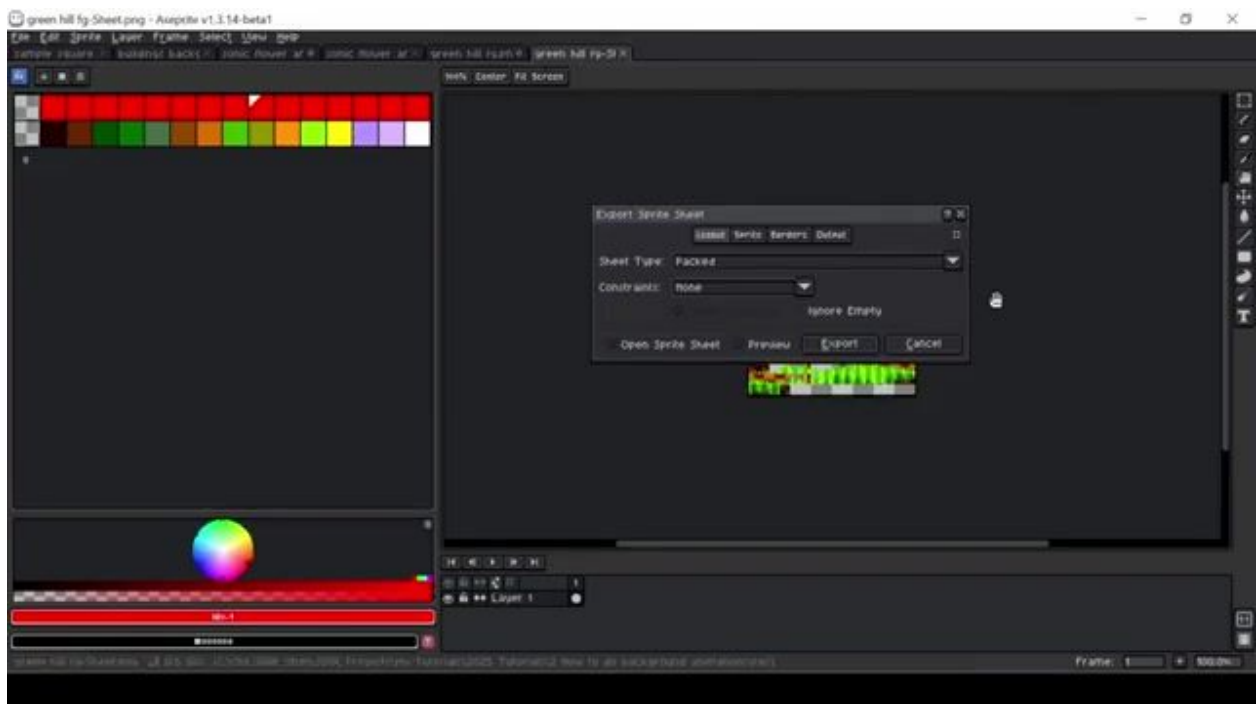


Figure 50: Aseprite's sprite sheet export dialog, with the sheet type set to Packed

Once exported, the result is a new PNG image with the complete tileset, in the same order it ended up reorganized into, with the flower's 16 tiles grouped at the start of the file. Before saving it, just as in earlier chapters, you need to check that the color mode is Indexed and that the palette is the correct one.

In SGDK, this exported file is used as the source for the foreground's tileset instead of generating it automatically from the map; the level map itself doesn't change, only the way its tileset is built. With this change in place, changing the test index from +38 to +1 is enough to point back to the flower's first tile, now at a position that's much simpler to remember and use:

```
VDP_loadTileData(square_image.tileset->tiles, background_tile_index[1] + 1, 1,
    ↪ DMA_QUEUE);
```

9. Animating the flower with a timer

With the flower's 16 tiles now contiguous, animating it requires two images: the one already on the level map (the flower's resting position) and a second image with the flower in its other position in the animation cycle, the one that will be transferred to VRAM while the game runs. Both are imported into `resources.res` as images, not as sprites, with the last two parameters set to `NONE`.

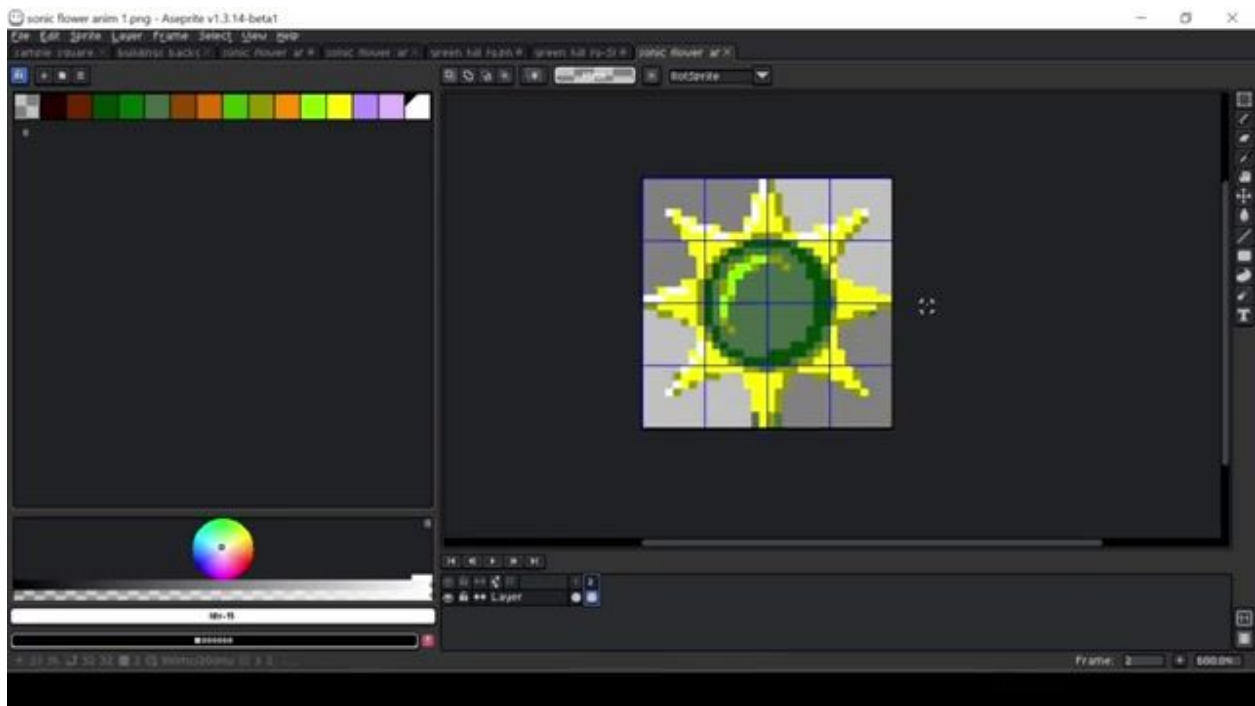


Figure 51: The flower's second image, with its 16 tiles in the rotated position of the animation cycle

The flower's second image, with its 16 tiles in the rotated position of the animation cycle, ready to be imported into `resources.res`.

In the main loop, `VDP_loadTileData` is called with this second image, `background_tile_index[1] + 1` as the index (the flower's contiguous position within the foreground), and 16 as the tile count, since the whole flower is now replaced in one go:

```
VDP_loadTileData(flower1_image.tileset->tiles, background_tile_index[1] + 1, 16,
↪ DMA_QUEUE);
```

With this single call, the flower immediately switches to its second position, with its tips oriented vertically and horizontally instead of diagonally, and the change applies equally to every flower in the level that shares that same tile.

For the animation to alternate on its own, without manual intervention, a timer and a conditional structure are needed. It's best to declare the counter as u16 instead of u8: even though in principle a smaller range would be enough, in practice it's common for a counter that starts small to end up needing more range as the code grows, and it's easy to forget having declared it with too tight a type.

```
u16 timer = 0;

while (TRUE)
{
    // ... rest of the game logic, including the per-tile scrolling from chapter 17
    ↪ ...

    if (timer == 60)
    {
        VDP_loadTileData(flower1_image.tileset->tiles, background_tile_index[1] +
↪ 1, 16, DMA_QUEUE);
    }
    else if (timer == 120)
    {
        VDP_loadTileData(flower0_image.tileset->tiles, background_tile_index[1] +
↪ 1, 16, DMA_QUEUE);
        timer = 0;
    }

    timer++;

    SYS_doVBlankProcess();
}
```

With the timer running inside the main loop, the if / else if pair checks its value: upon reaching 60 frames (one second, at 60 frames per second), the flower switches to its second position; upon reaching 120, it goes back to the first and the timer resets to zero. The timer increment happens at the end of each pass through the loop, after these checks. The result is the flowers alternating position once per second, continuously, exactly as in the original level.

10. The risk of ALL compression with duplicate tiles

There's an easy mistake to make when importing the animation images into `resources.res`: using the ALL compression option, the same one that is indeed appropriate for ordinary background images, instead of NONE. With ALL, SGDK detects that the flower's second image has several tiles that are duplicates of each other (the four corners of the 4x4 tile square are left completely blank, and are therefore identical to one another) and only loads one copy of each unique tile into VRAM, to save space.

The problem is that `VDP_loadTileData` expects to write exactly 16 distinct tiles, one for each

position in the 4x4 block; if SGDK only loaded, say, 12 unique tiles because it dropped the 4 blank duplicates, the function ends up reading tile data that doesn't correspond to what it should, and the result on screen ends up visibly broken: the flower appears deformed, with parts incorrectly repeated.

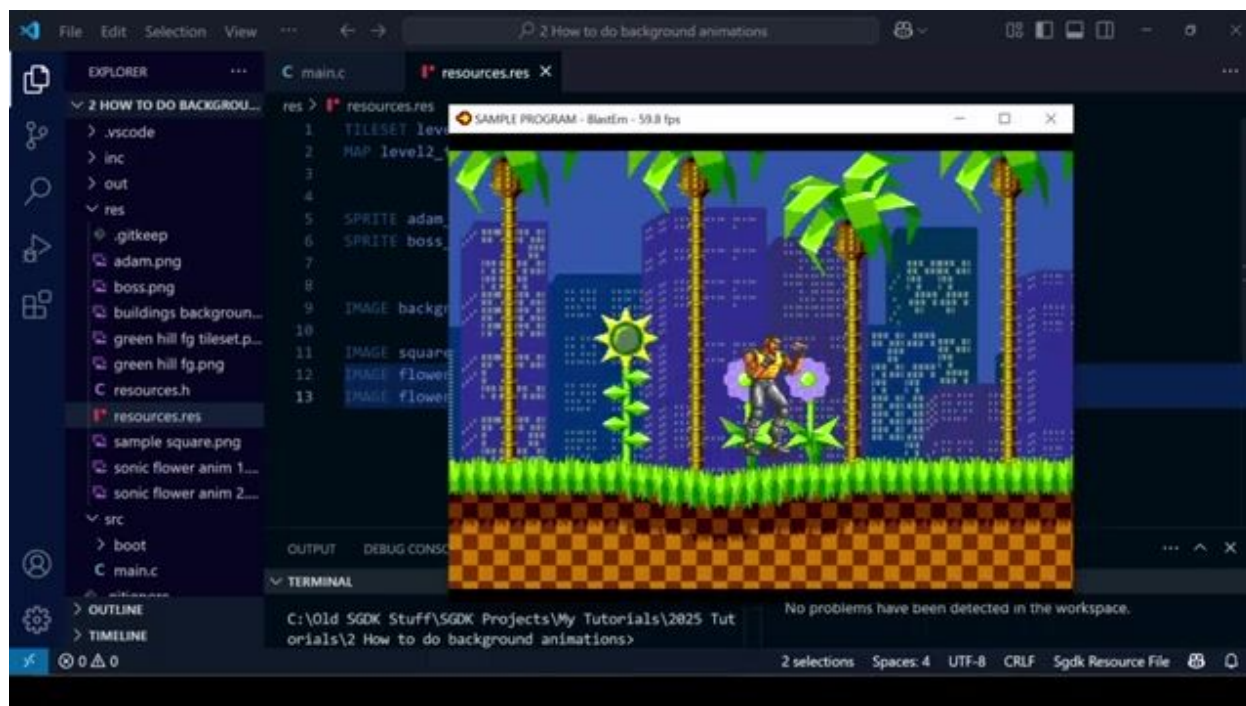


Figure 52: The result of importing the animation image with ALL compression: the flower ends up deformed

The visual result of importing the animation image with ALL compression: the flower appears deformed, with parts incorrectly repeated.

The solution is to use NONE as the compression option for any image that will be used with `VDP_loadTileData` in this kind of animation, so that SGDK loads absolutely every tile of the image into VRAM, including duplicates, and the number of tiles loaded always matches the number the function expects. This is also the reason why, earlier in this chapter, both the test tile and the two flower images were imported with NONE instead of ALL.

11. Other uses of this technique and its limits

Timer-driven animation, like the one in this chapter, is just one of the possible uses of changing VRAM in real time. Another common use case is very large levels, with more graphical detail variety than fits at once in the available VRAM: in that scenario, instead of triggering the change with a timer, it's best to trigger it based on the player's position within the level, discarding tiles that are no longer on screen and loading new tiles in their place, similar to how chapter 17 shifted map tiles based on scrolling. This is possible with this chapter's method, but it requires careful planning of the tileset itself for it to work well; these are VRAM streaming techniques seen in some Mega Drive homebrew projects for handling levels with more graphical variety than fits in VRAM at once.

There is one case, however, where this chapter's technique falls short: individual destructible blocks. When those blocks are placed on the background layer (a common decision

when there are too many of them to handle as sprites, since the Mega Drive can't display a large number of sprites on screen without noticeable flicker), changing a VRAM tile with this chapter's method affects every instance of that tile at once. That means destroying a single block would, by mistake, destroy every identical block in the level. This case needs a different solution, based on additional VRAM management techniques handled per block rather than per shared tile; it's a problem that has already been solved in various Mega Drive homebrew projects, though it falls outside the scope of this chapter.

Review exercises

1. Explain the difference between `VDP_loadTileData` and `VDP_loadTileSet`, and in which situation each one should be used.
2. Why is it best to store the background's and foreground's starting addresses in a dedicated array, instead of relying on whatever final value is left in the `ind` variable after loading the level?
3. Describe how to use Aseprite's tileset mode to find the position of a given tile within a background or a foreground.
4. Why is it worth reorganizing the tileset in Aseprite so that the tiles of an animated element, like this chapter's flower, end up contiguous with each other?
5. Explain in your own words what problem arises from importing an animation image with ALL compression instead of NONE, and why that problem doesn't show up with ordinary background images.
6. Why isn't this chapter's method enough to handle individual destructible blocks placed on the background layer?

Chapter 19 — Complex Level Maps with Aseprite

Chapters 13 through 16 showed how to build the map for a complete Mega Drive level with 16Tile: a free tool, designed specifically for this hardware, that lets you distribute a background's tiles across several palettes and assign them priority relative to sprites. This chapter picks up those same two problems — multiple palettes and tile priority — and adds a new one, generating a level collision array, but solves them with a different tool: Aseprite, the graphics editor already used in Chapter 8 to create this manual's first graphics.

The change of tool is not a whim. 16Tile was born as a project meant to cover a specific need and, although it was made available free of charge to the whole community, it retains the typical limitations of a purpose-built program without much development time behind it: it lacks copy and paste, and also undo, inside the map editor. Building a large level tile by tile, without being able to repeat a piece already made or revert a mistake, turns what should be mainly creative work into something tedious, and it also discourages experimenting with the level design, because any change of mind means manually redoing everything already built.

Aseprite, being a commercial pixel art editor with years of continuous development, has none of those shortcomings: it offers copy, paste, and undo out of the box, and, since version 1.3 — released in early 2024 — also a native tileset tool, originally designed for indie PC games. What Aseprite doesn't include out of the box are Mega Drive-specific features: multiple palettes within a single tileset, tile priority, and level collision arrays. That gap can be filled with scripts written in Lua, the scripting language built into Aseprite. Sections 6 and 7 of this chapter describe how a script of this kind would behave, in practice, for assigning priority and another for generating the collision array, understood as a possible avenue of automation within the editor and not as a piece of software with a single guaranteed source.

This chapter covers, in order, how to set up an Aseprite canvas as a tileset, how to paint it while taking advantage of the Mega Drive's four palettes within a single map, and how to solve — with the help of Lua scripts — assigning priority to certain tiles and generating the level's collision array. By the end, readers will be able to judge, based on the complexity of their own project, whether it's worth sticking with 16Tile or switching to this Aseprite-based workflow.

Prerequisites

Before continuing, it helps to have:

- Aseprite 1.3 or a later version installed: the native tileset tool used in this chapter does not exist in versions 1.2 or earlier.
- Having read Chapters 13 through 16, on building level maps and using multiple palettes and tile priority with 16Tile: this chapter assumes those same concepts are already

- known, and only changes the tool used to apply them.
- Basic familiarity with Aseprite, covered in Chapter 8: layers, the color palette, and the difference between indexed color mode and RGB mode.
- Some familiarity with SGDK’s resource compiler (rescomp), in order to interpret the error message that appears in the section on palette mixing.

You don’t need to know how to program in Lua: the scripts mentioned in this chapter are described as finished tools, without any need to write them or understand them line by line.

1. From 16Tile to Aseprite: what changes and what doesn’t

The underlying problem with 16Tile isn’t that the final result is worse: ultimately, both tools produce the same kind of file, an indexed image that SGDK converts into a tileset, so the result is equivalent. The problem lies in the process of getting to that result. In a map built tile by tile, with no way to copy or undo, repeating the same structure — for example, a building that appears several times throughout a level — requires rebuilding it from scratch every time, and a misplaced tile that isn’t caught in time can mean losing a good part of an entire session’s work.



Figure 53: A level map in 16Tile, built tile by tile with no way to copy or undo

A level map assembled in 16Tile: each building was built tile by tile, with no way to copy or move it as a block.

Two factors explain why it makes sense to solve this same problem with Aseprite. The first is the release of version 1.3, which introduced a tileset tool designed to separate a map’s logic (which tile goes in which cell) from a tileset’s logic (what each unique tile looks like), something that until then only specialized tools like 16Tile or Tiled offered. The second is the existence of Lua scripts capable of handling, within Aseprite, the two functions it lacks for working with the Mega Drive: assigning priority to a tile and generating a level collision array from a finished map.

2. Preparing the canvas and converting it into a tileset

The first step is to create a new file in Aseprite in RGB color mode, with a transparent background. Both the width and the height must be multiples of 16 pixels, not just of 8.

Attention: this requirement only makes sense once you reach section 7 of this chapter: the collision array is generated by copying this same map into a reference tileset that works in 16×16 pixel blocks. If the canvas dimensions aren't exact multiples of 16, that copy ends up misaligned with the collision grid, and the final result won't match the actual map.

With the canvas created, it's a good idea to rename the default layer — for example, to foreground — to distinguish it later from the other layers that will be added. The next step is to convert that ordinary layer into a tileset layer: right-click on the layer, *Convert to*, and *Tileset*.

That action opens a dialog box with several options. The grid's width and height should both be set to 8 pixels, since that's the size of a Mega Drive tile. The *base index* decides whether the first tile in the tileset is counted as number 0 or number 1; it isn't entirely clear which of the two conventions Aseprite uses internally in this dialog, so it's best to leave it at its default value, verify the actual behavior in practice before relying on it, and in any case not base any step of the subsequent workflow on this number. Lastly, the two *Allowed Flips* checkboxes — horizontal and vertical mirroring — tell Aseprite that, when counting unique tiles, it should treat two tiles that only differ by being mirrored as the same tile.

Tip: this mirroring setting only affects Aseprite's internal count of unique tiles while drawing. When importing the final tileset into SGDK, rescomp does its own independent mirroring analysis, without taking this checkbox into account.

3. Organizing the palette in index mode: four rows for the four palettes

The Mega Drive has four hardware palettes — PAL0 through PAL3 — of 16 colors each. Aseprite's palette panel, on the other hand, can hold many more than 16 colors at once, so it's worth taking advantage of that and organizing all 64 possible colors (4 rows of 16) within a single panel: from then on, each row represents one of the four hardware palettes.

While defining this organization, it helps to assign each row a base color quite different from the others — for example, red for the first row, green for the second, blue for the third, and yellow for the fourth — without worrying yet about the actual colors that will be used in the final artwork. This trick is just a temporary visual aid: it lets you spot at a glance where one palette ends and the next begins, and it gets replaced later by the final colors.

The next step is to change the file's color mode from RGB to Indexed (*Sprite*, *Color Mode*, *Indexed*). The difference isn't merely cosmetic: in RGB mode, each pixel stores a pure color value, with no reference to which palette row it came from; in indexed mode, on the other hand, each pixel stores the number of the palette slot it uses. That distinction is what makes it possible, later on, to reliably detect whether a tile mixes colors from two different palettes, even when those two palettes share a visually identical color — black, for example — that would look indistinguishable at a glance.

Finally, it's worth creating a second file, separate from the tileset, in RGB mode, with a reference strip of the tones the Mega Drive reproduces accurately (an informal way to refer to these tones is to call them "step-32" colors). Rather than typing the tileset's RGB values by hand and risking that they don't match any real console tone, it's better to copy the exact colors from that reference file and paste them into the tileset's indexed palette.

Tip: even if the exact colors aren't copied from a reference file, SGDK automatically adjusts any RGB value to the closest tone the console can reproduce when importing the resource. The reference file isn't mandatory: it's a way to preview, accurately, within Aseprite, what the colors will actually look like in the game, instead of discovering it only after compiling.

4. Drawing the tileset: the two tile-creation modes

When painting onto a tileset layer, Aseprite offers, in a dropdown menu specific to this type of layer, different tile-creation modes that change what happens each time you draw over a tile that already exists on the map.

In one of those modes, every time a tile is modified, a new, unique tile is automatically created within the tileset, without altering any other cell on the map that uses the original tile. This is the recommended mode for adding new pieces to the map.

The other mode works the opposite way: if you modify a tile that's already used in several cells of the map, the change is applied to all those cells at once, because you're actually editing the tile's shared definition rather than a new copy. This option is especially useful once the map is practically finished and all that's needed are small color tweaks to tiles that repeat many times.

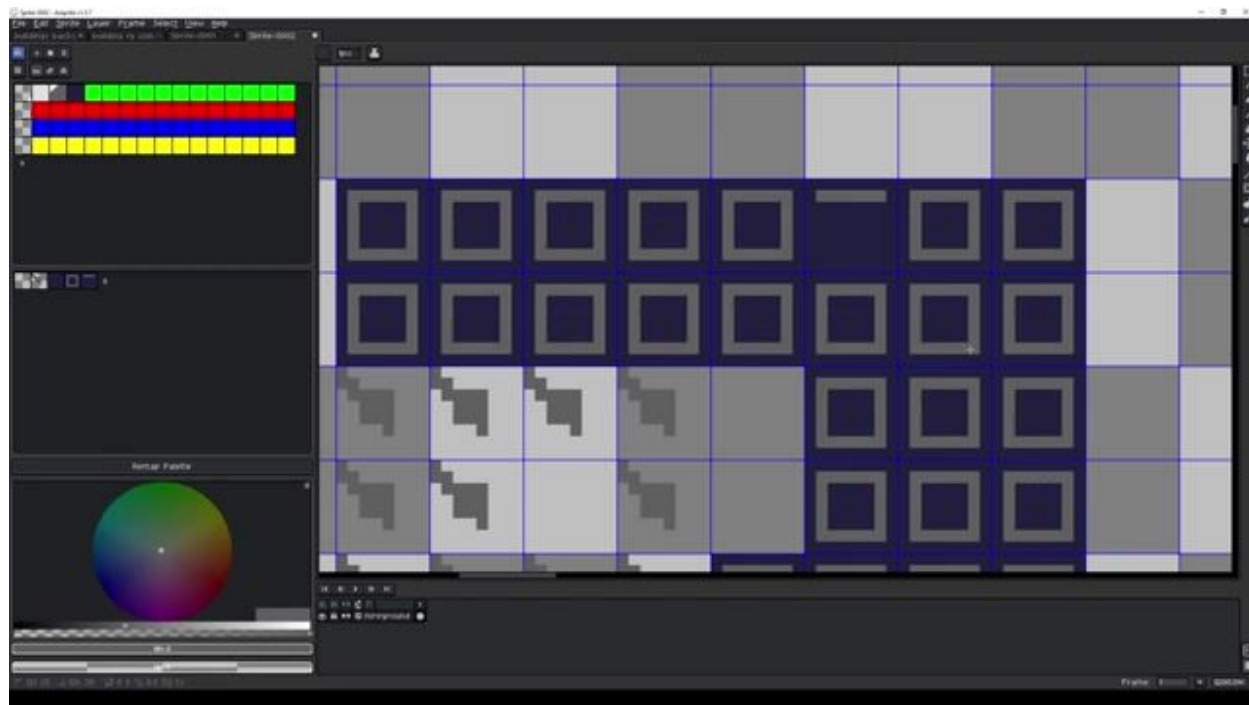


Figure 54: With the change-propagating mode active, modifying an existing tile updates every map cell that uses it at the same time

With the change-propagating option active, modifying an existing tile updates every map cell that uses it at the same time.

Aseprite also compares each new tile against the ones that already exist, including their horizontally and vertically mirrored versions — as long as the *Allowed Flips* checkbox was enabled when converting the layer. If it finds a match, it doesn't create an additional tile: it reuses the already-known tile and marks the map cell with a mirror indicator. Holding down the Ctrl

key over a tile in the panel reveals these marks: an X next to the index number indicates a horizontal flip, a Y indicates a vertical flip, and both letters together indicate the tile is flipped in both directions.

Tip: if Aseprite's gray background checkerboard makes it hard to distinguish the gray tones of the artwork itself, it can be changed from *Edit, Preferences, Background*, choosing a different color combination for that background grid.

5. Using several palettes within a single tileset

With the tileset already organized into four palette rows, the next step is to actually distribute the map's tiles across those four palettes, the same way it was already done with simple backgrounds in earlier chapters, but now on a complete map built by hand in Aseprite. As an example of how to allocate colors, consider an urban background that needs 6 colors — including the screen's overall background color — which leaves 10 free colors in that same palette to reuse in the foreground.

Reusing a color between the background and the foreground isn't a problem in itself — in fact, it's what allows you to make better use of each palette's 16 colors — as long as that reused color doesn't cause two different elements to visually blend together by ending up adjacent and the same tone.

Attention: the rule that truly admits no exceptions is a different one: within a single 8×8 pixel tile there can't be colors coming from two different palette rows. A tile's palette attribute is a single value for the whole tile; if its pixels reference colors from two different rows, the hardware will only honor one of those two palettes when drawing it, and the tile will look wrong on the console even though it looks correct in the editor.

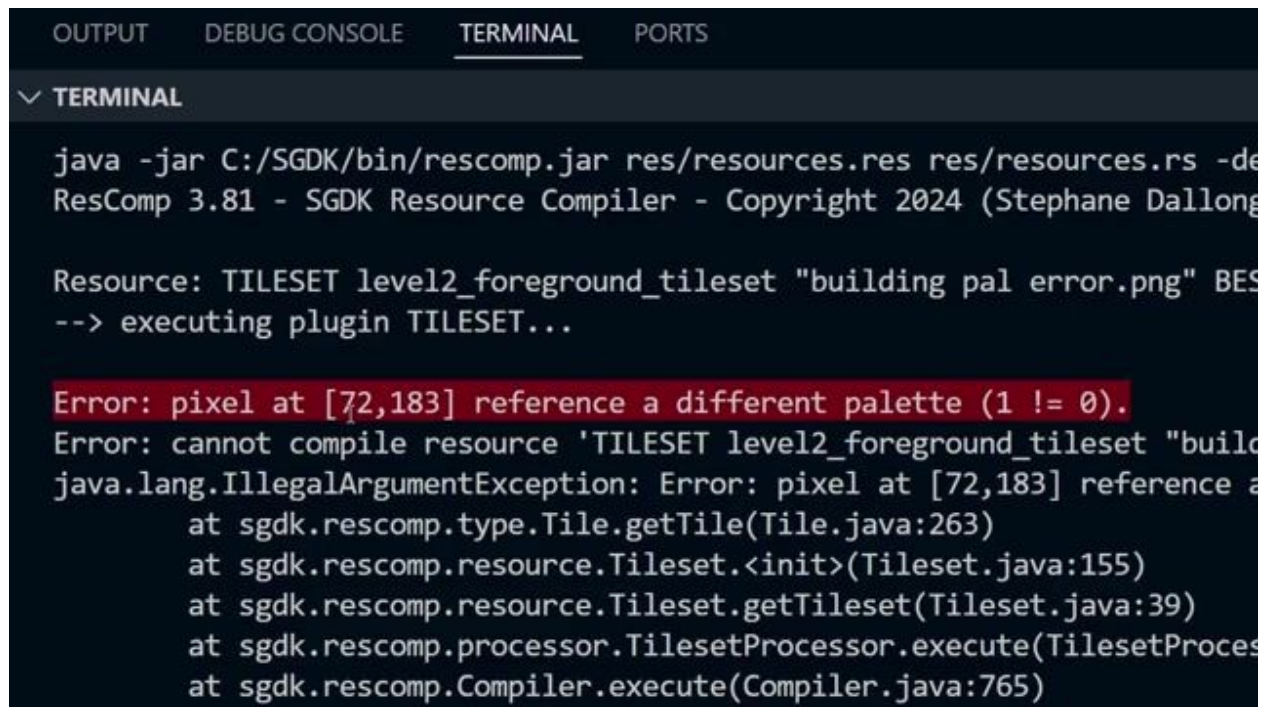
This mistake is particularly easy to make, and hard to spot at a glance, when two palettes share an identical color: for example, if both PAL0 and PAL1 include a pure black, you could accidentally paint a pixel with PAL1's black inside a tile that should exclusively use PAL0, and the result looks exactly the same on screen while you're drawing it. A simple way to audit this is to temporarily fill every color in one palette row with the same eye-catching color — all green for PAL0, for example — and visually scan the whole map for pixels that haven't changed color: any pixel that keeps its original tone reveals that it actually belongs to a different palette row.

If this mistake isn't caught in time and you try to compile the project anyway, *rescomp* detects it on its own while processing the tileset and halts compilation with a specific error message that gives the exact pixel coordinates of the conflicting pixel within the image.

rescomp's error message points to the exact pixel coordinates of the conflicting pixel within the image.

Tip: that error message, cryptic as it may look at first glance, saves a lot of search time: just open the tileset image and locate those exact coordinates to find the misassigned pixel, instead of eyeballing the entire map.

In a typical split between background, foreground, and sprites, palette 0 can be used for both the background and part of the foreground, palette 1 reserved exclusively for the rest of the foreground, and palettes 2 and 3 assigned to different sprite groups — the player's and the enemies', for example. This split highlights an idea already introduced in earlier chapters: the Mega Drive's four palettes are a resource shared between backgrounds and sprites, not a resource exclusive to each one separately.



```
OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

▼ TERMINAL

java -jar C:/SGDK/bin/rescomp.jar res/resources.res res/resources.rs -de
ResComp 3.81 - SGDK Resource Compiler - Copyright 2024 (Stephane Dallong

Resource: TILESET level2_foreground_tileset "building pal error.png" BES
--> executing plugin TILESET...

Error: pixel at [72,183] reference a different palette (1 != 0).
Error: cannot compile resource 'TILESET level2_foreground_tileset "build
java.lang.IllegalArgumentException: Error: pixel at [72,183] reference a
    at sgdk.rescomp.type.Tile.getTile(Tile.java:263)
    at sgdk.rescomp.resource.Tileset.<init>(Tileset.java:155)
    at sgdk.rescomp.resource.Tileset.getTileset(Tileset.java:39)
    at sgdk.rescomp.processor.TilesetProcessor.execute(TilesetProces
    at sgdk.rescomp.Compiler.execute(Compiler.java:765)
```

Figure 55: rescomp error message when it detects a tile with pixels from two different palette rows

6. Assigning tile priority with a Lua script

Chapters 15 and 16 showed what tile priority is — the bit that makes a background tile draw in front of sprites instead of behind them — and how to assign it with 16Tile. Aseprite has no built-in equivalent concept to that priority. That function can be covered by a Lua script that goes through the tiles marked as priority and applies a recognizable marker to them within the image itself.

The mechanism such a script would use requires extending the palette: to the four rows of 16 colors that already represent the four hardware palettes, you need to add eight more rows of 16 colors each, always in that order and always present, even if the project ends up using only one or two of the four real palettes. Of those eight new rows, the first four are filler — their color doesn't matter, because they won't be used to paint anything — and the last four are the priority rows, which by convention (and only as Aseprite's default color when creating a new row) usually appear in pink.

In depth: it isn't possible to confirm with certainty, from the code, exactly why twelve palette rows are needed, but the observable behavior is consistent with a reasonable explanation: for each tile marked as priority, the script needs some way to leave a record of that mark within the final image itself, without adding any data outside the graphics file. If the four priority rows reserve a range of color indices that's never used for painting directly, the script can use that range as a recognizable signal within the tileset itself. This reading is a reasonable inference from observed behavior, not a confirmation of how the script actually works internally.

With that palette extension already in place, the workflow for marking tiles as priority, assuming you have a Lua script installed in Aseprite's scripts folder (*File, Scripts, Open Scripts Folder*, then *File, Scripts, Rescan Scripts Folder* so it shows up in the list), would be as follows:

1. Create a new layer above the tileset layer (*New, New Layer*) and rename it, for example, to priority.
2. In the tileset layer, within the color index panel (not the tiles panel), select the tiles that should have high priority, copy them with Ctrl+C, and paste them with Ctrl+V onto the new priority layer: this copies those same tiles, in the same map position, to the new layer.
3. Save a backup copy of the file in .aseprite format — with a name like foreground_master, for example — before continuing: the next step is irreversible, and without this backup there would be no way to change which tiles are priority later on.
4. Convert the tileset layer back into an ordinary layer (right-click, *Convert to Layer*), because the script would work on ordinary pixels rather than on the internal structure of a tileset layer.
5. With the priority layer selected, run the priority script from the *File, Scripts* menu.

Attention: on large maps, a script of this kind can take several minutes to process all the tiles. It's best not to touch anything or close Aseprite during that time, even if the operating system warns that the process isn't responding: it's still working, and all you need to do is wait.

Once the process finishes, the final steps are: right-click on the priority layer and choose *Merge Down* to combine it with the original tileset layer; save the result as a new PNG file, separate from the .aseprite backup copy; and import that PNG into SGDK with no automatic tile attribute assignment, the same way it was already done with multiple palettes in earlier chapters, because both each tile's palette and its priority have already been determined by its color within the image itself.

Tip: if the priority script is accidentally run on the original tileset layer instead of on the priority layer, the effect gets inverted: the tiles that had been marked as priority become the only ones without priority, and every other tile on the map becomes priority. It's worth keeping this behavior in mind: it can actually be faster to invert it on purpose when only a few tiles need to end up without priority, rather than manually marking most of the map.

7. Generating the collision array with a second script

The same way 16Tile lets you export a level collision array, that same result can be automated within Aseprite with a second Lua script, using a separate reference tileset made of 16×16 pixel tiles instead of 8×8, because this workflow's collision array operates in 16×16 blocks.

That reference tileset is set up once and reused across all projects: in RGB color mode, it contains one tile for each type of collision needed — for example, an S for a solid tile, a P for a platform you can pass through from below but not from above, and additional variants for slopes — and, in that same file's palette, a distinct flat color for each of those tiles.

Attention: the order in which the tiles appear within the reference tileset and the order in which the colors appear within the palette must match exactly: index 0 must be the transparent color and the transparent tile, index 1 the color and tile for the S, index 2 the color and tile for the P, and so on. A script of this kind would work by reading, for each 16×16 pixel block of the collision map, only the color of its top-left pixel, and writing the index number of that color into the output array; if the order of tiles and colors doesn't match, the number that gets written won't correspond to the collision type that was actually drawn.

With the reference tileset already prepared, the procedure for generating the collision array of a specific map would be as follows:

1. Open the finished map (the tileset layer with the background and foreground result), select everything within the tiles panel, and copy it with Ctrl+A and Ctrl+C.
2. Open the collision reference tileset and adjust its canvas size (the *C* button, or *Sprite Size*) so it matches exactly the pixel dimensions of the copied map.
3. Paste the copy (Ctrl+V) onto a new layer, as a visual guide, and paint onto a separate collision layer the S, P, or slope tiles that correspond to each area of the level, using that background copy as a guide to know where there's ground, platforms, or elevation changes.
4. Save a backup copy of the file before continuing, for the same reason as in the previous section: the script step would be irreversible.
5. Delete the guide layer (the copy of the original map), leaving only the collision layer visible, and change the file's color mode to Indexed.
6. Run the collision script from the *File, Scripts* menu, and when prompted, provide an output file name with the .txt extension.

The resulting text file would contain a declaration ready to paste directly into the code: a `const u8` array with one number for each 16×16 block of the map, and, at the end, a constant with the total length of that array, already calculated.

Tip: it's worth checking whether there's a trailing comma after the array's last number before pasting it into the code: the compiler generally tolerates it anyway, but it's a two-second check.

The result would be equivalent, in form and in index logic, to a collision array built by hand with 16Tile: a linear array of unsigned 8-bit integers, with a fixed row width and one value per collision tile. The logic for converting pixel coordinates into indices of that array, and for interpreting each numeric value according to the collision type it represents, is the same regardless of whether the array was built by hand or generated with a script.

8. When this workflow is worth it compared to 16Tile

Adopting this workflow has a real organizational cost: you have to set up the palette with a specific row structure from the start, save a backup copy before every irreversible step, and depend on Lua scripts that aren't part of either Aseprite or SGDK. In exchange, you get back everything 16Tile doesn't offer: copy and paste, undo, and generally, a more comfortable editor for iterating on a level's design.

This workflow pays off more the bigger and more changeable the level is: a project with many large levels, or with a design that's still in a trial-and-error stage, benefits from being able to copy repeated structures and undo mistakes without redoing work. For simpler projects, there's no need to feel obligated to add this complexity if 16Tile is enough for the size of the level being built: it's better to save the more elaborate techniques for when the project actually needs them, rather than adopting them ahead of time just because they're available.

Review exercises

1. List at least two concrete limitations of 16Tile that motivate considering a switch to Aseprite, and explain why those limitations don't affect a commercial pixel art editor in the same way.
2. Explain why a tileset canvas must be created with dimensions that are multiples of 16, not just of 8, taking into account how that same image is used in the section on collision.
3. Describe the difference between Aseprite's two tile-creation modes, and what effect each one has when modifying a tile that's already used several times on the map.

4. Why is it necessary to work in indexed color mode, rather than RGB mode, in order to reliably detect an accidental mix of palettes within a single tile?
5. Explain, in your own words, why a tile priority script would need more palette rows than the four already used by normal tiles, and what special precaution should be taken before running it.

Chapter 20 — Converting Many-Color Images to the Mega Drive Palette

Reducing a high-color-range illustration to the console's four 16-color palettes through tile-based quantization

All the previous chapters in this part of the manual started from a comfortable premise: the source graphic was already designed with the Mega Drive in mind, drawn by hand in an editor with an indexed color mode, with the 16-color-per-palette limit taken into account from the very first stroke. This chapter closes out the part of the manual devoted to backgrounds, tiles, palettes, and scrolling by tackling the reverse problem — one that's considerably more common than it might seem: what to do when the starting image wasn't designed for the console at all. A digital illustration, a render, or a screenshot from a modern game can have thousands of distinct colors, far beyond the 61 the Mega Drive can display on screen at once. Manually converting such an image, color by color, is possible but extremely slow; this chapter presents an automated technique that reduces that work to a process of minutes, along with its limits, which are worth knowing before relying on it for any kind of graphic.

The example used throughout the chapter is a Sonic illustration with a color range far beyond what the console can reproduce, treated as if it were the artwork for a title screen or a cover.

Prerequisites

Before continuing, you should be clear on:

- The concept of an indexed palette, the limit of four 16-color palettes (PAL0 through PAL3), and the idea that a tile doesn't store color but rather an index into a palette, covered in Chapter 2.
- Working with a graphics editor in indexed color mode — Aseprite or an equivalent — including the use of a reference grid and an eyedropper-type tool to inspect the color index of a specific pixel, covered in Chapter 8.

1. The problem: from an arbitrary color range to four palettes per tile

The already-familiar restriction on the Mega Drive is that any graphic on screen is built from 8×8-pixel tiles, and each tile is assigned a single palette out of the four available. When a graphic is drawn from scratch with the console in mind, that restriction is naturally respected, tile by tile, as the drawing progresses. The problem arises when the image already exists,

comes from another source, and has far more colors than any combination of four palettes can represent.

Reducing that image by hand — choosing which 61 colors to keep and manually remapping every pixel — is feasible for a small illustration, but becomes impractical as the size and detail of the graphic grow. The automated alternative is to use a color quantization algorithm that, in addition to reducing the total number of colors, understands the Mega Drive's particular constraint: that the reduction isn't done over the whole image as if it were a single palette, but tile by tile, letting each 8×8 block use whichever of the four available palettes suits it best. There are community tools, such as tile-based palette quantization generators, built specifically for this problem; they usually run directly in the browser, with no installation, and accept a plain PNG image as input. Since these are free projects maintained by hobbyists, their exact name, address, or availability may change over time, so it's worth searching for whatever option is current when you need it rather than relying on a fixed link; what matters here is understanding the parameters this kind of tool exposes and what each one means, since they're common to any implementation of the same problem.

2. Preparing the source image: canvas size, not image size

The first step, before touching any quantization tool, is to adjust the resolution of the source image to the console's screen size. In the graphics editor this is done through the **canvas size** option, not the image size or sprite size option: the latter rescales the content, distorting or blurring the original drawing, whereas a canvas size change simply crops or adds space around it without touching the existing pixels.

The width is set to 320 pixels. If the aspect ratio is locked, the resulting height would be 240, appropriate for a PAL project; for an NTSC project, that height should be manually adjusted to 224. Since this is a canvas change rather than a scaling operation, the result crops a strip off the original image (typically from the top and bottom) instead of distorting it, which is usually preferable to losing proportions in a graphic that's later going to be shown at actual size.

Adjusting the canvas size to 320 pixels wide, with the aspect ratio still locked before manually correcting the height.

The color mode of the image at this point doesn't matter much, because the quantization tool used next accepts both RGB images and already-indexed ones. It's enough to save the cropped image as a regular PNG file.

3. Tile quantization parameters

Once the prepared image is loaded into the quantization tool, there's a set of parameters that define how the available color is going to be distributed. Although the exact name of each field can vary depending on the specific implementation, the concepts behind them are always the same and apply to any equivalent tool:

- **Tile size:** left at 8×8, which is the Mega Drive VDP's native tile size. Any other value wouldn't make sense for this console.
- **Number of palettes:** 1 to 4 on the Mega Drive. When the image is going to be used in isolation, such as a title screen with no other graphics sharing a palette with it, it's worth taking advantage of all four full palettes to maximize the amount of available color. If the image is going to coexist with other graphics (enemy sprites, item cards, etc.) that need their own palettes, fewer should be reserved — two, for example — leaving the rest for those other elements.

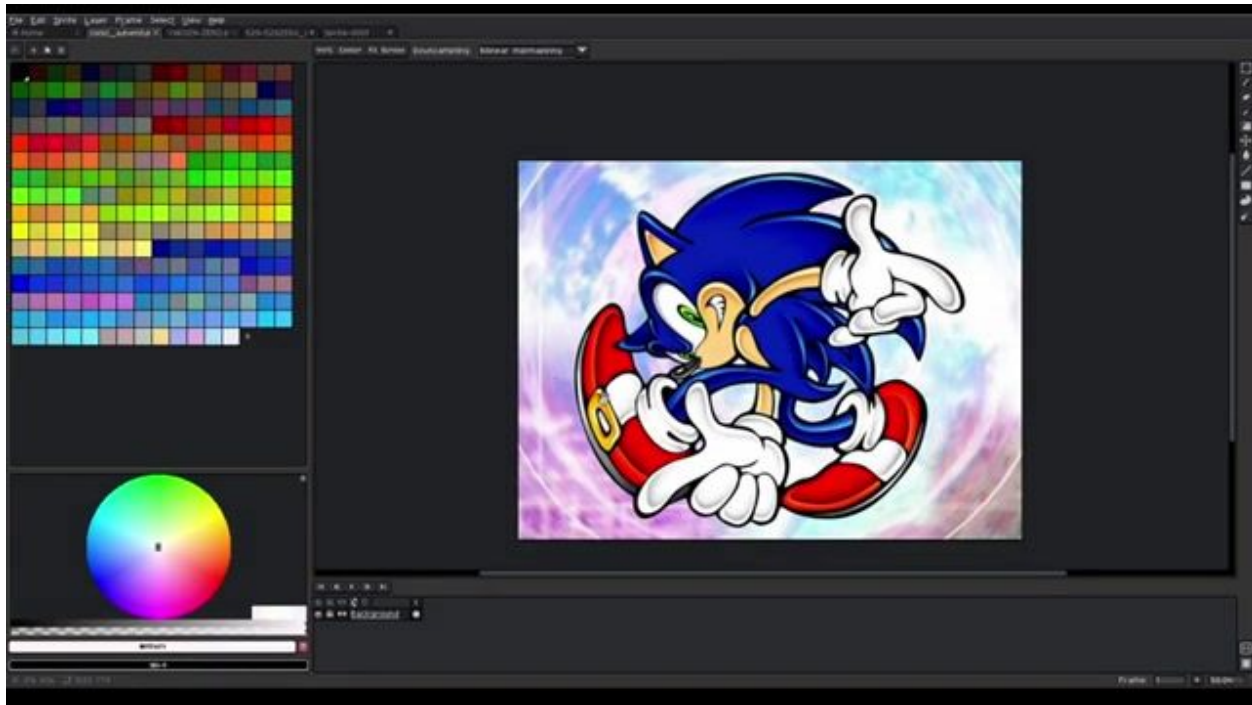


Figure 56: Adjusting the canvas size to 320 px wide

- **Colors per palette:** 16, the maximum the hardware supports.
- **Bits per channel:** 3, the correct value for the Mega Drive, which reproduces 8 intensity levels per RGB channel (the same 9 total bits per color seen in Chapter 2). Other 8-bit and 16-bit systems use different values in this same type of tool — more bits per channel on consoles with greater color depth, fewer on more limited ones — so it's worth making sure this is specifically configured for the Mega Drive and not for another system.
- **Index-0 color behavior:** determines what happens to the first color of each of the four palettes. If the graphic is a sprite or any element with transparency, it's best to use the transparent color option (usually represented as magenta), making sure the background of the original image is also magenta. If, instead, it's an isolated background image with no need for transparency, it's best to use the shared color option set to black: that way index 0 matches across all four palettes, that black can be reused within the drawing itself, and in practice you end up with 61 usable colors instead of 60.
- **Sampled pixel fraction:** controls how long the algorithm takes to produce the result. Higher values are slower but, in theory, produce a more accurate quantization; in practice, the difference between a low value (0.1) and the maximum (1.0) is usually much smaller than expected, so it's best to start with a low value and raise it only if the result isn't convincing.

The parameters already adjusted for Mega Drive: 4 palettes, 16 colors per palette, and 3 bits per channel.

Besides these parameters, this kind of tool usually offers **dithering** options to smooth the transition between colors when their count is reduced. It's worth turning these on when working with an isolated image, although the exact difference between the variants typically offered — usually described as a “slow” mode and a “fast” mode — is best explored by trying each one directly on the image itself, rather than memorizing a fixed rule: the result depends too much on the particular graphic content to generalize. What is fairly consistent across implementations is that a lower dithering weight produces a subtler effect and a higher one

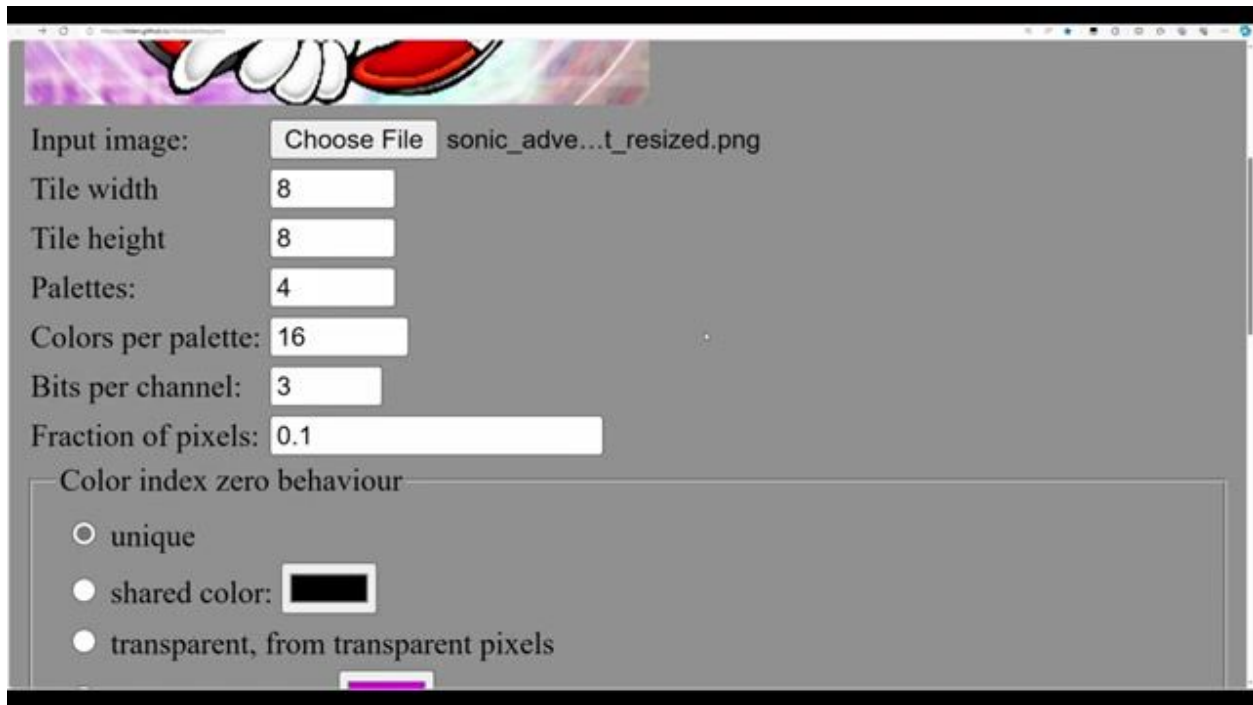


Figure 57: Quantization parameters configured for Mega Drive

intensifies it, and that different dithering patterns are usually offered (diagonal, horizontal, vertical, in 2- or 4-color variants) which are also worth comparing visually before settling on one.

4. Generating and downloading the quantized image

With the parameters set, the quantization is run. With a low pixel-fraction value the process is usually done in seconds; with higher values it can take longer, but in any case it's a matter of minutes, not hours, even on full-screen images.

The result is shown together with the generated palettes: in the example with four full palettes, four rows of 16 colors each, with the shared color (black, in this case) as the first color in all four. The resulting image can be downloaded and used directly: the produced file is usually imported into SGDK without any additional conversion step, even if the tool outputs a BMP instead of a PNG, since SGDK's resource compiler accepts both formats without distinction.

The already-quantized image together with the four resulting palettes, with the shared black as the first color of each.

5. Inspecting the result in the graphics editor

When opening the downloaded file in the graphics editor, it's essential to confirm that the color mode stays indexed and not switch it to RGB or any other mode under any circumstances: doing so would destroy the per-tile palette assignment the tool just built.

The result of this kind of quantization is usually quite satisfactory for an isolated illustration like the one used as an example, although there are almost always specific spots — high-



Figure 58: Quantized image together with its four generated palettes

contrast areas or ones with very fine color transitions, like a character’s eyes — where the seam between neighboring tiles, each with its own assigned palette, is more clearly noticeable.

A close-up of the eye area, where the seam between neighboring tile palettes is more clearly noticeable.

In the editor’s palette panel you can clearly see the structure of four rows of 16 colors, with the shared color as the first color of each; whatever other blacks might appear after that first color simply go unused and don’t need to be removed. Each 8×8-pixel tile in the image is assigned a single one of the four palettes (PAL0 through PAL3), so that the image as a whole makes use of all four palettes available on the console.

6. The rule that can’t be broken: one tile, one palette

Before manually touching up any already-quantized image, there’s a restriction that needs to be kept in mind at all times: within a single 8×8 tile, colors from two different palettes can’t be mixed. If that happens, SGDK’s resource compiler can’t resolve a single palette for that tile and halts compilation with an error.

To check which palette a given color belongs to, most editors let you inspect the exact index of a pixel with a modifier-key-plus-click combination (Alt+click, for example), which highlights the exact row and position of that color in the palette panel. This is especially useful before manually painting over a quantized image: if you add a color that belongs to a different palette row than the rest of the tile is already using, you unknowingly introduce a palette conflict that will only show up as an error once you compile.

When you need to visually combine elements that in practice belong to different palettes — for example, a foreground detail with its own palette over a background

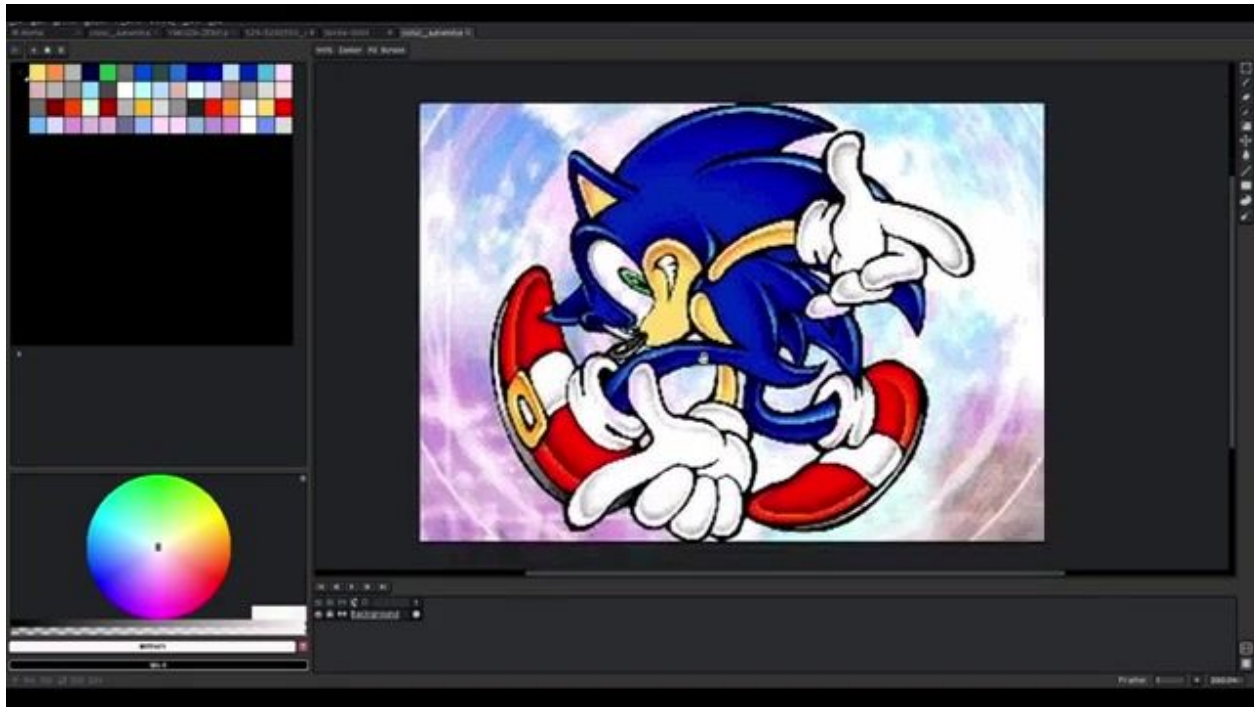


Figure 59: Seam between different tile palettes in the eye area

with its own — the solution isn't to force them into the same tile, but to overlay two independent background images, each with its own palette assignment.

An effective way to visually verify that no tile mixes palettes is to temporarily replace, in the editor itself, all the colors of each of the four palettes with a single, clearly distinguishable flat tone. The result is an image reduced to four solid colors, where each uniformly colored region of tiles corresponds exactly to one palette: any mixing between palettes would immediately show up as an irregular pattern within a single region, instead of clean blocks.

The full image with the four palettes each reduced to a flat color, showing that no tile mixes colors from more than one palette.

7. Manual touch-up: polishing the problem areas

Most images quantized this way are going to need some degree of manual touch-up, especially in high-contrast or fine-detail areas where the algorithm has less room to accommodate the available color. How much time is worth investing in that polishing is up to whoever is doing it: automatic quantization handles most of the work, but rarely one hundred percent of it.

A practical technique for deciding whether to merge or replace a color is, before making the final call, to temporarily change it to a very bold tone quite different from the rest of the image — an intense yellow, for example. That immediately reveals exactly where in the image that color is used, information that's otherwise hard to appreciate when the color in question is very similar to another nearby one in the same palette. Once you've confirmed the replacement doesn't cause any noticeable loss, the final color is applied.

The problem color recolored in intense yellow, to spot at a glance where it appears before deciding on its replacement.

After replacing a color, there can still be a somewhat abrupt transition between two tones

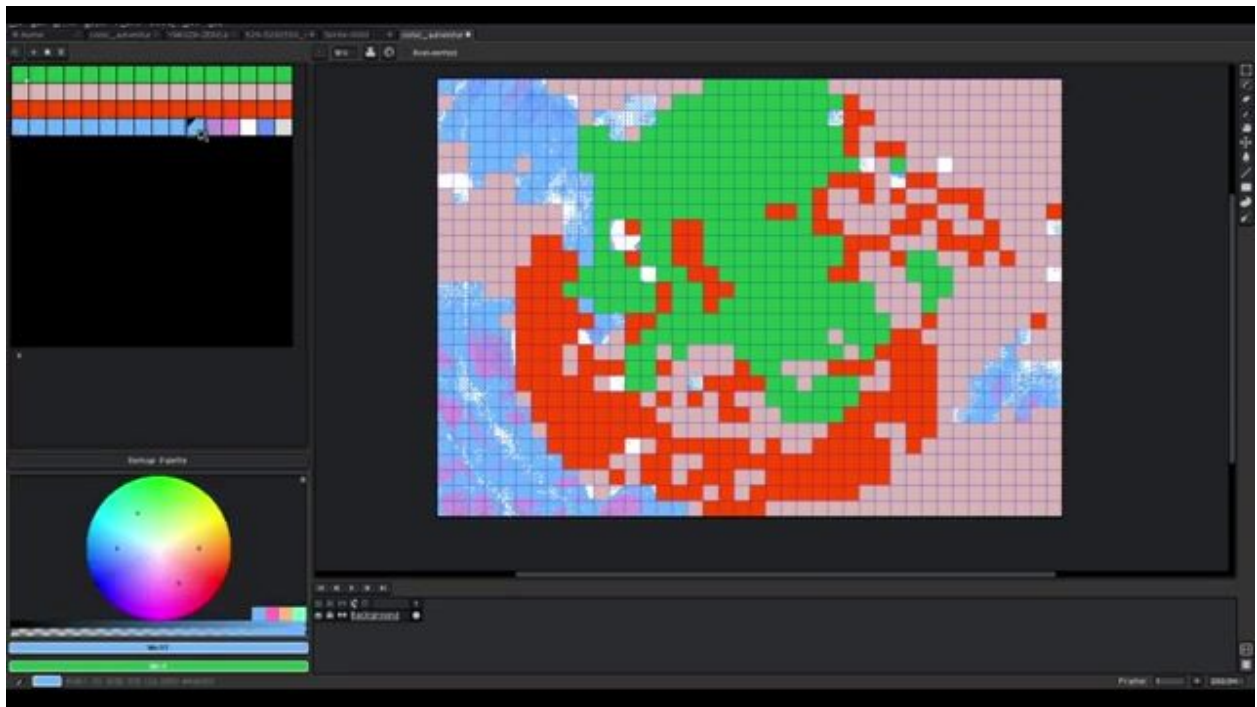


Figure 60: Each palette reduced to a flat color, with no mixing between tiles

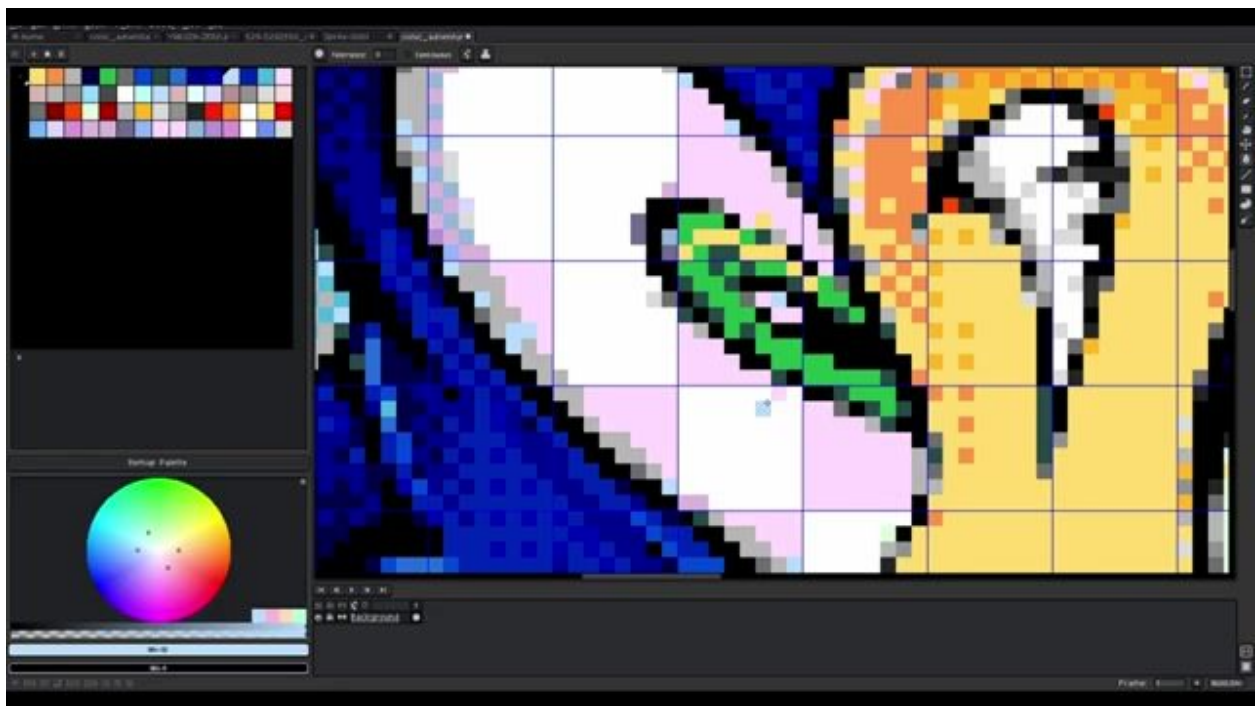


Figure 61: Problem color temporarily recolored in intense yellow

within the same tile; in those cases, adding manual dithering — alternating pixels of both colors in a regular pattern — can smooth out the edge, always respecting the rule from the previous section: any color added by hand has to belong to the same palette as the rest of the tile it's drawn in.

8. Integrating the converted image into SGDK

Once you're satisfied with the result, the already-quantized image is integrated into the project exactly like any other IMAGE resource, with one difference from the basic flow seen in Chapter 7: since the image carries four full palettes generated by the external tool, instead of loading them directly with `PAL_setPalette()`, it's better to apply them with a fade-in, to avoid the image suddenly appearing with all its colors in the very first frame.

In `resources.res`:

```
IMAGE quantized_bg "sonic_quantized.bmp" NONE
```

And in `main.c`:

```
#include <genesis.h>
#include "resources.h"

int main(bool hardReset)
{
    u16 palette[64];

    // All colors set to black before drawing, so the
    // image doesn't suddenly appear with its real colors
    PAL_setColors(0, palette_black, 64, CPU);

    // Draw the already-quantized image, using all four palettes
    VDP_drawImageEx(BG_A, &quantized_bg,
        TILE_ATTR_FULL(PAL0, FALSE, FALSE, FALSE, TILE_USER_INDEX),
        0, 0, FALSE, TRUE);

    // Copy to RAM the four real palettes carried by the image
    memcpy(palette, quantized_bg.palette->data, 64 * sizeof(u16));

    // And apply them with a smooth fade-in
    PAL_fadeIn(0, 63, palette, 100, TRUE);

    while (TRUE)
    {
        SYS_doVBlankProcess();
    }

    return 0;
}
```

The pattern is always the same: set all 64 color slots to black with `PAL_setColors()`, draw the image with `VDP_drawImageEx()` (which internally already associates each tile with its corresponding palette, as defined during quantization), copy the four real palettes carried by the imported resource into an array in RAM, and apply those palettes gradually with `PAL_fadeIn()` instead of an abrupt jump. This same fading technique — setting everything to black and then

bringing up the real palette progressively — can be reused in any other screen transition in the game, not just in the image conversion covered in this chapter.

Once compiled, the ROM faithfully reproduces the original illustration already reduced to the Mega Drive's four palettes, with a smooth fade-in transition. The result is remarkable relative to how little time it takes to get there, compared to a fully manual color reduction.



Figure 62: The converted Sonic illustration, running in an emulator

The final result in the emulator: the Sonic illustration already reduced to the four Mega Drive palettes, with the fade-in applied.

9. The limits of the technique

The example worked through in this chapter — an isolated illustration used as a title screen, with all four full palettes available just for it — is the scenario where this technique performs best. The situation changes quite a bit when the graphic being converted is a playable-level background that has to coexist with other elements: if only two of the four palettes can be reserved for the background — leaving the other two for, say, enemy sprites and interface elements — the result of the automatic quantization tends to look noticeably poorer and more “washed out” than in the case of an image that has all four full palettes at its disposal. This is especially noticeable in highly detailed art styles, with many color nuances and smooth transitions — the kind of graphic you might see in an exploration-heavy platformer with a lot of environmental detail — where half the available color capacity isn’t enough to preserve the visual richness of the original.

This automatic quantization works very well as a starting point and as a complete solution for fixed, isolated graphics — covers, title screens, standalone illustrations — but it doesn’t replace hand-done color reduction work when the graphic is highly detailed and has to share palettes with other elements in the game. In those cases, the most realistic approach is to treat automatic quantization as a rough first draft and assume that a good part of the color

adjustment is going to end up being done by hand, tile by tile, with the same care you'd take drawing a graphic designed for the console from the very start.

In the Mega Drive homebrew development community, there are experiments floating around that take this same idea of tile-based quantization to full video sequences, frame by frame, to achieve something resembling full-motion video (FMV) playable on the console, making use of up to 61 colors per frame. It's more a demonstration of the method's possibilities than a technique in everyday use: an actual video-playback project on the Mega Drive would probably also need additional hardware, such as the Mega CD add-on, to handle the data volume and read speed that kind of content demands.

This chapter closes out the part of the manual devoted to backgrounds, tiles, palettes, and scrolling. The next part picks back up with sprites and their animation, building on the same vocabulary of tile, palette, and color index covered so far.

Review exercises

1. Explain the difference between adjusting the canvas size and adjusting the image or sprite size when preparing a source illustration for this technique, and why the former option is chosen.
2. Describe the number-of-palettes, colors-per-palette, and bits-per-channel parameters of a tile quantization tool, and state the correct values for the Mega Drive and why.
3. What's the difference between choosing a shared color and a transparent color for index 0 of each palette, and for which type of graphic — background or sprite — would each option be preferable?
4. Explain the fundamental rule that has to be respected within each 8×8 tile when manually touching up an already-quantized image, and what happens in SGDK if it's broken.
5. Based on the contrast between converting an isolated illustration with all four full palettes available and converting a level background that can only use two, explain which type of content this technique performs best on and which requires more additional manual work.

Chapter 21 — How to Display Sprites

SGDK's sprite engine, pixel coordinates, and automatic VRAM allocation

The previous chapters dealt entirely with background planes: how to display them, how to scroll them, and how to draw them while respecting the hardware's rules. Planes build the game's stage, but not what the player controls or what they play against. That role belongs to sprites: the character that moves through the world, the enemies, the projectiles, any graphical element that animates or moves independently of the background. This chapter opens Part III by showing how to put a first sprite on screen, and along the way there turn out to be as many similarities as differences compared to the background pipeline already familiar to us.

Chapter 2 already introduced, at a conceptual level, the hardware behind sprites: their own control table in VRAM (the Sprite Attribute Table, or SAT), independent of the planes' name tables; the 32×32-pixel size limit per hardware sprite, combining up to 4×4 tiles; and the simultaneous limits depending on the resolution mode (80 sprites on screen and 20 per line in H40; 64 and 16 in H32). That vocabulary is taken as known here; this chapter deals with the code needed to put an actual sprite on screen.

The sprite used as an example in this chapter is taken as already prepared, without repeating the creation process covered in chapter 8. Everything said there still applies: a sprite is, first and foremost, a graphic, and as such it has to respect the size-multiple-of-8 rule, the 16-color-per-palette limit, and the indexed PNG format. What this chapter adds is the other half of the job: how to declare that file as a resource of a different type than a background, and how to put it on screen through an SGDK mechanism completely separate from the one used for planes, the sprite engine.

Prerequisites

Before continuing, it helps to have:

- Completed chapter 7, “How to Display Backgrounds on the Mega Drive”: the resource pipeline, the three sections of `main()`, `PAL_setPalette()`, and the concept of a palette (PAL0 through PAL3) are taken as known.
- Completed, or at least reviewed, chapter 8, “Creating Graphics with Aseprite”: a sprite is just another graphic, subject to the same size and palette rules.
- An understanding of the sprite hardware presented in chapter 2: the SAT, the maximum size of 32×32 pixels per sprite, and the simultaneous quantity limits depending on the resolution mode.
- A base SGDK project, either empty or built from one of the examples in previous chapters, with a sprite PNG file already copied into the `res` folder.

1. The SPRITE Resource: What Sets It Apart from an IMAGE

Loading a sprite into the ROM starts, just as with a background, by copying the PNG file into the `res` folder and declaring it in `resources.res`. But the directive is different: instead of `IMAGE`, `SPRITE` is used, and its list of fields is longer.

```
SPRITE our_sprite "sonic.png" 6 6 FAST 0
```

The first two fields, the name the C code will use (`our_sprite`, in this example) and the path to the PNG file, play the same role as in an `IMAGE`. The third and fourth fields, on the other hand, have no equivalent there: you have to explicitly specify the width and height of a single sprite frame, in tiles (6 and 6, in this case: a 48×48-pixel sprite). The reason for this difference is picked up in detail in chapter 22, dedicated to creating animation sheets: a single sprite file can contain several frames laid out side by side, and SGDK needs to know the size of an individual frame in order to be able to separate them. With a static, single-frame sprite like the one in this example, that size matches the size of the entire image.

The fifth field is, again, compression, with the same options as for an `IMAGE` — `NONE`, `FAST`, and `BEST` — but with an additional consideration that is specific to sprites. `NONE` compresses nothing; `FAST` prioritizes fast decompression at the cost of less space savings; and `BEST` offers the biggest savings, but demands more CPU work to decompress every time the data is used. `BEST` corresponds, in the `rescomp` documentation, to the algorithm identified as `APLIB`; in recent versions of SGDK the same word may instead indicate an automatic selection of the most convenient algorithm for each resource. It's worth confirming which of the two behaviors applies in the installed version before assuming that `BEST` always means the most aggressive, and slowest, compression.

That distinction in decompression speed matters far more for a sprite than for a background. A background, once drawn, normally stays the same in VRAM for a good part of the game; the cost of decompressing it is paid only once. An animated sprite, on the other hand, changes frames several times per second, and each frame change can involve decompressing data again and uploading it to VRAM. The effect is measurable: in a project with several animated characters on screen at once, changing their sprites' compression from `FAST` to `BEST` can cause a noticeable drop in frames per second, to the point of making the game perceptibly slower.

Fast compression for anything that animates often. The practical takeaway is to prefer `FAST` for sprites that animate frequently, and to reserve `BEST` for data that loads only once, like backgrounds. The alternative (loading every possible animation frame into VRAM up front, so nothing needs to be decompressed during gameplay) is also valid, but has the opposite cost: that VRAM space is no longer available for anything else, such as more background tiles. It's a trade-off between space and speed, not a one-size-fits-all answer.

The last field on the line, left at 0 in this example, is the animation timing: how many frames each animation frame lasts before moving on to the next one. At 0, automatic animation is disabled, which is exactly what's expected for a single-frame sprite like the one in this chapter; chapter 23, dedicated to animating sprites, returns to this field in detail.

2. The Three Sections of `main()` Again, and a Sprite That's a Pointer

With the resource already declared, it's necessary, as with any resource, to include `resources.h` in `main.c` so the code can refer to it by name. From there, the same three-section

organization used in chapter 9, on scroll and parallax scrolling, applies again: a global section, before `int main()`, for anything that needs to exist for the entire lifetime of the program; an initial setup section, between the opening of `main()` and the `while(1)`, for anything that runs exactly once; and the main loop, inside the `while(1)`, for anything that repeats every frame.

The first declaration in this chapter goes in the global section, and has a different form from any variable used so far:

```
Sprite *player;
```

The asterisk indicates that `player` is not itself a `Sprite`, but a pointer: a variable that, instead of storing a value, stores the memory address where another piece of data lives. The reason for this design becomes clearer looking ahead: as shown in section 5, the function that adds a sprite to the scene (`SPR_addSprite()`) doesn't create an independent copy for the game code, but instead returns a pointer to the `Sprite` that SGDK's own sprite engine keeps managing internally frame by frame. Storing that pointer, rather than a value of its own, is what lets the code, in later chapters, keep referring to this particular sprite in order to move it, animate it, or query its state, without having to look it up again each time. For now, it's enough to follow the recipe: `Sprite`, asterisk, a name of your choosing (`player`, in this example, though it could be called anything else), and a semicolon.

3. Initializing the Sprite Engine: `SPR_init()`

The sprite engine is the part of SGDK responsible for managing all active sprites at once: which VRAM they've been allocated, what screen position they're at, with what attributes, and how to pack all that information into the format the VDP's sprite hardware expects at every vertical blank (VBlank). Before adding the first sprite, that engine has to be initialized, and this is done exactly once, in the initial setup section, with `SPR_init()`:

```
SPR_init();
```

Since SGDK 1.4 (May 2019), `SPR_init()` takes no parameters at all: internally it reserves the maximum available number of sprites without needing to be told to. If older code or documentation turns up calling it with three arguments (`SPR_init(0, 0, 0)`, for example), that form still exists for backward compatibility under a different name, `SPR_initEx()`, but it's no longer the recommended form for new code: `SPR_init()` with no arguments is the current form, and the one used throughout the rest of this manual.

There's a more fundamental point beyond the difference in form: no other sprite function — not `PAL_setPalette()` for the sprite's palette, not `SPR_addSprite()`, not `SPR_update()` — has any way of warning you that it's failing because the engine isn't ready. The symptom of forgetting this call is, at best, that nothing appears on screen, with no error message pointing to the real problem. Treat this line as non-negotiable in any project that uses sprites.

4. Assigning the Sprite's Palette: The Same Function, a Different Data Source

With the engine initialized, the next step is exactly the same function used for backgrounds in chapter 7: `PAL_setPalette()`. The only thing that changes is where the color information comes from.

```
PAL_setPalette(PAL2, our_sprite.palette->data, DMA);
```

`our_sprite.palette->data` pulls the color information directly from the structure `rescomp` generated when processing the `SPRITE` resource, the same way `bg1.palette->data` pulled it from an `IMAGE` resource in chapter 7. The palette number chosen, `PAL2`, reflects an organizational convention here rather than a hardware rule: reserving `PAL0` and `PAL1` for the background and foreground planes, and `PAL2` and `PAL3` for the player-controlled character and for enemies, respectively. The VDP doesn't assign any special meaning to any of the four palettes; it's simply a criterion worth keeping consistent within a given project so as not to lose track of things.

5. Adding the Sprite to the Scene: `SPR_addSprite()` and Automatic VRAM Allocation

This is where a sprite's workflow starts to diverge clearly from that of a background. To display a background, chapter 7 needed a global variable (`ind`) to manually keep track of which VRAM position belonged to each image, incremented by hand after every call to `VDP_drawImageEx()`. For a sprite, that bookkeeping work isn't necessary: the sprite engine handles it on its own.

```
player = SPR_addSprite(&our_sprite, 0, 0, TILE_ATTR(PAL2, FALSE, FALSE, FALSE));
```

The first argument, `&our_sprite`, is the address of the sprite definition structure generated from the `SPRITE` resource (hence the `&` symbol, "address of"). The next two, `x` and `y`, are the sprite's initial position on screen, with an important difference from `VDP_drawImageEx()` that is picked up again in section 7. The fourth argument is the sprite's attribute, built with a different macro than the one used for backgrounds:

```
u16 TILE_ATTR(u16 pal, u16 prio, u16 flipV, u16 flipH);
```

`TILE_ATTR()` takes the same first four parameters as `TILE_ATTR_FULL()`, seen in chapter 7 for drawing backgrounds — that is, palette, priority, vertical flip, and horizontal flip — but, unlike that one, it doesn't take a VRAM index at all. That absence isn't an oversight: it's the flip side of the automatic resource allocation that `SPR_addSprite()` offers by default. The sprite engine decides on its own which VRAM position each sprite goes into, and it does so dynamically as sprites are added, removed, or change animation frame; the programmer doesn't have to reserve or keep track of anything, unlike the manual discipline with `TILE_USERINDEX` that backgrounds required. SGDK does offer a function, `SPR_setVRAMTileIndex()`, to set a sprite's VRAM position by hand in cases where that fine-grained control is needed, but the automatic behavior is the one used by default and is more than enough for practically any introductory case like this one.

With this line, the sprite is added to the scene, and its pointer, returned by `SPR_addSprite()`, is stored in `player`, the variable declared in section 2. One last step is still needed, inside the main loop, for the sprite to actually appear correctly on screen.

6. Updating Sprites Frame by Frame: `SPR_update()`

Unlike a background plane, which, once drawn with `VDP_drawImageEx()`, stays on screen without needing any further instruction in the main loop, sprites need the engine to update them every frame. The function in charge of this is `SPR_update()`, and it's called with no arguments inside the `while(1)`:

```
while(1)
{
```

```
    SPR_update();  
    SYS_doVBlankProcess();  
}
```

`SPR_update()` walks the internal list of active sprites, computes their updated state (position, animation frame if applicable, VRAM priority), and prepares that information into a list that gets sent to the VDP during the next vertical blank. Omitting this call doesn't stop the program from compiling or running, but sprites added with `SPR_addSprite()` never actually get shown or updated on screen: the engine keeps managing them behind the scenes, but never gets around to telling the hardware any of it.

7. Compiling and Experimenting: Pixels, Not Tiles, and the Flip Flags

With all six pieces now in place (`resources.res` with the `SPRITE` resource, the `resources.h` include, the player pointer, `SPR_init()`, `PAL_setPalette()`, and `SPR_addSprite()` in the initial setup, and `SPR_update()` in the loop), saving both files and recompiling the project already shows the sprite on screen, at position (0, 0), the top-left corner.

A good first experiment is to change the sprite's initial position and observe the result. Changing `x` and `y` from (0, 0) to (10, 10) only shifts the sprite a little to the right and down. This small a shift isn't a coincidence: unlike the `x` and `y` coordinates of `VDP_drawImageEx()`, which in chapter 7 were measured in tiles (a value of 5 shifted the image 40 pixels), the coordinates of `SPR_addSprite()` are measured directly in pixels. A value of 10 moves the sprite 10 pixels, not 10 tiles (80 pixels); to get a shift visually comparable to that of a background plane, a considerably larger number is needed, such as 100.

Tiles for planes, pixels for sprites. This difference in units between the two functions isn't a whim or an inconsistency in SGDK: it has to do with the fact that sprites, unlike planes, move freely pixel by pixel across the whole screen (which is why the Mega Drive's own sprite hardware positions them in pixel units), whereas a background plane, even though it can scroll in pixels (as seen in chapter 9), is initially drawn tile by tile. Keep firmly in mind which of the two units applies to each function: mixing them up produces sprites that barely move or backgrounds that jump abruptly.

The last two parameters of `TILE_ATTR()`, `flipV` and `flipH`, let you flip the sprite without needing another image file or any additional function. Changing `flipV` to `TRUE` flips the sprite vertically, upside down; changing `flipH` to `TRUE` flips it horizontally, facing the opposite direction from the original. Placed side by side, the effect is easy to picture: the standing silhouette exactly as it is in the PNG, that same silhouette flipped top to bottom with `flipV`, and that same silhouette facing the opposite way with `flipH`, without any of the three variants requiring a second sprite to be loaded or anything new to be drawn. It's the usual way of handling, for example, a character changing direction while walking.

The first parameter of `TILE_ATTR()`, `priority`, works the same way as seen in chapter 7 for planes: by default, the Mega Drive's drawing order goes from the background plane (`BG_B`) at the back, the foreground plane (`BG_A`) in the middle, and sprites in front of everything. Marking a sprite with high priority doesn't change that overall order relative to the planes (sprites are already drawn in front), but if the foreground plane is also marked with high priority, as explained in chapter 15 when discussing depth through tile priority, that plane can end up being drawn in front of the sprite in the relevant areas. It's the same tool that allows, for example, a character to appear behind a lamppost or a tree trunk in certain columns of the screen and in front of it in others.

Review exercises

1. Explain in your own words what each of the fields in a `SPRITE` line in `resources.res` is for, and how it differs from the equivalent field in an `IMAGE` line.
2. A teammate notices that their game, very smooth during the early tests, starts dropping frames per second as soon as they add animations to several characters on screen. They check their `resources.res` and find that all of their sprites use `BEST` compression. Explain why this is a reasonable suspect and what change you would try first.
3. Why is `player` declared as `Sprite *player` and not simply as `Sprite player`? Relate your answer to what `SPR_addSprite()` returns.
4. A sprite is added with `SPR_addSprite(&our_sprite, 5, 5, TILE_ATTR(PAL2, FALSE, FALSE, FALSE))` and doesn't appear on screen at all, even though the code compiles without errors. List at least two distinct possible causes, related to the functions covered in this chapter, and how you would tell them apart.
5. Why are the `x` and `y` coordinates of `SPR_addSprite()` measured in pixels while those of `VDP_drawImageEx()`, seen in chapter 7, are measured in tiles? What practical problem could assuming both functions use the same unit cause?

Chapter 22 — Creating Sprites and Animation Sheets

Chapter 21 showed how to put an already-prepared sprite on screen, using `SPR_init()`, `SPR_addSprite()`, `SPR_update()`, and the tile attributes described by `TILE_ATTR`. That chapter deliberately left the other end of the process pending: how that sprite actually gets drawn. This chapter closes that loop: how to put together your own animation sheet in a pixel art editor, prepare it so that the SGDK resource compiler (`rescomp`) slices it correctly into frames, and what real limits the Mega Drive’s sprite hardware imposes beyond what was already introduced in Chapter 2.

Much of this workflow directly reuses what was explained in Chapter 8 about creating graphics with Aseprite: the same editor, the same Mega Drive palette, the same discipline of working in RGB mode and switching to indexed mode only at the very end. What gets added here is everything that process needs when the graphic isn’t a single fixed image but a sequence of frames that, on top of that, has to be turned into a sheet with an exact row-and-column layout — the layout that Chapter 23, dedicated to sprite animation, explains how to interpret on the SGDK side.

Prerequisites

Before continuing, it’s a good idea to have:

- Completed Chapter 8 (“Creating Graphics with Aseprite”): the Mega Drive palette, the 16-color limit, and the “RGB first, indexed at the end” order are assumed to be known and won’t be repeated in detail here.
- Completed Chapter 21 (“Displaying Sprites”): the `SPRITE` resource in `resources.res`, the sprite engine, and the general idea that a sprite can contain several frames are assumed to be known.
- A pixel art editor with support for animation frames and tags. The examples that follow use Aseprite and its terminology, although the concept carries over to any equivalent editor.

1. A sprite’s canvas: the size of one frame, not the whole sheet

Creating a sprite starts the same way as creating a background: a new canvas, in RGB mode (for the same reason already seen in Chapter 8: avoiding color-banding issues when switching to indexed mode with a palette that’s still too large), but this time with a Transparent background instead of white or black, since a sprite needs real transparency so it doesn’t cover whatever is behind it in the game.

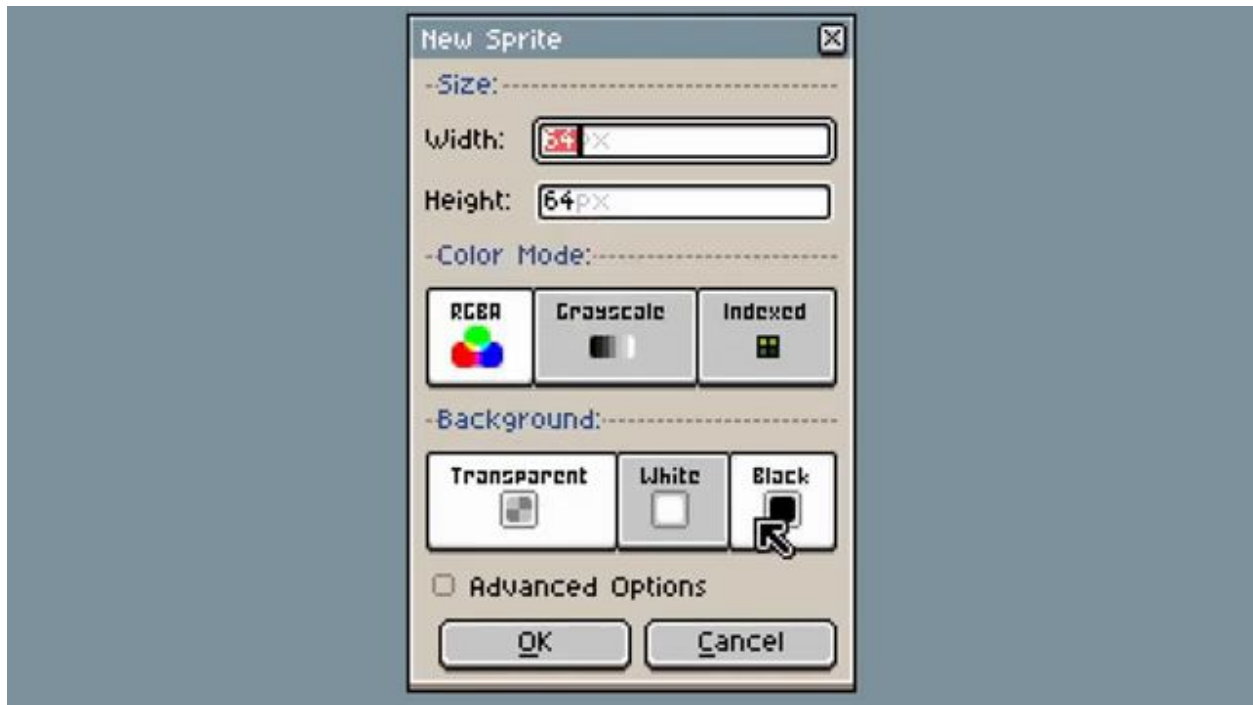


Figure 63: New sprite creation dialog

New sprite creation dialog in Aseprite: 64×64 pixels, RGB mode, Transparent background.

It’s worth clarifying a point where the editor itself uses the same word for two different things: when you choose a transparent background, Aseprite draws a checkered pattern as a visual reference for that transparency, with a default checker size of 16×16 pixels. That checker size is purely a visual aid from the editor and has no relation whatsoever to the frame width and height declared in the SPRITE line of resources.res (seen in Chapter 21 and revisited in Chapter 23): they’re two different uses of the idea of a “cell” that are worth not confusing. To make the visual reference match the hardware’s real 8×8-pixel tiles, you can change that checker size to 8×8 in the editor’s preferences.

The 64×64-pixel size chosen in this example (8×8 tiles) is merely the size of a single frame of the animation, not a hardware limit or the final size of the whole sheet: as more frames get added, the sheet will grow horizontally or vertically well beyond this initial canvas. Section 5 comes back to this point when preparing the sheet for export.

2. Animation frames: adding, copying, and the role of digital tracing paper

With the first frame drawn, adding a new one is as simple as pressing the corresponding button in the frames panel or the Alt+N keyboard shortcut; each press creates an additional, empty frame, ready to be drawn. For a simple animation — a bouncing ball, say — it’s enough to copy the content of the previous frame, paste it into the new one, and nudge it slightly, instead of redrawing the whole element frame by frame.

A feature that makes this work much easier, especially in animations more elaborate than a simple ball, is digital tracing paper (*onion skinning*, by analogy with the semi-transparent onion paper traditional animators used to draw a frame while seeing the previous one through

it). When enabled, the editor shows the preceding and following frames overlaid and dimmed behind the one currently being drawn.

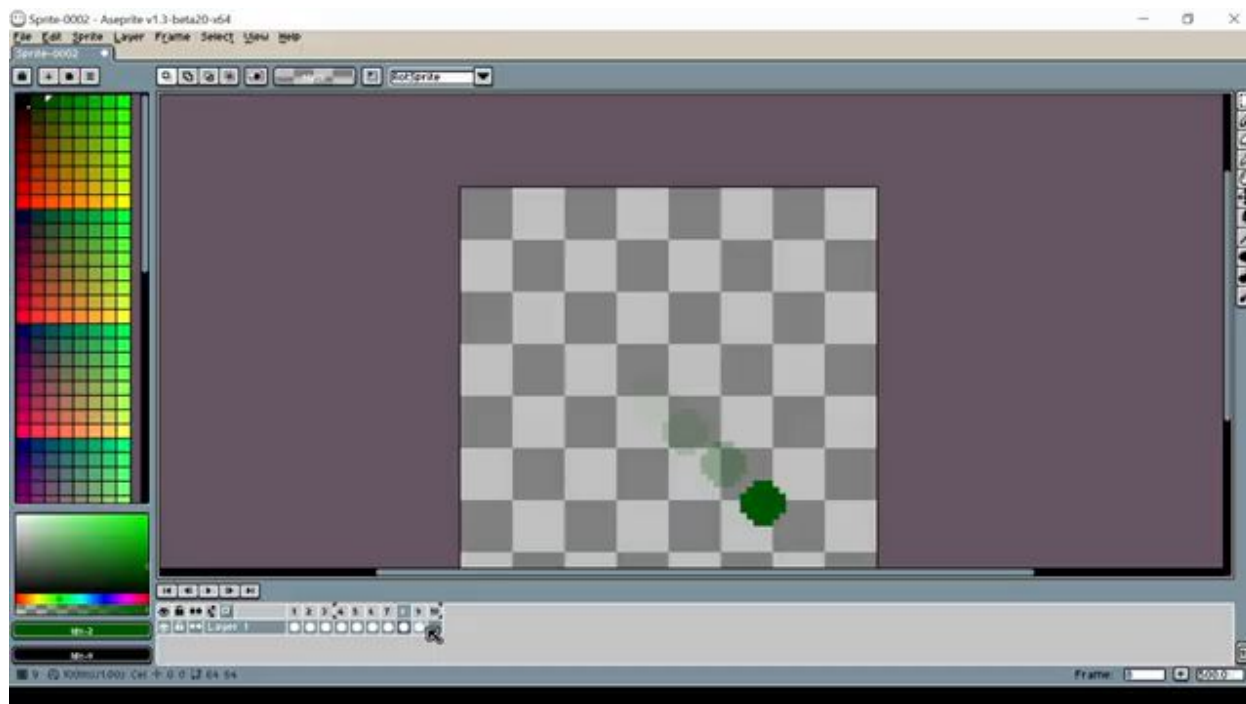


Figure 64: Digital tracing paper enabled

Digital tracing paper (onion skinning) enabled: the current frame is drawn while seeing the previous frames of the same animation, dimmed, underneath.

This makes it possible to keep trajectory and proportions consistent between frames without constantly jumping back and forth to compare, which is especially valuable in walking or attack animations, where each frame visually depends on the previous one. When playing back the animation with the play button, it's a good idea to turn off onion skinning so you can preview the final result without the overlaid ghost frames.

The 16-colors-per-palette limit (seen in Chapter 2 and revisited in Chapter 8) still applies here exactly the same way, counting the color reserved for transparency: a sprite as simple as a ball, with a single fill color plus the transparency index, is already using two of the sixteen available colors.

3. Several animations on a single sheet: the problem and the solution with tags

The ball example has only one animation, but a real character — think, for instance, of a sprite sheet for an Axel Stone-style character, the protagonist of *Streets of Rage 2* — usually needs several: standing still, walking, attacking, and so on. All of those animations live on the same sheet, one after another along the editor's timeline.

The sheet's 17 frames, still ungrouped: playing them in order mixes the static stance, idle, walking, and special attack together, one after another, as if they were a single continuous animation.



Figure 65: Timeline with ungrouped frames

Playing that sequence as-is mixes several animations into one continuous pass, which isn't what's needed: each one has to be playable and previewable on its own. Aseprite's solution (and that of most editors with animation support) is **tags**: a range of consecutive frames grouped under a named label. By selecting the corresponding frames and creating a new tag for each block — for example `stationary` for the first static frame, `idle` for standing idle, `walking` for the walk cycle, and `grand_upper` for a special hit — each animation becomes identifiable and can be played back in isolation.

The animations already grouped into tags visible on the timeline: with a tag selected, the play button plays back only that animation.

This grouping into tags isn't just a convenience for previewing inside the editor: as shown in Section 5, it's also the mechanism Aseprite uses to export the final sheet with the row separation that `rescomp` needs in order to recognize each animation as distinct — the same row-and-column logic that Chapter 23 explains in detail on the SGDK side: each row of the sheet corresponds to one animation, and each column within that row, to a successive frame.

4. The editor's speed is not SGDK's speed

Each frame within a tag can have its own duration inside the editor, adjustable by right-clicking a frame and selecting its properties. Aseprite measures that duration in milliseconds, with a default value of 100.

Individual frame properties dialog: the duration is measured in milliseconds, not in the unit SGDK uses later.

Lengthening the duration of a particular frame is a legitimate and common animation technique: it's common, for example, for the last frame of a hit to be held a bit longer than the rest to give it more visual impact, stretching it from 100 to 500 milliseconds. It's also possible



Figure 66: Frames grouped into tags on the timeline



Figure 67: Individual frame properties

to select several frames at once and apply the same duration to all of them in one go, instead of adjusting them one by one.

The duration set in the editor doesn't travel with the exported image. It's internal Aseprite information, useful only for previewing the animation while drawing it, and it disappears completely once the sheet is exported as a PNG.

Here lies the most common source of confusion: the animation's actual speed in the game depends exclusively on the animation-timing field of the SPRITE line in `resources.res` (introduced in Chapter 21 and detailed in Chapter 23), measured in screen frames (sixtieths of a second), not in milliseconds.

Since both values use different units, going from one to the other requires a manual conversion. If an animation was timed in Aseprite at 60 milliseconds per frame, that value has to be converted to SGDK's unit by dividing by the approximate duration of one sixtieth of a second (16.67 ms): $60 \div 16.67 \approx 3.6$. The result is almost never a whole number, so it has to be rounded up or down, with the resulting small difference between the pace previewed in the editor and the one that ultimately runs in the game. There's no way to avoid this manual step with the tools covered so far; it's best to simply keep it in mind as part of the workflow, rather than assuming that the value seen in Aseprite is the one that will end up in the game.

5. Preparing the sheet for export: palette, indexed mode, and rows per animation

With all the frames and tags already defined, preparing the final file repeats, with one caveat, the same steps from Chapter 8. The caveat is that "New Palette from Sprite" this time has to take into account the colors used across all frames at once, not in a single image: all the frames on the same sheet share a single 16-color palette on the Mega Drive (the same idea, already seen in Chapter 21, that `PAL_setPalette()` loads a single palette for the whole resource), so the color reduction has to be done with the complete set of the animation in mind, not frame by frame.

"New Palette from Sprite" processing all the frames on the sheet at once, so as to keep only the colors actually used in any of them.

With the palette already trimmed to 16 colors or fewer, the next step is, just as with a background, changing the color mode from RGB to indexed.

Changing the color mode to Indexed, once the palette has already been trimmed to 16 colors or fewer.

Before saving the file there's a step that didn't exist for a single-frame background: converting the editor's individual frames into a single sprite sheet, using the Export Sprite Sheet option from the file menu. There are two settings there that really matter for the result to be compatible with what `rescomp` expects: the sheet type has to be "By Rows,"

Sprite sheet export dialog, Layout tab: sheet type set to "By Rows."

and, in the sprite settings tab of the same dialog, the Split Tags option has to be enabled. That option is what actually puts each animation — each tag created in Section 3 — on its own row of the final sheet, instead of leaving all frames in a single continuous row.

Sprite tab of the same dialog, with Split Tags enabled: the preview already shows each animation on its own row, exactly the layout `rescomp` expects.

This correspondence isn't a coincidence: it's exactly the same row-and-column logic on the SGDK side that Chapter 23 picks back up. Setting up the export this way is what makes both sides of the process — how Aseprite organizes the animations and how `rescomp` interprets

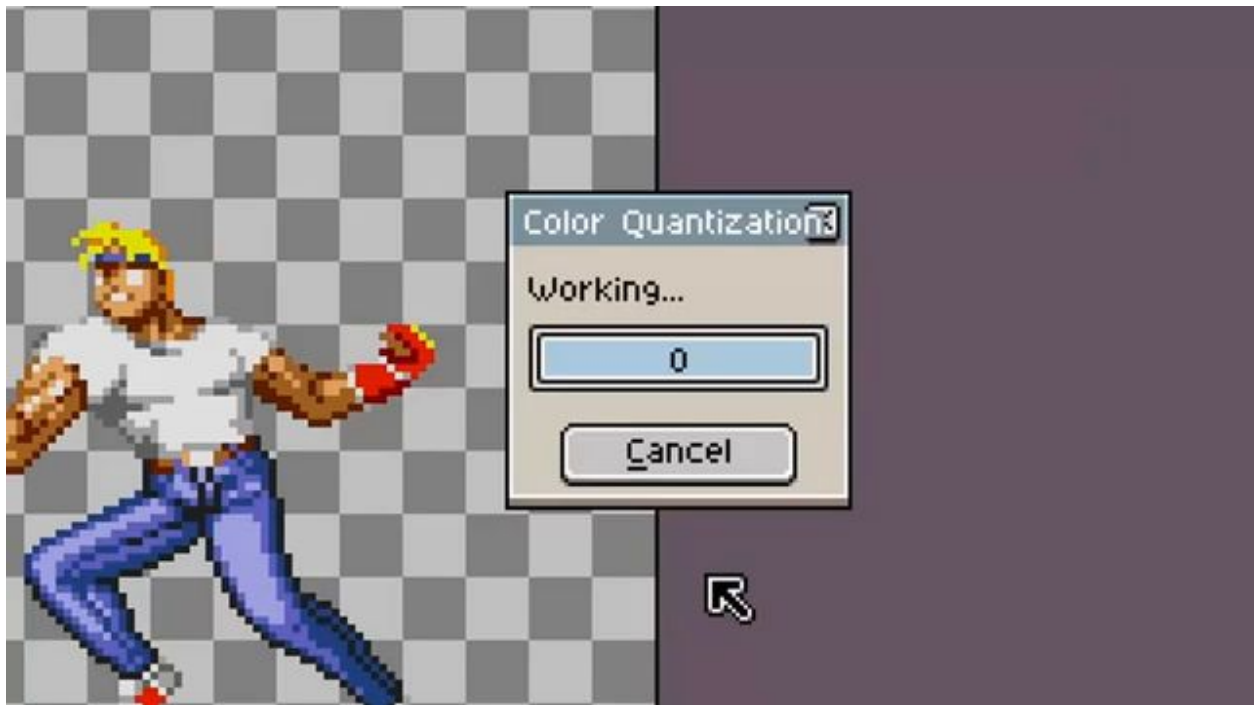


Figure 68: New Palette from Sprite

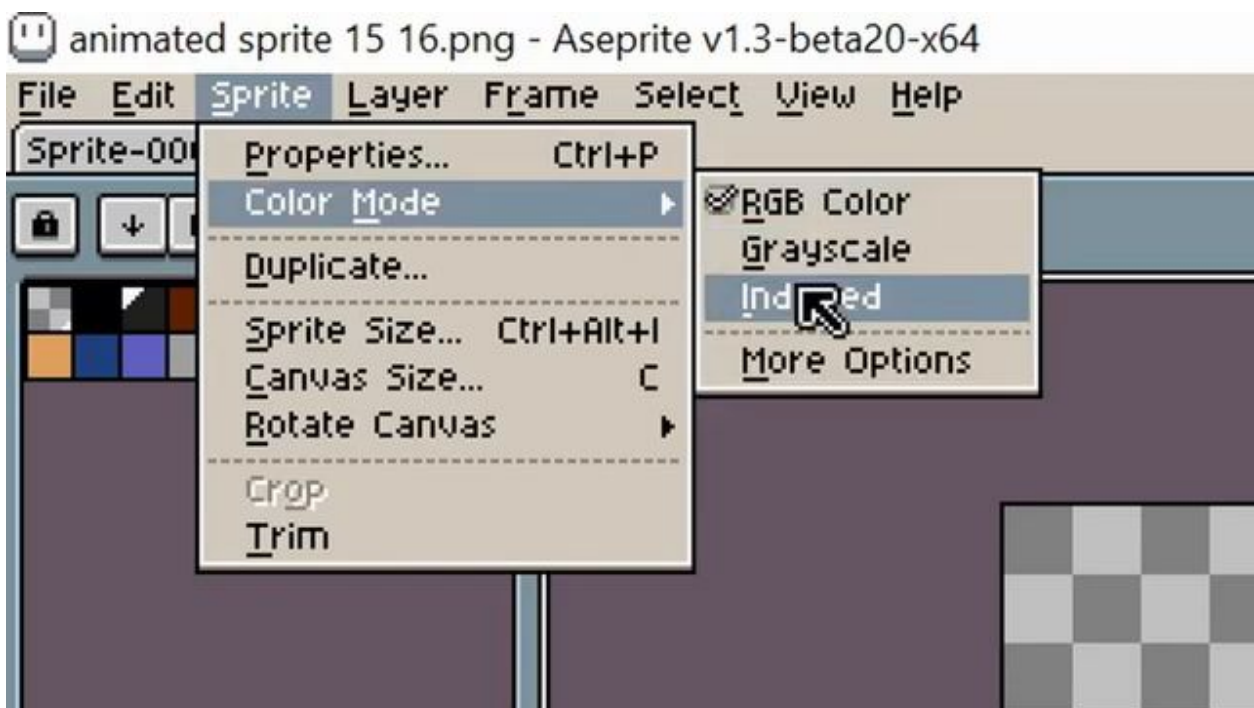


Figure 69: Changing color mode to Indexed



Figure 70: Sprite sheet export dialog, Layout tab

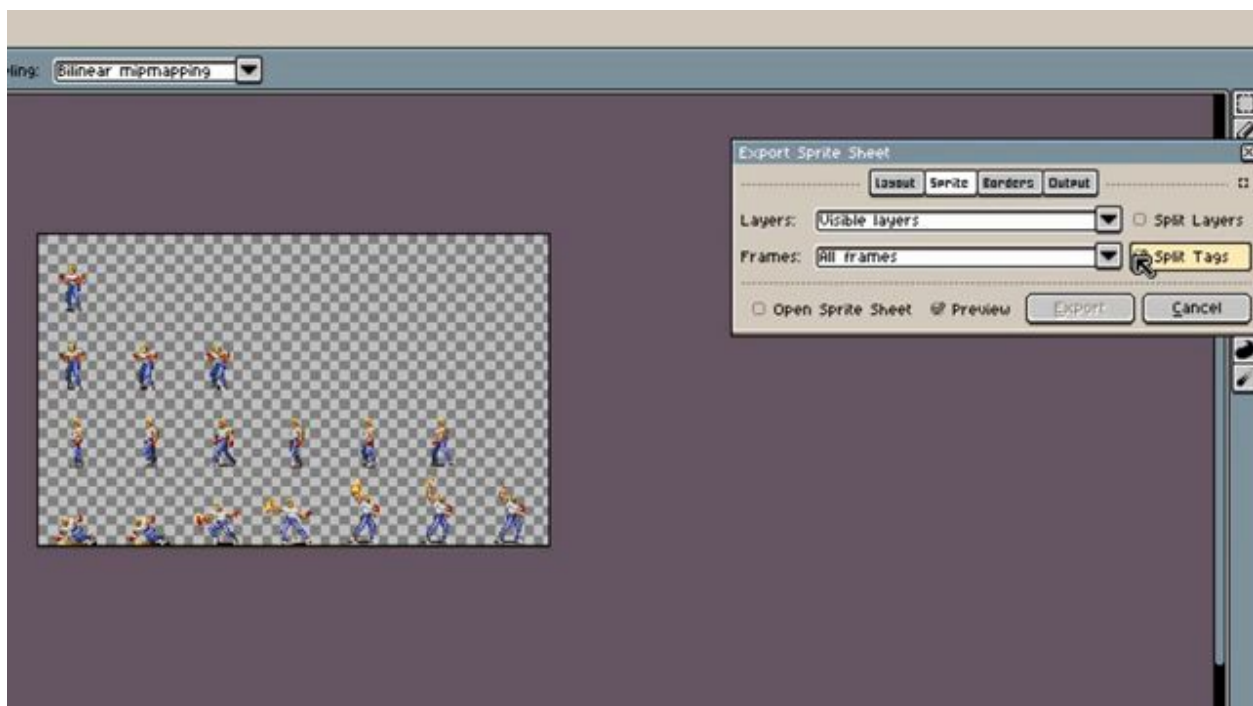


Figure 71: Sprite tab with Split Tags enabled

them — line up without any further manual adjustment. With the sheet already generated and previewed, the last step is to open it (Open Sprite Sheet) and save it as a PNG, the same way as any other Mega Drive graphic.

6. Real hardware limits for sprites

With the file now ready to import, this section steps away from Aseprite to talk about something that has so far been left implicit: how big sprites can be, how many fit on screen at once, and what happens when a project goes past those limits. To illustrate it, picture a fight against a flying final boss — in the style of a *Castlevania* boss — that fires many fireballs at once, while the playable character and the HUD are also on screen at the same time.



Figure 72: Scene with a final boss and many sprites on screen

A boss fight scene: the playable character, the flying enemy, several fireballs, and the HUD on screen at the same time, all of them sprites.

Chapter 2 already mentioned that the VDP’s sprite hardware has a maximum size per individual sprite of 32×32 pixels (4×4 tiles). A character visually bigger than that — like an Alucard-style protagonist from *Castlevania*, or a final boss — isn’t, strictly speaking, a single hardware sprite, but several sprites of up to 32×32 (or another allowed size, from 8×8 to 32×32 in any combination) combined to form the complete figure. That assembly doesn’t have to be programmed by hand: SGDK’s sprite engine, already used in Chapter 21 through `SPR_addSprite()`, takes care of automatically splitting a `SpriteDefinition` that’s too large into however many hardware sprites are needed, and of moving and animating them as if they were a single one from the code’s point of view.

That automatic assembly, however, doesn’t remove the quantity limits already seen in Chapter 2: in 320-pixel-wide mode (H40), the VDP can display a maximum of 80 hardware sprites in total per frame, and no more than 20 sprites (or 40 tiles wide, whichever comes first) on any single horizontal line of the screen. Exceeding that limit on a particular line doesn’t produce

an error or halt the game: the VDP simply stops drawing the sprites that exceed that line's quota for that particular frame, and since the set of dropped sprites can vary from frame to frame, the visible result is flickering (*sprite flicker*) in the affected elements. A scene with several fireballs, a large boss, and a HUD all competing for sprites on the same horizontal lines starts showing exactly that flicker as soon as enough projectiles pile up on screen, both in an emulator and on real hardware.

Sprite flicker was a much more frequent and noticeable problem on 8-bit consoles like the NES or the Master System, with considerably tighter per-line sprite limits; the Mega Drive, with its 20 sprites per line in H40, suffers from it less often, but isn't completely immune either, as any scene deliberately loaded with sprites demonstrates.

6.1 Transparent space costs nothing

Comparing two different ways of organizing the same animation sheet offers a specific lesson. Suppose a first version of a sheet where the character is drawn pushed up against one side of each cell, so the cell is as small as possible, under the reasonable-looking assumption that a smaller cell saves VRAM and reduces the risk of flicker.

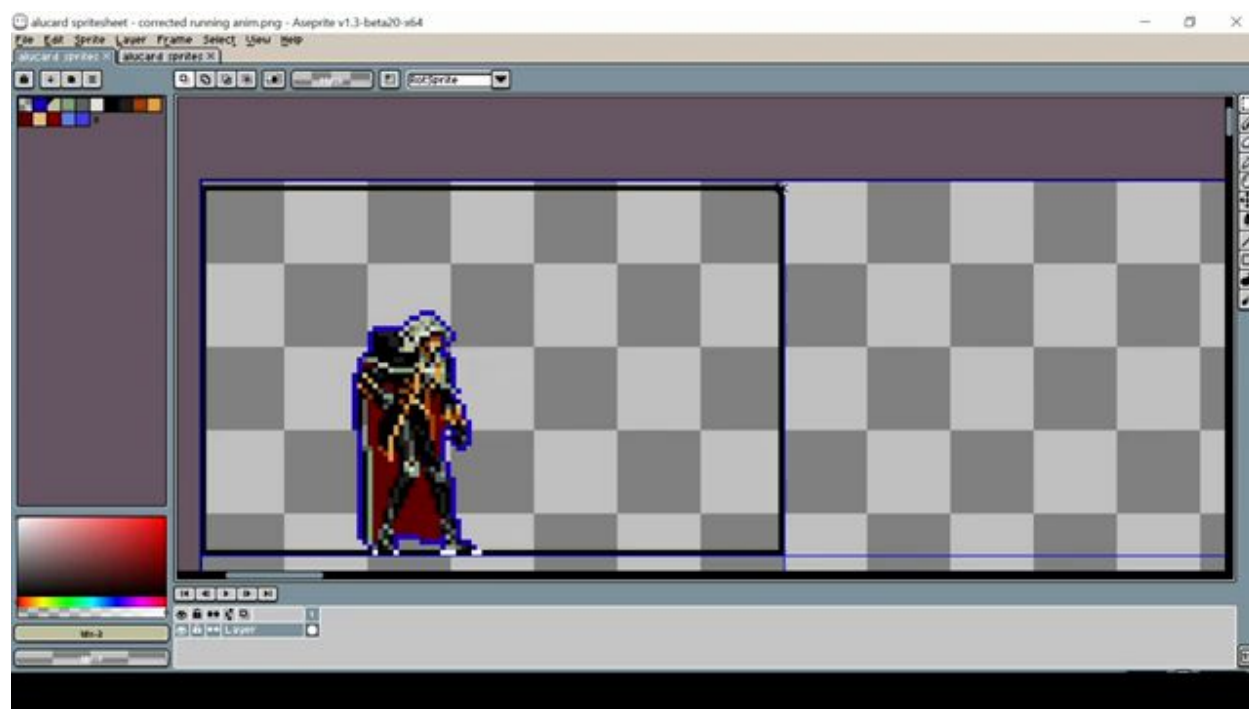


Figure 73: Character shifted to one side of the cell

First version of the sheet: the character drawn shifted toward the left side of each cell, to minimize the required cell size.

That adjustment, however, brings a cost of a different kind: since the character isn't centered within its cell, flipping it horizontally (with `flipH`, already used in Chapter 21) to make it face the opposite direction also shifts its apparent position on screen, which forces the collision code to constantly account for whether the character is facing left or right in order to correct for that shift. A second version of the same sheet, with the character properly centered in each cell — at the cost of a lot more transparent space around it — solves the problem: when flipped, the apparent position no longer shifts.

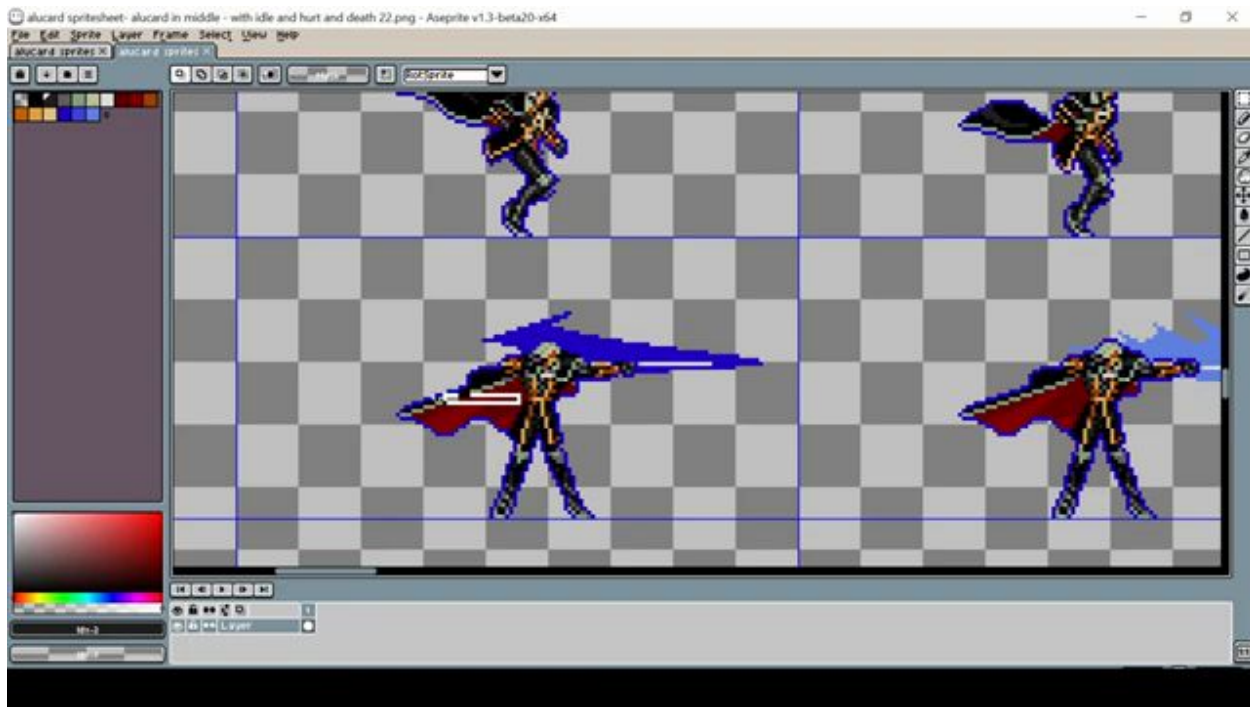


Figure 74: Character centered in the cell

Second version of the sheet: the character centered in each cell, with considerable transparent space on the sides; when flipped horizontally, the apparent position no longer shifts.

The counterintuitive part is that this extra transparent space has no real cost at all: rescomp detects the completely empty areas of each frame and excludes them both from the VRAM tile count and from the number of hardware sprites needed to represent the character, in the same way Chapter 8 already explained that SGDK detects and reuses flipped tiles. Centering the character on the sheet, then, doesn't cost any extra memory or make flicker worse: it's a code-organization improvement (fewer special cases for left and right) that comes free in terms of hardware resources. The same optimization applies to animation frames that are nearly identical to each other — for example, if a character's head doesn't change position between two consecutive frames: rescomp can recognize duplicate frames and avoid storing them again in VRAM, with no extra manual work required, although more demanding cases of space optimization may call for hand-tuning the sheet.

6.2 When it's not worth using a sprite

For truly enormous characters — a final boss that takes up a large part of the screen, for example — neither automatic VRAM allocation nor transparent-space optimization is enough to fully avoid the cost of so many combined hardware sprites. In those cases, a trick inherited from the 8-bit era is still valid on the Mega Drive: instead of building the enormous character entirely out of sprites, it's worth using one of the two background planes (BG_A or BG_B, seen in Chapter 2) to draw it, reserving sprites for the parts that genuinely need to move independently and with the pixel-level positioning freedom that sprites provide.

Review exercises

1. Explain the difference between the transparency checker size configured in the editor (8×8 pixels, for example) and the frame width/height declared in the SPRITE line of resources.res. Why is it a mistake to think they're the same setting?
2. A teammate carefully adjusts the duration of each frame of an attack animation inside Aseprite, making the final hit slower than the rest to give it impact. When testing the game, the animation plays back at a uniform speed, with no change of pace on the final hit. Explain why this happens and what they would need to do to achieve the effect they were after.
3. Why is it necessary to enable the *Split Tags* option when exporting a sprite sheet by rows, instead of letting rescomp guess where one animation ends and the next begins?
4. A character is drawn pushed up against the edge of each cell of its animation sheet, to minimize cell size. What programming problem does that decision introduce when the character is flipped horizontally, and why does centering it in the cell have no real cost in terms of VRAM or hardware sprites?
5. A final boss takes up half the screen and, when tested, the game shows noticeable flicker on several elements. Mention at least two different strategies, covered in this chapter, for reducing or avoiding that problem.

Chapter 23 — Sprite Animation

Animation sheets, `SPR_setAnim()`, frame speed, and why it's better to use `#define` instead of loose numbers

Chapter 21 left us with a sprite on screen that could appear, move, and flip, but it looked stiff: a single fixed image, with no motion of its own. This chapter fixes that and, along the way, changes protagonists: instead of a single-frame character, the example switches to an animation sheet for Axel Stone, the hero of *Streets of Rage 2*.

The change of file isn't cosmetic. A single-frame sprite and a full animation sheet are, at bottom, the same kind of resource (an indexed PNG that follows the palette and size rules covered in Chapter 22), but SGDK needs extra information to know how to slice an animation sheet into its individual frames and animations. That's the first obstacle in this chapter, and it shows up in the form of a compile error.

Prerequisites

Before continuing, you should already have:

- Chapter 21, “Displaying Sprites”: the `SPRITE` resource, the sprite engine (`SPR_init()`, `SPR_addSprite()`, `SPR_update()`), `TILE_ATTR()`, and the fact that a sprite's coordinates are measured in pixels.
- Chapter 22, “Creating Sprites and Animation Sheets”: an animation sheet is still, first and foremost, an indexed graphic subject to the 16-colors-per-palette limit and the multiple-of-8 rule.
- Basic C notions: variable declarations, and a first idea of what a constant is (this is picked up again and expanded on in section 5 of this chapter and in Chapter 6).

1. From a sprite to an animation sheet: what changes in the `SPRITE` resource

The starting point is the same project from the previous chapter. Switching sprites is, in principle, a single step: copy the new PNG file into the `res` folder and update the file name on the `SPRITE` line in `resources.res`, leaving the rest of the line untouched for now (still with `6 6`, the tile size of the previous chapter's sprite). Trying to compile as-is produces an error:

```
width (840) is not a multiple of cell width (48)
```

The message refers to a “cell width” of 48 pixels — exactly the 6 tiles of 8 pixels configured on the resource line — and a total image width of 840 pixels, which isn't a multiple of that 48. The problem isn't that the new file breaks the multiple-of-8 rule (840 does satisfy it): it's that the width declared in `resources.res` describes the size of a single animation frame, not the size of the whole image, and that size changed when the sprite changed.

The reason for this field, which went unnoticed with a single-frame sprite, becomes clearer when you look at the full animation sheet: unlike a static sprite, this file is one large image that contains, arranged in a grid, every frame of every animation for the character. For this particular sheet, each individual frame is 15×16 tiles (120×128 pixels). That's the size that needs to be declared where it previously said 6 6 (not the size of the whole sheet, which is considerably larger), so that rescomp knows where one frame ends and the next begins:

```
SPRITE our_sprite "animated_sprite_15_16.png" 15 16 FAST 0
```

With these corrected values, the project compiles without errors. Once rescomp knows the size of a frame, its slicing logic is straightforward: each full row of the sheet is interpreted as a distinct animation, and each column within that row as a successive frame of that animation. This idea is picked up in more detail in section 3.



Figure 75: Axel Stone's animation sheet divided into a grid

Figure 1: Axel's full sheet: each row is a distinct animation, and each column within a row is a successive frame of that animation.

2. Reminder: position is still measured in pixels

With the sprite now compiling, it's worth doing a quick visual check before moving on to the actual animation. Running the project with the position inherited from the previous chapter (100, 100), the new sprite — larger than the original — ends up with much of its body off the visible screen, near the bottom edge. Adjusting the y coordinate from 100 to 50 centers it better.

This adjustment is simply a direct application of what was already covered in Chapter 21: x and y in `SPR_addSprite()` are pixel coordinates, not tile coordinates, so the exact value that centers a sprite depends on its actual size in pixels, not on a fixed formula.

3. Playing an animation: `SPR_setAnim()`

With the sprite properly positioned, what's still missing is the core of the chapter: making it play one of its animations instead of staying on a single fixed frame. The function that chooses which animation to play is:

```
void SPR_setAnim(Sprite *sprite, s16 anim);
```

The first parameter is the pointer to the sprite already returned by `SPR_addSprite()` (player, in this example); the second, `anim`, is the index of the animation to play. To understand what that index means, we need to look at the full sheet again: each row is a distinct animation. On this particular sheet, from top to bottom: row 0 is a single static frame, with no real animation; row 1 is the idle animation; row 2 is the walk cycle; and row 3 is the Grand Upper punch, Axel's signature move in *Streets of Rage 2*.

As is customary in programming, numbering starts at 0. Calling the function with index 0, then, produces no visible motion, because that particular row contains only a single frame:

```
SPR_setAnim(player, 0);
```

Changing the second argument to 1 should, in principle, play the idle animation. But doing so and recompiling, the sprite still doesn't move: there's no error in this line, just a value that's still left unconfigured, as shown in the next section.

4. Animation speed: the last field in `resources.res`

The reason animation 1 doesn't move lies in the last field of the `SPRITE` line in `resources.res`, the one that's been left at 0 so far. That field is the duration of each frame, expressed in screen frames (1/60 of a second on the NTSC variant of the console). At 0, automatic animation is disabled entirely, which is why no animation index has produced any movement so far, so it needs to be set to a value greater than zero for the sprite engine to advance frames on its own.

A value of 60 turns out, in practice, to be too slow: with 60 screen frames per animation-frame change, and running at 60 frames per second, the animation changes only once per second — a noticeably slower pace than expected. Lowering the value makes the animation advance more often; a value of around 15 for the idle animation gives a visually reasonable result:

```
SPRITE our_sprite "animated_sprite_15_16.png" 15 16 FAST 15
```

Changing the animation index to 2 (the walk cycle) and adjusting the speed independently, in this case to 8 — noticeably faster than the idle animation's — completes the example:

```
SPRITE our_sprite "animated_sprite_15_16.png" 15 16 FAST 8
```

```
SPR_setAnim(player, 2);
```

This mechanism, as used so far, has a limitation: the speed field in `resources.res` is a single value per resource, not one per animation, so changing it affects all four rows of the sheet equally. `rescomp` offers, for anyone who needs a different speed per frame or per animation within the same sheet, a more advanced syntax with nested brackets that goes beyond the scope of this introductory chapter and is picked up later in this manual. It's worth knowing it exists, because sooner or later the difference between animations becomes a real issue — for example, a slow idle animation coexisting with a fast attack animation on the same character.

5. From loose numbers to meaningful names: why #define

Up to now, the animation index was written as a loose number directly in the call to `SPR_setAnim()`: 0, 1, 2, or 3. In programming this is known as a *magic number* (or, more informally, *hardcoding* a value), and it's a practice worth avoiding as code grows: a loose number says nothing about its purpose, and finding and fixing every occurrence of that number if its meaning ever changes becomes a tedious, error-prone task.

The natural first step is to replace the number with a variable:

```
int anim = 3;
SPR_setAnim(player, anim);
```

This already improves readability, but an ordinary variable (`int`) can change value at any point in the program, and the index of the *Grand Upper* animation on this particular sheet is never going to change: it will always be 3, no matter what else the rest of the code does. For that case — a fixed value that should never change — C offers the `const` qualifier:

```
const int ANIM_UPPER = 3;
```

Besides fixing the value, changing the name from `anim` to `ANIM_UPPER` solves the other real problem with a magic number: you no longer need to remember by heart that “the special move animation is number 3” — reading the name is enough. The convention of writing constants entirely in uppercase, with underscores between words, is a widespread practice in C (though not required by the language) that helps you tell, just by looking at the code, which identifiers are constants and which aren't.

For this kind of value, though, it's worth going one step further and replacing `const` with `#define`. As explained in Chapter 6, the Mega Drive has very little RAM available — barely 64 KB in total (Chapter 1) — and that's a resource worth actively conserving throughout the game's code. `#define` is a text substitution that the preprocessor resolves before the compiler ever sees the code, so it doesn't reserve any memory for the constant at run time; `const`, on the other hand, depending on the compiler and optimization level used, can end up taking up real space (the exact comparison between the two forms is well beyond what this chapter needs, and depends on specific details of the toolchain; it's enough to know that `#define` is, in principle, the safer choice in such a memory-constrained environment). The syntax of `#define` is also deliberately different from that of a variable declaration: it has no equals sign and no semicolon.

```
#define ANIM_STILL 0
#define ANIM_IDLE 1
#define ANIM_WALK 2
#define ANIM_UPPER 3
```

With the four constants now defined, the final code is considerably more readable than the loose-number version from the start of the chapter:

```
SPR_setAnim(player, ANIM_UPPER);
```

The behavior is identical to writing `SPR_setAnim(player, 3)` directly, but the code now reads on its own: anyone opening this file knows, without having to go check the animation sheet, that this line plays the character's special move. From here on, this manual uses `#define` consistently for this kind of fixed value, including sound management later in the book.

Review exercises

1. A teammate replaces one animation sheet with another that has larger frames, but forgets to update the width and height values on the `SPRITE` line in `resources.res`. What error should they expect to see, and why does the message talk about a “cell width” rather than the width of the whole image directly?
2. What relationship is there between the rows and columns of an animation sheet and the parameters of `SPR_setAnim()` and `SPR setFrame()` — the latter being a function that sets a specific frame within an animation, instead of letting it advance on its own? Explain in your own words what index 0 means in each case.
3. A sprite is correctly added to the scene, `SPR_setAnim(player, 2)` is called, and yet no movement is observed on screen. List at least two possible causes related to what was covered in this chapter, and how to tell them apart.
4. Explain, in your own words, the difference between hardcoding a number, using a variable (`int`), using a constant (`const`), and using `#define` to represent an animation index. Which of these four options would you choose for a value that should never change during the game’s execution, and why?
5. Why does changing the animation speed in `resources.res` affect all four animations on Axel’s sheet equally, rather than just the one being tested at a given moment? What would need to be done, in general terms, to make the idle animation slower than the walk animation without having to pick a single compromise value for both?

Chapter 24 — 2D Camera

A level map larger than the screen (chapter 11) and a character that moves with decimal precision (chapter 6) are, separately, half the solution. What's missing is the piece that connects them: a camera that follows the character as it moves across that large map, without letting either the camera or the character itself go past its limits. This chapter builds that feature step by step, using a *Zelda*-style map and sprite as an example, although the resulting logic is independent of any specific game and can be reused as-is in any project with scrolling.



Figure 76: The final result of this chapter

A character moves across a large map while the camera follows it and keeps it centered on the screen.

Prerequisites

Before continuing, you should already have in place:

- A level map larger than the screen, with the `TILESET/MAP/PALETTE` resources already declared and loaded with `MAP_create()`, and the `MAP_scrollTo()` function to scroll it (chapter 11).

- The `fix32` type, the `FIX32()` macro, and conversion to integer with `F32_toInt()` (chapter 6).
- A sprite already created and positioned with `SPR_setPosition()`, which takes `s16` coordinates (chapter 21).
- A `handle_input()` function that already moves the character according to the D-pad; this chapter reuses it without changing its internal logic, and simply adds the camera on top of it.

1. The starting point: a large map with no camera

We start from a project that already displays a large map and moves a sprite over it: the resources are declared, the map is loaded with `MAP_create()`, the sprite exists, and `handle_input()` already moves the character with the D-pad. The only thing missing is, precisely, the subject of this chapter. If this starting point is compiled and run as-is, the character can roam across the whole map, but the camera never moves: as soon as the character gets far enough from its starting point, it walks off the screen with nothing following it.

2. Organization: function prototypes

Before writing the camera, it's worth making an organizational adjustment to the existing code: move the entire body of `handle_input()` to the end of the file, leaving only the prototype in its original place (the function's first line, ending in a semicolon, with no body). Unlike more modern languages, the C compiler does not scan the rest of the file looking for a function used before its definition; without a prototype declared before the point where it's used, it simply refuses to compile. Organizing the file this way — prototypes at the top, full definitions at the bottom — is a common C convention that lets you read the program's main flow without constantly jumping to read the body of each function, something that becomes more valuable the more functions of your own you add, like the camera function this chapter is about to define:

```
static void handle_input();
static void camera();
```

3. Revisiting `fix32` for character movement

The second preparatory adjustment is changing the character's position variables (`link_x`, `link_y`) from `int` to `fix32`, as recommended in chapter 6 for any character position on a large map: it allows decimal values (a speed of 1.5 pixels per frame, instead of a whole number) and offers enough range for large maps. Since `SPR_addSprite()` and `SPR_setPosition()` expect integers rather than `fix32` values, every use of `link_x` or `link_y` inside those functions must first go through `F32_toInt()`:

```
fix32 link_x;
fix32 link_y;
fix32 link_vel = FIX32(1.5);

s16 current_camera_x;
s16 current_camera_y;
```

With this, the character now moves with decimal precision, although in this example the difference from whole-number movement isn't very noticeable. The change is made anyway

because `fix32` is the recommended type for any character or enemy position on a map, and it's worth getting used to using it from the start.

`current_camera_x` and `current_camera_y` are declared as `s16` instead of `int`: they take up half the memory and are more than enough to represent a position within a map of this size. This holds as long as the map doesn't exceed roughly 32,000 pixels per side, which is the limit of an `s16`; for maps larger than that threshold you'd need to go back to `int` (or `u32`, depending on the case).

4. The concept: the screen as a box that follows the character

Before writing code, it's worth having the general idea clear. The screen — 320 by 224 pixels in H40 mode — can be thought of as a box that moves across the whole map, trying to keep the character near its center. Inside that box the character has its own coordinates (`link_x` and `link_y`, the top-left corner of its sprite), and the box itself has its own coordinates within the full map: this chapter calls those coordinates `camera_x` and `camera_y`.

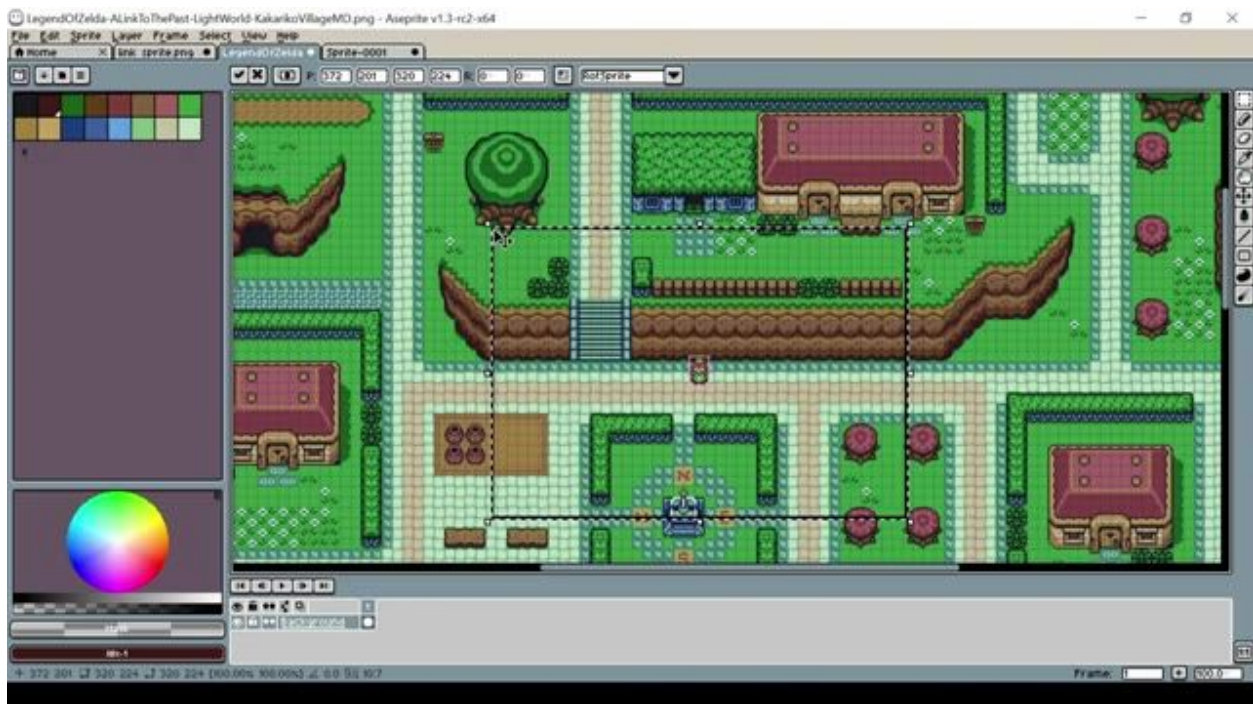


Figure 77: The camera concept as a box over the map

The box marks the visible area on screen within the full map, and the character has its own coordinates inside that box.

`current_camera_x` and `current_camera_y`, declared in the previous step, keep track of where the camera is at any given moment. Unlike a variable declared inside a function (which is destroyed as soon as the function ends and recreated the next time around), these need to persist from one frame to the next, so they have to be global, declared before `main()`.

The next step is to decide, via `#define` constants, how far from each edge of the screen the character has to be before the camera starts moving: one constant for each of the four edges, with starting values that can be tweaked later:


```
#define HOW_FAR_TO_LEFT_BEFORE_CAMERA_MOVES    140
#define HOW_FAR_TO_RIGHT_BEFORE_CAMERA_MOVES   140
#define HOW_FAR_TO_UP_BEFORE_CAMERA_MOVES      90
#define HOW_FAR_TO_DOWN_BEFORE_CAMERA_MOVES    90
```

5. The character's position on the map and on the screen

With the global variables and constants already in place, the rest of the code goes inside a new function, `camera()`. The first step is to convert the character's `fix32` position into a plain integer, since the comparisons and calculations that follow don't need decimal precision. Unlike `current_camera_x/current_camera_y`, this variable can — and should — be local to the function: it's only needed during this same frame, so it isn't worth having it permanently occupy RAM the way a global variable does.

```
s16 player_x_map_integer = F32_toInt(link_x);
s16 player_y_map_integer = F32_toInt(link_y);
```

Now we need to figure out where the character is relative to the screen, rather than to the full map: it's enough to subtract the camera's current position from the character's position on the map, since that is precisely the coordinate of the screen's top-left corner within the map:

```
s16 player_x_position_on_screen = player_x_map_integer - current_camera_x;
s16 player_y_position_on_screen = player_y_map_integer - current_camera_y;
```

6. Calculating the camera's new position

With the character's on-screen position already calculated, the next step is to decide whether the camera needs to move, by comparing that position against the four constants defined earlier. If the character has crossed the screen's right-hand limit, the camera's new position is calculated by subtracting that same limit distance from the character's position on the map, so that the character ends up exactly at the allowed edge instead of continuing to move further away. The same reasoning, with the comparison sign flipped, covers the left side; and if neither case applies, the camera simply keeps its current position:

```
s16 new_camera_x;
s16 new_camera_y;

if (player_x_position_on_screen > HOW_FAR_TO_RIGHT_BEFORE_CAMERA_MOVES)
    new_camera_x = player_x_map_integer - HOW_FAR_TO_RIGHT_BEFORE_CAMERA_MOVES;
else if (player_x_position_on_screen < HOW_FAR_TO_LEFT_BEFORE_CAMERA_MOVES)
    new_camera_x = player_x_map_integer - HOW_FAR_TO_LEFT_BEFORE_CAMERA_MOVES;
else
    new_camera_x = current_camera_x;
```

That same block of three cases (edge crossed on one side, on the other side, or no change) is repeated in an entirely analogous way for the vertical axis, with one caveat worth pointing out: since the screen's Y coordinate grows downward (0 at the top, 224 at the bottom), the condition that triggers downward movement uses a "greater than," which at first glance might seem counterintuitive if you think in terms of "up" and "down" rather than in terms of the numeric coordinate:

```

if (player_y_position_on_screen > HOW_FAR_TO_DOWN_BEFORE_CAMERA_MOVES)
    new_camera_y = player_y_map_integer - HOW_FAR_TO_DOWN_BEFORE_CAMERA_MOVES;
else if (player_y_position_on_screen < HOW_FAR_TO_UP_BEFORE_CAMERA_MOVES)
    new_camera_y = player_y_map_integer - HOW_FAR_TO_UP_BEFORE_CAMERA_MOVES;
else
    new_camera_y = current_camera_y;

```

7. Applying the scroll: MAP_scrollTo() only when needed

Now that the camera's new position has been calculated, we still need to decide whether it actually changed from the previous position: if the character hasn't moved far enough to cross any of the limits, the camera doesn't need to move, and calling MAP_scrollTo() every frame unnecessarily would be a waste of CPU work. The comparison uses the logical equality operator (two equal signs, as opposed to assignment with a single one) and the logical "or" operator (two vertical bars), since a change on either axis is enough to justify a new scroll:

```

if ((current_camera_x != new_camera_x) || (current_camera_y != new_camera_y)) {
    current_camera_x = new_camera_x;
    current_camera_y = new_camera_y;

    MAP_scrollTo(level_map, current_camera_x, current_camera_y);
}

```

If the condition is true, the code updates the global variables with the new values (so that next frame's comparison has the correct reference) and only then calls MAP_scrollTo(), the same function already used in chapter 11, this time with the freshly calculated coordinates.

8. First results, and the adjustment to center the character

With the camera function already written, we still need to add the call inside the main loop, right after handle_input():

```

while(1) {
    handle_input();
    camera();

    SPR_update();
    SYS_doVBlankProcess();
}

```

With that adjustment the camera now moves following the character, but there's still a problem left: the sprite is still being drawn at its position within the full map, instead of at its position relative to the current screen. The solution is the same subtraction already used to calculate player_x_position_on_screen: SPR_setPosition() needs the character's position minus the camera's position, not the absolute position on the map. This line is moved from the main loop to the end of the camera() function itself, since that's where it makes sense to update it:

```

SPR_setPosition(link_sprite, F32_toInt(link_x) - new_camera_x, F32_toInt(link_y) -
↪ new_camera_y);

```

Using new_camera_x/new_camera_y instead of current_camera_x/current_camera_y works in either case: when the camera actually moved, both variables are already synchronized by the

assignment in the previous step; when it didn't move, `new_camera_x/new_camera_y` already held the same value as `current_camera_x/current_camera_y` from the edge-comparison block. `SPR_setPosition()` takes s16 coordinates, as seen in chapter 21, and this subtraction between two already-calculated integers fits that type directly.

9. Keeping the camera from leaving the map

With the character now correctly centered, there's still a visible problem when approaching any edge of the map: the camera keeps scrolling beyond the map itself, showing repeated tiles or visual artifacts in the areas that fall outside the actual bounds.



Figure 78: The camera scrolling past the actual edge of the map

Without bounding the camera, the graphics repeat incorrectly in the area that no longer corresponds to any valid map tile.

The solution is to clamp `new_camera_x` and `new_camera_y` to a valid range, which requires four new constants: the horizontal and vertical screen resolution, and the width and height of the full map (1024 by 1024 pixels in this particular map, although the exact value depends on each project):

```
#define HORIZONTAL_RESOLUTION 320
#define VERTICAL_RESOLUTION   224
#define MAP_WIDTH             1024
#define MAP_HEIGHT            1024
```

With those constants defined, the left and top limits are simple: if `new_camera_x` or `new_camera_y` fall below 0, they're set to 0. The right and bottom limits require a small calculation: the camera can't exceed the total width (or height) of the map minus the screen resolution, since otherwise it would show content beyond the map itself:

```

if (new_camera_x < 0)
    new_camera_x = 0;
else if (new_camera_x > MAP_WIDTH - HORIZONTAL_RESOLUTION)
    new_camera_x = MAP_WIDTH - HORIZONTAL_RESOLUTION;

if (new_camera_y < 0)
    new_camera_y = 0;
else if (new_camera_y > MAP_HEIGHT - VERTICAL_RESOLUTION)
    new_camera_y = MAP_HEIGHT - VERTICAL_RESOLUTION;

```

This block goes after calculating `new_camera_x/new_camera_y` in step 6, and before the comparison against `current_camera_x/current_camera_y` in step 7. With this, the camera now respects the map's four edges: upon reaching any corner, it stops scrolling instead of showing content outside the bounds.

10. Keeping the character itself from leaving the screen

With the camera now bounded, one last detail remains: even though the map no longer scrolls beyond its limits, the character can still visually keep walking off the screen in those same edge areas, because nothing stops it from continuing to move even after the camera has stopped following it. The solution is analogous to the camera's own, but applied to `link_x` and `link_y`, and using the character sprite's width and height (two additional constants) instead of the screen resolution:

```

#define PLAYER_WIDTH    32
#define PLAYER_HEIGHT   32

```

```

if (link_x < 0)
    link_x = 0;
else if (link_x > FIX32(MAP_WIDTH - PLAYER_WIDTH))
    link_x = FIX32(MAP_WIDTH - PLAYER_WIDTH);

if (link_y < 0)
    link_y = 0;
else if (link_y > FIX32(MAP_HEIGHT - PLAYER_HEIGHT))
    link_y = FIX32(MAP_HEIGHT - PLAYER_HEIGHT);

```

Since `link_x` and `link_y` are `fix32`, the limits are expressed with `FIX32()` so that comparisons and assignments happen on the same scale. This block is placed at the beginning of the `camera()` function, before calculating anything related to the camera itself, since conceptually it corrects the character's position before the rest of the function uses it for any other calculation. With this adjustment, both the camera and the character itself respect the map's limits in all four directions.

The complete function

Putting the ten steps above together in order, `camera()` ends up looking like this:

```

static void camera() {
    // Correct the character if it went past the map's limits
    if (link_x < 0)
        link_x = 0;
    else if (link_x > FIX32(MAP_WIDTH - PLAYER_WIDTH))

```

```

    link_x = FIX32(MAP_WIDTH - PLAYER_WIDTH);

    if (link_y < 0)
        link_y = 0;
    else if (link_y > FIX32(MAP_HEIGHT - PLAYER_HEIGHT))
        link_y = FIX32(MAP_HEIGHT - PLAYER_HEIGHT);

    // Character's integer position on the map
    s16 player_x_map_integer = F32_toInt(link_x);
    s16 player_y_map_integer = F32_toInt(link_y);

    // Character's position relative to the screen
    s16 player_x_position_on_screen = player_x_map_integer - current_camera_x;
    s16 player_y_position_on_screen = player_y_map_integer - current_camera_y;

    // New camera position based on the four edges
    s16 new_camera_x;
    s16 new_camera_y;

    if (player_x_position_on_screen > HOW_FAR_TO_RIGHT_BEFORE_CAMERA_MOVES)
        new_camera_x = player_x_map_integer - HOW_FAR_TO_RIGHT_BEFORE_CAMERA_MOVES;
    else if (player_x_position_on_screen < HOW_FAR_TO_LEFT_BEFORE_CAMERA_MOVES)
        new_camera_x = player_x_map_integer - HOW_FAR_TO_LEFT_BEFORE_CAMERA_MOVES;
    else
        new_camera_x = current_camera_x;

    if (player_y_position_on_screen > HOW_FAR_TO_DOWN_BEFORE_CAMERA_MOVES)
        new_camera_y = player_y_map_integer - HOW_FAR_TO_DOWN_BEFORE_CAMERA_MOVES;
    else if (player_y_position_on_screen < HOW_FAR_TO_UP_BEFORE_CAMERA_MOVES)
        new_camera_y = player_y_map_integer - HOW_FAR_TO_UP_BEFORE_CAMERA_MOVES;
    else
        new_camera_y = current_camera_y;

    // The camera can't go past the map
    if (new_camera_x < 0)
        new_camera_x = 0;
    else if (new_camera_x > MAP_WIDTH - HORIZONTAL_RESOLUTION)
        new_camera_x = MAP_WIDTH - HORIZONTAL_RESOLUTION;

    if (new_camera_y < 0)
        new_camera_y = 0;
    else if (new_camera_y > MAP_HEIGHT - VERTICAL_RESOLUTION)
        new_camera_y = MAP_HEIGHT - VERTICAL_RESOLUTION;

    // Scroll the map only if the camera position changed
    if ((current_camera_x != new_camera_x) || (current_camera_y != new_camera_y)) {
        current_camera_x = new_camera_x;
        current_camera_y = new_camera_y;

        MAP_scrollTo(level_map, current_camera_x, current_camera_y);
    }

    // Reposition the sprite relative to the camera, not the full map

```

```

    SPR_setPosition(link_sprite, F32_toInt(link_x) - new_camera_x,
↪    F32_toInt(link_y) - new_camera_y);
}

```

11. Adjusting the feel of the camera

Now that the function is working without errors, changing the four `HOW_FAR_TO_*_BEFORE_CAMERA_MOVES` constants is the simplest way to adjust how strictly the camera keeps the character centered: smaller values keep it almost always at the exact center of the screen, while larger values give it more freedom of movement before the camera reacts, at the cost of a character that's less centered at any given moment. How tight the camera should be is a design decision for each particular game, not a fixed technical rule: more elaborate cameras, for example, push the view ahead in the direction the character is facing to show more of the path forward, although that behavior can feel uncomfortable if the camera shifts too abruptly every time the character changes direction.

The complete set of constants governing the camera's behavior are the four screen margins, the horizontal and vertical resolution, and the dimensions of the map and the sprite. Adjusting just the four margins is enough to completely change the feel of the camera without touching the rest of the logic.

12. Adapting the camera to H32 resolution mode

To wrap up, let's confirm that the same camera function works just as well in H32 mode (256 pixels wide, seen in chapter 10). The resolution change itself is still the same line of code as always:

```
VDP_setScreenWidth256();
```

The only additional adjustment needed is updating `HORIZONTAL_RESOLUTION` to 256 and the `HOW_FAR_TO_LEFT_BEFORE_CAMERA_MOVES/HOW_FAR_TO_RIGHT_BEFORE_CAMERA_MOVES` constants so they still represent, roughly, half of the new horizontal resolution, so that the character stays centered in this mode too. The rest of the `camera()` function needs no changes at all: all the limit and scroll logic is already independent of the specific resolution, because it relies on constants rather than on raw numbers written directly into the code.

Review exercises

1. Explain the difference between a character's position "on the map" and its position "on screen," and how the latter is calculated from the former.
2. Why is `player_x_map_integer` declared as a local variable inside `camera()` rather than as a global variable, unlike `current_camera_x`?
3. Explain what the check `if ((current_camera_x != new_camera_x) || (current_camera_y != new_camera_y))` is for before calling `MAP_scrollTo()`, and what problem it avoids.
4. A teammate implements the limits that keep the camera from leaving the map, but not the limits that keep the character itself from leaving the screen. Describe what incorrect behavior would be observed when approaching any edge of the map.
5. Explain why `SPR_setPosition()` must receive the character's position minus the camera's position, rather than the character's position on the map directly.

Chapter 25 — Parallax Scrolling Over a Camera

Combining a parallax background with the camera of a large map: inverted parallax, multiplication versus subtraction, and the real limit of a repeating image

Chapter 9 introduced basic parallax scrolling: two backgrounds moving automatically at different speeds, with no relation to any character. Chapter 24, for its part, showed how to scroll a large map based on a character's position, but without any additional background behind it. This chapter combines both ideas: a parallax background that scrolls according to the very same camera that already follows the character over the main map, with two new complications that did not appear in either chapter on its own.

Prerequisites

Before continuing, it's a good idea to have on hand:

- Chapter 24 ("2D Camera") already worked through: the `camera()` function, the `current_camera_x/current_camera_y` variables, and the map and character limits.
- Chapter 9 ("Scrolling and Parallax Scrolling"): `VDP_setScrollingMode()` and the use of two background planes (`BG_A` and `BG_B`).
- Chapter 7 ("How to Display Backgrounds on the Mega Drive"): the `IMAGE` resource and `VDP_drawImageEx()`.

1. Starting point: two different graphics resources

The example in this chapter combines a foreground map (the same kind of `TILESET + MAP` resource already seen in Chapter 11) with a distant background image. The foreground map measures 6176 pixels wide by 352 pixels tall — considerably taller than the screen's 224 pixels, so as to also demonstrate vertical scrolling — while the background image respects the already familiar limit of an `IMAGE` resource: 512×256 pixels at most.

The starting point is, quite simply, the finished code from the camera chapter, with the graphics resources swapped out and the map and sprite width/height constants updated to the new values. On top of that code, it's worth applying two minor adjustments as well:

Use `s16` instead of `int` for the coordinates. As explained in Chapter 6, it's always best to use the smallest type that will do the job. This example's map, at 6176 pixels wide, remains well below the roughly 32,000-pixel limit of an `s16`, so declaring the camera coordinates as `s16` instead of `int` requires less CPU work and less ROM, without losing any useful range.

Simplify single-statement `ifs`. In C, when the body of an `if` or an `else if` contains a single statement, the braces delimiting that body are optional, and removing them to leave the state-

ment on the same line changes nothing about the program's behavior. Blocks that execute more than one statement (for example, within input handling, where it's often necessary both to move the character and, eventually, to change its animation) still need braces, since there they are indeed mandatory.

2. Adding the background image: palettes and planes

With the foreground map already on BG_A using PAL1 for its colors — and not PAL0, precisely to highlight a useful effect worth recognizing: the first color of any palette other than 0 is always transparent, so if a background ever shows up as an unexpected solid color (pink, say) without having loaded any new palette, that's a sign that plane is displaying transparency over the VDP's background color instead of its actual colors — adding the distant background image requires its own call to `PAL_setPalette()`, this time with PAL0:

```
PAL_setPalette(PAL0, bg_far.palette->data, DMA);
PAL_setPalette(PAL1, map_tileset.palette->data, DMA);
PAL_setPalette(PAL2, player_sprite.palette->data, DMA);
```

With the palette already assigned, displaying the image on screen uses the same `VDP_drawImageEx()` function already seen in Chapter 7, this time targeting the BG_B plane (the back plane, behind the foreground map that occupies BG_A):

```
VDP_drawImageEx(BG_B, &bg_far, TILE_ATTR_FULL(PAL0, FALSE, FALSE, FALSE, ind), 0,
↪ 0, TRUE, TRUE);
```

Compiling at this point, the foreground map scrolls normally thanks to the camera already built in Chapter 24, and the background image shows all its colors correctly, but is completely static: without additional code, an image loaded with `VDP_drawImageEx()` doesn't scroll on its own. This would be exactly the desired behavior for a fixed, single-screen background, but it isn't the goal of this chapter.

3. Enabling background scrolling

The first step toward making the background scrollable is, just as in Chapter 9, to declare the VDP's scrolling mode with `VDP_setScrollingMode()`, choosing full-plane scrolling — rather than per-tile or per-line scrolling, two more advanced modes covered in Part II that aren't available for a MAP resource, only for IMAGE — for both the horizontal and vertical axes:

```
VDP_setScrollingMode(HSCROLL_PLANE, VSCROLL_PLANE);
```

With the scrolling mode already chosen, two new variables are needed — `b_hscroll` and `b_vscroll`, also declared as `s16` — to store how much to shift the background on each axis, along with two calls to `VDP_setHorizontalScroll()` and `VDP_setVerticalScroll()` inside the camera function, in the same block where `MAP_scrollTo()` is already called for the foreground map:

```
VDP_setHorizontalScroll(BG_B, b_hscroll);
VDP_setVerticalScroll(BG_B, b_vscroll);
```

4. An unexpected problem: inverted parallax

The simplest way to give `b_hscroll` and `b_vscroll` some value is to assign them directly from `new_camera_x` and `new_camera_y`, the same coordinates already used to scroll the foreground

map. Compiling with that change, vertical scrolling behaves as expected (background and foreground moving exactly the same, with no depth effect yet), but horizontal scrolling reveals a problem: the background moves in the opposite direction from what's expected, an inverted and visually unpleasant parallax effect.

The cause is conceptual: `MAP_scrollTo()` expects the camera's position in order to scroll the map in the correct direction, but `VDP_setHorizontalScroll()` interprets its parameter differently for a background image, so assigning it the same camera coordinate directly produces the opposite of the intended effect.

5. Fixing the direction: multiplication versus subtraction

The most intuitive solution is to flip the sign of `new_camera_x` before assigning it to `b_hscroll`, by multiplying it by `-1`:

```
b_hscroll = new_camera_x * -1;
```

This solution works, but it's worth keeping an underlying issue in mind: the Mega Drive's CPU (a Motorola 68000, which, as explained in Chapter 1, doesn't have a multiplication or division unit as fast as its basic arithmetic and logic operations) isn't especially quick at performing multiplications or divisions, so it's best to avoid them whenever a simpler alternative exists. In this case, subtracting the value from 0 achieves exactly the same result as multiplying by `-1`, without needing any multiplication at all:

```
b_hscroll = 0 - new_camera_x;
```

6. Depth with powers of two: bit shifting instead of dividing

With the direction now fixed, the next step is to make the background scroll more slowly than the foreground, to create a sense of depth: the farther away a background element is supposed to be, the more slowly it should move relative to the character. The most intuitive approach is to divide `new_camera_x` by some number — 2 for half speed, 3 for a third, and so on — but division, like multiplication, is a relatively costly operation for this CPU. The recommended alternative is bit shifting: shifting all the bits of a number one position to the right is exactly equivalent to dividing it by 2; shifting two positions is equivalent to dividing by 4; three positions, by 8; and so on, always in powers of 2. Since the CPU works in binary, shifting bits is a much simpler and faster operation than a generic division:

```
b_hscroll = (0 - new_camera_x) >> 1; // half speed
```

Trying out different shift amounts is a quick way to fine-tune the sense of depth until finding the one that best matches how distant that landscape element is supposed to be. For a background of distant mountains and forest, shifting three positions (dividing by 8) on both axes tends to be a good starting point:

```
b_hscroll = (0 - new_camera_x) >> 3;  
b_vscroll = (0 - new_camera_y) >> 3;
```

This value for `b_vscroll`, however, isn't final yet: the next section adjusts it again to solve a different problem with vertical scrolling.

7. A problem when scrolling vertically: the background repeating

With horizontal parallax now working correctly, scrolling vertically reveals a different problem: at some point, the background image starts repeating vertically, showing its top part again before finishing showing the bottom. This is a visually incorrect effect for a platformer, though it might be acceptable in a vertical shooter where a repeating background makes sense. The cause is the difference between the image's height (256 pixels, the maximum allowed for an IMAGE resource) and the screen's height (224 pixels): a margin of only 32 pixels before the vertical scroll exceeds the image's actual height and starts showing it again from the beginning.

The solution is to clamp `b_vscroll` to that 32-pixel range, using the same logic already used in Chapter 24 to keep the camera from going outside the map: a lower limit of 0, and an upper limit of 32 (the difference between the image's height and the screen's height):

```
if (b_vscroll < 0)
    b_vscroll = 0;
else if (b_vscroll > 32)
    b_vscroll = 32;
```

With this limit in place, the background stops repeating vertically no matter how far the camera scrolls up or down, although — as was already the case with horizontal scrolling of a repeating map — the background keeps repeating continuously along the horizontal axis, which is perfectly correct for a background image meant to repeat along that axis.

This lower-limit check might turn out to be unnecessary if the camera's Y coordinate, which `new_camera_y` depends on, is already clamped to a minimum of 0 inside the `camera()` function from Chapter 24. Even so, keeping it costs nothing and leaves the code safe against future changes to that function.

8. Why not use two maps instead of a map and an image?

A reasonable question is why not use two MAP resources (one for each plane) instead of combining a MAP for the foreground with an IMAGE for the background, since in principle both could be scrolled with `MAP_scrollTo()`. The reason is twofold. On one hand, an IMAGE resource supports the per-tile or per-line scrolling modes seen in Part II of this manual, capable of achieving visual effects that a MAP cannot reproduce, although this particular chapter only used full-plane mode. On the other hand, and more importantly: a large map with no repetitions exists, above all, to define the level's playable design (what obstacles there are and where), while a background is purely decorative and doesn't need that same level of detail. A 512×256-pixel image that repeats indefinitely is usually more than enough for a background, without spending extra memory on a full map that was never going to provide any relevant gameplay information.

Review exercises

1. Explain why assigning `new_camera_x` directly to `b_hscroll` produces horizontal scrolling in the wrong direction, and what operation fixes it.
2. Why is bit shifting (`>>`) preferred over division for reducing the speed of a parallax background?
3. If `b_hscroll` is calculated as `(0 - new_camera_x) >> 2`, at what fraction of the foreground's speed is the background scrolling?

4. Explain why a background image that is 256 pixels tall needs its vertical scrolling clamped to a range of just 32 pixels when the screen is 224 pixels tall.
5. Mention at least one reason why, in most cases, it's better to use an IMAGE resource for the background instead of a second MAP resource.

Chapter 26 — Transparency with Shadow and Highlight

The Mega Drive has no genuine way of blending colors to achieve transparency, the way more modern consoles can with alpha layers. What the VDP does have is a special mode called **Shadow and Highlight**, which darkens or lightens certain colors on screen in a controlled way. Many commercial games took advantage of this mode to simulate transparency, lighting, shadows, and ghosts, even though it isn't real transparency in any technical sense.

It's a feature with a reputation for being esoteric and hard to understand, but the programming side of it is actually quite simple: enabling the mode takes a single line of code. The real difficulty lies in understanding the particular rules that govern its behavior — which palette is affected, which colors within that palette, and how it interacts with tile priority — so you can make the most of it without surprises. This chapter covers those rules through a small practice project: a ghost that can turn translucent against the background of a haunted kitchen.

Prerequisites

Before continuing, it helps to be clear on two concepts already covered in this manual:

- The concept of the Mega Drive's color palettes (PAL0 through PAL3, with the first color of each palette reserved as transparent), covered in Chapter 2.
- The tile priority parameter (TILE_ATTR_FULL / TILE_ATTR), covered in Chapter 15.

1. Starting point: a background and two sprites

The project for this chapter starts out already with a haunted kitchen background and two sprites loaded in `resources.res`: a ghost (`spectre_sprite`) and a sprite meant purely as a visual palette reference (`sh_palette_sprite`), which is nothing more than a strip of colors drawn specifically to clearly show the effect on each individual color. Handling both sprites on screen and the D-pad controls to move them are already taken care of beforehand, so the focus of this chapter is entirely on palette design and the effect itself.

The project's `resources.res` file, with the kitchen background and the two starting sprites: the ghost and the reference palette strip.

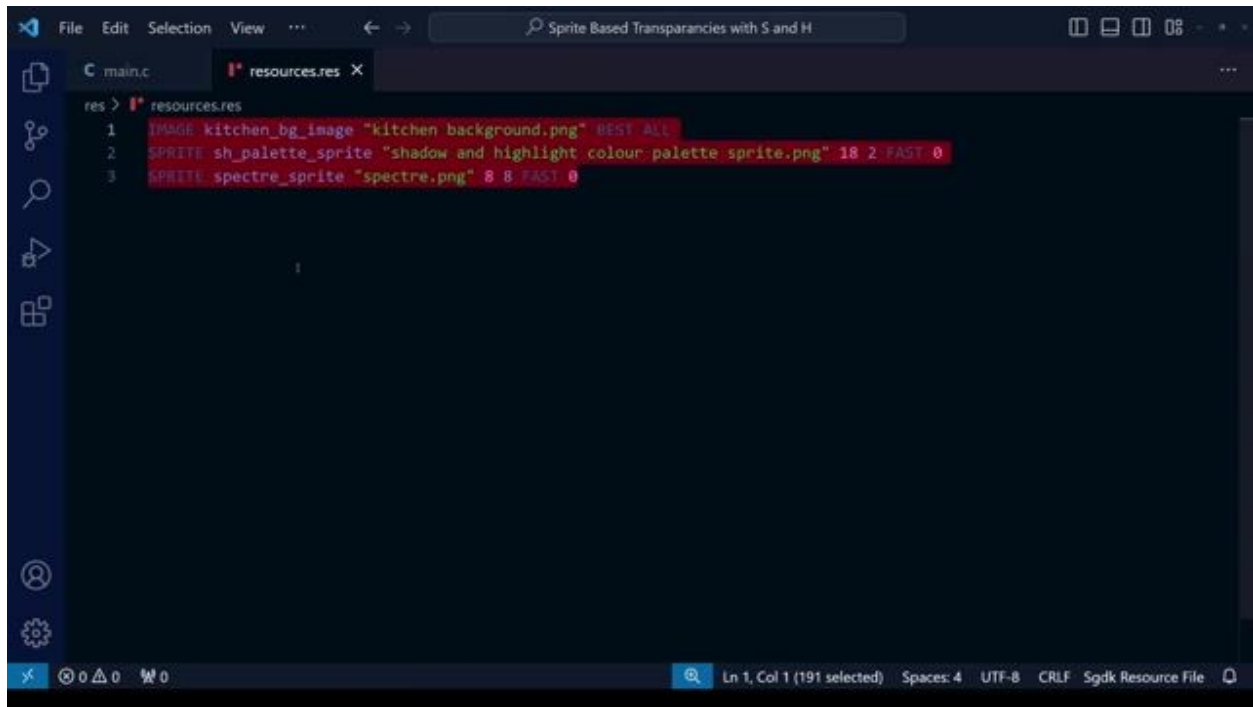


Figure 79: resources.res with the kitchen background and the starting sprites

2. The key palette: how to design a sprite for Shadow and Highlight

The most important aspect of this chapter isn't the code itself, but how each sprite's color palette is designed. The first color of the palette, as always on the Mega Drive, is ignored and acts as the sprite's transparent background color (it's a good idea to draw it in a bright, eye-catching color like blue, just so it's clear where the actual artwork ends during editing). The next thirteen colors are normal colors, which behave exactly the same as in any palette without Shadow and Highlight enabled.

It's the last two colors of the palette, color 15 and color 16, that have special behavior:

- Color 15 is used as the **highlight**.
- Color 16 is used as the **shadow**.

What matters about these two colors isn't the hue assigned to them in the palette, but only their **position** (15 and 16); they can, in principle, be any color at all, although it's worth picking hues that make the effect easier to visualize while testing.

The reference palette strip: the first color (blue) is transparent, the next thirteen are normal colors, and the last two are the positions reserved for highlight and shadow.

3. Placeholder fill colors in the ghost sprite

In the ghost sprite, palette positions 15 and 16 can initially be filled with any flat color as a placeholder, simply to occupy those two indices while the rest of the sprite is being drawn. In this project, the ghost's white ends up taking the highlight slot, and the black of its outlines and its own shadows takes the shadow slot. The only thing that matters at this point is the position of those colors within the palette, not their apparent hue during the drawing process.

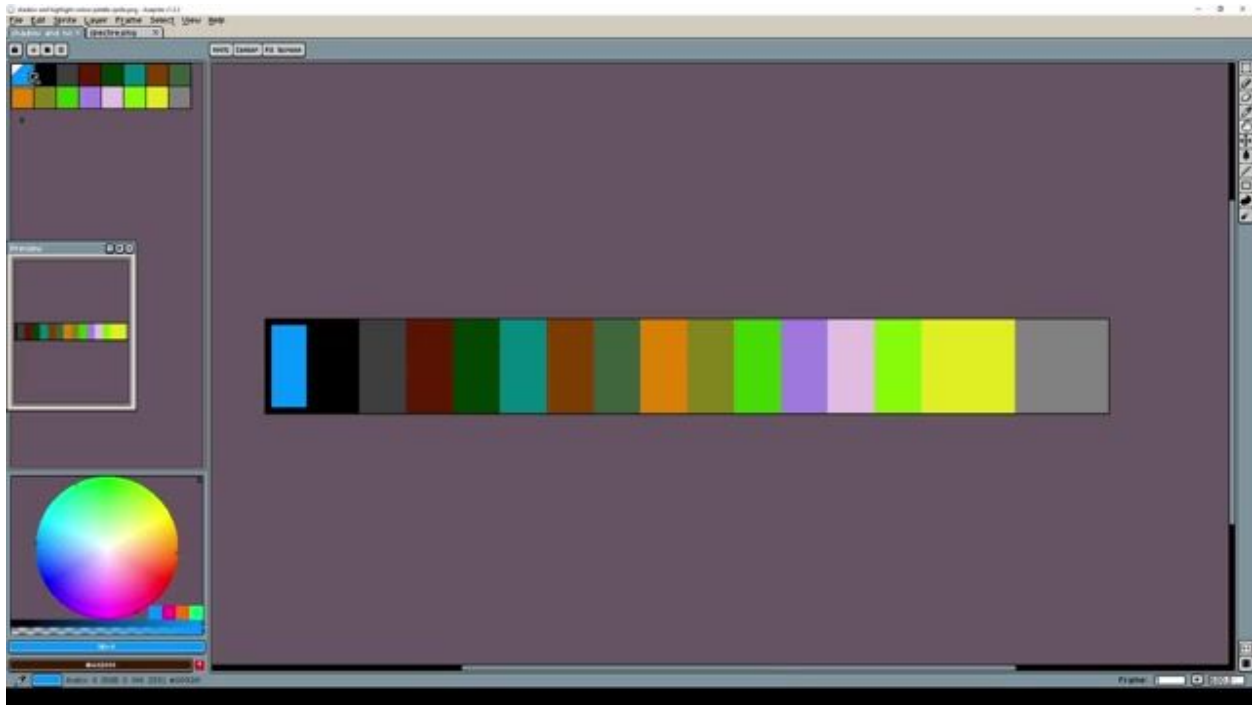


Figure 80: The reference palette strip, with the transparent color, the normal colors, and the highlight and shadow positions

4. Compiling with the mode disabled

Before turning anything on, it's worth compiling the project as it stands, to establish a clear point of comparison. Since Shadow and Highlight mode is disabled by default on the Mega Drive, the result is as expected: both the background and both sprites look normal, with no special effect yet, even though the palette is already set up for the mode.

To test these effects properly, it's best to do so, as much as possible, on real hardware or on an emulator that reproduces the mode accurately — Kega Fusion is a reasonable option — since several popular emulators don't simulate Shadow and Highlight behavior with full fidelity to the original hardware. Chapter 4 reviews various emulator options and their quirks.

5. Enabling Shadow and Highlight with a single line

Enabling the mode itself is extremely simple: just a single line,

```
VDP_setHiLightShadow(TRUE);
```

placed inside `main()` before entering the game's main loop. Passing it `FALSE` disables it again, which is useful for turning the effect on and off dynamically during gameplay, for example when entering and leaving a dark room. With that line added, the result changes immediately upon compiling: the last two colors of any palette that has them defined start showing the highlight and shadow effect.

6. Tile priority: how to keep an element's original color

Enabling Shadow and Highlight globally darkens, by default, everything drawn with low priority (FALSE), regardless of whether that element actually has highlight or shadow colors defined in its palette. For a background or a sprite to keep its original colors without being darkened, it needs to be explicitly marked with high priority (TRUE): for the background, within the priority parameter of `VDP_drawImageEx()` (or of `TILE_ATTR_FULL`, depending on how the tile is built); for sprites, within `TILE_ATTR()` when calling `SPR_addSprite()`.

This graphics mode has a reputation for having caused more than one headache during the original development of Mega Drive and Mega CD games: it's said, without any official confirmation, that some Mega CD release shipped with a scene meant to be shown in broad daylight instead appearing abnormally dark, as if Shadow and Highlight mode had been left enabled by mistake on elements without the correct priority. Whether or not that particular anecdote is accurate, it serves as a reminder that this parameter is worth checking carefully before considering a scene finished.

Going back to the chapter's project, the priority parameter is set to TRUE for both the background and both sprites, so they keep their original colors despite having Shadow and Highlight enabled globally:

```
// Background with high priority: keeps its original colors
VDP_drawImageEx(BG_B, &kitchen_image,
    TILE_ATTR_FULL(PAL0, TRUE, FALSE, FALSE, TILE_USER_INDEX), 0, 0, FALSE, TRUE);

// Sprites with high priority: also keep their original colors
SPR_addSprite(spectre, x, y, TILE_ATTR(PAL2, TRUE, FALSE, FALSE));
SPR_addSprite(sh_palette, x2, y2, TILE_ATTR(PAL3, TRUE, FALSE, FALSE));
```

Compiling again, the screen regains its normal brightness on all the elements marked with high priority, and now the highlight and shadow effect can be clearly seen on the reference palette strip, since it does have valid colors defined in positions 15 and 16.

With the background and sprites now marked with high priority, the colors return to normal and the highlight and shadow effect is clearly visible on the reference palette strip.

7. A key restriction: the effect only works on the fourth palette (PAL3)

At this point, the ghost sprite shows no highlight or shadow effect at all, even though it also has colors defined in positions 15 and 16 of its palette. The reason is a fundamental hardware restriction of Shadow and Highlight mode: **the effect only takes place on the Mega Drive's fourth palette, PAL3**. Looking back at the project's code, the reference palette strip (the one that does show the effect) is assigned to PAL3, while the background uses PAL0, PAL1 isn't used, and the ghost uses PAL2 — which is why it shows no effect.

This restriction is neither negotiable nor configurable: it's a limitation of the VDP itself, and it's worth keeping in mind from the design stage of a project, since it effectively reserves an entire palette (PAL3) for any element that needs this effect.

8. Swapping which sprite gets the effect

To confirm this restriction in practice, it's illustrative to swap the palette assignments between the two sprites: the ghost switches to using PAL3, and the reference strip switches to using

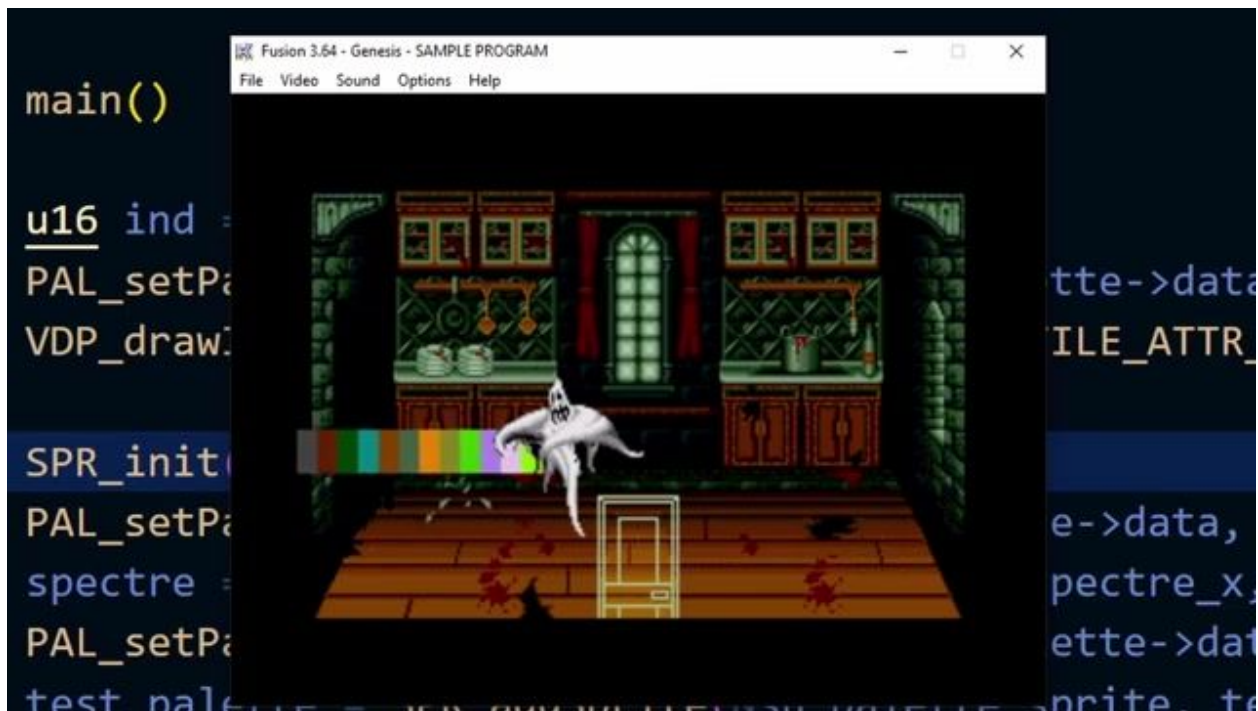


Figure 81: The screen with high priority on the background and sprites, showing the effect on the reference strip

PAL2. This means updating both the call to `PAL_setPalette()` (which copies the actual color data to the corresponding hardware palette) and the palette parameter within `TILE_ATTR()` in each sprite's call to `SPR_addSprite()`, since both must match:

```
PAL_setPalette(PAL2, sh_palette_sprite.palette->data, DMA);
PAL_setPalette(PAL3, spectre_sprite.palette->data, DMA);

SPR_addSprite(spectre, x, y, TILE_ATTR(PAL3, TRUE, FALSE, FALSE));
SPR_addSprite(sh_palette, x2, y2, TILE_ATTR(PAL2, TRUE, FALSE, FALSE));
```

With the palettes now swapped, the result is reversed exactly as expected: the reference palette strip loses the effect and goes back to showing normal colors, while the ghost, now on PAL3, clearly shows highlight and shadow in the corresponding areas of its artwork. The ghost can now be partially “switched off” whenever needed, giving the impression of transparency against the kitchen background.

After swapping palettes, the ghost — now on PAL3 — shows the highlight and shadow effect, while the reference strip, on PAL2, goes back to displaying normal colors.

9. An additional limitation: which sprite covers which

There's an important limitation left in the per-sprite transparency effect: if two sprites overlap on screen, the one drawn “on top” completely covers the one underneath, with no real color blending between them. It isn't possible, for instance, for a semi-transparent sprite to show through it whatever is on another sprite placed behind it; drawing order is decided solely by the order in which each sprite was added with `SPR_addSprite()` in the code, and you can confirm this by reversing that order to see how it changes which sprite ends up in front.

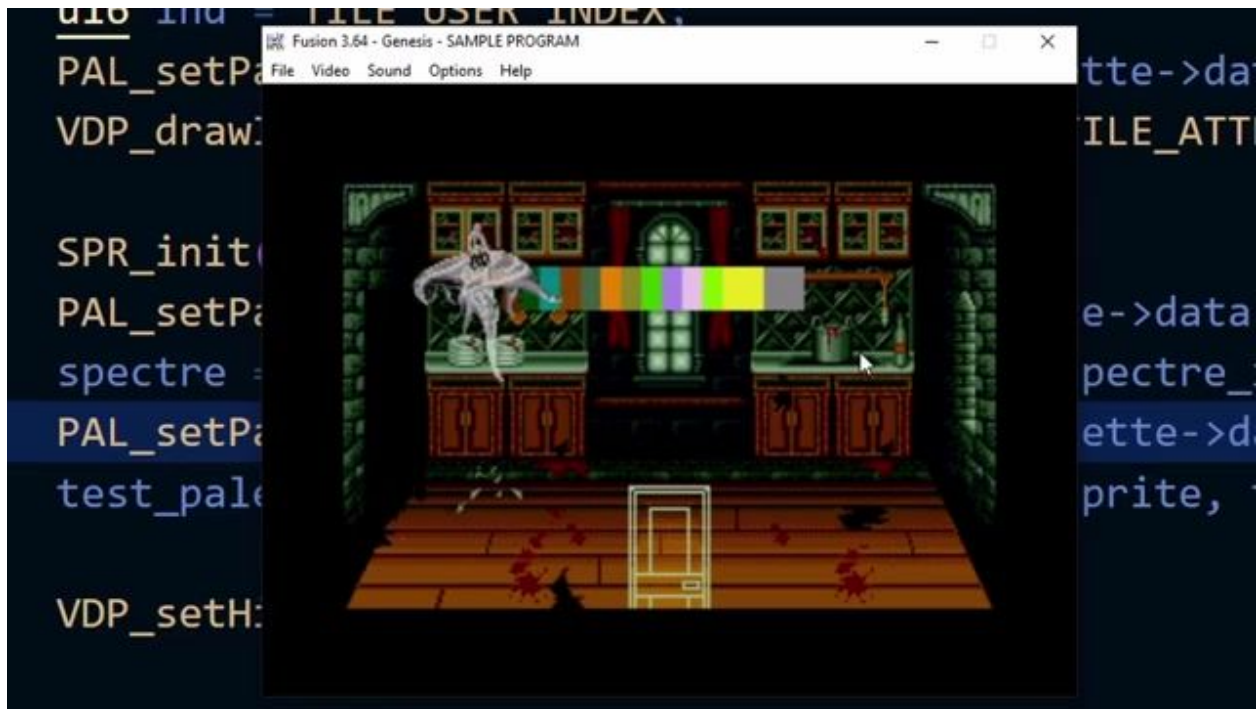


Figure 82: The ghost on PAL3 with the effect active, and the reference strip on PAL2 with normal colors

SGDK does offer ways to reorder sprites dynamically at runtime — something essential, for example, in a beat 'em up in the style of *Streets of Rage*, where drawing order depends on each character's vertical position on screen — but that topic is revisited later in this manual.

Review exercises

1. Explain specifically what the colors in positions 15 and 16 of a palette set up for Shadow and Highlight represent, and what happens with the thirteen colors before them.
2. Why did the ghost initially show no highlight or shadow effect, despite having colors defined in the correct positions of its palette?
3. Explain the dual role of the tile priority parameter (TILE_ATTR) in relation to Shadow and Highlight, and why that can cause complications in a real project.
4. What's the relationship between marking an element with low priority and the risk of an entire scene appearing darkened by mistake? Relate your answer to the restriction described in section 6.
5. Describe the limitation of the per-sprite transparency effect when two sprites overlap on screen, and what determines which of the two ends up in front.

Chapter 27 — Color Transparency Without Shadow and Highlight

Flicker, mesh, and horizontal flip: three ways to simulate transparency in any color

Chapter 26 showed how to use the VDP's Shadow and Highlight mode (covered in detail in Chapter 2) to simulate transparency, but that mode has two important limitations. The first is about color: Shadow and Highlight can only darken (toward black) or lighten (toward white) existing colors, so if you need yellow, green, red, or any other colored transparency, that mode simply won't do the job. The second is about overlap: as already shown, if a sprite with the effect is drawn on top of another one, it completely covers it instead of casting a shadow or highlight over it.

This chapter covers three alternative techniques that don't rely on Shadow and Highlight and that let you simulate transparency in any color: flicker, the mesh pattern, and a variant of the mesh pattern that uses the sprite's horizontal flip to disguise its grainy look.



Figure 83: An example of transparency in a commercial Mega Drive game

Sonic the Hedgehog 2 using color transparency, the kind of effect this chapter aims to reproduce without relying on Shadow and Highlight.

Prerequisites

Before continuing, it helps to have:

- Read Chapter 26 (“Transparency with Shadow and Highlight”): this chapter builds on the same base project, removing the line that enabled that mode.
- An understanding of the basic data types (in particular u8), covered in Chapter 6.
- Knowledge of how to assign palettes to sprites and backgrounds using more than one palette at the same time, covered in Chapter 14.

1. Starting point: the same project, without Shadow and Highlight

The code in this chapter picks up the same background and the same two sprites (the ghost and the reference palette strip) from the previous chapter, with two differences: the background now uses two different palettes at once (the technique covered in Chapter 14), and the line `VDP_setHighlightShadow(TRUE)` is removed entirely, since none of the techniques in this chapter depend on that mode.

The ghost is declared before the palette strip, so it appears drawn on top of it. With Shadow and Highlight disabled, both sprites simply look like normal, opaque sprites, with no transparency effect yet.

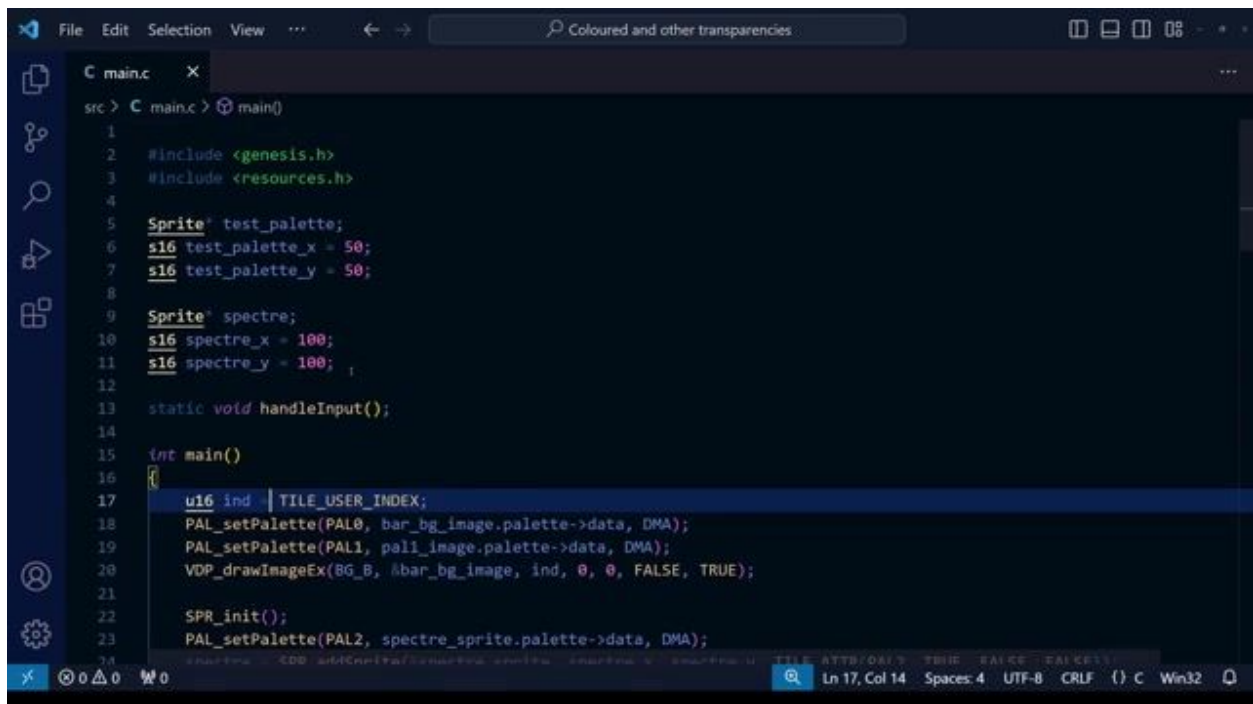


Figure 84: The background with two palettes and the two opaque sprites

The background with PAL0 and PAL1 active and the two sprites from the previous chapter, now without Shadow and Highlight and still without any transparency effect.

2. First technique: flicker

The idea behind flicker is simple: if a sprite is made visible and invisible very quickly, alternating frame by frame, the human eye perceives a blend between the sprite and whatever is behind it, giving the impression of transparency. This effect looks better on an actual CRT television than in a video recording or a screenshot, since a CRT's image persistence and refresh rate tend to smooth out the frame-by-frame alternation in a way that a camera or a modern high-frequency display doesn't always reproduce the same way.

Controlling that alternation requires a simple timer: a u8 variable (an unsigned 8-bit integer, more than enough since it only needs to alternate between 0 and 1), initialized to 0. It's worth organizing the flicker logic into a separate function, `transparencyFunction()`, declared with its prototype before `main()` and defined at the end of the file, so it doesn't get mixed in with the rest of the main loop.

Inside that function, the key piece is `SPR_setVisibility()`, an SGDK function that takes the sprite to affect and the desired state, using values of the `SpriteVisibility` type: `VISIBLE` to show it and `HIDDEN` to hide it. The full logic alternates between two branches of an `if`: if the timer is 0, the sprite is set visible and the timer is incremented by one; otherwise (that is, if it's 1), the sprite is set invisible and the timer is reset to 0. When writing this condition, be careful not to confuse `=` (assignment) with `==` (comparison), an easy mistake to make even though it's a basic concept, and one that can leave the timer in an unexpected state without the compiler warning you.

```
u8 timer = 0;

void transparencyFunction(void);

// ...

void transparencyFunction(void)
{
    if (timer == 0)
    {
        SPR_setVisibility(spectre, VISIBLE);
        timer++;
    }
    else
    {
        SPR_setVisibility(spectre, HIDDEN);
        timer = 0;
    }
}
```

With the function written, one last step remains that's easy to forget: calling it on every pass of the game's main loop, inside the `while(TRUE)`, alongside the other per-frame update calls (for example, reading input and `SPR_update()`).

The resulting behavior, frame by frame, is as follows: on the first frame the timer is 0, so the sprite is set visible (no apparent change, since it already was) and the timer moves to 1; on the second frame, since the timer is no longer 0, the sprite is set invisible; on the third frame the timer goes back to 0 and the cycle repeats, alternating visible and invisible on each successive frame.

The ghost alternating between visible and invisible frame by frame, letting the background and the palette strip behind it show through.



Figure 85: The ghost flickering between visible and invisible

It's worth keeping in mind that, when capturing or recording this effect to show it outside of a CRT television, the result may not look as convincing as on real hardware, where the frame-by-frame flicker tends to be perceived more smoothly and continuously. Even so, even in a recording, the transparency effect is clearly noticeable.

3. A second example: a semi-transparent lamp, *Streets of Rage* style

To test the same technique in a different context, it's worth switching to a scene inspired by the interiors of *Streets of Rage*, with a lamp sprite (simplified and reduced in size compared to the original game, where it's actually part of the background layer rather than a separate sprite) and a basic animation of a character walking behind it.

Before applying any effect, it's worth confirming the starting point: with the call to `transparencyFunction()` temporarily commented out inside the main loop, the lamp sprite looks like a solid, opaque yellow color, with no effect applied yet.

The lamp with no effect applied yet: a solid, opaque yellow.

Re-enabling the call to the function (now adapted to the `lamp` variable instead of `spectre`, since we don't want the character behind it to flicker) produces a lamp with a convincing look of yellow semi-transparency, something *Shadow* and *Highlight* could never have achieved, being limited to black and white.

With the flicker function applied to lamp, the lamp looks semi-transparent in a yellow shade that Shadow and Highlight couldn't reproduce.

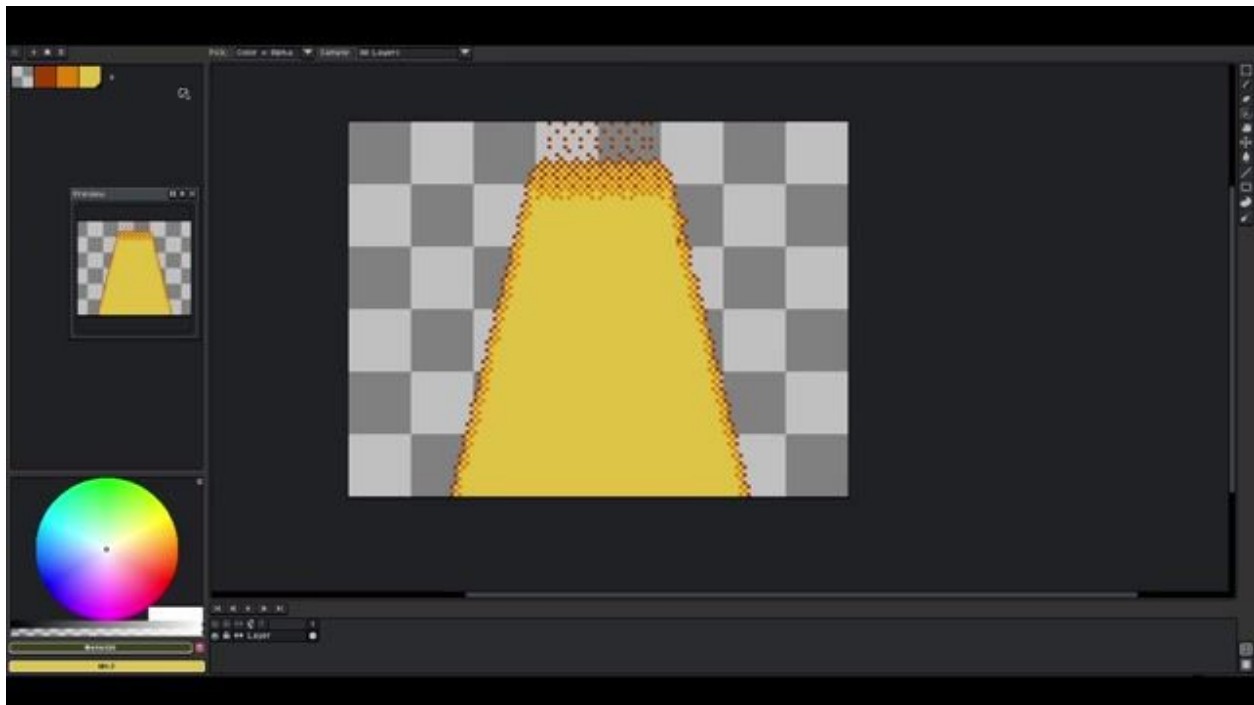


Figure 86: The lamp sprite as a solid color



Figure 87: The lamp flickering with yellow semi-transparency

4. Second technique: the mesh pattern

The original game that inspired this example, *Streets of Rage*, doesn't use flicker for its semi-transparent lamp, but rather a mesh pattern: instead of a solid color, the sprite's own graphic is drawn with a pattern of alternating pixels (a checkerboard-style dithering), so that the background color visually shows through the gaps in that pattern with no software trickery needed at all.

It's often mentioned that the Sega Saturn, generations later, has a dedicated hardware command for automatically converting a solid sprite into this kind of mesh pattern; this claim could not be confirmed against technical documentation for that console, so it should be taken with caution. In any case, the Mega Drive has nothing equivalent: the sprite's own graphic must already be drawn with the mesh built in from the start, in the graphics editor, before converting it into an SGDK resource.

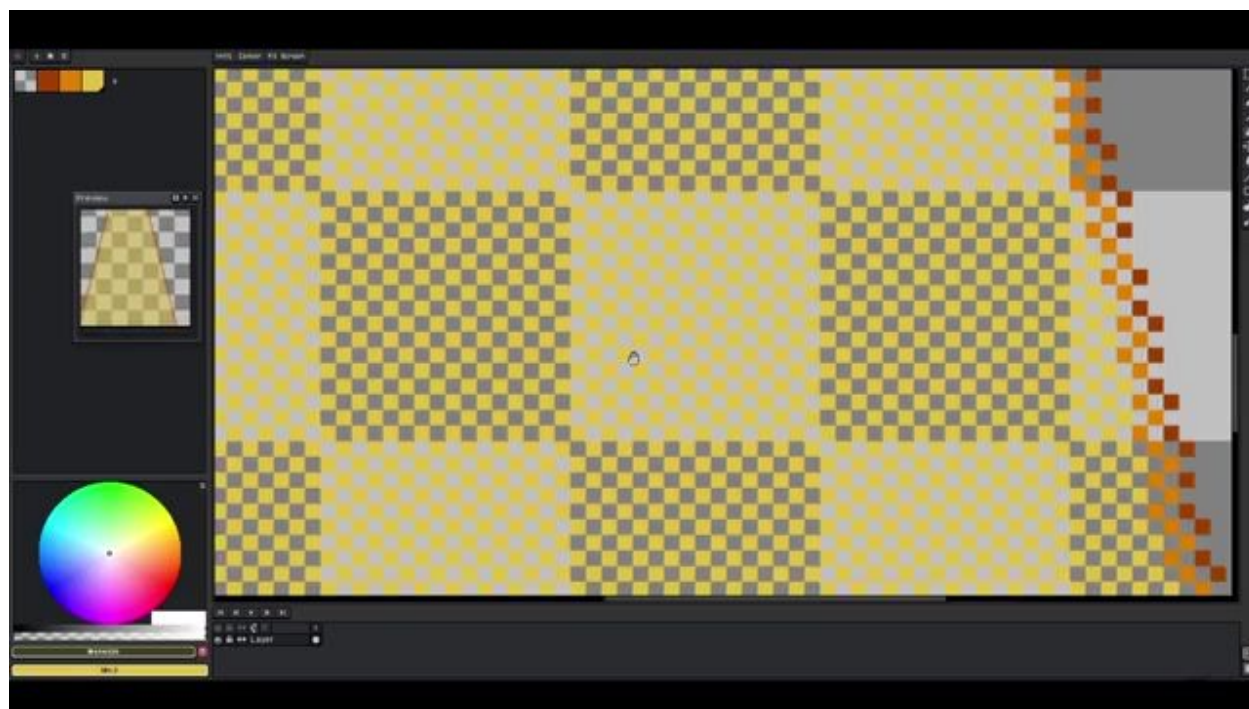


Figure 88: The lamp sprite redrawn with a mesh pattern

The lamp redrawn with a pattern of alternating pixels instead of a solid color, the technique used by the original Streets of Rage.

The visual result of a mesh pattern depends heavily on the hardware or video filter used to display it. With certain smoothing filters enabled in an emulator, the effect looks reasonably convincing; turning them off, to get closer to what you'd see with a crisp connection like RGB on a real CRT, the pattern shows up as hard lines and the sense of transparency is lost. An older or lower-quality connection, like RF, slightly blurs the whole image and, in the process, also softens the mesh (historical examples of this phenomenon include the waterfall in *Sonic the Hedgehog*, or, fittingly, the lamps in *Streets of Rage*). It's a trade-off: a sharp image with a very visible mesh, or a somewhat blurry image with a more disguised mesh. Since this perception changes depending on the television or emulator, it's worth testing the effect in more than one environment before settling on it.

Combining this mesh pattern with the flicker technique from the previous section is also possible, and gives an acceptable result, though not necessarily better than either technique

on its own.

5. Third technique: horizontal flip over the mesh pattern

There's a variant that tends to give a more even result than the mesh pattern combined with flicker: instead of alternating the sprite between visible and invisible, it's kept visible at all times, but flipped horizontally on every other frame using `SPR_setHFlip()`, another SGDK function already used in earlier chapters of Part III. Since the mesh pattern isn't perfectly symmetrical, flipping it back and forth frame by frame makes the light areas and the patterned areas seem to constantly swap positions, producing a smoother transparency effect than simply flickering the whole sprite.

The change from the flicker technique is minimal: `SPR_setVisibility()` is replaced with `SPR_setHFlip()` inside the same two branches of the `if`.

```
void transparencyFunction(void)
{
    if (timer == 0)
    {
        SPR_setHFlip(lamp, FALSE);
        timer++;
    }
    else
    {
        SPR_setHFlip(lamp, TRUE);
        timer = 0;
    }
}
```

Compiling with this change, the result is usually considered the most convincing of the three techniques presented in this chapter for this kind of effect: the lamp looks evenly semi-transparent, without the abrupt flicker of the first technique or the fixed visual harshness of the static mesh pattern from the second. As with any comparison of this sort, the final impression can vary depending on the television or emulator used to view it, so it's worth trying all three variants on the target hardware or emulator before deciding which one to use in a given project.

The lamp with a mesh pattern alternating its horizontal flip frame by frame: the most even result of the three techniques.

Nothing stops you, moreover, from combining these techniques with Shadow and Highlight within the same project: for example, using Shadow and Highlight for the parts of a sprite that need to look truly black or white in a transparent way, and one of this chapter's techniques for any other color. Which combination works best in each case depends on the visual needs of the game in question.

Review exercises

1. Explain the two limitations of Shadow and Highlight (Chapter 26) that motivate looking for alternative transparency techniques like the ones in this chapter.
2. Describe step by step, frame by frame, how the `timer` variable and the `SPR_setVisibility()` function produce the flicker effect.
3. Why doesn't a mesh pattern drawn directly onto the sprite require any special hardware support on the Mega Drive?



Figure 89: The lamp alternating its horizontal flip

4. Explain why the same mesh pattern can look very different depending on the type of video connection (RGB versus RF) used on a CRT television, and why this observation shouldn't be generalized without qualification to any television or emulator.
5. Why does alternating the horizontal flip of a sprite with a mesh pattern tend to produce a more even effect than simply flickering the whole sprite?

Chapter 28 — Ranger X-Style Dynamic Lighting

Chapters 26 and 27 covered two ways to simulate transparency over sprites: first with Shadow and Highlight, then with alternative software techniques based on flicker and mesh patterns. This chapter returns to Shadow and Highlight, but this time applies it to the background layers instead of individual sprites, in order to reproduce one of the most memorable effects in the Mega Drive library: the light beams from *Ranger X*, which illuminate both the stage and the player's own robot as it passes beneath them. Another famous example of this same technique is the stained-glass section in *Castlevania: Bloodlines*.

Unlike the version seen in Chapter 26, where priority was assigned sprite by sprite, here the switch is the priority of the background *tiles* themselves: any tile marked with high priority is shown with increased brightness, and any sprite that passes over that area is lit up along with it. This difference brings one advantage (a single change lights up the background and everything passing over it) and one new complication, because tile priority doesn't only control Shadow and Highlight: it also decides the draw order between layers and sprites, and the two functions can conflict with each other.

Prerequisites

Before continuing, it's worth having these points settled:

- The basic rules of Shadow and Highlight — PAL3 reserved for sprites with high-light/shadow, colors 15 and 16 reserved for those states, and the call to `VDP_setHighlightShadow()` — were already covered in Chapter 26 and are assumed to be known here.
- Handling two background layers (BG_A and BG_B) and the concept of tile priority, presented in Chapter 15.
- Creating and importing maps with per-tile priority using 16Tile, covered in Chapter 13.

1. Starting point: a robot over a background with several light sources

This chapter's practice project uses, as its character, a sprite inspired by the robot from *Ranger X*, over a simplified stage that revisits the setting of *Revenge of Shinobi*, chosen because it includes several light sources well suited to this effect. When working with this type of lighting, it's also best not to exceed a map that's 512 pixels wide by 256 pixels tall: larger maps can cause performance problems, which are revisited later in this manual.

The layer structure comes already set up: a regular background assigned to PAL0 and drawn on the BG_B layer with `VDP_drawImageEx()` (Chapter 7), and a foreground built in 16Tile,

loaded with `MAP_create()` and assigned to the `BG_A` layer using `PAL1`. Since the effect is going to be turned on, both the background and the foreground must start with all their priority set to `FALSE` (low), so that `Shadow` and `Highlight` can later darken or lighten them tile by tile:

```
#define MAP_WIDTH  512
#define MAP_HEIGHT 256

// Regular background on PAL0 / BG_B, starting low priority
VDP_drawImageEx(BG_B, &background_image, TILE_ATTR_FULL(PAL0, FALSE, FALSE, FALSE,
↪  TILE_USER_INDEX), 0, 0, FALSE, TRUE);

// Foreground on PAL1 / BG_A, with per-tile priority already defined in 16Tile
Map* foreground = MAP_create(&foreground_map, BG_A, TILE_ATTR_FULL(PAL1, FALSE,
↪  FALSE, FALSE, TILE_USER_INDEX + background_image.tileset->numTile));
```

The robot's sprite, for its part, is already assigned to `PAL2` with low priority, and the project includes the `VDP_setHiLightShadow(TRUE)` line needed to turn the mode on. The rest — controls, camera, and map scrolling — follows the same pattern already covered in previous chapters.

2. The foreground map in 16Tile: everything at low priority

The foreground is built beforehand in 16Tile with a tileset inspired by *Revenge of Shinobi*. All of its tiles are assigned to `PAL1` and, to start, all of them have their priority set to low. Building the project at this point, the result is a completely darkened scene: since `Shadow` and `Highlight` is already active but nothing has high priority, the background, the foreground, and the robot's own sprite all appear equally darkened.

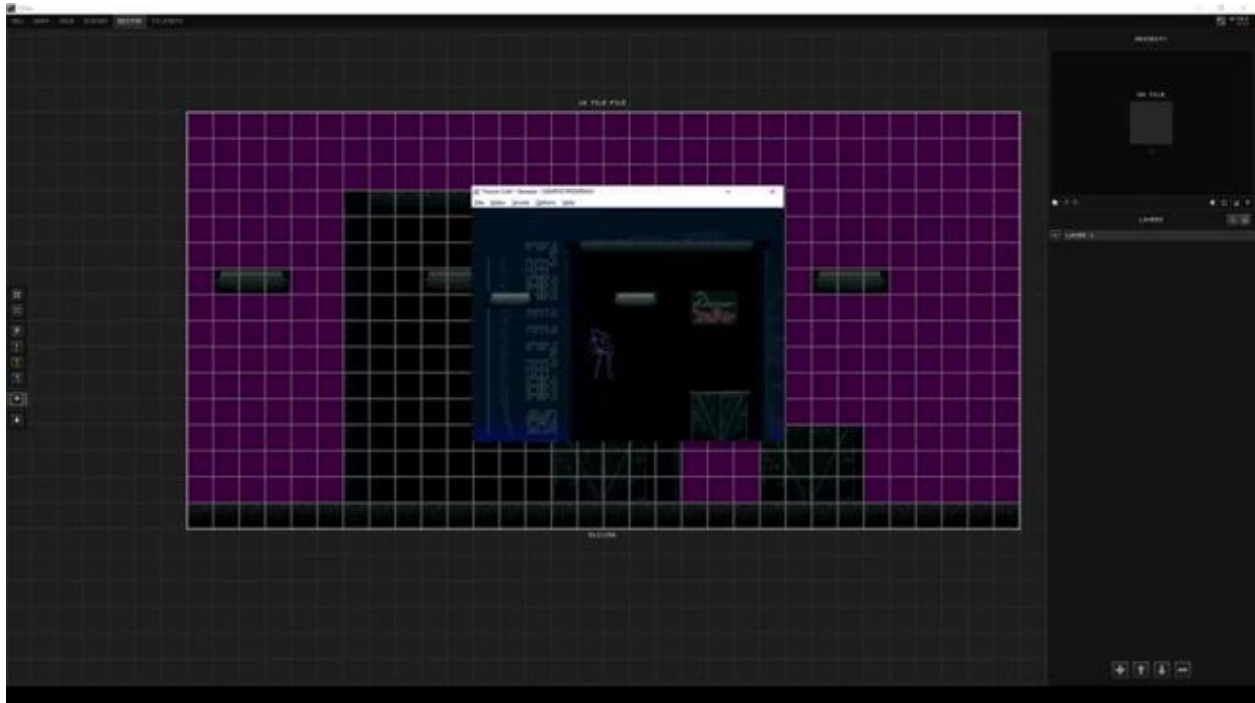


Figure 90: The priority layer in 16Tile, all set to low, alongside the on-screen result
 With the entire priority layer in magenta (low), background, foreground, and sprite are all

equally darkened.

3. Painting the first light source at high priority

The first concrete step is to paint, within 16Tile, only the tiles directly under the stage's first lamp at high priority. With this change the intended effect already shows up: that area of the background looks noticeably brighter than the rest, and the same happens to the robot's sprite as it passes underneath, because Shadow and Highlight applied to the background also affects any sprite located over an area marked at high priority.



Figure 91: The area under the first lamp already marked at high priority, with the background and the robot lit up as they pass underneath

By marking those tiles at high priority, both the background and the robot light up while passing through that area.

4. A color the effect doesn't reach: palette position 15

Looking closely at the robot's sprite as it passes under the lit area, there's one detail that doesn't change brightness along with the rest: a small dot of color in its design. The cause is an additional rule of the Shadow and Highlight mode applied to the background: for any sprite, regardless of whether it uses PAL2 or PAL3, the color located at position 15 of its palette is never affected by the effect. In this example, that dot was deliberately placed at position 15 — a position not used in the original design — and position 16 was filled with another color just to complete the 16-color palette.

There are at least three reasonable ways to deal with this rule when designing a sprite that's going to receive the effect:

- Avoid using position 15 altogether, even if that means sacrificing an available palette color.
- Accept that this particular color won't be affected, when it takes up little space on the sprite and is unlikely to be noticed during gameplay.
- Turn the limitation into a design asset: for example, a robot's or an enemy's glowing eyes could occupy exactly position 15, so that they keep standing out even under shadow. This is one option among the previous ones, not necessarily a better one, and it's worth choosing for whoever values that constant glow as part of the character's design.

5. A difference from the original *Ranger X*: should the player's sprite be lit up?

Comparing the result obtained so far with the original game reveals a difference: in *Ranger X*, the player's own robot sprite isn't affected by the lighting effect, and always keeps the same brightness, whether or not it passes through a lit area. That behavior can be reproduced with a single change: setting the robot sprite's priority to high (TRUE) instead of low.

```
// High priority: the sprite is exempt from Shadow and Highlight
SPR_setAttribute(ranger, TILE_ATTR(PAL2, TRUE, FALSE, FALSE));
```

There's no official confirmation of why the original game behaves this way, but there's a reasonable explanation: a player sprite that gets too dark under certain lights could make it harder to see clearly during gameplay, so keeping it at a constant brightness would be a design decision rather than a technical limitation. That said, the opposite stance is just as valid: letting the player's sprite (and the enemies' sprites) be affected by the effect too, since many consider it visually more appealing and it reinforces the feeling that the character is part of the same lit space as the stage. Both are legitimate design decisions; for the rest of this chapter, the robot's priority is set back to low (FALSE), since lighting up the player's own sprite is the central goal of this technique.

6. Adding the remaining light sources

With the pattern already established, the rest of the stage's lamps are added the same way: painting, within 16Tile, the tiles that correspond to each additional light source at high priority. Working over a dark background inside the editor itself helps to more clearly distinguish which tiles are already at high priority and which are still at low.

With the new lights already painted in, however, a different problem shows up next to the second light source: part of a wall and a trash can from the foreground, which should be drawn in front of the robot sprite, end up appearing behind it when passing through that area.

The robot should pass in front of the wall and the trash can, but ends up covered by them.

7. The underlying conflict: priority serves two purposes at once

The cause of this problem is the same limitation already explained in Chapter 15: the priority parameter of a tile or a sprite serves two purposes at the same time. On one hand, it determines whether that element is affected by Shadow and Highlight; on the other, it also determines the draw order between background layers and sprites, since any high-priority



Figure 92: The robot ending up hidden behind the wall and the trash can near the second light source

tile is drawn in front of any low-priority sprite, without exception. Marking the wall and trash can tiles at high priority to light them up had a side effect: those same tiles started being drawn in front of the robot sprite, which is still at low priority. It's the same mechanism that, applied the other way around, lets a character correctly stay behind one foreground element and in front of another within the same scene with several overlapping layers.

8. The solution: replacing the conflicting foreground with low-priority sprites

The chosen solution avoids sacrificing either the lighting effect or the correct draw order: within 16Tile, the foreground tiles causing the conflict — the wall and the right side of the trash can — are removed and replaced with the transparent tile, so that the background becomes directly visible in that area.

At this point the lighting effect already works correctly in that area, but the actual drawing of the wall and the trash can, just removed from the foreground, is now missing. That's where the core of the solution comes in: instead of background tiles, those elements are recreated as new low-priority sprites, using the same graphic they had as part of the foreground, placed exactly where the removed tiles used to be.

```
// New sprites, using the same palette the foreground had (PAL1)
Sprite* ranger = SPR_addSprite(&sprite_ranger, x, y, TILE_ATTR(PAL2, FALSE, FALSE,
    ↪ FALSE));
Sprite* wall = SPR_addSprite(&sprite_wall, wallX, wallY, TILE_ATTR(PAL1,
    ↪ FALSE, FALSE, FALSE));
Sprite* bin = SPR_addSprite(&sprite_bin, binX, binY, TILE_ATTR(PAL1, FALSE,
    ↪ FALSE, FALSE));
```

Since these new sprites remain at low priority, just like the player's robot, they are indeed subject to Shadow and Highlight and light up along with the rest of the scene under that lamp. Being sprites just like the robot itself, moreover, the draw order between them is controlled normally, without the previous conflict between background tile and sprite.

It's best to add the robot sprite first and only afterward the new wall and trash can sprites, so that the robot ends up correctly in front of them when it should: the order in which `SPR_addSprite()` is called determines which sprite is drawn in front of which. It's also necessary to update the position of these sprites within the camera function, just like with any other element that needs to scroll along with the map.

Replacing foreground tiles with sprites solves the conflict between lighting and draw order, but it comes at a cost: each element turned into a sprite starts competing for the same hardware resources as the rest of the sprites on screen. As explained in Chapter 2, the Mega Drive can display around 80 simultaneous hardware sprites before flicker starts showing up due to a lack of available slots on a given scanline, so this resource should be used sparingly, and not every conflicting foreground element should be turned into a sprite.

With these changes, the problem is resolved: the wall and the right side of the trash can once again appear in front of or behind the robot as appropriate, and all three light up correctly when passing under the lamp.

9. Why the order of `SPR_addSprite()` matters

To underscore the importance of this order, it's worth deliberately trying to reverse it: adding the robot sprite after the wall and trash can sprites, instead of before. With that change, the player's robot ends up drawn behind the trash can instead of in front of it, as it should be, since the order of `SPR_addSprite()` directly determines the order of the hardware sprite table, and therefore what gets drawn over what. Reversing the order back — leaving the robot sprite as the first one added — restores the correct result.

Review exercises

1. Explain the main advantage of applying Shadow and Highlight to the background instead of to individual sprites, and contrast it with the approach seen in Chapter 26.
2. Which palette position of a sprite is always exempt from the Shadow and Highlight effect applied to the background, and what design strategies exist for dealing with that limitation?
3. Explain the conflict that arises when marking the tiles of a foreground element that should be drawn in front of a sprite at high priority, and why it happens.
4. Describe the solution applied to resolve that conflict, noting what care must be taken with the order of `SPR_addSprite()` and with the limit on simultaneous hardware sprites.
5. Mention at least two limitations that Shadow and Highlight applied to the background shares with the version applied to individual sprites.

Chapter 29 — Palette Cycling Animations

Palette cycling was a very common technique in the 16-bit era for bringing backgrounds to life: it was used to simulate moving water or fire, day-to-night lighting transitions, or sci-fi flickers. Its appeal isn't just visual. Since it doesn't require extra tiles in VRAM or updating them frame by frame, it's an extremely cheap technique in terms of system resources, and combined with other hardware tricks it can even contribute to pseudo-3D effects. This chapter covers how to prepare graphics for this technique, how to program it in SGDK using `PAL_setPalette()`, and how to get a quick preview of the effect in Aseprite before writing a single line of code.

Prerequisites

Before continuing, make sure you're clear on:

- The use of `PAL_setPalette()` and loading a background image with `VDP_drawImageEx()`, as well as how CRAM is organized into 16-color banks (chap. 2, "The VDP in Depth") and how a background is displayed for the first time (chap. 7, "Displaying Backgrounds on the Mega Drive").
- Working with integer-based timers inside the main loop and the signed types `s8/s16` (chap. 6, "C Applied to Retro Development").
- How to work with indexed color mode in Aseprite (chap. 8, "Creating Graphics with Aseprite").
- How to extract a function from the main loop by declaring its prototype before `main()`, a pattern already used in previous chapters on animation.

1. A conceptual starting point: "8-bit art"

Before diving into the code, it's worth keeping in mind an idea that runs through much of classic pixel art for animated backgrounds: the so-called "8-bit art," understood not as a reference to a system or console, but to the number of colors shown on screen: 256 in total. Displaying that many simultaneous colors on the Mega Drive would require an add-on like the 32X, but even with the 61 colors effectively available on the base console (four palettes of 16 colors, with index 0 of each reserved for transparency), very convincing effects can be achieved using palette cycling. The key is building the original artwork with color shifting in mind from the start, so that the palette change reads as a smooth animation rather than a simple pixel flicker. Achieving this is, above all, a palette design problem rather than a programming one.

The "Canvas Cycle" demo, with 256 colors on screen using only palette cycling over a coastal landscape.



Figure 93: Canvas Cycle, Mark Ferrari's demo

2. Preparing the graphics: a background with an indexed palette

The example in this chapter uses as its base a waterfall background, edited in Aseprite with the color mode set to **indexed** (not RGB, as explained in the chapter on creating graphics with Aseprite), an essential requirement for being able to manipulate palette positions directly instead of the color of each pixel. The first color in the palette corresponds to the background black, and the colors relevant to the effect are the whites, greens, and blues that make up the waterfall.

The waterfall background in indexed color mode, with the corresponding palette visible in the left panel.

The advantage of palette cycling is that, instead of keeping many unique tiles in VRAM that have to be swapped out to create an animation, it's enough to shift the colors within the palette positions to achieve the same visual effect. This can be verified right in Aseprite: if you move the white color to the first position of the palette (for example, undoing and redoing the change with Ctrl+Z/Ctrl+Y to compare before and after), the waterfall gives the impression of movement, since all the colors in the gradient shift one position. Repeating that shift several times completes the entire animation cycle.

After shifting the colors by one position, the waterfall's water already shows slight movement compared to the original state.

3. Saving each step of the cycle as a separate image

To keep the programming process as simple as possible, it's better not to try to save the palette animation as such, but instead to create a series of tiny images (8x8 pixels, i.e., a

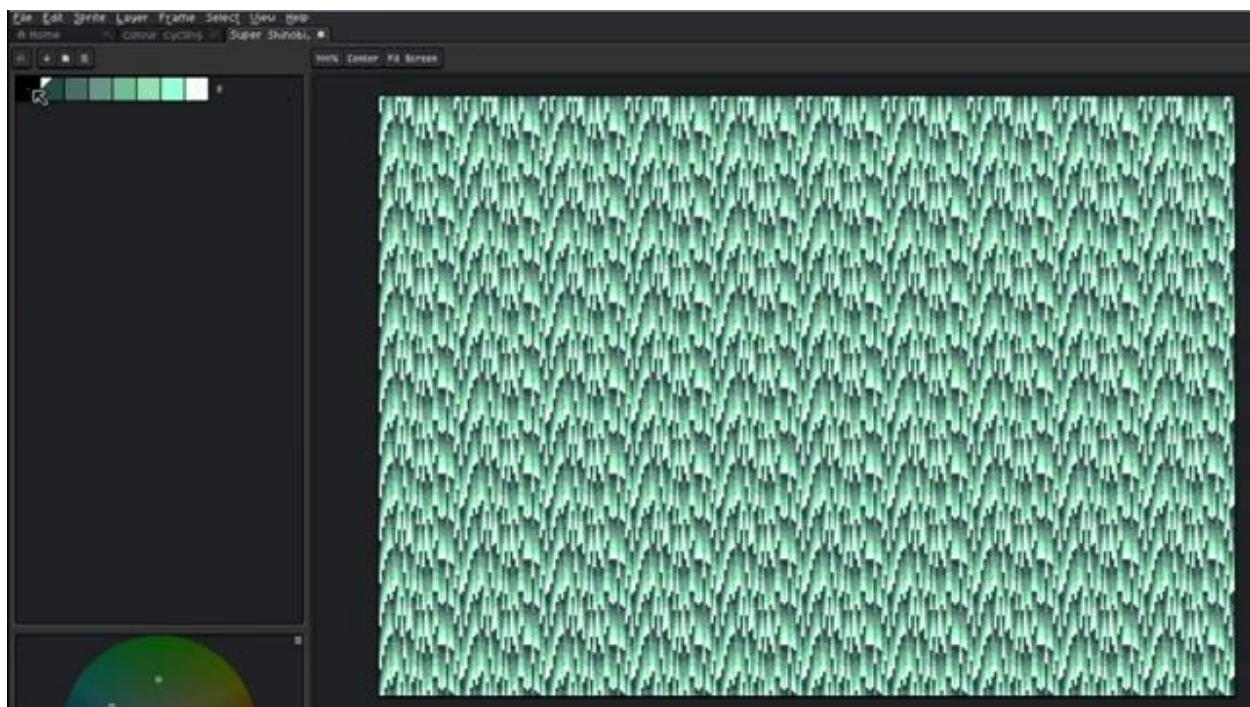


Figure 94: The waterfall background in Aseprite

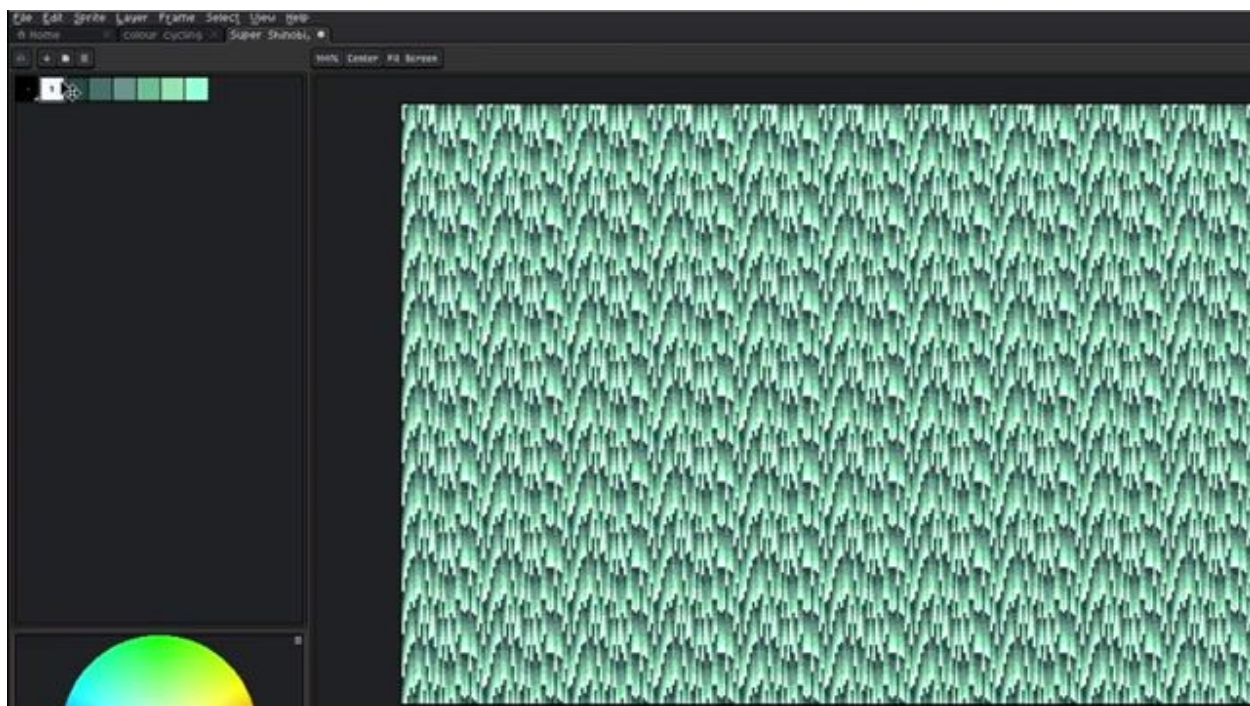


Figure 95: The waterfall palette after shifting a color

single tile), each with a slightly different (shifted) version of the waterfall's palette. These images aren't used for their graphical content: they serve solely as containers from which SGDK can read the palette data corresponding to each step of the cycle.

The first file is saved with the palette in its original position (in indexed color mode, exported as PNG); then, for each subsequent step, the palette is shifted one position further and saved under a different filename, until seven files in total are completed, one for each step of this waterfall's full cycle. The number of steps depends directly on how many colors make up the gradient you want to animate: a waterfall with seven relevant tones needs seven steps to complete a full cycle.

4. The starting point in SGDK

The example project already includes the seven 8x8 images just described and the waterfall background, all declared in `resources.res`. The background is called `BG1`, and each of the small palette images is named `PC0` through `PC6` (from "palette cycle"):

```
IMAGE BG1 "gfx/shinobi_waterfall.png" NONE
```

```
IMAGE PC0 "gfx/pc0.png" NONE
IMAGE PC1 "gfx/pc1.png" NONE
IMAGE PC2 "gfx/pc2.png" NONE
IMAGE PC3 "gfx/pc3.png" NONE
IMAGE PC4 "gfx/pc4.png" NONE
IMAGE PC5 "gfx/pc5.png" NONE
IMAGE PC6 "gfx/pc6.png" NONE
```

The starting code in `main.c` is deliberately simple: it loads the palette and draws the waterfall background on the `BG_B` layer, with no animation yet, just as was done when displaying a background for the first time in chapter 7:

```
#include <genesis.h>
#include "resources.h"

int main(bool hardReset) {
    VDP_drawImageEx(BG_B, &BG1, TILE_ATTR_FULL(PAL0, FALSE, FALSE, FALSE,
↪  TILE_USER_INDEX), 0, 0, FALSE, TRUE);
    PAL_setPalette(PAL0, BG1.palette->data, DMA);

    while (TRUE) {
        SYS_doVBlankProcess();
    }

    return 0;
}
```

Compiling at this point results in a completely static waterfall.

5. Creating the palette cycle timer

The first step in animating the effect is to create a timer: an integer variable called `palette_timer`, initialized to 0 and declared before `int main()` so it can also be used inside a separate function later on. As is typical with this kind of timer (chap. 6), it's incremented

once per pass through the main loop using `palette_timer++` (equivalent to `palette_timer += 1`), so its value advances from 0 to 1, 2, 3, and so on, once per frame.

```
static s16 palette_timer = 0;

int main(bool hardReset) {
    // ...

    while (TRUE) {
        palette_timer++;
        SYS_doVBlankProcess();
    }

    return 0;
}
```

6. Changing the palette based on the timer

To make something happen when the timer reaches a given value, `if` statements are used to compare `palette_timer` against a specific number. In the first step (when `palette_timer` equals 0), the same `PAL_setPalette()` function already familiar from previous chapters on backgrounds is used, but instead of taking the palette data from the background itself, it's taken from one of the small PC0 to PC6 images created earlier (in this case, PC0, the palette in its original position).

From there, the rest of the sequence is built with the same chained `else if` pattern, deciding how many frames should pass before the cycle advances (ten frames in this example, i.e., one-sixth of a second at 60 frames per second) and changing, at each step, both the comparison number and the palette file used: PC0 at frame 0, PC1 at frame 10, PC2 at frame 20, and so on up to PC6 at frame 60:

```
if (palette_timer == 0)
    PAL_setPalette(PAL0, PC0.palette->data, DMA);
else if (palette_timer == 10)
    PAL_setPalette(PAL0, PC1.palette->data, DMA);
else if (palette_timer == 20)
    PAL_setPalette(PAL0, PC2.palette->data, DMA);
else if (palette_timer == 30)
    PAL_setPalette(PAL0, PC3.palette->data, DMA);
else if (palette_timer == 40)
    PAL_setPalette(PAL0, PC4.palette->data, DMA);
else if (palette_timer == 50)
    PAL_setPalette(PAL0, PC5.palette->data, DMA);
else if (palette_timer == 60)
    PAL_setPalette(PAL0, PC6.palette->data, DMA);
```

7. Restarting the cycle and choosing the right data type

With the seven conditions now in place, one problem remains: `palette_timer` will keep incrementing indefinitely past 60, with no condition set up to handle those future values. If it had been declared as `int` (a 32-bit integer), the timer would take an extremely long time to overflow and wrap back to 0 on its own, so in practice the cycle would never restart naturally.

The solution is to add an eighth condition that, upon reaching a reference value, manually resets the timer.

There's an important detail about how to reset it: since `palette_timer++` runs at the end of every pass through the loop, after all the if comparisons, assigning `palette_timer = 0` directly in that condition would mean that, after the subsequent increment, the timer would end up at 1 on the next frame, completely skipping the `palette_timer == 0` condition and its PC0 palette. The solution is to assign `palette_timer = -1` instead, so that, after the increment, the value effectively ends up at 0 for the following frame, closing the cycle correctly:

```
else if (palette_timer == 70)
    palette_timer = -1;
```

There's no fixed answer regarding the exact reference value for the reset (70 in the example above): shifting that point to 69 could, in theory, produce a slightly more even transition between the last step of the cycle and the first, but there's no definitive reason to prefer one value over the other. It's worth trying both (69 and 70) experimentally on your own project and keeping whichever looks smoother, rather than assuming one is "the correct" one.

This need for a transient negative value also affects the data type used for `palette_timer`: since `int` is a 32-bit integer, that's more space than is needed for this purpose. An `s8` (an 8-bit signed integer, roughly -128 to 127) would be more than enough for a 70-frame cycle, but it's better to use `s16` (a 16-bit signed integer, with a range of several thousand in both directions) as a more flexible option in case higher timer values are needed in the future (for longer or slower cycles, for example), without going as large as a 32-bit `int`. In any case, the essential point is that the chosen type must be signed, since without that, the -1 assignment wouldn't have the intended effect.

8. Compiling the result and extracting a function

Once saved and compiled, the resulting effect strongly resembles a waterfall animated by moving water. When reconstructing the palette of an existing commercial game from memory or from a screenshot, it's normal not to achieve a perfect match with the original: it's enough for the result to be visually convincing for your own project. The effect looks even better once foreground elements and sprites are added in front of the waterfall. By changing the numbers used in the conditions (both the interval between frame changes and the reset point), the perceived speed of the cycle can be adjusted, faster or slower as the design requires.

Finally, since keeping all this palette-changing code directly inside the main loop is untidy, it's worth extracting it into a separate function, following the same pattern already used in previous chapters: a prototype declared before `main()`, the function body defined afterward, and a single call to that function inside the main loop:

```
#include <genesis.h>
#include "resources.h"

static s16 palette_timer = 0;

static void paletteCycler();

int main(bool hardReset) {
    VDP_drawImageEx(BG_B, &BG1, TILE_ATTR_FULL(PAL0, FALSE, FALSE, FALSE,
↪  TILE_USER_INDEX), 0, 0, FALSE, TRUE);
    PAL_setPalette(PAL0, BG1.palette->data, DMA);
```

```

while (TRUE) {
    paletteCycler();
    SYS_doVBlankProcess();
}

return 0;
}

static void paletteCycler() {
    if (palette_timer == 0)
        PAL_setPalette(PAL0, PC0.palette->data, DMA);
    else if (palette_timer == 10)
        PAL_setPalette(PAL0, PC1.palette->data, DMA);
    else if (palette_timer == 20)
        PAL_setPalette(PAL0, PC2.palette->data, DMA);
    else if (palette_timer == 30)
        PAL_setPalette(PAL0, PC3.palette->data, DMA);
    else if (palette_timer == 40)
        PAL_setPalette(PAL0, PC4.palette->data, DMA);
    else if (palette_timer == 50)
        PAL_setPalette(PAL0, PC5.palette->data, DMA);
    else if (palette_timer == 60)
        PAL_setPalette(PAL0, PC6.palette->data, DMA);
    else if (palette_timer == 70)
        palette_timer = -1;

    palette_timer++;
}

```

9. Quick preview in Aseprite before programming

Before building a full palette cycle in SGDK, it can be useful to get a quick preview of the effect directly in Aseprite, without going through the file-creation and coding process described in the previous sections yet. Aseprite doesn't natively offer a dedicated feature for this, but it does support community scripts that extend its functionality: there are scripts of this kind that automate palette cycling animation right in the editor, generating several animation frames (each with the palette shifted one step) from a range of color indices specified by the user. It's worth looking up the current version of such a script in the Aseprite community before using it, since its availability and installation method may change over time.

Generally speaking, installing an Aseprite script involves downloading it, copying the file into the program's scripts folder (accessible from *File > Scripts > Open Scripts Folder*), and then going to *File > Scripts > Rescan Scripts Folder* so Aseprite recognizes it. A typical palette cycle script asks for the index numbers of the colors to cycle (counting from the left, starting at 0) and, once confirmed, automatically generates the corresponding frames, letting you play them back right inside Aseprite to preview the effect before exporting anything.

The dialog box of the Colour Cycle script, with the color indices to cycle entered and the buttons to generate the frames.

If a script like this produces no visible result, the most likely cause is that the file is still in RGB color mode instead of indexed (chap. 8), so that setting should be checked and corrected

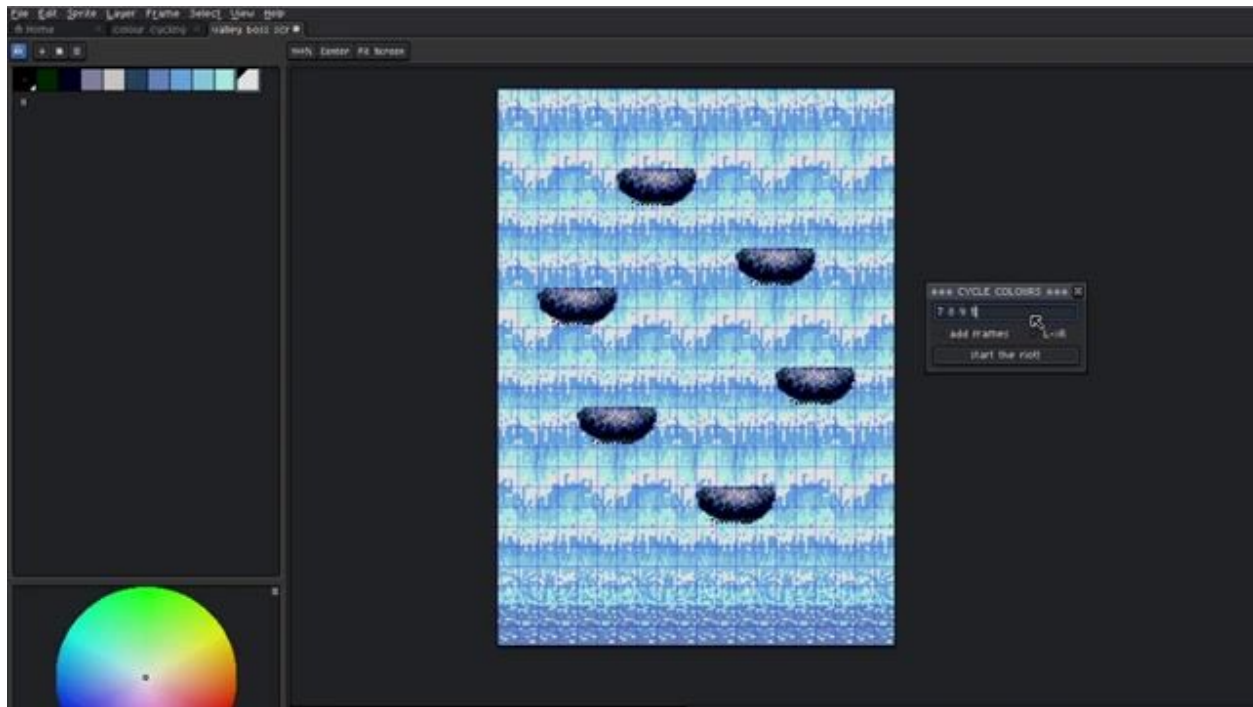


Figure 96: Dialog box of the Colour Cycle script

before repeating the process. The speed of the resulting animation can usually be adjusted by selecting all the generated frames and using the *Properties* option from the context menu (right-click) to change the duration of each frame.

Tools of this kind typically don't automatically generate the palette files that SGDK needs (the manual process described in sections 2 and 3 of this chapter): they mainly serve as quick visual prototyping within Aseprite, before investing time in programming the full effect.

Review exercises

1. Explain why palette cycling is cheaper in resources than animating individual tiles to achieve a similar effect.
2. Describe the procedure for preparing the image files that `PAL_setPalette()` will use at each step of the cycle.
3. Why must the timer reset to -1 rather than 0 in the cycle's final condition? Explain the role of the order of operations (the increment happening after the comparisons) in that decision.
4. Why is it better to use an `s16` type rather than `int` for `palette_timer`, and what role does the data type's signedness play in that decision?
5. Modify this chapter's example so the cycle resets at frame 69 instead of 70, and compare both results by eye. Do you notice any real difference in the smoothness of the transition?

Chapter 30 — Palette Fades

The previous chapter covered palette cycling: a technique for animating a scene by repeatedly alternating between several pre-prepared palettes. This chapter closes out Part III with a related but distinct technique: instead of assigning palettes directly and immediately with `PAL_setPalette()`, the color data is taken, stored in an array in RAM, and that array is used together with SGDK's `PAL_fadeIn()` function to make the palette appear gradually from black, rather than all at once.

The practical goal is to eliminate the “flicker” or abrupt-load effect that often occurs when the different graphical elements of a screen (background, foreground, sprites) load their palettes in a staggered way that’s visible to the player. Many commercial games solve exactly this detail so that a level’s opening screen looks polished instead of improvised.

Prerequisites

Before continuing, it helps to be clear on:

- The use of `PAL_setPalette()` to assign palettes to different planes and sprites from graphic resources defined in `resources.res` (Chapter 7, “Displaying backgrounds on the Mega Drive”).
- The concept of CRAM as the VDP’s color memory, with its four palettes of 16 colors each (Chapter 2, “The VDP in depth”).
- The concept of an array in C as a structure capable of storing multiple values of the same type under a single name.
- SGDK’s build cycle via `resources.res` and resource generation.
- Basic handling of a graphics editor in indexed color mode (for example, Aseprite).

1. The problem: a staggered, visible palette load

A simple *beat ‘em up*-style example serves as a starting point, with four palettes assigned independently via `PAL_setPalette()`: one for the background (PAL0), another for the foreground (PAL1), another for the player sprite (PAL2), and another for the enemy sprite (PAL3), each pulling its color data from its own graphic resource.

```
PAL_setPalette(PAL0, background_image.palette->data, DMA);
PAL_setPalette(PAL1, foreground_image.palette->data, DMA);
PAL_setPalette(PAL2, player_sprite.palette->data, DMA);
PAL_setPalette(PAL3, enemy_sprite.palette->data, DMA);
```

Reloading the ROM repeatedly in the emulator shows that the entry transition looks noticeably staggered: the background appears first, and then, with a slight offset, the foreground and the sprites follow. The result feels unpolished. If instead the whole graphic set appeared

through a slow fade from black, the transition would look far more deliberate, and that's the effect built in this chapter.

2. Consolidating the four palettes into a single image

The first step is to comment out (disable) the four `PAL_setPalette()` calls used so far, since they'll be replaced by a different mechanism based on an array.

Instead of maintaining four independent palettes, the technique consists of storing all 64 available colors (16 colors for each of the Mega Drive's 4 palettes) within the palette of a single image — in this case, the background image. That image's palette is expanded to 64 slots — 16 per row — so that the first row corresponds to what would be PAL0, the second row to PAL1, the third to PAL2, and the fourth to PAL3. The procedure essentially consists of copying and pasting, in the graphics editor, the colors from the palettes of the other images (foreground, player sprite, enemy sprite) into the corresponding rows of the background image's palette.



Figure 97: Consolidated palette of the background image, with 64 colors in four rows of 16

The full consolidated palette, showing the range from 0 to 63 with the four rows of 16 colors corresponding to the four original palettes.

Each palette must occupy exactly its row of 16 colors, even if the original graphic uses fewer colors than are available: the leftover slots are filled with black or another filler color, and are never left out of order, because otherwise the colors from the different palettes would end up mixed and misaligned relative to the indices the code will later use.

If the background image was already loaded previously in `resources.res`, SGDK only regenerates resources when it detects a change in that file, or when a full rebuild (*clean*) is requested. So, after modifying only the palette of an already-existing image, it's a good idea to force resource regeneration by adding and then removing any character in `resources.res` (saving both times), or by doing a full

project clean before rebuilding. Otherwise, the ROM will keep using the previous palette data.

3. Creating the array and copying the color data with `memcpy()`

The next step takes place in `main.c`: an array is created that can hold the 64 color values. Since each color is represented as an unsigned 16-bit integer, the array is declared as `u16`, with a descriptive name and a size of 64 slots, declared before `main()` just like any other global variable.

```
u16 our_level_palette[64];
```

To move the color data from the background image into that array in RAM, `memcpy()` is used — a standard C function (not specific to SGDK) that copies blocks of memory and takes three parameters: the copy destination, the copy source, and the number of bytes to copy.

```
memcpy(&our_level_palette[0], background_image.palette->data, 64 * 2);
```

The first parameter is the destination: the name of the array just created, preceded by the `&` symbol (memory address) and with `[0]` as the starting position, since the write should begin at the start of the array. The second parameter is the source of the data: taken from the background image's palette, using the same expression used earlier to assign palettes directly, i.e., `background_image.palette->data`.

The third and final parameter indicates the size of the data to transfer, expressed in bytes. Since each color takes up two bytes, it's worth writing `64 * 2` explicitly rather than `128`, precisely to make the math itself document the fact that each of the 64 colors occupies two bytes of data. The result is equivalent to writing `128` directly, but the `64 * 2` form reads better as implicit documentation of the calculation.

At this point, the `our_level_palette` array already holds the color data in RAM, but that copy by itself has no visible effect on screen: `memcpy()` only moves bytes in memory, without interacting at all with the VDP or with the console's active palette.

4. Applying the fade with `PAL_fadeIn()`

For the color data stored in the array to actually affect what's shown on screen, and to do so through a gradual transition, SGDK's `PAL_fadeIn()` function is used, documented as an effect that makes a palette appear gradually from black to the specified color (*fade in, black to specified color effect*):

```
void PAL_fadeIn(u16 fromCol, u16 toCol, const u16 *pal, u16 numFrame, bool async);
```

The function takes five parameters:

- **fromCol**: the starting color index of the fade, within the range 0-63. Since the consolidated palette holds 64 colors in total, and each original palette (PAL0 through PAL3) occupies 16 consecutive slots within that range (PAL0 from 0 to 15, PAL1 from 16 to 31, and so on), it's possible to treat the fade as a single continuous block from 0 to 63 instead of four separate blocks.
- **toCol**: the ending index of the fade, also within 0-63, and this last index is included in the effect. To make all four palettes fade in at once from black, the values 0 and 63 are used, covering all 64 colors (index 63 corresponds to the last of the palette's 64 colors).

- **pal**: the source of the color data, i.e., the name of the array created earlier, without needing to prefix it with the & symbol or specify any position in brackets, since the function expects a pointer to the whole array directly.
- **numFrame**: the duration of the fade expressed in frames. Since the Mega Drive runs at 60 frames per second (in the region used as the reference throughout this manual), a value of 60 corresponds to one second of duration, 120 to two seconds, and 30 to half a second.
- **async**: a boolean that determines whether the rest of the game's processes can continue while the fade is in progress (true), or whether, on the contrary, all other game logic must wait for the fade to finish before continuing (false). For now it's fine to leave it as true; section 7 revisits this parameter in detail.

```
PAL_fadeIn(0, 63, our_level_palette, 60, true);
```

5. First test: the fade works, but with an initial flash

With the new `PAL_fadeIn()` call in place, the project is saved, built, and run. The fade can be seen, though it's hard to observe clearly at a duration of 60 frames, so it helps to temporarily increase the frame count — say, to 300 — to make it much slower and more obvious during testing:

```
PAL_fadeIn(0, 63, our_level_palette, 300, true);
```

For most screen transitions, a value between 20 and 30 frames tends to work better in practice, reserving much longer durations for deliberately dramatic effects, such as the gradual appearance of an imposing enemy.

Although the fade itself works correctly, a brief flash of some graphic elements can still be seen right at the start, before the fade becomes noticeable. The cause is the following: the call to `PAL_fadeIn()` happens as the last step before loading all the graphic resources (tileset, map, sprites), and `memcpy()` only moves the color data into RAM without touching the VDP's active palette. As a result, while those graphic resources are loading, the console is still using an SGDK default palette, and it's precisely that default palette, briefly applied to the newly loaded graphics, that shows up as an unwanted flash on screen before the fade begins.

The flash of the SGDK default palette, with colors clearly unrelated to the palette defined for the level, briefly visible before the fade begins.

6. Removing the flash with `PAL_setColors()`

The solution is to explicitly set every color in the palette to black before loading any graphic resources, so that even though the VDP uses its default palette during loading, that default palette is already black and produces no visible flash. This is done with the `PAL_setColors()` function, placed right before loading the tileset:

```
void PAL_setColors(u16 index, const u16 *pal, u16 count, TransferMethod tm);
```

```
PAL_setColors(0, palette_black, 64, DMA);
```

The first parameter (`index`) is the color index to start writing at; since the whole palette should be affected, 0 is used. The second parameter (`pal`) is the set of RGB intensities to apply; instead of manually specifying the values for black, SGDK already provides a predefined palette called `palette_black`, ready to use directly.



Figure 98: Flash of the SGDK default palette before the fade

The third parameter (count) behaves differently from how the first two parameters of `PAL_fadeIn()` did: whereas `PAL_fadeIn()` took a starting index and an inclusive ending index, in `PAL_setColors()` this parameter doesn't represent an index at all, but a count of colors to affect starting from the initial index. This is a difference in convention between the two functions worth keeping in mind so as not to make a mistake by analogy: to cover all 64 colors of the palette (indices 0 through 63 inclusive), you don't write 63, but 64, since you're counting 64 colors starting from index 0.

The fourth and final parameter (`tm`, *transfer method*) indicates the method used to transfer to video memory; for this purpose, DMA is enough, with no need to dig into the available alternatives yet (CPU, DMA_QUEUE, DMA_QUEUE_COPY).

With this change, the full sequence looks like this:

```
memcpy(&our_level_palette[0], background_image.palette->data, 64 * 2);
PAL_setColors(0, palette_black, 64, DMA);

VDP_loadTileSet(...);
// ... rest of graphic resource loading (map, sprites) ...

PAL_fadeIn(0, 63, our_level_palette, 60, true);
```

First the color data is copied into the `our_level_palette` array (no visible effect yet); then every color of the active palette is set to black with `PAL_setColors()`; next the graphic resources are loaded, which now produce no flash at all because the palette is already black; and finally `PAL_fadeIn()` is applied to gradually bring in the real colors from that black.

Saving, building, and running the ROM again with this order produces a clean fade from black, with no flash of graphic resources using the default palette.

The order of the instructions is decisive: if `PAL_setColors()` is placed after loading the graphic

resources instead of before, the same default-palette flash observed originally reappears. `PAL_setColors()` must always run before loading the graphic resources, never after.

7. The fifth parameter of `PAL_fadeIn()`: true versus false

Returning to the fifth parameter of `PAL_fadeIn()`, deferred from section 4: when it's set to true, the rest of the game's processes (sprite updates, input handling, etc.) keep running normally while the fade is in progress. When it's changed to false, on the other hand, all other game logic stays paused until the fade finishes completely.

```
PAL_fadeIn(0, 63, our_level_palette, 300, false);
```

Testing the ROM with this parameter set to false shows that, while the image fades in very slowly from black, the sprites remain completely still: the main loop code responsible for updating sprites and handling player input doesn't run until the fade finishes.

Setting the parameter back to true, the sprites can move normally again while the fade's colors keep appearing gradually. In most cases true is the more suitable choice, reserving false for specific scenarios where you deliberately want to fully pause the game's logic for the duration of the transition (for example, a narrative transition screen where no interaction should be possible).

8. Partial fades: fading only part of the palette

The first two parameters of `PAL_fadeIn()` determine which range of color indices is affected by the fade, and that range doesn't have to cover the whole palette. If the range is set to 0 and 15, only the first 16 colors of the palette are faded (the ones that were `PAL0`, i.e., the background), while the rest of the palette stays black:

```
PAL_fadeIn(0, 15, our_level_palette, 60, true);
```

With the range limited to 0-15, the background appears with its real colors while the foreground and sprites remain black, as silhouettes.

Extending the range to 0 and 31 includes the first 32 colors of the palette — that is, both the background and the foreground — while the player and enemy sprites (`PAL2` and `PAL3`) still remain black.

This is the foundation on which more elaborate staggered fade effects are built, where different parts of the scene appear at different moments instead of all at once. Achieving that effect requires preparing additional arrays with different palettes and experimenting with `PAL_fadeIn()` calls staggered over time (for example, triggering each call with a slight delay relative to the previous one), though working out that example with full code is beyond the scope of this chapter: the partial ranges 0-15 and 0-31 are already enough to demonstrate the underlying principle.

Review exercises

1. Explain why copying an image's color data into an array with `memcpy()` produces, by itself, no visible change on screen.
2. Describe the five parameters of `PAL_fadeIn()` and the effect of each one, paying special attention to the difference between the index range it takes and the color count taken by `PAL_setColors()`.



Figure 99: Fade restricted to the 0-15 range, only the background with real colors

3. Why is it necessary to call `PAL_setColors()` with the black palette before loading the graphic resources, and what happens if that call is placed after loading instead?
4. Compare the behavior of the fifth parameter of `PAL_fadeIn()` when set to `true` versus `false`, and give a scenario where `false` might be the more appropriate choice.
5. Explain how a partial range of indices in `PAL_fadeIn()` (for example, 0-15 or 0-31) could be used as a starting point for building a staggered fade effect with several parts of a scene appearing at different moments.

Chapter 31 — D-Pad Controls

From sprites that are visible to sprites that move: position variables, a function of your own, the if statement, and dynamic animation assignment based on player input

Part III left the graphical side of a playable character fully resolved: how to display it on screen, how to animate it, and how to build its own animation sheet. What’s missing is the piece that turns all of that into a game: letting the player control the character. This chapter tackles that piece through the simplest possible example — the D-pad of a single joypad — and along the way introduces two programming tools that will accompany the rest of this manual: functions of your own and the if statement.

The starting point is a character already on screen, standing still at the center, playing a fixed animation. Everything that follows aims, first, to make that character move and, afterward, to make that movement respond to what the player presses on the D-pad instead of being an automatic motion.

Prerequisites

Before continuing, it’s best to have covered:

- Chapter 21, “How to Display Sprites”: the `Sprite*` pointer, `SPR_addSprite()`, `SPR_setPosition()`, and `SPR_update()`, which are reused directly here.
- Chapter 22, “Creating Sprites and Animation Sheets”: in particular, the idea that an animation sheet is usually drawn facing naturally toward one side, and that the opposite side is handled with a horizontal flip instead of drawing a second sheet.
- Chapter 23, “Sprite Animation”: `SPR_setAnim()` and the practice of identifying each animation with a named constant (for example, via `#define`) instead of a bare number.
- Chapter 6, on data types: in particular, why it’s best to declare a screen coordinate as `s16` rather than `int`.

1. From fixed coordinates to position variables

Up to now, a sprite’s position was passed to `SPR_addSprite()` as a pair of numbers hardcoded into the code, for example 100 and 50. For the character to be able to move, those two values need to stop being constants and become variables that the program can modify while the game runs. The solution is to declare two variables, with names that make it clear they represent the player’s position — so as not to confuse them, for instance, with any background-scrolling variable that might exist in the same program — and use them wherever there used to be bare numbers:

```
s16 player_x = 100;
s16 player_y = 50;
```


The type chosen here is `s16`, not `int`, deliberately: as explained in Chapter 6, `SPR_setPosition()` expects precisely `s16` coordinates (a signed 16-bit integer, from -32,768 to 32,767), which is more than enough for any screen or map position of reasonable size. Declaring `player_x` and `player_y` as `int` would also work — with values of at most a few hundred pixels there's no practical problem — but it's less tidy: each `int` takes up twice the memory unnecessarily, and its value has to be implicitly converted every time it's passed to a function expecting `s16`. Using `s16` from the start avoids that conversion and is, moreover, the type that appears in much of SGDK's example code for this same purpose.

Compiling at this point shouldn't change anything visible yet: the character appears in exactly the same place as before, because the variables start out with the same values as the numbers they replace. What changes is that these two variables can now be modified at any later point in the program, and every time they change, the sprite engine needs to be told the new position by calling `SPR_setPosition(player, player_x, player_y)` — the same function already used in Chapter 21 — followed, as always, by `SPR_update()` so the change actually reaches the screen.

As a first test, incrementing `player_x` at some point inside the main loop (`while(1)`) makes the character move one pixel per frame to the right — that is, 60 pixels per second, given that the Mega Drive updates the screen at 60 frames per second in NTSC mode — completely automatically, without the player intervening yet. That automatic result is only an intermediate step: the ultimate goal is for that increment to depend on what the player presses, rather than happening unconditionally on every frame.

2. Isolating the code into a function of your own: `handle_input()`

Before getting to the controls themselves, it's worth tidying up the code. Everything related to the player's position and movement is going to grow quite a bit over the course of this chapter, and leaving it loose inside the main loop, alongside the rest of the game's initialization, quickly makes the code hard to read. The usual solution — and the first time this manual builds a function of its own, rather than using only functions already provided by SGDK — is to create a function dedicated to handling player input:

```
static void handle_input()
{
}
```

The syntax of a function has three recognizable parts. The type before the name (`void` in this case) indicates what the function returns: `void` means it doesn't return any value, something revisited in more detail later in this manual. The word `static` in front limits the function's visibility to this same file, a detail that also doesn't need to be explored in depth yet. The parentheses after the name are where the data the function needs to receive from the calling code would go (`SPR_setAnim()`, for example, needs the sprite and the desired animation to be indicated there), but `handle_input()` doesn't need to receive any external data, so the parentheses are left empty. Finally, the braces delimit the body: all the code the function actually executes when it's called.

With the function already created, the code that incremented `player_x`, incremented `player_y`, and called `SPR_setPosition()` is cut from the main loop and pasted inside those braces:

```
static void handle_input()
{
```

```

    player_x += 1;

    SPR_setPosition(player, player_x, player_y);
    SPR_update();
}

```

The last step is to call the function from the main loop, simply writing `handle_input();` in the spot where the code used to be:

```

while (TRUE)
{
    handle_input();

    SYS_doVBlankProcess();
}

```

When the program reaches that line, it jumps to the function’s definition, executes everything inside its braces, and then resumes right after the call. The visible result is identical to what there was before moving the code: what changes is the organization, not the behavior. As programs grow, this pattern — pulling a block of code out of the main loop and encapsulating it in a function with a name that describes what it does — will recur frequently throughout the rest of this manual.

3. Order matters: declare before use

Before moving on, let’s see what happens if the order is reversed, because the resulting error shows clearly how the C compiler processes a file. If `handle_input()` is written above the declaration of `player_x` and `player_y`, instead of below as in the previous example, compilation fails with a message along the lines of `player_x undeclared (first use in this function)`.

Unlike other, more modern languages that analyze an entire file before running anything, the C compiler used by SGDK processes the file from top to bottom, in a single pass. When it reaches the line `player_x += 1;` inside `handle_input()`, if it hasn’t yet seen any declaration of `player_x` further up in the file, it has no way of knowing that variable will exist a little further down: it simply doesn’t recognize it, reports the error, and halts compilation right there.

The practical rule is simple, and it applies generally, not just to this particular case: any variable (or function) has to be declared earlier, further up in the file, than the first place where it’s used.

Moving `handle_input()` back to its original position, after the declarations of `player_x` and `player_y`, makes the error disappear.

4. Reading the D-pad: `JOY_readJoypad()` and the first `if`

With the code now tidied up, the next step is to replace the automatic increment of `player_x` with a conditional increment: one that only happens if the player is pressing the D-pad to the right. Reading the joypad’s state on a given frame is the job of a single SGDK function, `JOY_readJoypad()`, which needs to be told which joypad to read — `JOY_1` for the first controller, the only one used in this chapter — and which returns a value where each bit indicates whether a button is pressed or not:


```
u16 value = JOY_readJoypad(JOY_1);
```

Reacting to that value requires this manual’s first `if` statement. An `if` has two parts: a condition in parentheses and a block of code in braces that executes only if that condition is true — “if (this is true), then (do this).”

The condition for this first case combines the value read from the joypad with the `BUTTON_RIGHT` constant — provided by SGDK, one among several with analogous names for each button, including the D-pad’s four directions — using the `&` operator (a single symbol, not two). That operator compares the two values bit by bit, and the result is nonzero only when the bit corresponding to the right button is actually active in the value read: in practice, that expression works as a question of “is the right button pressed right now?”.

```
static void handle_input()
{
    u16 value = JOY_readJoypad(JOY_1);

    if (value & BUTTON_RIGHT)
    {
        player_x += 1;
    }

    SPR_setPosition(player, player_x, player_y);
    SPR_update();
}
```

It’s worth paying attention to the fact that the names of these constants are case-sensitive, just like any other identifier in C: writing them differently from how SGDK defines them produces a compilation error. With the condition complete, compiling and testing in the emulator confirms that the character now moves to the right only while the D-pad is held in that direction, and stops as soon as it’s released.

As already noted in Chapter 4, different emulators can differ in specific compatibility details. If an input example like this behaves differently between two emulators, it’s best to first rule out your own code by testing it in more than one emulator before assuming the problem lies in the program.

5. The four directions and efficiency with `else if`

Repeating the same recipe — an `if`, a `BUTTON_*` constant, an increment or decrement of `player_x` or `player_y` — for all four D-pad directions would already be enough for complete movement in all four directions, including diagonals, if each direction is written as an independent `if`. However, there’s a simple, zero-cost optimization available here: the physical D-pad makes it impossible to press right and left at the same time (they’re the same axis, in opposite directions), and the same goes for up and down. That physical impossibility can be exploited by replacing the second `if` of each pair with an `else if`: if it’s already been confirmed that the right button is pressed, there’s no need to even evaluate the condition for the left button, because both can’t be true at once.

```
if (value & BUTTON_RIGHT)
{
    player_x += 1;
}
else if (value & BUTTON_LEFT)
```

```

{
    player_x -= 1;
}

if (value & BUTTON_UP)
{
    player_y -= 1;
}
else if (value & BUTTON_DOWN)
{
    player_y += 1;
}

```

The detail that needs care is that these two optimizations must remain in separate pairs — one for the horizontal axis, another for the vertical one — and not have all four directions chained into a single continuous if / else if / else if / else if sequence. That distinction isn't cosmetic: if the four directions end up chained as a single sequence, the result would no longer support diagonal movement, because once the condition for the right button (or left) turns out true, the rest of the chain — including the up and down conditions — is skipped entirely, even if the player is also pressing a vertical direction. This is the code that should **not** be written:

```

/* Incorrect version: breaks diagonal movement */
if (value & BUTTON_RIGHT)
{
    player_x += 1;
}
else if (value & BUTTON_LEFT)
{
    player_x -= 1;
}
else if (value & BUTTON_UP)
{
    player_y -= 1;
}
else if (value & BUTTON_DOWN)
{
    player_y += 1;
}

```

The difference between the two versions is subtle to read but very visible when playing: with two independent pairs, pressing right and up simultaneously moves the character diagonally without issue, because each axis is evaluated in its own if, independent of the other. With the single chain, pressing up while right is already being pressed has no effect at all, because the program never gets to evaluate that condition. It's worth trying both versions in the emulator: it's an easy mistake to make without realizing why the outcome changes.

6. Animations that respond to movement

The character already moves in all four directions, but it still shows the same fixed animation the whole time, the one it started the chapter with. The next step is for the animation played to also depend on whether the player is moving or not, reusing the animation constants defined in Chapter 23 (ANIM_STILL, ANIM_IDLE, ANIM_WALK, ANIM_UPPER, or whatever equivalent names

were chosen for the sprite sheet in use).

With that set of constants already available, each of the four `if / else if` blocks from the previous section can include, in addition to the coordinate change, a call to `SPR_setAnim(player, ANIM_WALK)`: while the player presses any direction, the character walks. What's still missing is the opposite case — which animation to show when no direction is being pressed — which requires a new condition: that none of the four directions is active at the same time. That's written by negating each individual condition with the `!` ("not") operator and combining the four negations with the `&&` operator (two `&` symbols, unlike the single `&` used to read an individual button):

```
static void handle_input()
{
    u16 value = JOY_readJoypad(JOY_1);

    if (value & BUTTON_RIGHT)
    {
        player_x += 1;
        SPR_setAnim(player, ANIM_WALK);
    }
    else if (value & BUTTON_LEFT)
    {
        player_x -= 1;
        SPR_setAnim(player, ANIM_WALK);
    }

    if (value & BUTTON_UP)
    {
        player_y -= 1;
        SPR_setAnim(player, ANIM_WALK);
    }
    else if (value & BUTTON_DOWN)
    {
        player_y += 1;
        SPR_setAnim(player, ANIM_WALK);
    }

    if (!(value & BUTTON_RIGHT) && !(value & BUTTON_LEFT) &&
        !(value & BUTTON_UP) && !(value & BUTTON_DOWN))
    {
        SPR_setAnim(player, ANIM_IDLE);
    }

    SPR_setPosition(player, player_x, player_y);
    SPR_update();
}
```

Using two different symbols for operations that look similar at first glance — `&` to ask whether a specific bit of a value is active, `&&` to combine several true-or-false conditions with each other — isn't an arbitrary style choice: they're two distinct operators in C, with distinct meanings, and although in some cases using one instead of the other compiles just fine, the result isn't the same. It's best to treat them as separate tools — the first for inspecting a bit-value like the one returned by `JOY_readJoypad()`, the second for combining boolean conditions like those in an `if` — rather than swapping them freely.

The result combines both behaviors: the character walks while any direction is pressed and returns to idle as soon as all of them are released. This same scheme — a walking animation shared by all four directions and an idle one when there’s no input — is common even in games where up and down don’t have a dedicated walking animation distinct from the sideways one; each project can decide that detail on its own.

7. Orienting the sprite: `SPR_setHFlip()` and the “moonwalk” problem

One last visual detail remains. As explained in Chapter 22, it’s common for an animation sheet to be drawn facing naturally toward just one side; to show the character facing the opposite direction, a horizontal flip is used, either fixed when the sprite is loaded — via `TILE_ATTR()`, seen in Chapter 21 — or, as is needed here, changed dynamically while the game runs. SGDK provides `SPR_setHFlip()` for this, which takes the sprite to modify and a boolean value indicating whether it should be flipped or not:

```
SPR_setHFlip(player, FALSE);
```

Adding that line inside the leftward-movement block, the character is shown in its natural orientation when moving to that side. But moving to the right — which, before this change, displayed correctly because the sprite was originally loaded with the flip enabled in `TILE_ATTR()` — now keeps the flip applied from the last time, even after moving right again: the character walks facing away from its own direction of travel, an effect informally known as “moonwalking.” The cause is simply that there’s still no instruction telling the sprite to stop being flipped when moving right. The solution is to add the equivalent call, with the opposite value, in the right-button block:

```
if (value & BUTTON_RIGHT)
{
    player_x += 1;
    SPR_setAnim(player, ANIM_WALK);
    SPR_setHFlip(player, TRUE);
}
else if (value & BUTTON_LEFT)
{
    player_x -= 1;
    SPR_setAnim(player, ANIM_WALK);
    SPR_setHFlip(player, FALSE);
}
```

With both calls in place, `TRUE` when moving right and `FALSE` when moving left, the character stays correctly oriented no matter which direction it moves in.

Everything covered so far addresses input by polling: `JOY_readJoypad()` is queried once per frame, inside the main loop, and the code reacts to whatever it finds at that instant. SGDK also offers an event-based mechanism, `JOY_setEventHandler()`, which allows reacting to joypad state changes without having to poll it frame by frame; it’s a valid alternative for certain cases, though the direct polling used in this chapter is sufficient and is the simplest starting point for understanding how player input relates to the game’s state.

Review exercises

1. Explain, in your own words, why declaring a function before the variables it uses produces a compilation error in C, and what general rule should be followed to avoid it.
2. What's the difference between the `&` operator used in `(value & BUTTON_RIGHT)` and the `&&` operator used in the idle-state condition? Why can't they be used interchangeably?
3. A teammate writes the control logic for the four directions as a single `if / else if / else if / else if` chain, instead of two independent pairs. What concrete symptom will they notice while playing, and why does it happen?
4. Explain why the character in this chapter needs a call to `SPR_setHFlip()` in the rightward-movement block in addition to the one already in the leftward-movement block, and what would happen if one of the two were omitted.
5. Describe how you would extend this chapter's code so that the character, instead of only walking or standing still, could also run while holding down an additional button. What new variable or condition would need to be added?

Chapter 32 — Button Controls and Event-Based Reading

Chapter 31 solved character movement through continuous polling: on every frame, `handle_input()` asks `J0Y_readJoypad()` all over again what's being pressed at that instant. That approach works well for the D-pad, but it isn't the only way SGDK offers to handle player input, and in fact it isn't the best fit for everything. This chapter introduces the second approach: an event-based mechanism, where the code doesn't actively ask on every frame, but instead SGDK automatically notifies it whenever something changed on the joypad.

The title speaks of “button controls” following the more common convention, but it's worth clarifying from the start that the real distinction isn't D-pad versus buttons, but continuous versus one-shot: in a menu navigated with the D-pad, the event mechanism is the right fit, and in an attack that's held down, continuous polling is the right fit, even though in both cases the physical control is the same kind of button. The second half of the chapter picks up this distinction with a concrete example: an attack animation that needs to play out in full even if the player releases the button before it finishes, which requires introducing a timer.

Prerequisites

- Having completed Chapter 31 (“D-pad Controls”): the `handle_input()` function, `J0Y_readJoypad()`, the `BUTTON_*` constants, the `&` operator for reading a single bit, and the use of `if / else if / && / !` already explained there won't be repeated in detail here.
- Having completed Chapter 6 (“C Applied to Retro Development: Data Types, Limited Memory, and Pointers”): the `u8`, `u16`, and `s16` types, and the criterion of choosing the smallest type that fits the expected range of values, are revisited in this chapter for the attack timer variables.
- Ideally, having completed Chapter 23 (“Sprite Animation”): the timing field of an animation in `resources.res` is measured in sixtieths of a second per frame, a unit this chapter uses again to calculate how long an attack animation should last.

1. Two ways of reacting to player input

Before writing code, it's worth looking at how controls behave in existing games, because the difference is easier to notice by playing than by reading. In a typical platformer or action game, moving with the D-pad is a continuous process: while the player holds the right side down, the character keeps moving right, frame after frame, without needing to release and press again. Attacking or jumping, on the other hand, is usually a one-shot event: pressing the button once triggers the action exactly once, and even if the player keeps the button held down, the action doesn't repeat until it's released and pressed again.



Figure 100: Comparison between continuous movement and a one-shot attack

Holding down the attack button doesn't produce repeated hits: you have to release and press again for each one, unlike the continuous movement of the D-pad.

There are, however, situations where the D-pad, normally associated with continuous movement, also benefits from one-shot behavior. The clearest example is options menus: if moving the cursor between "1 Player," "2 Players," "Options," and "Battle" responded to the D-pad the same continuous way character movement does, updating the selection on each of the 60 frames per second while the button stays pressed, it would be practically impossible to stop on the desired option without overshooting it by several positions.

In an options menu, each D-pad press moves the cursor exactly one position, no matter how long the button is held down.

In short, the choice between continuous polling and a one-shot event doesn't depend on whether it's the D-pad or a button, but on what kind of response that particular action needs. As a general rule and a starting point — not as an absolute law — character movement usually belongs in the continuous polling function (`handle_input()`, already built in Chapter 31), and single-shot actions belong in the event mechanism this chapter introduces. But, as we'll see further on, there are legitimate exceptions in both directions.

2. The event mechanism: `JOY_setEventHandler()` and a function with a fixed signature

SGDK offers a second way to read the joypad, different from calling `JOY_readJoypad()` inside the main loop: registering your own function that SGDK will call automatically every time the state of some joypad changes. That function has to have an exact signature, predefined by SGDK through the `JoyEventCallback` type:



Figure 101: Options menu navigated with one-shot D-pad control

```
typedef void JoyEventCallback(u16 joy, u16 changed, u16 state);
```

Its three parameters indicate, in order, which joypad changed (*joy*), which buttons changed state since the last reading (*changed*), and what the full current state is (*state*). Unlike `handle_input()`, written with total freedom of name, parameters, and how it's called, this function has to respect that exact three-parameter `u16` signature so SGDK can invoke it correctly. The function's name is indeed free — in this manual it's called `joyEvent`, but you can choose any other name — as long as the order and type of the parameters are preserved.

Inside it, *changed* indicates, bit by bit, which buttons changed state since the last reading, and *state* indicates, also bit by bit, which ones are pressed right now. Combining both with the `&` operator — for example, `changed & state & BUTTON_RIGHT` — is equivalent to asking “did the right button change state, and is that new state pressed?”, which is exactly the definition of “the right button was just pressed”: if the button was already pressed on the previous frame, *changed* would be zero for that bit and the whole expression would come out to zero, even though *state* still marks the button as pressed.

It's worth pausing on this expression for a moment, because there's an alternative form, documented in the official SGDK wiki, that separates the two operators: `(changed & BUTTON_X) && (state & BUTTON_X)`. For a single button, both forms are mathematically equivalent. Since `BUTTON_X` has a single bit set to 1, the result of `changed & state & BUTTON_X` is nonzero exactly when that bit is set to 1 in both *changed* and *state*; and that's precisely the same thing evaluated separately by `changed & BUTTON_X` (nonzero if the bit changed) and `state & BUTTON_X` (nonzero if the bit is pressed), combined with `&&`. This equivalence holds **only for testing a single button at a time**: it can't simply be generalized to a mask that combines several different buttons with `|` (for example, `BUTTON_A | BUTTON_B`). In that case, `changed & state & mask` requires that there be a bit within the mask that's set simultaneously in *changed* and in *state*, while `(changed & mask) && (state & mask)`

is satisfied if *some* bit of the mask changed and *some* bit of the mask (not necessarily the same one) is pressed — two different conditions that can produce different results. For a single button this distinction disappears, but it's worth keeping in mind before copying the pattern into a condition with multiple buttons.

The other point where this function departs from what's been seen so far is how it gets triggered. The custom functions built so far, like `handle_input()`, need an explicit call inside the main loop to run on every frame. This function, by contrast, is never called directly anywhere: it's registered once, in `main()`, with `JOY_setEventHandler()`:

```
void JOY_setEventHandler(JoyEventCallback *CB);
```

passing it the function's name (without parentheses, since here it isn't being called but rather pointed out to SGDK):

```
JOY_setEventHandler(joyEvent);
```

From that registration onward, SGDK takes care of invoking `joyEvent()` automatically every time it detects a change on the joypad, without the main loop having to actively ask about it. To check the difference in behavior against Chapter 31, you can duplicate the same movement code from `handle_input()` inside `joyEvent()` — replacing only `value & BUTTON_*` with `changed & state & BUTTON_*` — and test it in the emulator. The result confirms the idea from the previous section: pressing and holding a direction moves the character exactly once, and you have to release and press again to move it further — precisely the one-shot behavior that doesn't work for smooth character movement, but does work for a menu.

The `BUTTON_A`, `BUTTON_B`, and `BUTTON_C` constants, just like `BUTTON_RIGHT` and the rest of the direction constants already used in Chapter 31, remain unchanged in SGDK 2.11.

3. Commenting out code versus deleting it

A habit worth adopting early, and one that becomes especially relevant when switching back and forth between testing movement in `joyEvent()` and returning it to `handle_input()`, is telling apart when commenting out code with `//` makes sense and when it doesn't. Commenting out code makes sense when there's a real intention to bring it back later — for example, to compare two alternative behaviors while deciding which one to use — but any line that's known to be useless going forward should simply be deleted rather than left commented out. Commented-out code that piles up without a clear purpose is, over time, a source of bugs: it becomes hard to tell at a glance which commented block is intentional and which is just something that was never cleaned up, and a stray line that gets accidentally uncommented along with the rest of a block can introduce unwanted behavior, such as a position variable that keeps incrementing with no condition attached, hard to trace if the file has too many commented-out fragments scattered without any criteria.

4. A first attempt at an attack, and why it falls short

With the event mechanism already tested, the rest of the chapter focuses on a more realistic example: an attack animation triggered with the B button, using the character's animation sheet already seen in previous chapters. The least elaborate reaction would be to add, directly inside `handle_input()`, one more condition for the B button alongside the ones that already exist for the four directions:

```
if (value & BUTTON_B) SPR_setAnim(player, ANIM_UPPER);
```

The problem with this approach shows up immediately when testing it: since `handle_input()` re-evaluates this condition on every one of the 60 frames per second for as long as continuous polling runs, holding down the B button plays the attack animation uninterrupted, instead of a single hit. This is exactly the continuous behavior section 1 identified as unsuitable for an attack action: the player would expect to press the button once, see the full hit play out, and only then be able to attack again.

Simply pulling this code out of `handle_input()` and putting it into `joyEvent()`, using `changed` & `state` instead of continuous polling, fixes the repetition problem but introduces a new one: the event only triggers `SPR_setAnim()` during the single frame in which the change is detected, so the full animation never gets to play — you barely see one loose frame of the hit before the idle code (still polling continuously in `handle_input()`) interrupts it and instantly snaps it back to the idle or walking animation.

What's missing, then, isn't simply choosing one of the two mechanisms, but combining them: use the event to trigger the attack exactly once, and some sort of elapsed-time count to keep that animation on screen for as long as it lasts, and, in the meantime, to keep the D-pad from interrupting the attack or being interrupted by it.

5. A timer to sustain the animation: `attack_timer` and `attack_duration`

The solution consists of adding two new variables, declared alongside the rest of the program's global variables: `attack_timer`, which starts at zero and keeps count of how many frames the current attack animation has been running, and `attack_duration`, a constant indicating how many frames that animation should last before returning to idle. Since both quantities are always non-negative and small in range — there's no need to represent large or negative numbers — it makes sense to declare them as `u16` rather than `int`, following the same criterion from Chapter 6 about choosing the type best suited to the expected range of values (it's also the same type already used by `changed` and `state` in the `joyEvent()` signature):

```
u16 attack_timer = 0;
u16 attack_duration; // computed further below
```

The value of `attack_duration` isn't picked by eye: it's computed from the same data already used in Chapter 23 to define an animation's speed in `resources.res`. If the attack animation has, say, seven frames, and each frame is configured to last eight sixtieths of a second (the same timing field of the `SPRITE` resource explained in that chapter), the total duration, in that same unit of sixtieths of a second, is the product of both numbers:

```
#define ATTACK_DURATION (7 * 8) // 7 frames x 8 sixtieths of a second = 56
```

This calculation is exactly reproducible for any other animation: it's enough to multiply the number of frames by the configured duration of each frame in `resources.res`, as long as — as is usually the case — all the frames of that animation share the same duration.

6. Starting the attack and suspending the D-pad while it lasts

With both variables already declared, the code inside `joyEvent()` that triggers the attack looks like this:

```

void joyEvent(u16 joy, u16 changed, u16 state)
{
    if ((changed & state & BUTTON_B) && attack_timer == 0)
    {
        SPR_setAnim(player, ANIM_UPPER);
        attack_timer += 1;
    }
}

```

Besides changing the animation, the condition increments `attack_timer` by one as soon as the B button press is detected, and requires `attack_timer` to be zero before starting: without this last addition, holding B down could restart the attack animation on every frame in which SGDK detected the button as just pressed. In practice this would no longer happen anyway, because, as we'll see next, `handle_input()` stays suspended during the attack; even so, it's a cheap check in terms of CPU that avoids restarting the count unnecessarily.

The other change is in the main loop: the call to `handle_input()`, which in Chapter 31 was made unconditionally, is now guarded:

```

if (attack_timer == 0) handle_input();

```

Since this condition has just one line of code to run if it's true, there's no need to wrap it in braces — both forms are equivalent in C, and using the brace-less version for a single statement is simply a matter of style, though it's worth going back to braces as soon as the block grows past one line, to avoid mistakes when adding code later without noticing it ended up outside the `if`. As long as `attack_timer` is zero, `handle_input()` runs as usual and the D-pad works just like in Chapter 31; as soon as the player presses B, `attack_timer` stops being zero and `handle_input()` simply stops being called, which freezes the character's movement during the attack, exactly as intended.

There's a syntax detail here that's a classic source of bugs in C: the condition `attack_timer == 0` uses two equal signs, because it's a logical comparison ("is it equal to zero?"), while assigning a value to a variable — as in `attack_timer = 0`, which appears further down — uses a single equal sign. Confusing the two operators is an easy mistake to make, one the compiler doesn't always catch, since `attack_timer = 0` inside an `if` condition is also a valid expression in C (it assigns zero and then evaluates that zero as false), only with a completely different effect than intended.

7. Keeping the count going while the attack plays out

Incrementing `attack_timer` just once, when B is pressed, isn't enough: it needs to keep adding one for every frame that passes while the attack is in progress, so that it eventually reaches `attack_duration` and the D-pad is freed again. That continuous increment can't live in `joyEvent()`, which only runs at the one-off instant the button's state changes; it has to run frame by frame, like the rest of the main loop. The solution is to add, alongside the `if (attack_timer == 0) handle_input();` already written, an `else if` condition covering the rest of the frames while the attack is underway:

```

if (attack_timer == 0)
    handle_input();
else if (attack_timer > 0 && attack_timer < attack_duration)
    attack_timer += 1;

```

This condition introduces two comparison operators — greater than (`>`) and less than (`<`) —

that haven't appeared in this manual until now, though their meaning is the same as in ordinary arithmetic. The full condition, `attack_timer > 0 && attack_timer < attack_duration`, reads as “while the attack has already started but hasn't finished yet,” and when it holds, it simply adds one more to the counter, frame after frame.

Before tackling the missing step, it's worth verifying that the `attack_duration` calculation actually matters: deliberately changing it to a value that's too high — 60 instead of 56, say, for the example of seven frames at eight sixtieths of a second each — causes the animation to complete its run and start over from the first frame before the counter gets around to releasing the D-pad, producing a small, visible repeat of the hit.

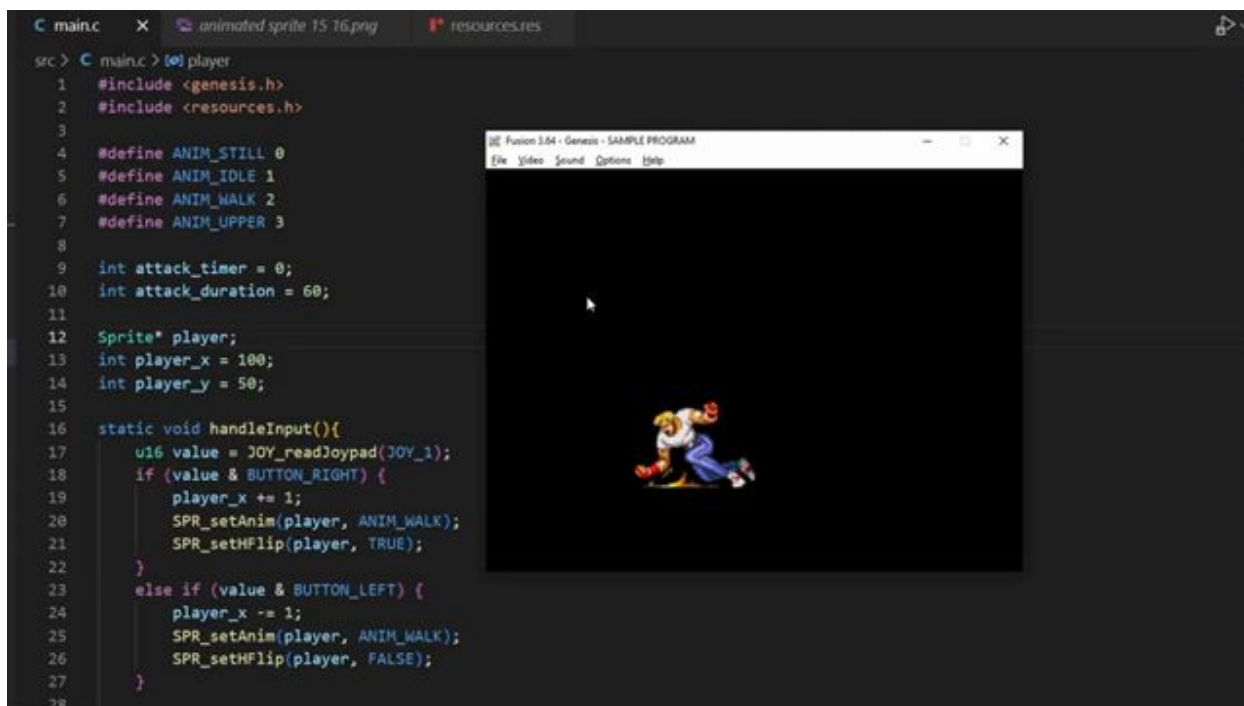


Figure 102: A miscalculated `attack_duration` produces a repeat of the hit

With `attack_duration` set to 60 instead of 56, the seven-frame animation has already finished its run but the timer still hasn't released the D-pad, so the hit repeats from the start before the character can move again.

8. Closing the loop: resetting the timer, and simplifying with else

One step is still missing: at some point, `attack_timer` has to go back to zero so the cycle can repeat. Without that last step, once the count up to `attack_duration` finishes, the counter would simply stop incrementing (because neither of the two conditions already written would still be true), but it would never go back to zero, leaving the D-pad frozen forever.

The missing step is adding a third condition that detects when `attack_timer` has reached exactly `attack_duration`, and in that case resets it to zero, freeing the D-pad again. That condition could be written explicitly, comparing `attack_timer == attack_duration`, but since it's exactly the one and only case not already covered by the two previous conditions (neither zero, nor a value between zero and `attack_duration`), a plain `else`, with no condition of its own, is enough to catch everything left over:

```

if (attack_timer == 0)
    handle_input();
else if (attack_timer > 0 && attack_timer < attack_duration)
    attack_timer += 1;
else
    attack_timer = 0;

```

This kind of simplification is convenient, but it demands close attention: an `else` with no condition runs its code every time none of the previous branches held, so you have to be sure that “everything else” matches exactly the case you want to cover, and not some unforeseen one. In this particular case, with only three possible states for `attack_timer` (zero, in progress, or finished), the simplification is safe and saves an explicit comparison, but it isn’t a technique to apply without thinking in more complex chains of conditions.

With the logic now complete and tested, one last purely organizational adjustment is worth making, in the same spirit that already motivated creating `handle_input()` in Chapter 31: rather than leaving these three loose lines inside the main loop, wrap them in their own function and call it from there, so the main loop stays readable at a glance:

```

void attack_function(void)
{
    if (attack_timer == 0)
        handle_input();
    else if (attack_timer > 0 && attack_timer < attack_duration)
        attack_timer += 1;
    else
        attack_timer = 0;
}

```

And in `main()`:

```

JOY_setEventHandler(joyEvent);

while (TRUE)
{
    attack_function();
    SPR_update();
    SYS_doVBlankProcess();
}

```

There’s one remaining difference worth highlighting between the two custom functions coexisting in this program: `handle_input()` and `attack_function()` are ordinary functions, needing an explicit call inside `while (TRUE)` to run every frame, while `joyEvent()` is the exception, registered once with `JOY_setEventHandler()` and never called directly from your own code. With this reorganization, the result behaves exactly the same as before — the D-pad moves the character normally, the B button triggers a single hit that plays all the way through, and only then does the D-pad respond again — but with a considerably tidier main loop.

9. So, `handle_input()` or `joyEvent()`?

With the full example already built, it’s worth revisiting the question from section 1 with more to go on. The general rule — continuous movement in `handle_input()`, one-shot actions in `joyEvent()` — remains a good starting point, but this very chapter showed two concrete exceptions worth keeping in mind when designing the controls of your own game. On one

hand, an options menu navigated with the D-pad is one of the few common situations where it makes sense to move the direction code into `joyEvent()`, precisely because that's where a one-shot movement is wanted, one per press, rather than continuous. On the other hand, an action that needs to measure how long a button is held down (a charged attack, for example) has to rely on continuous polling inside `handle_input()` (or an equivalent function), since `joyEvent()` only reports the instant something changes, not how long a state is held.

The choice, then, isn't a mechanical rule but a design decision that depends on what each particular action needs: if the player expects holding the button down to produce a continuous effect, use polling; if they expect each press to count as an isolated event, use the event mechanism. This chapter's attack example, with its timer, also shows that both mechanisms can be combined within a single action when needed: an event to trigger something once, and continuous polling to sustain it over time.

Review exercises

1. Explain in your own words the difference between the changed parameter and the state parameter of an event function like `joyEvent()`, and why both need to be combined (with `&`) to detect the instant a button is pressed, rather than using just one of the two.
2. Why is an options menu navigated with the D-pad usually implemented with the event mechanism (`joyEvent()`) rather than the continuous polling (`handle_input()`) used to move the character?
3. A teammate writes `if (value & BUTTON_B) SPR_setAnim(player, ANIM_UPPER);` inside `handle_input()` to program an attack. When testing it, holding down the B button produces a continuous, unbroken attack instead of a single hit. Explain why this happens and what's missing to achieve the intended behavior.
4. A jump animation has 5 frames, each configured with a duration of 10 sixtieths of a second in `resources.res`. Calculate what a constant equivalent to `attack_duration` should be worth for that animation.
5. Show, with a concrete example (values for changed, state, and a mask combining two different buttons, for example `BUTTON_A | BUTTON_B`), a case where `changed & state & mask` and `(changed & mask) && (state & mask)` produce different results, and explain why the equivalence between the two forms only holds for a single button.

Chapter 33 — 2-Player Mode

Doubling the sprite, the joypad, and the attack timer without doubling the bugs

This chapter doesn't introduce any new SGDK functions: it takes everything built in the previous two chapters —`handle_input()` for the D-pad and `joyEvent()` / `attack_function()` for the timed attack— and extends it from one player to two, with Axel and Sonic sharing the screen and each responding to their own joypad. The technical result is modest: duplicate variables, duplicate blocks of code. But doing it properly exposes several practical problems that don't show up with a single player: how to copy code without making mistakes when renaming variables, why two different sprite sheets don't necessarily share the same row layout, and a logic bug that ends up moving both characters at twice the expected speed.

Extending a single-player system to several players is almost never as simple as “just add more code”: it often forces you to revisit code that already worked, because assumptions that were valid with one player —such as a given function being called exactly once per frame— stop holding as soon as more than one character shares the same logic.

Prerequisites

Before continuing, you should already have in place:

- D-pad controls (chap. 31): `handle_input()`, `JOY_readJoypad()`, and the `if / else if` structure for the four directions.
- Button controls (chap. 32): `joyEvent()`, `JOY_setEventHandler()`, and the `attack_timer / attack_duration / attack_function()` mechanism for sustaining an attack animation.
- The fact that the Mega Drive has four hardware palettes, PAL0 through PAL3 (chap. 2), and that each sprite is assigned one of them when it's loaded.

This last point is not a minor detail for this chapter: as we'll see shortly, a second playable character uses up the last available palette.

1. A second sprite: Sonic on palette PAL3

The first step is a resource one: add Sonic's sprite sheet to the project with a `SPRITE` line in `resources.res`, the same way it was done for Axel back in the sprite creation chapter, specifying name, file, frame size, compression, and playback speed. With the resource already compiled, you need to declare a second sprite pointer and its own position variables, alongside the ones that already existed for Axel:

```
Sprite* player2;  
s16 player2_x = 300;  
s16 player2_y = 150;
```

Loading the sprite repeats exactly the same steps already seen for Axel, with `SPR_addSprite()` this time pointing to Sonic's resource and to palette PAL3 instead of PAL2:

```
player2 = SPR_addSprite(&sonic_sprite, player2_x, player2_y,  
    TILE_ATTR(PAL3, FALSE, FALSE, TRUE));
```

This palette choice isn't arbitrary, nor is there any room to pick something else: the Mega Drive has exactly four hardware palettes (PAL0 through PAL3), and in this program all four are already in use: one for the background and one for each playable character. With Axel using PAL2, PAL3 is the only palette left free for Sonic. This means that, as the example currently stands, there's no room to add a third sprite with its own palette without reusing one of the four already assigned: an additional enemy or item would have to share a palette with one of the existing elements, or give up part of its color range.

Compiling at this point—with Sonic already on screen but still without any controls attached—reveals a visual detail that at first glance looks like a bug: even though `player_x`, `player_y`, `player2_x`, and `player2_y` have exactly the same numeric values, Axel and Sonic don't appear at the same height on screen.

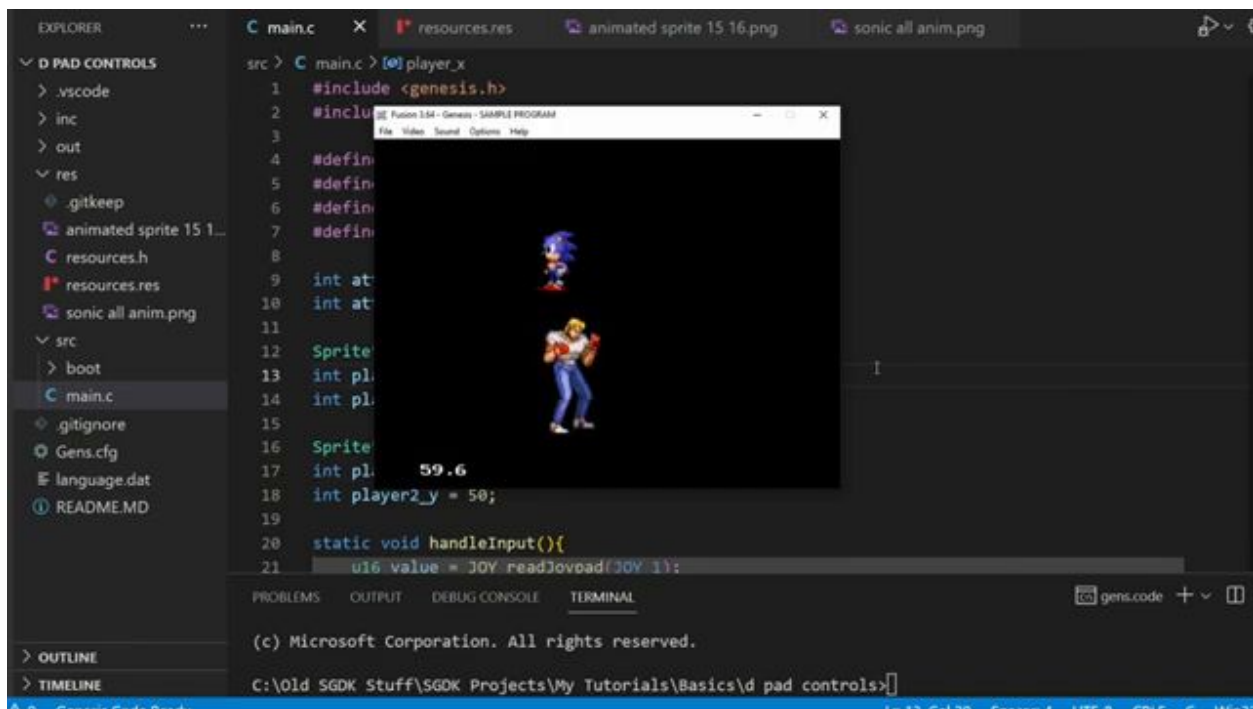


Figure 103: Axel and Sonic with the same position coordinates

Axel and Sonic share the same numeric position values, but appear at different heights on screen.

The explanation is the same idea already covered when displaying sprites: the position passed to `SPR_addSprite()` is the top-left corner of the entire animation frame, not of the figure drawn inside that frame. Since Axel's sheet has a larger frame size than Sonic's sheet—each sheet was created separately, with its own canvas—the same pair of numeric coordinates ends up placing the top-left corner of frames of different sizes, and the figure drawn inside each frame appears at a slightly different apparent position.

2. Avoiding manual copying: a throwaway file and find-and-replace

The next step —duplicating all the code in `handle_input()` and `joyEvent()` so that an equivalent version exists for `player2`— could be done by copying the `player` block and manually changing every instance of `value`, `player`, `player_x`, and `player_y` to its `player2` equivalent. With just a few lines this is tedious but manageable; in a larger program it would be slow and error-prone, because it's easy to forget to rename some stray instance. The cleaner alternative is to take advantage of find-and-replace, but applied carefully.

The first precaution is not to apply it directly on `main.c`: since the search scans the whole file, replacing `player` with `player2` would also affect any occurrence of `player` that already belongs to `player1`'s code, breaking it. The practical solution is to create a new, throwaway file inside the project's `src` folder —one that isn't included anywhere in the program or compiled—, paste a copy of `player1`'s code block into it, and make the replacements only on that isolated copy.

The second precaution has to do with two search options that are worth enabling deliberately. With neither enabled, searching for the word `value`, for example, finds any occurrence of those characters, even as part of a longer word. Enabling case sensitivity makes the search distinguish `value` from `VALUE`; enabling whole-word matching makes searching for `player` skip matches inside `player_x` or `player_y`, finding only the standalone word `player`.

That distinction matters because, in this particular case, it's fine to replace every occurrence of `value` with `value2` with no special restriction, but when replacing `player` with `player2` you need to enable whole-word matching. Without it, `player_x` would end up indirectly turned into `player2_x`, and then, on a later replacement pass over `player_x`, the result would come out duplicated or inconsistent. Before hitting “replace all,” it's worth checking how many matches the search tool reports: if the number doesn't match what you expected, that's a sign the search is finding something other than what you meant to replace. Once the copy has been fully renamed, the final step is to paste it back into `main.c`, below the original `player1` code, and discard the temporary file.

This same problem —manually duplicating a set of variables for every new game element— shows up much more noticeably once a game needs not two but several characters or enemies on screen at once. Later in this manual, when a game's entities are organized using structs (chap. 53), this same need to generalize a repeated pattern comes back, but with a tool designed specifically for that purpose.

3. The second joypad in `handle_input()`, and a mismatched mirror

With the block already duplicated and renamed, the first real change inside `handle_input()` is reading a second joypad in addition to the first, leaving a comment to visually mark where each player's code begins:

```
void handle_input() {
    u16 value = JOY_readJoypad(JOY_1);
    u16 value2 = JOY_readJoypad(JOY_2);

    // PLAYER1
    if (value & BUTTON_RIGHT) {
        player_x += 1;
        SPR_setHFlip(player, FALSE);
    }
}
```

```

        SPR_setAnim(player, ANIM_WALK);
    } else if (value & BUTTON_LEFT) {
        player_x -= 1;
        SPR_setHFlip(player, TRUE);
        SPR_setAnim(player, ANIM_WALK);
    }
    // ... rest of player1's directions ...

    // PLAYER2
    if (value2 & BUTTON_RIGHT) {
        player2_x += 1;
        SPR_setHFlip(player2, TRUE);
        SPR_setAnim(player2, ANIM_WALK);
    } else if (value2 & BUTTON_LEFT) {
        player2_x -= 1;
        SPR_setHFlip(player2, FALSE);
        SPR_setAnim(player2, ANIM_WALK);
    }
    // ... rest of player2's directions ...

    SPR_setPosition(player, player_x, player_y);
    SPR_setPosition(player2, player2_x, player2_y);
}

```

This separation with comments (*// PLAYER1*, *// PLAYER2*) is a simple but useful convention as a single function grows to handle more than one character.

Testing this, both characters already move in all four directions with their respective D-pads, but a visual problem shows up with Sonic: when moving left or right, he appears to be walking backwards relative to his actual direction of movement.

Sonic walks to the left, but the sprite ends up facing the opposite of his actual direction of movement.

This isn't a logic error but, once again, a difference between the two sprite sheets: Axel's sheet was drawn facing left by default (as already seen in the D-pad controls chapter), while the Sonic sheet used here was drawn facing right by default. Since the `SPR_setHFlip()` code copied for `player2` assumes the same default orientation as Axel, it ends up mirroring Sonic exactly backwards from what's needed. The fix —already applied in the block above— is to swap the `TRUE` and `FALSE` values in the two `SPR_setHFlip()` calls in the `player2` block, relative to the ones in the `player1` block, so they match Sonic's sheet's actual default orientation.

4. A second attack timer in `joyEvent()`

The D-pad already works for both characters; the attack button is still missing. The first obstacle is that `value` and `value2`, declared inside `handle_input()`, stop existing as soon as that function finishes executing. Since `joyEvent()` is a different function, it needs its own local variables to read both joypads, even though the name and purpose are the same as in `handle_input()`. With those variables declared, each player's attack code is wrapped in an additional condition that checks which of the two joypads generated the event:

```

void joyEvent(u16 joy, u16 changed, u16 state) {
    u16 value = JOY_readJoypad(JOY_1);
    u16 value2 = JOY_readJoypad(JOY_2);

```

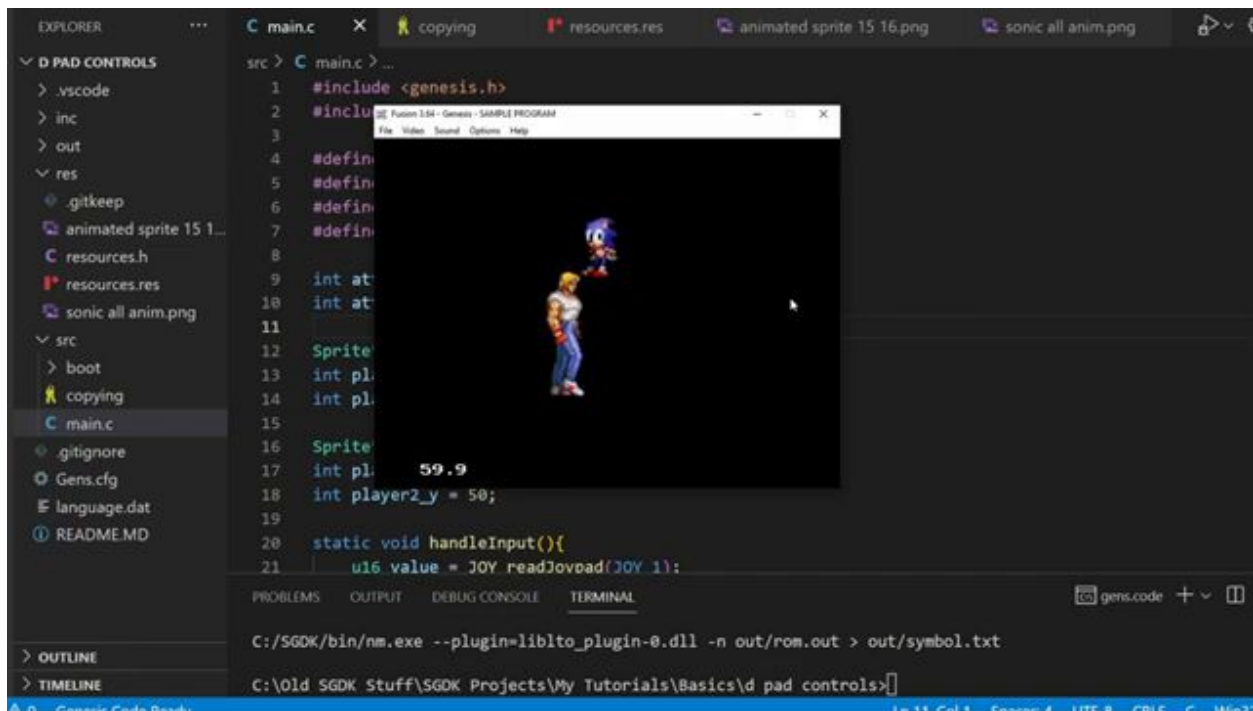


Figure 104: Sonic with the sprite orientation flipped while moving

```
// PLAYER1
if (value) {
    if ((changed & BUTTON_B) && (state & BUTTON_B)) {
        SPR_setAnim(player, ANIM_UPPER);
        attack_timer = 1;
    }
}

// PLAYER2
if (value2) {
    if ((changed & BUTTON_B) && (state & BUTTON_B)) {
        SPR_setAnim(player2, ANIM_SPIN);
        attack_timer2 = 1;
    }
}
}
```

Each player also needs their own timer and their own attack duration —`attack_timer2` and `attack_duration2`—, declared alongside the ones that already exist for `player1`, since both characters can start attacking at different moments and completely independently of one another.

There's a detail here that isn't obvious until you check it directly against the sprite sheet: on Axel's sheet, the attack animation is row number 3 (counting from zero), identified by the constant `ANIM_UPPER`. On Sonic's sheet, the equivalent attack animation isn't on the same row: it's row number 7. This difference doesn't follow any rule from SGDK or `rescomp`: it's simply the result of each sprite sheet having been laid out independently by whoever drew it, and

there's no requirement that two different sheets share the same animation order. In practice, this means the row number used for Sonic's attack has to be defined with its own constant—for example `#define ANIM_SPIN 7`—instead of reusing `ANIM_UPPER`, which in this program still refers specifically to row 3 of Axel's sheet.

Before reusing an animation constant for a second character, it's worth checking the corresponding sprite sheet instead of assuming it shares the same layout as the first one. This isn't an SGDK rule: it's a decision each artist makes when organizing their sheet, and it varies from one sprite set to another.

5. A classic bug: calling `handle_input()` twice per frame

With `joyEvent()` complete, what's left is to extend `attack_function()` —the function that decides when the D-pad should be suspended during an attack— so it also accounts for player2. The most direct approach, copying the same pattern already used for player1, would be to add a second, independent block with its own condition on `attack_timer2`:

```
// Version with the bug: do NOT use this form
void attack_function() {
    if (attack_timer != 0) {
        attack_timer++;
        if (attack_timer == attack_duration) {
            attack_timer = 0;
        }
    } else {
        handle_input();
    }

    if (attack_timer2 != 0) {
        attack_timer2++;
        if (attack_timer2 == attack_duration2) {
            attack_timer2 = 0;
        }
    } else {
        handle_input();
    }
}
```

Compiling and testing this code reveals an immediate problem: both characters visibly move faster than expected, roughly at twice the speed.

After the same number of frames, both Axel and Sonic end up displaced twice as far as expected.

Once identified, the cause turns out to be obvious in hindsight: `handle_input()` is a single function that already contains, internally, the movement code for both players (player1's and player2's together, since section 3). The code above calls that same complete function once when `attack_timer` is zero, and again, separately, when `attack_timer2` is zero. In the normal situation —neither player attacking, both timers at zero— both conditions are true in the same frame, so `handle_input()` runs twice in a row within the same frame. Since each run moves both characters one more pixel in the pressed direction, the visible result is double the displacement per frame for both of them, not just for one.

The correct fix isn't to call `handle_input()` conditionally from `attack_function()`, but to move that condition inside `handle_input()` itself, wrapping each player's code block in its own check

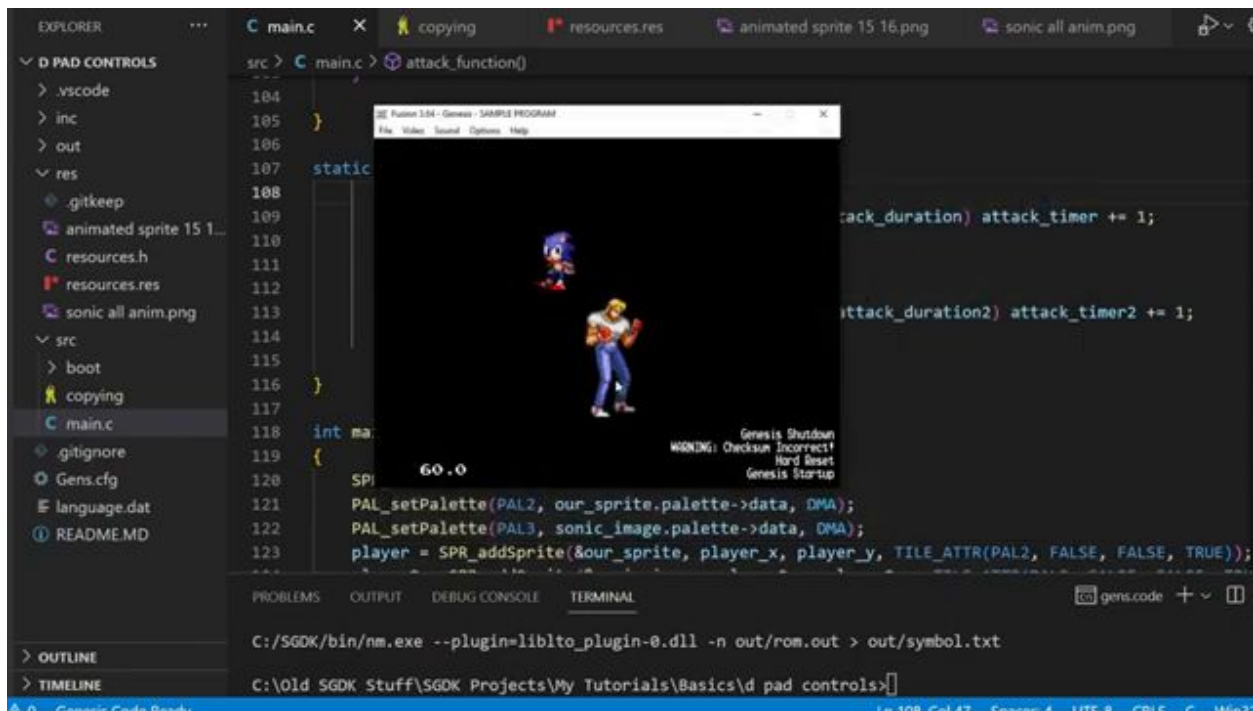


Figure 105: Axel and Sonic moving at twice the speed due to the bug

of whether that player is allowed to move this frame:

```
void handle_input() {
    u16 value = JOY_readJoypad(JOY_1);
    u16 value2 = JOY_readJoypad(JOY_2);

    // PLAYER1
    if (attack_timer == 0) {
        if (value & BUTTON_RIGHT) {
            player_x += 1;
            SPR_setHFlip(player, FALSE);
            SPR_setAnim(player, ANIM_WALK);
        } else if (value & BUTTON_LEFT) {
            player_x -= 1;
            SPR_setHFlip(player, TRUE);
            SPR_setAnim(player, ANIM_WALK);
        }
        // ... rest of player1's directions ...
    }

    // PLAYER2
    if (attack_timer2 == 0) {
        if (value2 & BUTTON_RIGHT) {
            player2_x += 1;
            SPR_setHFlip(player2, TRUE);
            SPR_setAnim(player2, ANIM_WALK);
        } else if (value2 & BUTTON_LEFT) {
            player2_x -= 1;
        }
    }
}
```



```

        SPR_setHFlip(player2, FALSE);
        SPR_setAnim(player2, ANIM_WALK);
    }
    // ... rest of player2's directions ...
}

SPR_setPosition(player, player_x, player_y);
SPR_setPosition(player2, player2_x, player2_y);
}

```

With this rearrangement, `attack_function()` stops calling `handle_input()` altogether: it goes back to being limited purely to keeping track of both timers (incrementing them while the attack is in progress, resetting them to zero once they finish), exactly the same role it had in the previous chapter:

```

void attack_function() {
    if (attack_timer != 0) {
        attack_timer++;
        if (attack_timer == attack_duration) {
            attack_timer = 0;
        }
    }

    if (attack_timer2 != 0) {
        attack_timer2++;
        if (attack_timer2 == attack_duration2) {
            attack_timer2 = 0;
        }
    }
}

```

The call to `handle_input()` then goes back to being made exactly once, unconditionally, directly from the main loop, alongside the call to `attack_function()` (not replaced by it):

```

while (TRUE) {
    handle_input();
    attack_function();

    SPR_update();
    SYS_doVBlankProcess();
}

```

With this adjustment, the final result behaves as expected from the start: each character moves at their normal speed with their own D-pad, and each player's attack button only suspends that player's movement —without affecting the other— while the attack animation plays out in full.

The lesson behind this bug goes beyond this specific case: when a function already internally bundles the logic for several elements —in this case, the movement of both players inside a single `handle_input()`— conditioning how many times that entire function gets called from outside can have side effects on everything inside it, not just on the part you meant to control. When a condition depends on a specific element (for example, whether `player2` is attacking), that condition has to be applied precisely to that element's code, not to the whole function that also contains the others.

An acknowledged limitation: this doesn't scale beyond two players

The pattern used throughout this chapter —adding a numeric suffix to each variable (`value2`, `player2_x`, `attack_timer2`, `attack_duration2`) and duplicating each code block— works reasonably well for two players, but it's worth being honest about its limits: it doesn't scale comfortably to three or more. Each additional player would require manually repeating the same set of loose variables and the same block of code inside `handle_input()`, `joyEvent()`, and `attack_function()`, with the same risk of copy-paste errors already seen in section 2, multiplied by each new character.

The natural path to generalizing this pattern beyond two players is to group all of a character's information —their sprite, their position, their attack timer— into a single data structure, and replace the loose variables with an array of those structures (for example, `Player players[MAX_PLAYERS]`), so that adding another player is a matter of increasing the array's size, not writing a new block of code. This manual doesn't develop that tool yet: C structs are introduced later, when the enemy data for the capstone project is organized (chap. 53). That same approach solves, in a much cleaner way, the scaling problem this chapter solved by hand.

Review exercises

1. Axel and Sonic have the same numeric position coordinates but appear at different heights on screen. Explain why this happens without it being a code bug.
2. Why is it worth creating a throwaway file, separate from `main.c`, before using find-and-replace to duplicate a player's code? What specific problem does that precaution avoid?
3. Explain the difference between searching for `player` with the “whole word” option enabled versus disabled, and why that difference matters when renaming variables like `player_x` and `player_y`.
4. A teammate adds a second character by reusing the `ANIM_UPPER` constant for its attack animation, assuming it will work the same way it did for Axel. Explain why this can be a mistake and what they should check before assuming two sprite sheets share the same animation layout.
5. `attack_function()` calls `handle_input()` once for each player timer that reaches zero, and both characters end up moving at twice the speed. Explain the exact cause of this behavior and describe the structural change —not just the specific code fix— that solves it.

Chapter 34 — The Sound Chips - YM2612 and PSG

How the Mega Drive's sound is organized under the hood, before writing the first line of code to play it

This part of the manual is devoted entirely to sound: how to play music and sound effects with SGDK, and how to create those assets from scratch. Before getting there, it's worth spending an entire chapter understanding the hardware behind it, the same way chapter 2 paused on the VDP before showing the first line of graphics code. Chapter 1 already gave a brief preview: the Mega Drive's sound is handled by a Zilog Z80 coprocessor that controls two separate chips — a Yamaha YM2612 FM synthesizer and a programmable sound generator (PSG). This chapter picks that idea back up with the depth needed to understand, in the following chapters, why SGDK's sound functions are organized the way they are.

This is a purely conceptual chapter: there's no code yet. The goal is to build a mental map of “what each chip does and why two are needed,” so that when chapter 35 starts showing how to play music with the XGM and XGM2 drivers, those design decisions already have a foundation.

1. A separate sound subsystem, not a simple speaker

On a modern computer, playing a sound usually comes down to handing a series of numbers to an audio card. On the Mega Drive, sound depends on a whole subsystem with its own processor: the Z80. That architecture shapes, from the outset, how any sound function has to be written in SGDK.

The 68000, which runs the game's main program, doesn't have direct, permanent access to the sound chips' registers. To write to them, it first needs to hand control of the data bus over to the Z80 (a bus arbitration mechanism: while the 68000 has control, the Z80 is halted, and vice versa). The Z80, for its part, has its own 8 KB of memory (already mentioned in the memory table in chapter 1) where it can run its own program completely independently of the 68000.

This enables two ways of working with sound, and understanding the difference helps explain why sound drivers exist at all:

- **Direct control:** the 68000 requests the bus from the Z80, halts it for an instant, writes directly to the YM2612 or PSG registers, and hands control back to the Z80. It's simple to understand, but every note or every sound change requires halting the Z80 and spending 68000 cycles in the process.
- **A resident program on the Z80:** the 68000 loads, just once (typically at game startup), a small program into the Z80's own memory. That program then runs autonomously, reading commands from a shared memory area and translating them into the low-level

writes that the YM2612 and PSG need. The 68000 no longer has to halt the Z80 for every sound event: it's enough to drop commands into that shared memory whenever needed.

The second approach is, in essence, that of a **sound driver**, and it's the one SGDK uses. Section 4 of this chapter comes back to this idea; for now it's enough to keep this in mind: while the 68000 keeps running the game, the Z80 has its own program dedicated exclusively to coordinating the two sound chips described below.

2. The YM2612: frequency modulation synthesis

The **Yamaha YM2612** is the chip responsible for the Mega Drive's most recognizable sound: those metallic timbres, bells, thick basses, and orchestral textures that set much of its music apart from other consoles of the era. It has **6 channels**, each capable of playing in stereo (that is, with its own balance between the left and right audio channels).

The technique the YM2612 uses is called **frequency modulation (FM) synthesis**. In very general terms — and this is already a deliberate simplification, without going into formulas or the exact details of the circuitry — FM synthesis doesn't record or play back a preexisting sound wave: it generates one mathematically by combining several simple waves (usually called "operators") in such a way that some modulate the frequency of others. Depending on how those operators are connected to each other and with what parameters (a combination known as an "algorithm"), the result can sound like a bell, a bass, an organ, or a completely artificial timbre with no equivalent in any real instrument. That's where much of the Mega Drive's sonic identity comes from: the music doesn't try to imitate real instruments with recorded samples, but generates its own timbres from scratch, chip by chip. The details of how a specific FM instrument is designed — how many operators are involved, what parameters exist, and how they're adjusted — are covered in chapter 36, dedicated to music creation; for now it's enough to know that each of the YM2612's six channels can have its own instrument configured independently.

There's a second function of the YM2612 worth knowing about from the start: it can play back **digitized audio samples (PCM)**, that is, recorded sound instead of synthesized sound. But that capability has a direct cost: playing PCM on the Mega Drive always occupies the chip's sixth channel, which then becomes unavailable for FM synthesis for as long as the sample lasts. In practice, this means a game that uses digitized voices, recorded gunshots, or sampled percussion is giving up, during those moments, one of the six available melodic channels. This is a sound-design decision that has to be factored in from the moment the composition is conceived, not a secondary technical detail.

3. The PSG: four channels inherited from the previous generation

The second sound chip is the **PSG (Programmable Sound Generator)**, a much simpler sound generator than the YM2612, inherited directly from the Master System, Sega's previous 8-bit console. The PSG has **4 channels**:

- **3 tone channels**, capable of generating simple square waves at a given frequency. This is the same kind of "chiptune" sound characteristic of many 8-bit consoles and computers.
- **1 noise channel**, designed to simulate explosions, percussion, wind, or any effect that needs a more chaotic texture than a tuned note. In several PSG chips of this family, the noise channel can also be synchronized with the frequency of the third tone channel, which makes it possible to generate noises with a recognizable pitch instead of flat

noise; this is a relatively fine-grained usage detail that will be revisited, if relevant, in the chapter devoted to creating sound effects.

The PSG doesn't have the tonal richness of the YM2612 (there's no FM synthesis or the ability to shape a complex timbre), but it's much simpler to work with and uses far fewer resources to produce a sound. Its most common use in Mega Drive games is twofold: reinforcing the YM2612's music with additional melodic lines or arpeggios (adding up to three more voices to the six FM channels), and above all, handling short sound effects — jumps, shots, coins, hits — without having to sacrifice any of the FM synthesizer's more valuable six channels.

The two chips have different origins: the YM2612 is exclusive to the Mega Drive and didn't exist in earlier Sega hardware, while the PSG is the same type of chip that was already used in the Master System. That continuity is one of the mechanisms that lets the Mega Drive run Master System games in its compatibility mode: that legacy mode only needs the Z80 and the PSG, without relying on the YM2612.

4. Why a sound driver is needed

With two chips of such different characteristics — six FM channels with PCM capability on one side, four simple tone-and-noise channels on the other — coordinating both from scratch, writing directly to their registers every time a change is needed, would be impractical for a real game. A game needs to simultaneously play a background music track (which may be using several YM2612 and PSG channels at once) and trigger specific sound effects (a jump, a shot, an explosion) without those effects cutting off or stepping on the music that's playing.

This is why SGDK doesn't expose sound as a simple function that “writes to the chip,” but rather through a **sound driver**: a program that runs on the Z80 (following the second approach described in section 1) and is responsible for deciding, at every moment, which channel of which chip is doing what. The game code, running on the 68000, doesn't need to know exactly which YM2612 register to write to in order to play a given note: it asks the driver to “play this music” or “play this effect,” and it's the driver that internally figures out how to distribute those requests among the available channels, including what to do when a sound effect needs a channel that's currently occupied by the music.

SGDK includes more than one sound driver, with different capabilities and different data formats — the most important ones are XGM and its successor, XGM2 — and choosing between them is one of the first practical decisions that will come up in chapter 35. For now, the important thing is to hold on to the concept: the driver is the layer that translates the game's intentions (“play this song,” “play this effect”) into the low-level, channel-by-channel handling of two chips with very different rules from each other.

5. Thinking in channels, not just instruments

A practical consequence of everything above, and perhaps the most important idea to take away from this chapter, is that composing or programming the sound of a Mega Drive game forces you to think in terms of **concrete channels**, not just abstract instruments. On a modern computer with practically unlimited audio synthesis, a composer can think “I need a bass, a drum kit, two melodic voices, and a background layer” without worrying about whether there's physical room for all of that at once. On the Mega Drive, that question has a strict numerical answer: there are six FM channels and four PSG channels, period.

This has very concrete effects on how a game's music and sound are designed:

- If a song uses all six of the YM2612's channels to sound with its full tonal richness, there's no FM channel left free for a sound effect that needs to play over the music

without interrupting it; that effect will have to be handled by the PSG, or the driver will have to decide to temporarily sacrifice one of the music channels.

- If the game needs to play a digitized voice or a PCM effect, you have to plan ahead for that to consume the YM2612's sixth channel for as long as it lasts, and compose the music with the awareness that this channel may not always be available.
- Reinforcing a song with an extra PSG channel only makes sense if that channel isn't simultaneously needed for the game's sound effects at that same moment.

In other words, the programmer and the composer of a Mega Drive game end up jointly making channel-allocation decisions very similar to the ones a graphics programmer makes with VRAM or the number of sprites on screen: a fixed, scarce resource that has to be allocated deliberately, rather than assuming that "everything that should sound, will sound." This way of thinking about sound — in terms of available channels and how they're split between music and effects — underlies the tools and functions presented starting in the next chapter.

Review exercises

1. Explain in your own words why the 68000 can't simply write directly, at any moment, to the YM2612 or PSG registers, and what role the Z80 plays in that process.
2. What's the difference between FM synthesis and playing back recorded audio (PCM), and why does that difference explain much of the Mega Drive's characteristic sonic identity?
3. If a game needs to play a digitized voice effect while the background music is playing, what consequence does that have for the YM2612's available channels at that moment?
4. Why is the PSG said to be "inherited" from the Master System, and how does that fact relate to the Mega Drive's compatibility mode with games from that earlier console?
5. Explain why SGDK doesn't expose sound as a function that writes directly to the chips, but rather through a driver that runs on the Z80. What concrete problem does that intermediate layer solve?

Chapter 35 — Music Playback - XGM and XGM2 Drivers

Chapter 34 explained why the Mega Drive needs an intermediate sound driver: the YM2612 and the PSG are powerful but low-level chips, and programming them directly to play back a full track—with all its channels, instruments, and PCM samples synchronized—would be impractical for a game. SGDK solves that problem with two music drivers already built in: **XGM**, the original one, and **XGM2**, added in an SGDK 2.00 update. This chapter compares the two and walks through the functions needed to play music with each. The code itself is short; what really deserves attention is the decision of which driver to use.

Prerequisites

- A working SGDK project, with `resources.res` and `main.c`, like the one used since the early chapters of the manual.
- An understanding of the general structure of a `resources.res` file for declaring resources (already covered in the chapters on graphics and maps).
- It helps to have read chapter 34, since this chapter assumes you already know what the YM2612 and the PSG are and why a software driver is needed to handle them.
- No musical knowledge is required: this chapter is limited to playing back already-composed tracks. Composing original music for the Mega Drive is covered in chapter 36.

1. Two drivers, one design decision

XGM is SGDK's original music driver. It offers four PCM channels in total: one is normally reserved for music, and the remaining three are available for sound effects.

XGM2 was added later, in an SGDK 2.00 update, with a specific goal: reducing the final ROM size. In practice, music tends to be what takes up the most space on a cartridge, so any savings there are significant. XGM2 achieves this in two ways: it uses smaller PCM samples for sound effects, and it allows a single PCM sample to be shared across multiple tracks. Under XGM, if the same drum sound appeared in four different tracks, that sample had to be included four times in the ROM, once per track; with XGM2 it's included only once, and every track that needs it reuses it.

That advantage comes with a trade-off: XGM2 offers only three PCM channels in total (one for music and two for sound effects), one fewer than XGM. This may matter depending on how many simultaneous sounds a game needs, and it's worth taking into account before choosing a driver for a project, since switching drivers later means redeclaring the audio resources.

These PCM channels are separate from the six FM channels of the YM2612 described in chapter 34: the driver manages them independently, and this manual doesn't go further into the internal details of that management.

When XGM2 was introduced, it generated some excitement in the community over two specific features: fade in/out and the ability to adjust the music tempo without having to manually count frames. Of those two features, only fade in/out is actually exclusive to XGM2: tempo adjustment already existed in the original driver, as shown later in this chapter.

2. Declaring the music resource in resources.res

Declaring a music track follows the same pattern as any other SGDK resource: first the type, then a name to identify the track within the code, and finally, in quotes, the source file. For the XGM driver:

```
XGM my_track "musica.vgm";
```

Both drivers require files in **uncompressed VGM** format; they do not accept compressed formats such as MP3. If a downloaded file is in VGZ format (a compressed version of VGM), it's enough to decompress it with any common tool and then rename the resulting extension from .vgz to .vgm.

If you're going to use music from existing games in your own project, keep in mind that a commercial project cannot include third-party music without authorization. Non-commercial fan projects tend to have more leeway, but it's worth checking the license of each track before distributing anything.

3. Playing music with the XGM driver

With the resource declared, starting playback takes a single line:

```
XGM_startPlay(my_track);
```

This call must be made **only once**, typically alongside the other initializations, after `SYS_init()` and before the main game loop starts. A common mistake is placing it inside the `while(1)`: if that happens, the system would try to restart the track on every frame, instead of letting it play normally.

```
int main(bool hardReset) {
    SYS_init();
    // ... other initializations ...

    XGM_startPlay(my_track);

    while (TRUE) {
        // game logic, without calling XGM_startPlay() again here
        SYS_doVBlankProcess();
    }

    return 0;
}
```

4. Controlling the number of repeats: XGM_setLoopNumber()

By default, `XGM_startPlay()` plays the track with 255 repeats, which in practice amounts to an indefinite loop for most purposes. When different behavior is needed—for example, having a title screen theme play just once and then stop—you can use `XGM_setLoopNumber()`, specifying the number of *additional* repeats desired:

```
XGM_setLoopNumber(0); // plays only once, without repeating
```

- -1 corresponds to the default behavior (255 repeats).
- 0 makes the track play exactly once.
- 1 plays it twice in total, and so on.

5. Adjusting the tempo: XGM_setMusicTempo()

`XGM_setMusicTempo()` allows changing the playback speed in real time. The default value is 60 (60 frames per second on NTSC; 50 on PAL); half that speed corresponds to 30, and double corresponds to 120:

```
XGM_startPlay(my_track);  
XGM_setMusicTempo(120); // double speed
```

This function must be called **after** `XGM_startPlay()`, never before, otherwise it has no effect. It's rarely essential, but it can be useful in genres like shoot 'em ups, where it might make sense to speed up the music as the ship's speed increases or as more enemies appear on screen.

6. Pausing and resuming: XGM_pausePlay() and XGM_resumePlay()

`XGM_pausePlay()` stops playback without resetting it, which is useful for example for cutscenes or pause menus. `XGM_resumePlay()` picks the track back up right where it left off:

```
if (paused) XGM_pausePlay();  
else XGM_resumePlay();
```

Unlike the previous functions, meant to be called only once before the main loop, these are normally invoked inside the game loop, conditioned on some event such as a button press. Due to the nature of the FM chip, resuming isn't always perfect: some notes may be lost until the next key event in the track itself.

7. Switching to the XGM2 driver

Moving from XGM to XGM2 doesn't require restructuring anything: in `resources.res` it's enough to replace the keyword XGM with XGM2, leaving the rest of the line unchanged:

```
XGM2 my_track "musica.vgm";
```

In `main.c`, the function equivalent to `XGM_startPlay()` is `XGM2_play()`:

```
XGM2_play(my_track);
```

The rest of the functions covered so far have direct equivalents with the `XGM2_` prefix: `XGM2_setLoopNumber()`, `XGM2_pause()`, and `XGM2_resume()` cover, respectively, the number of repeats, pausing, and resuming, with the same behavior described for the original driver.

8. What's truly exclusive to XGM2: fade in and fade out

Fade in and fade out is indeed an XGM2-exclusive feature, with no equivalent in the original driver. The syntax is straightforward: `XGM2_fadeIn()` takes as a parameter the number of frames the transition should last. At 60 frames per second, 120 equals 2 seconds:

```
XGM2_play(my_track);  
XGM2_fadeIn(120);    // 2-second fade in
```

`XGM2_fadeOut()` works analogously, with the same type of parameter, to fade the track out instead of cutting it abruptly. If the effect seems barely noticeable with low values, increasing the duration (for example to 240 frames, about 4 seconds) usually makes it more evident during testing.

Review exercises

1. Explain the main advantage of XGM2 over XGM in terms of final ROM size, and the specific mechanism that makes it possible.
2. What is the main disadvantage of XGM2 compared to the original driver, in terms of available PCM channels?
3. Explain why the call that starts music playback should not be placed inside the game's main loop.
4. Of the functions covered in this chapter (play, pause, resume, adjust repeats, adjust tempo, fade in/out), state which one is exclusive to XGM2 and which ones exist in both drivers.
5. Describe the procedure for using, with either of SGDK's two drivers, a music file downloaded in VGZ format.

Chapter 36 — Creating Music for the Mega Drive

Chapter 35 showed how to play back, within an SGDK project, a music file that had already been composed, using the XGM and XGM2 drivers. What remained pending, however, was the previous step in the chain: where that music file comes from in the first place. This chapter covers that earlier step, and it does so with a particularity compared to the rest of the manual: not a single line of C code is written. All the work happens in an external composing program called DefleMask, designed specifically for creating music targeted at the sound hardware of retro systems, the Mega Drive among them.

The goal of this chapter is modest: it does not aim to turn the reader into a composer, but rather to show how to start experimenting with sound on the Mega Drive without needing any prior musical training. For anyone who wants to go deeper into composition proper, particularly into designing FM instruments from scratch, there are community tutorials devoted entirely to that subject, with more detail than this chapter can offer.

Prerequisites

Before continuing, it's worth having on hand:

- Having completed Chapter 35, “Playing Music: the XGM and XGM2 Drivers”: this chapter assumes you already know what role the music file plays within an SGDK project and how it's played back once imported.
- Having DefleMask installed. There is a paid version and a free legacy version; the latter is enough for everything shown in this chapter, so there's no need to buy the application just to follow along.
- No prior musical background is required. This chapter is limited to importing and trying out instruments already created by other people, not composing from scratch.

1. Why this chapter doesn't teach music composition from scratch

Designing an FM instrument from the very first parameter (choosing algorithm, operators, envelopes, and frequency modulation until arriving at a specific timbre) is beyond the scope of this introduction; the DefleMask community covers that in more detail in tutorials devoted entirely to the subject. Rather than starting from zero, this chapter relies on instruments that have already been created, extracted from real game music, in the same way other chapters of this manual use graphics or level maps from commercial games as practice material (see, for example, Chapter 8 on creating graphics).

This approach has a concrete pedagogical advantage: it lets you hear right away what a well-designed FM instrument sounds like, the same kind that makes up the soundtracks of well-known Mega Drive games, without yet needing the full technical vocabulary of the YM2612 chip (algorithms, operators, envelopes) presented in Chapter 34. Importing and playing existing instruments is, in that sense, a much more accessible entry point than starting with design from scratch.

2. DefleMask: the tool used in this chapter

DefleMask is today one of the most widely used programs for composing music intended for retro sound hardware, including the Mega Drive. Its interface follows the classic tracker model: a pattern editor organized into columns, one per sound channel, with a virtual piano available at the bottom for writing notes and trying out instruments interactively.

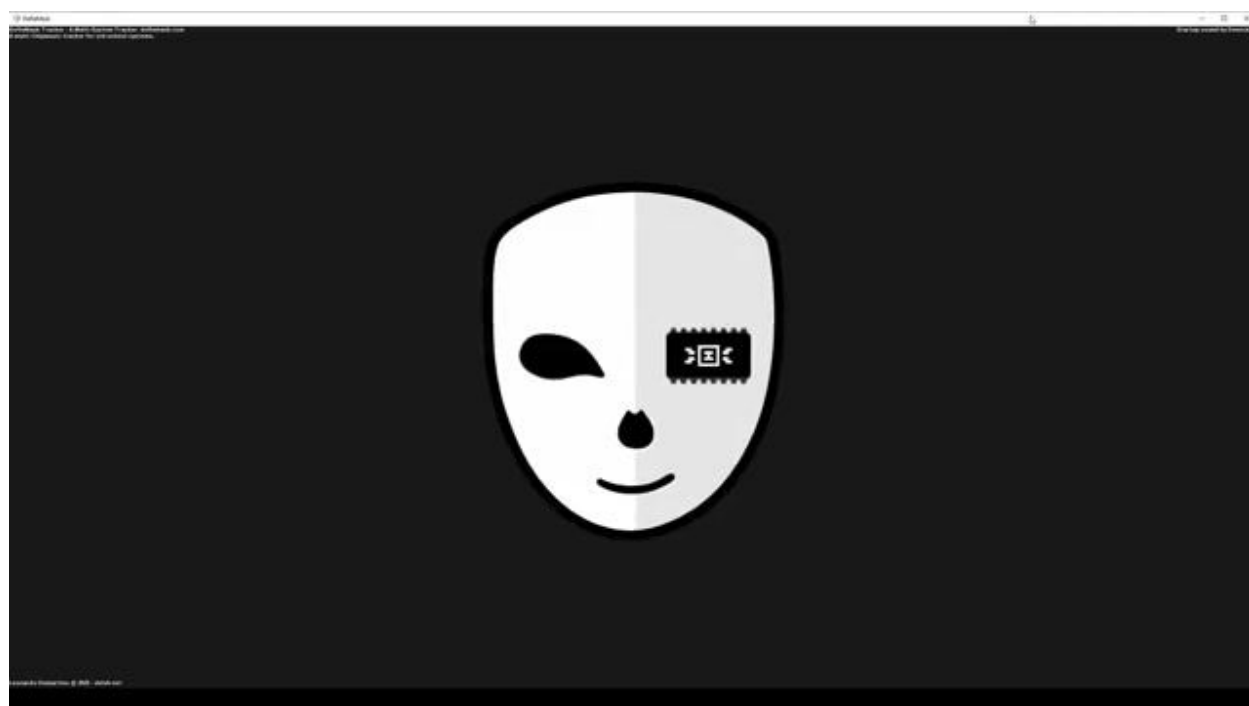


Figure 106: DefleMask welcome screen

DefleMask's welcome screen on startup.

The columns of the pattern editor correspond directly to the Mega Drive sound hardware channels already described in Chapter 34: the FM channels of the YM2612 chip (the sixth of which can be reconfigured as a PCM channel) and the PSG channels, including the noise channel. Seeing those channels represented as concrete columns in the editor helps make the hardware model, up to this point rather theoretical, tangible: each note written in a given column actually sounds through a specific channel of the real chip.

DefleMask's pattern editor: the virtual piano sits at the bottom, and the columns above correspond to the Mega Drive's FM and PSG channels.

There is also a free legacy version of DefleMask, which retains all the functionality needed to import instruments, write patterns, and play them back: none of the steps in this chapter require the paid version.



Figure 107: DefleMask pattern editor

3. The instrument panel: importing an already-created instrument

Instead of creating an FM instrument from scratch, DefleMask lets you import an existing one. At the top of the instrument panel there's a small folder icon; clicking it opens a file browser that lets you navigate to the folder where the downloaded instruments are stored.

Instrument panel with slot "0: Ins 0" still empty and the folder icon used to import an instrument.

DefleMask recognizes two different instrument formats:

- **.dmp**: DefleMask's native instrument dump format.
- **.tfi**: another instrument format supported by the program, used by some collections of instruments extracted from games.

When you select a file in either format, DefleMask loads it directly into the active instrument slot, replacing the generic name ("Ins 0") with the real name of the imported instrument.

An instrument already loaded into slot 0, replacing the initial generic "Ins 0".

Pressing the "+" button on the same panel and repeating the file selection process lets you add as many additional instruments as needed to the composition file.

4. Community instrument banks

To avoid depending on hand-made instruments, there are community instrument banks and tutorials for DefleMask focused specifically on the Mega Drive, organized in different ways. As an example of the kind of material you can find:

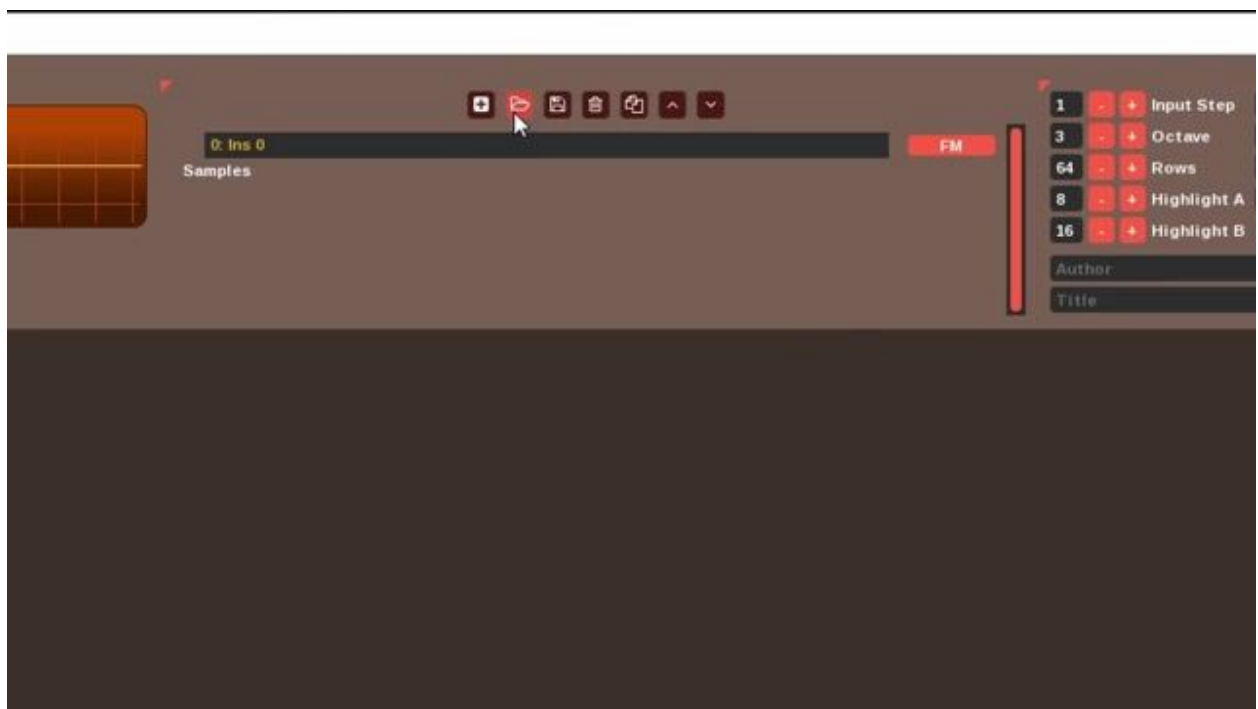


Figure 108: DefleMask instrument panel

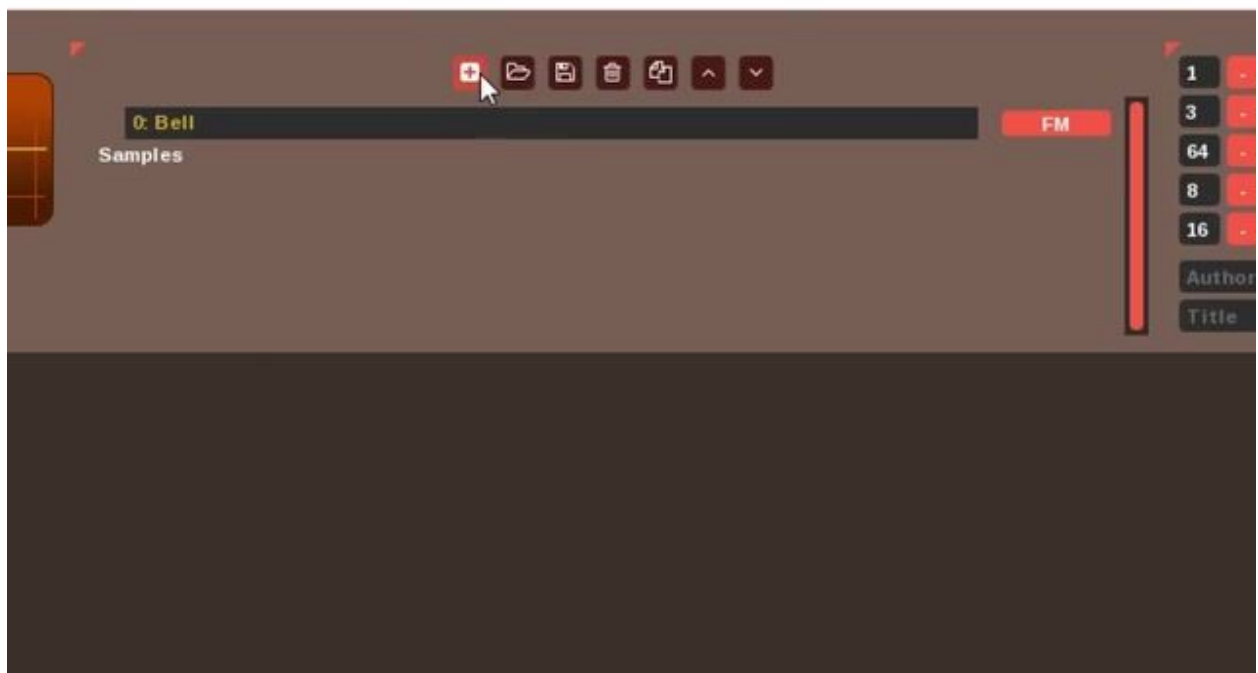


Figure 109: Instrument loaded into slot 0

An instrument bank extracted from arcade games might be organized by company (for example, separate folders for Atari, Capcom, Konami, Namco, and Sega), with individual .dmp files inside each one. Inside the folder for a specific Mega Drive game, such as Golden Axe, you'll usually find a collection of instruments with descriptive names ("Bass Drum", "Bell", "Brass"), ready to be imported as-is using the procedure from the previous section.

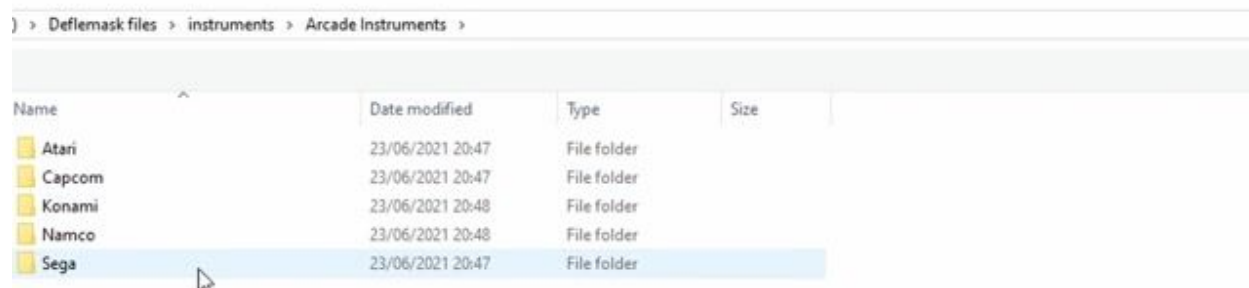


Figure 110: Instrument folder organized by company

Arcade instrument folder organized by company: Atari, Capcom, Konami, Namco, and Sega, each with its own .dmp files.

There are also larger banks in circulation, organized by game title rather than by company, with instruments in .tfi format extracted from the complete soundtrack of a specific game. A bank of this kind might include, for example, a folder for Streets of Rage with files named after the track and instrument of origin, making it possible to recreate the sound of well-known themes from that soundtrack with fairly good fidelity.

Contents of the folder corresponding to Streets of Rage, with .tfi files named after the track and instrument of origin.

In both cases, the way to bring the instruments into the project is the same one already seen in section 3: the folder icon in the instrument panel, repeated as many times as needed instruments.

5. Trying out instruments and exploring existing songs

With the instruments already loaded, the natural next step is to play them directly from the virtual keyboard built into DefleMask, to hear how each one sounds in isolation and mentally compare it to the original song it comes from. This ear-training exercise — identifying which well-known theme an instrument belongs to as soon as you hear it play — is, in itself, a useful way to get familiar with the characteristic timbre of the YM2612 and PSG combined, beyond the technical vocabulary already covered in Chapter 34.

(C:) > Deflemask files > instruments > TFI_WITH_FRIENDLY_NAMES > tfi_patches_names_2 > 04 > streets_of_rage_[bare_knuckle__ikari_no_tekken]_1990123

Name	Date modified	Type	Size
01_the_street_of_rage_77.tfi	23/06/2021 19:03	TFI File	1 KB
01_the_street_of_rage_80.tfi	23/06/2021 19:03	TFI File	1 KB
01_the_street_of_rage_81.tfi	23/06/2021 19:03	TFI File	1 KB
01_the_street_of_rage_82.tfi	23/06/2021 19:03	TFI File	1 KB
01_the_street_of_rage_83.tfi	23/06/2021 19:03	TFI File	1 KB
01_the_street_of_rage_84.tfi	23/06/2021 19:03	TFI File	1 KB
01_the_street_of_rage_85.tfi	23/06/2021 19:03	TFI File	1 KB
01_the_street_of_rage_86.tfi	23/06/2021 19:03	TFI File	1 KB
02_player_select_87.tfi	23/06/2021 19:03	TFI File	1 KB
02_player_select_88.tfi	23/06/2021 19:03	TFI File	1 KB
02_player_select_89.tfi	23/06/2021 19:03	TFI File	1 KB
02_Player_Select_90.tfi	23/06/2021 19:03	TFI File	1 KB
02_player_select_91.tfi	23/06/2021 19:03	TFI File	1 KB
03_fighting_in_the_street_40.tfi	23/06/2021 19:03	TFI File	1 KB
03_fighting_in_the_street_92.tfi	23/06/2021 19:03	TFI File	1 KB
03_fighting_in_the_street_93.tfi	23/06/2021 19:03	TFI File	1 KB
03_fighting_in_the_street_94.tfi	23/06/2021 19:03	TFI File	1 KB
03_fighting_in_the_street_95.tfi	23/06/2021 19:03	TFI File	1 KB

Figure 111: Streets of Rage instrument folder

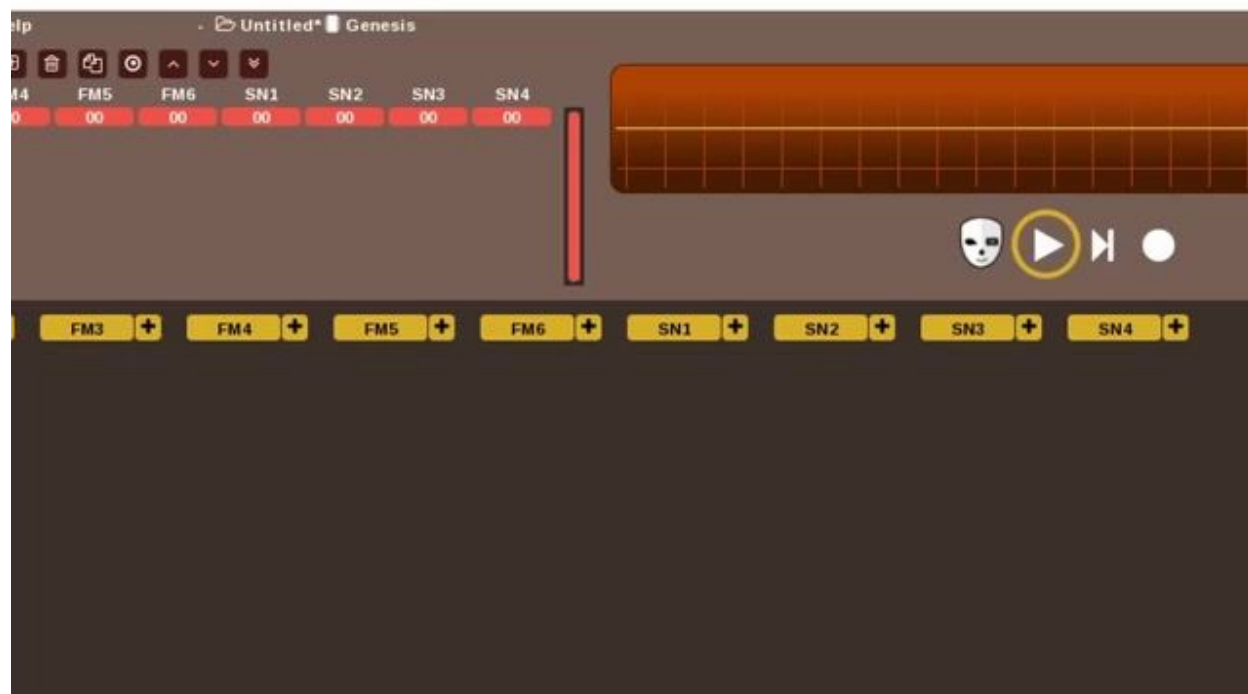


Figure 112: Blank DefleMask project

Main screen of a blank DefleMask project, with the FM and PSG channels (including the noise channel) available for the Mega Drive's sound chip.

As additional practice, searching online for piano tutorials of a well-known theme (adding terms like “tutorial” or “easy piano tutorial” to the search) can serve as a reference for recreating a basic version of that song inside DefleMask, note by note, using the already-imported instruments. It's also common to find in the community complete recreations of classic Mega Drive themes made in DefleMask, with the project file available for direct analysis: reviewing how an already-finished pattern is put together is often more instructive than starting from scratch.

6. From DefleMask to SGDK: the missing link

Everything shown so far happens entirely within DefleMask, with no connection yet to an SGDK project. Chapter 35 explained that XGM and XGM2 expect, as input, a file in VGM format, already converted to each driver's specific format using the corresponding conversion tools. That VGM file is the final destination of a composition made in DefleMask: the program allows a finished project to be exported to that format, closing the loop between this chapter and the previous one.

The exact procedure for performing that export, which specific menu or button in DefleMask to use, and which options are best chosen to maximize compatibility with XGM/XGM2, is not documented in this chapter with the same level of detail as the rest of the workflow. Before taking that final step, it's worth reviewing DefleMask's own documentation and, in case of doubt about any specific export option, following the same verification standard this manual applies to the rest of the code: if something isn't confirmed with certainty, it shouldn't be casually assumed.

Review exercises

1. Explain the difference between the `.dmp` and `.tfi` instrument formats mentioned in this chapter, and what type of bank each one usually comes from.
2. Describe the procedure for importing an already-existing instrument into DefleMask's instrument panel.
3. Why does this chapter rely on instruments extracted from existing games instead of teaching how to design FM instruments from scratch? Relate the answer to the level of detail of the YM2612 chip covered in Chapter 34.
4. What relationship do the columns of DefleMask's pattern editor have with the Mega Drive sound channels described in Chapter 34?
5. What file format connects the work done in DefleMask with what was explained in Chapter 35 about playing music with XGM/XGM2? Why is that conversion step essential before being able to hear the composition within an SGDK project?

Chapter 37 — Programming Sound Effects

Along with music, graphics, and animation, sound effects are one of the elements that bring the most life to a game, even when they serve no gameplay function whatsoever. Think of a platformer where the character makes a small sound while walking, or where certain elements of the scenery react with a sound effect when stepped on or touched: none of that affects the mechanics, but it reinforces the feeling that the represented world is real.

This chapter covers how to upload sound files to the ROM, assign them an identifying number within the range reserved for effects, play them in response to controls, and — the most important point of the chapter — how to manage sound channels and priorities when several effects might play at the same time.

Prerequisites

Before continuing, it helps to have:

- Read Chapter 35, “Music Playback: the XGM and XGM2 Drivers”: this chapter assumes familiarity with the XGM driver and the `resources.res` file, since sound effects are also declared there and played with functions from the same family.
- Read Chapter 32, “Button Controls and Event-Based Reading”: the example in this chapter reuses that same joystick event-handling structure to trigger the effects.
- An understanding of using `#define` to associate readable names with constant numbers, already covered in the chapter on sprite animation (Chapter 23).

1. Starting Point: Handling Several Buttons with Independent `if` Statements

The example in this chapter starts from a `joyEvent()` function like the one in Chapter 32, which checks whether buttons A, B, or C were pressed. There’s an important design detail in that structure: the three buttons are checked with three independent `if` statements, never with `else if`.

```
void joyEvent(u16 joy, u16 changed, u16 state)
{
    if ((changed & state) & BUTTON_A)
    {
        // play effect A
    }
}
```

```

if ((changed & state) & BUTTON_B)
{
    // play effect B
}

if ((changed & state) & BUTTON_C)
{
    // play effect C
}
}

```

The reason for using independent if statements instead of else if is that, with else if, only one sound effect could ever play at a time, even if the player presses two or three buttons simultaneously. With independent if statements, each button can trigger its own effect regardless of the state of the others.

To illustrate the rest of the chapter, three example sound files are used, with generic names you can replace with your own: `sfx_shout.wav`, `sfx_rise.wav`, and `sfx_yell.wav`. They don't need to be original or especially elaborate sounds to follow along with the chapter: any three short WAV files will do for experimenting with channels and priorities.

2. Uploading Sound Effects: the WAV Keyword in `resources.res`

The first step in programming a sound effect is uploading it to the ROM by declaring it in `resources.res`. Unlike music (which uses the XGM or XGM2 keyword, as seen in Chapter 35), individual sound effects are declared with the WAV keyword, in uppercase, since the source file must be in uncompressed .wav format.

After the keyword comes a freely chosen name to identify the resource within the code, followed by, in quotes, the file name with its .wav extension. The last piece of data the declaration requires is the sound driver associated with the effect:

```

WAV sfx_shout "sfx_shout.wav" XGM
WAV sfx_rise  "sfx_rise.wav"  XGM
WAV sfx_yell  "sfx_yell.wav"  XGM

```

This chapter uses the XGM driver instead of XGM2. Besides the fact that, as explained in Chapter 35, XGM2 has one fewer PCM channel than XGM, XGM2 is a more recent addition to SGDK and its documentation on effect playback is still even sparser. The procedure for making sound effects under XGM2 would be similar to what's shown in this chapter: declare the resource with XGM2 instead of XGM and use the equivalent functions from that API. It isn't covered in detail here; it's left as an exercise for anyone who prefers that driver.

Regarding the exact format the source .wav file must have (sample rate, bit depth), this chapter can't provide concrete, verified figures. In practice, `rescomp` (the SGDK tool that processes `resources.res`) usually accepts the WAV file as exported from a common audio editor and automatically adjusts it to the format the sound driver needs; if you notice distortion or unexpected noise when testing an effect of your own, it's worth checking the quality of the source file first before suspecting the console's sound driver.

3. The Sound Effect Number Range: Why Start at 64

Within SGDK, each sound effect in a game receives a unique identifying number, within a range from 0 to 255. However, the numbers from 0 to 63 are reserved exclusively for music, so sound effects must use numbers between 64 and 255: a total of 192 available numbers, more than enough for practically any game.

This distinction becomes clearer when using the playback functions: the declaration in `resources.res` (seen in the previous section) identifies the sound file itself, while the number defined in this section identifies the effect within the game's playback system. They are two related but distinct concepts.

4. Naming Effect Numbers with `#define`

Rather than using the numbers 64, 65, and 66 directly in the code — a practice that quickly becomes confusing when you have to remember which number corresponds to which effect — it's better to define a readable name for each one with `#define`, following the same pattern already used in Chapter 23 for sprite animation. It's also a good idea to choose names different from those used in `resources.res`, so as not to confuse the file resource's name with the effect's identifying number within the game:

```
#define SFX_SHOUT 64
#define SFX_RISE  65
#define SFX_YELL  66
```

5. Associating Each Effect Number with Its File: `XGM_setPCM()`

With the files already declared in `resources.res` and the effect numbers already defined, the next step is to tell SGDK which file corresponds to which effect number. This is done with the `XGM_setPCM()` function, which takes three pieces of data: the sound effect identifier (for which the name defined with `#define` can be used directly instead of the number), the name of the sound resource as declared in `resources.res`, and the size of the file in bytes.

For this last piece of data, rather than calculating the size manually, it's best to use the C language's own `sizeof()` function, passing it the name of the sound resource as an argument; SGDK resolves that calculation automatically at compile time:

```
int main()
{
    XGM_setPCM(SFX_SHOUT, sfx_shout, sizeof(sfx_shout));
    XGM_setPCM(SFX_RISE,  sfx_rise,  sizeof(sfx_rise));
    XGM_setPCM(SFX_YELL,  sfx_yell,  sizeof(sfx_yell));

    // ... rest of initialization
}
```

These three calls go before the main loop, along with the rest of the game's initialization.

6. Playing the Effect: XGM_startPlayPCM(), Priority, and Channel

The last step is to tell the system when to play each effect, inside the `joyEvent()` function presented at the start of the chapter. The function used is `XGM_startPlayPCM()`, which also takes three pieces of data: the sound effect identifier (again, using the defined name instead of the number), the effect's priority, and the sound channel it should play on.

Priority is a value from 0 to 15, where 15 is the highest priority. This value determines what happens if two sound effects try to play at the same time on the same channel. As for the channel, the XGM driver has four sound channels in total (`SOUND_PCM_CH1` through `SOUND_PCM_CH4`), but the first of these is usually reserved for music, so under normal circumstances it's best to avoid it and use channels 2, 3, or 4 for effects.

Assigning each effect to a different channel, with the same priority for all three:

```
void joyEvent(u16 joy, u16 changed, u16 state)
{
    if ((changed & state) & BUTTON_A)
        XGM_startPlayPCM(SFX_SHOUT, 15, SOUND_PCM_CH2);

    if ((changed & state) & BUTTON_B)
        XGM_startPlayPCM(SFX_RISE, 15, SOUND_PCM_CH3);

    if ((changed & state) & BUTTON_C)
        XGM_startPlayPCM(SFX_YELL, 15, SOUND_PCM_CH4);
}
```

When compiled and tested in the emulator, all three sound effects work correctly on their own and can also all play at the same time if all three buttons are pressed at once, since each one occupies a different channel. If the same button is pressed repeatedly, each new playback cuts off the previous one and restarts it from the beginning, since both share a channel.

7. What Happens When Several Effects Share the Same Channel

In a real game, it's common to need more than three simultaneous sound effects — for example, several different enemies on screen at once — which eventually forces more than one effect to share the same sound channel. To see what happens in that case, it's enough to modify the previous example so that all three effects use the same channel, leaving the same priority for all three for now:

```
if ((changed & state) & BUTTON_A)
    XGM_startPlayPCM(SFX_SHOUT, 15, SOUND_PCM_CH2);

if ((changed & state) & BUTTON_B)
    XGM_startPlayPCM(SFX_RISE, 15, SOUND_PCM_CH2);

if ((changed & state) & BUTTON_C)
    XGM_startPlayPCM(SFX_YELL, 15, SOUND_PCM_CH2);
```

When compiled and tested, the result is that any new effect cuts off whatever was playing at that moment on the same channel, regardless of which one it was: since all three have the same priority, the most recent one always replaces the previous one.

8. Controlling Which Effect Wins: Adjusting Priority

Depending on the game’s design, it may not be desirable for any new effect to interrupt the previous one on the same channel; priority allows this to be controlled more precisely. For example, you could leave the first effect with the lowest priority of the three (13), the second at an intermediate level (14), and the third with the highest priority (15):

```
if ((changed & state) & BUTTON_A)
    XGM_startPlayPCM(SFX_SHOUT, 13, SOUND_PCM_CH2);

if ((changed & state) & BUTTON_B)
    XGM_startPlayPCM(SFX_RISE, 14, SOUND_PCM_CH2);

if ((changed & state) & BUTTON_C)
    XGM_startPlayPCM(SFX_YELL, 15, SOUND_PCM_CH2);
```

The practical effect is that, while the priority-15 sound is playing, no other lower-priority effect can interrupt it on that channel; the priority-13 one, on the other hand, can be interrupted by both the priority-15 and priority-14 ones; and the priority-14 one can only be interrupted by the priority-15 one.

A higher-priority effect can always interrupt a lower-priority one on the same channel; two effects of equal priority interrupt each other, with the most recent one always winning.

The concrete way to apply channels and priorities in a real game depends entirely on each project’s design. As an example, in an action game with a main character, it might make sense to assign the character’s own effects and their “take damage” sound to a channel where that damage sound has high priority over their own attacks (so it isn’t lost while the character is attacking), reserve another channel for enemies, and a third for scenery elements like interactive objects or collectibles, so that these different types of sound don’t interrupt each other.

There’s a common mistake in this kind of design: if an enemy’s attack and its final speech or scream upon being defeated end up assigned to the same channel with the attack at a higher priority, it can happen that, if the attack is still playing at the moment the enemy is defeated, the second effect gets completely cut off and never plays at all. It’s an easy oversight to make and a hard one to notice during testing unless specific attention is paid to the assignment of channels and priorities among effects that might coincide in time.

Review exercises

1. Explain why it’s best to use three independent `if` statements for buttons A, B, and C in `joyEvent()`, rather than a chain of `else if`.
2. Why must sound effects in SGDK be numbered starting from 64 instead of from 0?
3. Describe the three parameters taken by `XGM_setPCM()` and the purpose of each one.
4. If two sound effects with the same priority share the same channel, what happens when both try to play at the same time? What about if they have different priorities?
5. Describe a game design situation (it can be a different genre or mechanic from the examples in this chapter) where a poor assignment of channel or priority could cause an important sound effect to be silenced without it being obvious why.

Chapter 38 — Creating Sound Effects

The previous chapter showed how to program sound effects that were already prepared for a SGDK project: how to declare a .wav file with the WAV keyword in `resources.res` and how to trigger it from code with the corresponding sound driver. This chapter deals with the step before that: creating those sound files from scratch, using the free program Audacity.

The focus here is deliberately kept narrow, limited to the specific requirements and limitations of the Mega Drive's sound hardware. For more general sound design techniques, how to convincingly create the sound of a punch, a sword, or a synthetic voice, it's worth turning to more general sound design resources once the console's own technical requirements are understood, which is precisely the subject of the following sections.

Prerequisites

Before continuing, it helps to be clear on:

- How to declare a WAV resource in `resources.res` and play it from code with the corresponding sound driver (chapter 37, "Programming Sound Effects").
- Having Audacity installed (free) or an equivalent audio editor that lets you change a track's sample rate, convert it to mono, and export it as unsigned 8-bit PCM.
- A microphone or some audio source to record with. It's also perfectly valid to start from an existing sound file instead of recording your own.

1. Where to draw inspiration from

Many sound effects from the classic Mega Drive library have a less handcrafted origin than they might seem at first listen. The first *Golden Axe*, for example, is known for lifting sound effects directly from 1980s Hollywood movies, while its sequel, *Golden Axe II*, with noticeably weaker enemy death effects, illustrates the other end of the spectrum.

The practical takeaway from these examples is simple: while taking sounds from movies or someone else's music might have gone unnoticed in the eighties, today it's preferable to create your own original sound effects, whether by recording your own voice, everyday objects, or by turning to properly licensed sound banks. The rest of this chapter proceeds from that premise: a sound effect is recorded from scratch with Audacity and adapted step by step to the Mega Drive's requirements.

2. Recording a sound effect in Audacity

The first step is to set up the recording device in Audacity, from the audio settings menu. It's best to choose the microphone as the input source, rather than the speakers, which would instead be used if you wanted to capture audio already being played back by the computer itself (the sound from a video, for example). The project's channel configuration and sample rate can be left at their default values for this initial recording, usually 44100 Hz, because the goal is to capture the best possible quality before adapting the file to Mega Drive's requirements later on.

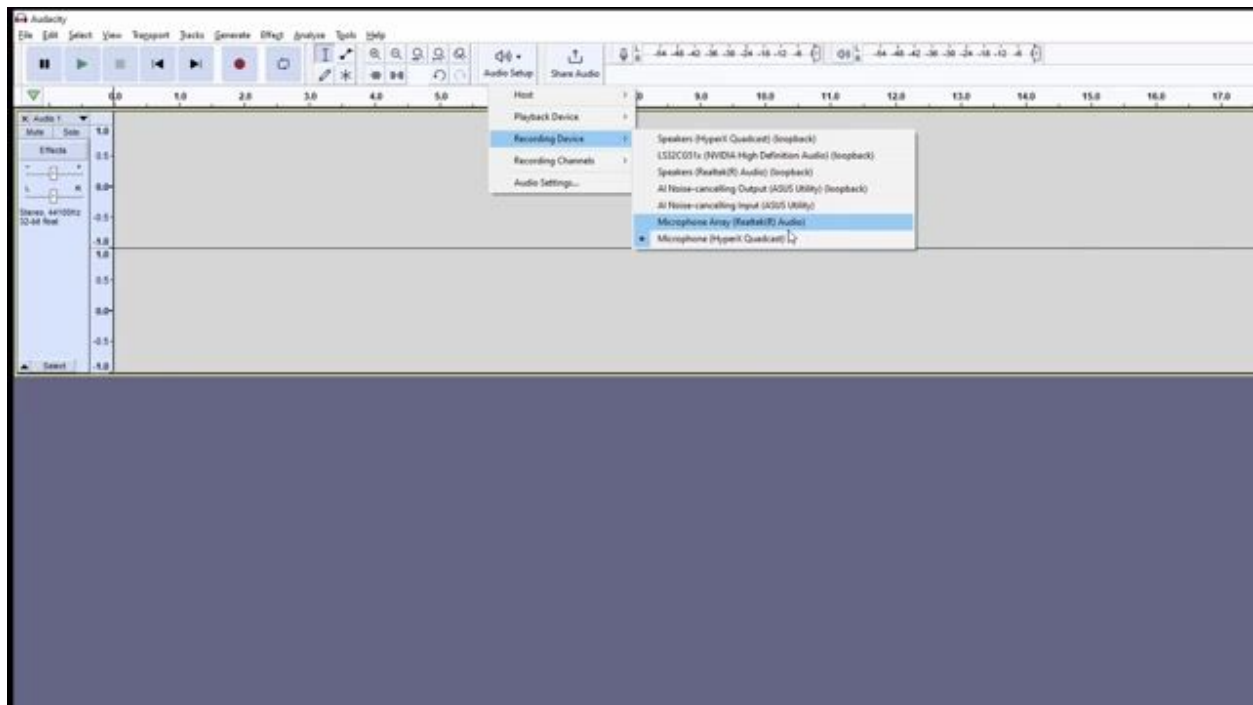


Figure 113: Audacity's Audio Setup menu, with the microphone selected as the recording device

Audacity's Audio Setup menu, selecting the microphone as the recording device instead of the speakers.

With the device already configured, it's just a matter of pressing the record button, producing the desired sound (a shout to serve as a "hit" effect for a *beat 'em up*, for example) and pressing the stop button when finished.

3. Removing background noise (optional)

Anyone recording in an environment with noticeable background noise (traffic, fans, the hum of some equipment) can turn to a noise reduction tool before continuing. The procedure consists of first selecting a stretch of the recording where there's no voice or desired sound, only the background noise, and then, from the *Effect* menu, choosing *Noise Removal and Repair* and then the noise reduction option, pressing the button that captures the "profile" of that noise.

Audacity's Effect menu, with the "Noise Removal and Repair" section expanded.

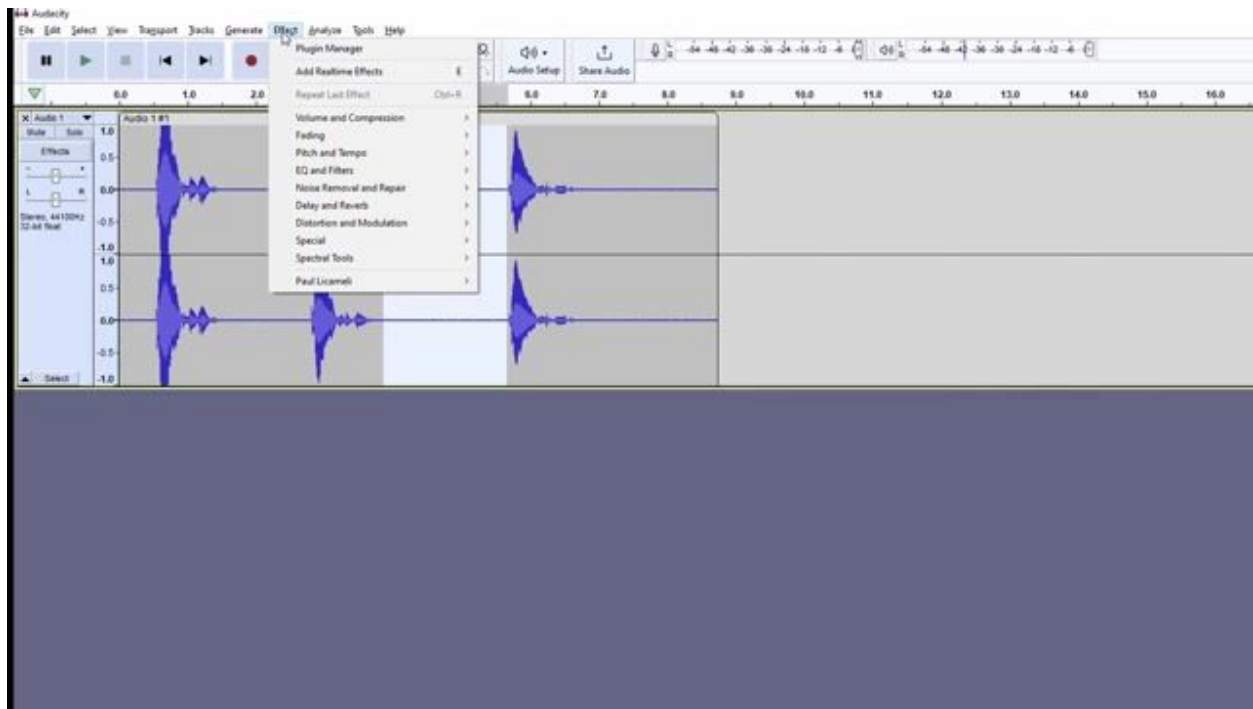


Figure 114: Audacity's Effect menu with the Noise Removal and Repair section expanded

Once the profile has been captured, the entire recording is selected (not just the silent stretch) and the same tool is opened again, this time confirming the operation instead of capturing the profile again, which applies the noise reduction to the whole file. This step is entirely optional and depends on how much background noise is present in the original recording: if the ambient noise is barely noticeable, it can be skipped without any appreciable loss of quality.

4. Isolating the useful fragment of the recording

After recording (and optionally cleaning up the background noise), it's best to select only the audio fragment you want to keep as the sound effect, copy it, and paste it into a new Audacity file, leaving behind any silence or discarded takes surrounding the useful fragment.

Selecting a specific fragment within several takes recorded on the same track.

Before continuing with any conversion specifically aimed at the Mega Drive, it's good practice to save the project at this point (using *Save Project As*) to keep a copy of the high-quality version of the sound effect, in case you want to reuse it later in a context other than the console: a PC version of the same game, for instance, or simply as a backup.

5. Adapting the file to Mega Drive's requirements: resampling and converting to mono

With the fragment already isolated and saved in its original version, the next step is to adapt it to a format better suited to the Mega Drive's sound hardware. The first adjustment is to reduce the sample rate: selecting the whole fragment and using the *Resample* option from



Figure 115: Selecting a specific fragment among several takes recorded on the same track

the *Tracks* menu, the value is changed to 16000 Hz, considerably lower than the 44100 Hz typical of a standard quality recording.

Audacity's resampling dialog box, changing the rate to 16000 Hz.

The second adjustment is to convert the stereo track to mono, using the *Mix Stereo Down to Mono* option from the same *Tracks* menu. After both steps, the track's side panel confirms the file's new format, showing "Mono, 16000Hz".

These two values, mono and 16000 Hz, serve as a reasonable rule of thumb for a Mega Drive sound effect, and are the ones used throughout this chapter, but they don't represent a strict limit or the only valid format: good results can be achieved with moderate variations in sample rate, depending on the effect and each project's own judgment.

6. Adjusting the pitch (optional)

Audacity offers numerous additional effects that can be applied to a recording. One of them is *Change Pitch*, which lets you alter a recording's pitch without changing its speed. It's the same kind of tool that, according to Yuzo Koshiro himself, he used to transform his own voice into that of Blaze, one of the characters in *Streets of Rage*.

The Change Pitch dialog box, with the Percent Change field set to -14.000, making the voice deeper.

A *Percent Change* value of 0 leaves the pitch unchanged; a negative value makes the voice deeper, and a positive one makes it higher-pitched. In practice, negative values tend to produce reasonably convincing results, while positive values tend to produce an unnatural-sounding high-pitched voice rather than a believable female voice; achieving that effect with more nuance generally requires additional processing that goes beyond the scope of this chapter. For the sound effect used as an example here, no pitch change is applied, leaving the

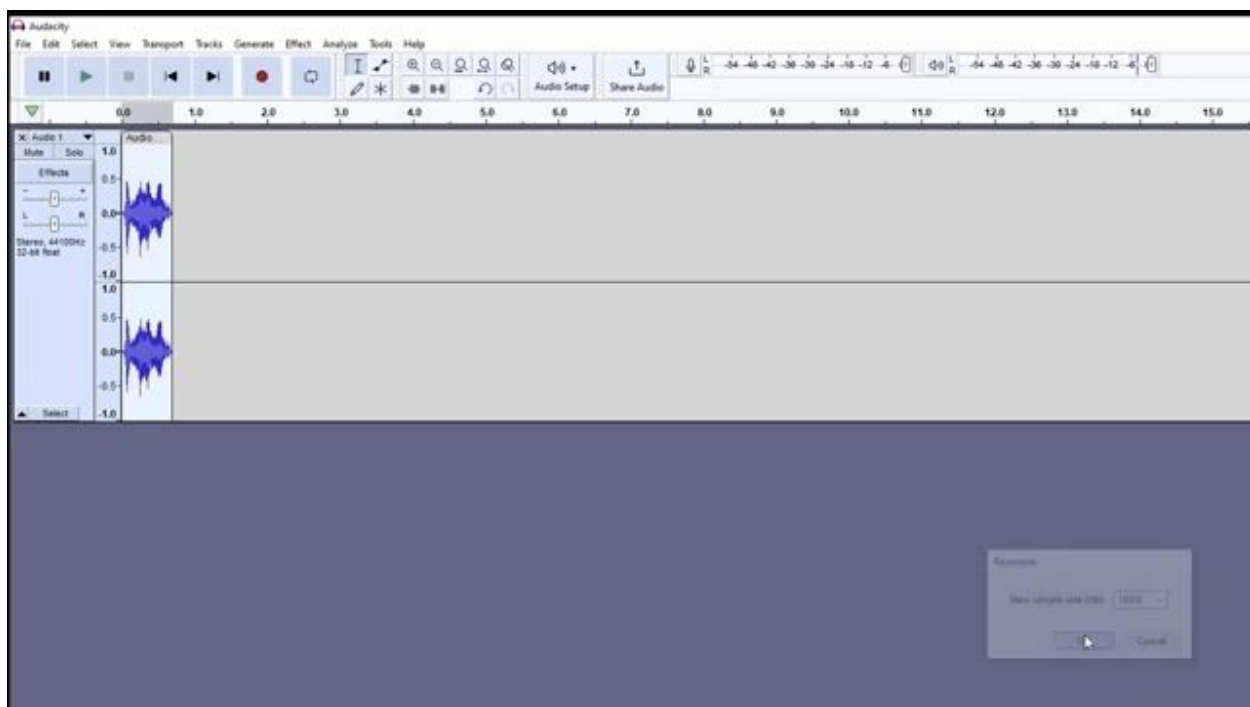


Figure 116: Audacity's resampling dialog box, changing the rate to 16000 Hz

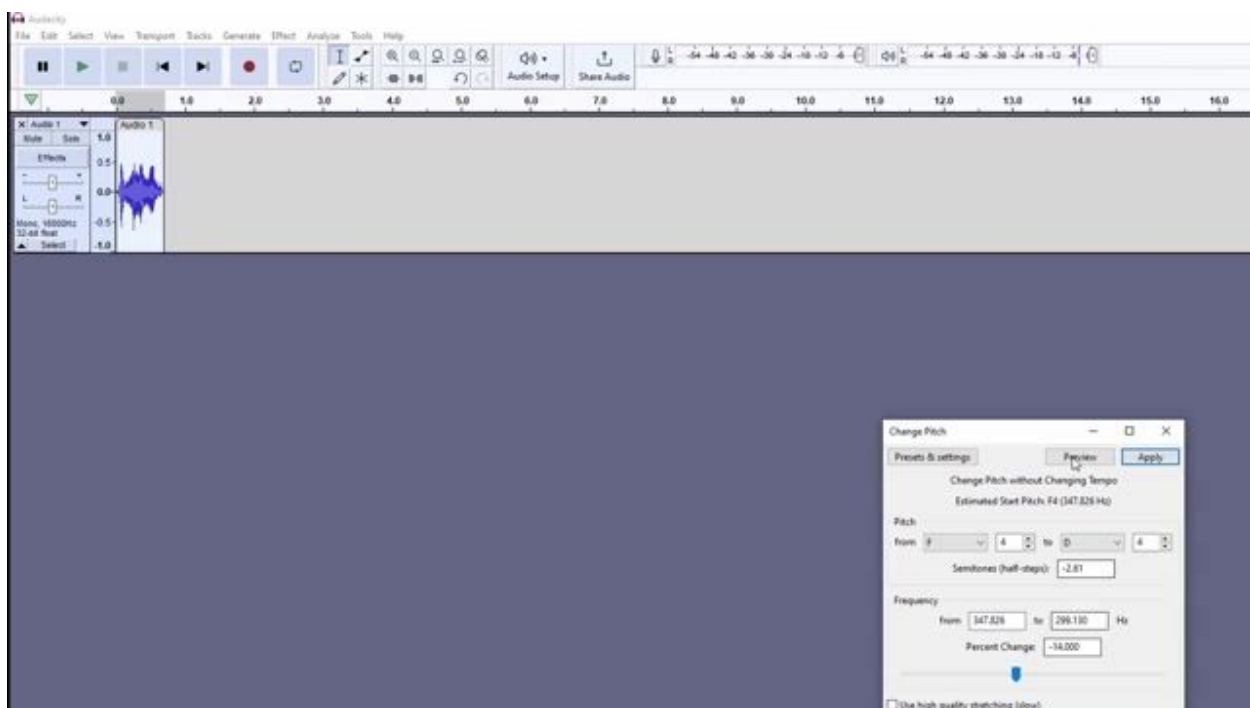


Figure 117: The Change Pitch dialog box, with the Percent Change field set to -14.000

recording exactly as it originally sounded; the pitch adjustment remains a tool available for whenever an effect calls for it.

7. Exporting the final file: unsigned 8-bit WAV format

The last step is to export the already-adapted file (resampled and converted to mono) in the format SGDK expects. From the *File* menu, choose *Export* and then export as an audio file, giving it whatever name you prefer. The most important detail in this step is the *Encoding* field, which must be specifically set to “Unsigned 8-bit PCM”.

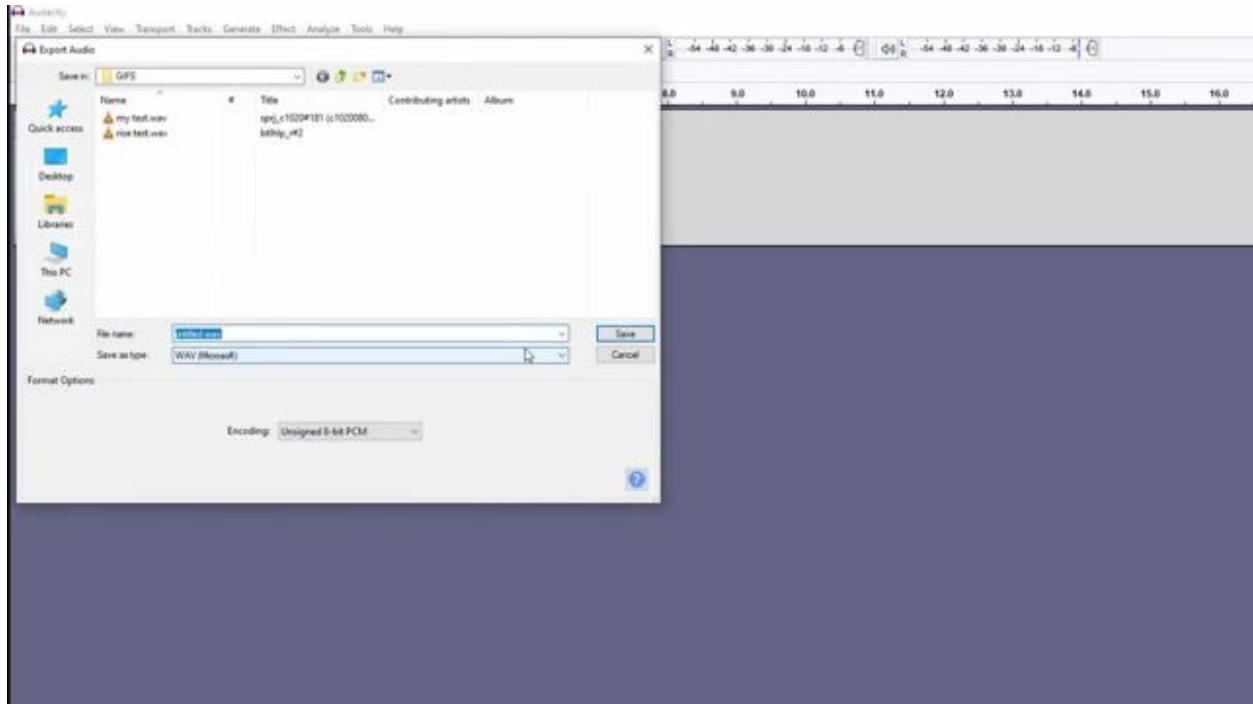


Figure 118: Audacity’s export dialog box, with Encoding set to Unsigned 8-bit PCM

Audacity’s export dialog box, with the Encoding field set to “Unsigned 8-bit PCM”.

Once exported with this encoding, the resulting .wav file can be added to the SGDK project in exactly the same way seen in the previous chapter, declaring it with the WAV keyword in `resources.res`:

```
WAV sfx_hit "sfx/hit.wav" FAR
```

8. Does the file always need to be converted manually?

It’s possible to add a .wav file to `resources.res` that doesn’t meet the recommended format (in stereo and at a sample rate other than 16000 Hz, for example) and have the project still compile. SGDK appears to perform some automatic conversion of .wav files at compile time, even when they aren’t mono or at the recommended sample rate, as long as the file is reasonably small; it isn’t precisely documented exactly how far this goes or under what exact conditions that automatic conversion kicks in, so it’s best to treat it as a fallback behavior rather than a reliable way of working.

That’s why, even though the final result might sound equivalent (the same voice effect, compared in its original high-quality version and in its version manually converted for Mega Drive, can turn out to be practically indistinguishable once played back from the ROM in an emulator), it’s generally best not to rely on that automatic conversion and to always carry out the manual process described in this chapter, because there’s no guarantee that behavior will work equally well in every case.

It’s also worth keeping source files as short and as low in quality as is acceptable, since this reduces the final size taken up inside the ROM, even though SGDK already compresses these sound files reasonably well. On that note, the XGM2 driver (covered in chapter 35, “Playing Music: the XGM and XGM2 Drivers”) is often pointed to as being able to fit even smaller sound effects than the original XGM driver used throughout this chapter, although the exact comparison between the two formats hasn’t been verified here in detail and should be taken as a general reference rather than a precise figure.

9. Adjusting the sound effect’s volume

One important recommendation to close out the chapter: SGDK doesn’t offer any way to adjust a sound effect’s volume directly from code, so any volume adjustment has to be made beforehand in the audio file itself, within Audacity. This is a common problem with sound effects taken from external sources, which often turn out too quiet or too loud relative to the rest of the game’s music and effects.

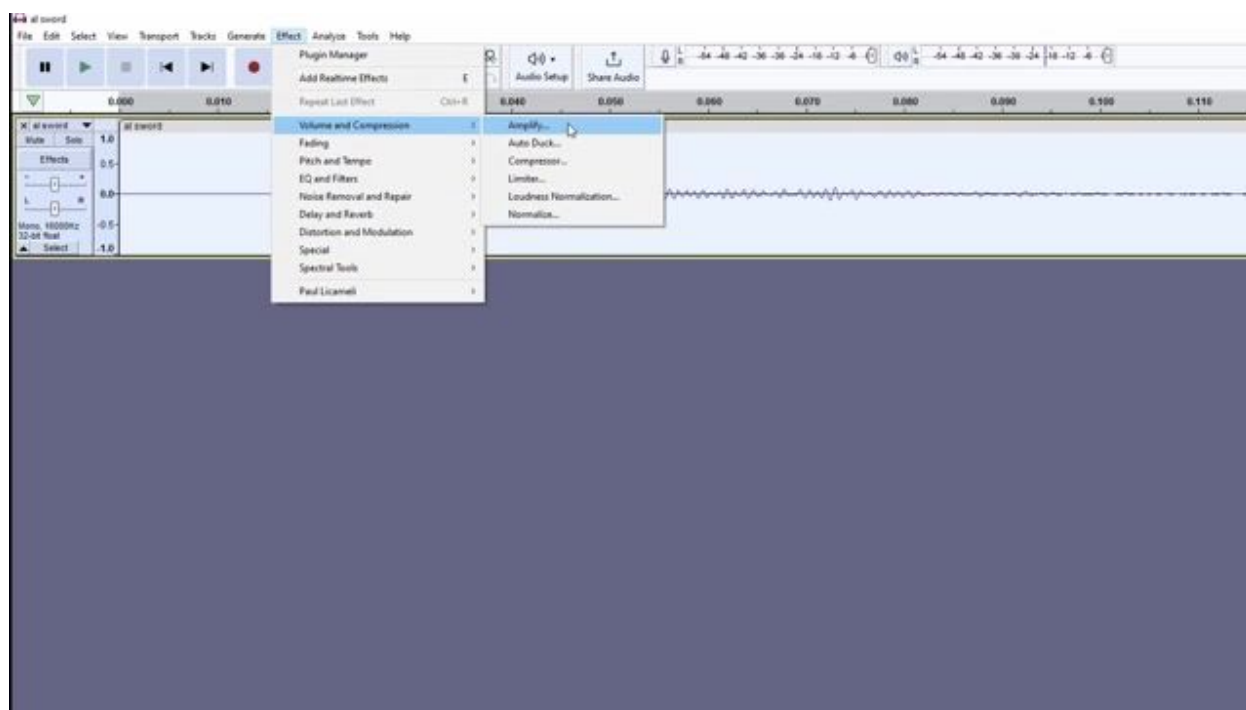


Figure 119: Audacity’s Effect menu with the Volume and Compression section expanded, showing Amplify

Audacity’s Effect menu, with the “Volume and Compression” section expanded, showing the Amplify option.

The procedure consists of selecting the entire sound effect and applying the *Amplify* effect, found within the *Volume and Compression* category. The dialog box lets you preview the result

before applying the change, and by default it suggests the greatest amplification possible without the sound becoming distorted (without clipping the signal).

If an even higher volume than the default suggestion is desired, the *Allow Clipping* checkbox can be checked, which allows raising the amplification beyond the recommended limit. Doing so, however, can cause part of the signal to clip and lose quality, so it's generally best not to enable that checkbox and to stick with the highest amplification level that doesn't produce clipping, leaving the final desired volume as a design decision that depends on the overall balance between music and effects in each game.

A sound effect's volume can't be fixed afterward, once it's already compiled into the ROM: any adjustment has to be resolved in Audacity, before exporting the final .wav file.

Review exercises

1. Describe the two main technical adjustments, beyond the content of the sound itself, that should be applied to an audio recording before exporting it for Mega Drive.
2. What specific encoding should be chosen when exporting the final .wav file from Audacity, and why does that setting matter?
3. Why is it best to manually convert a sound effect to meet Mega Drive's requirements, rather than relying on SGDK's automatic conversion at compile time?
4. Explain why a sound effect's volume adjustment should be done in Audacity rather than in SGDK code.
5. What risk does enabling the "Allow Clipping" option involve when using the Amplify effect, and why is it generally best to avoid it?

Chapter 39 — The Resource Compiler in Depth

A unified view of rescomp: resource types, compression modes, and the organization of resources.res

From Chapter 7 through Chapter 38, practically every chapter of this manual used rescomp, SGDK's resource compiler, to solve a specific problem: displaying a background, assembling a large level, animating a sprite, playing music, or triggering a sound effect. In each of those cases, rescomp appeared as a tool in service of a concrete goal, and each chapter explained only what was needed for that goal. This chapter closes Part VI by taking a step back: instead of adding a new resource, it brings together in one place the complete map of what rescomp can do, what each resource type is for relative to the others, and what general criteria are worth applying when choosing compression and organizing a resources.res file that, in a real project, ends up growing well beyond two or three lines.

There is no new code to learn in this chapter: every keyword, function, and structure that appears here was already used in some earlier chapter. The goal here is different — this is a reference chapter, a chapter of judgment, meant to be revisited whenever a real project forces a decision, such as whether to compile a giant map with BEST or FAST, or how to keep a three-hundred-line resources.res from becoming impossible to maintain.

1. What rescomp is and its place in the pipeline

Chapter 7 first introduced the central idea: a graphic, audio, or level resource does not automatically become available inside a Mega Drive program. Before it can be used from main.c, it has to be declared in a text file — by convention, resources.res, inside the project's res folder — and left for rescomp to process during compilation.

The full circuit always has the same shape, regardless of the type of resource involved:

1. A source file (PNG, WAV, VGM, TMX, depending on the type) is placed in the project's res folder.
2. A line in resources.res declares the resource: its type, a name chosen by the programmer, the path to the source file, and, depending on the type, additional parameters (compression, frame size, sound driver, etc.).
3. When the project is compiled, rescomp reads resources.res, processes each source file, and generates two new files, resources.h and resources.c, within the project structure.
4. main.c includes resources.h and, from that point on, can refer to each resource by the name given to it in the .res file, as if it were a global variable already initialized with the data from the original file.
5. The processed content — already in the binary format expected by the Mega Drive hardware, and compressed as indicated — ends up embedded in the final ROM (rom.bin)

when the project is linked.

As Chapter 7 already pointed out, the `.res` file is the source that gets edited by hand, while `resources.h/resources.c` are products generated automatically on every build: they are never edited directly, and any change has to be made in the source `.res` file. Also, a newly declared resource name is not available to the editor or to the compiler for `main.c` until the project is saved and built at least once.

2. The resource types seen in this manual

The table below brings together every resource type used by this manual so far, showing what each one generates and the chapter in which it first appeared. It is not an exhaustive list of everything `rescomp` supports — some additional types, such as those related to custom text fonts, fall outside the scope of this manual — but it covers the core resources for graphics, levels, and sound.

Type	What it generates	First used
IMAGE	A complete Image structure: tileset, tile map, and palette, all together, intended for a graphic of up to 512×256 pixels that gets drawn in one shot with <code>VDP_drawImageEx()</code> .	Chapter 7
TILESET	A set of unique 8×8-pixel tiles extracted from an image, discarding duplicates and — with the <code>ALL</code> option — also mirror images, to be loaded with <code>VDP_loadTileSet()</code> .	Chapter 11
PALETTE	The color data (<code>.data</code>) of an image, on its own, for cases where the main resource doesn't include its own palette.	Chapter 11
MAP	A <code>MapDefinition</code> structure with the complete tile layout of a level, referencing an already-declared <code>TILESET</code> , used to build a scrollable map with <code>MAP_create()</code> and <code>MAP_scrollTo()</code> .	Chapter 11
TILEMAP	A <code>TileMap</code> structure, generated from a <code>.tmx</code> file exported by Tiled, with the width, height, and list of tile indices for a layer. It can be combined with <code>TILESET/PALETTE</code> and <code>MAP_create()</code> , or traversed directly for small maps without scrolling.	Chapter 12

Type	What it generates	First used
SPRITE	A <code>SpriteDefinition</code> structure, with one or more animation frames organized by row, meant to be used with the sprite engine (<code>SPR_addSprite()</code> , etc.) rather than with the plane functions.	Chapter 21 (declaration) / Chapter 22 (preparing the source sheet)
XGM	A music track for SGDK's original audio driver, produced from an uncompressed VGM file.	Chapter 35
XGM2	An equivalent music track for the newer driver, with a smaller ROM footprint at the cost of one fewer PCM channel.	Chapter 35
WAV	An individual sound effect produced from an uncompressed <code>.wav</code> file, tied to one of the two music drivers (XGM or XGM2) so it can be played through a PCM channel.	Chapter 37

Looking at the table as a whole, a pattern emerges that was already hinted at chapter by chapter: `rescomp` doesn't offer a single "graphics" resource type or a single "sound" resource type, but rather variants designed for different usage scenarios. `IMAGE` and `SPRITE` bundle everything needed into a single declaration because they're designed for the simple case: one screen, one character. `TILESET`, `MAP`, `TILEMAP`, and `PALETTE`, on the other hand, split those same pieces apart because a large level needs to share tiles across many positions and, eventually, across several maps. Something similar happens with sound: `XGM/XGM2` handle the complete track with their own channel management, while `WAV` handles the one-off effect triggered in response to a game event. Choosing the right resource type is, in practice, the first design decision for any new graphic or sound, even before thinking about compression.

3. Compression modes, in a unified view

Several of the resource types from the table above — `IMAGE`, `TILESET`, `MAP`, `SPRITE` — accept a compression field, and Chapters 7 and 21 already introduced, separately, the same three alternatives:

- **NONE**: the resource is included uncompressed. It takes up more space in ROM, but requires no decompression work at runtime: the data is already in the final format the hardware expects.
- **FAST**: a lighter compression algorithm, which produces a smaller file than **NONE** but not as compact as **BEST**, in exchange for decompression that is quick and cheap in terms of CPU at runtime.
- **BEST**: the mode that produces the smallest file. In `rescomp`'s documentation this corresponds to the `APLIB` algorithm, though in recent versions of SGDK the same keyword may also imply an automatic selection of whichever algorithm best suits each particular

resource — a behavior Chapter 21 already pointed out, and one worth checking against the actually installed SGDK version before assuming BEST is always synonymous with “the most aggressive compression and the slowest to decompress.”

The criterion for choosing among the three doesn’t change depending on the resource type: what changes is how many times per second that resource will be decompressed during the game, and how much CPU time is available at that moment.

- A resource that gets decompressed **only once**, typically when loading a screen or a level — a static background, the tileset and map of a full level, a menu’s palette — can afford the cost of BEST with no perceptible impact: the decompression happens once, during a loading screen or a transition, and after that the data already lives decompressed in VRAM or in working RAM. This is exactly the case of the IMAGE resource from Chapter 7 and the TILESET/MAP from Chapter 11: there’s no practical reason not to use BEST there.
- A resource that gets decompressed **many times per second** — the central case being an animated sprite, which changes frames several times per second and may need to decompress data on every change — pays the cost of BEST over and over, frame after frame. Chapter 21 already showed that this cost is measurable: in a project with several animated characters on screen at once, switching their sprites from FAST to BEST can cause a noticeable drop in frame rate. For this type of resource, FAST is usually the more sensible choice, reserving BEST for cases where ROM size is a more pressing constraint than speed.
- NONE makes sense in two specific situations: when the resource is already so small that compressing it wouldn’t yield a meaningful saving, or when the fastest possible access is needed and there’s no room even for the light cost of FAST — for example, data that is read extremely frequently and where every additional CPU cycle is critical.

The question worth asking about each resource, then, is not “how big is this file?” but “how many times per second does it need to be decompressed, and how tight is the CPU budget at that moment in the game?” A huge level background, even though it’s a heavy file, gets decompressed only once when entering the level and can safely go in BEST; a small sprite, if it’s animated constantly, may justify FAST even though it weighs much less in raw terms.

This same trade-off between space and speed reappears, in another form, in the alternative already mentioned in Chapter 21 for sprite compression: loading every possible animation frame into VRAM up front, thereby avoiding any decompression during gameplay, at the cost of permanently occupying the VRAM space those frames need — space that then becomes unavailable for anything else. This isn’t a better or worse alternative to adjusting compression: it’s the same space-versus-speed trade-off, resolved at the opposite end of the problem.

4. Organizing a growing resources.res

Every example in this manual, chapter after chapter, worked with a `resources.res` of just a few lines, designed to introduce one resource at a time. A real project ends up accumulating dozens or hundreds of lines: tilesets and maps for several levels, sprite sheets for every character and enemy, a music track per scene, and a long list of sound effects. Without some extra care, that file quickly becomes hard to navigate and maintain, especially on a team of more than one person.

The safest way to keep that file organized, within what this manual can confirm based on the information already used in earlier chapters, is a discipline of organization within a single `resources.res` file, rather than relying on splitting resource declarations across several separate files. Specifically:

- **Group lines by area of the game**, with a comment marking each block — for example, one block for the playable character’s resources, another for each level, another for the

interface, and another for audio — in the same order in which it makes sense to review them.

- **Adopt a consistent prefix in resource names** that identifies which area or which level each one comes from — for example, `lvl1_tileset`, `lvl1_map`, `lvl1_palette` for the first level, and `player_walk`, `player_idle` for the character's animation sheets — so that the resource name is already readable in `main.c`'s code without having to go back to `resources.res` to remember what it corresponds to.
- **Keep related resource lines together**, as already shown in Chapter 11 with `TILESET`, `PALETTE`, and `MAP` for the same map, or in Chapter 12 with `TILESET`, `PALETTE`, and `TILEMAP` for a level designed in Tiled: declaring those lines one right after another makes the relationship between them far clearer than scattering them across the file.
- **Review compression at the end of each block**, since it's easy to copy and paste a line from an already-declared resource as a starting point for a new one and forget to adjust the compression to match the actual use case for that new resource (for example, copying the line for a background compressed with `BEST` as the basis for a sprite that would actually need `FAST`).

As for the possibility of splitting resource declarations across more than one `.res` file per area of the game — a reasonable idea in the abstract, especially in large projects — this manual cannot confirm with certainty, based on what has been covered in earlier chapters, whether `rescomp` natively supports processing multiple `.res` files or whether some additional inclusion mechanism between them would be needed. Before assuming that path is available, it's worth checking against the documentation for the installed `SGDK` version; organizing by blocks and prefixes within a single file, on the other hand, is a recommendation that doesn't depend on any uncertain feature of the tool, and it's already enough, in practice, to keep a sizable project organized.

Review exercises

1. Without looking at the table in Section 2, list the nine resource types covered in this manual and, for each one, the chapter where it was first used.
2. A teammate declares the tileset and map of a full level with `FAST` compression, “so it loads faster,” and the main character's animated sprite with `BEST`, “to save space.” Explain why that choice is probably backwards, based on this chapter's space-versus-speed criterion.
3. Why does a `MAP` resource additionally need a separately declared `PALETTE` resource, while an `IMAGE` resource doesn't need one?
4. Explain in your own words the difference in purpose between the `TILESET`/`MAP` resources (Chapter 11) and `TILEMAP` (Chapter 12), and in what situation you would choose each one.
5. Propose an organization scheme (blocks and name prefixes) for the `resources.res` of a platform game with three levels, a playable character with several animations, two different enemies, and music and sound effects specific to each level.

Chapter 40 — Level Collision for a Top-Down 2D Game

With backgrounds, large maps, sprites, the camera, controls, and sound already covered, there's one problem left that shows up in practically any game with free movement: keeping the character from walking through walls, furniture, or any other obstacle in the scene. This chapter opens Part VII of the manual, dedicated to physics and collisions, and builds a level collision system from scratch for a game viewed from above, in the style of the early Zelda titles.

The example in this chapter reuses the single-screen room scene (320×224 pixels) with beds, furniture, and walls as obstacles. That scene, with its narrow gaps between objects, works as a good stress test for the code we're about to build: if the collision correctly resolves the tight corners between two nearby pieces of furniture, it will have no trouble with the simpler cases.

Prerequisites

Before continuing, it helps to have already covered:

- Chapter 6, on data types (u8, u16, s16, const) and the Mega Drive's memory constraints.
- Chapter 13, on creating maps with 16Tile, since this chapter reuses that workflow to generate an additional layer.
- Chapter 21, on displaying sprites, and Part III in general, particularly Chapter 24 on the 2D camera: the camera code from that chapter is reused here unmodified.
- An understanding of what an array is: a variable capable of storing several values of the same type, accessible via a numeric index starting at 0.

1. Where to place the collision box within the sprite

The first conceptual step, before writing a single line of code, is deciding where the collision box will sit within the character's sprite: the rectangle whose four corners are checked against the map to decide whether a movement is valid. That box doesn't have to match the full sprite; in fact, it's almost always better for it to be smaller.

For a top-down character, it usually works best to place it around the feet rather than over the entire sprite. That way, the character can visually overlap with the top of a piece of furniture or the headboard of a bed without that counting as an actual collision — something already possible thanks to tile priority, covered in Chapter 15: a tile with priority can be drawn on top of the sprite even though, in the pixels further down, the character is still physically blocked.

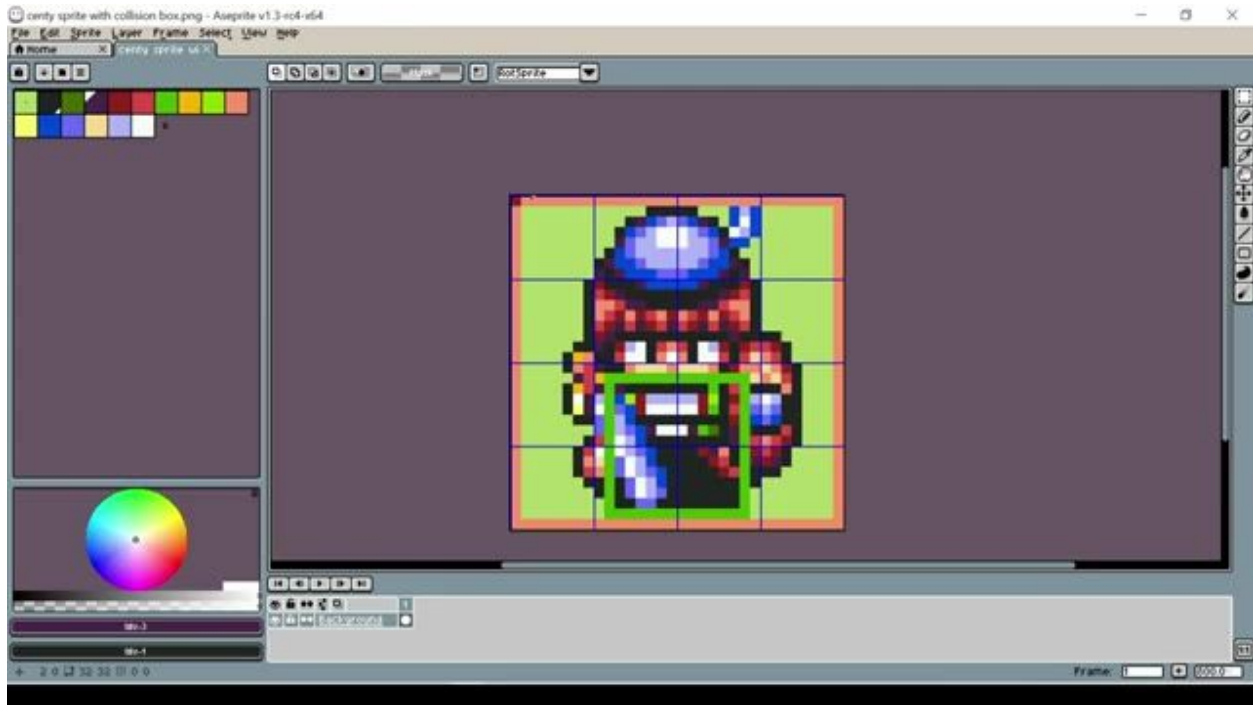


Figure 120: The character sprite with an overlaid box marking the collision box

The reference box marks where the collision box will be placed: around the character's feet, smaller than the full sprite.

2. A second map layer, dedicated solely to collision

The core technique in this chapter is to create, within the same 16Tile project used for the background (Chapter 13), a second layer dedicated exclusively to collision. That layer uses its own tileset, completely different from the graphical one: a file of 16 blocks of 16×16 pixels, each a distinct flat color. Each color will represent a type of collision.

For the purposes of this chapter, two colors are enough: one (black, for example) for a free tile, through which the character can move without restriction, and another (red, for example) for a fully solid tile. The remaining 14 colors in the tileset stay available for more elaborate collision types that aren't needed yet: a water tile passable only with certain equipment, or a one-way tile that lets you drop down a ledge without being able to climb back up, like the ones found in several classic games from that era.

With the layer created and named (collision, for example), the painting itself works exactly like any other 16Tile layer you've already used: select the red block and paint it over every map tile that should behave as solid, leaving the rest in the color that represents "free." The layer's visibility icon (shaped like an eye) lets you toggle between viewing it opaque, semi-transparent over the background, or hidden, which makes it easier to paint the collision while still seeing the actual background as a reference.

Painting the collision layer at reduced opacity, to keep seeing the actual background as a reference while placing the solid tiles.

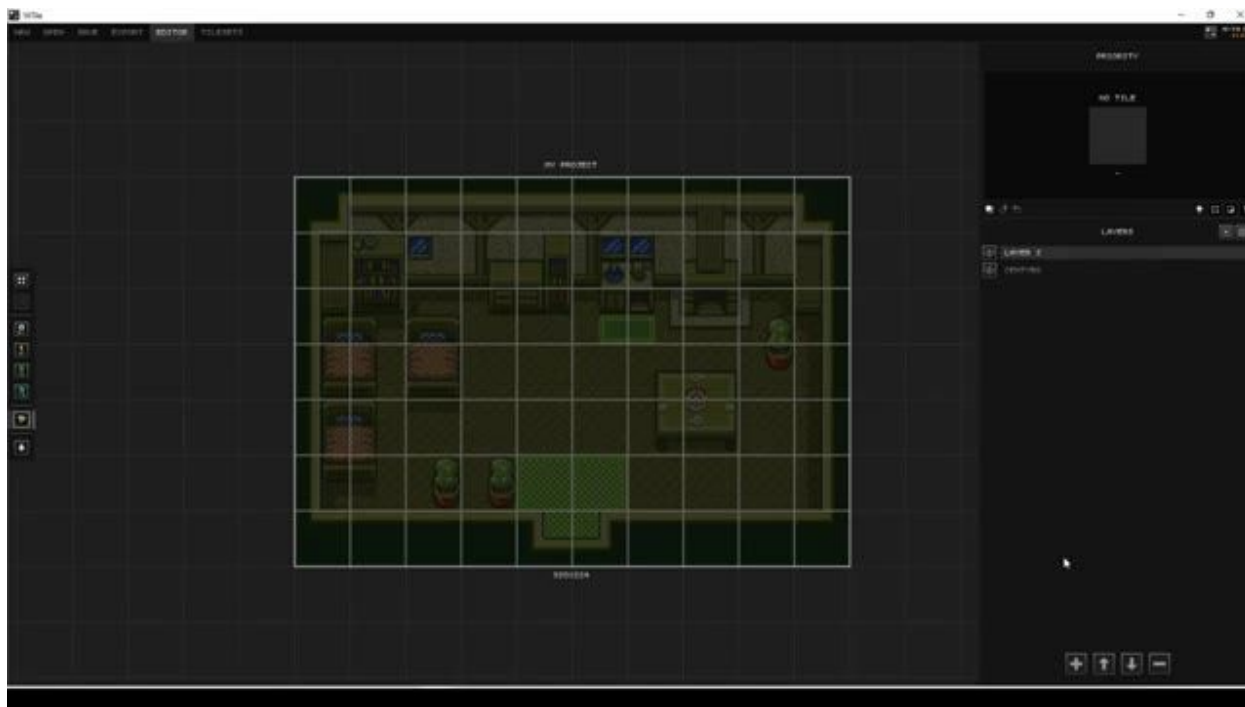


Figure 121: The semi-transparent collision layer over the background

3. Exporting the layer as a collision array

Before exporting, the collision layer needs a special setting within 16Tile's layers panel: an option that tells the tool this particular layer should be exported as a data array rather than as a graphical background. When you export the full project, the result is two quite distinct files: one for the background layer (an image, just like the ones seen in earlier chapters) and another for the collision layer, which is a plain text file with C array syntax, ready to paste into the project: a series of 1s (solid tile) and 0s (free tile), one per map tile, in row order.

Automating this step is one of the reasons 16Tile becomes especially valuable as levels grow larger: writing the collision array by hand, tile by tile, is slow and error-prone, especially on maps bigger than a single screen, like the ones worked with using `MAP_create()` in Chapter 11.

4. The collision array in `main.c`: `const u8`, not `u16`

The text file exported by 16Tile declares the array as `u16` by default. Since in this example its only possible values are 0 and 1, it's worth shrinking it to `u8`, which takes up half the space in ROM without losing any information: for a 320×224 map (20×14 tiles, 280 elements), that's already 280 bytes saved, and it scales proportionally with the size of the level.

The second adjustment, even more important, is declaring the array as `const`, as explained in Chapter 6 on data types and memory. A value marked `const` records in the code that it won't change during gameplay, and in practice it makes the compiler place that array in ROM instead of copying it to RAM, a far scarcer resource on this console (only 64 kilobytes total). Since the collision map never changes during the game, it's a perfect candidate to live permanently in ROM. There's no published measurement confirming how many cycles it costs to read from ROM instead of RAM for an access like this, so it's worth not assuming the

difference is negligible; either way, declaring the array as `const` carries no practical downside and does save real RAM, so it's the recommended way to declare it.

```
// Room collision array (320x224 px = 20x14 tiles = 280 elements).
// Generated by 16Tile from the map's collision layer,
// shrunk to u8 and declared const so it lives in ROM (see Chapter 6).
const u8 level_col[] = {
    1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,
    1,0,0,0,0,0,0,0,0,0,0,0,0,0,1,0,0,0,0,0,1,
    1,0,0,1,1,0,0,0,0,0,0,0,1,0,0,0,0,0,0,0,1,
    1,0,0,1,1,0,0,0,0,0,0,0,1,0,0,0,0,0,0,0,1,
    // ... the rest of the rows, up to the full 14 rows of 20 columns
    1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,
};
```

5. Two helper constants

To work comfortably with the array, two additional constants are needed.

The first is the array's horizontal length in tiles: how many values there are in each row before moving to the next (20, in this map). That value can exceed 255 in levels larger than a single screen, like the ones in Chapter 11, so it's worth declaring it as `u16` from the start instead of `u8`, even though in this example it's well below that limit:

```
const u16 level_col_width = 20;
```

The second constant attaches a readable name to the numeric value that represents a solid tile within the array, using the same `#define` technique already used in earlier chapters for sprite animations:

```
#define SOLID_TILE 1
```

Using `SOLID_TILE` instead of the raw number 1 in the code makes any future comparison much more readable, especially if other tile types get added later (water, slopes, and so on) and you need to remember which number corresponds to each one.

6. The collision box offsets

`player_x` and `player_y` always represent the top-left corner of the full sprite, not of the collision box defined in section 1. To get that box's actual coordinates, you need to calculate, for each of its four sides, the pixel distance to the corresponding edge of the sprite. Those four values are measured once, on the reference sprite, and declared as constants alongside `PLAYER_WIDTH` and `PLAYER_HEIGHT`:

```
#define PLAYER_WIDTH      32
#define PLAYER_HEIGHT     32

#define PLAYER_COLBOX_LEFT 10
#define PLAYER_COLBOX_RIGHT 23
#define PLAYER_COLBOX_TOP  18
#define PLAYER_COLBOX_BOTTOM 31
```

7. Setting up collision()

All the logic in this chapter is concentrated in a new function, `collision()`, which follows the same pattern already used for `handle_input()` and `camera()`: a prototype declared near the top of the file, the function called inside the main loop (right after `handle_input()` and before `camera()`), and the body written at the end of the file.

```
while (TRUE)
{
    handle_input();
    collision();
    camera();

    SPR_update();
    SYS_doVBlankProcess();
}
```

To verify each calculation as the code is written, it's worth reusing the on-screen debugging technique already seen in Chapter 6: a text buffer and a call to `VDP_drawTextBG()` inside `collision()`, continuously displaying the value of some variable of interest.

Inside `collision()`, the first step is to calculate the box's actual coordinates in pixels on the map, by adding each offset to the sprite's coordinate (converted from `fix32` to an integer with `F32_toInt()`):

```
s16 player_left_collision_coord  = F32_toInt(player_x) + PLAYER_COLBOX_LEFT;
s16 player_right_collision_coord = F32_toInt(player_x) + PLAYER_COLBOX_RIGHT;
s16 player_top_collision_coord   = F32_toInt(player_y) + PLAYER_COLBOX_TOP;
s16 player_bottom_collision_coord = F32_toInt(player_y) + PLAYER_COLBOX_BOTTOM;
```

8. From pixels to tiles: dividing by 16 with a bit shift

The collision array doesn't work in pixels but in 16×16 tiles, so the coordinates above need to be converted into a tile column or row number by dividing them by 16. As already explained in Chapter 6, dividing by a power of 2 is best done with a bit shift instead of the division operator, since the Mega Drive's CPU handles generic division poorly: dividing by 16 is equivalent to shifting four positions to the right (`>> 4`).

```
s16 xtilecoord_left  = player_left_collision_coord >> 4;
s16 xtilecoord_right = player_right_collision_coord >> 4;
s16 ytilecoord_top   = player_top_collision_coord  >> 4;
s16 ytilecoord_bottom = player_bottom_collision_coord >> 4;
```

9. From tile coordinates to array index

With the tile coordinates for each corner already calculated, the next step is to translate them into the corresponding index within the linear collision array, which numbers its 280 elements continuously from 0 to 279, row by row. The formula is:

$$\text{index} = (\text{row} \times \text{level_col_width}) + \text{column}$$

A couple of concrete examples help confirm the formula is correct: a tile in column 0, row 1 gives $(1 \times 20) + 0 = 20$; a tile in column 2, row 6 gives $(6 \times 20) + 2 = 122$. Both match the real position of those tiles within the array.

Applying this formula to the four corners of the collision box gives the four indices that need to be looked up, and with them, the collision type at each corner:

```
u16 array_index_topleft_colbox    = (ytilecoord_top    * level_col_width) +
  ↳ xtilecoord_left;
u16 array_index_topright_colbox   = (ytilecoord_top    * level_col_width) +
  ↳ xtilecoord_right;
u16 array_index_bottomleft_colbox = (ytilecoord_bottom * level_col_width) +
  ↳ xtilecoord_left;
u16 array_index_bottomright_colbox = (ytilecoord_bottom * level_col_width) +
  ↳ xtilecoord_right;

u8 topleft_col_type    = level_col[array_index_topleft_colbox];
u8 topright_col_type   = level_col[array_index_topright_colbox];
u8 bottomleft_col_type = level_col[array_index_bottomleft_colbox];
u8 bottomright_col_type = level_col[array_index_bottomright_colbox];
```

Displaying these last four variables with `VDP_drawTextBG()` while the character moves around the scene lets you visually confirm that the value switches to `SOLID_TILE` when approaching a piece of furniture or a wall, and goes back to `0` over open floor.

10. Boolean movement variables

With all the collision information available, the next step is to use it to control actual movement. The approach is to introduce four boolean variables, one per direction, defaulting to false:

```
bool player_move_left  = FALSE;
bool player_move_right = FALSE;
bool player_move_up    = FALSE;
bool player_move_down  = FALSE;
```

In `handle_input()`, instead of moving the character directly when a D-pad key is detected as pressed, each direction now toggles the corresponding boolean on or off: `TRUE` if the button is pressed, `FALSE` if it isn't, or if the opposite direction is being pressed. The actual movement is moved entirely into `collision()`, which from now on is the only function responsible for deciding whether the character actually moves.

```
if (JOY_readJoypad(JOY_1) & BUTTON_LEFT)
{
    player_move_left = TRUE;
    player_move_right = FALSE;
}
else if (JOY_readJoypad(JOY_1) & BUTTON_RIGHT)
{
    player_move_right = TRUE;
    player_move_left = FALSE;
}
else
{
    player_move_left = FALSE;
    player_move_right = FALSE;
}
// The same pattern repeats for up and down.
```

11. Resolving leftward movement

Inside `collision()`, an `if (player_move_left)` block brings together three steps in order: move the character to the left, calculate whether that movement causes the top-left or bottom-left corner of the collision box to land on a solid tile, and if so, correct the position so it ends up flush against the edge of that tile. Since all of this happens within the same animation frame, the player never actually sees the character visually penetrating the obstacle before being pushed back.

It's worth moving the pixel, tile, and index calculations for the left side into this block, since there's no point calculating them if the character isn't moving in that direction:

```
if (player_move_left)
{
    player_x = F32_sub(player_x, FIX32(PLAYER_SPEED));

    s16 left_coord = F32_toInt(player_x) + PLAYER_COLBOX_LEFT;
    s16 top_coord  = F32_toInt(player_y) + PLAYER_COLBOX_TOP;
    s16 bottom_coord = F32_toInt(player_y) + PLAYER_COLBOX_BOTTOM;

    s16 xtile_left = left_coord >> 4;
    s16 ytile_top   = top_coord  >> 4;
    s16 ytile_bottom = bottom_coord >> 4;

    u16 idx_topleft    = (ytile_top * level_col_width) + xtile_left;
    u16 idx_bottomleft = (ytile_bottom * level_col_width) + xtile_left;

    if (level_col[idx_topleft] == SOLID_TILE || level_col[idx_bottomleft] ==
        ↪ SOLID_TILE)
    {
        // Align the left edge of the collision box (not the sprite)
        // with the right edge of the solid tile it collided with.
        s16 blocked_coord = (xtile_left << 4) + 16 - PLAYER_COLBOX_LEFT;
        player_x = FIX32(blocked_coord);
    }
}
```

The calculation for the bounce-back position takes the tile column where the obstacle lies, multiplies it by 16 using a left bit shift (multiplying by a power of 2 allows the same shortcut as dividing), adds another 16 — the width of a tile, to land on its right edge instead of its left edge — and finally subtracts the collision box offset, since `player_x` describes the position of the full sprite, not the box. Skipping that final subtraction leaves the character pushed noticeably farther from the obstacle than expected, so it's worth paying attention to it: the correct result stops the character exactly where the green reference collision box touches the edge of the furniture, with no visible extra bounce.

12. Right, up, and down

Rightward movement follows the same logic as leftward movement, with two differences: it uses the coordinates of the box's right corners instead of the left ones, and its bounce-back formula doesn't need to add the full width of a tile, since aligning the box's right edge with the solid tile's left edge doesn't require that extra adjustment (though it does still subtract the corresponding offset, in this case `PLAYER_COLBOX_RIGHT`):


```
s16 blocked_coord = (xtile_right << 4) - PLAYER_COLBOX_RIGHT;
player_x = FIX32(blocked_coord);
```

Vertical movement (up and down) repeats the same structure, using the top or bottom corner coordinates as appropriate.

Horizontal movement is deliberately resolved before vertical movement: since both axes share some of the same tile coordinates, resolving the horizontal collision first simplifies the code needed later for more advanced collision types, such as slopes. This is a reasonable design decision for this case, not a mandatory engine rule: with other collision schemes, the reverse order might be preferable, or even resolving both axes together.

13. The “wall jump” problem and its fix

With all four movement directions now working individually, a problem shows up when combining two directions at once: holding down, for example, right and up at the same time near a wall makes the character momentarily get stuck against it, instead of sliding smoothly. This happens because, when the horizontal axis is resolved first, the corrected position leaves the character flush against the edge of the solid tile (zero distance), and that exact same coincidence can cause the vertical axis calculation, right afterward, to interpret that position as a collision all over again.

The fix is to subtract a fraction of a pixel (0.1) from the character’s coordinate immediately after resolving the horizontal collision — just enough to nudge it slightly away from the wall without the human eye noticing the difference, but enough that the vertical axis no longer gets stuck against it:

```
if (level_col[idx_topleft] == SOLID_TILE || level_col[idx_bottomleft] ==
    ↪ SOLID_TILE)
{
    s16 blocked_coord = (xtile_left << 4) + 16 - PLAYER_COLBOX_LEFT;
    player_x = F32_sub(FIX32(blocked_coord), FIX32(0.1));
}
```

The same adjustment, subtracting 0.1 from player_y after vertical collision, fixes the equivalent problem when combining downward movement with horizontal movement.

14. The final result

With all four movement directions and both “wall jump” fixes in place, the character can move around the entire scene, consistently colliding with beds, furniture, and walls, including the narrow gaps between objects mentioned at the start of the chapter.

After subtracting the collision box offset in the formula, the character stops right at the edge of the obstacle, with no visible bounce or penetration.

Review exercises

1. Explain the difference between declaring the collision array as u16 or as u8, and why it’s also worth declaring it as const in the specific context of the Mega Drive.
2. Why does a character’s collision box need its own offset values relative to the full sprite, instead of using player_x and player_y directly?



Figure 122: The character stopped exactly against the edge of a piece of furniture

3. Given a tile in column 3 of row 4, and a collision array with a row width of 20, calculate the corresponding index within the array using the formula presented in this chapter.
4. Explain why, when calculating the leftward bounce-back position, it's necessary to add 16 (the size of a tile) in addition to multiplying the column by 16, while the rightward calculation doesn't need that same step.
5. Describe the "wall jump" problem mentioned in this chapter and the solution applied to prevent it. Why does the order in which the axes are resolved (horizontal before vertical) affect whether this problem shows up?

Chapter 41 — Jump and Gravity for a 2D Platformer

Chapter 40 left us with a complete level collision system for a top-down game: a collision box around the character, a constant array describing which map tiles are solid, and a `collision()` function capable of converting pixels into indices of that array to stop movement in all four directions. This chapter takes that same code as a starting point and adapts it to the genre that defined a large part of the 16-bit catalog: the side-scrolling platformer. Most of the heavy lifting — converting pixel coordinates to tile coordinates, the array index formula, checking each corner of the collision box — is already solved; what changes is how that information gets used. In a top-down game the player is free to decide to move up or down; in a platformer, gravity constantly pulls the character downward, and the player can only fight it by jumping.

Prerequisites

Before continuing, it helps to already have covered:

- Level collision for a top-down 2D game (chapter 40): this chapter directly reuses and modifies that code, in particular the `collision()` function and the collision array.
- Button controls and event-based reading with `JoyEvent()` (chapter 32): the jump button is added following that same pattern.
- The `fix16/fix32` types, the `FIX32()` macro, and the `F32_toInt()` conversion (chapter 6): all the physics in this chapter — gravity, jump velocity, terminal velocity — are expressed using `fix32`.

1. From top-down to side-scrolling: what changes and what stays the same

The tileset used as an example in this chapter — possibly related to the one from *Chakan: The Forever Man*, though there's no way to confirm this for certain beyond the visual resemblance — includes common ground tiles, some fully solid platforms (meant for practicing jumping: they can't be passed through from the sides, from above, or from below), and a semi-solid platform type, common in the genre, that lets the character pass freely through the sides and from below but stops them when they fall onto it from above. This chapter focuses on the ground and the fully solid platforms along with gravity and jumping; semi-solid platforms are left for chapter 43, and adapting this same system to a larger sprite is left for chapter 42.

The tileset used in the chapter, with its ground tiles, solid platforms, and semi-solid platform grouped together in the original sheet.

The map and its collision array are prepared in `16Tile` using the same procedure already covered in chapter 40: a background layer and a collision layer exported as an array of ones and

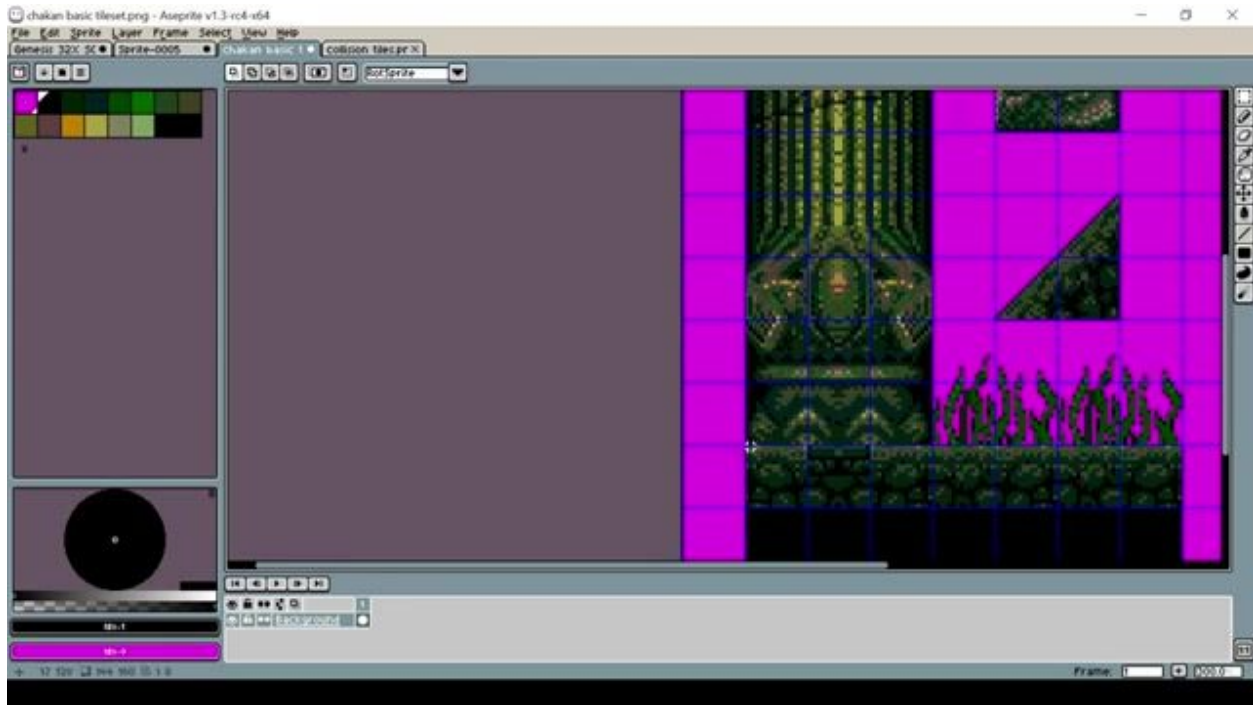


Figure 123: The tileset for this chapter, with ground tiles, solid platforms, and the semi-solid platform

zeros. Since this map is considerably larger than the single-screen room from the previous chapter, `MAP_WIDTH` and `MAP_HEIGHT` need to be updated to the new dimensions in pixels, the row length of the collision array needs to be recalculated (the new map width divided by 16), and both the array's contents and the total element count declared in brackets need to be replaced. A collision bug that shows up erratically on a new map is almost always caused by getting one of those two numbers wrong.

2. Removing free vertical movement

The first real change toward a side-scrolling platformer is eliminating the vertical movement that made sense in a top-down game but not in this genre: the boolean variables `player_move_up` and `player_move_down`, along with the lines in `handle_input()` that activated them using the D-pad directions, are removed entirely. Horizontal movement stays unchanged.

The collision calculation against the ground and the ceiling, however, isn't discarded: much of that work gets reused later on. The upward collision block is temporarily commented out (it's picked back up in section 6 of this chapter, for ceiling collision when jumping), while the downward collision block gets restructured from the ground up. In the top-down game, that code only ran if the player pressed the down direction; in a platformer, gravity acts on the character every single frame, whether or not any button is pressed, so the `if` that used to guard it goes away and the ground collision calculation stays permanently active.

3. Gravity as a constant vertical velocity

The heart of this chapter is a new variable, `gravity`, of type `fix32`, with an initial test value:

```
fix32 gravity = FIX32(0.8);
```

Also, since the character now has its own vertical velocity — independent of whether the player is pressing a key — another fix32 variable is needed:

```
fix32 player_y_vel = FIX32(0);
```

This exposes a naming problem: the variable `player_vel`, inherited from chapter 40, represented the fixed horizontal speed at which the character moved. To avoid confusing it with the new vertical velocity, it should be renamed to `player_x_vel` everywhere it appears.

The gravity logic itself is surprisingly simple: each frame, the value of `gravity` gets added to `player_y_vel` (the falling speed progressively accelerates, frame by frame), and then `player_y_vel` gets added to the `player_y` coordinate itself. Since the vertical axis on the Mega Drive grows downward (0 at the top, larger values toward the bottom), adding a positive velocity pulls the character downward:

```
// Gravity: accumulates onto the vertical velocity every frame
player_y_vel += gravity;
player_y += player_y_vel;
```

It's a good idea to reset `player_y_vel` to `FIX32(0)` at program start, along with the rest of the character's variables, and to adjust the character's starting position to fit the actual size of the new map before compiling for the first time. With this change, the character falls from its starting position until it hits the ground and can move freely side to side, but once on the ground there's no way to go back up: just like in real life, jumping is needed to counteract gravity.

The values 0.8 for gravity and, later on, 12 for jump velocity don't come from any exact physics formula or any standard SGDK convention: they're a reasonable, hand-picked starting point, meant to be tuned experimentally to match the feel you're after. A higher gravity value produces a sharper fall and a shorter jump; a lower one produces a floatier jump. Treat them as fine-tuning knobs, not constants to memorize.

4. The jump button

Adding the jump requires, first, a new fix32 variable:

```
fix32 jumping_vel = FIX32(12);
```

the upward velocity given to the character the moment the jump button is pressed. Next, button handling needs to be set up through `JoyEvent()`, following the same pattern already seen in chapter 32: a function prototype, `JOY_setEventHandler()` inside `main()`, and the function itself at the end of the file. Any action button works for this (C, B, or one of the extra buttons on the 6-button pad); what matters is the logic inside the handler. Pressing the jump button subtracts `jumping_vel` from `player_y_vel` (subtracts, rather than adds, because any upward movement in this coordinate system requires a negative value):

```
void handle_joy_event(u16 joy, u16 changed, u16 state)
{
    if (changed & state & BUTTON_C)
    {
        player_y_vel -= jumping_vel;
    }
}
```

With this change the character can now jump and climb back up over the map's steps and platforms. A problem shows up right away, though: pressing the button repeatedly and quickly enough lets the character rise indefinitely, since each press subtracts 12 from the vertical velocity while gravity only adds 0.8 per frame, and that difference isn't enough to offset very rapid presses.

This isn't necessarily a bug that needs fixing in every case: some games deliberately offer double or triple jumps, and others use the jump button to control the character's buoyancy underwater, pressing it repeatedly to avoid sinking. For this chapter's example, though, the goal is to limit the character to a single jump until it touches the ground again.

5. Limiting the jump to a single use

The solution is a new boolean variable, initialized to false:

```
bool jumping = false;
```

When the jump button is pressed, in addition to subtracting `jumping_vel`, this variable gets set to true. The button's condition is extended with an `&&` to require, besides the button being pressed, that the character not already be in the air (`!jumping` is a common, shorter way of writing `jumping == false`):

```
void handle_joy_event(u16 joy, u16 changed, u16 state)
{
    if ((changed & state & BUTTON_C) && !jumping)
    {
        player_y_vel -= jumping_vel;
        jumping = true;
    }
}
```

The only remaining step is setting `jumping` back to false at the right moment: when the character actually touches the ground, inside the existing lower collision block, alongside the rest of the position and velocity corrections applied there. With this change the character can only jump once per ground touch, without chaining jumps in mid-air.

6. Re-enabling ceiling collision

With the jump now limited to one use, a problem becomes obvious: since the upper collision block was left commented out in section 2, the character can jump straight through any platform above instead of bumping its head against it. To re-enable it, it helps to first reorder the lower collision block, and then reuse the same six lines of calculation (pixel coordinates, tile coordinates, array indices) already used for the ground, adapting them to the top corners of the collision box. The commented-out block gets uncommented and added right after the lower collision block, this time as an `else if` rather than a standalone `if`, since in any given frame the character can only be touching either the ground or the ceiling, never both at once:

```
if (player_bottomleft_col == SOLID_TILE || player_bottomright_col == SOLID_TILE)
{
    // Ground collision: snap the position to the top edge of
    // the solid tile, just like in chapter 40, and stop the fall
    player_y = FIX32((ytilecoord_bottom << 4) - PLAYER_COLBOX_BOTTOM);
    player_y_vel = FIX32(0);
    jumping = false;
}
```



```

}
else if (player_topleft_col == SOLID_TILE || player_topright_col == SOLID_TILE)
{
    // Ceiling collision: snap the position to the bottom edge of
    // the solid tile and stop the ascent
    player_y = FIX32(((ytilecoord_top + 1) << 4) - PLAYER_COLBOX_TOP);
    player_y_vel = FIX32(0);
}

```

The tile coordinates (`ytilecoord_bottom`, `ytilecoord_top`) and the collision box offsets (`PLAYER_COLBOX_BOTTOM`, `PLAYER_COLBOX_TOP`) are the same ones already calculated in chapter 40; the only thing that changes is which corners of the box each calculation applies to.

7. A design dilemma: what happens when the character hits its head

With upper collision working, a curious side effect appears: when the character bumps its head against a platform mid-jump, instead of falling immediately it seems to stay “stuck” for a moment against the ceiling before starting to fall. The explanation is simple: when jumping, `player_y_vel` starts at -12, and gravity only adds 0.8 per frame (-12, -11.2, -10.4, and so on); as long as that velocity stays negative, the character keeps “trying” to rise every frame, and the ceiling collision simply blocks it in place until the velocity reaches 0, at which point gravity finally starts pulling it back down.

There’s no single correct answer here, and both behaviors show up in real games on the console. At one extreme, hitting your head doesn’t immediately cut the upward momentum, which is allowed to run its course normally; level design that accounts for this behavior might, for example, avoid placing hazards directly beneath a low platform. At the other extreme, the character falls immediately the moment it hits its head, with no “stuck” moment against the ceiling at all. Achieving this second behavior is as simple as resetting `player_y_vel` to `FIX32(0)` at the same point where jumping already gets reset to false inside the upper collision block, exactly as is already done in the lower collision block upon touching the ground, as shown in the snippet from the previous section. Choosing between one behavior and the other is a design decision, not a matter of technical correctness.

8. A real danger: when velocity exceeds 16 pixels per frame

Since the collision system works with 16×16 pixel tiles and checks positions only once per frame (not continuously throughout the movement), any velocity — horizontal or vertical — that exceeds 16 pixels per frame runs the risk of completely skipping over a single-tile-thick layer without the collision code even detecting it. Raising gravity to a high value can let falling speed exceed 16 within a few frames and pass straight through a platform without stopping on it; likewise, a jumping_vel that’s too high can let the character pass through a platform from below if its upward velocity on the exact frame of impact still exceeds that limit.

With terminal velocity set to a low value, the fall becomes noticeably slow and “floaty” compared to a fall without that limit.

The solution has two parts. The first is simply keeping `jumping_vel` below 16, which guarantees the character never jumps faster than the collision system can detect. The second is introducing a terminal velocity: an upper limit on `player_y_vel`, checked every frame right after gravity is added to it, so that falling speed can never exceed that value no matter how long the character has been falling:

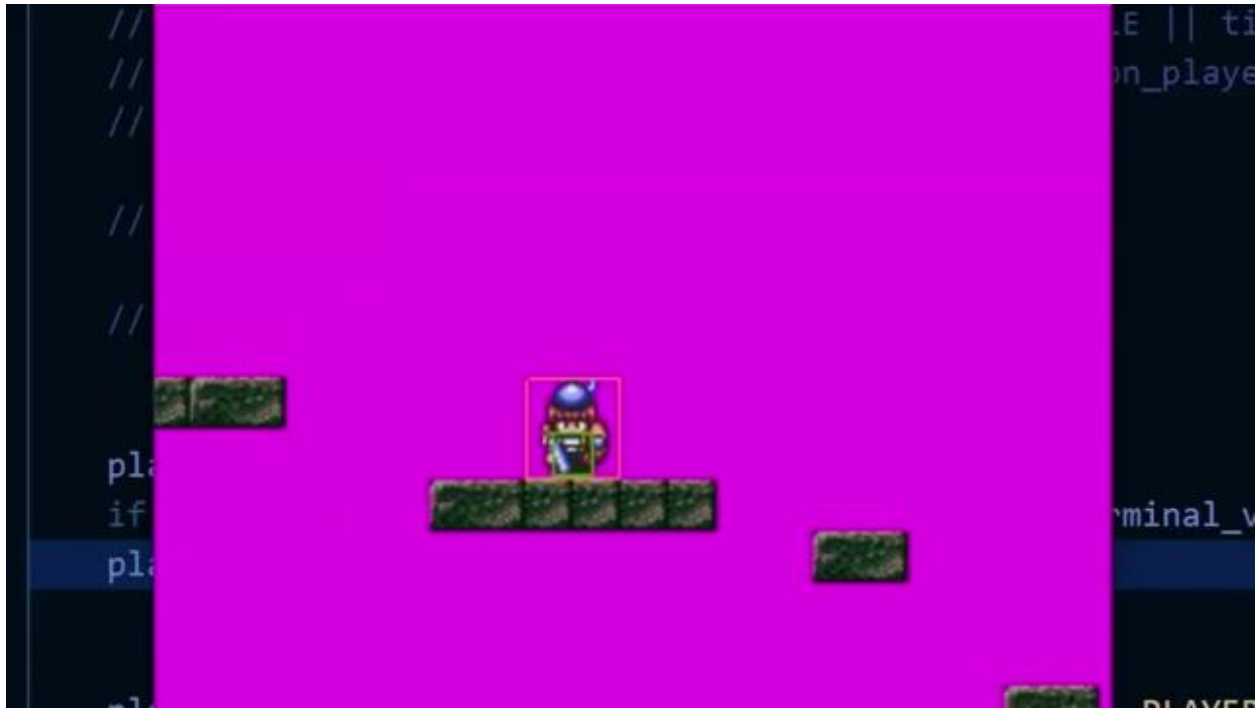


Figure 124: The character falling with terminal velocity set to a low value

```
fix32 terminal_vel = FIX32(8);  
  
// ... inside the physics loop, every frame:  
player_y_vel += gravity;  
if (player_y_vel > terminal_vel)  
{  
    player_y_vel = terminal_vel;  
}  
player_y += player_y_vel;
```

A very low terminal velocity value (say, 4) produces a visibly slow, almost floaty fall, which might suit a gliding power-up; most conventional games would probably prefer a higher value, such as 8, 10, or 12. Since this example’s horizontal movement stays fixed and below 16 pixels per frame, it’s in no danger of passing through tiles on that axis; but any game with acceleration-based horizontal physics would need to apply the same speed-limiting precaution on that axis too.

There’s still an edge case that the system built in this chapter doesn’t fully resolve: because of the exact order in which jumping and the character’s position get updated (between the moment it leaves the ground and the first collision frame already in the air), it would still be possible to jump again without having touched the ground. If the game requires it, the most direct way to close that gap is an additional flag (for example, `already_jumped`) that gets set to true at the very instant of the jump and is only explicitly cleared upon confirming ground contact on the following frame, rather than relying solely on jumping.

Review exercises

1. Explain, in terms of the gravity and `player_y_vel` variables, why the character's falling speed increases progressively instead of staying constant.
2. Why does pressing the jump button subtract a value from `player_y_vel` instead of adding to it?
3. Describe the mechanism (the boolean variable and the two points in the code where its value changes) that prevents the character from jumping more than once without first touching the ground, and propose how to reinforce it with an additional flag to close the edge case noted at the end of the chapter.
4. Explain the difference between letting the upward momentum run its course normally when hitting a ceiling, and making the character fall immediately at that same instant. Which line of code makes the difference between one behavior and the other?
5. Why can no character velocity — jumping, falling, or horizontal movement — exceed 16 pixels per frame in this collision system, and what mechanism was introduced specifically to guarantee that limit during the fall?

Chapter 42 — Level Collision for Larger Sprites

Extending a two-point-per-side system to several intermediate checkpoints

Chapter 41 left us with a fully working system for gravity, jumping, and level collision, but it was tested on a relatively small, compact sprite. 16-bit games, however, often feature protagonists that are considerably larger, and that exposes a problem that had until now stayed hidden: the collision system you've been using only checks two points per side, the top corner and the bottom corner of each edge of the collision box. With a small sprite, those two points are enough to cover the character's entire height. With a much taller sprite, though, a sizable gap is left between those two corners, through which the character can pass straight through the scenery without the game noticing.

This chapter replaces the protagonist with a considerably larger sprite (judging by the name of the source file, it appears to belong once again to *Chakan: The Forever Man*, though there's no way to confirm that from an independent source) and shows how to extend the collision system by adding intermediate checkpoints until the character's entire body is covered.

The new protagonist sprite, considerably taller than the one used in chapter 41, shown in the graphics editor next to its tile grid.

Prerequisites

Before continuing, you should already have worked through:

- Chapter 41, "Jumping and Gravity for a 2D Platformer": this chapter directly reuses and extends that same code (gravity, `player_y_vel`, jumping with `jumping_vel`, terminal velocity).
- The formula for converting pixel coordinates to tile coordinates (the `>> 4` bit shift) and the array index formula (`row × row_length + column`), covered in chapter 40, "Level Collision for a Top-Down 2D Game."
- The concept of terminal velocity introduced in chapter 41, since it comes up again toward the end of this chapter.

1. Importing the new sprite and adjusting its tile size

The first step is to import the new sprite file into the `res` folder and, in `resources.res`, replace the reference to the old sprite with this new file. Since the new sprite has different pixel dimensions, its size in tiles changes too: SGDK always works with 8×8 pixel tiles when importing graphics, even though level collision uses 16×16 tiles, so the two numbers that

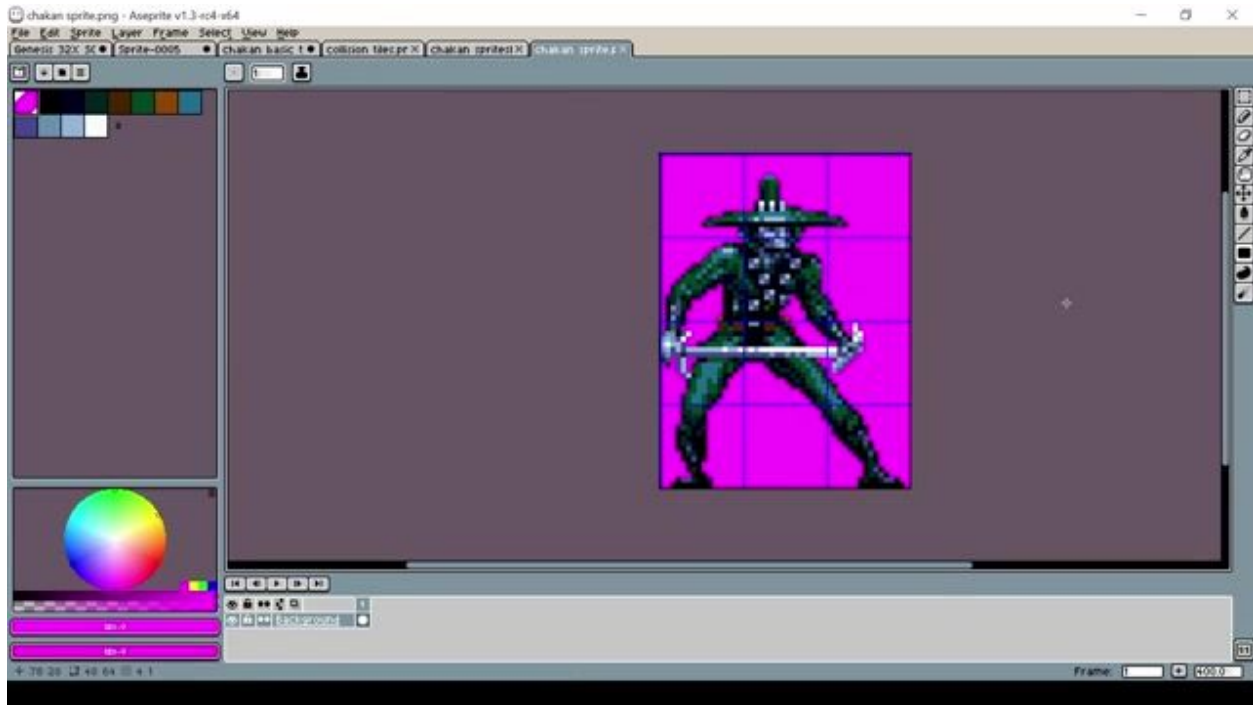


Figure 125: The new protagonist sprite

follow the file name on the SPRITE line in `resources.res` must indicate how many 8×8 tiles make up the width and height of the full sprite. For this particular sprite, that comes out to 6 tiles wide by 8 tiles tall:

```
SPRITE player_sprite "chakan_sprite.png" 6 8 FAST
```

It's an easy detail to overlook, because getting it wrong produces no compile error: the sprite simply gets clipped or tiled incorrectly when it's drawn.

2. Adjusting `PLAYER_WIDTH` and `PLAYER_HEIGHT` in `main.c`

With the resource already updated, `main.c` needs the `PLAYER_WIDTH` and `PLAYER_HEIGHT` constants corrected to reflect the new sprite's actual dimensions, this time measured in pixels (not in 8×8 tiles as in `resources.res`):

```
#define PLAYER_WIDTH 48 // 6 tiles of 8 px
#define PLAYER_HEIGHT 64 // 8 tiles of 8 px
```

These two constants aren't directly involved in the collision calculations, but they are used elsewhere in the code, such as when positioning the sprite on screen, so it's worth getting them right from the start even if their effect isn't immediately visible.

3. Redrawing the collision box: new offsets and a higher jump

The next step, just as in chapter 40, is to go back into the graphics editor and manually draw, on top of the sprite, the rectangle considered to be the character's actual collision box. It's usually somewhat narrower than the full sprite, since the hat, the sword, or other decorative details shouldn't block the character against the scenery. Exactly where to draw that rectangle is a design decision, not a mathematical calculation: it depends on the specific look of each sprite, and there's no general formula for deriving it.

From that rectangle, the four constants `PLAYER_COLBOX_LEFT`, `PLAYER_COLBOX_RIGHT`, `PLAYER_COLBOX_TOP`, and `PLAYER_COLBOX_BOTTOM` are measured, always as pixel offsets measured from the top-left corner of the full sprite (the reference point the Mega Drive uses to position any sprite on screen). For this sprite, for example:

```
#define PLAYER_COLBOX_LEFT    16
#define PLAYER_COLBOX_RIGHT   32
#define PLAYER_COLBOX_TOP     6
#define PLAYER_COLBOX_BOTTOM  60
```

Since this character is considerably taller than the one from chapter 41, it's also worth taking this opportunity to increase the jump speed: `jumping_vel` goes from 12 to 14, giving the character enough momentum to clear the new sprite's extra height. Like the collision box offsets, this value is an empirical adjustment based on the look of this particular sprite, not the result of a general formula.

```
fix32 jumping_vel = FIX32(14);
```

4. First results: jumping works, but the character passes through platforms from the sides

Compiling at this point, the result is partially encouraging: the character can jump and fall normally, and collision against the ground and against the ceiling, that is, the top and bottom corners of the box, works correctly. The problem shows up when moving sideways: the character passes straight through solid platforms from the side, both left and right, as if they weren't there.

The cause is easy to spot by looking at the code inherited from chapter 40: for each side of the collision box, the system only checks two points, the top corner and the bottom corner of that side. With the compact sprite from chapter 41, those two points were enough, because the entire height of the collision box fit within a single 16-pixel collision tile. With this much taller sprite, the collision box spans several tiles in height, and there's a considerable gap between the top corner and the bottom corner that neither point manages to check: if a solid platform happens to sit at exactly that intermediate height, the character can pass through it without the game detecting it.

Jumping and vertical collision already work, but the character still passes through platforms when moving sideways.

5. A first intermediate point: one alone isn't enough

The conceptual solution is to add extra checkpoints between the top and bottom corners of each side. Conveniently, the tile coordinate of the top corner has already been calculated, so



Figure 126: The character standing on one platform while passing through another sideways

the coordinates of the intermediate tiles are obtained simply by adding 1, 2, 3 (and so on, as needed) to that same coordinate, rather than repeating the whole calculation from scratch.

For the right side, a first attempt adds a single intermediate point, two tiles below the top point:

```
s16 ytilecoord_right_mid1 = ytilecoord_top + 2;
```

Testing it, it doesn't work well: that point lands roughly at the character's waist height, too low to block the chest or shoulders against a platform. Adjusting the offset from +2 to +1 tiles, the point ends up just below the hat, and that's enough to properly block the upper part of the body:

```
s16 ytilecoord_right_mid1 = ytilecoord_top + 1;
```

6. Completing coverage for the right side

Even with that adjustment, the problem isn't fully solved: although the upper body no longer passes through the platform, the lower half of the character can still slip through it while jumping, since there's still a stretch of the body without any nearby checkpoint. The solution is to add two more intermediate points, at +2 and +3 tiles below the top point, completing a total of five checkpoints for the right side: top, three intermediate points, and bottom.

```
s16 ytilecoord_right_mid1 = ytilecoord_top + 1;
s16 ytilecoord_right_mid2 = ytilecoord_top + 2;
s16 ytilecoord_right_mid3 = ytilecoord_top + 3;
```

Each of these intermediate points needs, in addition to its own tile coordinate, its corresponding array index and its own collision type lookup: exactly the same pattern already used for

the corners in chapter 40, simply repeated several times.

```
u16 array_index_topright_colbox = (ytilecoord_top *
    ↳ length_of_levelcol_array) + xtilecoord_right;
u16 array_index_rightmid1_colbox = (ytilecoord_right_mid1 *
    ↳ length_of_levelcol_array) + xtilecoord_right;
u16 array_index_rightmid2_colbox = (ytilecoord_right_mid2 *
    ↳ length_of_levelcol_array) + xtilecoord_right;
u16 array_index_rightmid3_colbox = (ytilecoord_right_mid3 *
    ↳ length_of_levelcol_array) + xtilecoord_right;
u16 array_index_bottomright_colbox = (ytilecoord_bottom *
    ↳ length_of_levelcol_array) + xtilecoord_right;

u8 collision_topright = level_col[array_index_topright_colbox];
u8 collision_rightmid1 = level_col[array_index_rightmid1_colbox];
u8 collision_rightmid2 = level_col[array_index_rightmid2_colbox];
u8 collision_rightmid3 = level_col[array_index_rightmid3_colbox];
u8 collision_bottomright = level_col[array_index_bottomright_colbox];
```

And it's also added to the if condition that decides whether the character gets blocked on that side:

```
if (collision_topright == SOLID_TILE ||
    collision_rightmid1 == SOLID_TILE ||
    collision_rightmid2 == SOLID_TILE ||
    collision_rightmid3 == SOLID_TILE ||
    collision_bottomright == SOLID_TILE)
{
    player_x = FIX32((xtilecoord_right << 4) - PLAYER_COLBOX_RIGHT - 1);
}
```

With all five points in place and tested, the result is conclusive: walking across the entire map, there are no longer any gaps through which the character can slip through a platform's right side, no matter what height along the body it's at.

7. A bug found along the way: mid-air jumping

Before repeating the same work for the left side, it's worth fixing a bug that has nothing to do with the sprite's size, but rather with the jump button condition inherited from chapter 41: if the character walks off the edge of a platform and starts falling without having jumped first, it's still possible to press the jump button and get a jump while in mid-air. This happens because the only condition being checked until now was that the jumping variable be false, and jumping never gets set to true when walking off an edge, only when actually jumping.

The fix is to add an additional condition: the character can only jump if, in addition to not already being in a jump, their vertical velocity (player_y_vel) is less than or equal to zero, meaning they aren't falling:

```
if (JOY_readJoypad(JOY_1) & BUTTON_C && !jumping && player_y_vel <= 0)
{
    player_y_vel -= jumping_vel;
    jumping = true;
}
```


8. Repeating the process for the left side

With the right side finished and the mid-air jump bug fixed, the same work is repeated for the left side: three more intermediate checkpoints (at +1, +2, and +3 tiles below the top point), each with its own coordinate, array index, and collision type lookup, added to the existing ones at the top-left and bottom-left corners. It's worth creating entirely new variables for the left side instead of reusing the ones from the right side, even though many of the height values coincide between both sides, simply to keep the code clearer and avoid mixing up one side with the other:

```
s16 ytilecoord_left_mid1 = ytilecoord_top + 1;
s16 ytilecoord_left_mid2 = ytilecoord_top + 2;
s16 ytilecoord_left_mid3 = ytilecoord_top + 3;

u16 array_index_topleft_colbox = (ytilecoord_top * length_of_levelcol_array)
    ↪ + xtilecoord_left;
u16 array_index_leftmid1_colbox = (ytilecoord_left_mid1 *
    ↪ length_of_levelcol_array) + xtilecoord_left;
u16 array_index_leftmid2_colbox = (ytilecoord_left_mid2 *
    ↪ length_of_levelcol_array) + xtilecoord_left;
u16 array_index_leftmid3_colbox = (ytilecoord_left_mid3 *
    ↪ length_of_levelcol_array) + xtilecoord_left;
u16 array_index_bottomleft_colbox = (ytilecoord_bottom *
    ↪ length_of_levelcol_array) + xtilecoord_left;

u8 collision_topleft = level_col[array_index_topleft_colbox];
u8 collision_leftmid1 = level_col[array_index_leftmid1_colbox];
u8 collision_leftmid2 = level_col[array_index_leftmid2_colbox];
u8 collision_leftmid3 = level_col[array_index_leftmid3_colbox];
u8 collision_bottomleft = level_col[array_index_bottomleft_colbox];
```

With so many conditions piling up (five possible checkpoints per side, each capable of blocking movement), the if line becomes long and hard to read if written all on one line. It's perfectly valid to reformat that if so that each condition joined by an || ("logical or") occupies its own line, as long as the opening brace stays at the end and the closing braces line up correctly; this doesn't change the code's behavior at all, but it greatly improves readability:

```
if (collision_topleft == SOLID_TILE ||
    collision_leftmid1 == SOLID_TILE ||
    collision_leftmid2 == SOLID_TILE ||
    collision_leftmid3 == SOLID_TILE ||
    collision_bottomleft == SOLID_TILE)
{
    player_x = FIX32(((xtilecoord_left + 1) << 4) - PLAYER_COLBOX_LEFT);
}
```

If you prefer, you can store the three intermediate offsets in a small array and loop over them instead of declaring each variable separately; the result would be equivalent, though with five points per side the explicit version is still reasonably readable.

9. Final result: a robustness test

With both sides now covered by five checkpoints each, the character can freely roam the test map, jump between platforms of different heights and sizes, and collide with the scenery from

any angle without getting stuck or passing through anything. It's normal for some of the more demanding jumps in a map originally designed for a smaller character to become difficult to pull off with such a large sprite; that's a level design issue, not a flaw in the collision code itself.

It's always worth testing this kind of system by having the character move through narrow spaces, since that's where detection gaps would most easily show up if any checkpoint were missing.



Figure 127: The character jumping between platforms of different heights

With the five checkpoints per side now in place, the character jumps between platforms of different heights without passing through any of them.

10. What if 8×8 collision tiles were used instead of 16×16?

A common question at this point is why not just use a finer collision grid, 8×8 pixels instead of 16×16, since that would eliminate the problem of gaps between checkpoints at the root. It's perfectly possible (you'd just need to adapt the pixel-to-tile conversion formulas and all related calculations), but it brings some downsides that explain why 16×16 is the more common convention.

On one hand, since there would be twice as many tiles both horizontally and vertically, the system has to perform many more collision checks per frame, which represents an extra load on the CPU, especially noticeable if there's a second player or several enemies that also need to resolve their own collision. The material doesn't quantify that overhead with any concrete figure, so it should be taken as a qualitative warning rather than as a reference number.

There's also a memory cost that can be calculated precisely: an 8×8 grid doubles both the width and height of the collision array in tiles. Since the area of a grid is the product of its two dimensions, doubling both at the same time quadruples the total number of tiles in the array, and with it the space that array takes up in ROM: a map that requires, say, 2232 elements

with 16×16 tiles would need around 8928 with 8×8 tiles. On large levels, that wasted space becomes significant.

And then there's the terminal velocity introduced in chapter 41: with 8×8 tiles, the safe speed limit per frame to avoid passing through a tile undetected is cut in half, that is, to 8 pixels per frame instead of 16. That's a considerably stricter constraint on how fast any object in the game can move, and it has to be taken into account for `terminal_vel`, for `jumping_vel`, and for any horizontal speed.

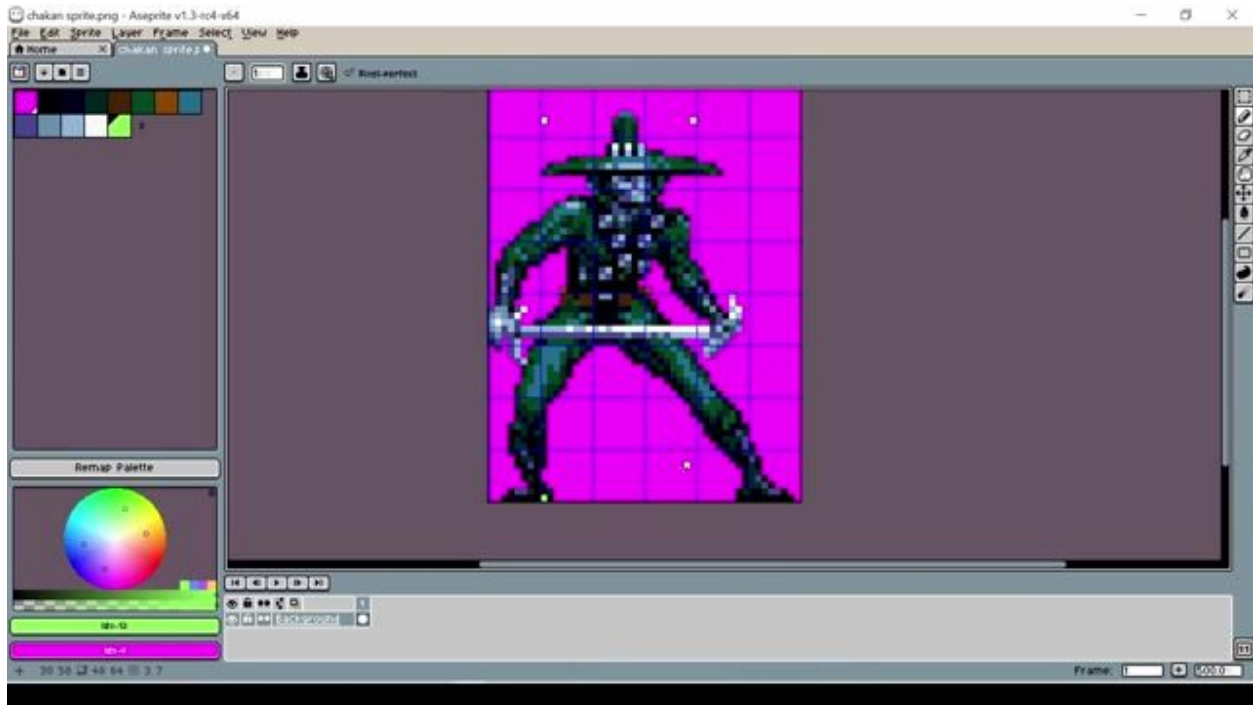


Figure 128: An 8×8 grid overlaid on the sprite

A reference 8×8 pixel grid overlaid on the sprite: the granularity that collision of that size would imply, much finer than the 16×16 used so far.

Review exercises

1. Explain why a collision system that only checks the top and bottom corners of each side stops being sufficient when the character's sprite is much taller than a collision tile.
2. Why are the intermediate checkpoints calculated by adding 1, 2, or 3 to the tile coordinate of the top corner, rather than being calculated independently from the character's position?
3. Describe the mid-air jump bug found in this chapter and the additional condition that fixes it.
4. Name the three downsides pointed out in this chapter regarding the use of an 8×8 pixel collision grid instead of a 16×16 one, indicating which of them is a verifiable mathematical relationship and which are qualitative statements.
5. If a map has a collision array of 2232 elements with 16×16 tiles, how many elements would the equivalent array have if 8×8 tiles were used instead, and why is that relationship a factor of four rather than a factor of two?

Chapter 43 — Semi-Solid Platforms

Almost every platformer from the 16-bit era includes a very particular kind of tile: the semi-solid platform. Unlike a fully solid floor or wall tile, this type of platform can be freely passed through by jumping up through it and moving past its sides, but it stops the character the moment it falls onto it from above, as if it were solid.

This chapter adds that behavior to the collision system built in the previous chapters of this part, and along the way several design nuances and several interesting bugs come up, typical of this kind of mechanic.

Prerequisites

Before continuing, it helps to already have in place:

- Level collision for larger sprites (chapter 42), since this chapter reuses and directly extends that same code.
- The distinction between `handle_input()` (continuous controls, like the D-pad) and `JoyEvent()` (single button presses), covered in the chapter on button controls.
- The concept of terminal velocity and gravity-driven jumping (chapter 41), since this chapter revisits that topic with a new twist.

1. The new semi-solid platform tile in 16Tile

The first step is purely level-design work: a new element is added to the map in `16Tile`, specifically meant to serve as a test semi-solid platform. In the collision layer, the tile corresponding to that column is painted neither black (free) nor red (solid), but in a third color, green, reserved specifically to represent this new collision type.

Just as with solid tiles, there's no need to paint the whole column green: most of it is left passable (black), and only the top row — the edge where the character is meant to get caught when falling — is painted green.

The collision layer with the third color already added: black for free tiles, red for solid ones, and now green for the semi-solid platform.

2. Compiling with no code changes

With the map and the collision array already updated and imported into SGDK, the `LEVEL_COL` array now contains ones, zeros, and also a new value, 2, in the positions corresponding to the

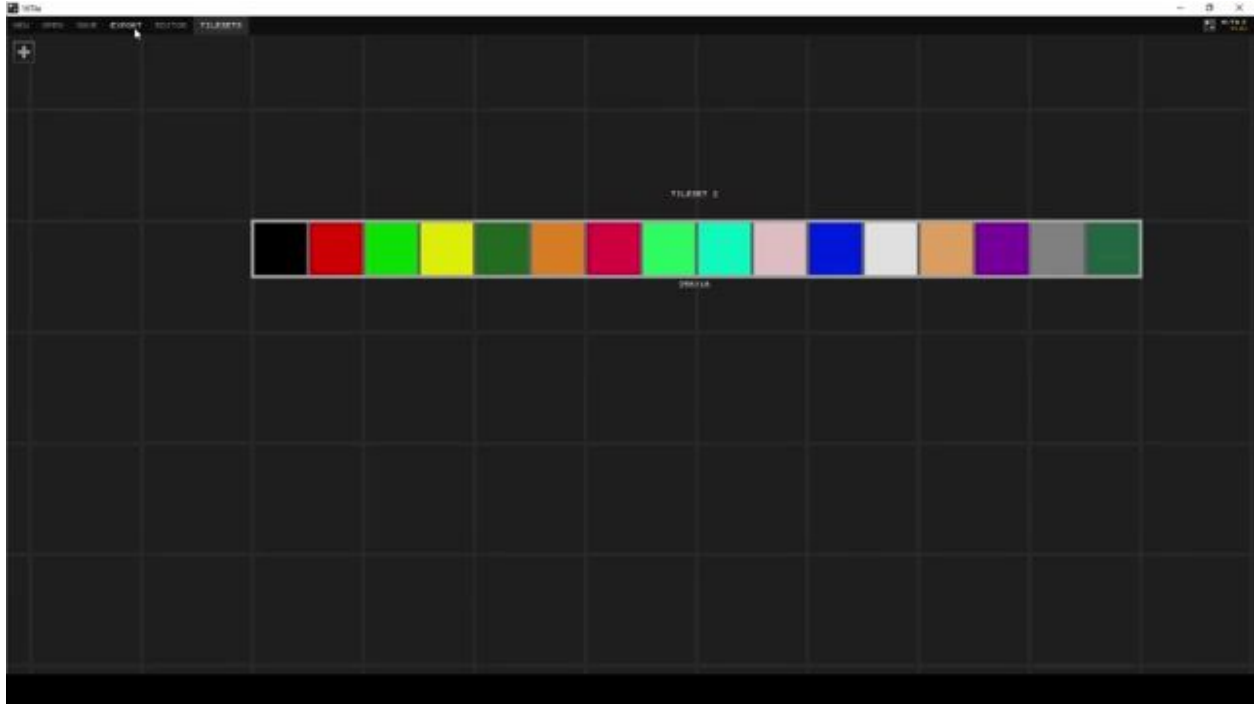


Figure 129: Color palette of the collision layer in 16Tile

semi-solid column. However, since there's still no code in `collision()` that knows what to do with the value 2, the result when compiling is that the character passes through the new column exactly as it would pass through any free tile: the data is already in the array, but the game's behavior doesn't take it into account yet.

3. Creating PLATFORM_TILE and reusing the floor collision block

The first code step is a new constant:

```
#define SOLID_TILE 1
#define PLATFORM_TILE 2
```

following exactly the same pattern as `SOLID_TILE`. With the constant declared, the fastest way to get started is to copy the collision block that already exists for when the character's feet touch a solid tile, paste it right below, and replace every mention of `SOLID_TILE` in that copy with `PLATFORM_TILE`.

4. The problem: the character gets stuck above the platform when jumping through it

Compiling with this first copied block, collision does appear on the semi-solid platform, but in an incorrect way: instead of letting the character pass freely upward, the jump gets blocked as soon as the head touches the bottom edge of the platform, as if it were just another solid tile.

The reason is simple: the copied code block doesn't distinguish which direction the character is moving relative to the platform, so it stops them both when going up and when going down alike — exactly the opposite of what's expected from a semi-solid platform.

5. The conceptual solution: comparing the current position with the previous frame's

One possible solution would be to simply check whether the character's vertical velocity is positive or negative, similar to how the mid-air jump bug was solved in chapter 41. However, checking only the sign of the vertical velocity, instead of comparing against the previous frame's position, can end up causing bugs in certain particular circumstances with this type of platform. That's why a different approach is preferable: storing, on every frame, the character's Y position from the previous frame, before gravity and the rest of that frame's vertical movement are applied.

By comparing that previous position with the current one, it's possible to know for certain whether the character is moving downward (falling) or upward (jumping): since the Mega Drive's Y axis grows downward, if the current position is numerically greater than the previous one, the character is moving downward.

```
fix32 previous_player_y;
```

This variable is declared at the beginning of `collision()` and is updated to the value of `player_y` before gravity and the rest of that frame's vertical movement are applied. With it declared, the `PLATFORM_TILE` block's condition is expanded: besides requiring that the bottom-left or bottom-right tile be of platform type, it now also requires that `player_y` be greater than `previous_player_y`, meaning the character must actually be falling on that frame and not rising:

```
if ((tile_left == PLATFORM_TILE || tile_right == PLATFORM_TILE)
    && player_y > previous_player_y)
{
    // collision against the semi-solid platform
}
```

With this change, the result is now as expected: the character can jump and pass through the semi-solid platform freely upward, and only gets caught on top of it when falling.

6. A design nuance: how far should the character sink before getting caught?

With the basic mechanic working, a detail comes up that isn't exactly a bug, but rather a design decision: as the code stands at this point, as soon as the character's feet touch any part of the platform — even its lowest edge — the character is immediately pushed up to the platform's surface. For a fairly thick collision tile, like the one in this example, that behavior can feel unnatural: it's probably better for the character to only get caught if their feet reach a reasonably close distance to the platform's top edge, and if they fall much further below that, they should simply keep falling through it normally.

This is also a good time to reorganize the code: since in a given frame the character can only be colliding with the floor, with the ceiling, or with a platform (never two at once), the standalone `if` block for `PLATFORM_TILE` becomes another `else if` in the same chain already used for `SOLID_TILE`, saving unnecessary CPU checks.

7. Implementing the pixel margin before getting caught

To be able to compare the character’s position against the platform’s top edge in whole-pixel units, an additional variable is needed to convert `player_y` (of type `fix32`) into a plain integer, since `blocked_coord` — the coordinate of the platform’s top edge, calculated earlier — is an integer value:

```
u16 player_y_int = F32_toInt(player_y);
```

With that variable ready, the whole platform collision block is wrapped inside a new `if`, so that it only runs when `player_y_int` is less than `blocked_coord` plus a pixel tolerance margin, meaning the character’s feet haven’t yet fallen past that margin relative to the top edge:

```
else if ((tile_left == PLATFORM_TILE || tile_right == PLATFORM_TILE)
        && player_y > previous_player_y
        && player_y_int < (blocked_coord + 3))
{
    // collision against the semi-solid platform
}
```

With a tolerance margin of a few pixels like this one, the character only gets caught if their feet land within that distance from the platform’s top edge; if they fall further below that, they simply keep falling through it. This value — like others in this chapter, such as the timer limit that appears later on or the adjustments to `jumping_vel` — is empirical: it’s best tested in the actual game and tuned according to the desired feel of the jump, rather than derived from a fixed formula. A margin that’s too wide (say, 8 pixels) can make the character get caught sooner than expected; one that’s too narrow may require increasing `jumping_vel` so the character can more easily reach that narrow band when jumping from below.

8. A serious bug: passing through platforms at high speed

Playing around with the result a bit more reveals a much more serious bug, directly related to the terminal velocity introduced in chapter 41. Since the platform now only catches the character within a narrow pixel margin instead of across the tile’s full 16 pixels, the safe per-frame speed margin becomes much stricter: if the character falls from high enough and their vertical velocity on the exact frame of impact exceeds that margin — which happens easily, since gravity builds up speed quickly frame by frame — their feet can land already below the margin in a single frame, without the collision code ever detecting it. The result is that the character passes straight through the platform as if it weren’t there.

One possible solution would be to simply lower the terminal velocity below that margin, but that option isn’t recommended: with such a low terminal velocity, the character’s fall becomes noticeably slow and floaty, an undesirable effect for this type of game. Instead, it’s better to add an exception to the existing condition.

This problem is, in essence, the same kind of “high-speed tunneling” that already came up when solving axis-by-axis level collision in chapter 40: when a frame’s displacement is greater than the detection margin, the object can skip clean over the contact zone.

9. The exception: letting a character that was already falling from above fall normally

The exception consists of adding, with an `||` (logical “or”), a second way to satisfy the condition: if the character’s Y position in the previous frame (`previous_player_y`) was already above the platform’s top edge (`blocked_coord`), then the collision is allowed to happen anyway, no matter how many pixels the character sank on this frame. The logic is that if the character was above the platform in the previous frame, it’s because they were genuinely falling onto it and not passing through it from below, so it’s appropriate to catch them anyway, even if the falling speed made them skip past the narrow pixel margin in a single frame.

Since `previous_player_y` is of type `fix32` while `blocked_coord` is a plain integer, they can’t be compared directly without first converting one of the two. It’s possible to write the `F32_toInt()` conversion directly inside the `if` itself, but it’s better to create a separate variable for the already-converted value: writing the conversion inline, inside the condition, tends to produce bugs that are hard to track down later.

```
u16 previous_player_y_int = F32_toInt(previous_player_y);
```

10. Replacing the magic number with a descriptively named variable

As a final touch to this part, the pixel margin used so far as a loose number in the code becomes a variable with a long but descriptive name, so that anyone reading the code months later immediately understands what that value represents without having to infer it from context:

```
u16 how_far_below_platform_before_pushup = 3;
```

In general, it’s good practice to avoid so-called “magic numbers” scattered through the code, for exactly this reason. With the variable now declared, the full condition reads as follows:

```
else if ((tile_left == PLATFORM_TILE || tile_right == PLATFORM_TILE)
        && player_y > previous_player_y
        && (player_y_int < (blocked_coord + how_far_below_platform_before_pushup)
            || previous_player_y_int < blocked_coord))
{
    // collision against the semi-solid platform
}
```

As with the rest of this chapter’s empirical values, the number chosen for `how_far_below_platform_before_pushup` should be tuned by testing the running game, rather than treated as final from the start.

11. Passing through the platform at will: holding down and jumping

As a last functional addition, it’s common in games with semi-solid platforms to let the player hold down the D-pad’s down direction and press the jump button to deliberately drop through the platform. The implementation combines the two control functions already used in previous chapters: inside `handle_input()`, a boolean variable, `dpad_down`, is created, which stays true while the player holds down the down direction, following the same pattern already used for the left and right directions; and inside `JoyEvent()`, the jump button’s condition is

expanded to also check whether `dpad_down` is active at that moment, and if so, instead of jumping, it activates a new boolean variable, `fall_through_platform`.

With `fall_through_platform` now declared, the condition of the platform's collision block is expanded once more, also requiring that variable to be false (with `!fall_through_platform`) for the collision to take effect. This way, while the player holds down and presses the jump button, collision against the semi-solid platform is completely disabled, letting the character fall freely through it.

12. One last bug: getting stuck passing through every platform

Testing this new control immediately reveals a problem: once `fall_through_platform` becomes true, there's no point in the code that ever sets it back to false, so the character ends up passing through absolutely every semi-solid platform on the map forever, instead of just the one it was standing on when the button was pressed.

The solution is a timer: two new variables, declared alongside the rest of the character's physics variables (`gravity`, `jumping_vel`, `terminal_vel`):

```
bool dpad_down = FALSE;
bool fall_through_platform = FALSE;
u8 fall_through_platform_timer = 0;
u8 max_fall_through_platform_time = 4;
```

`fall_through_platform_timer` is a counter that starts at 0, and `max_fall_through_platform_time` is the limit value it must count up to before collision is reactivated. This limit can be reasoned out from gravity itself: since falling speed builds up quickly frame by frame (0.8, 1.6, 2.4, and so on), the character passes completely through the platform edge's pixel margin in very few frames, so a long timer isn't necessary; a value of 4 frames turns out to be enough in practice, although — like the other values in this chapter — it's worth tuning experimentally based on the game's actual falling speed.

Inside `collision()`, while `fall_through_platform` is true, the counter is incremented every frame, and as soon as it reaches its maximum value, both the counter and `fall_through_platform` itself are reset (the latter to false), reactivating normal collision against the next platform the character encounters:

```
if (fall_through_platform)
{
    fall_through_platform_timer++;
    if (fall_through_platform_timer >= max_fall_through_platform_time)
    {
        fall_through_platform_timer = 0;
        fall_through_platform = FALSE;
    }
}
```

With the timer in place, the final result works as expected: holding down and jumping, the character falls through the platform beneath them, but gets caught normally again on the next semi-solid platform it encounters on its way down, without carrying the “falling through” state any further than necessary.

Review exercises

1. Explain why comparing `player_y` against `previous_player_y` makes it possible to know whether the character is falling or jumping, and why that approach is preferable to checking the sign of `player_y_vel` directly.
2. Why can adding a pixel tolerance margin before catching the character on the platform cause collision to fail at high speed, and how does this relate to the terminal velocity introduced in chapter 41?
3. Describe the exception added to the platform's collision condition to fix the high-speed pass-through bug, and explain why a separate `previous_player_y_int` variable was needed.
4. Explain the mechanism (variables and functions involved) that allows passing through a semi-solid platform by holding down and pressing the jump button.
5. Why was it necessary to add a timer (`fall_through_platform_timer`) instead of simply turning `fall_through_platform` on and off immediately?

This closes Part VII: over the course of these four chapters, an axis-by-axis collision system was built piece by piece, capable of handling floors, jumping, large sprites, and semi-solid platforms. Part VIII shifts to a different theme: instead of game physics, the focus moves to how to move data efficiently within the Mega Drive — DMA, VRAM limits, and performance — issues that become increasingly relevant as a project grows in size and in the number of sprites and animated tiles on screen.

Chapter 44 — DMA - Direct Memory Transfers

The VDP's DMA controller, SGDK's `dma.c/dma.h` unit, and the transfer queue as an optimization technique

The `dma` parameter has already shown up several times in earlier chapters without much attention being paid to it: in `PAL_setPalette(PAL0, bg1.palette->data, DMA)` and in `VDP_drawImageEx(BG_B, &bg1, ..., TRUE)`, where it was explained in passing that DMA stands for *Direct Memory Access* and that it's a copy mechanism that, unlike copying with the CPU, is cheaper in terms of processor time. That explanation was enough to keep moving forward with backgrounds and sprites, but it leaves out questions that become unavoidable as soon as a game grows: why is DMA faster than a CPU copy if, at some point, the same bytes still have to be moved? Can DMA be controlled more finely than with a simple TRUE/FALSE? What happens if, in a single frame, more VRAM needs to be updated than DMA can move?

This chapter opens Part VIII of the manual, dedicated to VRAM optimization and performance, and answers those questions in detail. There isn't a flashy new function to learn from scratch here — the underlying mechanism has already been used, unknowingly, since chapter 7 — but rather a deeper understanding of something that was already being used only halfway, plus access to an SGDK unit, `dma.h`, that until now has stayed completely hidden behind the `dma/DMA` parameter of other functions.

Prerequisites

Before continuing, it helps to have:

- An understanding of the VDP, VRAM, CRAM, and VSRAM as presented in chapter 2.
- Some prior experience, even without going deep, using the `dma/DMA` parameter of functions like `PAL_setPalette()` or `VDP_drawImageEx()`, covered in chapter 7.
- A general sense of how SGDK organizes its resources in ROM (chapter 39), since the examples in this chapter load tilesets generated from an `IMAGE` resource.

1. What DMA does at the hardware level

The Mega Drive's CPU, a Motorola 68000, can write data to the VDP's VRAM, but it can only do so one word (16 bits) at a time, and each of those writes is one more instruction within the program: it involves decoding the instruction, calculating the destination address, and executing the write, all of which consumes 68000 clock cycles. Copying an entire tileset of several thousand bytes this way, word by word, in a software loop, is a slow operation that keeps the processor busy the whole time.

The VDP solves this with its own DMA controller: a dedicated circuit, separate from the 68000's logic, capable of taking control of the system bus and moving an entire block of data from memory (the cartridge ROM or the 68000's working RAM) directly into VRAM, CRAM, or VSRAM, without the 68000 having to execute one instruction per word transferred. The programmer simply tells the VDP where to copy from, where to copy to, and how much, and the VDP itself takes care of the rest.

It's worth being precise about what this does and doesn't mean. It's tempting to imagine DMA as something that happens "in parallel," leaving the CPU free to keep calculating game logic while the data travels to VRAM on its own. That's not exactly how it works: during a DMA transfer, the DMA controller takes the system bus for itself, and the 68000 is effectively halted — unable to access memory or keep executing — until the transfer finishes. The gain from DMA isn't that the CPU "does something else in the meantime," but rather two distinct, more concrete gains:

- The data movement itself is much faster than the equivalent loop written in C or assembly, because it's done by a circuit dedicated to that single task, without the cost of decoding instructions.
- That pause time can be deliberately placed at a moment when the CPU had nothing better to do: the *vertical blank* period (vblank), the instant between when the VDP finishes drawing one frame and starts the next, during which there's nothing on screen to update, and much of the time games are already waiting for that moment anyway with `SYS_doVBlankProcess()`.

It's precisely that second point that explains why, in chapter 7, it was said that DMA "doesn't take up CPU time": in practice, when the transfer is triggered during vblank, it isn't taking useful time away from any game calculation, because that time was already earmarked for waiting. The nuance this chapter adds is that this claim depends on when the transfer happens, and it isn't free if DMA is triggered in the middle of game logic, during active screen drawing.

2. Beyond the boolean: TransferMethod

The functions seen so far, like `VDP_drawImageEx()`, expose DMA in the simplest possible way: a `bool dma` parameter that's either `TRUE` (use DMA) or `FALSE` (copy with the CPU). But the actual mechanism behind it, which other SGDK functions expose directly, is a richer enumerated type:

```
typedef enum
{
    CPU = 0,
    DMA = 1,
    DMA_QUEUE = 2,
    DMA_QUEUE_COPY = 3
} TransferMethod;
```

`PAL_setPalette()`, in fact, was already receiving this type back in chapter 7 (`PAL_setPalette(PAL0, bg1.palette->data, DMA)`), although at that point only two of its four possible values were used:

- **CPU** — copies the data with the CPU itself, word by word. It's the slowest option, but it doesn't require the source data to remain intact after the call: the copy is already finished by the time the function returns.
- **DMA** — triggers a DMA transfer immediately, in a blocking way: the function doesn't return until the VDP has finished moving the data. This is the option used so far in this

manual.

- **DMA_QUEUE** — instead of transferring right away, queues the operation to run automatically during the next vblank.
- **DMA_QUEUE_COPY** — a variant of the previous one meant for data that isn't stable in memory (for example, a temporary buffer or a local variable): before queuing, it copies the data into an internal area reserved by SGDK, so that the original content can be modified or discarded immediately after the call without affecting the actual transfer, which will only happen at the next vblank.

These last two values are the gateway to the real subject of this chapter: the DMA queue.

3. The dma.h unit and its low-level functions

Everything above — the bool `dma` of `VDP_drawImageEx()`, the `TransferMethod` of `PAL_setPalette()` — is, underneath, a convenience layer over an SGDK unit dedicated specifically to DMA: `dma.h`. This unit exposes functions to trigger transfers explicitly, without going through a higher-level drawing function, which is useful when you need to move data that isn't necessarily a whole image (for example, updating just part of a tileset, or writing data generated at runtime).

The most direct function is `DMA_doDma()`, which triggers an immediate, blocking transfer — the explicit equivalent of using DMA as the `TransferMethod`:

```
void DMA_doDma(u8 location, void *from, u16 to, u16 len, s16 step);
```

- **location** — the destination of the transfer within the VDP: `DMA_VRAM`, `DMA_CRAM`, or `DMA_VSRAM`.
- **from** — a pointer to the source data (in ROM or in the 68000's RAM).
- **to** — the destination address within VRAM, CRAM, or VSRAM, depending on `location`.
- **len** — the amount of data to transfer, measured in 16-bit words (not bytes).
- **step** — how much the destination address advances after each word written; for a normal, contiguous copy the usual value is 2.

There's also `DMA_transfer()`, which adds a `TransferMethod` parameter at the front and delegates to CPU, DMA, `DMA_QUEUE`, or `DMA_QUEUE_COPY` as appropriate, acting as the single entry point used internally by functions like `PAL_setPalette()`:

```
bool DMA_transfer(TransferMethod tm, u8 location, void *from, u16 to, u16 len, u16  
↪ step);
```

To upload, for example, the tileset from an `IMAGE` resource to VRAM without going through `VDP_drawImageEx()`, the explicit call would be:

```
u16 ind = TILE_USERINDEX;  
  
// Immediate, blocking transfer of level1's tiles  
// into VRAM, starting at tile index 'ind'.  
DMA_doDma(DMA_VRAM, level1.tileset->tiles, ind * 32,  
          level1.tileset->numTile * 16, 2);
```

The calculation `ind * 32` converts a tile index into a byte address in VRAM (each tile takes up 32 bytes), and `numTile * 16` converts the tile count into the number of 16-bit words that need to be transferred (32 bytes per tile equal 16 words). These are the same calculations `VDP_drawImageEx()` performs internally when called with `dma = TRUE`; the difference is that here they're visible and under the programmer's direct control.

4. The DMA queue: queuing instead of forcing

Triggering an immediate transfer like the one in the previous section has a practical limitation: if a single frame needs to upload several tilesets, several palettes, and maybe also update VSRAM for a scroll effect, each `DMA_doDma()` call blocks execution until it finishes, one after another, with no centralized control over how much total time is being used or over exactly when each transfer happens within the frame.

SGDK solves this with a queuing system. Instead of triggering the transfer at the moment it's requested, it gets recorded in an internal list:

```
bool DMA_queueDma(u8 location, void *from, u16 to, u16 len, u16 step);
```

The parameters are the same as those of `DMA_doDma()`, with one fundamental difference: `DMA_queueDma()` doesn't move any data at the moment it's called. It just stores the operation (destination, source, length, step) in an internal queue. That queue is processed automatically, as a batch, during the next vblank — the same period during which `SYS_doVBlankProcess()` runs — at which point SGDK's engine triggers, one after another, all the transfers that have been queued since the previous vblank.

This has an important consequence for the source data: since the actual transfer happens later, the `from` pointer passed to `DMA_queueDma()` needs to keep pointing to valid, unmodified data until the vblank actually processes it. For a tileset living in ROM this isn't a problem, since its content never changes. But if the source is a temporary buffer, a local variable, or any data the program is going to modify or discard before the next vblank, you need the variant that copies the data immediately into an internal area reserved by SGDK:

```
bool DMA_copyAndQueueDma(u8 location, void *from, u16 to, u16 len, u16 step);
```

`DMA_copyAndQueueDma()` copies the data pointed to by `from` into an SGDK-owned buffer at the moment it's called (a CPU copy, fast since it's usually a small block) and queues the actual transfer from that internal buffer, not from the original `from`. It's the same idea behind `DMA_QUEUE_COPY` as a `TransferMethod`, and it's the option to pick whenever the source data isn't a constant resource in ROM.

That internal copy buffer has a limited size, configurable with:

```
void DMA_setBufferSize(u16 value);  
u16 DMA_getBufferSize(void);  
void DMA_setBufferSizeToDefault(void);
```

And the queue itself — the number of distinct operations it can hold, not the number of bytes — also has a configurable size:

```
void DMA_setMaxQueueSize(u16 value);  
u16 DMA_getMaxQueueSize(void);  
void DMA_setMaxQueueSizeToDefault(void);
```

Sizing both values is, essentially, a trade-off decision between available RAM and the amount/size of transfers a game needs to group into a single frame: a game that only updates palettes and the occasional stray tile per frame can get along perfectly well with the default values; a game that loads entire tilesets at once when changing levels may need a bigger queue, or may need to resort directly to a copy buffer with more room if much of that data is assembled at runtime rather than already coming packaged in ROM.

5. The real limits of the VDP's DMA

DMA isn't magic: it's still subject to the limitations of real VDP hardware, and understanding them avoids surprises when deciding how much VRAM to update at once.

The source data can't be just any memory address. The VDP's DMA controller can only read from the cartridge ROM or from the 68000's working RAM; it can't, for instance, read directly from the Z80's RAM (the sound processor) or from any other memory region outside that map. On top of that, real Mega Drive hardware has a well-known quirk: a DMA transfer can't cross a 128 KB boundary within that source memory without the address “wrapping around” incorrectly within that same 128 KB block. It's a real limitation of the chip, not of SGDK, and SGDK handles it transparently: when a queued or immediate transfer would cross one of those boundaries, `DMA_queueDma()` and `DMA_doDma()` automatically split it into two consecutive operations, one on each side of the boundary, without the code calling these functions having to worry about detecting it.

The vblank time available for DMA is finite. The VDP can only move a limited amount of data during a single vblank period — on the order of a few thousand bytes per frame, with a somewhat higher figure on PAL systems than on NTSC due to a longer vblank — and SGDK exposes that figure as a configurable limit:

```
u16 DMA_getMaxTransferSize(void);
void DMA_setMaxTransferSize(u16 value);
void DMA_setMaxTransferSizeToDefault(void);
```

When the sum of all the transfers queued for a given frame exceeds that limit, not all of them get processed in the immediate vblank: depending on how the behavior for that overflow is configured (`DMA_getIgnoreOverCapacity()` / `DMA_setIgnoreOverCapacity()`), the excess is either ignored or left pending. Either way, SGDK's queue **doesn't automatically spread the leftover work across the following frames**: it's the game code's own responsibility not to queue, all at once, more than a single vblank can process, and to deliberately split a large load across several frames when needed — for example, uploading one portion of a new level's tileset per frame over a handful of frames, instead of all at once.

During active drawing, DMA is even slower still. Outside of vblank, while the VDP is busy reading VRAM to draw the screen, it can only spare DMA the memory access slots left over between one video read and the next, which are very few. The practical result is that a transfer triggered in the middle of active drawing crawls along at a fraction of the speed it would have during vblank, on top of risking pausing the 68000 at a moment when it's probably in the middle of some time-sensitive calculation, such as a frame's physics. That's why the general recommendation — and the reason the DMA queue exists — is to concentrate large transfers into vblank as much as possible, rather than firing them off individually during game logic.

Thought of as a timeline, a video frame is split into an active-drawing period, fairly long, followed by a vblank period, much shorter but with all of its capacity available for DMA. When the amount of data to move exceeds what fits into that vblank, the leftover portion isn't transferred in that frame: it's left pending for the next one, unless the game code itself takes care of splitting it up beforehand.

6. Why queuing is an optimization technique

With the pieces above in place, it's now possible to answer the question that motivated this chapter: why queue DMA transfers instead of forcing them all immediately?

The short answer is that queuing moves control over *when* each transfer happens from the

point in the code where *what* to transfer is decided to a single place, vblank processing, where SGDK can take the frame's real capacity limit into account. If a game needs, at the same instant of game logic, to update a new enemy's palette, upload an animated tile, and refresh a row of a scrolling map, forcing three immediate transfers with `DMA_doDma()` one after another blocks execution three times in a row, with no guarantee that the whole set will comfortably fit into the next vblank. Queuing all three with `DMA_queueDma()` (or with `DMA_QUEUE/DMA_QUEUE_COPY` in the higher-level functions that support it) lets the game code keep running without pauses until vblank, at which point all three are processed together, respecting the frame's capacity limit.

This same mechanism becomes even more relevant when the amount of data to move isn't small, as happens when loading the entire graphics set for a new level at once: several tile-sets, several palettes, and potentially several tile maps, all ready for the first frame in which the level becomes visible. It's easy for that amount of data to exceed what a single vblank can process. The DMA queue doesn't solve that problem automatically — as noted in the previous section, it doesn't spread the excess across future frames on its own — but it does provide the tools (`DMA_canQueue()` to find out whether an operation fits within the frame's remaining budget, `DMA_getQueueSize()/DMA_getQueueTransferSize()` to inspect how much is already queued) on which an explicit strategy of loading in stages across several frames can be built. That concrete strategy — how to spread out the loading of a new level across several frames without it showing up as a noticeable pause — is picked up in detail later in this manual, in the chapter dedicated to loading graphics for a new level.

Review exercises

1. Explain why saying that DMA “doesn't take up CPU time” is a simplification, and what actually makes a DMA transfer more efficient than a copy done with 68000 instructions.
2. What's the practical difference between `DMA_QUEUE` and `DMA_QUEUE_COPY` as `Transfer-Method` values? Given a buffer with data computed at runtime that's going to be overwritten as soon as the current function ends, which of the two should be used, and why?
3. A teammate triggers, in the middle of the main loop (not in vblank), several consecutive calls to `DMA_doDma()` to upload the graphics for a new enemy. The game starts showing stutters right at that moment. Explain why, and what change you would propose.
4. Why doesn't the existence of the 128 KB limit on DMA source addresses tend to be a problem for someone programming with SGDK, even though it's a real limitation of the Mega Drive's hardware?
5. A new level needs to upload far more data to VRAM than fits within the capacity of a single vblank. Explain why simply queuing it all at once with `DMA_queueDma()` isn't enough to solve the problem on its own, and what tool SGDK provides to find out, before queuing, whether an operation still fits within the current frame's budget.

Chapter 45 — Managing VRAM Limits

How VRAM is divided between background and sprites, how to spot an allocation conflict, and how to adjust that split with `SPR_initEx()`

Chapter 44 showed how DMA moves data into VRAM without stopping the CPU. But before asking *how* a piece of data gets into VRAM, it's worth asking something more basic: *whether there's room* for it. Chapter 2 already established that VRAM has 64 KB in total, and that, in theory, this is enough for 2048 8×8-pixel tiles if all of that memory were dedicated exclusively to storing them. In practice, that theoretical figure is never fully available: part of VRAM is reserved for the plane name tables, the sprite attribute table (SAT), and other internal data, and SGDK, on top of that, automatically splits whatever remains between background tiles and sprite tiles. That automatic split usually goes unnoticed while the graphics are modest, but as soon as a project starts using backgrounds with many unique tiles (a full illustration converted to tiles, for example, rather than a level tileset with repeating patterns), it can become a real problem: sprite tiles end up being written on top of background tiles, producing visual corruption.

This chapter explains that automatic split, how to detect when it runs out, how to measure ahead of time how many unique tiles a graphic needs, and how to take manual control of the allocation with `SPR_initEx()`.

Prerequisites

Before continuing, it helps to have:

- The VRAM, tile, and palette vocabulary introduced in Chapter 2.
- The manual VRAM accounting and use of `TILE_USERINDEX` covered in Chapter 7, since this chapter revisits that same idea of “occupied space” but applied to the split between background and sprites.
- Basic use of the sprite engine (`SPR_addSprite()`, `SPR_update()`, `SPR_init()`) introduced in Chapter 21.
- Basic handling of a pixel-art editor with indexed palette support (Chapter 8), and, in particular, of Aseprite's tileset tool already introduced in Chapter 19, which in this chapter is reused only for counting tiles, without revisiting how it works internally.
- An emulator with VDP/VRAM debugging tools, such as the Gens emulator introduced in Chapter 4.

1. A typical scenario: a nearly full background and an overlapping sprite

Picture a project that already has a finished background (a full illustration converted to the Mega Drive palette, not a level tileset with lots of repeating patterns) and that adds a new element as a sprite: a title-screen logo, a UI icon, a large character. The sprite itself is declared, loaded, and drawn without any compile errors, and when the ROM runs it appears correctly on screen. The problem shows up somewhere else: in the part of the background that was located “furthest along” in the memory occupied by its tiles, fragments of graphics appear that don’t belong there, as if something had been written on top. If the new sprite has an animation, even one as simple as alternating between two frames, the problem becomes even more visible: the corrupted area of the background changes appearance in step with the sprite’s animation, because it’s literally the sprite’s tiles that are showing up there.

This isn’t a coincidence or a hardware bug: it’s an exact symptom of the background tiles and the sprite tiles competing for the same VRAM positions.

2. Why it happens: two automatic allocations sharing the same space

SGDK doesn’t reserve one fixed block of VRAM for the background and another fixed, independent block for sprites: it automatically splits the available space between both uses, and that split has a limit. Background tiles, loaded through IMAGE, TILESET, or MAP resources, are placed starting at TILE_USERINDEX (seen in Chapter 7) and moving forward, occupying as many positions as the image has unique tiles. The sprite engine, for its part, automatically reserves its own block of VRAM to upload the tiles of active sprites into, and that block has a default size.

As long as the background uses few tiles, both blocks coexist without any problem: there’s plenty of room for both. But if the background gets close to the tile limit that the automatic split assigns to it, the block reserved for sprites starts to overlap with the tail end of the background tiles, and the sprite engine ends up writing its own data there, overwriting background tiles that were already in use. The result is exactly the corruption described in the previous section: it’s not that the background is physically damaged, but rather that another set of data now occupies that same VRAM address.

3. How much VRAM is available: rough figures, not universal constants

To be able to reason about this problem without memorizing memory addresses, it helps to have a reference figure in mind: in a typical configuration, the VRAM available for user tiles can be thought of as a space of roughly **1,400 tiles** in total, of which SGDK automatically assigns around the first **1,000** to the background and leaves the remaining **400** for sprites, through a default reservation of **420 tiles** that `SPR_init()` applies when initializing the sprite engine.

It’s worth stressing that these numbers are a rough simplification, not a guaranteed constant of the hardware or of SGDK: the theoretical maximum figure for VRAM, per Chapter 2, is 2048 tiles if all of memory were dedicated exclusively to tiles, but in practice that figure shrinks because of the space taken up by the plane name tables, the SAT, and other internal data, and the exact split between background and sprites can vary depending on the resolution mode, how many planes are in use, and other details specific to each project. Take these figures

as a practical guide for reasoning about the problem, and for getting a rough sense of how much headroom there is before the automatic split starts to fail, not as an exact limit that will always match, tile for tile, in every configuration.

The practical rule worth remembering. A background built from a fully converted image (as opposed to a level tileset with repeating patterns, where many tiles get reused) tends to use a number of unique tiles very close to the total number of tiles the image occupies, because almost no 8×8 cell repeats. That type of graphic is, by far, the one most likely to approach the available limit and trigger the conflict described in the previous section.

4. Counting unique tiles with Aseprite’s tileset tool

Before touching a single line of code, it’s worth measuring how many unique tiles each image in the project actually uses. Chapter 19 already introduced, within Aseprite, the editor’s native tileset tool (available since version 1.3) as a way to paint a level map while taking advantage of several palettes at once; that same panel, without needing to revisit how it works internally, serves here for something much simpler: counting how many distinct tiles are in an image.

When converting an image to a tileset within Aseprite and enabling the 8×8 grid over the canvas, the tileset panel numbers each unique tile it detects and lets you visually match each cell of the image to its corresponding number. In a fully converted image, like the background in the scenario laid out in section 1, the vast majority of tiles turn out to be unique, except for the occasional coincidence in areas of very flat color (a plain white surface, for example), where several cells share exactly the same tile. A background of this kind can end up around, say, 1,000 unique tiles: a figure that, compared with the automatic background reservation mentioned in the previous section, immediately explains why there’s almost no headroom left before the sprite block starts to overlap.

Repeating the same procedure on the sprite’s graphic (the logo or added element) might turn up, for example, a considerably lower figure, somewhere in the low hundreds of unique tiles. Added to the background tiles, that figure might still fall within the rough 1,400-tile total limit, and yet the conflict still occurs, because SGDK’s automatic split doesn’t know in advance how many tiles the background is actually going to need: it simply applies a fixed reservation of 420 tiles for sprites, regardless of whether the background ends up using 600 or 1,000. When the sum of both actual uses exceeds the total available, the excess is what gets written over the tail of the background.

5. Adjusting the split with `SPR_initEx()`

The solution isn’t to shrink the background or sacrifice the sprite, but to tell SGDK exactly how much VRAM to reserve for the sprite engine, instead of accepting the automatic value of 420 tiles. Instead of the usual function:

```
SPR_init();
```

you use:

```
SPR_initEx(300);
```

`SPR_initEx()` takes a single parameter, `vramSize`, indicating how many VRAM tiles to reserve for sprites; passing 0 is equivalent to using the default value of 420 tiles, the same one `SPR_init()` applies internally. By lowering that number — from 420 to 300, for example —

additional space is freed up for background tiles before the area reserved for sprites begins, and the sprite tiles start being written further along in VRAM, leaving the tail end of the background tiles untouched.

Only one of the two sprite-engine initialization functions should be used, never both: if the project already calls `SPR_init()`, that call is replaced with `SPR_initEx()` rather than being added alongside it.

After recompiling with this change, the background from the scenario laid out at the start of the chapter should now display without visible corruption, with the sprite — animated or not — occupying an area of VRAM that no longer interferes with the background tiles. Opening the emulator’s VDP/VRAM viewer again (Chapter 4) lets you confirm this visually: between the end of the background tiles and the start of the sprite tiles there should now be a gap of free space, instead of an overlap. Lowering the `vramSize` value even further — to 200, for example — pushes the start of the sprite tiles even further along in memory, widening that gap.

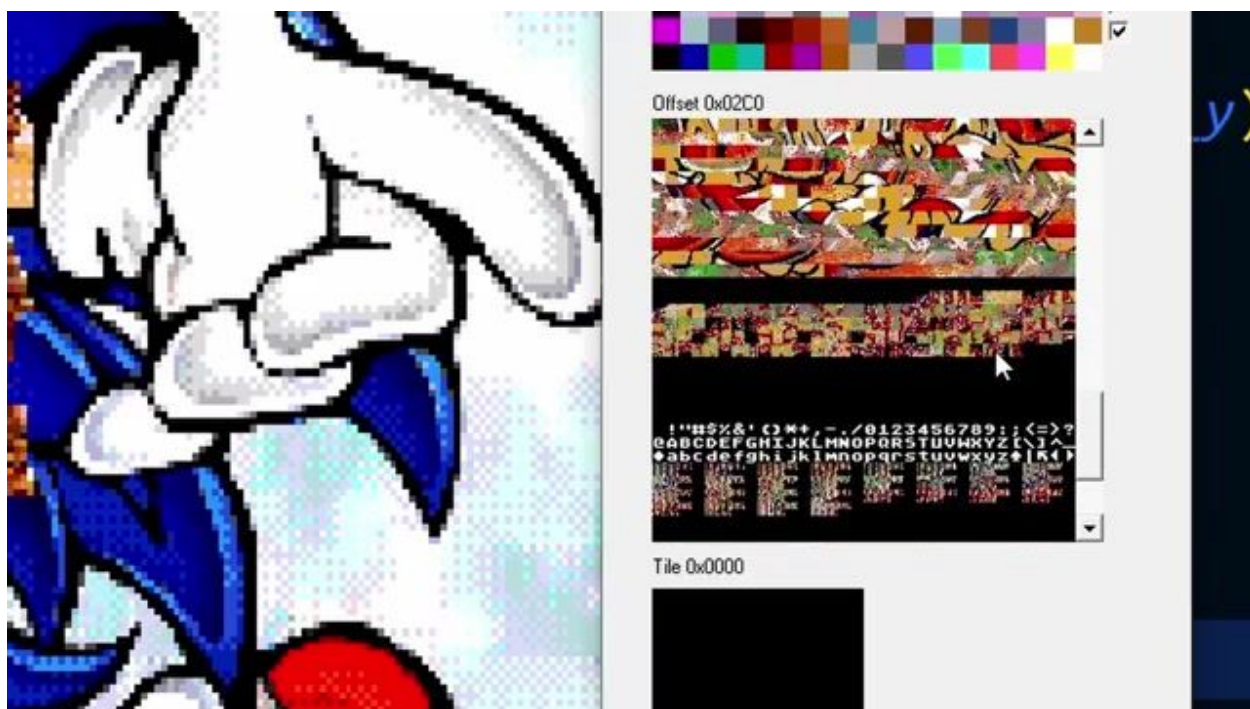


Figure 130: VDP/VRAM viewer after adjusting to `SPR_initEx(300)`

The emulator’s VDP/VRAM viewer after the change to `SPR_initEx(300)`: between the end of the background tiles and the start of the sprite tiles there is now a gap of free space, instead of the previous overlap.

6. How much VRAM to reserve in a real project

`SPR_initEx()` doesn’t have a single correct value: how much VRAM to reserve for sprites depends entirely on the project’s needs. A game with very detailed backgrounds, close to using a full screen as a single illustration with few repeated tiles, will need to reduce the sprite reservation to leave more room for the background. A game with large sprites, highly detailed ones, or many distinct animation frames on screen at once — several enemies with

elaborate sprites, for example — will need the opposite approach: keeping or even expanding the sprite reservation, at the cost of a simpler background or one with more repeated tiles.

That tension — more VRAM for the background means less for the sprites, and vice versa — is, to a large extent, unavoidable: there's no combination that maximizes both resources at once, because they share the same physical space. What's generally worth avoiding is needlessly exhausting either side: reserving far more VRAM than a sprite actually needs leaves less room for the background with no real benefit, and the same holds in reverse. Just as with the number of colors in a palette, the practical recommendation is to measure first — using the tileset tool described in section 4 — how much VRAM each part of the project actually needs, and set `SPR_initEx()` to that real number, leaving just a small margin, rather than reserving extra “just in case” or thoughtlessly accepting the automatic value of 420.

Review exercises

1. Explain, roughly, how SGDK splits the available VRAM by default between background data and sprite data, which function and which parameter are responsible for that split, and why those figures should be taken as rough guidelines rather than a guaranteed exact limit.
2. Describe how you can check, before writing any code, how many unique tiles a background image uses, and why a fully converted image tends to use far more unique tiles than a level tileset with repeating patterns.
3. What visual symptom does a VRAM allocation conflict between background and sprites produce, and what is its exact cause in terms of shared memory positions?
4. Explain the role of the `vramSize` parameter of `SPR_initEx()`, which value corresponds to the default behavior of `SPR_init()`, and what happens when it's reduced.
5. Discuss the trade-off between dedicating more VRAM to the background versus to sprites, and what criteria should guide the decision of how much to reserve in one's own project, rather than simply accepting the automatic value or pushing either resource to its limit.

Chapter 46 — The New SGDK 2.x VRAM Configuration

Chapter 45 introduced the practical limits of Mega Drive VRAM and how to manage them as a project grows. This chapter continues along the same line, but from a different angle: rather than reducing how many tiles a graphic occupies — a topic picked up again in Chapter 47 — the goal here is to understand precisely how SGDK organizes the available VRAM, and to take advantage of a configuration option in the framework itself that lets you reclaim space which, in earlier versions of SGDK, was reserved but left unused.

Understanding the full anatomy of VRAM and knowing how to read SGDK's configuration documentation is useful even in projects with plenty of room to spare: sooner or later, any game with enough levels, enemies, or effects ends up needing every available tile.

Prerequisites

Before continuing, you should be clear on:

- The VRAM limits of the Mega Drive and the general strategies for managing them, from Chapter 45.
- What `TILE_USERINDEX` is and why it's useful to reserve VRAM ranges starting from that constant, from Chapter 7.
- The general anatomy of VRAM — tiles, planes, sprites — presented in Chapter 2.
- What `SPR_init()` does, along with its variant `SPR_initEx()`, explained in Chapter 21.
- A working installation of SGDK 2.11, since this chapter modifies a configuration file belonging to the framework itself and requires recompiling it before the change takes effect in any project.

1. VRAM anatomy: what's there, and in what order

Before touching any configuration, it's worth walking through the entirety of VRAM using an emulator's VRAM viewer (BlastEm and Gens KMod, among others, include one), tile by tile, to see exactly what information lives in each area. In a typical SGDK project, that walkthrough reveals a fairly consistent order:

1. Sixteen solid-color tiles at the very start, reserved for SGDK's internal use.
2. The background and foreground tiles of the current level or screen.
3. The tiles that change constantly: the sprites, both the player's and those of enemies or other animated elements.
4. SGDK's basic text font, used to display numbers and letters on screen (the same default font that already appeared when drawing the first graphics back in Chapter 2).

5. A block that at first glance looks like a set of corrupted or visually meaningless tiles: it's actually the scroll table and other internal structures associated with the background foreground plane (BG_A).
6. A second block that looks just as chaotic, corresponding to the same kind of information but associated with the background plane (BG_B).
7. The sprite attribute table, the structure SGDK uses internally to keep track of every active sprite at a given moment.

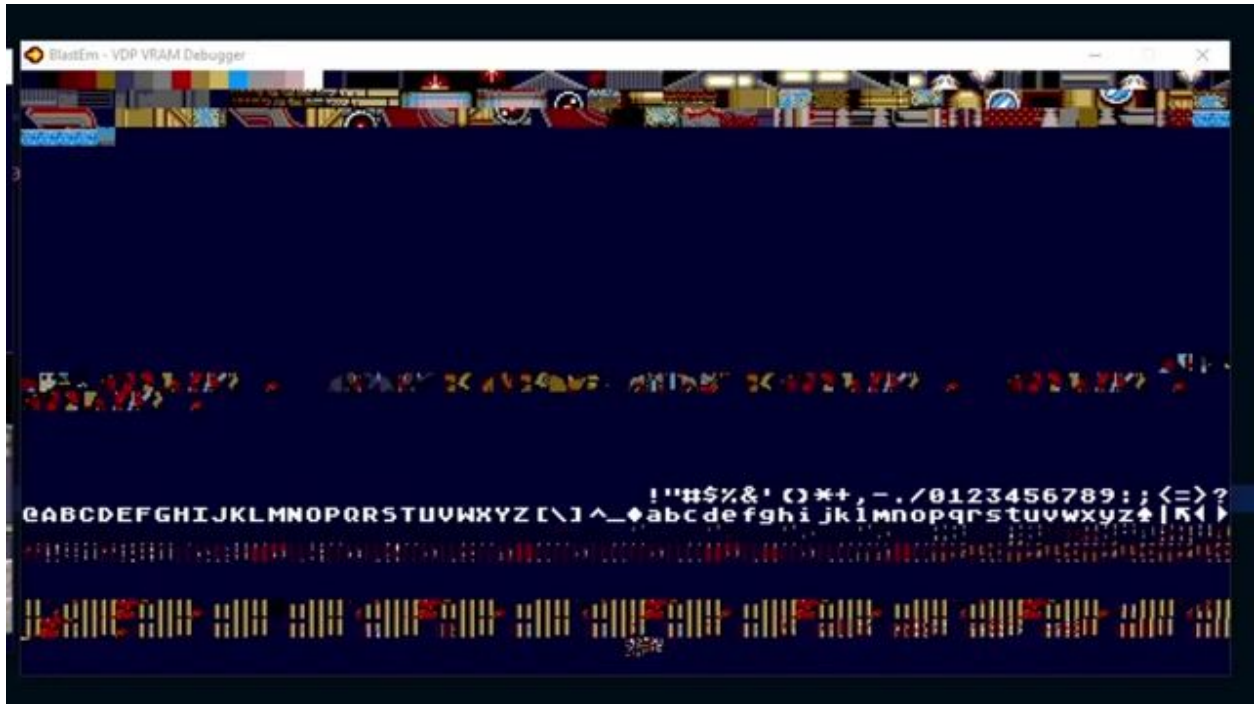


Figure 131: VRAM viewer showing the 16 internal-use tiles, the background and sprite tiles, and the text font

BlastEm's VRAM viewer, with the 16 internal-use tiles at the start, followed by the background and sprite tiles and, further along, the text font.

Don't mistake the "corrupted"-looking blocks for an actual error. The scroll table and the sprite attribute table aren't meant to represent visible graphics: they store control data that the Mega Drive's VDP constantly reads during rendering. When viewed through a VRAM viewer that interprets them as if they were ordinary graphic tiles, it's expected that they'll look like visual noise with no recognizable pattern.

2. config.h: a file for reconfiguring SGDK

Inside the inc folder of the SGDK installation — the same folder that header files used earlier in this manual, like `genesis.h`, come from — you'll find `config.h`, designed specifically to let you enable or disable certain SGDK features before compiling it.

Among the options `config.h` offers that this chapter doesn't touch are, for example, one that keeps the sound processor held during DMA (direct memory access) transfers, meant to avoid compatibility issues on certain Mega Drive models; the choice of which sprite engine to use internally, which by default points to the newer and more efficient engine unless the project

needs sprite multiplexing (a technique for exceeding the console's usual 80-hardware-sprite limit, which is beyond the scope of this chapter); and the enabling of *bank switching*, needed only for ROMs larger than 4 megabytes.

The option that matters here is `LEGACY_FONT_LOCATION`, which is enabled by default (with a value of 1). With that default value, SGDK places the text font in VRAM exactly where it appeared during the walkthrough in the previous section: after the sprites. Disabling this option — changing its value from 1 to 0 — moves the font to the start of VRAM, immediately after SGDK's 16 internal-use tiles, and frees up the space it used to occupy.

```
// config.h, inside the inc/ folder of the SGDK installation
#define LEGACY_FONT_LOCATION 0
```

3. Applying the change: recompiling SGDK

No change to `config.h` takes effect until SGDK itself is recompiled, since this is a compile-time option of the framework, not of the game project. After saving `config.h` with `LEGACY_FONT_LOCATION` set to 0, the `build_lib.bat` file — located in the root folder of the SGDK installation — handles that recompilation: running it opens a console that fully recompiles SGDK and, when finished, asks you to press any key to close.

With SGDK recompiled, all that's left is to go back to your own project and recompile it too, so it uses the updated version of the framework. Opening the VRAM viewer again, the change is immediately obvious: the text font no longer appears after the sprites, but right at the very start of VRAM.

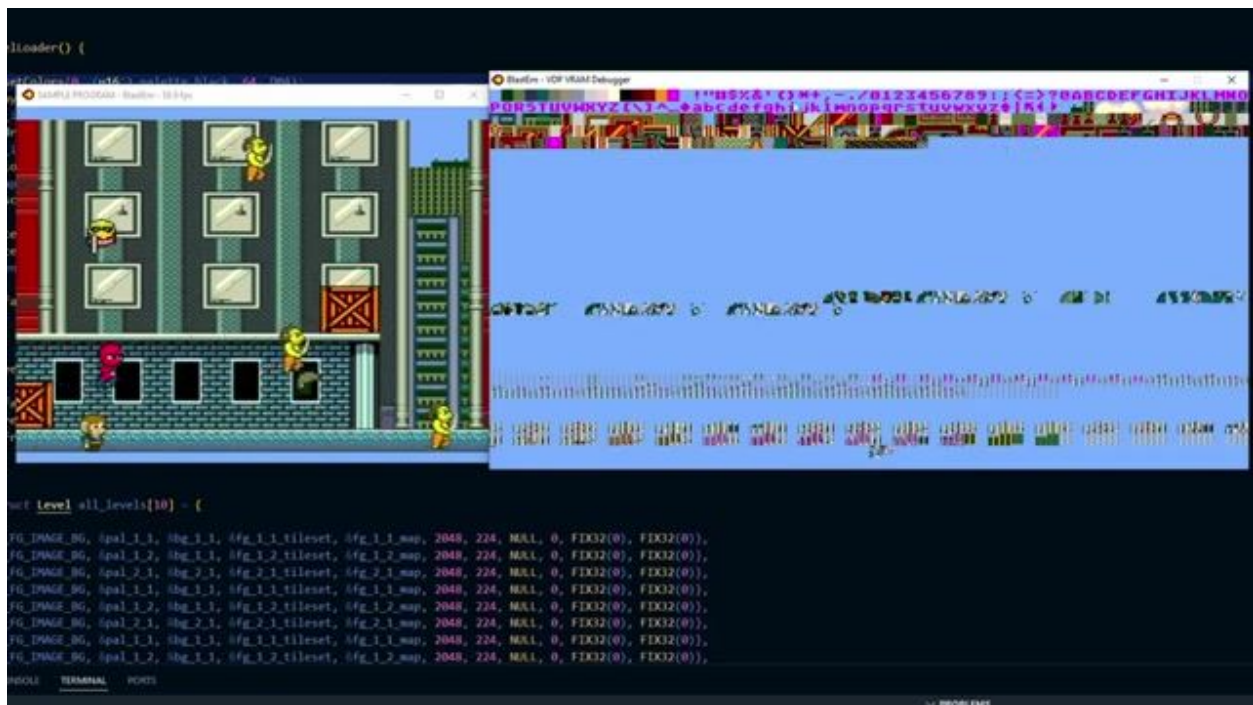


Figure 132: The text font shifted to the start of VRAM after recompiling SGDK

After recompiling SGDK with `LEGACY_FONT_LOCATION` set to 0, the font is placed at the start of VRAM, immediately after the 16 internal-use tiles.

4. Why it matters: space reserved even when unused

The font changing position within VRAM might seem like a detail with no real practical consequence, but it isn't. Consider, for example, a level that doesn't need to display any text on screen, and where you decide, manually, not to load the font tiles at all. With SGDK's old configuration (`LEGACY_FONT_LOCATION` set to 1), that leaves what looks, at first glance, like empty space in VRAM, located right before the background foreground plane's scroll table.

The intent in not loading the font is usually to make that free space available for additional sprites. And for a while, it does seem to work: as more sprites are generated, the empty space shrinks normally. But at some point, it stops shrinking no matter how many new sprites are generated; instead, SGDK starts looking for free space in other areas of VRAM, reusing gaps left behind by earlier sprites that are no longer in use.

The reason is that the old configuration reserves that space for the text font unconditionally, regardless of whether the program ever actually loads a single font tile. That reservation, invisible to the programmer but very real from SGDK's point of view, means that neither the sprite engine nor the background tile loader can ever use that range, no matter how completely free it appears to be.

5. Recalculating the starting point of available VRAM

With `LEGACY_FONT_LOCATION` set to 0, that space is finally no longer held captive. To take advantage of it in your own project's code, it's worth revisiting how the starting index of available VRAM is calculated at the beginning of the routine that loads a level's graphics.

Until now, that index was usually initialized with the value of `TILE_USERINDEX`, the SGDK constant marking the first tile free for the programmer after the tiles reserved by the system (introduced in Chapter 7). Looking at its definition in SGDK's code, that value is calculated as the sum of three parts: `TILE_SYSTEM_INDEX` (the position where system tiles start, typically 0), plus `TILE_SYSTEM_LENGTH` (the length of those system tiles), and `FONT_LENGTH` (the length of the font).

In the SGDK 2.11 installation used for this manual, `TILE_SYSTEM_LENGTH` was 16 and `FONT_LENGTH` was 96, but these two numbers should be treated as illustrative, not fixed: they're internal SGDK constants and could change in future versions of the framework. Before relying on a specific numeric value, it's worth checking it directly in your own installation, either by letting your editor show the value of `TILE_USERINDEX` when hovering over the constant, or by checking its definition in SGDK's source code.

With those reference values, the example from the previous section wasted 96 whole tiles, reserved for a font that this particular level didn't even load. With `LEGACY_FONT_LOCATION` set to 0, there's no longer any need to reserve that space before starting to load the level's own tiles. Subtracting that number of tiles from `TILE_USERINDEX` works as a first test, but it leaves the code unexpressive and fragile against future SGDK changes. A cleaner alternative is to use the `TILE_FONT_INDEX` constant directly as the starting value, since it represents exactly the position where the font used to start and which is now available for the rest of the project:

```
// In the routine that loads the current level's graphics,  
// before reserving space for the background and sprites  
u16 vram_index = TILE_FONT_INDEX;
```

The SGDK maintainer recommends not going any further than this and not overwriting the 16 system tiles at the start of VRAM, since doing so can cause problems in certain map-handling functions. This is an explicit recommendation from the SGDK project itself, so it's worth taking seriously in any production-bound code; anyone who still wants to experiment

with that margin can use `TILE_SYSTEM_INDEX` instead of `TILE_FONT_INDEX` as the starting point, knowingly taking on that risk:

```
// Riskier: ignores the 16 system tiles.  
// When in doubt, prefer TILE_FONT_INDEX.  
u16 vram_index = TILE_SYSTEM_INDEX;
```

6. Verifying the reclaimed space with `SPR_initEx()`

To concretely confirm that the space previously occupied by the font is now usable by the sprite engine, it's worth turning to `SPR_initEx()`, the variant of `SPR_init()` presented in Chapter 21 that takes as a parameter the number of VRAM tiles specifically reserved for the sprite engine. Passing 0 as the argument — the default value — makes `SPR_initEx()` behave exactly like `SPR_init()`, automatically reserving 420 tiles.

As a test, you can temporarily force a low tile count, chosen deliberately to push the sprite space until it butts right up against the start of the background foreground plane's scroll table:

```
// Temporary test: forces the sprite engine to use  
// only 37 tiles, to visually verify in the VRAM viewer  
// that no reserved gap remains.  
SPR_initEx(37);
```

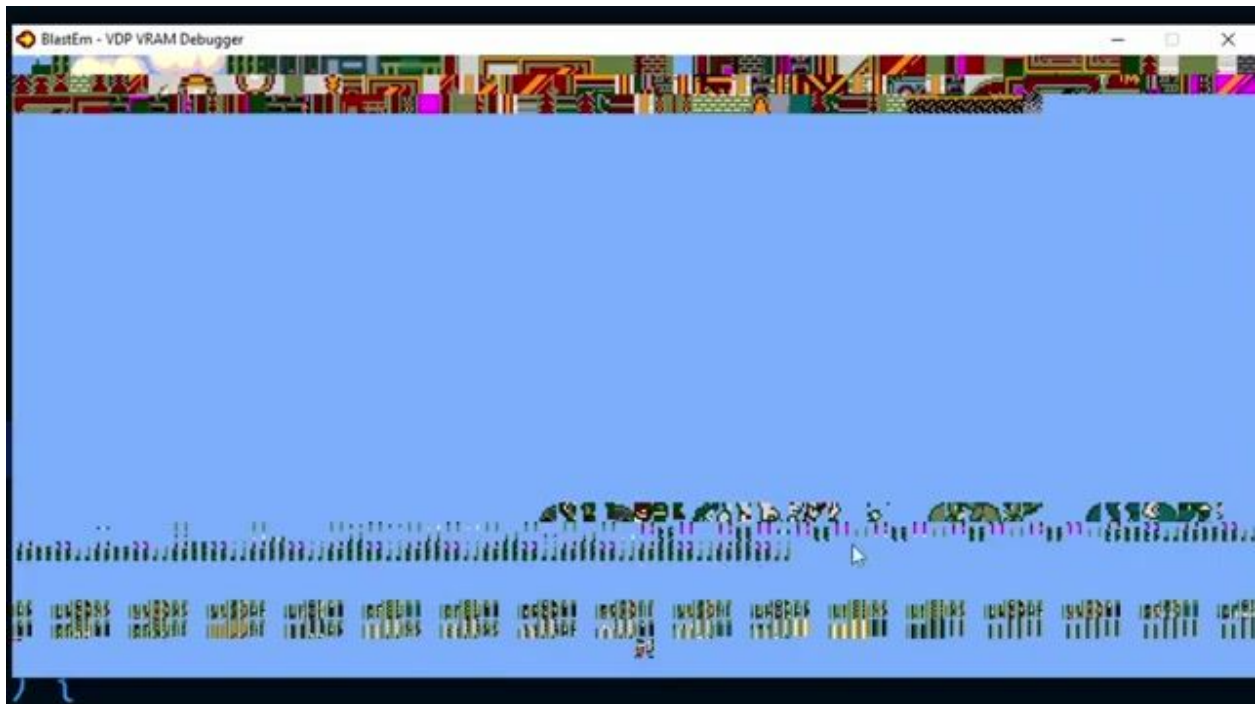


Figure 133: Background and sprite tiles packed with no gaps after testing with `SPR_initEx(37)`

With `SPR_initEx(37)`, the background and sprite tiles end up packed with no gap at all up to the start of the scroll table.

Upon recompiling and reopening the VRAM viewer, the background and sprite tiles end up packed with no gap at all up to the start of the scroll table, confirming that the space formerly

reserved for the font is now fully put to use. Once the verification is done, it's worth reverting to `SPR_initEx(0)` (or directly to `SPR_init()`), restoring the default behavior of 420 tiles for the sprite engine: the low value used in the test only serves to demonstrate the point, not as a final setting. In a real project, this parameter is usually tuned level by level, depending on how many background tiles each one needs — a topic revisited alongside the other VRAM-saving techniques later in this part of the manual.

Review exercises

1. List, in order, the different sections that make up the VRAM of a typical SGDK project, according to the walkthrough done at the start of this chapter.
2. Explain why the blocks corresponding to the scroll table and the sprite attribute table appear, in a VRAM viewer, as tiles with no apparent visual meaning.
3. What does the value of the `TILE_USERINDEX` constant represent, and from what three components is it calculated?
4. Describe the specific problem that a level can run into when it decides not to load the font tiles: why doesn't the space that looks empty in VRAM end up usable for new sprites under SGDK's old configuration?
5. What is the difference between using `TILE_FONT_INDEX` and using `TILE_SYSTEM_INDEX` as the starting value of the available VRAM index, and what risk does the SGDK maintainer point out regarding the second option?

Chapter 47 — Reducing Tiles and Saving VRAM

Up to this point, this manual has almost always assumed that a level's graphics fit comfortably into the available VRAM. That assumption is reasonable while designing a game from scratch for the Mega Drive, because the art creation process itself tends to stay within reasonable budgets: the artist knows from the start what the platform's limitations are and draws accordingly. But sooner or later, anyone tackling a more ambitious project — a port, a demake of a game designed for more powerful hardware, or simply a game with very detailed backgrounds — runs into the opposite problem: the graphics already exist, they already look the way they're supposed to, and the number of unique tiles they need simply doesn't fit in VRAM.

This chapter deals with what to do in that scenario: how to detect redundant or nearly identical tiles within an already-finished tileset, and how to merge them to reclaim space without sacrificing the visual quality of the final result. Unlike most chapters in this manual, the content here doesn't revolve around C code or SGDK functions, but around a technique applied directly to the artwork itself, supported by tile-editing tools.

Prerequisites

- Understanding what VRAM is and what a unique tile means, covered in chapter 2.
- Familiarity with Aseprite as a tile-based graphics editor, already used in chapters 8 and 19 to create graphics and level maps.
- It's worth having read chapters 45 and 46, on managing VRAM limits and configuring VRAM in SGDK 2.x, since this chapter assumes that context and focuses on a complementary technique: reducing the number of unique tiles at the source, before they ever take up space on the console.

There's no need to have any open-source project set up or compiling: the content of this chapter is independent of whatever C program is being developed and applies to any Mega Drive project in general.

1. Why it matters: the real limit on unique tiles in VRAM

In theory, the Mega Drive's 64 kilobytes of VRAM are enough to store up to 2048 unique tiles, as explained in chapter 2. In practice, however, that theoretical number is never fully available for level graphics, because part of VRAM is reserved for other structures the system needs, such as scroll tables (see chapter 45 for details on how VRAM is divided between those structures and graphics tiles).

A useful rule of thumb, born out of experience porting games from more powerful hardware to the Mega Drive, is this: if the combined unique tiles between a level's background and

foreground exceed a thousand, it's time to start worrying about not having enough room left for the player's sprites, enemies, power-ups, and everything else that also consumes VRAM. The concrete recommendation that follows from that experience is to keep the sum of background and foreground unique tiles below a thousand, and preferably below eight hundred, leaving a comfortable margin for everything else a level needs to display on screen at the same time.

This figure isn't an exact hardware rule, but a design guideline learned through practical experience, and it should be treated as such: a useful reference point, not an absolute limit etched into the console's silicon.

The distinction between making an original game for the Mega Drive and porting or remaking a game originally designed for more powerful hardware is central to this chapter. When designing from scratch, the tile budget tends to be respected almost as a matter of habit. When remaking a 32-bit game, on the other hand, the goal is to resemble the original as closely as possible, and that goal competes directly with the need to keep the number of unique tiles low: you don't always have the freedom to simplify the art as much as you'd like. That's why tile-reduction techniques are far more relevant in this second scenario.

2. How many tiles professional backgrounds use: some reference points

Before diving into the technique itself, it's worth getting an intuitive sense of how many unique tiles are needed depending on the type of scene. A fairly consistent observation across different games of the era is that nature is more expensive in tiles than human-made structures: a natural landscape, with its organic variation of shapes and shadows, resists tile repetition much more than a brick wall or a regular architectural structure does.



Figure 134: Cave background with rock formations

A cave background with organic rock formations: about 390 unique tiles, reflecting how costly it is in VRAM to convincingly represent a natural environment.

As an illustrative example (the specific numbers vary a lot from project to project and shouldn't be taken as an exact table, just as a sense of scale), a cave background with rock formations might come in at around 390 unique tiles, a relatively high number that reflects how expensive it is to convincingly represent a natural environment. A more artificial or architectural scene — a castle, a corridor, a building façade — on the other hand, tends to achieve visually rich results with far fewer tiles: a foreground of just 147 tiles alongside a background of 105, adding up to around 250 in total, or even a foreground of 86 tiles and a background of 129, both well below the recommended limit of a thousand combined tiles.

The most extreme case of tile economy usually shows up in well-designed repeating patterns: a castle corridor with a wall-and-tower pattern can be achieved with as few as twenty-one unique tiles in total, despite including trees, different wall heights, and shading on the stonework. It's a good reminder of just how much a well-thought-out pattern — with pieces that fit together and repeat with subtle variations — can deliver visually, far beyond what the actual tile count in VRAM would suggest.

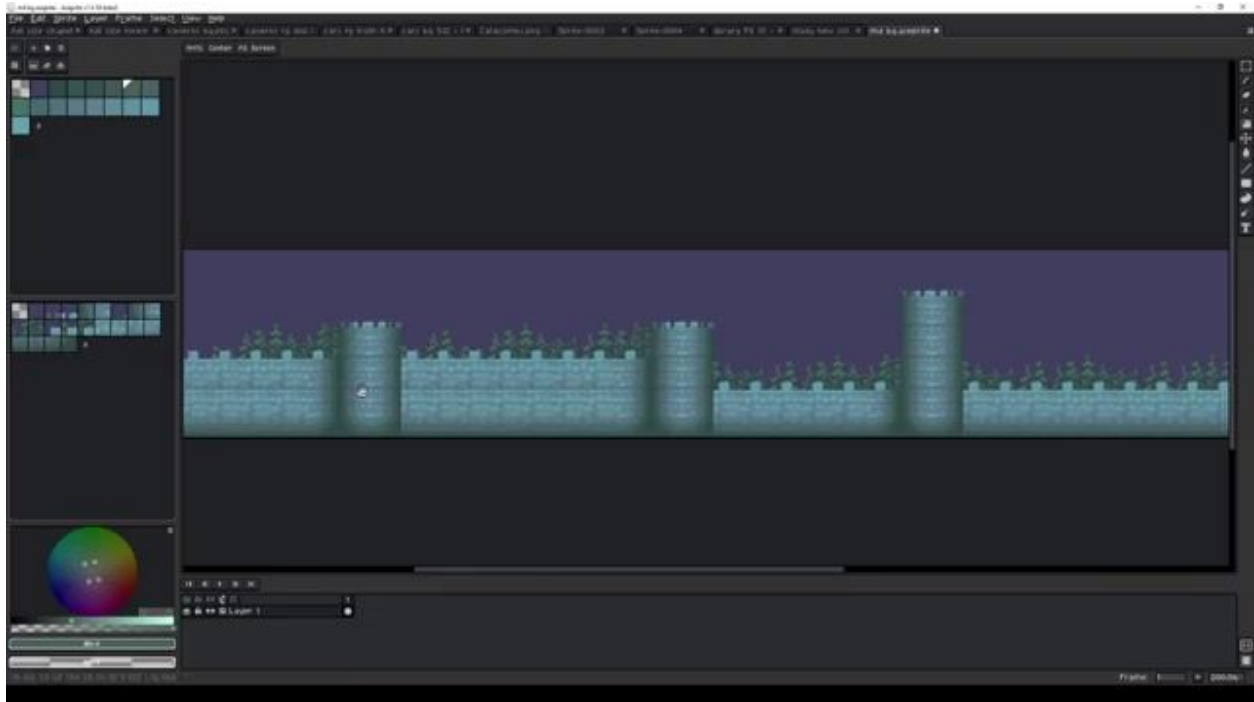


Figure 135: Castle wall with few unique tiles

A section of castle wall that, despite its apparent variety of shapes and shading, uses only 21 unique tiles.

3. Preparing the graphic for analysis

When, despite every design precaution, a background or foreground ends up exceeding the desired tile budget, it's worth analyzing the tileset for redundant tiles before cutting visual content. There are community scripts and tools for Aseprite that can automatically detect duplicate or nearly identical tiles — including mirrored variants of the same tile — within a tilemap-type layer. There's no need to use any particular tool: what matters is understanding

the technique, which applies to any equivalent script offering the two functions described below.

Before a graphic can be analyzed, the corresponding layer needs to be in tilemap layer mode rather than a regular layer of loose pixels. If the layer is still in normal mode, it should be converted using the context menu option “Convert to Tilemap,” also checking the option to detect mirrored tiles on the horizontal and vertical axes, an option that will become relevant later in this chapter.

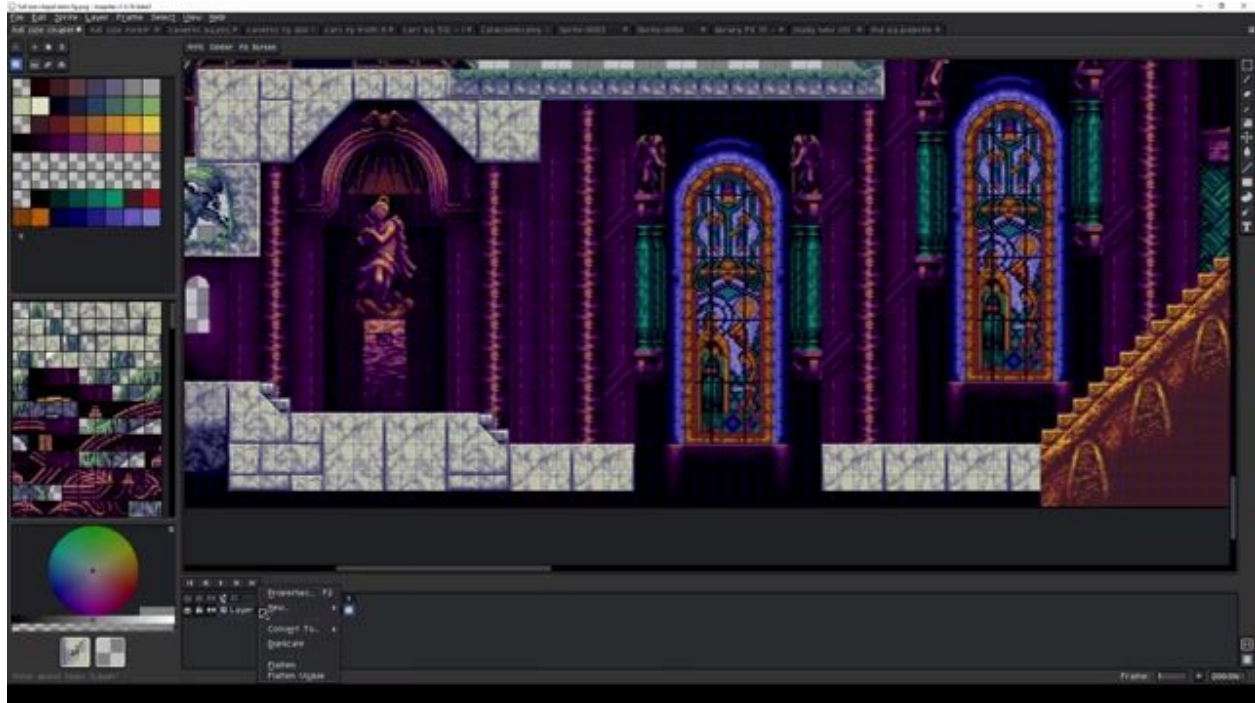


Figure 136: Layer converted to tilemap in Aseprite

A layer already converted to tilemap mode, ready for the reduction script to operate on it.

Before automated tools existed for this, a simple manual method served as a quick check: set the tile editing option to “single tile” mode (so that any change applies to the existing tile instead of creating a new one), add a very eye-catching color to the palette, and paint that color over a single instance of the tile you want to analyze. Since every copy of that same tile elsewhere in the graphic changes color at once, the result lets you see at a glance how many times that specific tile repeats within the level. It’s still a useful trick for a quick spot check, even if you have an automated tool for general analysis.

4. Frequency search: identifying rarely used tiles

A tool of this kind usually offers two main analysis functions: a search by usage frequency and a search by visual similarity. The first takes a threshold number indicating how many times a tile must appear in the graphic to be flagged. With the threshold set to 1, every tile that appears only once in the entire level is highlighted; with the threshold at 2, tiles that appear at most twice, and so on.

The tool’s panel, with its layer and threshold fields, and its three main buttons: Find By Count, Find Similar, and Apply.

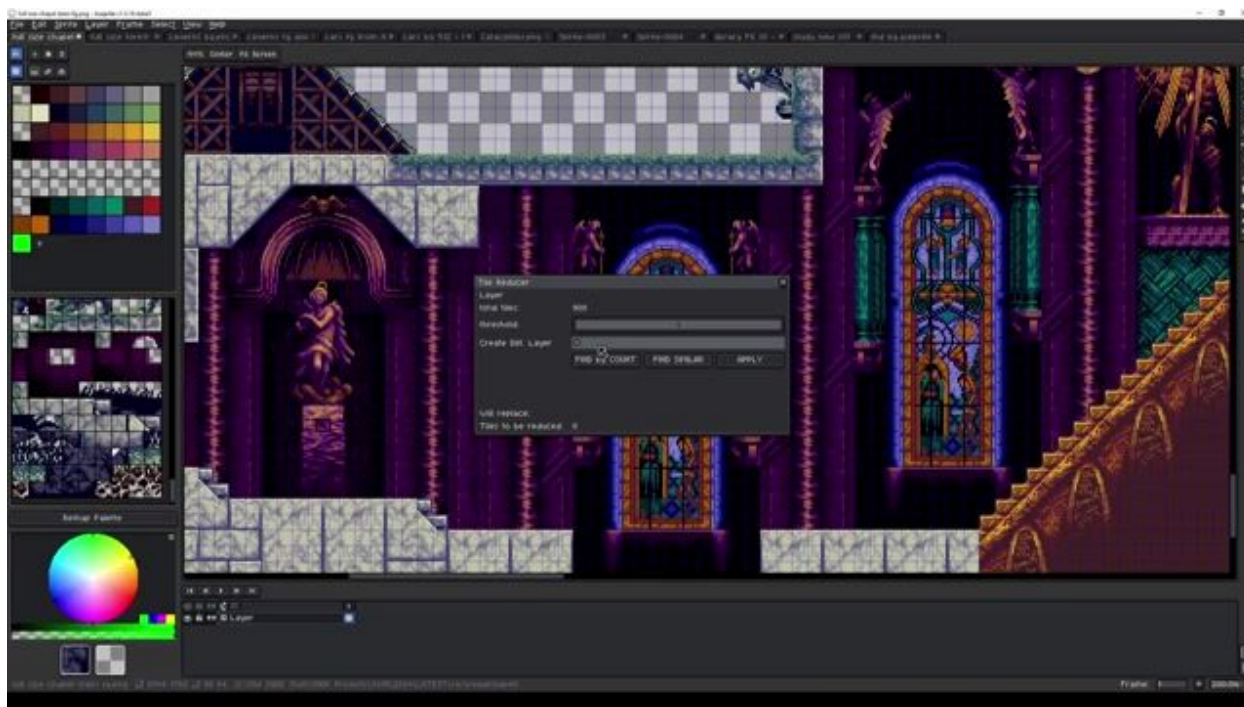


Figure 137: Panel of the tile reduction tool

This information should be interpreted with judgment, not simply treated as a list of tiles to delete. A tile that appears only once isn't, by itself, a problem: many levels deliberately include unique details — a tower, a statue, a decorative plant, a particular window — that give character to a specific area of the scenery, and removing them just because they're rare would impoverish the level's visual design. Each flagged case should be reviewed and a conscious decision made about which ones to leave as they are.

What's worth looking for among rarely used tiles are candidates for accidental merging: tiles that, at a glance, are practically identical to another tile used extensively elsewhere in the level, but which ended up registered as a separate tile due to some small discrepancy — sometimes just a single pixel of difference — usually caused by an error during the extraction (ripping) of existing graphics or by careless manual editing. Faced with a candidate like this, the manual fix is simple: copy the part of the graphic that repeats correctly elsewhere in the level and paste it over the area with the slightly different tile, unifying both versions into one.

The region of the graphic with the correct, already-repeated version of the tile is copied, to be pasted over the area where a slightly different variant appeared by mistake.

5. Similarity search: merging nearly identical tiles

The second function automates exactly this kind of correction. Unlike the frequency search, here the threshold doesn't represent a number of occurrences but a degree of visual similarity between tiles: the lower the number, the more alike two tiles must be to each other to be flagged as merge candidates.

Before using this function, it's always worth turning on the option to create a new layer with the changes (create diff layer or equivalent), so the original layer stays intact in case a change needs to be reverted. Always working on a copy is a cheap precaution against the cost of losing or damaging a level's original graphic.

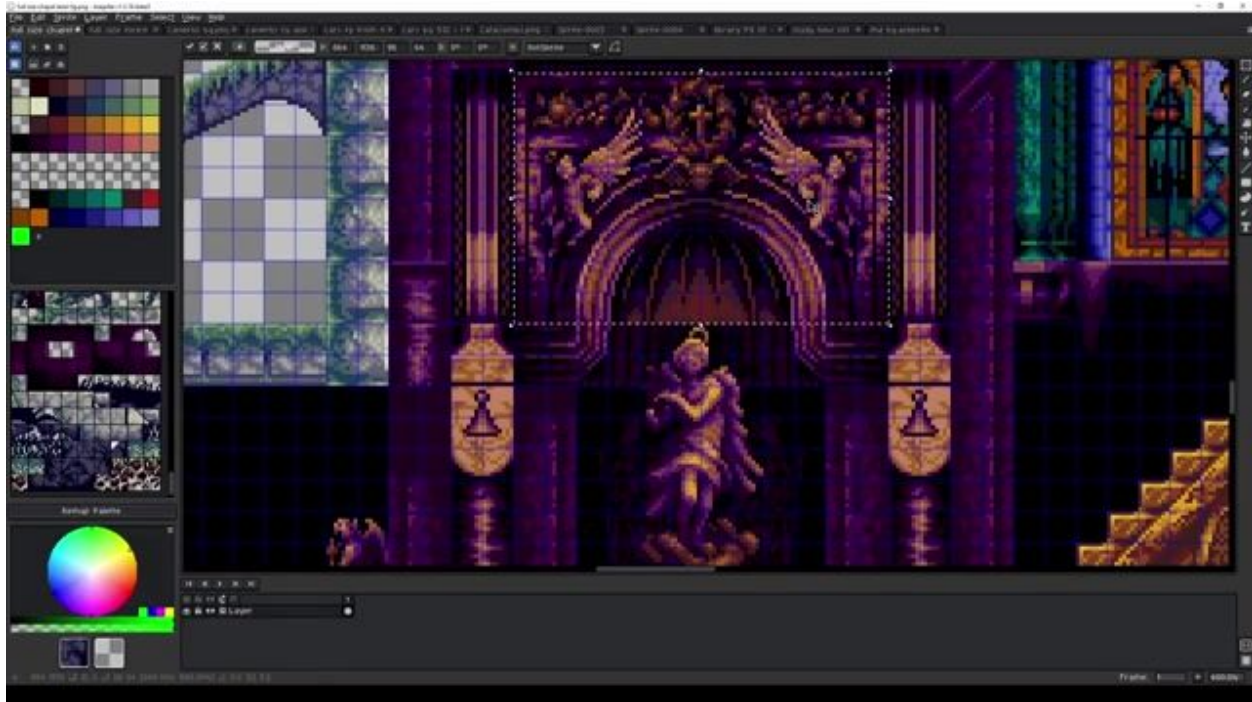


Figure 138: Manual correction of a tile with an erroneous variant

Running the similarity search highlights pairs of nearly identical tiles, indicating for each one how many times it appears within the level. To merge them, it's usually enough to select the less-used tile of each pair and confirm the replacement: that action replaces all its occurrences with those of the more-used tile in the pair, without needing to copy and paste manually as in the previous section.

The script detects a pair of nearly identical tiles, differing by just a couple of pixels, candidates to be merged into one.

It's worth repeating this operation across different tile pairs within the same graphic — nearly identical diagonal lines, symmetric parts of a decorative element — before applying all the selected merges at once, generating the new, independent layer with the result.

Tools of this kind can get somewhat slow when analyzing large graphics, with noticeable pauses while similarity is calculated across all the tiles in the level. This isn't a sign of misuse: it's simply the cost of comparing a considerable number of tiles against each other, and it should be treated as a performance limitation inherent to this kind of analysis, not as a sign that something went wrong.

6. Checking the result: a problem with mirrored tiles

After applying the changes and generating the new layer, the logical first step is to check how many unique tiles remain, since reducing that number was, after all, the whole point of the process. In some cases a counterintuitive result can appear: the new layer reports more unique tiles than the original, instead of fewer, despite having merged several nearly identical tiles.

After applying the merge and creating the new layer, the unique tile count counterintuitively comes out higher than that of the original layer.

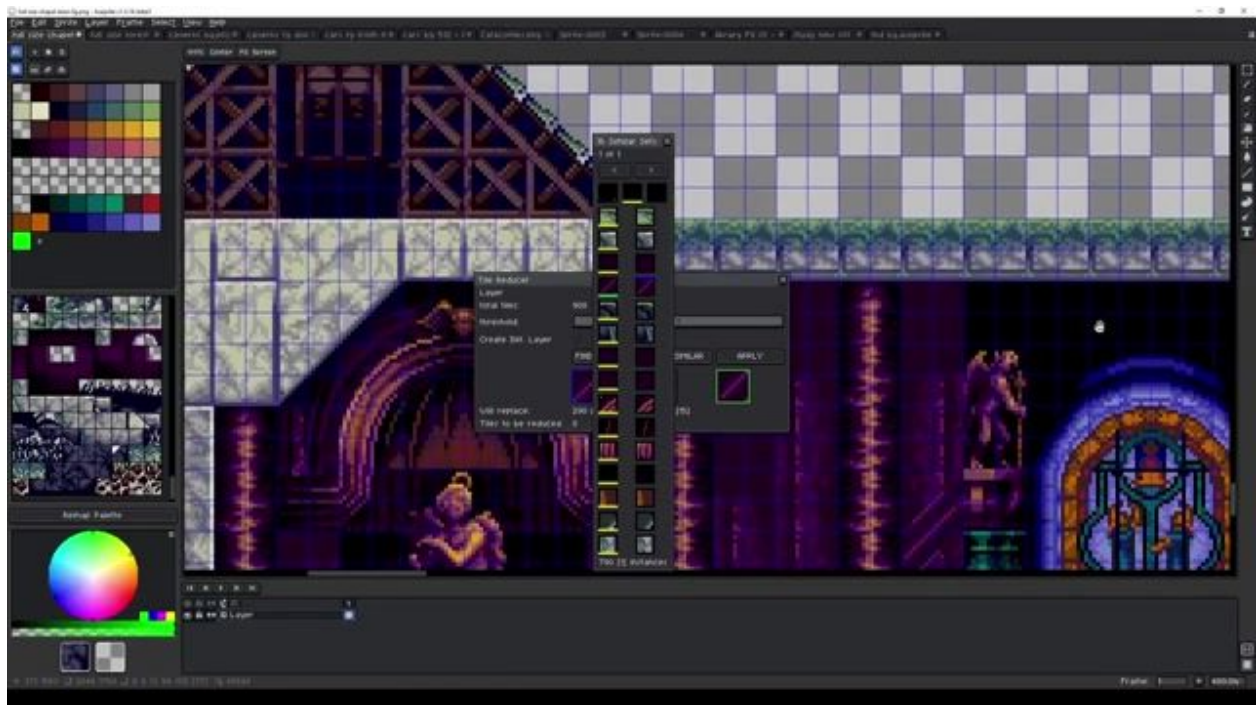


Figure 139: Pair of nearly identical tiles detected by similarity

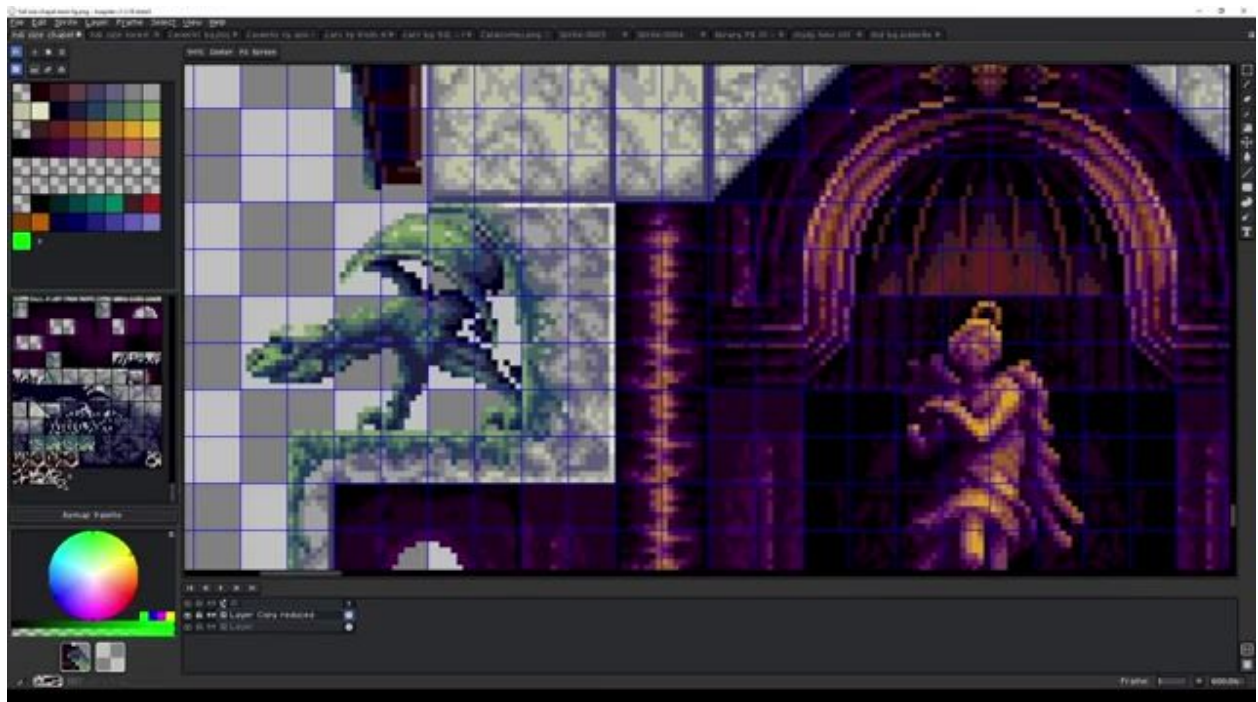


Figure 140: Tile count after merging, higher than the original

This behavior has been observed in specific cases as an issue related to mirrored tiles: when a layer uses horizontal or vertical mirror detection, the merging process can momentarily lose the relationship between a tile and its mirrored versions, counting them as additional tiles instead of recognizing them as reflections of the same base tile. It's not clear whether this is a documented limitation of some particular tool or a one-off quirk of the graphic being analyzed; in any case, it's worth staying alert to this possibility when working with layers that use mirroring.

The fix, when this happens, is simple: convert the new layer back to a normal pixel layer and then reconvert it to a tilemap, checking the horizontal and vertical mirroring options again. That extra step allows the mirroring relationships to be recalculated correctly, and the unique tile count then shows the expected reduction.

With the count now corrected, it's worth doing a visual comparison by toggling the visibility of the original layer and the new merged layer, to confirm that the differences between the two are minimal: small shading details or a handful of different pixels in specific areas of the graphic, differences subtle enough to be imperceptible during normal gameplay and not significant enough to justify keeping extra tiles in VRAM just to preserve them.

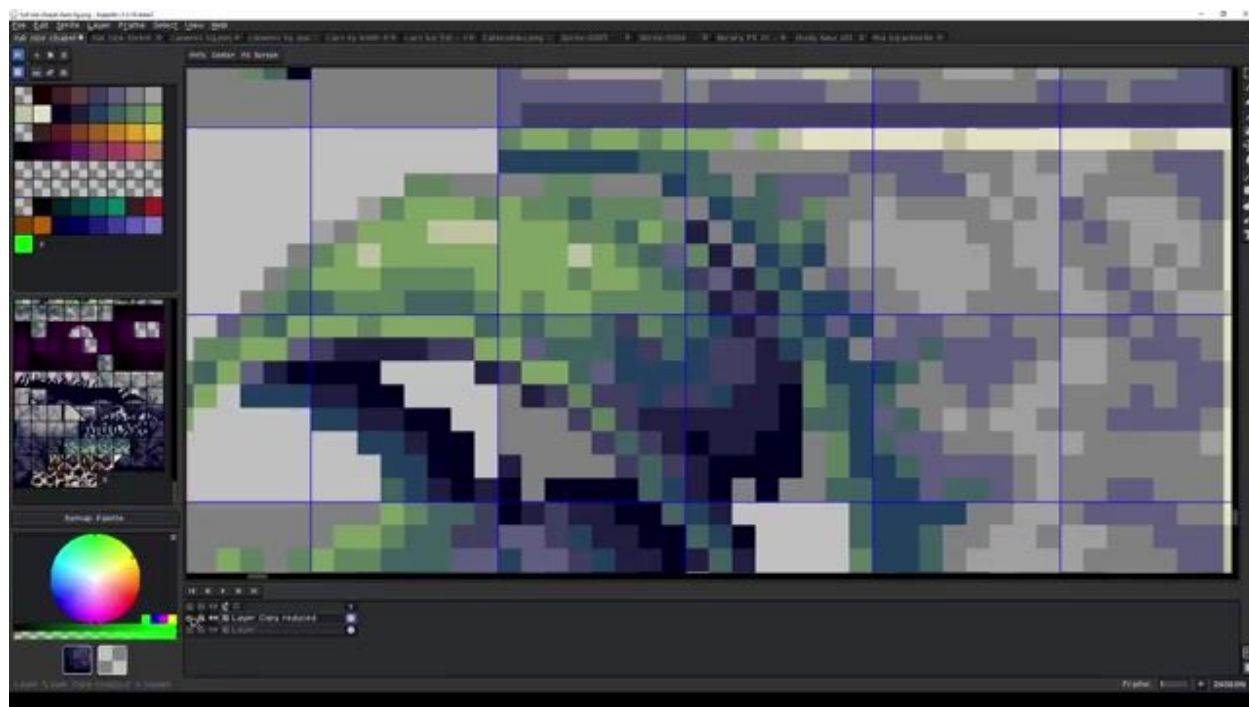


Figure 141: Detail comparison between original and merged layer

The difference between the original version and the merged one boils down, in practice, to a handful of shading pixels, imperceptible during gameplay.

7. How much you gain and who this technique is for

As an illustrative example of the typical magnitude of savings achievable with this kind of analysis — not as a guaranteed figure, since it depends entirely on the specific tileset — a pass of merging nearly identical tiles on an already fairly optimized graphic can recover on the order of fifty to a hundred fewer tiles overall. At first glance that may not sound like much, but that figure can be exactly the difference between being able to include one more enemy

in a level or not, when working close to the practical VRAM limit discussed at the beginning of the chapter. In projects where the available tile margin is tight, every tile recovered has a value out of proportion to how modest the number sounds in the abstract.

This technique is especially useful for those porting or remaking an existing game, because in that scenario the original graphics were created with no consideration whatsoever for the Mega Drive's tile limits, and small accidental discrepancies from the extraction or adaptation process are common. Even so, it retains value even when creating original art from scratch: it's easy to end up, without realizing it, with a pair of nearly identical tiles that could perfectly well be merged into one, so it's worth keeping this technique in mind as a general check, not just as a last resort when a level doesn't fit in VRAM.

Review exercises

1. Explain, in your own words, why a natural landscape tends to require more unique tiles than an architectural or artificial-looking scene.
2. Why is it worth keeping the sum of background and foreground tiles below a thousand, rather than getting closer to VRAM's theoretical limit of 2048 tiles?
3. A tile appears only once in an entire level. Explain why that doesn't automatically mean it should be deleted or merged with another tile.
4. Describe the difference between a frequency search and a similarity search when analyzing a tileset: what does the threshold number represent in each case?
5. Why is it worth always enabling the option to create a new layer (diff layer) before automatically merging tiles, rather than applying the changes directly to the original layer?

Chapter 48 — Saving VRAM Through Manual Sprite Loading

Chapter 21 showed the most direct way to put a sprite on screen: define it, add it to the scene with `SPR_addSprite()`, and let the SGDK engine handle everything else, including uploading the graphic's tiles to the VDP's VRAM. That convenience has a cost that isn't always obvious until many sprites are needed on screen at the same time, especially when several instances share exactly the same graphic. This chapter makes that cost concrete and teaches how to take manual control of VRAM so that repeated instances of the same sprite share a single copy of their tiles.

The case study is a background decoration element (trees that scroll with parallax between the background and foreground) within a sample project. The same reasoning applies to any sprite that repeats many times on screen with the same graphic: clouds, stars, collectible items, projectiles, and so on. Understanding this technique is also the gateway to a broader topic: the manual, strategic management of VRAM as a resource shared between background, foreground, and sprites, which was already raised in chapters 45 through 47 and continues to be developed here from another angle.

Prerequisites

- Knowledge of C and prior experience with SGDK: creating a project, compiling, and running it in an emulator.
- Familiarity with the basic lifecycle of SGDK's sprite engine: `SPR_init()`, `SPR_addSprite()`, `SPR_update()`, and the concept of `SpriteDefinition` as a container for a sprite's animation frames (chapters 21 and 23).
- A general understanding of the VDP's VRAM and what a tile is (the 8x8-pixel graphic cell, the minimum unit of VRAM storage), covered in chapter 2, along with manual VRAM accounting and `TILE_USERINDEX` from chapter 7. This chapter does not explain those concepts from scratch, but goes deeper into how they are managed when the automatic engine is no longer enough.
- An emulator with a VRAM debugger (BlastEm, for example), which lets you view in real time which tiles are loaded and in what position.

1. The hidden cost of the automatic sprite engine

By default, SGDK's sprite engine assumes it's best to stream each sprite's animations from ROM to VRAM as they're needed, frame by frame. This is one of the great advantages of cartridge-based systems over CD-based ones: ROM is read with the almost instantaneous latency and bandwidth of the system bus, while a CD-ROM drive of the era was too slow to feed VRAM on the fly with that same agility.

Deep dive: why the CD complicated animation

Consoles like the Mega CD, PlayStation, and Saturn couldn't rely on nimble tile streaming because accessing a CD-ROM is much slower and less predictable than accessing cartridge ROM. The usual solution was to load a character's entire animation sheet into VRAM during the level-loading stage, instead of bringing in frames on the fly. The best-known side effect of this limitation is the "stutter" suffered by some fighting games of the era — *Mortal Kombat* is a frequently cited case — during final battles, when the game needs to load the full animation of a new character entering the scene into VRAM.

On the Mega Drive, however, the fact that animation *can* be streamed from ROM doesn't mean it's always *advisable* to do so. Watching the VRAM debugger while various commercial games run reveals a pattern:

- In *Mortal Kombat* (Mega Drive version) there is constant VRAM activity: each character's animation tiles are continuously streamed from ROM, being replaced frame by frame.
- In *E-SWAT*, the player's animation is streamed the same way, but the enemies' animations are fully preloaded: even if an enemy moves or attacks, there's no new activity in VRAM, because those tiles were already there from the start.
- In *The Revenge of Shinobi* something similar happens and is easy to observe: if the player moves, new tiles enter VRAM and the scroll table shifts; but if the player stands still and only the enemies move, there is no change whatsoever in VRAM, because the enemy ninjas' animations were already preloaded beforehand.

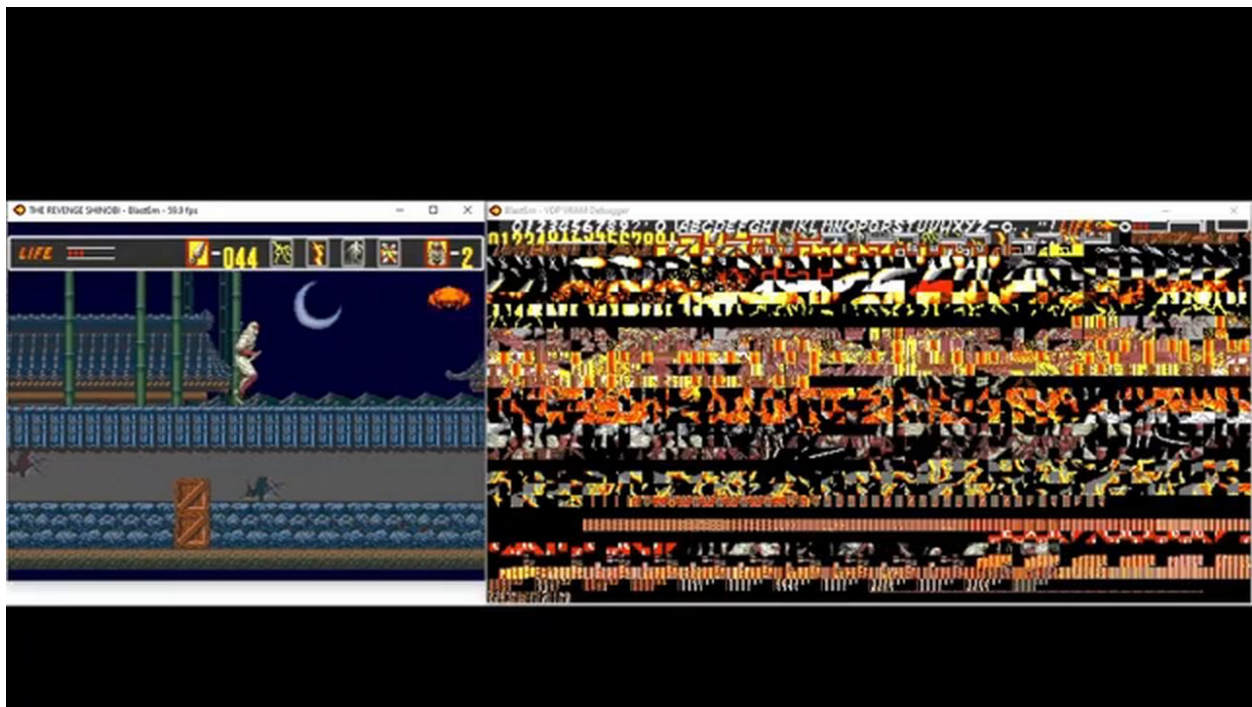


Figure 142: *The Revenge of Shinobi* alongside the VRAM debugger

The Revenge of Shinobi running in BlastEm alongside the VRAM debugger: the player's tiles are streamed live as they move, while the enemies' tiles are already preloaded and don't change even as they move or attack.

The most telling case is that of the rings in *Sonic the Hedgehog*. When many instances of

a ring appear on screen at the same time, they don't each stream their own copy of the animation: it is preloaded a single time, and every visible instance reuses it. In the VRAM debugger those tiles appear colored red instead of the yellow they show on screen, because the debugger doesn't apply the color palette in real time — it only shows the raw pixel pattern as it's stored.

Two reasons to prefer preloading over streaming

1. **Performance:** streaming tiles to VRAM every frame consumes DMA time and CPU cycles; overusing this technique with many sprites at once can cause slow-downs.
2. **VRAM savings:** when the same graphic is needed for several simultaneous instances (rings, clouds, background trees), uploading it once and having all instances point to that same copy is strictly more efficient than letting the sprite engine upload an independent copy for each instance.

2. The problem in concrete terms: four trees, four copies

The sample project adds a third parallax layer between the background and foreground using tree sprites placed manually into the scene: graphically they are just four trees distributed with some irregularity so the scrolling looks natural. The problem appears when you look at VRAM.



Figure 143: Four decorative trees and the VRAM debugger

On the left, the sample program with the four decorative trees over the background. On the right, the VRAM debugger: four identical strips are visible, corresponding to four copies of the same tree graphic.

All four trees use exactly the same graphic. However, since each one was added to the scene with an independent call to `SPR_addSprite()`, the sprite engine uploaded four identical copies of those tiles to VRAM, one per instance. This is exactly the default behavior described in the

previous section: each sprite manages its own tile upload without knowing that another sprite has already loaded the same graphic. The more repeated instances there are on screen, the more VRAM is wasted this way.

3. Modeling reusable background sprites

Before solving the VRAM problem, it's worth looking at how the code that manages these background sprites is organized, since the solution builds directly on that structure. The design closely follows the one typically used for a game's enemies: a header file (`background_sprites.h`) with the data types, and an implementation file (`background_sprites.c`) with the concrete data and functions.

3.1 The data types

Two structures are defined. The first describes each unique background sprite graphic that exists in the game (a tree, for example, and later on clouds and stars):

```
struct BGSpriteType {
    const SpriteDefinition* image;
    Palette palette;
    bool highPriority;
};
```

The `image` field points to the graphic's `SpriteDefinition`. The `palette` field indicates which of the VDP's four color palettes that sprite uses, under the assumption that all instances of a given type share a palette. The `highPriority` field determines whether the sprite is drawn in front of or behind the foreground plane: the trees, clouds, and stars in this chapter use low priority because they must stay behind the foreground, but the field exists because that isn't always the case.

When a background sprite needs high priority

The `highPriority` field is not superfluous: there are legitimate cases where a decorative element must be drawn in front of everything. The arcade version of *Mortal Kombat*, for example, has hanging chains that appear in front of the player in a particular stage to reinforce the sense of depth. The same background-sprite mechanism described in this chapter can be used for that kind of effect just by changing this field.

The second structure describes each concrete instance of a background sprite within a level:

```
struct BGSprite {
    u8 status;
    u8 type;
    fix32 x;
    fix32 y;
    Sprite* sprite;
};
```

`status` indicates whether the instance is active or inactive (unlike enemies, which also have attack, chase, or recovery states, background sprites for now only need these two possibilities). `type` identifies, via an enum, which of the unique types in `BGSpriteType` it corresponds to. `x` and `y` are the position in fixed-point (`fix32`), and `sprite` holds the pointer to the actual instance managed by the SGDK engine once it's created.

3.2 The data: catalog of types and per-level layout

In `background_sprites.c`, two constant arrays are defined (they live in ROM). The first is the catalog of unique background sprite types that exist in the game:

```
const struct BGSpriteType bg_sprites_list[TOTAL_NUM_UNIQUE_BG_SPRITES] = {
    {&bg_sprite_tree, PAL2, FALSE},
    {&bg_sprite_tree, PAL2, FALSE},
    {&bg_sprite_tree, PAL2, FALSE},
    {&bg_sprite_tree, PAL2, FALSE},
    {&bg_sprite_tree, PAL2, FALSE},
};
```

At this stage of the project, all five planned types (tree, cloud, stars, and two more not yet defined) point to the same tree graphic as a placeholder; they will be replaced as the final graphics for each one become available. The second array is the layout of background sprites for each section of the game that actually has any:

```
const struct BGSprite
↪ bg_sprites[TOTAL_NUM_LEVELS_WITH_BG_SPRITES][NUM_BG_SPRITES_PER_LEVEL] = {
    {
        {ACTIVE_BG_SPRITE, CLOUD_BG_SPRITE, FIX32(10), FIX32(10), NULL},
        {ACTIVE_BG_SPRITE, CLOUD_BG_SPRITE, FIX32(100), FIX32(100), NULL},
        {ACTIVE_BG_SPRITE, CLOUD_BG_SPRITE, FIX32(0), FIX32(0), NULL},
        {INACTIVE_BG_SPRITE, CLOUD_BG_SPRITE, FIX32(0), FIX32(0), NULL},
        {INACTIVE_BG_SPRITE, CLOUD_BG_SPRITE, FIX32(0), FIX32(0), NULL},
        // ...
    },
    // ...
};
```

Note the parallel with an eventual enemy system: a list of unique types and a table of per-level positions, both in ROM. Since this data doesn't change at runtime, it's best kept as `const`: the compiler places it in ROM instead of reserving space for it in the scarce working RAM.

Also notice that this level's entries use the type `CLOUD_BG_SPRITE`, even though the graphic it currently maps to in `bg_sprites_list` is the placeholder tree: the level is conceptually cataloged as one that will have background clouds, even though the final cloud graphic doesn't exist yet. This distinction between "which category it belongs to" and "which graphic it uses today" is what allows the graphic to be replaced later without touching the per-level layout.

NUM_BG_SPRITES_PER_LEVEL is not the actual number of active sprites

This constant sets the maximum size of the per-level array (10, in the example), not the number of sprites that will actually be active. The remaining entries are filled with `INACTIVE_BG_SPRITE`. This slack makes it easier to add new sprites to a level without having to resize arrays, at the cost of a bit of unused ROM memory.

3.3 From ROM to working RAM

The data above lives in ROM and is read-only. To manipulate the state of background sprites during gameplay (activating them, moving them, assigning them their Sprite instance), a working copy in RAM is needed:

```
struct BGSprite current_bg_sprites[NUM_BG_SPRITES_PER_LEVEL];
```

which is filled, when each level loads, by the function `copyBGSpriteArray()`:

```

void copyBGSpriteArray() {
    for (u8 i = 0; i < NUM_BG_SPRITES_PER_LEVEL; i++) {
        current_bg_sprites[i] =
            bg_sprites[all_levels[current_game_section].background_sprites][i];
    }
}

```

This function walks through the ROM template corresponding to the current game section (`current_game_section`) and copies it into the working array in RAM.

4. Integration with the level loader

Enemies appear, to a greater or lesser extent, in practically any section of the game. Background sprites are the exception: in a game with hundreds of sections (title screens, menus, levels), perhaps only a handful have background sprites. Reserving a background-sprite array entry for every section, as is done with enemies, would waste ROM in the vast majority of cases where it isn't needed.

For that reason, the `Level` structure (used to describe each game section in the `all_levels` array) includes an additional field:

```

struct Level {
    // ...
    const u8 background_sprites;
};

```

This field takes a value from an enum listing the different background-sprite zones in the game (for example, the zone corresponding to the city or the one for the harbor), with a special value `NO_BG_SPRITES` for sections that have none. The function that loads each level, `levelLoader()`, checks this field before creating the background sprites:

```

void levelLoader() {
    // ...
    createPlayer();
    createEnemies();

    if (all_levels[current_game_section].background_sprites != NO_BG_SPRITES)
        createBGSprites();

    current_game_state = RUNNING_LEVEL;
    // ...
}

```

Why this field is optional and the enemies one isn't

This design asymmetry is not accidental. In a full game with several hundred sections, only a minority tend to include background sprites, while almost all include some enemy. A mandatory field for background sprites, analogous to the one for enemies, would mean reserving an empty `background_sprites` entry in ROM for every section without background sprites — a systematic waste. The sentinel value `NO_BG_SPRITES` solves the problem at a minimal memory cost (a single byte per section) in exchange for an extra check at load time.

The other key element for what follows is the variable `u16 vram_ind`, declared as `extern` in `levels.h`. It keeps track of the position (the tile index) up to which VRAM is occupied as the

various graphic elements of a level are loaded, in order:

- It's initialized to `TILE_USERINDEX`, the first free index after the tiles reserved by the system (see chapter 7).
- It's used to load the background image (`VDP_drawImageEx`) and is then incremented by the number of tiles that image occupied.
- It's used to load the foreground tileset (`VDP_loadTileSet`) and is incremented again.
- From that point on, it's available so that `createBGSprites()` (or any other system that uploads tiles manually) knows from which index it can write without overwriting data already loaded.

vram_ind is a manual counter: getting it wrong throws everything off

Since `vram_ind` is managed by hand, adding the number of tiles used after each load, an accounting error (forgetting to increment it, or incrementing it by too much or too little) causes two different elements to end up writing over the same VRAM region. The symptom is predictable: tiles from one element replacing or mixing with those of another, producing corrupted graphics. Section 7 of this chapter deliberately shows that error so it's recognizable if it happens by accident.

5. The naive version: one copy per instance

With the data model already built, the first version of `createBGSprites()` (the function responsible for putting a level's active background sprites on screen) is as straightforward as possible: copy the level's template and, for each active entry, ask the sprite engine to add it.

```
void createBGSprites() {
    copyBGSpriteArray();

    for (u8 i = 0; i < NUM_BG_SPRITES_PER_LEVEL; i++) {
        if (current_bg_sprites[i].status != INACTIVE_BG_SPRITE) {
            current_bg_sprites[i].sprite = SPR_addSprite(
                bg_sprites_list[current_bg_sprites[i].type].image,
                F32_toInt(current_bg_sprites[i].x),
                F32_toInt(current_bg_sprites[i].y),
                TILE_ATTR(bg_sprites_list[current_bg_sprites[i].type].palette,
                    bg_sprites_list[current_bg_sprites[i].type].highPriority,
                    FALSE, FALSE));
        }
    }
}
```

This code compiles and works: the four trees appear on screen exactly in the expected positions. The problem, as already shown in section 2, is that each call to `SPR_addSprite()` independently uploads the tiles of `bg_sprites_list[...].image` to VRAM, with no instance aware that another has already uploaded exactly the same graphic. Four instances of the same tree produce four identical copies in VRAM.

6. The solution: load once and share the VRAM index

SGDK offers a mechanism designed exactly for this scenario: manually upload a `SpriteDefinition`'s tiles a single time, then tell each `Sprite` instance to use that same copy instead of managing its own. Three pieces work together here: the function `SPR_loadAllFrames()`, which performs the manual load, and the methods `SPR_setAutoTileUpload()` and `SPR_setVRAMTileIndex()`,

which reconfigure a Sprite instance so that it no longer manages its own tiles and instead points to the ones already loaded.

6.1 SPR_loadAllFrames(): uploading the tiles a single time

The function's signature, according to the SGDK documentation, is:

```
u16 SPR_loadAllFrames(const SpriteDefinition *sprDef, u16 index, u16
↳ *totalNumTile);
```

sprDef is the SpriteDefinition whose tiles are to be uploaded to VRAM; index is the position (the tile index) where they'll be written; totalNumTile, if not NULL, lets you retrieve how many tiles in total were occupied by all the loaded animation frames. The function also returns a dynamically allocated index table that reveals the VRAM position of each frame of each animation; since this chapter doesn't need to manipulate animations frame by frame, that return value and the totalNumTile parameter are set aside for now (passing NULL) and revisited later in this manual.

This function can take several frames: call it only during loading

The SGDK documentation itself warns about this: SPR_loadAllFrames() can take quite a while (several game frames) because it uploads via DMA the entirety of the tiles from every animation in the given SpriteDefinition. For that reason it must be called during initialization or level loading, never inside the game's main loop.

In the project's code, this call is added a single time, before the loop that iterates over the background sprites: unlike SPR_addSprite(), this load must not be repeated for each instance.

```
void createBGSprites() {
    copyBGSpriteArray();

    SPR_loadAllFrames(bg_sprites_list[current_bg_sprites[0].type].image,
                      vram_ind = vram_ind + 10, NULL);

    for (u8 i = 0; i < NUM_BG_SPRITES_PER_LEVEL; i++) {
        if (current_bg_sprites[i].status != INACTIVE_BG_SPRITE) {
            current_bg_sprites[i].sprite = SPR_addSprite(/* ... as before ... */);

            SPR_setAutoTileUpload(current_bg_sprites[i].sprite, FALSE);
            SPR_setVRAMTileIndex(current_bg_sprites[i].sprite, vram_ind);
        }
    }
}
```

Two details of this call deserve explanation. The first is where the graphic to load comes from: rather than writing the tree graphic's name literally, the code takes bg_sprites_list[current_bg_sprites[0].type].image, i.e., the graphic used by the array's first active entry. This keeps the code a bit more flexible than a hardcoded value, though it only works correctly as long as all the background sprites in a given level share a single graphic, which is the case in this example.

This technique assumes a single graphic shared per level

Taking the graphic from current_bg_sprites[0] is only correct if, within a given level, all background sprites use the same SpriteDefinition. If a level needed to mix, say, trees and clouds with different graphics at the same time, this scheme

would need to be extended by calling `SPR_loadAllFrames()` once for each distinct graphic present in the level, rather than once for the whole array.

6.2 The spacing margin in `vram_ind`

The second detail is the addition of 10 positions to the accumulated value of `vram_ind` before loading the trees. As explained in section 4, `vram_ind` has already been incrementing since `levelLoader()` as the level's background and foreground are loaded. That 10-tile margin serves no technical purpose: it's a deliberate gap so that, when looking at the VRAM debugger, a clear blank space is visible between the end of the foreground tiles and the start of the tree tiles, making the effect easier to follow. In a real project that margin can be removed without any issue, simply leaving `SPR_loadAllFrames(..., vram_ind, NULL)` right after the previous loads.

Care should be taken with expressions like `vram_ind = vram_ind + 10` used directly as a function argument: they're valid in C, but mixing an assignment with a side effect inside an argument list can be confusing if someone later modifies that line without noticing it's also reassigning a global variable. It's preferable to put the assignment on its own line before the call, as was implicitly done here by explaining the reason for the increment.

6.3 `SPR_setAutoTileUpload()` and `SPR_setVRAMTileIndex()`: reconfiguring each instance

With the tiles already uploaded a single time, each Sprite instance created with `SPR_addSprite()` still needs to be told not to manage its own copy of tiles but instead to use the one already in VRAM. This is achieved with two additional calls, placed inside the loop, one per active background sprite:

```
void SPR_setAutoTileUpload(Sprite *sprite, bool value);
```

The value parameter set to `TRUE` (the default behavior) tells the sprite engine to keep uploading this sprite's tiles automatically; set to `FALSE`, it tells the engine that responsibility now rests with the programmer. That is exactly what's needed here: it's called with `FALSE` so the engine stops trying to upload its own tiles for this instance.

The second call, `SPR_setVRAMTileIndex(sprite, index)`, tells the engine which VRAM position to look at for that instance's tiles when drawing it, instead of the position the engine would have chosen on its own. The value passed to it must be exactly the same index used earlier in `SPR_loadAllFrames()`:

```
SPR_setAutoTileUpload(current_bg_sprites[i].sprite, FALSE);  
SPR_setVRAMTileIndex(current_bg_sprites[i].sprite, vram_ind);
```

The two indices must match exactly

If the index used in `SPR_setVRAMTileIndex()` doesn't match the one used in `SPR_loadAllFrames()`, the instance will end up displaying on screen whatever tiles happen to be at that other VRAM position, which could belong to any other element of the game, instead of the correct graphic. It's a silent error: the program compiles and runs without failing, but the visual result is wrong.

Why keep using `SPR_addSprite()` if VRAM is managed by hand?

It might seem that, if automatic tile uploading is going to be disabled anyway, there's no need to go through `SPR_addSprite()`. However, that call is still necessary: it's what reserves the Sprite instance within the engine, assigns it a hardware slot, incorporates it into the priority/depth ordering, and makes it available for the rest

of the engine's functions (position, animation, flipping). The only thing being taken over is VRAM management, not the rest of the sprite's lifecycle (see chapter 21).

6.4 The result

With these changes (one call to `SPR_loadAllFrames()` before the loop and two additional calls inside it), the project is recompiled and the VRAM debugger is checked again. On screen the result is visually identical to the one in section 2: the same four trees, in the same positions. The difference is in VRAM, where there is now a single copy of the tree tiles instead of four.

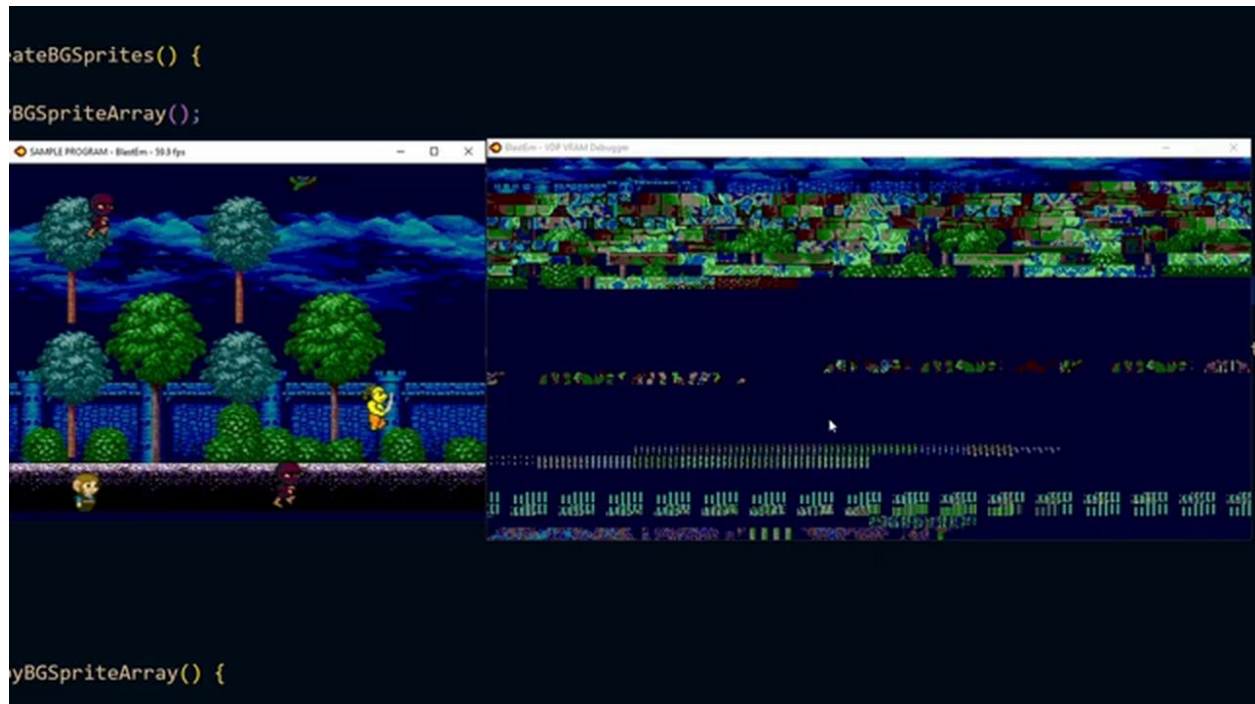


Figure 144: VRAM debugger after sharing the tree tiles

Same sample program after the change: where there used to be four identical strips of tree tiles, there is now only one, and the four instances visible on screen share that single copy.

That difference is exactly the space recovered for other elements of the game (more sprites, larger tilesets, other backgrounds) to use. It's the same VRAM-optimization logic already worked on from a different angle in chapter 47 when reducing the number of unique tiles in a tileset: there, the amount of data to upload was reduced; here, uploading data that was already there multiple times is avoided.

7. Choosing the right VRAM position

Sharing a single copy of tiles across several instances solves much of the problem, but it isn't the only relevant decision: where, within VRAM, that copy is placed also matters. The VDP's VRAM has 2048 tile positions in total; BlastEm's VRAM debugger represents them as a grid of 32 rows of 64 tiles each. Not all of those positions are free: part is occupied by the scroll table and the sprite table, whose size depends on how large the level is and what type of scroll is used.

To illustrate the risk of choosing a bad position, the sample project first tries setting `vram_ind` to 264, which places the tree tiles at the start of the second row of the debugger's grid. The result isn't what's expected: since that region was already occupied by background tiles, they get overwritten, and the background clouds appear corrupted, showing fragments of the tree graphic instead of their original graphic.

In the VRAM debugger that strip of tree tiles appears right where background data used to be, and on screen the effect is just as direct: the clouds lose their original graphic and show recognizable fragments of the tree in its place.

A carelessly chosen VRAM index corrupts other graphics

This experiment deliberately reproduces the error warned about in section 4: manually writing tiles to a VRAM position already in use by another element produces no compilation or runtime error, but it does visually corrupt whatever was stored there. Before choosing a VRAM index by hand, it's important to first check, via the debugger, which range of positions is actually free.

The more productive alternative is to take advantage of the gaps: the tiles left unused between the end of the background/foreground data and the start of the scroll table and sprite table, at the bottom of VRAM. How much free space there is depends on the level's size and scroll configuration, but there's usually some margin. The sample project tries placing the trees at position 1984 (2048 minus 64, i.e., the start of the grid's last row) and manages to fit them into a space that was previously unused, without overwriting any other data.

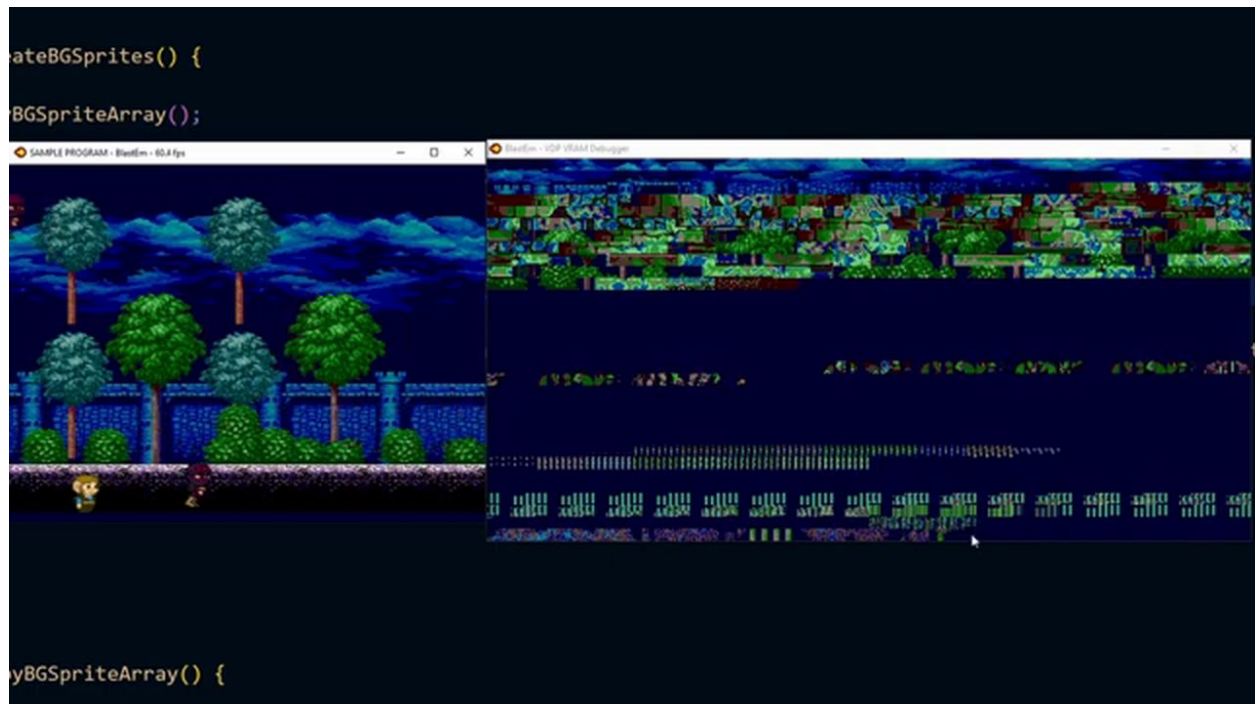


Figure 145: Tree tiles relocated to the end of VRAM

The tree tiles relocated to position 1984, at the start of the debugger grid's last row: a gap that used to sit unused between the sprite table and the scroll table.

This strategy has a double advantage: not only is a single copy of tiles shared across instances (section 6), but that copy is also placed in a space that would otherwise go to waste, leaving more tiles available both for the background and foreground and for the rest of the sprites that do use SGDK's automatic engine. This idea of "fitting" manual data into VRAM's free

gaps is a technique revisited later in this manual for other interface elements, such as health hearts or sub-weapon icons.

Review exercises

1. Explain in your own words why four sprite instances that share the same graphic can end up occupying four separate copies in VRAM if they're created with `SPR_addSprite()` without any further modification.
2. What's the difference between the loading `SPR_addSprite()` performs by default and the one `SPR_loadAllFrames()` performs? Mention at least one advantage and one limitation of using the latter.
3. Suppose that, due to a typo, `SPR_setVRAMTileIndex()` is called with an index different from the one used in `SPR_loadAllFrames()` for the same graphic. Will the program fail to compile or to run? What visual symptom would you expect?
4. The counter `vram_ind` is incremented by hand after each tile load in `levelLoader()`. Propose an alternative design that reduces the risk of someone forgetting to increment it correctly when adding a new graphic element to the level.
5. Describe a situation where it would not be advisable to apply this chapter's technique (sharing a single copy of tiles across several sprite instances). Justify your answer by considering what would happen if those instances needed to animate independently.

Chapter 49 — Profiling and CPU Load

How to know, while the game is running, whether a frame's work fits into the time available

The previous three chapters of this part each dealt, from a different angle, with the same resource: video memory. Chapter 44 showed how DMA moves data into VRAM without stalling the CPU; chapters 45 and 46 explained how much VRAM is actually available and how to split it between background and sprites; chapter 47 showed how to reduce the number of unique tiles a graphic needs in order to fit into that space. This chapter, the last in Part VIII, shifts the axis: instead of asking how much memory something takes up, it asks how much CPU time it consumes. These are two different questions — a game can have plenty of VRAM to spare and still run slowly, or it can have plenty of free CPU time to spare and still not fit in memory — but both answer the same underlying need: making the game work within the hardware's real limits, not just within the limits that happen to be convenient to program against.

Prerequisites

Before continuing, it helps to keep in mind:

- The main loop and the role of `SYS_doVBlankProcess()` in synchronizing with the VDP, introduced in chapter 5.
- Emulators with development tools — Gens KMod, BlastEm — and their VDP viewers and 68000 debuggers, introduced in chapter 4.
- DMA as a mechanism for moving data to VRAM without blocking the CPU (chapter 44), and managing VRAM limits (chapters 45 through 47), as a reference for the other half of optimizing a Mega Drive game.
- Bit shifting as an alternative to multiplication and division, seen when calculating parallax depth for a background in chapter 25.
- Collision arrays traversed frame by frame, introduced in Part VII (chapters 40 through 43).

1. A frame's budget: what “frame budget” means

A console doesn't draw a continuous image: it draws a succession of complete frames, one after another, at a fixed rate imposed by the video hardware itself. On a Mega Drive configured for NTSC output, that rate is 60 frames per second; on one configured for PAL, it's 50. That figure isn't a performance target to aim for: it's literally the rate at which the VDP sweeps the screen, the same regardless of how simple or complex the scene is.

A direct calculation follows from that figure. At 60 frames per second, each frame has 1/60 of a second — about 16.7 milliseconds — to complete. At 50 frames per second, that margin

risers to 20 milliseconds. That interval — the **frame budget** — is all the time the program has to do absolutely everything a frame needs: read the controls, update the game logic, resolve physics and collisions, decide what needs to be redrawn, and get all of that work ready before the VDP starts drawing the next frame.

Chapter 5 already introduced the piece of code that embodies that limit: `SYS_doVBlankProcess()`, called once per loop iteration, which synchronizes pending work with the VDP's vertical blanking interval — the brief span between one drawn frame and the next during which it's safe to modify certain graphics data. That call isn't a mere formality: it's, in practice, the point where each frame's budget gets settled. If all the work in one iteration of the main loop — reading input, moving characters, resolving collisions, updating animations — finishes before the next vblank arrives, `SYS_doVBlankProcess()` simply waits out whatever time is left and the game keeps running at 60 (or 50) frames per second without anything being noticeable. But if that work stretches past the vblank it was supposed to meet, `SYS_doVBlankProcess()` ends up waiting for the *next* vblank, a full frame later: the game drops an entire frame, the sense of smoothness suffers, and if this happens persistently, the actual game speed perceived by the player falls below the nominal 60 (or 50) frames per second.

Dropping a frame is not the same as running “a bit slower.” A game that occasionally goes over its budget drops isolated frames, which generally reads as a small, one-off jolt. A game that systematically goes over its budget — for example, because a poorly written collision routine inherently takes more than 16.7 milliseconds in a level with many enemies — ends up running at a constant fraction of its nominal speed (30 frames per second instead of 60, for instance), and any animation or movement that isn't designed to be independent of frame rate starts to look and feel different depending on how many elements are on screen at a given moment.

Within that frame budget there is, moreover, an even narrower margin: the vblank interval itself, just a fraction of the whole frame, is the only window in which it's safe to write to VRAM without risking corrupting whatever the VDP is drawing at that instant. This is exactly where the DMA covered in chapter 44 comes into play: the faster VRAM transfers complete, the more room is left within that narrow window to queue up graphics updates without having them cut off halfway through. The frame budget and the vblank window are, in that sense, two sides of the same timing problem: one bounds how long the game's complete logic can take, the other bounds specifically how long writes to VRAM can take.

2. Measuring CPU load: `SYS_getCPULoad()` and `VDP_showCPULoad()`

Knowing that a frame budget exists is useful as a concept, but it says nothing about whether a specific project is respecting it. For that, SGDK offers a way to estimate, frame by frame and while the game is running, what percentage of that budget is actually being used.

`SYS_getCPULoad()` returns a value that estimates the CPU load for the most recent frame, expressed as a percentage: SGDK measures how much time is left idle while waiting for vblank and compares it against the total time available in that frame, so that a value close to 100% indicates that nearly the entire frame budget is being used, while a value well above 100% indicates that the frame is going over its budget and, consequently, frames are being dropped.

Checking that number manually, frame by frame, isn't very practical during normal development, so SGDK also offers a way to display it directly on screen: `VDP_showCPULoad()`, which draws that percentage as overlaid text, using the same default font already used to display any other text on screen (chapter 5). As with any on-screen debugging information, it's best to call it once per iteration of the main loop, typically right alongside the call to

```
SYS_doVBlankProcess();
```

```
while (1)
{
    // input reading, game logic,
    // sprite and animation updates...

    VDP_showCPULoad(1, 1);    // <!-- VERIFY: exact parameter names -->

    SYS_doVBlankProcess();
}
```

With that overlay active, it's enough to play the project's most demanding level — the one with the most enemies on screen, the most elaborate parallax background, the explosion with the most particles — and watch how the number reacts: if it stays stable and comfortably below 100%, there's margin to spare; if it starts approaching or exceeding 100% in the most demanding situations, it's time to investigate which part of the frame is consuming that excess, before the problem becomes noticeable to the player.

3. Measuring real frames per second: VDP_showFPS()

The CPU load percentage is an internal estimate, calculated from how much idle time is left before vblank. A direct and more intuitive complement to that estimate is to directly measure how many frames per second are actually being drawn in practice: if the nominal target is 60 (NTSC) or 50 (PAL) and the counter shows a consistently lower number, that's the most concrete confirmation possible that frames are being dropped, with no need to interpret any percentage.

SGDK offers `VDP_showFPS()` for this purpose, with the same usage pattern as `VDP_showCPULoad()`: it's called once per iteration of the main loop and draws the frames-per-second count directly on screen using the default font.

```
while (1)
{
    // game logic...

    VDP_showFPS(TRUE);    // <!-- VERIFY: exact parameter name and meaning -->

    SYS_doVBlankProcess();
}
```

In some projects it may be useful to get that number without drawing it directly on screen — for example, to average it over several frames, log it into a custom variable, or display it in a format other than the one the default function offers. For that specific case, SGDK also exposes a function that only calculates the value without drawing anything, along the lines of `SYS_getFPS()`, although it's worth verifying its exact name and return type against one's own SGDK 2.11 installation before relying on it in production code.

Used together, the CPU load percentage and the frames-per-second counter answer different, complementary questions: the first indicates *how much margin* is left within each frame's budget, even when that margin is still enough to sustain the nominal 60 (or 50) frames per second; the second confirms, unambiguously, whether those nominal frames are actually being achieved. It's worth keeping both overlays active during much of development — and removing them, along with any other on-screen debugging tool, before generating a final build of the game — rather than turning them on only once a performance problem is already

suspected: by the time the problem is noticeable just by playing, it has usually been present for quite a while.

4. Emulators as a complement: visual profiling with BlastEm and Gens KMod

The CPU load percentage and the frames-per-second counter tell you *whether* a frame is going over budget, but they don't automatically tell you *which part of the code* is responsible. For that second step, the development tools of the emulators introduced in chapter 4 are a direct complement.

Both BlastEm and Gens KMod include a 68000 debugger capable of pausing execution, stepping instruction by instruction or frame by frame, and placing breakpoints at specific code addresses. Combining that capability with the `VDP_showCPULoad()` overlay itself makes it possible to narrow down the problem fairly precisely: if the load percentage spikes right when an explosion with lots of particles triggers, or right when entering a room with a certain number of enemies, pausing execution at that instant and stepping through the debugger lets you observe exactly which routine is running when the time runs out, instead of having to guess by reviewing the code blindly.

The VDP viewer in both emulators, already used in chapters 45 and 46 to visually inspect VRAM, plays a different but related role here: it doesn't measure CPU time, but it does let you confirm with certainty how many VRAM transfers are happening in a given frame and how large they are — a useful piece of data for deciding whether the real bottleneck is on the computation side (the CPU resolving logic) or on the data-transfer-to-VRAM side (a problem revisited in the next section). No emulator reproduces real hardware timings with perfect accuracy, so the performance figures they produce should be treated as a solid guide for finding the bottleneck, not as a certified measurement of behavior on physical console hardware.

5. What typically eats into a frame's budget in practice

With the measurement tools already introduced, it's worth reviewing the usual suspects in a real Mega Drive game when the CPU load percentage starts approaching the limit. This manual has already presented, in earlier chapters, the concrete solution for each of these three cases; what this chapter adds is the judgment needed to recognize them as the real cause of a performance problem, rather than treating them as an isolated technical curiosity.

Avoidable multiplications and divisions. The Mega Drive's Motorola 68000 resolves a multiplication or a division in considerably more cycles than an addition, a subtraction, or a bit shift. Chapter 25 already showed the concrete case of dividing a parallax background's position to achieve a sense of depth: replacing that division with a bit shift (`>>`) when the divisor is a power of two — 2, 4, 8, and so on — achieves exactly the same numeric result at a fraction of the cost. Any calculation that repeats once per frame, and even more so if it repeats once per sprite or per active entity on screen, is a natural candidate to review for multiplications or divisions that could be replaced with a lighter operation.

VRAM transfers without using DMA. Chapter 44 showed that copying data to VRAM by driving the CPU directly — instead of delegating that copy to the DMA — keeps the CPU fully occupied for the entire transfer, unable to do any other work in the meantime. A large transfer done this way, especially if it happens within the narrow vblank window described in section 1, can by itself consume a considerable portion of that frame's budget. DMA frees up the CPU during the transfer and, by completing it faster thanks to hardware dedicated to that task, leaves more margin available for the rest of that frame's work.

Traversing large collision arrays every frame. Part VII of this manual (chapters 40 through 43) built level collision from an array that represents, tile by tile, which areas of the map are walkable. Checking that array for the few positions relevant to a character — the corners of its collision box, nothing more — is a cheap operation; traversing the entire array, or checking it for far more positions than strictly necessary, once per active entity and once per frame, is a pattern that scales poorly: the cost grows with the number of enemies or objects on screen, exactly the kind of growth that can go unnoticed with a handful of test enemies and only become a real problem once a finished level packs in many at once — the scenario revisited in Part IX of this manual.

None of these three cases is, by itself, a reason to rewrite code preemptively: optimizing before measuring risks complicating code that wasn't actually responsible for the problem. The correct approach is the reverse of how these three sections are ordered: first measure with `VDP_showCPULoad()` and `VDP_showFPS()`, then confirm with the emulator's debugger which specific routine fires exactly when the percentage rises, and only then apply the appropriate technique — bit shifting, DMA, or narrowing an array traversal — at the actual point of the problem.

Review exercises

1. Calculate how many milliseconds of budget a frame has in NTSC (60 frames per second) and in PAL (50 frames per second), and explain exactly what happens when a frame's logic and drawing exceed that budget.
2. What's the difference between what `SYS_getCPULoad()`/`VDP_showCPULoad()` measure and what `VDP_showFPS()` measures, and why is it worth using both tools together during development instead of relying on just one?
3. Explain why an emulator's debugging tools (chapter 4) are a good complement to SGDK's CPU load overlay, and not a substitute: what information does each provide that the other can't offer on its own?
4. Name three typical operations in a Mega Drive game that tend to eat up a good part of a frame's budget, and for each one indicate the technique already covered in this manual for mitigating it.
5. A project has plenty of free VRAM space to spare (chapters 45 through 47), but the `VDP_showFPS()` counter shows sustained readings below 60 frames per second in NTSC. Explain why reducing VRAM usage even further won't solve this problem, and what steps should be taken instead.

Chapter 50 — VDP Planes and Enemies in a Beat 'em Up

Why an enemy placed only once in the code can end up appearing repeated several times on a large map, and how to avoid it by combining a custom positioning function with `SPR_setVisibility()`

This part of the manual takes on a longer project than any of the previous ones: over the next thirteen chapters, a side-scrolling action game with enemies on screen will be built step by step, in the style of *Streets of Rage*: a simple but mechanically complete beat 'em up, which serves as an excuse to introduce code organization topics —header files, structs, enums, function pointers, game states— that haven't been needed until now in smaller projects. The starting point is the same world that the previous chapters have been building: a large scrolling map, a camera that follows the player, and a parallax background behind everything. The only thing missing to start populating it is, precisely, the subject of this chapter: enemies.

The first obstacle that shows up when adding an enemy to a large map has nothing to do with artificial intelligence or combat, but with something more basic and, at first glance, baffling: an enemy placed only once in the code, with a single fixed position, can end up appearing repeated several times throughout the level. The cause has to do with the size of the VDP planes and with how the Mega Drive handles scrolling internally, and the solution combines a custom sprite positioning function with a constant check of whether each enemy falls within the camera's visible bounds.

Prerequisites

Before continuing, it helps to have:

- Chapter 25 (“Parallax scrolling on top of a camera”) already done: this chapter picks up that same project, with a foreground map on BG_A, a parallax background on BG_B, and the `camera()` function already working.
- The `fix32` type and the `F32_toInt()`/`F32_fromInt()` conversions, covered in Chapter 6.
- Chapter 27 (“Color transparency without Shadow and Highlight”), where `SPR_setVisibility()` was introduced along with the `VISIBLE` and `HIDDEN` values of the `SpriteVisibility` type.

1. A brief graphical analysis: how *Streets of Rage* achieves so much with so little

Before diving into code, it's worth pausing for a moment on the graphics of *Streets of Rage*, one of the most memorable beat 'em ups on the console, because they explain a good part of the art decisions worth making in a project of this genre. Zooming in on a brick in a wall

or a tile on a sidewalk, the pattern turns out to be surprisingly simple and repetitive, with very few colors; zooming back out, on the other hand, the whole thing looks solid and conveys the sense of a dirty, worn-out street typical of the genre, without ceasing to be colorful. The trick lies in sprinkling that repetitive base with unique elements —signs, graffiti, loose bricks, bent fences— that break up the visual monotony without spiking VRAM consumption: most of the screen is still made up of a handful of tiles reused over and over, and only a handful of additional tiles, placed thoughtfully, is enough for the result to not look repetitive at a glance.



Figure 146: Streets of Rage brick pattern zoomed in in Aseprite

The same brick pattern, zoomed in: up close it's barely a few repeated colors, but on the full screen it forms a solid, dirty wall.

This same idea —few base tiles, plus a small number of one-off variations— is one worth keeping in mind when preparing level graphics for this part of the manual's project, and it will be revisited in more detail in later chapters specifically devoted to preparing level graphics.

2. The problem: a single enemy that repeats across the whole map

On top of the stage already built in the previous chapter, the logical first step is to add an enemy: a boss (`boss_sprite`), with its own X and Y position in `fix32` coordinates (so it can later be combined with movement physics, just like the player), added to the screen after the player's sprite and updated inside the `camera()` function in exactly the same way as the player:

```
// Enemy global variables (added alongside the player's)
static Sprite* boss_sprite;
static fix32 boss_x = FIX32(900);
static fix32 boss_y = FIX32(160);
```

```
// Inside main(), after adding the player sprite
boss_sprite = SPR_addSprite(&boss_sprite_def, F32_toInt(boss_x),
    ↪ F32_toInt(boss_y),
    TILE_ATTR(PAL3, FALSE, FALSE, FALSE));

// Inside camera(), alongside the player's update
SPR_setPosition(boss_sprite, F32_toInt(boss_x) - new_camera_x, F32_toInt(boss_y) -
    ↪ new_camera_y);
```

The code compiles and, on the surface, everything works: the boss appears in its position and moves correctly along with the camera. The problem shows up when traveling a good stretch of the map to the right: that same enemy appears again, over and over, even though there is only a single instance of `boss_sprite` in the code, with a single fixed position.



Figure 147: The boss appearing next to the player despite being defined only once in the code

The boss (on the left) next to the player, at a point on the map where it shouldn't be according to its fixed position in the code.

3. The cause: plane size and scroll that repeats

The explanation goes back to something already seen in the chapter on parallax scrolling: when the scroll coordinate of a background plane advances far enough in one direction, that plane starts repeating from the beginning. The exact point at which the screen starts repeating depends on the plane's size, configurable with `VDP_setPlaneSize()`. By default, that size is 64 tiles wide by 32 tiles tall, which equals 512×256 pixels —precisely the maximum limit of an `IMAGE` resource mentioned in the previous chapter. And what happens to the background also happens, in practice, to any sprite positioned in map coordinates: once that repetition limit is reached, the scroll engine shows the same content again from the origin, and the en-

emy gives the impression of having been placed again, even though there's still only a single instance of it in the code.

`VDP_setPlaneSize()` takes the width and height of the plane in tiles (with possible values of 32, 64, or 128) and a third boolean parameter related to the internal VRAM addressing mode. A plane 128 tiles wide, for example, spans 1024 pixels: quite a bit more than the 320-pixel screen width in high-resolution mode, but the repetition point still exists, just pushed farther away.

4. Does changing the plane size solve the problem? No, and it also has its own drawbacks

Widening the plane to 128 tiles with `VDP_setPlaneSize(128, 32, TRUE)` looks, at first glance, like a reasonable solution. In practice, the result visibly scrambles both the background and the foreground—the map data already loaded doesn't fit the same way into a plane of a different size—and as for the enemy repetition, the change makes no difference at all: plane size affects background layers, not sprite positions. Furthermore, even if it worked, increasing the plane size requires more VRAM for the plane itself, so it wouldn't be a good fit for this particular problem either way.

Even so, `VDP_setPlaneSize()` does have a legitimate and fairly specific use: intro or cutscene scenes where graphical elements need to be kept off screen before sliding them in. Consider, for example, a scene where several different characters slide down from the top of the screen, one by one, before the level starts. With the default plane size (256 pixels tall), there isn't enough vertical space to keep all those characters hidden above the screen before the animation. The solution, in a case like that, is to increase the plane's height from 32 to 64 tiles while keeping the width at 64:

```
VDP_setPlaneSize(64, 64, TRUE); // 512x512 instead of 512x256, for cutscenes
```

This gives a 512×512 pixel space instead of the usual 512×256, enough vertical margin to hide several elements before bringing them down one by one via vertical tile scrolling, a technique revisited in a later chapter. The practical takeaway is that changing the plane size is best reserved for this kind of one-off cutscene or intro, where extra space is needed to hide elements off camera; for the game itself, it's best to leave it at its default size and solve enemy repetition another way.

5. Revisiting `SPR_setPosition()` and why a custom function is needed

With the plane-size option ruled out, the problem needs to be tackled from another angle. The function `SGDK` provides to position a sprite, `SPR_setPosition()`, takes three pieces of data: the sprite to move and its X and Y coordinates as signed 16-bit integers (`s16`). The problem is that the position of the player and the enemies in this project is handled as `fix32`, in order to represent fractional positions—essential as soon as jump or smooth-movement physics are added, as seen in previous chapters—so it needs to be converted to an integer with `F32_toInt()` before it can be used with `SPR_setPosition()`. In addition, the actual on-screen position of any sprite has to subtract the camera position (`new_camera_x/new_camera_y`): without that adjustment, the sprite stays fixed at the same point on the screen no matter where the camera moves, instead of moving along with the rest of the stage.

Repeating that conversion and subtraction by hand every time an enemy needs to be positioned works, but it's exactly the kind of repetitive code that's worth wrapping in a cus-

tom function, especially considering that this project will eventually have several enemies on screen at once.

6. Writing a custom function: `set_enemy_position()`

The solution is to write a custom function, similar in spirit to `handle_input()` or `camera()`, that takes the enemy's `fix32` coordinates directly —instead of requiring the conversion to integer before calling it— and does that conversion internally. While at it, it's worth having the function also take the width and height of the sprite in question, since those two values are needed later on to solve the visibility problem, and different enemies can have different sprite sizes. The prototype, declared before `main()` alongside the project's other custom functions, looks like this:

```
static void set_enemy_position(Sprite* enemy_sprite, fix32 enemy_x, fix32 enemy_y,
    s16 enemy_width, s16 enemy_height);
```

It's worth remembering a point about variable scope: the names used as function parameters —`enemy_sprite`, `enemy_x`, `enemy_y`, `enemy_width`, `enemy_height`— only exist inside that function. Trying to use them anywhere else in the code, such as inside the main loop, produces a compilation error stating that the variable is undefined. Any variable that needs to be used across several different functions, on the other hand, has to be declared globally, outside of any function.

Inside the body of the function, the first step is to convert the received `fix32` coordinates to `s16` integers, with descriptive names that make clear they've already gone through the conversion:

```
static void set_enemy_position(Sprite* enemy_sprite, fix32 enemy_x, fix32 enemy_y,
    s16 enemy_width, s16 enemy_height) {
    s16 enemy_x_as_s16 = F32_toInt(enemy_x) - new_camera_x;
    s16 enemy_y_as_s16 = F32_toInt(enemy_y) - new_camera_y;

    SPR_setPosition(enemy_sprite, enemy_x_as_s16, enemy_y_as_s16);
}
```

7. A second scope problem: the camera variables

When trying to subtract `new_camera_x` and `new_camera_y` inside the new function, the same kind of error shows up as with the parameters: those two variables had, until now, been declared inside the `camera()` function itself, so they only exist there and can't be used inside `set_enemy_position()`. The solution is the same as in any other case of this kind: move the declaration of `new_camera_x` and `new_camera_y` outside of `camera()`, to the global variables area near the beginning of the file, alongside `current_camera_x` and `current_camera_y`, to which they are related:

```
// Camera variables (now global, no longer local to camera())
static s16 current_camera_x;
static s16 current_camera_y;
static s16 new_camera_x;
static s16 new_camera_y;
```

The body of `camera()` doesn't need any other changes: since those two variables already existed under those names inside the function, pulling their declaration out and leaving it

outside doesn't change the logic at all, it just widens their scope so that any function in the program—including `set_enemy_position()`—can read them.

With this change, the function compiles without errors. Calling `set_enemy_position()` with the boss's coordinates instead of the manual call to `SPR_setPosition()` from section 2, the visual result is identical to what it was at the beginning of the chapter: the enemy moves correctly along with the camera, but the original problem—that it keeps reappearing several times while traveling across the map—still isn't solved, because this function, for now, is simply doing exactly the same thing as before, just accepting `fix32` directly and handling the conversion and camera adjustment itself.

8. The real solution: hiding the enemy when it falls outside the camera

The solution to the underlying problem is to take advantage of `SPR_setVisibility()`, already used in Chapter 27 for the flicker effect, to hide each enemy as soon as its on-screen position—already adjusted for the camera—falls outside the visible bounds, and to show it again as soon as it re-enters those bounds. The logic boils down to four checks inside a single `if`, joined with `||` (logical “or”): if the enemy's on-screen X coordinate is less than 0, or greater than the screen width, or if the Y coordinate is less than 0, or greater than the screen height, the enemy is off camera and must be hidden; otherwise, it must be shown:

```
static void set_enemy_position(Sprite* enemy_sprite, fix32 enemy_x, fix32 enemy_y,
    s16 enemy_width, s16 enemy_height) {
    s16 enemy_x_as_s16 = F32_toInt(enemy_x) - new_camera_x;
    s16 enemy_y_as_s16 = F32_toInt(enemy_y) - new_camera_y;

    if (enemy_x_as_s16 < 0 || enemy_x_as_s16 > HORIZONTAL_RESOLUTION ||
        enemy_y_as_s16 < 0 || enemy_y_as_s16 > VERTICAL_RESOLUTION) {
        SPR_setVisibility(enemy_sprite, HIDDEN);
    } else {
        SPR_setVisibility(enemy_sprite, VISIBLE);
        SPR_setPosition(enemy_sprite, enemy_x_as_s16, enemy_y_as_s16);
    }
}
```

Note the use of `HORIZONTAL_RESOLUTION` and `VERTICAL_RESOLUTION`, the same constants already defined in the camera chapter (320 and 224 in high-resolution mode), instead of writing those numbers directly into the condition: if the project were to switch resolution mode later on, it's enough to update those two constants in a single place. It's also worth paying attention to the comparison operator (`==`, with two equal signs, not one) and being careful, when writing the four checks, not to mix up an X for a Y—an easy mistake to make when copying and pasting the same condition four times. And if the project used the low-resolution mode (256 pixels wide instead of 320), the horizontal limit adjusts on its own, because it now depends on the constant rather than on a fixed number.

Compiling with this logic, the result is already fairly good: the enemy stops repeating across the whole map and only appears once, exactly where the code places it.

9. A subtle bug: the enemy disappears before actually leaving the screen

Moving the camera carefully, however, reveals a small flaw: on the left side of the screen, the enemy sprite disappears an instant before actually leaving the visible area, leaving a noticeable gap. This is a common behavior in games from earlier generations, but it looks out of place in a 16-bit game.

A good trick to clearly reproduce this flaw is to move the enemy's starting position much farther away on the map, so there's plenty of room to slowly approach it with the camera and observe the exact moment it appears or disappears.

10. The cause and the solution: using the sprite's width and height as the margin, not 0

The explanation for the flaw lies in how the Mega Drive positions any sprite: its coordinates always correspond to the top-left corner of the animation frame. If the enemy moves left and its on-screen X coordinate reaches 0, at that moment its left edge exactly coincides with the edge of the screen, but a good part of its body —the sprite's entire width— is still visible to the right of that corner. Since the condition hides the sprite as soon as its X is less than 0, the enemy disappears too soon, while it should still be visible on screen.

The solution is to replace that limit of 0 with a negative value equal to the sprite's own width, so that the enemy remains visible as long as any part of it —including its right edge— is still within the visible area, and only gets hidden once it has completely left the screen. The same reasoning applies to the top limit: if the map included vertical scrolling, the enemy would disappear too soon at the top as well, unless that limit is replaced with the sprite's negative height instead of 0. Although this particular project's map only scrolls left to right for now, it's worth applying the same adjustment to the vertical limit to keep the function ready for any kind of map in the future. This is precisely why `set_enemy_position()` takes `enemy_width` and `enemy_height` as parameters, instead of having those values hardcoded inside the function:

```
static void set_enemy_position(Sprite* enemy_sprite, fix32 enemy_x, fix32 enemy_y,
    s16 enemy_width, s16 enemy_height) {
    s16 enemy_x_as_s16 = F32_toInt(enemy_x) - new_camera_x;
    s16 enemy_y_as_s16 = F32_toInt(enemy_y) - new_camera_y;

    if (enemy_x_as_s16 < -enemy_width || enemy_x_as_s16 > HORIZONTAL_RESOLUTION ||
        enemy_y_as_s16 < -enemy_height || enemy_y_as_s16 > VERTICAL_RESOLUTION) {
        SPR_setVisibility(enemy_sprite, HIDDEN);
    } else {
        SPR_setVisibility(enemy_sprite, VISIBLE);
        SPR_setPosition(enemy_sprite, enemy_x_as_s16, enemy_y_as_s16);
    }
}
```

```
// Actual width and height of each sprite, used as the visibility margin
#define BOSS_WIDTH      56
#define BOSS_HEIGHT    80
```

```
// Updated call inside camera(), replacing the one from section 2
set_enemy_position(boss_sprite, boss_x, boss_y, BOSS_WIDTH, BOSS_HEIGHT);
```

It's worth first testing, as an experiment, a margin value smaller than the sprite's actual width (for example, half of it) to confirm that an insufficient margin still leaves part of the sprite visible off screen before it gets hidden. That confirms that the correct value has to match the sprite's actual width (and height) exactly, not an approximate number. With the margin set to `BOSS_WIDTH/BOSS_HEIGHT`, the enemy appears and disappears right at the edge of the screen, without leaving any noticeable gaps or reappearing more than once anywhere on the map.

11. Adding a second enemy

To confirm that the function is now reusable for any number of enemies, the next step is to add a second character to the map—for example, a stocky enemy called `fat_man` in the code—following exactly the same pattern: declare its sprite and coordinates, add it with `SPR_addSprite()` after the player's sprite, and call `set_enemy_position()` for it inside `camera()`, with its own dimensions:

```
static Sprite* fat_man_sprite;
static fix32 fat_man_x = FIX32(1400);
static fix32 fat_man_y = FIX32(180);

#define FAT_MAN_WIDTH  48
#define FAT_MAN_HEIGHT 64

// Inside main(), alongside the rest of the sprites
fat_man_sprite = SPR_addSprite(&fat_man_sprite_def, F32_toInt(fat_man_x),
    ↪ F32_toInt(fat_man_y),
    TILE_ATTR(PAL3, FALSE, FALSE, FALSE));

// Inside camera(), alongside the boss's call
set_enemy_position(fat_man_sprite, fat_man_x, fat_man_y, FAT_MAN_WIDTH,
    ↪ FAT_MAN_HEIGHT);
```

When adding a second enemy, it's worth carefully checking that each one has its own palette correctly assigned: it's easy, when copying and pasting one enemy's code to create another, to forget to change the palette and end up with two characters sharing the same colors by mistake, something that's usually noticeable right away when compiling because the visual result looks confusing. It's also worth keeping in mind, as more characters are added to the map, two hardware limits already covered in previous chapters: the number of simultaneous hardware sprites the VDP can display on a single line before they start flickering due to a lack of slots, and the amount of RAM available to keep track of the position and state of each active enemy. This second point—how to neatly organize each enemy's information, instead of piling up loose variables for each one—is exactly the problem tackled in the next chapters of this part of the manual, starting with organizing the project into separate header files.

Review exercises

1. Explain why an enemy defined only once in the code can reappear several times across a large map, in terms of VDP plane size.
2. Why isn't changing the plane size with `VDP_setPlaneSize()` a suitable solution for this chapter's problem, even though it does have a legitimate use in another context? Describe that other context.
3. Explain the variable scope problem that forced `new_camera_x` and `new_camera_y` to be

moved outside the `camera()` function, and why something similar happened with the parameters of `set_enemy_position()`.

4. Describe the four checks that determine whether an enemy sprite should be hidden for being off camera, and what each one represents.
5. Why must the visibility limit be set to the sprite's negative width (or height) instead of 0, and what visual problem occurs if that adjustment isn't made?

Chapter 51 — Organizing the Project with Header Files

How to split the code of a growing game into several .c and .h files, instead of piling it all up in main.c

The previous chapter left this beat 'em up project with the player, two enemies (boss_sprite and fat_man_sprite), the camera() function, the set_enemy_position() function, and a handful of global variables, all living together inside a single main.c file. It works, but it's already starting to show the typical problem of any growing project: it gets harder and harder to find a function among so much code, and every new enemy, weapon, or mechanic added is going to pile more lines onto a file that's already difficult to navigate. This chapter steps away for a moment from the game's specific content to address something more general, applicable to any project written in C: how to split that code into several files, organized by topic, without changing the program's behavior in the slightest.

It's worth clarifying from the start that none of what follows is a feature specific to SGDK or the Mega Drive: header files are a mechanism of the C language itself, and everything explained in this chapter would apply equally to any other project written in that language.

Prerequisites

Before continuing, it helps to have:

- Chapter 50 ("VDP planes and enemies in a beat 'em up") completed, since this chapter picks up exactly from the code left at the end of that one: player_sprite/player_x/player_y, boss_sprite, fat_man_sprite, set_enemy_position(), and camera().
- The basic C concepts covered in chapter 6: what a declaration is versus a definition, what a function prototype is, and what the #include directive does.
- Chapter 39 ("The resource compiler in depth"), about how rescomp generates a header file from a .res file.

1. Why split the code into several files

The central idea of this chapter can be summed up in one sentence: instead of writing all of a game's code inside main.c, it's better to group it by topic into separate sets of files. One set of files for everything related to the player, another for the enemies, another for the camera, and so on as the project requires. Each of these sets usually has, at minimum, a header file (.h extension) and a corresponding code file, with the same name but a .c extension.

With a project as small as the ones from the early chapters of this manual, splitting the code into files doesn't gain much: everything fits comfortably on a screen or two. But a beat 'em up

with a player, several enemy types, a camera, levels, and later secondary weapons or a game state system is a different case, and it's exactly the kind of project this mechanism exists for.

A common problem when setting up a new SGDK project in Visual Studio Code is that function autocompletion (IntelliSense) stops showing SGDK's own descriptions and suggestions. The problem is almost always in the `settings.json` file, inside the project's `.vscode` folder: the simplest way to fix it is to copy that file from a previous project where autocompletion does work, and paste it into the new project.

2. Splitting resources into several `.res` files

Before touching the code itself, it's worth applying the same idea to resources. Up to now, this manual has declared sprites, backgrounds, and music inside a single `resources.res` file. For a project with resources for the player, each enemy type, backgrounds, and music, it's also a good idea to split resources into several `.res` files, following the same criteria as for the code: one resource file per topic. For example, `res_player.res` for the player and `res_enemies.res` for the enemies:

```
// res_player.res
SPRITE player_sprite_def "player_assets/player.png" 5 5 FAST

// res_enemies.res
SPRITE boss_sprite_def    "enemies_assets/boss.png"    7 10 FAST
SPRITE fat_man_sprite_def "enemies_assets/fat_man.png" 6 8  FAST
```

Also note the organization into subfolders within `res`: `player_assets` for the player's images and `enemies_assets` for the enemies', instead of leaving all the image files loose directly in `res`. This doesn't change anything about the `rescomp` mechanism already covered in chapter 39; it just requires writing the full path starting from `res` in each `SPRITE` line, including the subfolder name.

The first time it compiles with these files, `rescomp` automatically generates the corresponding header files, `res_player.h` and `res_enemies.h`, with the same logic it already used to generate `resources.h` when there was a single resource file. There's no need to touch those generated files by hand, but it's worth reviewing their contents, because they use exactly the same include guard mechanism explained in section 4 of this chapter, just with a guard name automatically generated from the file's name.

3. Quotes or angle brackets: two ways to include a file

Any file that uses SGDK functions needs to include `genesis.h`, the library's general header, with `#include <genesis.h>`. The project's own header files, on the other hand, such as `res_player.h` or the ones created in this very chapter, are included using quotes: `#include "res_player.h"`.

This difference in syntax isn't an aesthetic whim. Angle brackets tell the compiler to search for the file among the system's or the library's standard include paths (that's where SGDK lives), while quotes tell it to search first in the project itself, next to the file doing the including. As a practical convention: `<genesis.h>` and any other library header go between angle brackets; anything that's the project's own code goes between quotes.

4. From static variables to variables shared across files

Before splitting the code into several files, there's a problem the previous chapter left pending without naming it: a good part of this project's variables and functions (`player_sprite`, `boss_sprite`, `camera()`, `set_enemy_position()`) were declared with the `static` keyword. Within a single file, `static` on a global variable or function limits its scope to that same file, preventing it from accidentally clashing with an identical name defined elsewhere in the project. It's good practice when everything lives in `main.c`, because that file is, in effect, the only place those variables are used.

The problem is that, as soon as that same code is split into several files, that restriction stops making sense: if `camera()` needs to call `set_enemy_position()`, which is now going to live in another file, or if `set_enemy_position()` needs to read `new_camera_x`, which is going to live in a third file, none of those variables or functions can remain `static` anymore. The keyword that replaces `static` for this purpose is `extern`, and the rest of this chapter is devoted to showing exactly where and how to use it.

5. The include guard

Every new header file created in this project — `player.h`, `enemy.h`, `camera.h` — has to start with a repetitive but indispensable block called an include guard:

```
#ifndef PLAYER_H
#define PLAYER_H

// (the file's content goes here)

#endif // PLAYER_H
```

This guard exists to prevent the same header file from being included more than once within a single program. A file like `player.h` can end up included, indirectly, from many different places in the project: for example, `enemy.h` might need to include `player.h` to make an enemy react to the player's position, and `camera.h` might need to include it too for some other reason. Without an include guard, each of those repeated inclusions would redeclare the same variables and functions, producing compilation errors — typically redefinition errors — that are sometimes hard to trace back to their real cause. The guard solves this by marking, with `#define`, that the file has already been included once; if the compiler encounters a second inclusion, `#ifndef` detects that the marker already exists and jumps straight to the corresponding `#endif`, without processing the content again.

The name of the marker (`PLAYER_H` in the example) is a convention, not a language requirement: it just needs to be unique within the project, and using the file's name in uppercase, with an underscore instead of the dot, is the simplest way to guarantee that.

It doesn't matter much whether `#include` directives are written inside the include guard or before it (the files `rescomp` generates automatically, for instance, put their `#include <genesis.h>` before the guard); both ways work the same. What does always need to be respected is that the rest of the content (variable, sprite, and function declarations) stays between the `#define` and the `#endif`.

6. Splitting out the player: `player.h` and `player.c`

With the guard now clear, the first set of files worth creating is the player's. Inside the project's `inc` folder, `player.h` is created, with declarations for everything other files are go-

ing to need from the player: the pointer to the sprite, its position in map coordinates, and a `create_player()` function that groups the code that used to be loose inside `main()`.

The important difference from how these variables were written in the previous chapter is that, inside a header file, both the variables and the functions that need to be usable from other files are declared with the `extern` keyword in front:

```
// inc/player.h
#ifndef PLAYER_H
#define PLAYER_H

#include <genesis.h>
#include "res_player.h"

// Player sprite and its position in map coordinates
extern Sprite* player_sprite;
extern fix32 player_x;
extern fix32 player_y;

// Creates the player's sprite and adds it to the scene
void create_player(void);

#endif // PLAYER_H
```

`extern` tells the compiler that this variable or function exists and can be used from any file that includes this header, but that its actual definition (the place where memory is actually reserved for it, or where the function's body is written) is somewhere else. That somewhere else is, by convention, a `.c` file with the same name, in the `src` folder:

```
// src/player.c
#include "player.h"

Sprite* player_sprite;
fix32 player_x = FIX32(40);
fix32 player_y = FIX32(160);

void create_player(void) {
    player_sprite = SPR_addSprite(&player_sprite_def, F32_toInt(player_x),
    ↪ F32_toInt(player_y),
        TILE_ATTR(PAL1, FALSE, FALSE, FALSE));
}
```

Unlike `player.h`, `player.c` doesn't need any include guard or the `extern` keyword: it's enough to include `player.h` at the top, to stay linked to that header's declarations, and then write the actual definition of each variable and the full body of each function there. Every `extern` declaration in a `.h` file needs, somewhere in the project, exactly one corresponding definition; if that definition were missing, the linker would fail with an undefined symbol error, and if there were more than one definition for the same variable spread across different `.c` files, the error would be the opposite: symbol defined more than once. This is, in fact, the reason the previous section insisted on moving from `static` to `extern`: declaring `Sprite* player_sprite`; directly inside `player.h`, without `extern`, would end up generating a new definition of that variable in every `.c` file that includes the header, and the project would stop linking as soon as more than one file included it.

7. Splitting out the enemies: enemy.h and enemy.c

The same pattern applies to the enemies' code, with the difference that here there's more than one variable of each type (one per enemy) and a function, `set_enemy_position()`, which was already written in the previous chapter and now simply changes files:

```
// inc/enemy.h
#ifndef ENEMY_H
#define ENEMY_H

#include <genesis.h>
#include "res_enemies.h"

// Actual widths and heights of each sprite, used as a visibility margin
#define BOSS_WIDTH      56
#define BOSS_HEIGHT     80
#define FAT_MAN_WIDTH   48
#define FAT_MAN_HEIGHT  64

extern Sprite* boss_sprite;
extern fix32 boss_x;
extern fix32 boss_y;

extern Sprite* fat_man_sprite;
extern fix32 fat_man_x;
extern fix32 fat_man_y;

// Creates the enemies' sprites and adds them to the scene
void create_enemies(void);

// Repositions an enemy according to the current camera, and hides it
// if its on-screen position falls outside the visible bounds
void set_enemy_position(Sprite* enemy_sprite, fix32 enemy_x, fix32 enemy_y,
    s16 enemy_width, s16 enemy_height);

#endif // ENEMY_H

// src/enemy.c
#include "enemy.h"
#include "camera.h"

Sprite* boss_sprite;
fix32 boss_x = FIX32(900);
fix32 boss_y = FIX32(160);

Sprite* fat_man_sprite;
fix32 fat_man_x = FIX32(1400);
fix32 fat_man_y = FIX32(180);

void create_enemies(void) {
    boss_sprite = SPR_addSprite(&boss_sprite_def, F32_toInt(boss_x),
↪ F32_toInt(boss_y),
        TILE_ATTR(PAL3, FALSE, FALSE, FALSE));
    fat_man_sprite = SPR_addSprite(&fat_man_sprite_def, F32_toInt(fat_man_x),
↪ F32_toInt(fat_man_y),
```

```

        TILE_ATTR(PAL3, FALSE, FALSE, FALSE));
    }

void set_enemy_position(Sprite* enemy_sprite, fix32 enemy_x, fix32 enemy_y,
    s16 enemy_width, s16 enemy_height) {
    s16 enemy_x_as_s16 = F32_toInt(enemy_x) - new_camera_x;
    s16 enemy_y_as_s16 = F32_toInt(enemy_y) - new_camera_y;

    if (enemy_x_as_s16 < -enemy_width || enemy_x_as_s16 > HORIZONTAL_RESOLUTION ||
        enemy_y_as_s16 < -enemy_height || enemy_y_as_s16 > VERTICAL_RESOLUTION) {
        SPR_setVisibility(enemy_sprite, HIDDEN);
    } else {
        SPR_setVisibility(enemy_sprite, VISIBLE);
        SPR_setPosition(enemy_sprite, enemy_x_as_s16, enemy_y_as_s16);
    }
}

```

Note that `enemy.c` includes `camera.h` in addition to `enemy.h`: `set_enemy_position()` needs to read `new_camera_x`, `new_camera_y`, `HORIZONTAL_RESOLUTION`, and `VERTICAL_RESOLUTION`, and all of that is going to live in the camera file, written in the next section. This is exactly the kind of dependency between files that a larger project naturally ends up having: `enemy.c` needs something from `camera.h`, just as later on other files are going to need things from `enemy.h` or from `player.h`. As long as each header clearly declares what it exposes and each include guard prevents repeated declarations, that kind of cross dependency doesn't cause any problem.

8. Splitting out the camera: `camera.h` and `camera.c`

The last set of files in this reorganization is the camera's, with the four variables the previous chapter had turned into globals (`current_camera_x`, `current_camera_y`, `new_camera_x`, `new_camera_y`) and the two resolution constants already in use since the 2D camera chapter:

```

// inc/camera.h
#ifndef CAMERA_H
#define CAMERA_H

#include <genesis.h>

#define HORIZONTAL_RESOLUTION 320
#define VERTICAL_RESOLUTION 224

extern s16 current_camera_x;
extern s16 current_camera_y;
extern s16 new_camera_x;
extern s16 new_camera_y;

// Recalculates the camera's position and repositions the player and the enemies
void camera(void);

#endif // CAMERA_H

// src/camera.c
#include "camera.h"

```

```

#include "player.h"
#include "enemy.h"

s16 current_camera_x;
s16 current_camera_y;
s16 new_camera_x;
s16 new_camera_y;

void camera(void) {
    // Calculation of new_camera_x / new_camera_y based on the player's
    // position and the map's boundaries (chapter 24)
    // ...

    SPR_setPosition(player_sprite, F32_toInt(player_x) - new_camera_x,
                    F32_toInt(player_y) - new_camera_y);

    set_enemy_position(boss_sprite, boss_x, boss_y, BOSS_WIDTH, BOSS_HEIGHT);
    set_enemy_position(fat_man_sprite, fat_man_x, fat_man_y, FAT_MAN_WIDTH,
    ↪ FAT_MAN_HEIGHT);

    current_camera_x = new_camera_x;
    current_camera_y = new_camera_y;
}

```

camera.h doesn't need to include either player.h or enemy.h: nothing it declares depends on them. It's camera.c that does need them, because the body of camera() uses player_sprite, player_x, player_y, boss_sprite, fat_man_sprite, and set_enemy_position(). This distinction — what goes in the .h because other files need to know about it, and what goes only in the .c because it's an internal implementation detail — is what keeps each header as small and clear as possible, and also avoids a circular dependency: player.h and enemy.h don't need to know anything about camera.h, and camera.h doesn't need to know anything about player.h or enemy.h.

9. main.c reduced to the essentials

With the three pairs of files already created, main.c is reduced to the include directives for its own headers and the body of main():

```

// src/main.c
#include <genesis.h>
#include "player.h"
#include "enemy.h"
#include "camera.h"

int main(bool hard_reset) {
    VDP_setScreenWidth320();

    SPR_init();

    create_player();
    create_enemies();

    while (TRUE) {

```

```

        camera();

        SPR_update();
        SYS_doVBlankProcess();
    }

    return 0;
}

```

The project's file tree, after this reorganization, looks like this:

```

projecto/
├── inc/
│   ├── player.h
│   ├── enemy.h
│   └── camera.h
├── res/
│   ├── player_assets/
│   │   └── player.png
│   ├── enemies_assets/
│   │   ├── boss.png
│   │   └── fat_man.png
│   ├── res_player.res
│   └── res_enemies.res
└── src/
    ├── main.c
    ├── player.c
    ├── enemy.c
    └── camera.c

```

When compiled, the on-screen result is exactly the same as at the end of the previous chapter: the player and the two enemies appear at their expected position, the camera follows them just as before, and none of them duplicate while moving around the map. That is, in fact, the central point of this chapter: splitting the code into header files and their corresponding .c files doesn't change the program's behavior at all, only its internal organization. The benefit isn't as noticeable in a project this size, but as more elements get added — levels, secondary weapons, more enemy types, a game state system — each with its own pair of files, main.c is going to stay small and readable, and all those files are going to end up linked to one another, directly or indirectly, through the headers that keep including each other.

Review exercises

1. Explain in your own words what an include guard (`#ifndef/#define/#endif`) is for in a header file, and what specific problem it prevents.
2. Why can't the `boss_sprite`, `boss_x`, and `boss_y` variables, which were static in the previous chapter, keep being static once `enemy.c` is a file separate from `camera.c`?
3. What's the difference between declaring a variable with `extern` inside a .h file and defining it in the corresponding .c file? What compilation or linking error appears if either of those two parts is left out?
4. Why is `genesis.h` included between angle brackets while a header of the project's own, like `player.h`, is included between quotes?
5. `enemy.c` includes `camera.h`, but `camera.h` includes neither `player.h` nor `enemy.h`. Explain why this organization avoids a circular dependency among the project's three headers.

Chapter 52 — Adding Enemies to the Game

From loose per-enemy variables to arrays, and how to decide how many enemies the screen can sustain at once

The previous chapter left the project already split into header files: `player.h/player.c` for the player, `enemy.h/enemy.c` for the enemies, and `camera.h/camera.c` for the camera, with `boss_sprite` and `fat_man_sprite` as the only two enemies on the map, each with its own pointer to `Sprite`, its own `fix32` position variables, and its own width and height constants. That pattern — one loose variable for each piece of data for each enemy — works perfectly well for two named characters, but a beat 'em up needs something more: waves of common enemies that appear, move, and disappear without each one deserving its own hand-written set of variables. This chapter adds that wave of enemies, shows why the loose-variable pattern stops being sustainable as soon as the number of enemies grows, and replaces it with arrays traversed with a `for` loop. It closes by returning to a more practical question: how many enemies can be on screen at the same time before the hardware, or the frame budget itself, starts to notice.

Prerequisites

Before continuing, it helps to have:

- Chapter 51 (“Organizing the project with header files”) completed, since this chapter picks up directly from `player.h/player.c`, `enemy.h/enemy.c`, and `camera.h/camera.c` exactly as they were left there, including `set_enemy_position()` and the variables `boss_sprite`, `fat_man_sprite`, `boss_x`, `boss_y`, `fat_man_x`, and `fat_man_y`.
- Arrays and `for` loops, covered in depth in chapter 17 (“Tile-based scrolling”); this chapter assumes that syntax is already known and focuses on applying it to a new problem.
- The limits on simultaneous sprites depending on resolution mode, introduced in chapter 10 (“H40 and H32 resolution modes”), and VRAM management and CPU profiling covered in Part VIII (chapters 45 to 49), which are briefly revisited toward the end of this chapter.

1. The limit of the “one variable per enemy” pattern

Adding a third enemy using the same approach as `boss_sprite` and `fat_man_sprite` is mechanical: a `Sprite*` variable, two `fix32` variables for position, two constants for width and height, a line inside `create_enemies()` to register it with `SPR_addSprite()`, and a call to `set_enemy_position()` inside `camera()`. None of that is hard, but it’s exactly the kind of work that doesn’t scale: adding a wave of four common enemies — a ninja, a sniper, a stealth

ninja, and a boomerang-throwing enemy, all four recruited from the same remake the previous chapters have been using — would multiply each of those steps by four, and adding ten more enemies would multiply them again. The code in `create_enemies()` and `camera()` would end up as a long string of nearly identical lines, one per enemy, with the added risk that copying and pasting those lines is also the most common way to introduce silent bugs: a palette changed by mistake, a Y copied where an X should have gone, a call to `set_enemy_position()` left pointing at the wrong enemy.

The solution to this kind of repetition already appeared earlier in this manual, applied to a different problem: instead of one variable per enemy, an array with one position per enemy, traversed with a `for` loop.

2. Assets for a wave of common enemies

Following the same approach already used for `boss.png` and `fat_man.png`, the four new graphics are added to the `enemies_assets` subfolder inside `res`, and `res_enemies.res` gets one `SPRITE` line for each of them, alongside the two that already existed:

```
// res/res_enemies.res
SPRITE boss_sprite_def      "enemies_assets/boss.png"      7 10 FAST
SPRITE fat_man_sprite_def   "enemies_assets/fat_man.png"    6 8  FAST
SPRITE ninja_sprite_def     "enemies_assets/ninja.png"      3 4  FAST
SPRITE sniper_sprite_def    "enemies_assets/sniper.png"     3 4  FAST
SPRITE stealth_sprite_def   "enemies_assets/stealth.png"    3 4  FAST
SPRITE boomerang_sprite_def "enemies_assets/boomerang.png"  4 4  FAST
```

The four common enemies are deliberately small in their number of animation frames, in keeping with an 8-bit game style: three of them measure 3×4 tiles (24×32 pixels), and the boomerang-throwing one, a bit wider because of its throwing animation, measures 4×4 tiles (32×32 pixels). When compiled, `rescomp` adds the four new definitions to `res_enemies.h`, alongside the ones that already existed for `boss_sprite_def` and `fat_man_sprite_def`.

The `enemies_assets` subfolder, with the graphics for boss, fat_man, and the four common enemies already copied inside.

3. From loose variables to arrays

The first change inside `enemy.h` is purely declarative: instead of four sets of loose variables (one for each common enemy), a single array is declared per attribute, with as many slots as there are common enemies. Each enemy is identified by its position within those arrays, starting at 0, following the usual C convention:

```
// inc/enemy.h
#ifndef ENEMY_H
#define ENEMY_H

#include <genesis.h>
#include "res_enemies.h"

// Actual widths and heights for boss and fat_man, used as a visibility margin
#define BOSS_WIDTH      56
#define BOSS_HEIGHT     80
#define FAT_MAN_WIDTH   48
#define FAT_MAN_HEIGHT  64
```

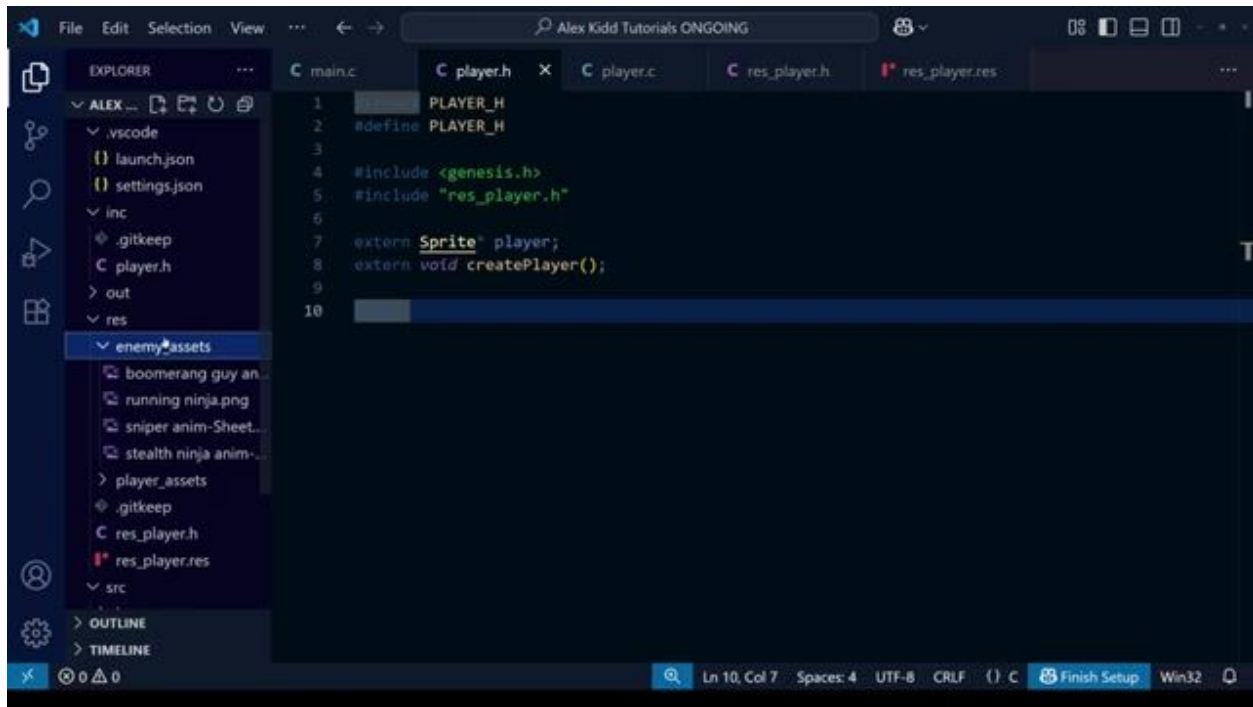


Figure 148: The enemies_assets subfolder with the project's enemy graphics

```
extern Sprite* boss_sprite;
extern fix32 boss_x;
extern fix32 boss_y;

extern Sprite* fat_man_sprite;
extern fix32 fat_man_x;
extern fix32 fat_man_y;

// Index of each common enemy within the arrays below
#define ENEMY_NINJA 0
#define ENEMY_SNIPER 1
#define ENEMY_STEALTH 2
#define ENEMY_BOOMERANG 3
#define NUM_ENEMIES 4

// One array per attribute, instead of one set of variables per enemy
extern Sprite* enemy_sprite[NUM_ENEMIES];
extern fix32 enemy_x[NUM_ENEMIES];
extern fix32 enemy_y[NUM_ENEMIES];
extern fix32 enemy_velx[NUM_ENEMIES];
extern const s16 enemy_width[NUM_ENEMIES];
extern const s16 enemy_height[NUM_ENEMIES];

// Creates boss, fat_man, and the four common enemies, and adds them to the scene
void create_enemies(void);
```

```
// Applies the common enemies' horizontal movement on every frame
void manage_enemies(void);

// Repositions an enemy according to the current camera, and hides it
// if its on-screen position falls outside the visible bounds
void set_enemy_position(Sprite* enemy_sprite, fix32 enemy_x, fix32 enemy_y,
    s16 enemy_width, s16 enemy_height);

#endif // ENEMY_H
```

boss_sprite and fat_man_sprite keep their loose variables: they are unique, named enemies with particular behavior, and there's no advantage in forcing them into an array meant to handle several enemies uniformly. The array does make sense, though, for the group of common enemies, which all behave the same way and differ only in their graphic, their position, and their speed.

4. create_enemies() with a for loop

In enemy.c, each array is initialized with one value per slot, and an additional array — of pointers to SpriteDefinition, one per common enemy type — lets a single for loop register all four on screen, instead of repeating the same call to SPR_addSprite() four times:

```
// src/enemy.c
#include "enemy.h"
#include "camera.h"

Sprite* boss_sprite;
fix32 boss_x = FIX32(900);
fix32 boss_y = FIX32(160);

Sprite* fat_man_sprite;
fix32 fat_man_x = FIX32(1400);
fix32 fat_man_y = FIX32(180);

Sprite* enemy_sprite[NUM_ENEMIES];
fix32 enemy_x[NUM_ENEMIES] = { FIX32(200), FIX32(500), FIX32(1100), FIX32(1800) };
fix32 enemy_y[NUM_ENEMIES] = { FIX32(180), FIX32(160), FIX32(190), FIX32(170) };
fix32 enemy_velx[NUM_ENEMIES] = { FIX32(0.5), FIX32(-0.3), FIX32(0.4), FIX32(-0.6)
    ↪ };

const s16 enemy_width[NUM_ENEMIES] = { 24, 24, 24, 32 };
const s16 enemy_height[NUM_ENEMIES] = { 32, 32, 32, 32 };

// One pointer to SpriteDefinition for each common enemy, in the same order
// as the ENEMY_NINJA..ENEMY_BOOMERANG indexes
static const SpriteDefinition* enemy_def[NUM_ENEMIES] = {
    &ninja_sprite_def, &sniper_sprite_def, &stealth_sprite_def,
    ↪ &boomerang_sprite_def
};

void create_enemies(void) {
    boss_sprite = SPR_addSprite(&boss_sprite_def, F32_toInt(boss_x),
    ↪ F32_toInt(boss_y),
```

```

        TILE_ATTR(PAL3, FALSE, FALSE, FALSE));
    fat_man_sprite = SPR_addSprite(&fat_man_sprite_def, F32_toInt(fat_man_x),
    ↪ F32_toInt(fat_man_y),
        TILE_ATTR(PAL3, FALSE, FALSE, FALSE));

    // The four common enemies share palette PAL2, distinct from
    // PAL1 (player) and PAL3 (boss and fat_man)
    for (u8 i = 0; i < NUM_ENEMIES; i++) {
        enemy_sprite[i] = SPR_addSprite(enemy_def[i], F32_toInt(enemy_x[i]),
    ↪ F32_toInt(enemy_y[i]),
            TILE_ATTR(PAL2, FALSE, FALSE, FALSE));
    }
}

```

boss_sprite and fat_man_sprite are still registered by hand, each with its own line, because there are only two of them and each explicitly uses its own graphic resource in the code; the four common enemies, on the other hand, are registered inside the loop, reading their starting position and their sprite definition from slot *i* of the corresponding arrays.

5. manage_enemies(): a first artificial intelligence with arrays and for loops

With the positions already in arrays, updating them frame by frame becomes a single function with a single loop, instead of one addition per enemy:

```

void manage_enemies(void) {
    // Each common enemy advances according to its own horizontal speed,
    // read from enemy_velx[i]: positive values move to the
    // right, negative ones to the left
    for (u8 i = 0; i < NUM_ENEMIES; i++) {
        enemy_x[i] += enemy_velx[i];
    }
}

```

This is, deliberately, the simplest possible version of an artificial intelligence: each common enemy walks in a straight line, at a constant speed, without reacting to anything. It's only a starting point — the following chapters add reaction to the player, patrol limits, and behavior states — but it's enough to confirm that the speed array works: if *enemy_velx* were a single value instead of one per slot, the four enemies would move at exactly the same speed and in the same direction, losing the variety they had when each one was an independent variable. Storing one speed per enemy in its own array slot is what lets that variety come back without writing a different addition for each one.

6. Integrating the array with the camera

camera() already called *set_enemy_position()* once for *boss_sprite* and once for *fat_man_sprite*; the four common enemies are added with the same for loop already used to create them, reading the width and height that correspond to each one from *enemy_width* and *enemy_height*:

```

// src/camera.c
#include "camera.h"

```

```

#include "player.h"
#include "enemy.h"

s16 current_camera_x;
s16 current_camera_y;
s16 new_camera_x;
s16 new_camera_y;

void camera(void) {
    // Calculation of new_camera_x / new_camera_y based on the player's
    // position and the map's limits (chapter 24)
    // ...

    SPR_setPosition(player_sprite, F32_toInt(player_x) - new_camera_x,
                    F32_toInt(player_y) - new_camera_y);

    set_enemy_position(boss_sprite, boss_x, boss_y, BOSS_WIDTH, BOSS_HEIGHT);
    set_enemy_position(fat_man_sprite, fat_man_x, fat_man_y, FAT_MAN_WIDTH,
↪ FAT_MAN_HEIGHT);

    for (u8 i = 0; i < NUM_ENEMIES; i++) {
        set_enemy_position(enemy_sprite[i], enemy_x[i], enemy_y[i],
↪ enemy_width[i], enemy_height[i]);
    }

    current_camera_x = new_camera_x;
    current_camera_y = new_camera_y;
}

```

manage_enemies() is called once per pass of the main loop, before camera(), so that the latter positions the enemies that have already moved on screen, rather than the ones from the previous pass:

```

// src/main.c
#include <genesis.h>
#include "player.h"
#include "enemy.h"
#include "camera.h"

int main(bool hard_reset) {
    VDP_setScreenWidth320();

    SPR_init();

    create_player();
    create_enemies();

    while (TRUE) {
        manage_enemies();
        camera();

        SPR_update();
        SYS_doVBlankProcess();
    }
}

```

```
    return 0;
}
```

Once compiled, all six enemies appear on screen: `boss_sprite` and `fat_man_sprite` standing still in their usual position, and the four common enemies each moving at their own speed and in their own direction, correctly hiding when they leave the camera thanks to `set_enemy_position()`.

7. How many enemies the screen can sustain at once

Before adding more enemies, how far can this be pushed? To answer that, it helps to revisit two hardware limits already seen in Part VIII.

The first is the number of simultaneous hardware sprites the VDP can draw, introduced in chapter 10: up to 80 in H40 mode (320 pixels wide) or 64 in H32 mode, with a maximum of 20 per horizontal line in H40 (16 in H32). That limit isn't counted per character, but per hardware sprite, and an SGDk Sprite larger than 32×32 pixels (4×4 tiles, the maximum size of a hardware sprite) is actually made up internally of several chained hardware sprites.

It's worth doing the math with the project's actual sizes: `player_sprite` (5×5 tiles) takes up $2 \times 2 = 4$ hardware sprites; `boss_sprite` (7×10 tiles) takes up $2 \times 3 = 6$; `fat_man_sprite` (6×8 tiles) takes up $2 \times 2 = 4$; each of the three 3×4-tile common enemies takes up $1 \times 1 = 1$, and the boomerang-throwing one, at 4×4 tiles, takes up another 1×1 . In total, this chapter's full scene uses $4 + 6 + 4 + 3 + 1 = 18$ hardware sprites, comfortably under the 80 available in H40.

The margin is comfortable today, but it shrinks fast as projectiles, items, and effects get added, each with its own sprite. The per-line limit is, in practice, the easiest one to hit without noticing: if several large enemies happened to overlap vertically within the same screen band, the sum of hardware sprites they occupy on that line could get close to the 20 available, and the VDP simply stops drawing the excess ones on that line, causing flicker. Spreading enemies across different screen heights, and avoiding stacking several large enemies one below another, helps keep that margin.

The second limit isn't the VDP's but the frame budget itself, covered in chapter 49: every active enemy adds CPU work on each pass of the main loop — its own movement logic, its animation update, and later on, collision detection against the player — and that work has to fit, along with everything else, within the 16.7 milliseconds of a frame at 60 frames per second. The correct way to know whether a given number of simultaneous enemies is sustainable isn't to guess it, but to measure it with `SYS_getCPULoad()` while the game runs with that many enemies active and moving, as explained in that chapter.

It's also worth remembering the VRAM cost of each new enemy type, covered in chapters 45 to 48: every distinct graphic adds its own unique tiles to video memory, and a common enemy that repeats many times on screen — unlike `boss_sprite` or `fat_man_sprite`, which are unique — is exactly the use case for the manual sprite loading from chapter 48, where several instances of the same enemy share a single copy of their tiles instead of reserving one per instance.

In practice, the concrete figure of “how many enemies at once” isn't a fixed number that holds for any project: it depends on the size of the sprites, how much else is on screen (background, effects, HUD), and how much logic runs per enemy. `NUM_ENEMIES` is 4 in this chapter because that's the size of the wave being added right now, but nothing prevents defining it larger as

the project needs it, as long as the hardware-sprite-per-line count and the `SYS_getCPULoad()` measurement confirm there's still margin.

8. The problem coming up: too many parallel arrays

The position array and the speed array already solve this chapter's problem, but they raise a new one as soon as you think about what else each common enemy is going to need later on: hit points (something the common enemies in this remake don't need, since they die in one hit, but which bosses will need), whether it's alive or already defeated, which direction it's facing, which animation state it's in. Each of those attributes, following the same approach used in this chapter, would require yet another four-slot array, indexed with the same enemy number as the others but declared and maintained separately. Even `enemy_sprite` is already, in effect, one more of these parallel arrays.

Technically, continuing down that path would work, but it would quickly become unmanageable: nothing in the language would prevent, for example, updating `enemy_x[2]` and accidentally reading the hit points from `enemy_life[1]`, because the compiler has no way of knowing that both slots should refer to the same enemy. The tidy way to solve this — grouping all of an enemy's attributes in one place, instead of keeping them spread across parallel arrays of the same size — is to create a custom data type with a structure (`struct`), a topic revisited later in this manual.

Review exercises

1. Explain why `boss_sprite` and `fat_man_sprite` kept their loose variables in this chapter, while the four common enemies moved into an array.
2. What would happen if `enemy_velx` were a single `fix32` variable instead of a four-slot array? Describe the concrete effect on the common enemies' movement.
3. Calculate how many hardware sprites an 8×8-tile enemy takes up, and explain why that number doesn't necessarily match the number of characters on screen.
4. Why is measuring with `SYS_getCPULoad()` more reliable than eyeballing how many simultaneous enemies the game can sustain?
5. Describe, in your own words, the “parallel arrays” problem this chapter closes with, and why a structure (`struct`) would solve it better than continuing to add arrays.

Chapter 53 — C Structs Applied to Game Entities

Replacing an enemy's parallel arrays with a single, purpose-built data type capable of gathering all of its information in one place

The previous chapter left a problem on the table that was only going to get worse over time: to handle several enemies at once, instead of a lone `boss_sprite` and `fat_man_sprite`, an array was needed for every attribute — one for the sprite pointers, another for the X coordinates, another for the Y coordinates, another for the horizontal speed, another for each enemy's graphic resource. It works, but every new attribute needed down the line (hit points, a behavior type, its own timer) is going to require adding yet another array, all the same size and always walked in parallel, with the constant risk that a position drifts out of sync between one array and another. This chapter introduces the C language tool designed exactly for this problem: the structure, or `struct`, which was already mentioned in passing back in chapter 6 as one of the data types available in C, but which is only now used in depth to model something as concrete as an enemy in the game.

A structure lets you define your own data type that groups several related values — potentially of different types — under a single name, so that each enemy becomes a single variable with several fields inside it, instead of a repeated position across half a dozen separate arrays.

Prerequisites

Before continuing, it helps to have:

- Chapter 52 (“Adding enemies to the game”) completed, since this chapter picks up directly from the problem it leaves open: several parallel arrays — one per attribute — to handle more than one enemy on screen.
- Chapter 51 (“Organizing the project with header files”), covering extern syntax and the project's organization into `player.h/.c`, `enemy.h/.c`, and `camera.h/.c`.
- Chapter 6 (“C applied to retro development”), covering the `u8`, `s8`, and `fix32` types, and C structures in general — although this chapter revisits the dot operator (`.`) from scratch for anyone who hasn't used it yet.

1. The starting point: several arrays for one single thing

By the end of the previous chapter, `enemy.h` and `enemy.c` had grown beyond the original two enemies from chapter 50 (`boss_sprite` and `fat_man_sprite`), in order to handle a small wave of enemies at once. The most direct way to achieve that, without any new tools, was to replace each standalone variable with an array of the same type, all indexed the same way: `position`

i of enemy_sprite corresponds to the same enemy as position i of enemy_x, of enemy_y, of enemy_vel_x, and of enemy_graphic.

```
// Project state at the end of the previous chapter: one array per attribute
#define MAX_ENEMIES 6

Sprite* enemy_sprite[MAX_ENEMIES];
fix32 enemy_x[MAX_ENEMIES];
fix32 enemy_y[MAX_ENEMIES];
fix32 enemy_vel_x[MAX_ENEMIES];
const SpriteDefinition* enemy_graphic[MAX_ENEMIES];
```

The problem isn't that this code is badly written: it compiles and works just as well as before. The problem shows up the moment one more attribute needs to be added. Hit points, for instance, would require declaring a sixth array, `s8 enemy_hitpoints[MAX_ENEMIES]`, and remembering, at every place in the code where an enemy is created or updated, to touch that array in addition to the other five. The more attributes pile up, the easier it is to forget one when writing a new function, or worse, to accidentally misalign one array's index against the others — for example, by inserting an enemy in the middle of the array and forgetting to shift one of the five arrays along with the other four.

2. Defining struct Enemy

The solution is to stop thinking in terms of “five arrays that live in sync” and start thinking in terms of “an array of a single thing, which already carries its five attributes inside it.” That “single thing” is exactly what defining a structure in C lets you do. The syntax is simple: the `struct` keyword, whatever name you want to give the new type, a pair of braces with the attributes inside, and a semicolon at the end.

Anyone already familiar with C might prefer `typedef struct` instead of the plain `struct TypeName` form used in this chapter. The difference is mostly stylistic — with `typedef` you can write `Enemy` instead of `struct Enemy` in every declaration — and this manual sticks with the form without `typedef` because it's a bit more explicit for anyone encountering structures for the first time.

The new type is called `Enemy`, and it's defined in `enemy.h`, alongside the sprite-size constants that have already been there since chapter 51. The first attribute worth adding is a state (status), as an unsigned 8-bit integer (`u8`): for now two values are enough, 0 for an inactive enemy and 1 for an active one, although the data type leaves room to represent other behaviors later on, such as attacking, taking damage, or chasing the player.

Next come the attributes that already existed as parallel arrays: the `x` and `y` coordinates and the horizontal speed `vel_x`, all three as `fix32`, and a pointer to the enemy's sprite, of type `Sprite*`. An attribute meant for future use is also added, hit points (`hitpoints`), as a signed 8-bit integer (`s8`) — most of the regular enemies in this project are going to die in a single hit, but this attribute is going to matter more for tougher level bosses — along with each enemy's graphic resource, `graphic`, of the same type `const SpriteDefinition*` already used by `boss_sprite_def` and `fat_man_sprite_def`. Finally, so that `set_enemy_position()` can keep calculating each enemy's visibility margin without depending on a fixed macro per type, it makes sense for the sprite's width and height to travel as part of the structure too:

```
// inc/enemy.h (excerpt)
struct Enemy {
    u8 status;                // 0 = inactive, 1 = active
    fix32 x;
```

```

fix32 y;
fix32 vel_x;
Sprite* sprite;           // NULL until create_enemies() sets it up
s16 width;
s16 height;
s8 hitpoints;
const SpriteDefinition* graphic;
};

```

3. An array of structures, instead of an array per attribute

With the Enemy type already defined, you can declare an array of elements of that type using exactly the same syntax already used for the fix32 arrays in section 1, except that now the data type is struct Enemy instead of fix32. The array is called current_enemies, and it keeps the same size, MAX_ENEMIES, that the parallel arrays it replaces already had:

```

// inc/enemy.h (excerpt)
extern struct Enemy current_enemies[MAX_ENEMIES];

```

Like any other variable that needs to be used from several files, it's declared with extern inside the .h file and defined, with its actual contents, in the corresponding .c file.

4. Filling in the enemy data

In enemy.c, the actual contents of the array are defined. Each element is written between braces, with its nine values in the same order in which the structure's attributes were declared: status, X position, Y position, speed, sprite, width, height, hit points, and graphic. The sprite attribute is left as NULL for all six enemies, because at this stage no sprite has been set up on screen yet — that only happens inside create_enemies(), in the next section.

```

// src/enemy.c
#include "enemy.h"
#include "camera.h"

struct Enemy current_enemies[MAX_ENEMIES] = {
    // status,    x,          y,          vel_x,          sprite, width,
    ↪ height,          hitpoints, graphic
    { 1, FIX32(900),  FIX32(160), FIX32(0),  NULL, BOSS_WIDTH,  BOSS_HEIGHT,
    ↪ 5, &boss_sprite_def },
    { 1, FIX32(1400), FIX32(180), FIX32(0),  NULL, FAT_MAN_WIDTH, FAT_MAN_HEIGHT,
    ↪ 3, &fat_man_sprite_def },
    { 1, FIX32(2100), FIX32(160), FIX32(-1), NULL, BOSS_WIDTH,  BOSS_HEIGHT,
    ↪ 5, &boss_sprite_def },
    { 1, FIX32(2600), FIX32(180), FIX32(1),  NULL, FAT_MAN_WIDTH, FAT_MAN_HEIGHT,
    ↪ 3, &fat_man_sprite_def },
    { 0, FIX32(0),    FIX32(0),  FIX32(0),  NULL, 0,          0,          0,
    ↪ NULL          },
    { 0, FIX32(0),    FIX32(0),  FIX32(0),  NULL, 0,          0,          0,
    ↪ NULL          },
};

```

The first four elements are left active (status set to 1): the first two reproduce exactly the

position of the boss and the fat man that already existed since chapter 50, and the following two add a second pair of the same graphics, further along the map, with a nonzero `vel_x` so they move on their own in addition to following the camera. The last two elements are left inactive (status set to 0), reserved for adding more enemies in the future without having to enlarge the array or recompile with a different size.

Storing a width and height of 0 for inactive enemies causes no problem: as long as status is 0, neither `create_enemies()` nor `manage_enemies()` ever reads those fields, thanks to the check added in the next two sections.

5. The dot operator: how to access an element's attributes

To access an attribute of an array element, the dot operator (`.`) is used: `current_enemies[i].status` accesses the state of the enemy at position `i`, `current_enemies[i].graphic` accesses its graphic, and so on for any other attribute of the structure. The reading is literal: first the array is indexed as usual (`current_enemies[i]`), and then you navigate inside that element to the field you need (`.status`, `.x`, `.sprite`, etc.).

This syntax replaces, in a single step, what the parallel arrays required: instead of `enemy_sprite[i]`, `enemy_x[i]`, `enemy_y[i]`, and `enemy_graphic[i]` — four separate arrays, coordinated only by convention — there is now a single array and a single index, and the dot operator picks the attribute within that element.

6. Simplifying `create_enemies()` with the dot operator

With all six enemies already defined in the array, `create_enemies()` can walk through them with a single for loop, instead of one call to `SPR_addSprite()` per enemy. Before creating each sprite it makes sense to check that the corresponding enemy is active: if its status is nonzero, it's set up with `SPR_addSprite()`, taking the graphic resource and the starting position directly from that array element's attributes.

```
// src/enemy.c (continued)
void create_enemies(void) {
    for (u8 i = 0; i < MAX_ENEMIES; i++) {
        if (current_enemies[i].status != 0) {
            current_enemies[i].sprite = SPR_addSprite(current_enemies[i].graphic,
                F32_toInt(current_enemies[i].x), F32_toInt(current_enemies[i].y),
                TILE_ATTR(PAL3, FALSE, FALSE, FALSE));
        }
    }
}
```

It's worth paying attention to a detail that's easy to overlook: the value returned by `SPR_addSprite()` — the pointer to the newly created sprite — has to be stored in `current_enemies[i].sprite`, not discarded. If that assignment were missing, the pointer would be lost as soon as that loop iteration ended, and `manage_enemies()` would have no way to move or hide that sprite later on: the sprite attribute would keep being NULL forever, and any later call to `SPR_setPosition()` or `SPR_setVisibility()` with that pointer would end up in a runtime error.

7. Simplifying position updates: manage_enemies()

The same approach applies to the function that updates the enemies' position every frame. Instead of a loop that added the speed to an array of positions and then called `set_enemy_position()` for each individual sprite (as `camera()` did directly at the end of chapter 51, with one call for `boss_sprite` and another for `fat_man_sprite`), a single loop now walks through the six elements of `current_enemies`, adding `current_enemies[i].vel_x` to `current_enemies[i].x` and delegating to `set_enemy_position()` — the same function from chapter 50, unchanged — the camera adjustment and the hiding of sprites outside the visible bounds.

This loop, as it stands, would also try to move the inactive enemies, whose `sprite` attribute is still `NULL` because no sprite was ever created for them; doing so would produce the same runtime error mentioned in the previous section. The solution is to add the same check already used in `create_enemies()`: process only the enemies whose status is nonzero.

```
// src/enemy.c (continued)
void manage_enemies(void) {
    for (u8 i = 0; i < MAX_ENEMIES; i++) {
        if (current_enemies[i].status != 0) {
            current_enemies[i].x += current_enemies[i].vel_x;

            set_enemy_position(current_enemies[i].sprite, current_enemies[i].x,
↪ current_enemies[i].y,
            current_enemies[i].width, current_enemies[i].height);
        }
    }
}

void set_enemy_position(Sprite* enemy_sprite, fix32 enemy_x, fix32 enemy_y,
s16 enemy_width, s16 enemy_height) {
    s16 enemy_x_as_s16 = F32_toInt(enemy_x) - new_camera_x;
    s16 enemy_y_as_s16 = F32_toInt(enemy_y) - new_camera_y;

    if (enemy_x_as_s16 < -enemy_width || enemy_x_as_s16 > HORIZONTAL_RESOLUTION ||
        enemy_y_as_s16 < -enemy_height || enemy_y_as_s16 > VERTICAL_RESOLUTION) {
        SPR_setVisibility(enemy_sprite, HIDDEN);
    } else {
        SPR_setVisibility(enemy_sprite, VISIBLE);
        SPR_setPosition(enemy_sprite, enemy_x_as_s16, enemy_y_as_s16);
    }
}
```

After this change, `enemy.h` ends up with the structure definition, the `current_enemies` array, and the prototypes of the three functions:

```
// inc/enemy.h
#ifndef ENEMY_H
#define ENEMY_H

#include <genesis.h>
#include "res_enemies.h"

// Actual widths and heights of each sprite, used as the visibility margin
#define BOSS_WIDTH 56
```

```

#define BOSS_HEIGHT      80
#define FAT_MAN_WIDTH    48
#define FAT_MAN_HEIGHT   64

#define MAX_ENEMIES      6

// All of an enemy's attributes, grouped into a single data type
struct Enemy {
    u8 status;                // 0 = inactive, 1 = active
    fix32 x;
    fix32 y;
    fix32 vel_x;
    Sprite* sprite;           // NULL until create_enemies() sets it up
    s16 width;
    s16 height;
    s8 hitpoints;
    const SpriteDefinition* graphic;
};

extern struct Enemy current_enemies[MAX_ENEMIES];

// Creates the sprite for each active enemy and adds it to the scene
void create_enemies(void);

// Updates the position of each active enemy according to its speed,
// and adjusts the on-screen visibility and position of its sprite
void manage_enemies(void);

// Repositions a sprite according to the current camera, and hides it
// if its on-screen position falls outside the visible bounds
void set_enemy_position(Sprite* enemy_sprite, fix32 enemy_x, fix32 enemy_y,
    s16 enemy_width, s16 enemy_height);

#endif // ENEMY_H

```

With `manage_enemies()` now centralizing the update of every enemy, `camera()` stops calling `set_enemy_position()` once for each enemy known in advance, and instead calls `manage_enemies()` a single time:

```

// src/camera.c (excerpt from camera(), updated)
void camera(void) {
    // Calculation of new_camera_x / new_camera_y based on the player's
    // position and the map's boundaries (chapter 24)
    // ...

    SPR_setPosition(player_sprite, F32_toInt(player_x) - new_camera_x,
        F32_toInt(player_y) - new_camera_y);

    manage_enemies();

    current_camera_x = new_camera_x;
    current_camera_y = new_camera_y;
}

```


With the functions already rewritten, the parallel arrays from section 1 (`enemy_sprite`, `enemy_x`, `enemy_y`, `enemy_vel_x`, and `enemy_graphic`) are removed from `enemy.h` and `enemy.c`, since all of that information now lives inside `current_enemies`. When compiled, the result on screen is the same as at the end of the previous chapter — the same four active enemies, moving and hiding correctly according to the camera — but organized in a way that’s much simpler to extend: adding a seventh attribute to each enemy, from now on, means adding a single field to struct `Enemy`, instead of a new array coordinated by hand with the rest.

8. A problem that remains unsolved: data repeated per enemy type

Structures solve the parallel-array problem, but they leave a different one unresolved. If it later became necessary to change one of the array’s enemies to a different type — for example, turning the second `fat_man` into a third boss — changing the `graphic` attribute isn’t enough: `width`, `height`, and `hitpoints` also have to be updated by hand, separately, since those values depend on the enemy type and end up repeated, independently, in every array element that uses that type. With only two enemy types and four active elements, the risk of forgetting one of those fields already exists; with more types and more enemies on screen, that risk is only going to grow.

Remembering which status number corresponds to which behavior is already a bit confusing even with just two possible values (0 and 1); as soon as more states are added — attacking, taking damage, dying — that problem is also going to need its own solution.

The solution to both problems — the data repeated per type and the magic numbers used for status — comes through a related tool that this manual revisits later on: a fixed list of the possible enemy types, each with its own constant characteristics (`graphic`, `hit points`, `size`), so that changing an enemy’s type within `current_enemies` is reduced to changing a single value, instead of several separate fields at once.

Review exercises

1. Explain in your own words what problem with parallel arrays the use of a structure (`struct`) solves in this chapter.
2. Describe the basic syntax for declaring a structure in C, and the syntax for declaring an array of elements of that type.
3. What is the dot operator used for when working with an array of structures? Give an example taken from this chapter.
4. Why is it necessary to check the `status` attribute before operating on an enemy inside `create_enemies()` and `manage_enemies()`?
5. What problem remains unsolved at the end of this chapter, related to enemies that share the same type?

Chapter 54 — C Enumerations (enums)

Replace the loose numbers in the status field with readable names, and use #define for the quantities that repeat throughout the code

The previous chapter left each enemy's status field as a plain integer: 0 for an inactive enemy, 1 for an active one, with the promise of adding more values in the future to represent different behaviors (attacking, hurt, dying, and so on). The problem is that memorizing which number represents which behavior becomes, as more values are added, increasingly hard to follow just by looking at the code: an `if (current_enemies[i].status != 0)` says nothing on its own about what that zero means. This chapter solves that problem with another tool from the C language that hasn't been used yet in this manual: the enumeration, or `enum`, which lets you give a readable name to each of those numbers.

Prerequisites

Before continuing, it helps to have:

- Chapter 53 ("C structs applied to game entities") completed, since this chapter works directly on the `Enemy` structure and the `current_enemies` array created there.
- Chapter 52 ("Adding enemies to the game"), where the `createEnemies` and `manageEnemies` functions were first written, and which this chapter will simplify even further.
- Basic knowledge of C: the `#define` directive, already used in earlier chapters to create named constants.

1. Why #define isn't enough for a list of related values

Before showing enums, it's worth reviewing the alternative this manual already knew about: using `#define` to give a name to each number, for example `#define ENEMY_INACTIVE 0` and `#define ENEMY_IDLE 1`, and then replacing the loose numbers in the code with those names. That technique works, but it falls short when dealing with a more or less long list of values related to one another, such as the different possible states of an enemy. Declaring each value separately with its own `#define` forces you to repeat the keyword over and over, says nothing anywhere in the code about those names being part of the same closed set of options, and leaves it up to the programmer not to accidentally repeat the same number in two different `#defines`. When the values are related to each other, it's usually cleaner to group them inside an enumeration instead of declaring each one separately.

2. Defining the EnemyStates enum

The syntax of an enum is similar to that of a struct: the enum keyword, the name you want to give the list, a pair of braces, and a semicolon at the end. This enum is called `EnemyStates`, and it's declared inside `enemies.h`, alongside the existing `Enemy` structure:

```
// enemies.h
#pragma once

#include <genesis.h>
#include "res_enemies.h"

// Possible values for an enemy's status field
enum EnemyStates {
    ENEMY_INACTIVE,
    ENEMY_IDLE,
    ENEMY_ATTACK,
    ENEMY_HURT,
    ENEMY_DYING
};
```

Inside the braces you write the list of possible values, one per line and separated by commas, following the usual C convention of writing them in uppercase. The order chosen, `ENEMY_INACTIVE`, `ENEMY_IDLE`, `ENEMY_ATTACK`, `ENEMY_HURT`, and `ENEMY_DYING`, leaves room to add more states in the future, as the enemies' artificial intelligence becomes more elaborate.

It's possible to manually assign a number to each value of the enum, by writing an equals sign followed by that number after each name. But if no number is specified, the compiler automatically assigns consecutive values starting from 0: the first value in the list becomes 0, the second 1, and so on, following the same order in which they were written. It's best to let the compiler handle this automatic assignment rather than writing the numbers by hand, unless there's a specific reason not to follow that consecutive order.

With this declaration, `ENEMY_INACTIVE` is worth 0 and `ENEMY_IDLE` is worth 1, exactly the same values that had already been used since the previous chapter, only now they have a name. `ENEMY_ATTACK`, `ENEMY_HURT`, and `ENEMY_DYING` end up defined with the values 2, 3, and 4 respectively, ready to be used as soon as the project incorporates those behaviors.

3. Replacing the array's numbers with the enum's names

With the enum already defined, you can go back to the `current_enemies` array, in `enemies.c`, and replace the 0 and the 1 that appeared as the first value of each element with `ENEMY_INACTIVE` and `ENEMY_IDLE` respectively:

```
// enemies.c
#include "enemies.h"

struct Enemy current_enemies[MAX_NUMBER_CURRENT_ENEMIES] = {
    { ENEMY_IDLE,      FIX32(40),  FIX32(160), FIX32(1),  NULL, 1, &ninja_sprite_def
      ↪ },
    { ENEMY_IDLE,      FIX32(120), FIX32(160), FIX32(-1), NULL, 1,
      ↪ &sniper_sprite_def },
    { ENEMY_IDLE,      FIX32(200), FIX32(160), FIX32(1),  NULL, 2,
      ↪ &stealth_ninja_sprite_def },
```

```

{ ENEMY_IDLE,      FIX32(280), FIX32(160), FIX32(-1), NULL, 1,
  ↪ &boomerang_sprite_def },
{ ENEMY_IDLE,      FIX32(360), FIX32(160), FIX32(1),  NULL, 1, &ninja_sprite_def
  ↪ },
{ ENEMY_INACTIVE,  FIX32(440), FIX32(160), FIX32(0),  NULL, 1,
  ↪ &sniper_sprite_def },
{ ENEMY_INACTIVE,  FIX32(520), FIX32(160), FIX32(0),  NULL, 2,
  ↪ &stealth_ninja_sprite_def },
{ ENEMY_INACTIVE,  FIX32(600), FIX32(160), FIX32(0),  NULL, 1,
  ↪ &boomerang_sprite_def },
{ ENEMY_INACTIVE,  FIX32(680), FIX32(160), FIX32(0),  NULL, 1,
  ↪ &ninja_sprite_def },
{ ENEMY_INACTIVE,  FIX32(760), FIX32(160), FIX32(0),  NULL, 1,
  ↪ &sniper_sprite_def },
};

```

Although in this example the active enemies all start out in the `ENEMY_IDLE` state, nothing prevents an enemy from starting directly in another state in the future, walking or attacking, depending on what's appropriate for each level situation. What matters is that this first value of each element now reads clearly: just looking at `ENEMY_IDLE` or `ENEMY_INACTIVE` tells you what state each enemy starts in, without having to remember what each number meant.

4. The behavior doesn't change

The names in an enum are, ultimately, just a more readable way of writing the same numbers as always: the compiled program behaves exactly the same as before this change. It's worth confirming this by compiling first without touching anything else, and checking that the on-screen result matches what you had at the end of the previous chapter. Afterward, as a test, you can change the state of one of the already active enemies from `ENEMY_IDLE` to `ENEMY_INACTIVE` and compile again: the expected result is that fewer enemies appear on screen, since that enemy no longer gets registered in `createEnemies`. Once that behavior is confirmed, it's best to set those values back to `ENEMY_IDLE` before continuing.

5. Simplifying the checks in `createEnemies` and `manageEnemies`

The same enum can be used to simplify the `if` checks that already existed in `createEnemies` and `manageEnemies` since the previous chapter. Instead of writing `if (current_enemies[i].status != 0)`, you now write `if (current_enemies[i].status != ENEMY_INACTIVE)`, with exactly the same meaning but much easier to read for anyone reviewing the code later on:

```

void createEnemies(void) {
    for (u8 i = 0; i < MAX_NUMBER_CURRENT_ENEMIES; i++) {
        if (current_enemies[i].status != ENEMY_INACTIVE) {
            current_enemies[i].sprite = SPR_addSprite(current_enemies[i].graphic,
                F32_toInt(current_enemies[i].x), F32_toInt(current_enemies[i].y),
                TILE_ATTR(PAL2, FALSE, FALSE, FALSE));
        }
    }
}

```

```

void manageEnemies(void) {
    for (u8 i = 0; i < MAX_NUMBER_CURRENT_ENEMIES; i++) {
        if (current_enemies[i].status != ENEMY_INACTIVE) {
            current_enemies[i].x += current_enemies[i].vel_x;
            SPR_setPosition(current_enemies[i].sprite,
                F32_toInt(current_enemies[i].x), F32_toInt(current_enemies[i].y));
        }
    }
}

```

Both functions now check `current_enemies[i].status != ENEMY_INACTIVE` instead of comparing against the number 0: `createEnemies` only registers the sprite of enemies that aren't inactive, and `manageEnemies` only updates the position of those same enemies, avoiding moving or drawing a sprite that was never actually created.

An enum is not the same thing as a `#define` macro, even though in practice they serve a similar purpose: giving a readable name to a value. The enemy states are handled with an enum because they form a closed list of values related to one another, all referring to the same idea, an enemy's state; in the following section, on the other hand, `#define` remains the right tool for a standalone constant that isn't part of any list.

6. A named constant for the array's size

There's another number that shouldn't be left loose in the code: the 10 that appears as the size of the `current_enemies` array, and which is also repeated as the upper bound in the `for` loops of `createEnemies` and `manageEnemies`. If that quantity ever needed to change in the future, for example to allow more simultaneous enemies on screen in a more demanding level, you'd have to find and fix each of those occurrences separately, with the risk of missing one.

The solution is the same one already used in earlier chapters for other constants: a `#define` directive with a descriptive name, `MAX_NUMBER_CURRENT_ENEMIES`, set equal to 10, declared in `enemies.h` alongside the enum and the structure:

```

// enemies.h
#pragma once

#include <genesis.h>
#include "res_enemies.h"

#define MAX_NUMBER_CURRENT_ENEMIES 10

enum EnemyStates {
    ENEMY_INACTIVE,
    ENEMY_IDLE,
    ENEMY_ATTACK,
    ENEMY_HURT,
    ENEMY_DYING
};

struct Enemy {
    u8 status;
    fix32 x;
    // One of the values from EnemyStates

```

```

    fix32 y;
    fix32 vel_x;
    Sprite* sprite;
    s8 hitpoints;
    const SpriteDefinition* graphic;
};

extern struct Enemy current_enemies[MAX_NUMBER_CURRENT_ENEMIES];

void createEnemies(void);
void manageEnemies(void);

```

With that macro now defined, the number 10 is replaced both in the array declaration (`current_enemies[MAX_NUMBER_CURRENT_ENEMIES]`) and in the condition of the two for loops (`i < MAX_NUMBER_CURRENT_ENEMIES`), so that changing that quantity in the future is reduced to modifying a single value, in a single place in the code. The `createEnemies` and `manageEnemies` snippets from the previous section already show the final result, with `MAX_NUMBER_CURRENT_ENEMIES` instead of the number 10 repeated in several places.

After modifying a header file this way, it's a good idea to do a project clean before compiling again. In some cases, changes made to a `.h` file aren't correctly reflected in the build unless that cleanup is forced first.

7. What's coming next: structs and enums, once again

Both the structs from the previous chapter and the enums from this chapter will be used many more times later on in this project, and not just for enemies. As an early example, consider the player's animations: in a sprite with several animation frames, instead of referring to the idle frame as number 0 and the walking frame as number 1, an enum would allow you to write something like `PLAYER_IDLE` and `PLAYER_WALKING`, with the same gain in readability seen in this chapter for enemy states. That same pair of tools, a struct to group the fixed attributes of a thing and an enum to replace its numeric values with names, comes up again later in this manual to solve a different problem: that of the different enemy types (the basic ninja, the sniper, the stealth ninja, the one that throws boomerangs), each with its own typical graphic, hit points, and speed, instead of having those values repeated by hand in each element of `current_enemies`.

Review exercises

1. Explain the difference between using several separate `#define` directives and using a single enum to represent a list of related values, such as an enemy's states.
2. Describe the basic syntax for declaring an enum in C, and what value each element receives if no number is assigned to it by hand.
3. Why doesn't replacing the numbers in the `current_enemies` array with the names from the `EnemyStates` enum change the behavior of the compiled program?
4. How does using an enum differ from using a `#define` macro, and when is it best to use each one?
5. What is the `MAX_NUMBER_CURRENT_ENEMIES` constant created in this chapter for, and what problem does it avoid compared to writing the number 10 directly in several places in the code?

Chapter 55 — Permanent Enemy Types

The previous two chapters introduced two tools from the C language: structs, to group an enemy's attributes in a single place, and enums, to give readable names to the different values of the status attribute. This chapter combines both tools to solve a problem that was left open at the end of the chapter on structs: the Enemy struct mixes together, in the same place, data that changes constantly while a level is being played — position, velocity, current state — with data that is actually permanent and specific to each enemy type, such as which graphic it uses, how many hit points it has at the start, or which color palette it's assigned.

The solution is to create a second struct, `EnemyType`, exclusively for that permanent data, and a catalog of enemy types built from it. That way, changing an enemy's type within a level comes down to modifying a single value, instead of manually updating several separate fields at once.

Prerequisites

- Chapters 53 (“C Structs Applied to Game Entities”) and 54 (“C Enumerations (enums)”), since this chapter works directly with the Enemy struct, the `current_enemies` array, the `EnemyStates` enum, and the `createEnemies` and `manageEnemies` functions defined there.
- The difference between ROM and RAM on the Mega Drive, covered in chapter 6, and in particular why the 64 kilobytes of available RAM are a much more limited resource than the cartridge ROM.

1. Separating what's permanent from what changes during the level

The starting point is to go over the Enemy struct field by field and ask which of those fields belong to each individual enemy instance within a level, and which, instead, belong to the enemy type and shouldn't change from one instance to another.

The status field — whether the enemy is inactive, active, attacking, etc. — changes constantly during the level, so it stays in `Enemy`. The same goes for the x and y coordinates, the `vel_x` velocity, and the `sprite` pointer, which identifies a specific instance already added to the screen. The graphic an enemy uses, on the other hand, is something permanent, specific to the enemy type rather than to any particular instance: the boomerang-throwing enemy will always use the same boomerang graphic, no matter how many instances of that type exist in a level.

2. Defining the EnemyType struct

The new struct, `EnemyType`, starts with the graphic field — the same `const SpriteDefinition*` pointer that `Enemy` already had — because that data really is permanent for each enemy type. Two more fields are added next: that type’s starting hit points, and the palette it uses.

```
// enemies.h

// Permanent data for an enemy type: it doesn't change between instances,
// only between different types (boomerang, ninja, sniper, etc.)
struct EnemyType
{
    const SpriteDefinition* graphic;
    s8 max_health;
    u8 palette_used;
};
```

The hit points field needs to exist in both structs at once, though with a slightly different nuance in each: in `Enemy` it represents an instance’s current hit points, which go down as the player hits it; in `EnemyType`, instead, it represents how many hit points any instance of that type should start with when created. To avoid confusing the two meanings, the field in `EnemyType` is called `max_health` instead of `hitpoints`.

Regarding the palette: it would be possible to reserve, say, `PAL3` exclusively for all enemies, following a simple scheme where `PAL0` is for the background, `PAL1` for the foreground, and `PAL2` for the player. But there’s no need to follow that scheme rigidly. Each enemy type can use any of the four palettes available on the Mega Drive, which is why it makes sense for the palette to be just another field of the enemy type, instead of always assuming the same one for everybody.

3. A catalog of enemy types, stored in ROM

With the `EnemyType` struct already defined, an array of that type is declared, `enemyTypesList`, following the same syntax already used for `current_enemies`. It’s given ten slots, on the assumption that a game of this size probably won’t need more than ten different enemy types, counting level bosses; that number can be expanded later if needed.

```
// enemies.h

extern const struct EnemyType enemyTypesList[10];
```

There’s an important difference from `current_enemies`: this array is declared with the `const` keyword before `struct`. A variable marked as `const` stays in the cartridge ROM and is never copied to the console’s RAM. This makes sense for `enemyTypesList` because its content — which graphic, how many maximum hit points, and which palette each enemy type gets — is fixed and never changes during play, unlike `current_enemies`, whose values change all the time and therefore do need to live in RAM.

The Mega Drive has only 64 kilobytes of RAM, a much scarcer resource than the cartridge ROM. That’s why it’s worth marking any data that won’t change during play as `const`, so it stays in ROM and doesn’t take up RAM space unnecessarily.

4. Filling in the catalog with real data and placeholder values

In `enemies.c`, the actual contents of the array are written, using the same brace-list syntax already used for `current_enemies`. Of the four enemy types that already exist as graphic assets — the boomerang enemy, the basic ninja, the stealth ninja, and the sniper — all three fields are filled in: the corresponding graphic, a number of maximum hit points, and a palette.

```
// enemies.c

#include <genesis.h>
#include "enemies.h"
#include "res_enemies.h"

// Catalog of enemy types. Being const, the whole array stays in the
// cartridge ROM and doesn't take up space in the much more limited RAM.
// The health and palette values are provisional: the game's final
// balance hasn't been defined yet, so they can be adjusted later
// without touching any other file.
const struct EnemyType enemyTypesList[10] =
{
    { &boomerang_enemy_sprite, 20, PAL3 }, // BOOMERANG_ENEMY
    { &basic_ninja_sprite, 10, PAL3 }, // BASIC_NINJA_ENEMY
    { &stealth_ninja_sprite, 15, PAL2 }, // STEALTH_ENEMY
    { &sniper_enemy_sprite, 10, PAL3 }, // SNIPER_ENEMY

    // Placeholder slots: there aren't any more enemy types defined
    // yet, so the boomerang's data is repeated as a placeholder
    // until new types are added.
    { &boomerang_enemy_sprite, 20, PAL3 },
    { &boomerang_enemy_sprite, 20, PAL3 },
    { &boomerang_enemy_sprite, 20, PAL3 },
    { &boomerang_enemy_sprite, 20, PAL3 },
    { &boomerang_enemy_sprite, 20, PAL3 },
    { &boomerang_enemy_sprite, 20, PAL3 }
};
```

The stealth ninja uses PAL2 instead of PAL3 in this example, precisely to show that the palette isn't tied to a fixed scheme per entity type: each enemy type can take whichever palette best suits its graphic. The array has ten slots, but there are only four real enemy types so far; the remaining six slots are filled with placeholder values, waiting for more types to be added in the future.

5. Updating the Enemy struct

With the catalog now in place, the graphic field is removed from the `Enemy` struct, since that information now lives exclusively in `EnemyType`. In its place, a new field is added, `enemy_type`, an unsigned 8-bit integer (u8) that indicates which slot in the `enemyTypesList` catalog corresponds to that particular instance.

```
// enemies.h

// Data specific to each enemy instance within a level
```

```

struct Enemy
{
    u8 status;
    fix32 x;
    fix32 y;
    fix32 vel_x;
    Sprite* sprite;
    s8 hitpoints;
    u8 enemy_type;
};

```

6. Readability with a second enum: EnemyNames

Writing bare numbers inside `current_enemies` to indicate each enemy's type would have the same problem already solved in the previous chapter with the `status` attribute: they're hard to remember and hard to read at a glance. The solution is the same: an enum, this time called `EnemyNames`, with one name per catalog slot, in the same order they appear there.

```

// enemies.h

// Readable names for each slot in the enemyTypesList catalog
enum EnemyNames
{
    BOOMERANG_ENEMY,
    BASIC_NINJA_ENEMY,
    STEALTH_ENEMY,
    SNIPER_ENEMY
};

```

With this enum defined, the bare numbers in `current_enemies` are replaced with these names: instead of writing a 0 to indicate that an enemy is of the boomerang type, `BOOMERANG_ENEMY` is written, and likewise for each of the other types.

```

// enemies.c

struct Enemy current_enemies[MAX_NUMBER_CURRENT_ENEMIES] =
{
    { ENEMY_IDLE,      FIX32(50),  FIX32(150), FIX32(1),  NULL, 0, BOOMERANG_ENEMY
      ↪ },
    { ENEMY_IDLE,      FIX32(90),  FIX32(150), FIX32(-1), NULL, 0, BASIC_NINJA_ENEMY
      ↪ },
    { ENEMY_IDLE,      FIX32(130), FIX32(150), FIX32(1),  NULL, 0, STEALTH_ENEMY
      ↪ },
    { ENEMY_IDLE,      FIX32(170), FIX32(150), FIX32(-1), NULL, 0, SNIPER_ENEMY
      ↪ },
    { ENEMY_IDLE,      FIX32(210), FIX32(150), FIX32(1),  NULL, 0, BOOMERANG_ENEMY
      ↪ },
    { ENEMY_INACTIVE,  FIX32(0),    FIX32(0),    FIX32(0),  NULL, 0, BOOMERANG_ENEMY
      ↪ },
    { ENEMY_INACTIVE,  FIX32(0),    FIX32(0),    FIX32(0),  NULL, 0, BOOMERANG_ENEMY
      ↪ },
    { ENEMY_INACTIVE,  FIX32(0),    FIX32(0),    FIX32(0),  NULL, 0, BOOMERANG_ENEMY
      ↪ },
};

```

```

    { ENEMY_INACTIVE, FIX32(0),    FIX32(0),    FIX32(0),    NULL, 0, BOOMERANG_ENEMY
    ↪ },
    { ENEMY_INACTIVE, FIX32(0),    FIX32(0),    FIX32(0),    NULL, 0, BOOMERANG_ENEMY
    ↪ }
};

```

The hitpoints field is left at 0 for all ten elements: as shown in the next section, `createEnemies` will fill it in automatically from the catalog, so there's no need to write it by hand for each element.

7. Reading the catalog from `createEnemies`

With the `graphic` field now removed from the `Enemy` struct, `createEnemies` needs to be adjusted: instead of reading the graphic directly from `current_enemies[i].graphic`, it now has to be looked up in the catalog, using that instance's enemy type as the index: `enemyTypesList[current_enemies[i].enemy_type].graphic`. If the enemy type is `BOOMERANG_ENEMY` (slot 0 of the catalog), this expression returns the boomerang's graphic; if it's any other type, it returns the graphic corresponding to that slot.

The same pattern is repeated twice more inside `createEnemies`. The fixed palette within `TILE_ATTR` is replaced with `enemyTypesList[current_enemies[i].enemy_type].palette_used`, so each enemy type appears with the palette that matches it according to the catalog. And the starting hit points are no longer written as a bare number: `current_enemies[i].hitpoints = enemyTypesList[current_enemies[i].enemy_type].max_health` makes each instance start the level with the maximum that its type is entitled to.

```

// enemies.c

void createEnemies()
{
    u16 i;

    for (i = 0; i < MAX_NUMBER_CURRENT_ENEMIES; i++)
    {
        if (current_enemies[i].status != ENEMY_INACTIVE)
        {
            // A pointer to the corresponding enemy type is stored,
            // to avoid repeating the catalog lookup three times
            const struct EnemyType* type =
                ↪ &enemyTypesList[current_enemies[i].enemy_type];

            current_enemies[i].sprite = SPR_addSprite(type->graphic,
                F32_toInt(current_enemies[i].x), F32_toInt(current_enemies[i].y),
                TILE_ATTR(type->palette_used, FALSE, FALSE, FALSE));

            current_enemies[i].hitpoints = type->max_health;
        }
    }
}

```

`manageEnemies` doesn't need any further changes: it still works only with the fields that change during the level — `x`, `vel_x`, and `sprite` — which remain in `Enemy` unmodified.

8. Result and the advantage of this approach

When compiled, enemies appear on screen with the graphic and palette that correspond to their type, read from `enemyTypesList` via their `enemy_type`.

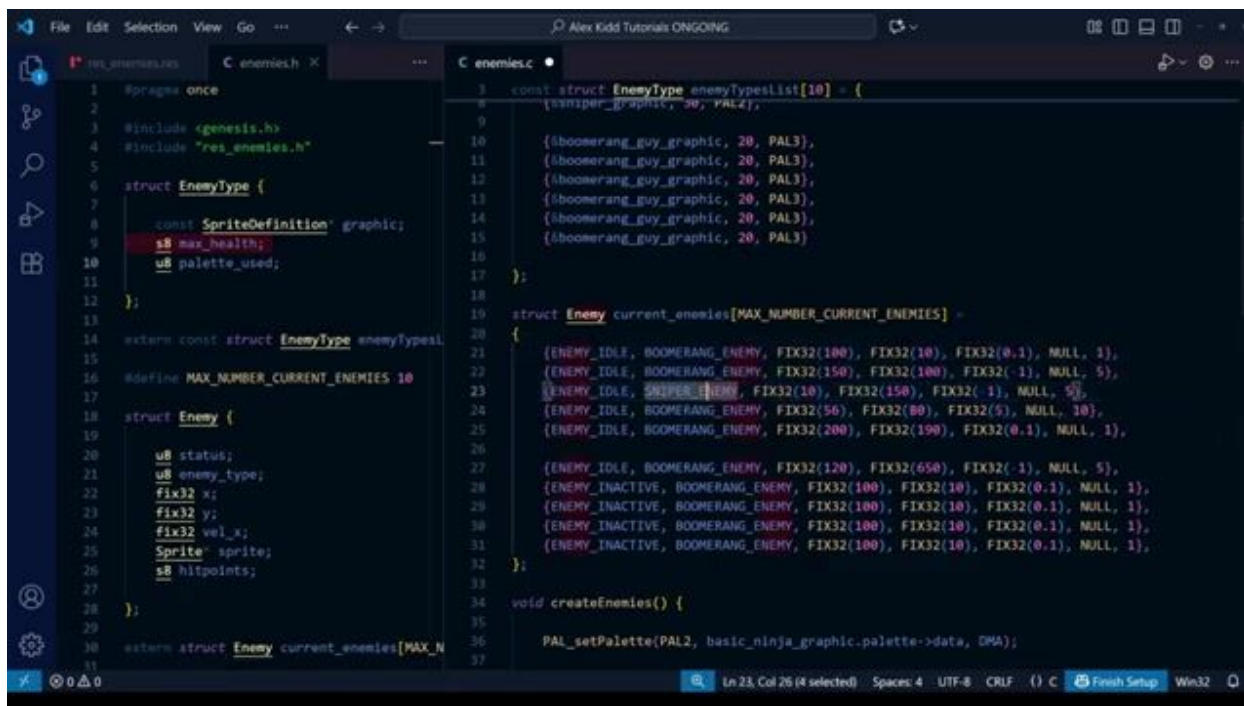


Figure 149: The `current_enemies` array now with `enemy_type`

The finished `current_enemies` array: each instance carries a single `enemy_type` field instead of repeating `graphic`, `health`, and `palette` separately.

Changing any enemy's type within `current_enemies` shows the advantage of this approach: it's enough to change the enum name in a single place, for example from `BOOMERANG_ENEMY` to `SNIPER_ENEMY`, and that enemy switches to the sniper's graphic, palette, and hit points without touching any other field by hand. Before this chapter, the same change would have required separately updating the graphic, velocity, and hit points in each affected instance, with the risk of forgetting one.

This same scheme — a lightweight per-instance struct, plus a const catalog indexed by an enum — is the one revisited throughout the rest of this part of the manual to solve similar problems, such as assigning a different artificial intelligence behavior to each enemy type without duplicating code between instances.

Review exercises

1. Explain, in your own words, the criterion this chapter uses to decide which fields go in the `Enemy` struct and which go in the new `EnemyType` struct.
2. Why is the `enemyTypesList` array declared with the `const` keyword, and what practical difference does that make compared to the `current_enemies` array, which doesn't use it?
3. What is the `enemy_type` field in the `Enemy` struct for, and what data type does it use?
4. Explain what the expression `enemyTypesList[current_enemies[i].enemy_type].graphic` does, and why it's necessary after removing the `graphic` field from the `Enemy` struct.

5. Why was it necessary to name the hit points field in `EnemyType` as `max_health` instead of using the same name as in `Enemy`?

Chapter 56 — Function Pointers for Enemy AI

How to store an enemy's behavior, not just its data, inside its type

Chapter 55 solved a real organizational problem: it separated the permanent data of each enemy type (its graphic, its maximum health, its palette) into a struct of its own, `EnemyType`, and gathered them into a catalog, `enemyTypesList`, indexed by the `enemy_type` field of each instance inside `current_enemies`. Thanks to that, changing an enemy's type within a level was reduced to changing a single value of the `EnemyNames` enum, instead of manually updating several separate fields.

However, that chapter left a pending problem, and a fairly obvious one at that: regardless of an enemy's type, `manageEnemies()` still applies exactly the same artificial intelligence (AI) to all of them. That AI, inherited from earlier chapters of the project, simply adds `vel_x` to the enemy's x coordinate and updates its sprite's position. That's a reasonable behavior as a starting point, but it makes no sense for a boomerang-throwing enemy, a regular ninja, a stealth ninja, and a sniper to all move the exact same way.

This chapter solves that problem in two stages. First, in the most straightforward way possible: a distinct AI function for each enemy type, selected with a chain of `if / else if`. That solution works, but it doesn't scale well as new enemy types get added. That's why the second stage brings in a C language mechanism that hasn't been used yet in this manual: **function pointers**, which allow the AI of each enemy type to be stored as just another piece of data inside struct `EnemyType`, alongside the graphic, health, and palette, and invoked directly without first asking what type it is.

Pointers in general (what they are, how they're declared, how they point to data in memory) were already explained in depth in chapter 6 of this manual. This chapter assumes that content is already known and focuses on what's new: a pointer that, instead of pointing to a piece of data, points to a function.

1. One AI function per enemy type

The first step is to create, for each of the four enemy types defined in the catalog (boomerang, basic ninja, stealth ninja, and sniper), an independent function with its own movement logic. Each of these functions needs a single piece of data: the enemy's position within `current_enemies`, that is, a number between 0 and `MAX_ENEMIES - 1`. That parameter is called `enemy_number` and is declared as `u8`; the name is a stylistic choice, not a language requirement.

The prototypes are added in `enemies.h`, alongside the already existing ones for `createEnemies()` and `manageEnemies()`, following the pattern `enemyName + AI`:


```
// enemies.h (excerpt)
void createEnemies(void);
void manageEnemies(void);

// AI functions, one per enemy type
void boomerangAI(u8 enemy_number);
void basicNinjaAI(u8 enemy_number);
void stealthAI(u8 enemy_number);
void sniperAI(u8 enemy_number);
```

It's worth remembering that `extern` only makes sense in a function's *declaration* inside a header file; it should never appear in its *definition* inside the corresponding `.c` file, since that's where the function's body is actually written. With that in mind, the bodies are filled in in `enemies.c`. For now the goal isn't a definitive, elaborate AI for each enemy (that will be revisited later in this manual), but simply to verify that the selection mechanism works, giving each function a visible difference from the others: some move along the `x` axis, and at least one moves along the `y` axis.

```
// enemies.c (excerpt)
void boomerangAI(u8 enemy_number) {
    current_enemies[enemy_number].x += FIX32(1.5);
    SPR_setPosition(current_enemies[enemy_number].sprite,
        F32_toInt(current_enemies[enemy_number].x),
        F32_toInt(current_enemies[enemy_number].y));
}

void basicNinjaAI(u8 enemy_number) {
    current_enemies[enemy_number].x += FIX32(2.5);
    SPR_setPosition(current_enemies[enemy_number].sprite,
        F32_toInt(current_enemies[enemy_number].x),
        F32_toInt(current_enemies[enemy_number].y));
}

void stealthAI(u8 enemy_number) {
    // Unlike the others, this one moves along the y axis
    current_enemies[enemy_number].y += FIX32(0.5);
    SPR_setPosition(current_enemies[enemy_number].sprite,
        F32_toInt(current_enemies[enemy_number].x),
        F32_toInt(current_enemies[enemy_number].y));
}

void sniperAI(u8 enemy_number) {
    // Combines both axes
    current_enemies[enemy_number].x += FIX32(1.0);
    current_enemies[enemy_number].y -= FIX32(0.5);
    SPR_setPosition(current_enemies[enemy_number].sprite,
        F32_toInt(current_enemies[enemy_number].x),
        F32_toInt(current_enemies[enemy_number].y));
}
```

Notice one design detail: each function uses a fixed speed value, written directly with `FIX32()`, instead of reading the `vel_x` field of `current_enemies[enemy_number]`. This is intentional. The `vel_x` field was defined in earlier chapters as a per-instance editable value, meant for a stage of the project where the concept of enemy type didn't exist yet; if two instances of the same

type (two boomerang enemies, for example) had different `vel_x` values loaded by hand into `current_enemies`, they would move at different speeds despite being the same type, which is inconsistent with the central idea of chapter 55: that an enemy's behavior depends on its type, not on loose values loaded per instance. That's why each AI function defines its own speed as an internal constant. The `vel_x` field can still exist in `struct Enemy` for other future uses (for example, an AI that does need to modify its speed at runtime), but it is no longer the source that the functions in this chapter draw their movement from.

2. Choosing the right AI in `manageEnemies()`

With the four functions already written, the next step is to modify `manageEnemies()` so it decides which one to run based on the current enemy's `enemy_type` field. The most direct way is a chain of `if / else if`, comparing that field against each value of the `EnemyNames` enum:

```
void manageEnemies(void) {
    for (u8 i = 0; i < MAX_ENEMIES; i++) {
        if (current_enemies[i].status != ENEMY_INACTIVE) {
            if (current_enemies[i].enemy_type == BOOMERANG_ENEMY)
                boomerangAI(i);
            else if (current_enemies[i].enemy_type == BASIC_NINJA_ENEMY)
                basicNinjaAI(i);
            else if (current_enemies[i].enemy_type == STEALTH_ENEMY)
                stealthAI(i);
            else if (current_enemies[i].enemy_type == SNIPER_ENEMY)
                sniperAI(i);
        }
    }
}
```

In every branch, the argument passed is `i`, the loop variable of the `for` loop, which is exactly the same value that each AI function internally calls `enemy_number`: the enemy's position within `current_enemies`.

Figure 1: The four enemy types on screen, each moving according to its own AI function.

With only four enemy types, this block is perfectly readable, and there's nothing wrong with leaving it as is. The problem appears when thinking about the scale of the full project: if the `enemyTypesList` catalog ended up having, say, fifty different enemy types — the maximum the array size would allow — `manageEnemies()` would end up with fifty conditions in a row just to decide which function to call. It's the same kind of scalability problem that's already been solved earlier in this project: with enums instead of a long list of `#defines`, and with structs instead of independent parallel arrays. The more enemy types the game has, the more obvious the need for a tidier solution becomes.

3. Storing behavior, not just data

The AI an enemy uses is, at bottom, a feature just as permanent to its type as its graphic, its maximum health, or its palette: every boomerang enemy always uses `boomerangAI`, every stealth ninja always uses `stealthAI`, and that association never changes during gameplay. If the graphic, health, and palette already live inside `struct EnemyType`, it makes sense for the AI function to live there too, instead of being decided every frame through a chain of comparisons inside `manageEnemies()`.

The difference from the other `EnemyType` fields is that, instead of storing a number or a pointer

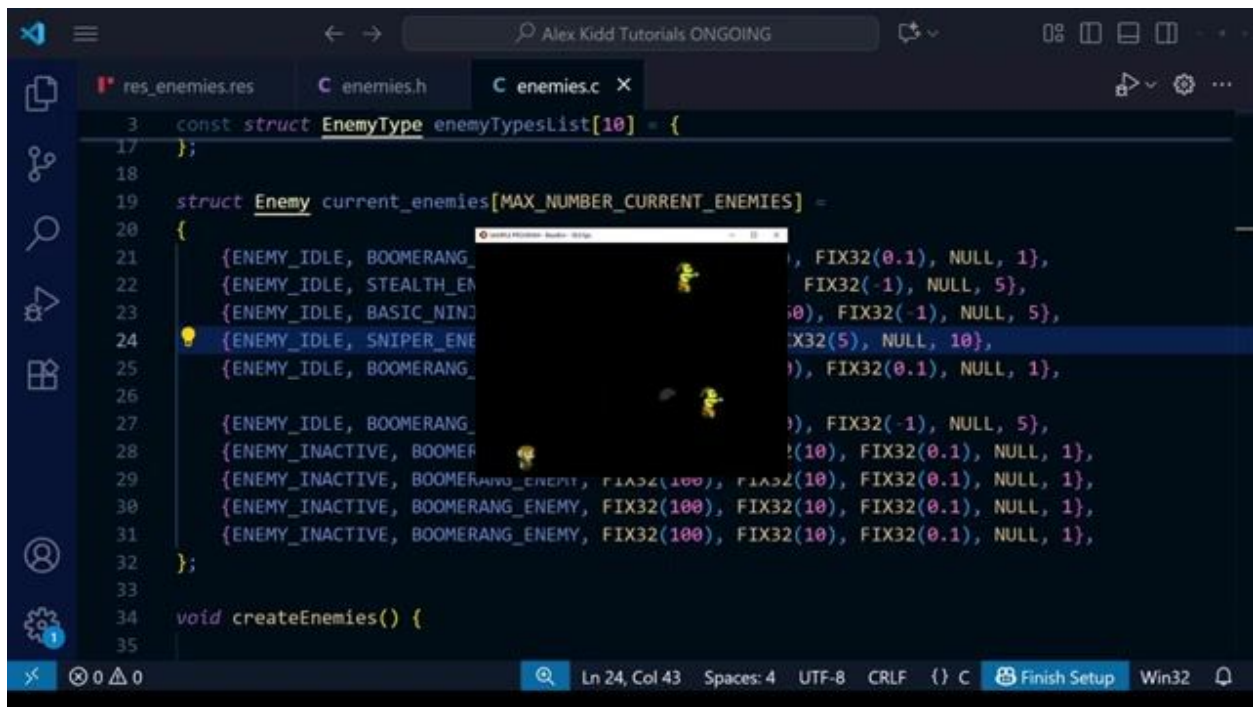


Figure 150: The four enemy types on screen, each moving according to its own AI function

to a graphical resource, what gets stored now is the function itself that must be executed. That's what **function pointers** are for: variables capable of storing the address of a function, as long as that function has the exact signature the pointer expects (same return type, same number and type of parameters).

3.1. The syntax of a function pointer

Declaring a field of this type inside struct EnemyType is written like this:

```
void (*enemyAI)(u8 enemy_number);
```

The syntax looks a little strange at first glance, because of the extra parentheses and the asterisk, but it's easier to understand by comparing it line by line with the normal prototype of one of the AI functions:

```
void (*enemyAI)(u8 enemy_number); // the function pointer
void boomerangAI(u8 enemy_number); // a normal function, for comparison
```

The underlying structure is the same in both cases: first the return type (void, returns nothing), then a name, and between parentheses the parameter list (a single u8 enemy_number). The only real difference is that, in the function pointer, the chosen name (enemyAI) is enclosed in parentheses together with an asterisk to its left: (*enemyAI). That asterisk is what indicates that this isn't declaring a normal function, but rather a variable capable of storing the address of any function that matches that signature: one that returns void and takes a single u8 parameter.

The general pattern for declaring a function pointer is returnType (*chosenName)(parameters);. The parentheses around *chosenName are mandatory: without them, void *enemyAI(u8 enemy_number); would mean something com-

pletely different (a normal function, called `enemyAI`, that returns a pointer to `void`).

With that, the field is added to struct `EnemyType`, right after `palette_used`:

```
// enemies.h (excerpt)
struct EnemyType {
    const SpriteDefinition* graphic;
    u8 max_health;
    u8 palette_used;
    // AI function that any instance of this enemy type runs
    void (*enemyAI)(u8 enemy_number);
};
```

3.2. Assigning the AI for each type in `enemyTypesList`

With the field now declared, the next step is to fill it in for each existing entry of `enemyTypesList`, adding the corresponding function name at the end of each brace block, after the graphic, health, and palette. In C, the name of a function used without parentheses or arguments evaluates to a pointer to that function, so no special syntax is needed for this assignment: writing the name is enough.

```
// enemies.c (excerpt)
const struct EnemyType enemyTypesList[MAX_ENEMY_TYPES] = {
    { &boomerang_sprite, 10, PAL3, boomerangAI },
    { &basic_ninja_sprite, 15, PAL3, basicNinjaAI },
    { &stealth_ninja_sprite, 12, PAL3, stealthAI },
    { &sniper_sprite, 8, PAL3, sniperAI },
    // Filler slots, waiting for new enemy types
    { &boomerang_sprite, 10, PAL3, boomerangAI },
    { &boomerang_sprite, 10, PAL3, boomerangAI },
    { &boomerang_sprite, 10, PAL3, boomerangAI },
    { &boomerang_sprite, 10, PAL3, boomerangAI },
    { &boomerang_sprite, 10, PAL3, boomerangAI },
    { &boomerang_sprite, 10, PAL3, boomerangAI },
};
```

Since the prototypes of the four functions are already declared in `enemies.h`, and `enemies.c` includes that header at the top, the compiler already knows each one's signature by the time it fills in this list, regardless of the order in which the function definitions appear further down in the same file.

3.3. `manageEnemies()` reduced to one line

With each type's AI now stored in the catalog, the chain of `if / else if` is no longer needed. It's enough to use the current enemy's `enemy_type` field as an index into `enemyTypesList`, access its `enemyAI` field, and call it with parentheses exactly like any other function, passing it `i` as the argument:

```
void manageEnemies(void) {
    for (u8 i = 0; i < MAX_ENEMIES; i++) {
        if (current_enemies[i].status != ENEMY_INACTIVE)
            enemyTypesList[current_enemies[i].enemy_type].enemyAI(i);
    }
}
```

The expression `enemyTypesList[current_enemies[i].enemy_type].enemyAI(i)` reads from the inside out: `current_enemies[i].enemy_type` gives the enemy type's position within the catalog; `enemyTypesList[...]` accesses that entry; `.enemyAI` retrieves the function pointer stored there; and `(i)` invokes it, passing `i` as `enemy_number`. Since the body of the `if` is now reduced to this single statement, the braces that used to surround it are no longer necessary and can be removed, leaving the whole statement on a single line.

The result on screen is identical to the one obtained with the `if / else if` chain: each enemy still moves according to the function corresponding to its type. What changed is the way of reaching that result. Now, adding a new enemy type to the game — enemy number fifty, if it came to that — doesn't require touching `manageEnemies()` at all: it's enough to write its AI function, declare its prototype in `enemies.h`, and add a new entry to `enemyTypesList` with that function assigned to the `enemyAI` field. `manageEnemies()` is done for good, no matter how many enemy types the game ends up having.

4. Complete reference code

For reference, here is the header file with everything this chapter contributes, integrated with what was already defined in chapters 53, 54, and 55:

```
// enemies.h
#ifndef ENEMIES_H
#define ENEMIES_H

#include <genesis.h>
#include "res_enemies.h"

#define MAX_ENEMIES      10
#define MAX_ENEMY_TYPES  10

// State of an enemy instance within the level (chapter 54)
enum EnemyStatus {
    ENEMY_INACTIVE,
    ENEMY_ACTIVE,
    ENEMY_ATTACKING
};

// Readable names for each position in the enemyTypesList catalog (chapter 55)
enum EnemyNames {
    BOOMERANG_ENEMY,
    BASIC_NINJA_ENEMY,
    STEALTH_ENEMY,
    SNIPER_ENEMY
};

// Data that changes instance by instance, within a level (chapter 53)
struct Enemy {
    u8 status;
    fix32 x;
    fix32 y;
    fix32 vel_x;
    u8 hitpoints;
    u8 enemy_type;
    Sprite* sprite;
};
```

```

};

// Permanent data, specific to each enemy type (chapter 55),
// plus the AI function that corresponds to it (this chapter)
struct EnemyType {
    const SpriteDefinition* graphic;
    u8 max_health;
    u8 palette_used;
    void (*enemyAI)(u8 enemy_number);
};

extern struct Enemy current_enemies[MAX_ENEMIES];
extern const struct EnemyType enemyTypesList[MAX_ENEMY_TYPES];

void createEnemies(void);
void manageEnemies(void);

void boomerangAI(u8 enemy_number);
void basicNinjaAI(u8 enemy_number);
void stealthAI(u8 enemy_number);
void sniperAI(u8 enemy_number);

#endif // ENEMIES_H

```

And here is `enemies.c`, with `createEnemies()` reading the graphic, palette, and initial health from the catalog (a mechanism already explained in chapter 55) and `manageEnemies()` now using the function pointer:

```

// enemies.c
#include "enemies.h"

struct Enemy current_enemies[MAX_ENEMIES] = {
    { ENEMY_ACTIVE,    FIX32(100), FIX32(160), 0, 0, BOOMERANG_ENEMY, NULL },
    { ENEMY_ACTIVE,    FIX32(140), FIX32(160), 0, 0, BASIC_NINJA_ENEMY, NULL },
    { ENEMY_ACTIVE,    FIX32(180), FIX32(160), 0, 0, STEALTH_ENEMY,    NULL },
    { ENEMY_ACTIVE,    FIX32(220), FIX32(160), 0, 0, SNIPER_ENEMY,      NULL },
    { ENEMY_INACTIVE,  0, 0, 0, 0, BOOMERANG_ENEMY, NULL },
    { ENEMY_INACTIVE,  0, 0, 0, 0, BOOMERANG_ENEMY, NULL },
    { ENEMY_INACTIVE,  0, 0, 0, 0, BOOMERANG_ENEMY, NULL },
    { ENEMY_INACTIVE,  0, 0, 0, 0, BOOMERANG_ENEMY, NULL },
    { ENEMY_INACTIVE,  0, 0, 0, 0, BOOMERANG_ENEMY, NULL },
    { ENEMY_INACTIVE,  0, 0, 0, 0, BOOMERANG_ENEMY, NULL },
};

const struct EnemyType enemyTypesList[MAX_ENEMY_TYPES] = {
    { &boomerang_sprite, 10, PAL3, boomerangAI },
    { &basic_ninja_sprite, 15, PAL3, basicNinjaAI },
    { &stealth_ninja_sprite, 12, PAL3, stealthAI },
    { &sniper_sprite, 8, PAL3, sniperAI },
    { &boomerang_sprite, 10, PAL3, boomerangAI },
    { &boomerang_sprite, 10, PAL3, boomerangAI },
    { &boomerang_sprite, 10, PAL3, boomerangAI },
    { &boomerang_sprite, 10, PAL3, boomerangAI },
    { &boomerang_sprite, 10, PAL3, boomerangAI },
};

```

```

    { &boomerang_sprite, 10, PAL3, boomerangAI },
};

void createEnemies(void) {
    for (u8 i = 0; i < MAX_ENEMIES; i++) {
        if (current_enemies[i].status != ENEMY_INACTIVE) {
            u8 type = current_enemies[i].enemy_type;

            current_enemies[i].sprite = SPR_addSprite(
                enemyTypesList[type].graphic,
                F32_toInt(current_enemies[i].x),
                F32_toInt(current_enemies[i].y),
                TILE_ATTR(enemyTypesList[type].palette_used, FALSE, FALSE, FALSE));

            current_enemies[i].hitpoints = enemyTypesList[type].max_health;
        }
    }
}

void manageEnemies(void) {
    for (u8 i = 0; i < MAX_ENEMIES; i++) {
        if (current_enemies[i].status != ENEMY_INACTIVE)
            enemyTypesList[current_enemies[i].enemy_type].enemyAI(i);
    }
}

void boomerangAI(u8 enemy_number) {
    current_enemies[enemy_number].x += FIX32(1.5);
    SPR_setPosition(current_enemies[enemy_number].sprite,
        F32_toInt(current_enemies[enemy_number].x),
        F32_toInt(current_enemies[enemy_number].y));
}

void basicNinjaAI(u8 enemy_number) {
    current_enemies[enemy_number].x += FIX32(2.5);
    SPR_setPosition(current_enemies[enemy_number].sprite,
        F32_toInt(current_enemies[enemy_number].x),
        F32_toInt(current_enemies[enemy_number].y));
}

void stealthAI(u8 enemy_number) {
    current_enemies[enemy_number].y += FIX32(0.5);
    SPR_setPosition(current_enemies[enemy_number].sprite,
        F32_toInt(current_enemies[enemy_number].x),
        F32_toInt(current_enemies[enemy_number].y));
}

void sniperAI(u8 enemy_number) {
    current_enemies[enemy_number].x += FIX32(1.0);
    current_enemies[enemy_number].y -= FIX32(0.5);
    SPR_setPosition(current_enemies[enemy_number].sprite,
        F32_toInt(current_enemies[enemy_number].x),
        F32_toInt(current_enemies[enemy_number].y));
}

```


}

The names `boomerang_sprite`, `basic_ninja_sprite`, `stealth_ninja_sprite`, and `sniper_sprite` correspond to the `SpriteDefinitions` generated by `rescomp` from `res_enemies.res`, using the same mechanism seen in earlier chapters of this manual.

Review exercises

1. Explain in your own words why applying the same AI function to every enemy in the game, without distinguishing its type, becomes a problem as more enemy types are added.
2. What does the `enemy_number` parameter represent in the `boomerangAI`, `basicNinjaAI`, `stealthAI`, and `sniperAI` functions? Which exact piece of data in `current_enemies` does it correspond to?
3. Write the general syntax for declaring, inside a struct, a function pointer that returns nothing and takes a single `u8` parameter. What role do the parentheses around the field name play?
4. Explain the difference between resolving AI selection with a chain of `if / else if` inside `manageEnemies()`, and resolving it with an `enemyAI` field of function pointer type inside struct `EnemyType`. What concrete advantage does the second option offer as the number of enemy types grows?
5. What does the expression `enemyTypesList[current_enemies[i].enemy_type].enemyAI(i);` mean, and exactly what does it replace inside `manageEnemies()`?

Chapter 57 — Game States

A minimal state machine to separate loading a level from running it, put to double use by turning the player into a struct along the way

Up to this point in the project, the main loop always does the same thing on every pass: it processes the controls, moves the player, updates the enemies in the `current_enemies` array, and refreshes the sprites on screen. That assumption—that the game is always playing a level—was convenient while the project was limited to a single scrolling screen, but it stops holding up as soon as the game needs to do something other than play: load a new level, show a menu, or present a pause screen. If the player-control code runs unconditionally, it will keep moving the character even while the game should be busy loading data, which makes no sense and can produce strange behavior.

This chapter introduces the concept of a **game state**: a variable that indicates at all times what the game is doing, and which both the main loop and the control functions check before acting. It's a simple idea to state, but it's the foundation on which the actual transition between levels will be built in the coming chapters. Along the way, and because it's worth resolving before moving further, this chapter also refactors the player's data—until now, loose variables with hand-written coordinates—into a struct `Player`, following the same pattern already used for the enemies.

Prerequisites

Before continuing, it helps to have on hand:

- The project as organized in Chapter 51 (“Organizing the project with header files”): the `player.h/player.c`, `enemy.h/enemy.c`, and `camera.h/camera.c` pairs, using `extern` to share variables and functions between files.
- Chapters 52 through 56 of this part, which already turned the enemies into a `current_enemies` array of struct `Enemy` (Chapter 53, “C structs applied to game entities”), with their own types and states declared as an enum (Chapter 54, “C enumerations (enums)”) and their AI resolved with function pointers (Chapter 56).
- The two control functions already used in this manual, `handle_input()` and `joyEvent()`, covered respectively in Chapters 31 (“D-pad controls”) and 32 (“Button controls and event-based reading”), along with the `controls.h/controls.c` pair where this project's input code lives.
- The difference between `fix32` (32-bit fixed point) and `s16` (16-bit signed integer), explained in Chapter 6.

1. Why the main loop needs to know what state the game is in

At the project's current stage, the player and several enemies already move on screen thanks to what was developed in previous chapters. All the control code runs unconditionally: as long as the program is running, the D-pad moves the player and the enemy AI keeps advancing. This works because, so far, the game only knows how to do one thing—play the level currently loaded in memory—so there has never been a need to distinguish any other situation.

The problem appears as soon as you think about what comes next: loading a new level, showing a title screen, pausing the game. Each of these situations requires the game to stop behaving as if it were playing a level. Without an explicit way to tell what situation the program is in, there's no way to tell the control code, "don't move the player right now, we're still loading."

The solution is to add a variable that explicitly records what state the game is in at any given moment, and to make both the main loop and the control functions check it before acting. This idea—a program that is, at all times, in one of several possible states, and that reacts differently depending on which state that is—is, in essence, a **finite state machine**, an extremely common design pattern in game development. It already showed up, without being named as such, when each enemy was given its own state enum in Chapter 54; now the exact same approach is applied, but at the scale of the whole game.

2. The GameStates enum and the current_game_state variable

2.1. Declaring the possible states

Following the same approach already used for enemy states and enemy types, the game states are declared as an enum. This avoids representing states as bare numbers: a 0 or a 1 with no name communicates nothing on its own, while `LOADING` and `RUNNING_LEVEL` do. In `levels.h`, a new enum called `GameStates` is created with two values to start: `LOADING`, which represents the moment when the game is loading the resources for a level or a new screen, and `RUNNING_LEVEL`, which represents the moment when a level that's already loaded is in progress.

```
// inc/levels.h
#ifndef LEVELS_H
#define LEVELS_H

#include <genesis.h>

// Possible game states. Just two for now: loading data,
// or playing the level that's already in memory. More will
// be added later: a title screen, a menu, maybe a level
// select map.
typedef enum {
    LOADING,
    RUNNING_LEVEL
} GameStates;

// Current game state, checked from controls.c and from
// the rest of the project before moving the player or the enemies
extern u8 current_game_state;
```

```
#endif // LEVELS_H
```

It's worth noting that, although the cartridge doesn't have the long loading times of an optical medium—loading a level's data can take as little as one or two sixtieths of a second—a dedicated loading state is still necessary. The reason isn't the length of the wait, but the need for the loading process not to be interrupted by the rest of the game's code: while data is being copied to VRAM or structures are being initialized, it's best that player control or enemy AI don't run at the same time.

For now only these two values are needed, but since the enum already exists, adding a new state in the future—a title screen, a menu—comes down to writing one more name inside its braces, without touching the rest of the code that already distinguishes between states by name.

2.2. Storing the current state in a variable

Declaring the possible states isn't enough: the game also needs a variable that records which of them is the current state at any given instant. For this, `current_game_state` is created as an unsigned 8-bit integer (`u8`), the same type already used for each enemy's state in Chapter 54: with a range of 0 to 255 there's more than enough room for any reasonable number of game states, and using a larger type would waste memory for no benefit.

Like any variable, `current_game_state` needs an initial value. In `levels.c` it's assigned `RUNNING_LEVEL`, so the game starts out directly playing the level that's already on screen:

```
// src/levels.c
#include "levels.h"

u8 current_game_state = RUNNING_LEVEL;
```

This choice is consistent with the project's current state—there's still no real level-loading mechanism, a subject picked back up in Chapter 58 ("Loading different enemies per level") and in Chapters 60 and 61—but once real loading is implemented, it will make more sense for the game to start directly in the `LOADING` state.

Going deeper. It's worth naming the pattern explicitly, because it's going to keep coming back throughout this manual: what's being built here is a finite state machine, a model in which a system can only be, at any given instant, in one of a fixed set of states, and where specific events, like pressing the Start button, trigger transitions from one state to another. In game engines, almost any element with complex behavior (the flow of the game, an enemy with AI, an options menu) ends up being modeled this way, because it keeps the code organized: instead of a tangle of conditions stepping on each other, each state knows what to do and which other states it can move to.

3. Changing state from `joyEvent()`

With the state already declared, a mechanism is needed to change it in response to the player's action. As already seen in Chapters 31 and 32, SGDK distinguishes between two input functions with different purposes: `joyEvent()`, which fires once per button press and is suited to one-off actions, and `handle_input()`, which runs on every pass of the main loop and is suited to continuous actions, such as moving the player while the D-pad is held down. Advancing

to the next screen when Start is pressed is exactly the kind of one-off action `joyEvent()` is meant for.

Inside `joyEvent()`, the transition logic is added: if the current state is `RUNNING_LEVEL` and the button that changed in this event was Start, `current_game_state` is set to `LOADING`. The check on the current state isn't a minor detail: it's what will allow pressing Start to mean different things depending on what part of the game you're in—jumping to the next screen during gameplay, but perhaps confirming an option inside a menu. Each new state added in future chapters will likely need its own branch inside these two control functions.

```
// src/controls.c
#include "controls.h"
#include "levels.h"
#include "player.h"

void handle_input(void) {
    if (current_game_state != RUNNING_LEVEL) return;

    u16 value = JOY_readJoypad(JOY_1);

    if (value & BUTTON_RIGHT) {
        player_character.x_f32 += FIX32(2);
    } else if (value & BUTTON_LEFT) {
        player_character.x_f32 -= FIX32(2);
    }
}

void joyEvent(u16 joy, u16 changed, u16 state) {
    if (current_game_state == RUNNING_LEVEL && (changed & state & BUTTON_START)) {
        current_game_state = LOADING;
    }
}
```

Watch out. A common mistake when introducing a new variable in a file other than the one where it was declared is forgetting the `#include` for the corresponding header. In this case, `controls.c` needs `current_game_state`, `RUNNING_LEVEL`, and `LOADING`, and none of the three live in `controls.h`: they live in `levels.h`. If that `#include "levels.h"` were missing, the compiler would flag each of those three identifiers as unrecognized, an error that disappears as soon as the missing `#include` is added. It isn't a conceptual problem with the state machine: it's simply a consequence of how C shares knowledge of symbols between files, and it's the same kind of error—an identifier that exists, but whose header wasn't included where it's needed—that already came up when the project was split apart in Chapter 51, and one worth recognizing at a glance whenever it reappears.

`controls.h`, for its part, only needs to declare the two functions (their bodies, and therefore their internal dependencies, stay in `controls.c`):

```
// inc/controls.h
#ifndef CONTROLS_H
#define CONTROLS_H

#include <genesis.h>

void handle_input(void);
void joyEvent(u16 joy, u16 changed, u16 state);
```

```
#endif // CONTROLS_H
```

4. From loose variables to a struct Player

4.1. Defining struct Player

Before wrapping up player control, it's worth resolving a design problem that had been dragging on since the earliest chapters of this project: the player's data—its sprite and its coordinates—still exist as loose variables, with its initial coordinates written directly as fixed numbers at the moment the sprite is created. Just as Chapter 53 grouped each enemy's attributes into a struct `Enemy`, here it makes sense to group the player's into a struct `Player`.

The new struct includes the player's sprite and two versions of its X and Y coordinates: one in `fix32`, the 32-bit fixed-point type, and another in `s16`, a 16-bit signed integer. The reason for keeping both versions, rather than just one, was already explained in Chapter 6: the player's movement needs numbers with a fractional part (`fix32`) because an acceleration of 0.5 or a jump with gravity rarely advances in whole pixel numbers; however, `SPR_setPosition()`—and, later on, the level collision routines and the collision box calculations—expect to receive 16-bit integers. Keeping both versions avoids converting between types on every use, though it requires remembering to keep them in sync.

```
// inc/player.h
#ifndef PLAYER_H
#define PLAYER_H

#include <genesis.h>
#include "res_player.h"

// Groups all of the player's data into a single structure,
// following the same approach already used in struct Enemy (Chapter 53)
typedef struct {
    Sprite* sprite;
    fix32 x_f32;
    fix32 y_f32;
    s16 x_s16;
    s16 y_s16;
} Player;

extern Player player_character;

// Creates the player's sprite and adds it to the scene
void create_player(void);

// Updates the on-screen position of the player's sprite
void manage_player(void);

#endif // PLAYER_H
```

Unlike the enemies, which are handled as an array of structs (`current_enemies`) because there can be several at once, this project's player is a single character, so it's enough to declare a single variable of type `Player`, called `player_character`, instead of an array. If a project needed more than one playable character at the same time—a two-player co-op mode, for

example—the same approach used for the enemies, an array of structs, would be the natural way to go.

4.2. Adapting player.c: create_player() and manage_player()

With the struct already declared, player.c now assigns values to the player_character instance: the fix32 fields receive the initial position values that used to be loose variables, and from them the s16 fields are calculated with F32_toInt(), the same function already used in enemy.c and in camera.c. The call to SPR_addSprite(), which only accepts 16-bit integers, uses those just-calculated values:

```
// src/player.c
#include "player.h"
#include "camera.h"

Player player_character;

void create_player(void) {
    player_character.x_f32 = FIX32(40);
    player_character.y_f32 = FIX32(160);
    player_character.x_s16 = F32_toInt(player_character.x_f32);
    player_character.y_s16 = F32_toInt(player_character.y_f32);

    player_character.sprite = SPR_addSprite(&player_sprite_def,
        player_character.x_s16, player_character.y_s16,
        TILE_ATTR(PAL1, FALSE, FALSE, FALSE));
}

void manage_player(void) {
    // Subtracts the camera position, just like set_enemy_position()
    // does for each enemy (Chapter 50)
    player_character.x_s16 = F32_toInt(player_character.x_f32) - new_camera_x;
    player_character.y_s16 = F32_toInt(player_character.y_f32) - new_camera_y;

    SPR_setPosition(player_character.sprite, player_character.x_s16,
        player_character.y_s16);
}
```

manage_player() is a new function, declared in player.h alongside create_player(). For now it's limited to recalculating the on-screen position of the player's sprite and calling SPR_setPosition(), exactly what used to be a loose line inside camera(); that line is removed from camera.c, which from now on only handles calculating new_camera_x/new_camera_y and repositioning the enemies with set_enemy_position():

```
// src/camera.c (updated excerpt)
void camera(void) {
    // Calculating new_camera_x / new_camera_y from the player's
    // position and the map's boundaries (Chapter 24)
    // ...

    set_enemy_position(boss_sprite, boss_x, boss_y, BOSS_WIDTH, BOSS_HEIGHT);
    set_enemy_position(fat_man_sprite, fat_man_x, fat_man_y, FAT_MAN_WIDTH,
        FAT_MAN_HEIGHT);

    current_camera_x = new_camera_x;
```



```

    current_camera_y = new_camera_y;
}

```

In future chapters, `manage_player()` will grow to handle the player's physics, level collision, and, eventually, its own mini state machine (standing, walking, jumping). Storing the player's position in two formats is convenient, but it demands discipline: any code that modifies its position, whether jump physics or level collision, has to update `x_f32/y_f32` first and let `manage_player()` recalculate `x_s16/y_s16` from them, or risk having the on-screen version fall out of sync with the actual version used for calculations. Centralizing that recalculation in a single place, rather than scattering it loosely across several functions, is precisely the reason `manage_player()` exists.

5. Wiring everything together in `main.c` and testing the result

With the three pieces now connected—the `GameStates` enum, the `current_game_state` variable checked in both `joyEvent()` and `handle_input()`, and the `Player` struct already integrated into `player.c`—`main.c` registers `joyEvent()` once with `J0Y_setEventHandler()` (as has already been done since Chapter 32) and calls `handle_input()`, `camera()`, and `manage_player()` on every pass of the main loop:

```

// src/main.c
#include <genesis.h>
#include "player.h"
#include "enemy.h"
#include "camera.h"
#include "controls.h"
#include "levels.h"

int main(bool hard_reset) {
    VDP_setScreenWidth320();

    SPR_init();
    J0Y_setEventHandler(joyEvent);

    create_player();
    create_enemies();

    while (TRUE) {
        handle_input();
        camera();
        manage_player();

        SPR_update();
        SYS_doVBlankProcess();
    }

    return 0;
}

```

When you compile and test the ROM, the player moves with the D-pad completely normally. As soon as the Start button is pressed, the state changes from `RUNNING_LEVEL` to `LOADING` and,

thanks to the check added in `handle_input()`, the D-pad stops having any effect on the player: the state machine is working, even though there still isn't any code that truly reacts to the `LOADING` state beyond blocking movement.



Figure 151: Comparison of the player before and after pressing Start

The player moves with the D-pad while the state is `RUNNING_LEVEL`; pressing Start changes the state to `LOADING` and the controls stop responding.

Review exercises

1. Explain, in your own words, why it isn't enough for the D-pad to always move the player unconditionally, once the game starts having more than one possible screen or situation.
2. Why is `current_game_state` declared as `u8` and not, say, a plain `int`? Relate your answer to what was covered in Chapter 6.
3. Describe the difference between `joyEvent()` and `handle_input()`, and explain why the state change on pressing Start is written inside the former and not the latter.
4. A classmate adds `current_game_state` to `controls.c` and gets an "unrecognized identifier" error, despite having typed the name correctly. What's the most likely cause, and how is it fixed?
5. Explain why `struct Player` stores the player's coordinates in both `fix32` and `s16`, rather than using a single data type for both purposes, and what role `manage_player()` plays in keeping those two versions in sync.
6. Describe, in your own words, what a finite state machine is and why it's a pattern likely to come up several more times throughout this manual.

Chapter 58 — Loading Different Enemies per Level

Everything built so far around enemies —the `Enemy` struct, the permanent type catalog `enemyTypesList`, and the AI selected through function pointers— coexists with a limitation that becomes obvious as soon as you think about a complete game: the `current_enemies` array represents a single enemy configuration, the one currently on screen, and until now that configuration could only be changed by editing the code by hand and recompiling. A real game, however, has dozens of screens or sections, each with its own combination of enemies, and recompiling the entire ROM every time the player advances to a new level is not a reasonable option.

This chapter separates two things that until now lived mixed together in a single array: on one hand, the complete catalog of which enemies correspond to each level or section of the game —data that never changes during a playthrough and can therefore live permanently in the cartridge’s ROM—, and on the other, the enemy configuration for the section currently being played, which continues to live in `current_enemies`, in RAM, because that is the only place where the data can be modified while the game is running. The bridge between the two is a new function, `copyEnemyArray()`, which copies the data for the corresponding section from the catalog into `current_enemies`.

Prerequisites

Before continuing, it helps to have already covered:

- The `Enemy` struct and the `current_enemies` array, as they stood at the end of chapter 53 (“C Structs Applied to Game Entities”), and the `EnemyStates` enum together with the `MAX_NUMBER_CURRENT_ENEMIES` constant from chapter 54 (“C Enumerations (enums)”).
- The `enemyTypesList` catalog, the `EnemyType` struct, and the practice of using `const` to keep permanent data in ROM, from chapter 55 (“Permanent Enemy Types”).
- The `enemyAI` field within `EnemyType` and the functions `boomerangAI`, `basicNinjaAI`, `stealthAI`, and `sniperAI` from chapter 56 (“Function Pointers for Enemy AI”).
- Familiarity with multidimensional arrays in C (an array of arrays), although having used them before is not strictly required: this chapter introduces them in practice.
- The `GameStates` enum and the notion of game state from chapter 57 (“Game States”), since `current_game_section` relies on the same idea of a variable that the rest of the code consults to know what situation the game is in.

1. The problem: a single array isn't enough for multiple levels

As things stand today, `main.c` calls `createEnemies()`, and that function takes the data from `current_enemies` and uses it to display the enemies on screen. This works perfectly as long as the game has a single screen, because `current_enemies` only needs to represent that one configuration. But as soon as the game has, say, four levels with several sections each, a different enemy configuration is needed for each of those sections, and `current_enemies`—designed to represent a single combination at a time— isn't enough to store all of them simultaneously.

The solution isn't to make `current_enemies` bigger, but to separate two distinct questions that until now were answered by the same array. The first is which enemies exist in each section of the game, a fixed answer known ahead of time. The second is which enemies are active right now, in the section currently being played, something that changes constantly as the AI acts. The first question is solved with a new catalog; the second remains `current_enemies`.

2. A complete catalog: the `all_enemies` array

2.1. Declaring an array of arrays

The catalog is declared in `enemies.h` by copying the syntax already used for `current_enemies`, with three changes. First, the name becomes `all_enemies`, to reflect that it contains the enemies for the whole game and not just the current screen. Second, a second pair of brackets is added: `current_enemies` is an array of ten `Enemy` structs (one for each possible enemy on screen), whereas `all_enemies` is an array of arrays, with one row per section of the game and, within each row, the same ten slots that `current_enemies` already has.

The number of sections is, in itself, a number that is best not left loose in the code, following the same approach already used for `MAX_NUMBER_CURRENT_ENEMIES` in chapter 54. That's why a new constant, `NUMBER_OF_SECTIONS`, is defined. This manual illustrates it with four sections—as many as the reference project's levels— but nothing prevents expanding it later if the game grows: it's enough to change a single value.

The third change is adding the word `const` before `struct Enemy`. Unlike `current_enemies`, whose values change constantly as the AI acts on each enemy, the `all_enemies` catalog represents the initial configuration of each section, data that is never modified during a playthrough. Marking it as `const`—the same approach already applied to `enemyTypesList`— ensures that the entire array is stored in the cartridge's ROM and doesn't take up space in the mere 64 kilobytes of available RAM.

```
// enemies.h
#ifndef _ENEMIES_H_
#define _ENEMIES_H_

#include <genesis.h>
#include "levels.h" // required for current_game_section, used by
    ↪ copyEnemyArray()

#define MAX_NUMBER_CURRENT_ENEMIES 10
#define NUMBER_OF_SECTIONS 4

enum EnemyStates
{
    ENEMY_INACTIVE,
```

```

    ENEMY_IDLE,
    ENEMY_ATTACK,
    ENEMY_HURT,
    ENEMY_DYING
};

enum EnemyNames
{
    BOOMERANG_ENEMY,
    BASIC_NINJA_ENEMY,
    STEALTH_ENEMY,
    SNIPER_ENEMY
};

struct EnemyType
{
    const SpriteDefinition *graphic;
    u8 max_health;
    u8 palette_used;
    void (*enemyAI)(u8 enemy_number);
};

struct Enemy
{
    u8 status;
    fix32 x;
    fix32 y;
    fix32 vel_x;
    Sprite *sprite;
    s8 hitpoints;
    u8 enemy_type;
};

extern const struct EnemyType enemyTypesList[MAX_NUMBER_CURRENT_ENEMIES];
extern struct Enemy current_enemies[MAX_NUMBER_CURRENT_ENEMIES];
extern const struct Enemy
    ↪ all_enemies[NUMBER_OF_SECTIONS][MAX_NUMBER_CURRENT_ENEMIES];

extern void createEnemies(void);
extern void manageEnemies(void);
extern void copyEnemyArray(void);

extern void boomerangAI(u8 enemy_number);
extern void basicNinjaAI(u8 enemy_number);
extern void stealthAI(u8 enemy_number);
extern void sniperAI(u8 enemy_number);

#endif

```

2.2. Filling in the data, section by section

In `enemies.c` the actual contents of `all_enemies` are written out. Since the syntax for each individual section is identical to what was already used for `current_enemies`, the fastest way

to fill in the catalog is to take the already-familiar block of ten enemies and repeat it as many times as sections need to be defined, separating each block with a comma and closing the entire array with a final semicolon.

To be able to tell at a glance when the game moves from one section to another during testing, it also helps to vary the contents of each block: the first section is left with a mix of enemy types, the second is filled entirely with boomerang enemies, the third entirely with stealth enemies, and the fourth with snipers. In a finished level, each section would also have its own number of active enemies and its own starting coordinates; for now it's enough to vary the type to verify that the copying mechanism works.

```
// enemies.c
#include "enemies.h"

const struct EnemyType enemyTypesList[MAX_NUMBER_CURRENT_ENEMIES] =
{
    { &boomerang_sprite, 10, PAL0, boomerangAI }, // BOOMERANG_ENEMY
    { &basic_ninja_sprite, 10, PAL0, basicNinjaAI }, // BASIC_NINJA_ENEMY
    { &stealth_sprite, 10, PAL0, stealthAI }, // STEALTH_ENEMY
    { &sniper_sprite, 10, PAL0, sniperAI }, // SNIPER_ENEMY
    { &boomerang_sprite, 10, PAL0, boomerangAI }, // filler, not yet used
    { &boomerang_sprite, 10, PAL0, boomerangAI }, // filler, not yet used
    { &boomerang_sprite, 10, PAL0, boomerangAI }, // filler, not yet used
    { &boomerang_sprite, 10, PAL0, boomerangAI }, // filler, not yet used
    { &boomerang_sprite, 10, PAL0, boomerangAI }, // filler, not yet used
    { &boomerang_sprite, 10, PAL0, boomerangAI }, // filler, not yet used
};

struct Enemy current_enemies[MAX_NUMBER_CURRENT_ENEMIES];

const struct Enemy all_enemies[NUMBER_OF_SECTIONS][MAX_NUMBER_CURRENT_ENEMIES] =
{
    // Section 0: mix of types, to test variety within a single screen.
    {
        { ENEMY_IDLE, FIX32(20), FIX32(150), FIX32(1.5), NULL, 10,
          ↪ BOOMERANG_ENEMY },
        { ENEMY_IDLE, FIX32(44), FIX32(150), FIX32(1.5), NULL, 10,
          ↪ BASIC_NINJA_ENEMY },
        { ENEMY_IDLE, FIX32(68), FIX32(150), FIX32(1.5), NULL, 10,
          ↪ STEALTH_ENEMY },
        { ENEMY_IDLE, FIX32(92), FIX32(150), FIX32(1.5), NULL, 10,
          ↪ SNIPER_ENEMY },
        { ENEMY_IDLE, FIX32(116), FIX32(150), FIX32(1.5), NULL, 10,
          ↪ BOOMERANG_ENEMY },
        { ENEMY_IDLE, FIX32(140), FIX32(150), FIX32(1.5), NULL, 10,
          ↪ BASIC_NINJA_ENEMY },
        { ENEMY_INACTIVE, FIX32(0), FIX32(0), FIX32(0), NULL, 0,
          ↪ BOOMERANG_ENEMY },
        { ENEMY_INACTIVE, FIX32(0), FIX32(0), FIX32(0), NULL, 0,
          ↪ BOOMERANG_ENEMY },
        { ENEMY_INACTIVE, FIX32(0), FIX32(0), FIX32(0), NULL, 0,
          ↪ BOOMERANG_ENEMY },
        { ENEMY_INACTIVE, FIX32(0), FIX32(0), FIX32(0), NULL, 0,
          ↪ BOOMERANG_ENEMY }
    },

```

```

// Section 1: entirely boomerang enemies, meant to make the section change
↪ obvious.
{
    { ENEMY_IDLE, FIX32(20),  FIX32(150), FIX32(1.5), NULL, 10, BOOMERANG_ENEMY
    ↪ },
    { ENEMY_IDLE, FIX32(44),  FIX32(150), FIX32(1.5), NULL, 10, BOOMERANG_ENEMY
    ↪ },
    { ENEMY_IDLE, FIX32(68),  FIX32(150), FIX32(1.5), NULL, 10, BOOMERANG_ENEMY
    ↪ },
    { ENEMY_IDLE, FIX32(92),  FIX32(150), FIX32(1.5), NULL, 10, BOOMERANG_ENEMY
    ↪ },
    { ENEMY_IDLE, FIX32(116), FIX32(150), FIX32(1.5), NULL, 10, BOOMERANG_ENEMY
    ↪ },
    { ENEMY_IDLE, FIX32(140), FIX32(150), FIX32(1.5), NULL, 10, BOOMERANG_ENEMY
    ↪ },
    { ENEMY_IDLE, FIX32(164), FIX32(150), FIX32(1.5), NULL, 10, BOOMERANG_ENEMY
    ↪ },
    { ENEMY_IDLE, FIX32(188), FIX32(150), FIX32(1.5), NULL, 10, BOOMERANG_ENEMY
    ↪ },
    { ENEMY_IDLE, FIX32(212), FIX32(150), FIX32(1.5), NULL, 10, BOOMERANG_ENEMY
    ↪ },
    { ENEMY_IDLE, FIX32(236), FIX32(150), FIX32(1.5), NULL, 10, BOOMERANG_ENEMY
    ↪ }
},
// Section 2: entirely stealth enemies.
{
    { ENEMY_IDLE, FIX32(20),  FIX32(150), FIX32(1.5), NULL, 10, STEALTH_ENEMY },
    { ENEMY_IDLE, FIX32(44),  FIX32(150), FIX32(1.5), NULL, 10, STEALTH_ENEMY },
    { ENEMY_IDLE, FIX32(68),  FIX32(150), FIX32(1.5), NULL, 10, STEALTH_ENEMY },
    { ENEMY_IDLE, FIX32(92),  FIX32(150), FIX32(1.5), NULL, 10, STEALTH_ENEMY },
    { ENEMY_IDLE, FIX32(116), FIX32(150), FIX32(1.5), NULL, 10, STEALTH_ENEMY },
    { ENEMY_IDLE, FIX32(140), FIX32(150), FIX32(1.5), NULL, 10, STEALTH_ENEMY },
    { ENEMY_IDLE, FIX32(164), FIX32(150), FIX32(1.5), NULL, 10, STEALTH_ENEMY },
    { ENEMY_IDLE, FIX32(188), FIX32(150), FIX32(1.5), NULL, 10, STEALTH_ENEMY },
    { ENEMY_IDLE, FIX32(212), FIX32(150), FIX32(1.5), NULL, 10, STEALTH_ENEMY },
    { ENEMY_IDLE, FIX32(236), FIX32(150), FIX32(1.5), NULL, 10, STEALTH_ENEMY }
},
// Section 3: entirely snipers.
{
    { ENEMY_IDLE, FIX32(20),  FIX32(150), FIX32(1.5), NULL, 10, SNIPER_ENEMY },
    { ENEMY_IDLE, FIX32(44),  FIX32(150), FIX32(1.5), NULL, 10, SNIPER_ENEMY },
    { ENEMY_IDLE, FIX32(68),  FIX32(150), FIX32(1.5), NULL, 10, SNIPER_ENEMY },
    { ENEMY_IDLE, FIX32(92),  FIX32(150), FIX32(1.5), NULL, 10, SNIPER_ENEMY },
    { ENEMY_IDLE, FIX32(116), FIX32(150), FIX32(1.5), NULL, 10, SNIPER_ENEMY },
    { ENEMY_IDLE, FIX32(140), FIX32(150), FIX32(1.5), NULL, 10, SNIPER_ENEMY },
    { ENEMY_IDLE, FIX32(164), FIX32(150), FIX32(1.5), NULL, 10, SNIPER_ENEMY },
    { ENEMY_IDLE, FIX32(188), FIX32(150), FIX32(1.5), NULL, 10, SNIPER_ENEMY },
    { ENEMY_IDLE, FIX32(212), FIX32(150), FIX32(1.5), NULL, 10, SNIPER_ENEMY },
    { ENEMY_IDLE, FIX32(236), FIX32(150), FIX32(1.5), NULL, 10, SNIPER_ENEMY }
}
};

```

When filling in an array like this, with several structurally identical blocks, it's

easy to lose count and write one block too many or too few. The compiler catches the mismatch and warns with a message along the lines of “too many initializer values” as soon as the number of blocks doesn’t match the array’s declared size (NUMBER_OF_SECTIONS rows of MAX_NUMBER_CURRENT_ENEMIES elements each). The error is fixed by deleting or adding the missing or extra block; even so, it’s worth counting the blocks carefully before compiling, rather than relying on the error message to catch the slip.

3. The `copyEnemyArray()` function: moving data from ROM to RAM

Having the complete catalog in `all_enemies` is useless if `createEnemies()` and `manageEnemies()` keep reading exclusively from `current_enemies`, which is where the data actually shown and modified on screen truly lives. What’s needed, then, is a function that, given the current section number, copies the corresponding block from `all_enemies` into `current_enemies`. That function is `copyEnemyArray()`, already declared as a prototype in the `enemies.h` listing in the previous section, alongside `createEnemies()` and `manageEnemies()`.

The body of `copyEnemyArray()` walks through the ten slots of `current_enemies` with a `for` loop and, on each pass, replaces the contents with those of the corresponding enemy within the indicated section of `all_enemies`. The expression `all_enemies[current_game_section][i]` uses two indices: the first selects the game section, and the second, the same `i` from the loop, selects which of the ten enemies in that section is being copied.

```
void copyEnemyArray(void)
{
    for (u8 i = 0; i < MAX_NUMBER_CURRENT_ENEMIES; i++)
    {
        // Copy entry i of the current section into current_enemies.
        current_enemies[i] = all_enemies[current_game_section][i];
    }
}
```

The C language offers a generic function for copying blocks of memory, `memcpy()`, which in principle could replace this `for` loop with a single line, something like `memcpy(current_enemies, all_enemies[current_game_section], sizeof(current_enemies))`. Both approaches are valid in C; this manual prefers the explicit loop because it makes visible, element by element, what is being copied and to where, without depending on the source and destination sizes matching exactly in bytes. If the reader tries `memcpy()` and finds it more convenient, there’s no reason not to use it.

4. Linking with the current level: `current_game_section`

One piece is still missing: where does the section number that `copyEnemyArray()` uses as an index come from? For that, a new variable called `current_game_section` is added in `levels.h`, in the same style as `current_game_state` from chapter 57: an unsigned 8-bit integer that records which section of the game the player is currently in. With 8 bits there’s room for 256 different sections, more than enough even for a game with a great many levels; if some project needed more, a `u16` would offer up to 65 thousand sections at no significant cost.

```
// levels.h
#ifndef _LEVELS_H_
```

```

#define _LEVELS_H_

#include <genesis.h>

enum GameStates
{
    LOADING,
    RUNNING_LEVEL
};

extern u8 current_game_state;
extern u8 current_game_section;

#endif

```

```

// levels.c
#include "levels.h"

u8 current_game_state = RUNNING_LEVEL;
u8 current_game_section = 0; // the game always starts in the first section

```

`current_game_section` plays, for the current level, a role similar to the one `current_game_state` plays for the overall flow of the game: both are variables that a set of functions constantly consults to decide what to do, and both live in `levels.h/levels.c` because, conceptually, they describe what part of the game the player is currently in. As the game grows, these two variables, together with other similar ones, will likely end up grouped into a single struct that fully describes the state of the current playthrough.

Note that `enemies.h` already includes `levels.h` in the listing in section 2.1: without that `#include`, the compiler would flag `current_game_section` as an unrecognized identifier within `enemies.c`, since that variable is declared in `levels.h` and not in any file `enemies.c` previously knew about. This kind of error—an identifier that exists, but whose header wasn’t included where it’s needed—doesn’t indicate any conceptual problem with the design; it’s simply a consequence of how C organizes the knowledge of symbols across files, and it’s worth getting used to recognizing it at a glance as the project grows.

5. Where to call `copyEnemyArray()`

With the function already written, it remains to decide where to call it from. One option is to call it in `main.c`, right before `createEnemies()`, so that the data is copied first and then displayed on screen. The option adopted here is to call it as the very first line inside `createEnemies()` itself, so that any code that in the future calls `createEnemies()` automatically triggers the data copy without having to remember to call `copyEnemyArray()` separately.

```

void createEnemies(void)
{
    // Copy the current section's catalog into current_enemies
    // before creating any sprite on screen.
    copyEnemyArray();

    for (u8 i = 0; i < MAX_NUMBER_CURRENT_ENEMIES; i++)
    {
        if (current_enemies[i].status != ENEMY_INACTIVE)

```

```

    {
        current_enemies[i].sprite = SPR_addSprite(
            enemyTypesList[current_enemies[i].enemy_type].graphic,
            F32_toInt(current_enemies[i].x),
            F32_toInt(current_enemies[i].y),

↪ TILE_ATTR(enemyTypesList[current_enemies[i].enemy_type].palette_used, TRUE,
↪ FALSE, FALSE));

        current_enemies[i].hitpoints =
↪ enemyTypesList[current_enemies[i].enemy_type].max_health;
    }
}

void manageEnemies(void)
{
    for (u8 i = 0; i < MAX_NUMBER_CURRENT_ENEMIES; i++)
    {
        if (current_enemies[i].status != ENEMY_INACTIVE)
            enemyTypesList[current_enemies[i].enemy_type].enemyAI(i);
    }
}

```

This decision does, however, leave a small design detail unresolved: both `copyEnemyArray()` and `createEnemies()` walk through the ten slots of `current_enemies` with an identical for loop, which means that, in practice, the same range is traversed twice in a row. It would be possible to merge both loops into a single one and eliminate `copyEnemyArray()` as a standalone function, gaining a bit of efficiency. The criterion adopted here prioritizes clarity: keeping `copyEnemyArray()` as a separate function, with a name that describes exactly what it does, turns out to be more readable than a single loop that does two things at once, and the cost of traversing a ten-element array twice per level is, in practice, negligible compared to the time available per frame.

This kind of decision —code clarity versus a marginal savings in CPU cycles— will come up again throughout this manual. As a general rule, it's best to prioritize readability while the performance cost is negligible, and reserve aggressive optimization for code that truly runs with a frequency or data volume that justifies it, such as routines that run once per frame over many elements at once.

6. Testing the result

Compiling and loading the ROM with `current_game_section` at 0, the result is identical to what was already there before this chapter, because the first section of `all_enemies` was deliberately filled with the same mix of enemies that `current_enemies` already had. Changing `current_game_section` to 1 and reloading the ROM shows, instead, the section made up entirely of boomerang enemies; repeating the test with `current_game_section` at 2 brings up the stealth enemy section, and with 3, the sniper section.

With `current_game_section` at 1, the ROM loads the section made up entirely of boomerang enemies, confirming that `copyEnemyArray()` copies the correct data from `all_enemies`.

With this settled, the final piece is still missing for the player to experience a real level change within the running game: for now, changing sections still requires editing `cur-`

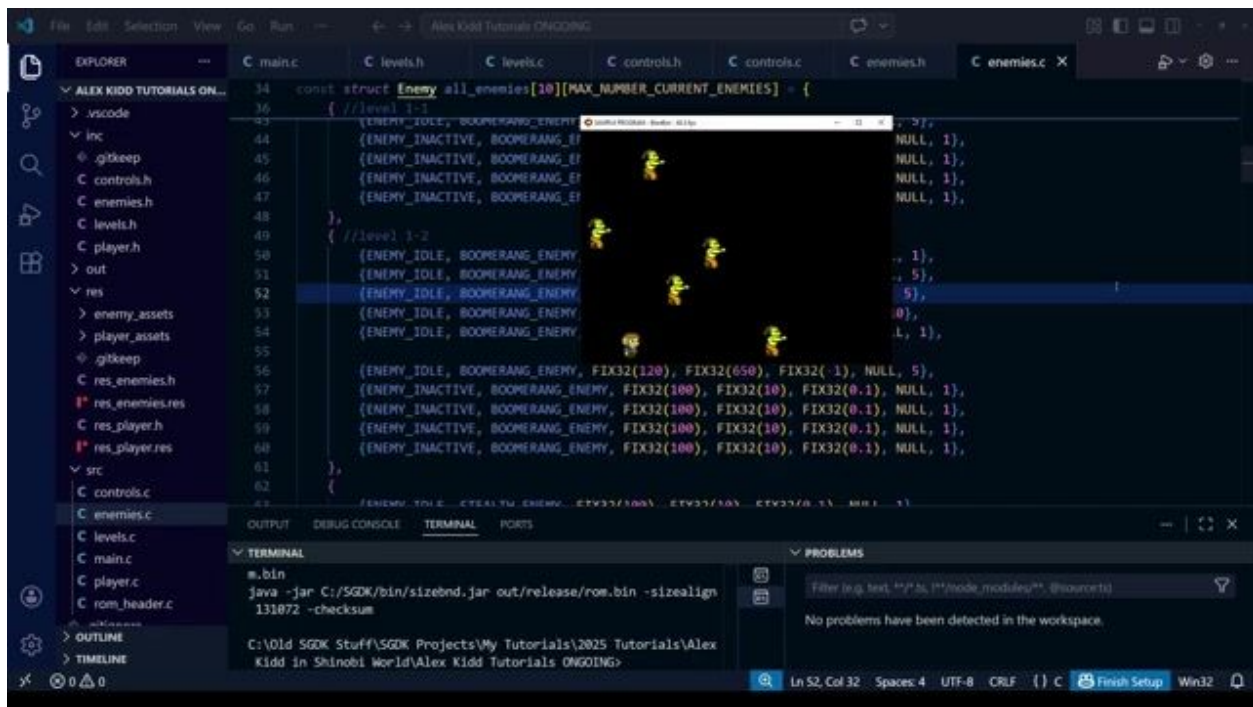


Figure 152: Section 1 loaded with boomerang enemies

rent_game_section by hand and recompiling. Chapter 59 (“Preparing Level Graphics”) takes a small detour into art and background preparation, before chapter 60 (“In-Game Level Transition”) picks the code back up to connect current_game_section with the Start button and the state machine from chapter 57, finally achieving level transitions that happen within the running game, without recompiling.

Review exercises

1. Explain, in your own words, why current_enemies alone isn’t enough to represent the enemies of a game with several sections or levels.
2. Describe what each of the two indices in the expression all_enemies[current_game_section][i] represents.
3. Why is all_enemies declared with the const keyword, while current_enemies is not?
4. Explain the relationship between copyEnemyArray() and createEnemies(): why is the former called from within the latter, and what would happen if it were never called?
5. The chapter describes a design alternative that would merge copyEnemyArray() and createEnemies() into a single loop. Why is it decided not to do so, despite the performance savings it would offer?

Chapter 59 — Preparing Level Graphics

Where the art for each level of a game with several distinct stages comes from, and how to organize tilesets, palettes, and maps by level within a single SGDK project

The previous chapters of this part left the project with the player and the enemies organized into separate files (chapter 51), and with the logic for each enemy to position and hide itself correctly relative to the camera (chapter 50). The next step in building this beat 'em up is to let the player move on from one level to the next, each with its own background, its own foreground, and, likely, its own cast of enemies. Before writing that transition logic, it's worth solving a more basic problem, one that none of the previous chapters had to face yet: where the art for each of those levels comes from, and how it's organized in the project so that loading level 2 doesn't depend on still having level 1's tiles in memory.

This chapter deals with those two questions. The first part is more a matter of artistic judgment and workflow than of code itself; the second picks up directly from the resource handling already covered with the rescomp compiler (part VI) and the VRAM limits (part VIII), applying them to the specific case of a game with multiple backgrounds that are shown, never two at once, over the course of the game.

Prerequisites

Before continuing, it's worth having fresh in mind:

- The TILESET, PALETTE, and MAP resources, and the `VDP_loadTileSet()`, `PAL_setPalette()`, `MAP_create()`, and `MAP_scrollTo()` functions, introduced in chapter 11.
- The Tiled workflow and the TILEMAP resource from chapter 12, as a valid alternative to the MAP resource when a level is designed by painting tiles rather than starting from a single, already-composed image.
- The split of the project into several .res files by topic, seen in chapter 51, which this chapter extends from “one resource file per character” to “one resource file per level.”
- The palette and VRAM constraints covered in the chapters on multiple background palettes (14), managing VRAM limits (45), SGDK 2.x's new VRAM configuration (46), and tile reduction (47).
- Basic familiarity with a tile-based graphics editor, such as the Aseprite already used in chapter 8.

1. Where a level's art comes from

Not every level of a game needs to be drawn entirely from scratch. Reusing and adapting art from an existing game — one's own, or, in hobby projects made for learning purposes, some

well-known classic — is a common production strategy in homebrew development: it lets you focus effort on improving and expanding art that’s already proven, instead of generating, from a blank page, the sheer volume of tiles a full level needs. An illustrative case, common in Mega Drive projects made by hobbyists, is starting from the graphics of an 8-bit game on an earlier console and adapting them to the Mega Drive’s capabilities while keeping the same drawing style and the same tile and sprite size, rather than redesigning the art at a higher level of detail.

Whatever the source of the art, it’s worth settling two criteria before you start preparing a level’s tiles:

- **The sprite size for the player and the enemies doesn’t change between levels.** If a level’s background art is drawn at a different level of detail than the rest, the player and enemies will look disproportionate as soon as they’re moved over that background. Any reference art being adapted — whether from an earlier game of your own or another source — has to be respected at the same scale the project’s sprites already use.
- **Tiles are still 8×8 pixels**, the minimum unit the VDP works with, as explained in chapter 2. This doesn’t change from one level to the next: it’s a hardware constraint, not a design decision.

When the reference art comes from an existing game that isn’t available as an editable image file, a practical technique for gathering reference material is to capture the game running inside an emulator that supports frame advance, rather than letting it run at normal speed. Freezing the image frame by frame avoids capturing a screen in the middle of a scroll movement or with an animated element in an unrepresentative in-between position, and lets you go through a level screen by screen to later combine those captures in a graphics editor and reconstruct the full layout as a single continuous image. Practically any emulator with an equivalent frame-advance feature works for this purpose: there’s no technical dependency on any particular emulator, beyond the convenience of using one you already know.

Once that reference image has been assembled, the editing work goes through a conventional tile-based graphics editor (Aseprite, already used in chapter 8): cleaning up the result, snapping it to an 8×8 grid, and starting to apply the color improvements described in the next section.

2. Making the most of the extra color: contrast and plane separation, now multiplied per level

The Mega Drive offers four hardware palettes of 15 colors each plus a shared background color, for a total of 61 simultaneous on-screen colors compared to the 31 of a typical 8-bit console. That difference isn’t put to use by having an individual tile or sprite draw from more than one palette — each one is still limited to 15 colors plus the background color, as explained in chapter 14 — but by spreading different palettes across different elements of the scene so they contrast clearly.

In this project, the four hardware palettes already have a fixed assignment inherited from previous chapters: PAL1 for the player and PAL3 for the enemies (chapters 50 and 51). That leaves PAL0 and PAL2 available for level backgrounds, and it’s worth keeping that assignment constant across the thirteen chapters of this part: PAL0 for the foreground (BG_A), where the level’s collision lives, and PAL2 for the parallax background (BG_B), which is purely decorative. Each new level replaces the contents of those two palettes, but never changes which hardware palette corresponds to which plane: that simplifies the loading code, because there’s no need to decide again, level by level, which palette number to use.

Within those two per-level palettes, the same separation criterion already practiced in chapter

14 now also applies between tiles that coexist on the same plane: it's worth reserving clearly distinct color ranges for walkable terrain and for the vegetation or decorative elements surrounding it, so that a player can tell at a glance where they can step and where they can't, instead of both kinds of tile sharing a similar color range and blending together. That contrast matters even more in a game with continuous horizontal scrolling like this one, where the player doesn't have time to stop and carefully examine the scenery.

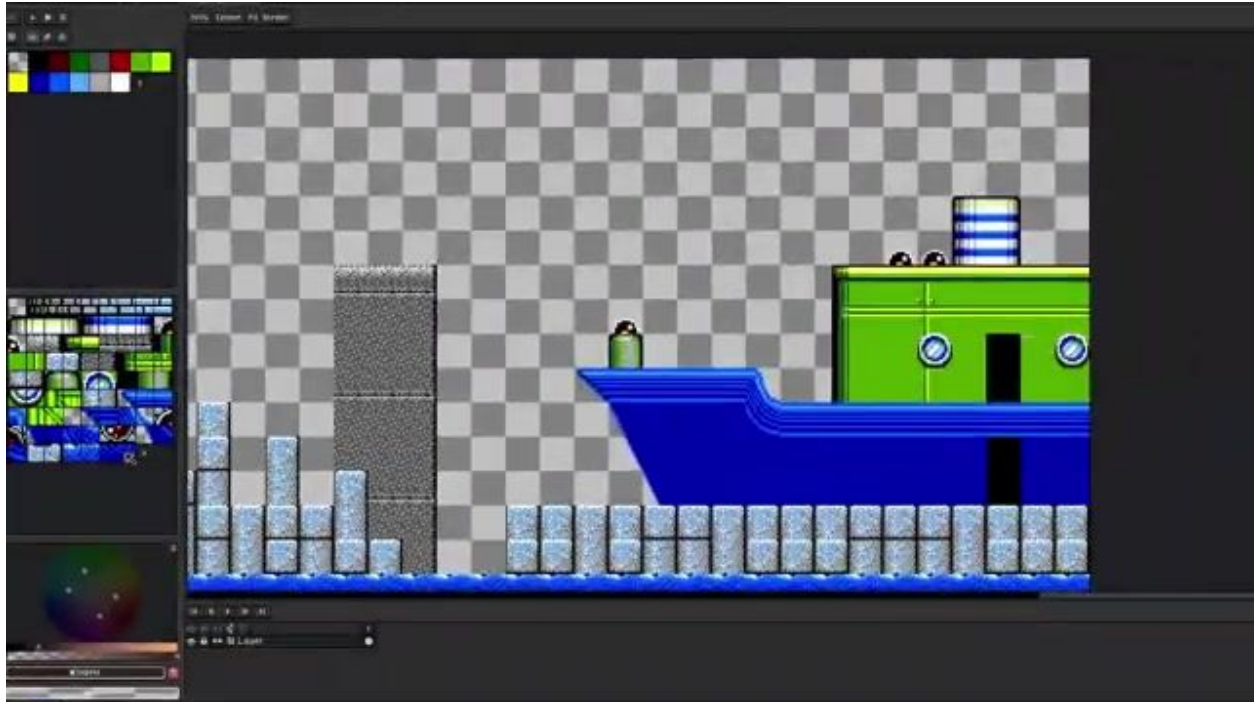


Figure 153: Tilesheet of a level with palettes differentiated by element

Tilesheet of a harbor area: structures, a boat, and water, each with its own palette so they stand out clearly while moving through the level.

Splitting into background and foreground planes is also a requirement for applying parallax scrolling (chapter 9): if a level's reference art comes from a game that showed everything on a single layer, you have to explicitly decide, while preparing the tiles, which elements move to the distant background and which stay in the walkable foreground, before declaring any resource.

3. Organizing resources by level

With the artistic criteria already set, the next step is to bring each level into the project as its own set of resources, following the same topic-based organization pattern that chapter 51 applied to the characters: one .res file per level, with its own image subfolders inside res. A level with a foreground and a parallax background needs, at minimum, a TILESHEET, a MAP (or a TILESHEET+PALETTE+TILEMAP if it comes from Tiled), and a PALETTE for each of the two planes:

```
// res/res_level1.res

// Level 1 foreground (BG_A): walkable terrain
TILESET level1_fg_tilesset "level1_assets/foreground.png" BEST ALL
MAP      level1_fg_map      "level1_assets/foreground.png" level1_fg_tilesset BEST 0
```



```

PALETTE level1_fg_palette "level1_assets/foreground.png"

// Level 1 parallax background (BG_B): distant scenery
TILESET level1_bg_tileset "level1_assets/background.png" BEST ALL
MAP      level1_bg_map     "level1_assets/background.png" level1_bg_tileset BEST 0
PALETTE level1_bg_palette "level1_assets/background.png"

// res/res_level2.res

TILESET level2_fg_tileset "level2_assets/foreground.png" BEST ALL
MAP      level2_fg_map     "level2_assets/foreground.png" level2_fg_tileset BEST 0
PALETTE level2_fg_palette "level2_assets/foreground.png"

TILESET level2_bg_tileset "level2_assets/background.png" BEST ALL
MAP      level2_bg_map     "level2_assets/background.png" level2_bg_tileset BEST 0
PALETTE level2_bg_palette "level2_assets/background.png"

```

Each new .res file generates, when compiled, its own header (res_level1.h, res_level2.h), through exactly the same mechanism already seen in chapter 51. If a particular level was designed by painting tiles in Tiled instead of starting from a single composed image, that level's MAP resource is replaced by the TILESET+PALETTE+TILEMAP combination from chapter 12; the rest of this chapter works the same regardless of the specific origin of each map, because MAP_create() and MAP_scrollTo() don't distinguish one origin from the other.

4. A catalog of graphics by level

With the resources for two levels already declared, what's needed now is a way to go through them from code without writing a completely different function for each new level added. The solution follows the same spirit as a configuration catalog indexed by number, already practiced in previous chapters for other data that varies from section to section: a structure that groups together everything needed to display a level, and an array of those structures indexed by level number.

```

// inc/level_graphics.h
#ifndef LEVEL_GRAPHICS_H
#define LEVEL_GRAPHICS_H

#include <genesis.h>

// Everything needed to display a full level: a tileset, a map,
// and a palette for the foreground (BG_A), and the same for the
// parallax background (BG_B).
typedef struct {
    const TileSet* foreground_tileset;
    const MapDefinition* foreground_map;
    const Palette* foreground_palette;
    const TileSet* background_tileset;
    const MapDefinition* background_map;
    const Palette* background_palette;
} LevelGraphics;

extern const LevelGraphics level_graphics_table[];
extern const u8 level_count;

```

```

extern Map* foreground_map;
extern Map* background_map;

// Loads the given level's graphics into VRAM and sets up the
// corresponding background planes, freeing the previous level's if there was one
void load_level_graphics(u8 level_index);

#endif // LEVEL_GRAPHICS_H

// src/level_graphics.c
#include "level_graphics.h"
#include "res_level1.h"
#include "res_level2.h"

const LevelGraphics level_graphics_table[] = {
    { &level1_fg_tileset, &level1_fg_map, &level1_fg_palette,
      &level1_bg_tileset, &level1_bg_map, &level1_bg_palette },
    { &level2_fg_tileset, &level2_fg_map, &level2_fg_palette,
      &level2_bg_tileset, &level2_bg_map, &level2_bg_palette },
};

const u8 level_count = sizeof(level_graphics_table) / sizeof(LevelGraphics);

Map* foreground_map;
Map* background_map;

void load_level_graphics(u8 level_index) {
    const LevelGraphics* level = &level_graphics_table[level_index];

    // If a previous level was already loaded, its Map structure in RAM
    // is freed before creating the new one; there's no need to separately
    // erase the tiles in VRAM, because the new VDP_loadTileSet() overwrites
    // them directly by loading into the same index range.
    if (foreground_map != NULL) MEM_free(foreground_map);
    if (background_map != NULL) MEM_free(background_map);

    // PAL0 and PAL2 remain reserved for level backgrounds throughout
    // the whole project; PAL1 and PAL3 remain the player's and the enemies'.
    PAL_setPalette(PAL0, level->foreground_palette->data, DMA);
    PAL_setPalette(PAL2, level->background_palette->data, DMA);

    // The background is loaded right after the foreground in VRAM,
    // using the foreground's actual tile count as the offset.
    VDP_loadTileSet(level->foreground_tileset, TILE_USER_INDEX, DMA);
    VDP_loadTileSet(level->background_tileset,
        TILE_USER_INDEX + level->foreground_tileset->numTile, DMA);

    foreground_map = MAP_create(level->foreground_map, BG_A,
        TILE_ATTR_FULL(PAL0, FALSE, FALSE, FALSE, TILE_USER_INDEX));
    background_map = MAP_create(level->background_map, BG_B,
        TILE_ATTR_FULL(PAL2, FALSE, FALSE, FALSE,
            TILE_USER_INDEX + level->foreground_tileset->numTile));
}

```

```
MAP_scrollTo(foreground_map, 0, 0);
MAP_scrollTo(background_map, 0, 0);
}
```

`load_level_graphics()` can be called both when the game starts, with `load_level_graphics(0)` for the first level, and later in the project once a level-transition function exists: it's enough to call it again with the number of the next level for the background, the foreground, and their palettes to change completely. The `if (foreground_map != NULL)` check prevents the first time the function is called from trying to free a structure that doesn't exist yet; on any later call, however, that pointer already points to the previous level's map and needs to be freed before it's replaced, so as not to leave allocated memory unused.

Note that the VRAM index where the background is loaded (`TILE_USER_INDEX + level->foreground_tileset->numTile`) is calculated from the actual tile count of *that particular* level's foreground, taken from the `numTile` field of the `TileSet` structure, rather than a fixed, hand-written number. This matters because different levels are almost certainly not going to have the same number of unique tiles in their foreground, so a fixed offset that works for level 1 could cause level 2's background to partially overlap its own foreground if the latter turned out to have more tiles.

5. The VRAM budget is no longer for a single background, it's for the whole level

The chapters on managing VRAM limits (45), SGDK 2.x's new configuration (46), and tile reduction (47) already explained how to calculate how many unique tiles fit into the space available starting from `TILE_USER_INDEX`. With a single background on screen, that calculation was done once. With a level split into a foreground and a parallax background, the number that matters is the **sum** of the unique tiles in both: `level->foreground_tileset->numTile + level->background_tileset->numTile` can't exceed the VRAM space free for user tiles, after also subtracting the tiles already used by the player's and enemies' sprites, which share the same tile memory even though they're managed with `SPR_addSprite()` instead of `VDP_loadTileSet()`.

This doesn't change any of the techniques already covered — reducing duplicate tiles with the `TILESET`'s `ALL` mode, reusing mirrored variants instead of drawing new tiles, or falling back on the manual sprite loading from chapter 48 if more space needs to be freed up — but it does change the arithmetic: in a single-level game, that budget was calculated once and stayed fixed; in a multi-level game, each level has its own combined foreground-plus-background budget, and it's worth checking it level by level as levels are added to the catalog, rather than assuming that whatever fit in the first one will automatically fit in all the rest.

Review exercises

1. Why is it worth keeping fixed, throughout the project, which hardware palette (`PAL0`, `PAL1`, `PAL2`, or `PAL3`) corresponds to the player, the enemies, the level foreground, and the parallax background, instead of deciding it again for each level?
2. Explain in your own words why the VRAM budget for a level with a foreground and a parallax background is calculated by adding up the unique tiles of both planes, rather than evaluating each one separately.
3. In `load_level_graphics()`, what problem would arise if the second `VDP_loadTileSet()` always loaded the background starting from a fixed index, instead of `TILE_USER_INDEX + level->foreground_tileset->numTile`?

4. Why is there no need to explicitly “erase” the previous level’s tiles in VRAM before loading the next level’s, and what does need to be explicitly freed with `MEM_free()`?
5. Describe what would change in this chapter’s `res_level1.res` file if level 1 had been designed in Tiled instead of starting from a single composed image, relating your answer to the `TILEMAP` resource from chapter 12.

Chapter 60 — Transitioning Between Levels in the Game

How to connect the state machine with the per-section enemy catalog to advance levels without recompiling, and what happens when an array overflows

The previous chapters in this part left ready, separately, two pieces that still didn't talk to each other: a minimal state machine, with the `current_game_state` variable and its `LOADING` and `RUNNING_LEVEL` values, and a complete catalog of enemies by section, with the `all_enemies` array, the `current_game_section` variable, and the `copyEnemyArray()` function. Up to this point, changing sections was still a manual procedure: editing the value of `current_game_section` by hand in the code, saving, and recompiling. This chapter finally connects both pieces so the player can advance to the next level by pressing the Start button while the game is running, without touching the source code again.

Along the way, the chapter shows an almost inevitable design mistake the first time a level transition is implemented: the sprites from the previous section remain on screen instead of disappearing. And, to close out, what happens if the game is forced to try to load a section that doesn't exist. Both are examples of a category of bugs that any Mega Drive programmer eventually becomes familiar with: the misuse of arrays and sprite memory.

Prerequisites

Before continuing, it helps to have:

- The `GameStates` enum and the `current_game_state` variable, from the chapter on game states.
- The `all_enemies` catalog, the `current_game_section` variable, and the `copyEnemyArray()` function, from the chapter on loading different enemies per level.
- The `createPlayer()` and `createEnemies()` functions already working, capable of placing the player and the current section's enemies on screen.
- Familiarity with `joyEvent()` to react to pressing the Start button, also covered in the chapter on game states.

The artistic preparation of each level's backgrounds, covered in the previous chapter, isn't needed yet: this chapter works exclusively on the transition logic, leaving the actual loading of background graphics for the next chapter.

1. The main loop based on the game state

The first change happens in `main.c`, where up to now the main loop always called the same functions on every pass, with no condition. From this chapter on, the body of the loop is

organized as a chain of `if/else if` statements that check the value of `current_game_state` before deciding what to do. If the state is `RUNNING_LEVEL`, the loop processes the controls, updates the player and the enemies, and refreshes the sprites, exactly as it has been doing until now. If the state is `LOADING`, on the other hand, the game needs to do something completely different: load the data for the new level.

One function is deliberately left out of this branching: `SYS_doVBlankProcess()`, the call that synchronizes screen updates with the console's vertical blank interval. This function needs to run on every pass of the loop regardless of the game's state, so it's left outside of any `if`, running unconditionally.

```
// src/main.c
#include <genesis.h>
#include "levels.h"
#include "player.h"
#include "enemies.h"

int main(bool hard_reset) {
    VDP_setScreenWidth320();
    JOY_init();
    JOY_setEventHandler(joyEvent);
    SPR_init();

    while (TRUE) {
        if (current_game_state == RUNNING_LEVEL) {
            handleInput();
            managePlayer();
            manageEnemies();
            SPR_update();
        } else if (current_game_state == LOADING) {
            levelLoader();
        }

        // Runs always, regardless of the game's state
        SYS_doVBlankProcess();
    }

    return 0;
}
```

Before writing the branch for the `LOADING` state, it's worth reviewing which existing functions should run only once when the program starts (setting the screen resolution, initializing the controller, initializing SGDK's sprite engine) and which ones should be repeated every time a new level is loaded. The former stay where they are, before the main loop; only `createPlayer()` and `createEnemies()` need to run on every level load, and it's precisely those two that get reorganized in the next section.

2. `levelLoader()`: a function that groups level loading together

2.1. The prototype and the general idea

Instead of calling `createPlayer()` and `createEnemies()` separately inside the `LOADING` branch of the main loop, it makes sense to group them inside a new function, `levelLoader()`, declared

in `levels.h`. This grouping isn't just about tidiness: as loading a level grows more complex (incorporating, in the next chapter, the loading of background graphics, and later music), `levelLoader()` will become the only place that needs to be touched to add one more step to the loading process, instead of having to remember to update the main loop every time.

```
// inc/levels.h
#ifndef LEVELS_H
#define LEVELS_H

#include <genesis.h>
#include "enemies.h"
#include "player.h"

// Possible game states
enum GameStates {
    LOADING,
    RUNNING_LEVEL
};

extern u8 current_game_state;
extern u8 current_game_section;

// Groups together all the steps needed to load the current section
// (for now, player and enemies; in the next chapter, backgrounds too)
void levelLoader(void);

#endif // LEVELS_H
```

Although the compiler doesn't flag any error if they're omitted at this point, it's worth including `enemies.h` and `player.h` inside `levels.h`, to make explicit that `levelLoader()` depends on the functions declared in those files. As the project grows and files reference each other more often, leaving these dependencies undeclared makes any future reorganization of the code much more fragile, even though it may compile fine for the moment thanks to other indirect `#includes`. The include guard already used in every header of this project ensures that these cross-inclusions between `levels.h`, `enemies.h`, and `player.h` don't end up causing any redefinition error.

This is also a good moment to set the initial values of `current_game_state` and `current_game_section` in `levels.c`: the former changes from `RUNNING_LEVEL` to `LOADING`, because as soon as the program starts, what's needed is to load a level, not run it; the latter is left at 0 so the game always starts from the first section.

2.2. The body of `levelLoader()`

The body of `levelLoader()`, in `levels.c`, for now simply calls `createPlayer()` and `createEnemies()` (the same two functions that used to be called directly from `main.c`) and, as a final step, changes `current_game_state` back to `RUNNING_LEVEL`. This last step is essential: if `levelLoader()` never updated the state, the game would keep loading the same level over and over on every pass of the main loop, never moving on to run it.

```
// src/levels.c
#include "levels.h"

u8 current_game_state = LOADING;
u8 current_game_section = 0;
```



```

void levelLoader(void) {
    createPlayer();
    createEnemies();

    // With the level now loaded, the next pass of the main loop
    // will run the RUNNING_LEVEL branch
    current_game_state = RUNNING_LEVEL;
}

```

3. Testing the initial load

With `levelLoader()` now being called from the `LOADING` branch of the main loop, it's enough to compile and test the ROM to verify the full flow: the program starts in the `LOADING` state, runs `levelLoader()` once (which places the player and the section 0 enemies on screen and changes the state to `RUNNING_LEVEL`), and from then on each subsequent pass of the loop runs the `RUNNING_LEVEL` branch, allowing the player to move around and the enemies to be observed behaving normally.

4. Changing sections by pressing Start

With level loading now automated, what remains is connecting the Start button to advancing sections. Inside `joyEvent()`, in the same condition that already changed `current_game_state` to `LOADING` when Start was pressed, one more line is added: `current_game_section++`, which increments the section number by one every time the button is pressed.

```

// src/controls.c (fragment of joyEvent(), already existing since the
// chapter on game states)
void joyEvent(u16 joy, u16 changed, u16 state) {
    if (current_game_state == RUNNING_LEVEL) {
        if (changed & state & BUTTON_START) {
            current_game_state = LOADING;
            current_game_section++;
        }
    }
}

```

The next time the main loop enters the `LOADING` branch, `levelLoader()` will call `createEnemies()` again, which in turn calls `copyEnemyArray()` as its first step; that function will read the new value of `current_game_section` and copy, from `all_enemies`, the enemies corresponding to the next section.

Compiling and testing this change, the initial result is encouraging: the first level loads normally, and pressing Start does advance to the next section, with its own enemies as defined in `all_enemies`. However, a slightly closer look reveals a problem.

5. A common bug: sprites that don't disappear between levels

Pressing Start several times in a row, the enemies of the new section appear correctly, but the player and the enemies from the previous section remain visible on screen instead of

disappearing, piling up each time a level is advanced. The cause is easy to explain: every time `levelLoader()` calls `createPlayer()` and `createEnemies()`, these functions allocate new sprites via `SPR_addSprite()`, but at no point is SGDK's sprite engine told to release the previous section's sprites before creating the new ones.

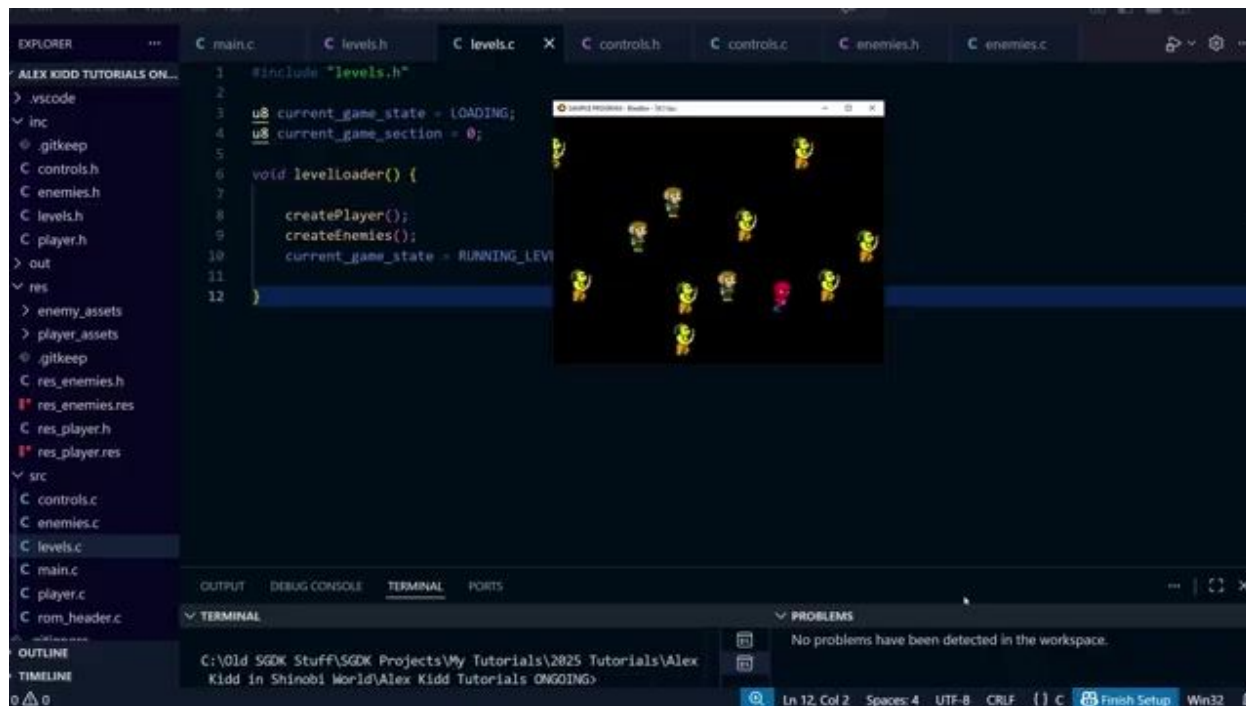


Figure 154: Sprites piling up on screen after pressing Start several times

The sprites from previous sections remain on screen together with those of the current section, instead of being replaced.

Beyond the anecdotal sight of dozens of overlapping sprites on screen before the problem is noticed or the system crashes, this bug illustrates a general principle valid in almost any sprite engine: allocating a resource (in this case, a hardware sprite) without releasing the previous one will sooner or later exhaust that resource, whether it's memory, sprite identifiers, or any other finite limit of the system.

6. The fix: `SPR_reset()`

SGDK offers a function designed exactly for this scenario: `SPR_reset()`, which releases every sprite allocated so far along with the resources associated with them. The fix consists of calling `SPR_reset()` as the first step inside `levelLoader()`, before anything else, so that every level load starts from a clean state, with no leftover sprites from the previous section:

```
// src/levels.c
#include "levels.h"

u8 current_game_state = LOADING;
u8 current_game_section = 0;

void levelLoader(void) {
    // Releases every sprite allocated so far –the player and the
```

```

// enemies from the previous section— before creating the
// ones for the new section
SPR_reset();

createPlayer();
createEnemies();

current_game_state = RUNNING_LEVEL;
}

```

After recompiling with this change, the problem disappears completely: the player and the enemies from each previous section are removed from the screen the instant the next section is loaded, and only the sprites belonging to the current section remain visible.

`SPR_reset()` releases absolutely every sprite allocated so far, without distinguishing what category they belong to. As long as the only thing on screen is the player and the enemies, this is exactly what's needed; but if in a future chapter the game adds a life counter or a score icon that should survive a level transition, that sprite would also disappear when `SPR_reset()` is called inside `levelLoader()`, and it would need to be recreated right there, or the question of which sprites get released and which don't would need to be rethought.

Look for the existing function first. Any function in an engine or framework that allocates a resource is usually accompanied by an equivalent function to release it. Faced with a resource-accumulation bug like the one in this section, it's worth asking first: does the framework being used already have a function designed exactly for this problem? In SGDK, the answer is almost always yes, and looking it up in the documentation usually saves a lot of time compared to trying to solve the problem by hand—for example, keeping a custom array of pointers to `Sprite` just to be able to loop through it and call `SPR_releaseSprite()` sprite by sprite.

7. A deliberate crash: what happens when the level array overflows

To close out the chapter, a question asked with a bit of mischief: what happens if Start is pressed more times than there are sections defined in `all_enemies`? That catalog was declared with capacity for ten sections (indices 0 through 9), so pressing Start eleven times brings `current_game_section` to the value 10, an index that falls outside the array's valid bounds.

The result, the first time the array overflows by a single position, isn't an immediate crash: a single enemy appears on screen, with data that clearly doesn't correspond to any section designed on purpose. This happens because index 10 falls right into the memory contiguous to `all_enemies`, which at that moment happens to contain some data that can be interpreted as a valid enemy. Continuing to press Start and overflowing the array by one more position, the program finally crashes with an *address error* message.

This is what the address error produced by the 68000 looks like when reading memory outside the bounds of `all_enemies`.

An *address error* of this kind is how the Mega Drive, more precisely its Motorola 68000 processor, reacts when the program tries to access a memory address in a way the hardware doesn't allow, something that happens frequently when an array overflows and ends up reading memory that doesn't belong to that array.

It's tempting to think that an immediate, loud crash is the worst possible outcome for a bug

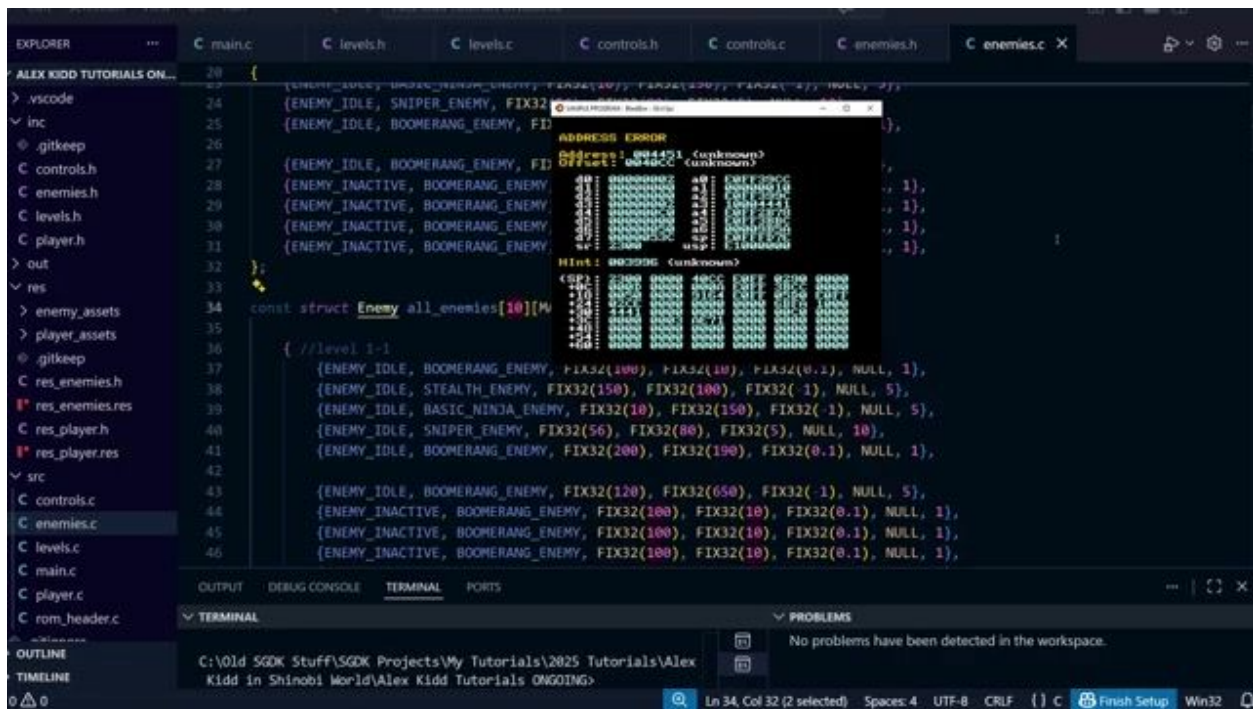


Figure 155: Emulator address error when overflowing the all_enemies array

like this, but it’s worth considering the opposite argument: the intermediate result (a single enemy with corrupted data, with no error message at all) is actually more dangerous, because it invites looking for the cause of the problem in the wrong place—the AI logic, for instance—instead of correctly suspecting an array overflow. An immediate, explicit crash, even though it’s alarming the first time it’s seen, unambiguously signals that something was written or read outside the memory bounds it was supposed to stay within, narrowing down considerably where to look for the bug.

This chapter closes without fixing the ten-section limit: this is a problem that naturally gets resolved as the game’s development progresses and defines its actual number of levels, and it doesn’t warrant further concern for now. It is worth remembering this behavior, though, the next time a strange, inconsistent bug shows up in a project of your own: before suspecting more complex logic, it’s worth ruling out an array overflow first.

Review exercises

1. Explain why `SYS_doVBlankProcess()` is deliberately left out of the `if/else if` chain that distinguishes the game states, unlike `handleInput()` or `manageEnemies()`.
2. Why is it essential that `levelLoader()` change `current_game_state` to `RUNNING_LEVEL` as its last step? What would happen if that line were left out by mistake?
3. Explain the exact cause of the bug where sprites from previous sections remained visible when transitioning between sections, and describe the fix that was applied.
4. Why doesn’t the first overflow of the `all_enemies` array (reaching index 10) produce an immediate crash, while the second overflow does?
5. If you had to increase the game’s level capacity from 10 to 20 sections, what changes would need to be made, at minimum, to the code described in this chapter and in the chapter on loading enemies per level?

Chapter 61 — Loading a New Level's Graphics

A Level struct, a level catalog, and the actual loading of background, foreground, and palette into VRAM via DMA, respecting the limits covered in Part VIII

The previous chapter left the full skeleton of a level transition working: pressing Start increments `current_game_section`, triggers the `LOADING` state, and `levelLoader()` takes care of returning the game to the `RUNNING_LEVEL` state. However, that skeleton still doesn't move a single tile onto the screen: no matter what section the player is in, the background shown is always the same, because `levelLoader()` currently only calls `createPlayer()` and `createEnemies()`. This chapter closes that loop, finally connecting `current_game_section` with the actual loading of each level's backgrounds, foregrounds, and palettes, and along the way answers a question left pending since Part VIII: what does it actually mean, in practice, to upload the complete graphics of a new level to VRAM without the transition showing up as a long pause or visual corruption?

The result isn't just showing different graphics per section: it's doing so cleanly, with the correct palettes from the very first visible frame, and with a fade transition that hides any flicker while the new data travels to VRAM. Along the way, the same kind of array-overflow crash that already showed up with the enemy catalog reappears, now applied to the level catalog.

Prerequisites

Before continuing, it helps to have on hand:

- The state machine (`current_game_state`, with its `LOADING` and `RUNNING_LEVEL` values) and the `current_game_section` variable, along with `levelLoader()` and `SPR_reset()`, as they were left in the chapter on level transitions.
- The `Enemy` struct and the `all_enemies` catalog as a direct reference: this chapter's `Level` struct follows the same design pattern, including the same kind of array-overflow crash if you advance past the catalog's limit.
- An understanding of the `BG_A/BG_B` planes, `VDP_drawImageEx()`, and `TILE_ATTR_FULL()`, and of why it's worth splitting a level into background and foreground to enable parallax scrolling, as explained in the chapter on preparing level graphics.
- Familiarity with managing multiple palettes and with `PAL_fadeIn()/PAL_fadeOut()`, covered in the chapter dedicated to palette fades.
- The DMA mechanism explained in depth in chapter 44 (`DMA_doDma()`, `DMA_queueDma()`, the transfer queue, and its per-vblank capacity limits) and the split of VRAM between background and sprites covered in chapter 45 (`TILE_USER_INDEX`, `SPR_initEx()`), since this chapter revisits both topics and applies them to a concrete case: loading a full level.

- A notion of how `rescomp` generates a header file from a `.res` file, as explained in the chapter dedicated to the resource compiler.

1. A level's graphic resources: background, foreground, and a separate palette

Each level in the game is made up of two independent images —a background and a foreground— prepared following the artistic criteria already explained in the chapter on preparing level graphics: separating both layers is precisely what enables parallax scrolling between them later on. A third, much smaller image is added to those two, whose sole purpose is to store color information: the screen's overall background color and the four system palettes (PAL0 through PAL3) in full, sixty-four colors in total. In this project, PAL0 and PAL1 are reserved for the level's background and foreground, PAL2 for enemies, and PAL3 for the player and the HUD.

Storing the palette as a separate resource, instead of letting each image carry its own, has a concrete advantage: it allows combining the same background with different foregrounds that use different palettes for PAL1, which will come in handy the day a single scene needs lighting or section variants. The three resources for each level are declared using the same `.res` file mechanism already used in previous chapters for sprites and backgrounds:

```
// res_levels.res

IMAGE level1_palette      "levels/level1_palette.png"      NONE
IMAGE level1_background   "levels/level1_background.png"  BEST ALL
TILESET level1_fg_tileset "levels/level1_foreground.png"  BEST ALL
MAP level1_fg_map         "levels/level1_foreground.png"  level1_fg_tileset BEST 0

IMAGE level2_palette      "levels/level2_palette.png"      NONE
IMAGE level2_background   "levels/level2_background.png"  BEST ALL
TILESET level2_fg_tileset "levels/level2_foreground.png"  BEST ALL
MAP level2_fg_map         "levels/level2_foreground.png"  level2_fg_tileset BEST 0

IMAGE level3_palette      "levels/level3_palette.png"      NONE
IMAGE level3_background   "levels/level3_background.png"  BEST ALL
TILESET level3_fg_tileset "levels/level3_foreground.png"  BEST ALL
MAP level3_fg_map         "levels/level3_foreground.png"  level3_fg_tileset BEST 0
```

The `level1_palette` resource uses the `NONE` compression mode because it's a tiny image: compressing it provides no real savings and only adds an unnecessary decompression step. When compiled, `rescomp` generates `res_levels.h` following the same logic already seen for any other resource: each name becomes available as a read-only global variable, ready to be included from `levels.c`.

2. A Level struct to describe each section of the game

2.1. Defining struct Level

With the resources already uploaded to the ROM, the next step is to describe each level of the game with a struct, using exactly the same criteria already applied to enemies: gathering, in a single place, all the information a level needs in order to be displayed on screen.

```

// levels.h
#pragma once

#include <genesis.h>
#include "res_levels.h"

// The three level configurations planned for this project.
// For now only LEVEL_PARALLAX is implemented; the other two are
// declared in advance so the struct and the catalog don't need to
// be redesigned the day they get implemented.
typedef enum {
    LEVEL_PARALLAX,    // independent background and foreground, with parallax
    ↪ scroll
    LEVEL_DOUBLE_MAP,  // two tile maps at different speeds (indoor scenes)
    LEVEL_STATIC       // a single static image with a collision map, no scroll of
    ↪ its own
} LevelType;

typedef struct {
    LevelType type;
    const Image* palette;                // 8x8 image used only to extract the
    ↪ level's 64 colors
    const Image* background;             // level background, drawn on BG_B
    const TileSet* foreground_tileset;    // unique foreground tiles
    const MapDefinition* foreground_map;

    // The following fields aren't used yet in this chapter: they're
    // declared from the start so the struct -and therefore every entry
    // already written into the catalog- doesn't need to change the day
    // the camera and level collision specific to this project get
    // implemented (see the general chapters on camera and collision
    // in Parts III and VII).
    u16 width;
    u16 height;
    const u8* collision_map;
    s16 player_start_x;
    s16 player_start_y;
} Level;

#define LEVEL_CAPACITY 10
extern const Level all_levels[LEVEL_CAPACITY];

// VRAM position currently being written to during loading
extern u16 vram_index;
// The foreground map, already prepared and ready to display on screen
extern Map* fg_map_def;
// Intermediate copy of the level's 64 colors, used as the source for the fade-in
extern u16 palette_full[64];

void levelLoader(void);
void screenResetManager(void);

```

Several of Level's fields are pointers (Image*, TileSet*, MapDefinition*), which

requires writing the corresponding asterisk when declaring each field. Leaving out the asterisk by mistake is an easy typo to make, one the compiler doesn't always flag right away with a clear message; instead, it can show up later as strange behavior when reading those fields. It's worth carefully checking the syntax of every pointer field when writing a struct like this one.

`vram_index`, `fg_map_def`, and `palette_full` are three support variables needed by the loading process in the next section: `vram_index` tracks the VRAM position currently being written to; `fg_map_def` holds the pointer to the foreground map returned by `MAP_create()`, needed later to free it; and `palette_full` serves as an intermediate step for copying a level's palette before applying a fade-in to it.

2.2. The all_levels catalog

With the struct already defined, an array of ten elements of type `Level`, `all_levels`, is declared, using the same provisional sizing criteria already used for `all_enemies`. The actual content is written in `levels.c`, marked as `const` so it's stored in ROM rather than occupying RAM, since each level's configuration doesn't change during gameplay:

```
// levels.c
#include "levels.h"
#include "player.h"
#include "enemies.h"

u8 current_game_section = 0;
u16 vram_index;
Map* fg_map_def;
u16 palette_full[64];

const Level all_levels[LEVEL_CAPACITY] = {
    { LEVEL_PARALLAX, &level1_palette, &level1_background, &level1_fg_tileset,
      ↪ &level1_fg_map, 0, 0, NULL, 0, 0 },
    { LEVEL_PARALLAX, &level2_palette, &level2_background, &level2_fg_tileset,
      ↪ &level2_fg_map, 0, 0, NULL, 0, 0 },
    { LEVEL_PARALLAX, &level3_palette, &level3_background, &level3_fg_tileset,
      ↪ &level3_fg_map, 0, 0, NULL, 0, 0 },
    { LEVEL_PARALLAX, &level1_palette, &level1_background, &level1_fg_tileset,
      ↪ &level1_fg_map, 0, 0, NULL, 0, 0 },
    { LEVEL_PARALLAX, &level2_palette, &level2_background, &level2_fg_tileset,
      ↪ &level2_fg_map, 0, 0, NULL, 0, 0 },
    { LEVEL_PARALLAX, &level3_palette, &level3_background, &level3_fg_tileset,
      ↪ &level3_fg_map, 0, 0, NULL, 0, 0 },
    { LEVEL_PARALLAX, &level1_palette, &level1_background, &level1_fg_tileset,
      ↪ &level1_fg_map, 0, 0, NULL, 0, 0 },
    { LEVEL_PARALLAX, &level2_palette, &level2_background, &level2_fg_tileset,
      ↪ &level2_fg_map, 0, 0, NULL, 0, 0 },
    { LEVEL_PARALLAX, &level3_palette, &level3_background, &level3_fg_tileset,
      ↪ &level3_fg_map, 0, 0, NULL, 0, 0 },
    { LEVEL_PARALLAX, &level1_palette, &level1_background, &level1_fg_tileset,
      ↪ &level1_fg_map, 0, 0, NULL, 0, 0 },
};
```

For testing purposes, the catalog alternates between three different pre-prepared levels, so the sections can be told apart at a glance while advancing with Start. Each entry's last six fields are placeholder values, not used yet, awaiting the camera and level collision.

Just like with `all_enemies`, `all_levels` also has a fixed capacity of ten entries. Advancing past that limit —by pressing Start eleven times in a row, for example— produces exactly the same kind of array-overflow crash (*address error*) already seen with the enemy catalog: the program tries to read an entry of `all_levels` that's outside its declared bounds. The cause and the fix are identical to that earlier case, so they aren't repeated in detail here; for now, it's simply best not to exceed the ten defined sections.

3. Loading the graphics in `levelLoader()`

With the catalog already complete, the loading process is added inside `levelLoader()`. The procedure starts by positioning `vram_index` at the beginning of the index reserved for user tiles (`TILE_USER_INDEX`), draws the current section's background with `VDP_drawImageEx()`, advances `vram_index` by the number of tiles that background occupied, uploads the foreground tileset with `VDP_loadTileSet()` starting from that new position, and finally builds the foreground map with `MAP_create()` and scrolls it with `MAP_scrollTo()`:

```
void levelLoader(void) {
    const Level* level = &all_levels[current_game_section];

    vram_index = TILE_USER_INDEX;

    VDP_drawImageEx(BG_B, level->background,
        TILE_ATTR_FULL(PAL0, FALSE, FALSE, FALSE, vram_index), 0, 0, FALSE, DMA);
    vram_index += level->background->tileset->numTile;

    VDP_loadTileSet(level->foreground_tileset, vram_index, DMA);
    fg_map_def = MAP_create(level->foreground_map, BG_A,
        TILE_ATTR_FULL(PAL1, FALSE, FALSE, FALSE, vram_index));
    MAP_scrollTo(fg_map_def, 300, 0);

    createPlayer();
    createEnemies();

    current_game_state = RUNNING_LEVEL;
}
```

Throughout this sequence, `vram_index` plays the same role already seen in the chapter on backgrounds: every graphic resource uploaded to VRAM needs to know what position to start writing from, and that position depends on how much space the resources uploaded earlier in that same load already took up. Keeping track in a single variable, instead of computing separate offsets on each call, is what lets the same block of code work unchanged no matter which level is being loaded at any given moment.

Compiling and testing the result at this point, the level's graphics do show up on screen, confirming that the loading mechanism works, though with a pending visual problem: since no level-specific palette has been loaded yet, `PAL0` shows up as white by default and `PAL1` as red, leaving some areas of the foreground visibly different from the rest.

The level's graphics are already on screen, but without their own palette: `PAL0` shows up white and `PAL1` red by default.

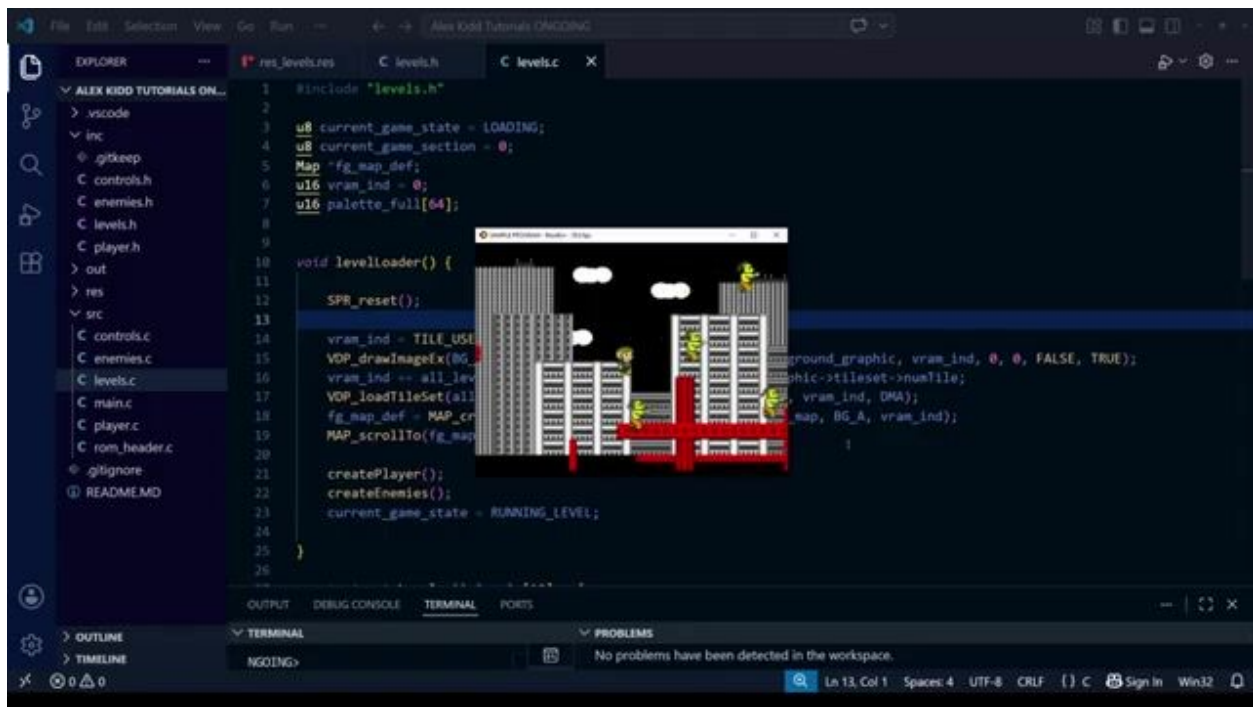


Figure 156: Level with PAL0 and PAL1 not yet loaded

4. Why the colors come out wrong: manual palettes versus per-level palettes

If you open the emulator’s palette debugging screen at this point, PAL2 and PAL3 —the enemy’s and the player’s palettes— show up with their correct colors, while PAL0 and PAL1 remain white and red. The reason is that `createPlayer()` and `createEnemies()` still call `PAL_setPalette()` manually to set their own palettes, something that made sense before a level system existed, but which now conflicts with the idea of loading all of a level’s palettes in a centralized way.

The fix is to remove those manual calls to `PAL_setPalette()` from both `createPlayer()` and `createEnemies()`. The goal is for the game’s four palettes to be loaded from a single, centralized place inside `levelLoader()`, instead of each function responsible for creating an on-screen element managing its own palette separately.

The first step of that centralization is copying the information from the current level’s small palette image —accessible as `all_levels[current_game_section].palette->data`— into the already-declared `palette_full` array, using `memcpy()` to copy all 64 colors in one go:

```
memcpy(palette_full, level->palette->data, sizeof(palette_full));
```

With the colors already copied, the final step is to call `PAL_fadeIn()` at the end of `levelLoader()`, passing it `palette_full` as the color source and a short duration, measured in frames:

```
PAL_fadeIn(0, 63, palette_full, 15, FALSE);
```

The 0, 63 range covers all four palettes at once, since `palette_full` carries the 64 colors in the same order they live in in CRAM. After recompiling, the level finally shows up with its real colors from the very first visible instant on screen.

5. Clean transitions: fade-out and memory release

With the fade-in already working, one detail still remains unresolved: when switching levels, the player briefly gets to see half-loaded tiles while the new data is being copied to VRAM. The fix is to add a fade-out, `PAL_fadeOut()`, at the start of the loading process, so the screen goes black before the new graphic data starts being written, and the new level is only revealed once everything is ready.

There's also a loose end of a different nature: every time `MAP_create()` is called to generate `fg_map_def`, SGDK allocates dynamic memory for that structure. If that memory is never freed before preparing the next level's map, every level transition leaves behind allocated memory that's no longer used and never reclaimed—something that isn't noticeable today because the game is small, but that would eventually cause an out-of-memory crash as the game grows. The fix is to call `MEM_free(fg_map_def)` before calling `MAP_create()` again when loading the next level.

These three cleanup tasks—fade-out, sprite reset, and memory release—are grouped into a separate function, `screenResetManager()`, whose sole purpose is to leave the screen and memory in a clean state before loading any new level:

```
// levels.c

void screenResetManager(void) {
    PAL_fadeOut(0, 63, 10, FALSE);
    SPR_reset();

    if (fg_map_def != NULL) {
        MEM_free(fg_map_def);
        fg_map_def = NULL;
    }

    // Leaves all four palettes black between one fade and the next,
    // to rule out any chance of a leftover color from the previous
    // level bleeding through visually while the new data is loading
    PAL_setColors(0, palette_black, 64, DMA);
}
```

`screenResetManager()` isn't called from `levelLoader()`, but from `controls.c`, inside the same condition that already increments `current_game_section` and changes `current_game_state` to `LOADING` when `Start` is pressed:

```
// controls.c (joyEvent excerpt)
if (changed & buttons & BUTTON_START) {
    screenResetManager();
    current_game_section++;
    current_game_state = LOADING;
}
```

The reason is visual synchronization: it's better for the screen to start fading to black the very instant the player presses `Start`, rather than on the frame in which `levelLoader()` actually runs, which could be slightly delayed depending on the rest of the main loop.

6. From a blocking transfer to the DMA queue

Everything above already leaves the level-change cycle working end to end, but it's worth taking a closer look at what's happening under the hood of `VDP_drawImageEx()` and `VDP_loadTileSet()` when uploading the graphics, because this is exactly the scenario chapter 44 left pending for this point in the manual.

Both `VDP_drawImageEx(..., DMA)` and `VDP_loadTileSet(..., DMA)` trigger, underneath, an immediate, blocking transfer equivalent to `DMA_doDma()`: the 68000 is halted until the VDP finishes copying the tiles. In `levelLoader()` this isn't a problem, because it happens only once per transition and with the screen already black thanks to `screenResetManager()`: there's nothing on screen depending on the CPU continuing to run at that exact instant. The real cost isn't visual, it's about time: while the DMA is busy copying a level's tiles, no other game calculation moves forward, and since this load doesn't necessarily happen during vblank — it could start in the middle of a frame's active drawing, depending on exactly when `levelLoader()` runs within the main loop— the transfer also runs at the reduced speed that chapter 44 described for that period.

For a level with a modest background and foreground, a few thousand bytes in total, this speed difference isn't noticeable. The problem shows up with the `LEVEL_DOUBLE_MAP` level type anticipated in the enum: an indoor scene with two independent tile maps can easily accumulate more unique tiles than a single vblank is able to move, based on the figure exposed by `DMA_getMaxTransferSize()`. Forcing that amount of data through `DMA_doDma()` all at once doesn't produce any error, but it does produce a proportionally longer CPU stall, potentially spilling partly into the active drawing of the next frame, with the speed penalty that implies.

The alternative, as laid out in chapter 44, is to split the load across several frames using the DMA queue instead of an immediate transfer, checking before each chunk how much budget remains available in the current frame with `DMA_canQueue()`:

```
// Uploads a full TileSet to VRAM in chunks that respect a single
// vblank's transfer limit, instead of one blocking call.
// Meant for large tilesets (for example, a LEVEL_DOUBLE_MAP), where
// a single DMA_doDma() call might not comfortably fit within one frame.
static void loadTileSetFractioned(const TileSet* tileset, u16 index) {
    u16 words_left = tileset->numTile * 16; // 16 words (32 bytes) per tile
    u16 words_sent = 0;
    u16 dest = index * 32;

    while (words_left > 0) {
        u16 max_chunk = DMA_getMaxTransferSize();
        u16 chunk = (words_left < max_chunk) ? words_left : max_chunk;

        // If the chunk doesn't fit within the current frame's remaining
        // budget, wait for the next vblank before trying again
        while (!DMA_canQueue(chunk)) {
            SYS_doVBlankProcess();
        }

        DMA_queueDma(DMA_VRAM, (u8*) tileset->tiles + words_sent * 2, dest, chunk,
↪ 2);

        words_sent += chunk;
        words_left -= chunk;
        dest += chunk * 2;
    }
}
```

```

        // Lets this chunk be processed on the next vblank before
        // queuing up the next one
        SYS_doVBlankProcess();
    }
}

```

Replacing the call to `VDP_loadTileSet(level->foreground_tileset, vram_index, DMA)` with `loadTileSetFractioned(level->foreground_tileset, vram_index)` spreads the same total amount of data across several vblanks instead of just one, without the screen —already black thanks to `screenResetManager()`— showing any hint that the load is happening in several steps. For this chapter’s `LEVEL_PARALLAX` levels, whose data volume is modest, the blocking version remains perfectly reasonable; the fractioned version is the tool to reach for the day a heavier level type requires it, rather than an optimization that needs to be applied across the board as a matter of course.

Before reaching that point, though, it’s worth measuring: chapter 45 already showed how to count unique tiles with a graphics editor’s tileset tool and how to split the available VRAM between background and sprites with `SPR_initEx()`. Applying that same criteria here, it’s useful to verify at runtime that no level in the catalog exceeds the tile budget the project reserved for the combined background and foreground:

```

#define LEVEL_TILE_BUDGET 1000 // tiles available for background + foreground,
    ↪ leaving the rest to SPR_initEx()

u16 tiles_needed = level->background->tileset->numTile +
    ↪ level->foreground_tileset->numTile;
if (tiles_needed > LEVEL_TILE_BUDGET) {
    KLog_U1("Level exceeds VRAM budget, tiles needed: ", tiles_needed);
}

```

This check doesn’t replace the earlier measurement with the graphics editor already explained in chapter 45; it’s more of a safety net for catching, in the emulator during development, a level whose art grew beyond what was planned before the problem shows up as visual corruption on screen.

Review exercises

1. Explain why a level’s palette information is stored in a separate resource of just 64 colors, instead of being included directly within the background or foreground image.
2. Describe the role of `vram_index` inside `levelLoader()`: why is it necessary to increment it after drawing the background, and what would happen if it weren’t?
3. Why did `PAL2` and `PAL3` show their correct colors from the start, while `PAL0` and `PAL1` showed up as white and red by default, before the changes in section 4?
4. `VDP_drawImageEx()` and `VDP_loadTileSet()` use, in this chapter, an immediate DMA transfer (DMA) instead of a queue. Explain under what condition that choice stops being harmless, and what alternative SGDK offers for that case.
5. Explain what problem `MEM_free(fg_map_def)` solves, why it makes sense to group it together with `PAL_fadeOut()` and `SPR_reset()` inside `screenResetManager()`, and why that function is called from `joyEvent()` in `controls.c` instead of from the start of `levelLoader()`.

Chapter 62 — Wrapping Up and Extending the Project

A look back at what was built throughout this part, and a guide to continuing on toward the HUD, sound, and save games

The previous twelve chapters built, piece by piece, a small functional beat 'em up: a player and several types of permanent enemies defined in a reusable catalog, a per-enemy artificial intelligence resolved with function pointers, a state machine that distinguishes between playing and loading a level, and a section transition that replaces the background, the foreground, and the entire palette without the loading being noticeable as a long pause. This is a good moment to stop, look at the project as a whole, and place it within the limits it has today, before continuing to add new layers.

Unlike the chapter that closes each of the other parts of this manual, this one does not introduce a new technical topic. Its purpose is different: to review how the pieces of this part fit together, to point out precisely what is still missing for the project to feel like a complete game, and to lay out the most direct path from what has already been built toward the topics that still lie ahead in this manual.

1. The project as it stands

It's worth reconstructing, in a single picture, the full journey of this part. It all started by organizing the VDP planes to clearly separate the game background from the character sprites, and by splitting that growing codebase into its own header files (`player.h`, `enemies.h`, `levels.h`, `controls.h`) so that each piece of the project lives in its own file without losing the ability to communicate with the others. On that foundation, enemies went from being loose variables to an array, and from an array to a full struct `Enemy`, indexed through an enum `EnemyTypes` instead of bare numbers with no meaning. A catalog of permanent types (`all_enemy_types`) centralized each enemy's fixed characteristics — maximum health, speed, sprite, palette — and a function pointer inside that same struct allowed each enemy type to have its own behavior without a single giant block of repeated `if/else` statements in the main loop.

In parallel, a state variable backed by an enum `GameStates` gave the game the ability to distinguish between playing and loading a new level, blocking player control during the transition; and a catalog of levels per section, with its own struct and its own enum, finished connecting that state machine to the actual loading of backgrounds, foregrounds, and palettes for each section, using the DMA mechanism covered in depth in Part VIII. The result is a player who moves, enemies with their own behaviors that appear depending on the section, and a button that advances from one section to the next, loading different graphics each time.

Put another way: the project in this part is not a finished game, but the backbone of one. It

has architecture, it has data organized in a scalable way, and it has the full level-loading cycle working end to end. What it still lacks is nearly everything a player would expect to see on screen beyond the character and the enemies: there's no health bar that drops when hit, no score, no sound, and no way to save progress between one play session and the next.

2. What's missing, and why it's deliberately left open

Closing this part with a complete game — with a main menu, a HUD, a scoring system, and win/lose conditions — would be possible, but it would come at a pedagogical cost: it would force several topics that deserve their own space to be introduced all at once, mixed together in a single closing chapter. That's why the project in this part is deliberately left open, unlike the project in Part XI, which does go through its entire pipeline all the way to a complete, playable game.

Three concrete extensions stand out as the most natural way to continue from here, and all three rely purely and exclusively on tools this manual has already covered:

A HUD with lives and score. Displaying the player's health and a point counter on screen doesn't require any new technique: it's enough to reserve a strip of the BG_A or BG_B plane — as explained in the chapter on VDP planes that opened this part — and update it with `VDP_drawText()` whenever either value changes, with the same care already noted in Part I about reserving one extra character in the destination string for negative numbers. The trickiest point of a HUD within this particular project is that `levelLoader()` calls `SPR_reset()` on every level transition, freeing *all* sprites reserved up to that point; if the HUD is ever implemented with sprites instead of text on a background plane, those sprites would need to be recreated inside `levelLoader()` after the `SPR_reset()` call, or `SPR_reset()` would need to be replaced with a more selective release that distinguishes the HUD's sprites from those of the player and the enemies.

Sound. The project in this part didn't play a single sound effect or a single music track, but everything needed to do so was already covered in Part V: loading a sound effect with `rescomp` and triggering it with `SND_startPlay_PCM()` or the corresponding driver when a hit is received, or changing the background music with each level transition inside `levelLoader()`, following the same pattern already used there to load new backgrounds and palettes.

Saving the game. Neither this project nor any earlier chapter has yet touched on preserving the player's progress between console power cycles. That topic is developed later in this manual, in Part X, and it's the natural complement to a level catalog like `all_levels`: without saving, `current_game_section` always resets to zero when the console restarts; with saving, that same value could be read from non-volatile memory as soon as the program starts, picking the player back up exactly where they had left off.

None of the three extensions requires redesigning what has already been built. All three rely on the same architecture of structs, enums, and enum-indexed catalogs that ran through this entire part, which is, in the end, the most important lesson the project leaves behind: once a game's data is well organized — instead of scattered across loose variables — adding new functionality rarely means rewriting what already works.

3. A pattern that keeps repeating: why it's worth recognizing

Before closing, there's a pattern that repeated at least three times throughout this part, under a different name each time: a catalog of fixed data, represented as a constant array of structs, indexed by an enum, and queried from code rather than hardcoded by hand. The enemy-type

catalog (`all_enemy_types`), the per-section enemy catalog (`all_enemies`), and the level catalog (`all_levels`) are three applications of the same pattern to three different problems.

Recognizing this pattern as a general tool, rather than a one-off solution tied to enemies or levels, is useful because it will resurface in whatever form a game needs to describe “several things of the same kind, with the same characteristics but different values”: collectible objects, projectile types, a character’s dialogue lines, difficulty settings. The same mold — struct, enum, constant array, an access function that validates the index before using it — serves all of those cases, with the same precaution noted in the chapter on level transitions: a fixed-size array can always overflow if the index traversing it isn’t validated against its actual bound.

Review exercises

1. Explain, in your own words, why the project in this part is considered “a backbone” rather than “a finished game.” What concrete elements does it lack to feel like a complete game?
2. Describe the design pattern that repeats across `all_enemy_types`, `all_enemies`, and `all_levels`: what do the three catalogs have in common, beyond the specific data each one stores?
3. If you had to add a HUD with sprites (instead of text over a background plane) to display health icons, what concrete conflict would you need to resolve with the `levelLoader()` function as it stood at the end of the previous chapter?
4. Why does saving a game require non-volatile memory, and why isn’t an ordinary C variable like `current_game_section` enough to preserve that value between one console power-on and the next?
5. Think of a game feature that wasn’t mentioned in this chapter (for example, collectible objects or different weapons) and describe, in broad strokes, how you would apply the struct/enum/constant-array catalog pattern to represent it.

Chapter 63 — HInterrupts and Raster Effects

The VDP's horizontal interrupt, `SYS_setHIntCallback()`, and differential scroll by screen bands

This chapter opens Part X of the manual, devoted to advanced topics that didn't fit comfortably into the previous parts: they aren't a direct continuation of a topic already underway, but rather specific tools worth knowing once the rest of the foundation — the VDP, DMA, the general structure of an SGDK project — is already settled. The first of these tools is the horizontal interrupt, or *HInterrupt* (also called *HInt*, or in English-language documentation, *horizontal interrupt*).

Up to this point, any change made to the VDP — moving a plane, changing a palette, updating a tile — has always happened at one of two possible moments: during the main loop, while the screen has already been drawn and what's showing is still the previous frame, or during the *vertical blank* (vblank), the interval between when one frame finishes drawing and the next one begins, which is what `SYS_doVBlankProcess()` targets. The HInterrupt adds a third, much finer-grained possibility: reacting in the middle of the drawing of a single frame, exactly when the VDP's scan beam finishes one line of video and is about to start the next. That makes it possible to change the VDP's state — the background color, a plane's horizontal offset, in theory almost any register — at a precise point on the screen, and it's the basis for a classic family of effects generically known as *raster effects*: water that ripples with a different scroll offset on each horizontal strip, a starry sky that scrolls at a different speed than the ground, a split screen where a HUD area stays fixed while the rest keeps scrolling.

Prerequisites

Before continuing, you should be comfortable with:

- Understanding of the VDP, its BG_A/BG_B planes, and the three granularities of horizontal scroll (`HSCROLL_PLANE`, `HSCROLL_TILE`, `HSCROLL_LINE`), introduced in chapter 2 and put into code in chapters 9 and 17.
- Having worked with `VDP_setScrollingMode()` and `VDP_setHorizontalScrollTile()`, from chapter 17, since this chapter extends the same idea to the finest granularity, per line.
- The general structure of an SGDK program, with its main loop organized around `SYS_doVBlankProcess()`.
- Ideally, having read chapter 44 on DMA, because one of this chapter's precautions is precisely which transfer mechanisms **cannot** be used inside an HInterrupt.

1. What a horizontal interrupt is at the hardware level

The Mega Drive's VDP draws the screen line by line, top to bottom, sweeping each line from left to right before moving to the next. That process is exactly the same mechanism, inherited from analog television, that makes the very concept of vblank possible: when the scan reaches the last visible line and has to return to the top for the next frame, there's an interval with nothing to draw, and that's where the vertical interrupt (VInt) that triggers `SYS_doVBlankProcess()` occurs.

The horizontal interrupt is the same idea, but applied to each individual line instead of to each complete frame. The VDP has an internal line counter and a dedicated register — register 0x0A, in the VDP's internal register numbering — that specifies how many lines should elapse between interrupt triggers. SGDK exposes that counter with:

```
void VDP_setHIntCounter(u8 value);
```

The value loaded there isn't "every how many lines," but "how many lines to wait before the first one": with `value = 0` the interrupt fires on every line, and in general, loading `N` makes the HInterrupt occur every `N + 1` scan lines. That counter reloads itself automatically after each trigger, so once configured it keeps producing interrupts at regular intervals without the program having to reprogram it line by line.

The interrupt itself has to be enabled separately, with:

```
void VDP_setHInterrupt(bool value);
```

`VDP_setHInterrupt(TRUE)` enables the trigger; `VDP_setHInterrupt(FALSE)` turns it off completely, without touching the counter value (which stays saved for the next time it's re-enabled). An important detail for understanding where it makes sense to use this mechanism: the HInterrupt fires while the VDP is scanning through the screen's active drawing area, line by line; when it reaches the end of the frame, it's the vertical interrupt that takes over during the vblank, not the horizontal one. This is precisely what makes the HInterrupt useful for this chapter: it provides a hook point exactly where the VInt doesn't reach, in the middle of the frame's own drawing, rather than only before or after it.

A necessary clarification: it's not magic, it's real time. The CPU doesn't "pause" the VDP's drawing to handle the interrupt at leisure. The screen's scan keeps going exactly at the pace set by the video clock, whether or not program code runs in the meantime. If the interrupt handler takes too long to finish, the VDP will already have started drawing the next line with the old state, or the moment to update it in time will simply be missed. Section 5 of this chapter returns to this idea in more detail.

2. Registering the handler in SGDK: `SYS_setHIntCallback()`

Enabling the interrupt in the VDP isn't enough: SGDK also needs to be told which C function to run each time it occurs. That's done with:

```
void SYS_setHIntCallback(VoidCallback *CB);
```

which takes a pointer to a function with no parameters and no return value. It's the same pattern already used with `SYS_setVBlankCallback()` to hook into the vblank, but with an important difference: the function registered as the HInterrupt handler has to be declared with a special macro, `HINTERRUPT_CALLBACK`, instead of the usual `void`:

```
HINTERRUPT_CALLBACK hintHandler(void)
{
    // the handler's body
}
```

HINTERRUPT_CALLBACK is nothing more than a void with an added compiler attribute (`__attribute__((interrupt))`), which tells the compiler that this function will be invoked directly as the target of a 68000 interrupt, rather than as a normal call from elsewhere in the C program. That makes the compiler generate a different function prologue and epilogue: it makes sure to save and restore every CPU register it uses, and to end with the 68000's interrupt-return instruction instead of an ordinary subroutine return. If you forget the macro and declare the handler as a regular function, the program may still compile but corrupt CPU registers at the worst possible moment, producing intermittent failures that are hard to track down.

With that, enabling an HInterrupt that fires, say, every 8 lines comes down to three lines of code, typically placed during initialization:

```
SYS_setHIntCallback(hintHandler);
VDP_setHIntCounter(7);          // 7 + 1 = every 8 lines
VDP_setHInterrupt(TRUE);
```

3. Per-line scroll: the other half of the effect

Chapter 17 showed HSCROLL_TILE, which allows an independent horizontal offset for each 8-pixel row of a plane. The VDP supports an even finer granularity, HSCROLL_LINE, which allows a different horizontal offset value for each individual line of the screen, without grouping them in sets of 8. It's enabled with the same function already covered:

```
VDP_setScrollingMode(HSCROLL_LINE, VSCROLL_PLANE);
```

and updated with the equivalent of VDP_setHorizontalScrollTile() for this granularity:

```
void VDP_setHorizontalScrollLine(VDPPlane plane, u16 line, s16 *values, u16 len,
    ↪ TransferMethod tm);
```

The parameters are analogous to those of VDP_setHorizontalScrollTile(): plane is BG_A or BG_B; line is the first screen line, counted from the top, from which the values are applied; values is an array holding one offset per line; len is the number of lines to update; and tm is the transfer method (CPU, DMA, DMA_QUEUE, or DMA_QUEUE_COPY, as covered in chapter 44).

Here's a key point worth emphasizing, because it tends to cause confusion: this table of per-line offsets is read by the VDP **on its own**, automatically, every time it draws each screen line. That means a rippling effect — like the classic water reflection, with each horizontal strip offset slightly differently from the one above it — **doesn't necessarily require an HInterrupt**: it's enough to compute the entire table of values once per frame, during the vblank, and upload it all at once with a single call to VDP_setHorizontalScrollLine(). The VDP then takes care, in real time throughout the whole active drawing pass, of reading a different value for each line without the program having to intervene line by line. This is, in fact, the simplest and most common way to achieve a rippling water effect in SGDK, and it needs none of what was explained in sections 1 and 2.

So what is the HInterrupt needed for? For everything that table **can't** resolve on its own:

- **Changing something that has no per-line table.** The VDP's background color

(VDP_setBackgroundColor()) is a single value, not a table: if you want a sky that changes shade line by line, or a different color band for the “ground” of a background, there’s no way to preload that into a one-shot table; you have to rewrite that value exactly when the scan beam crosses each line, and only an HInterrupt can do that.

- **Vertical scroll has no per-line granularity.** Unlike horizontal scroll, which supports up to one value per line (HSCROLL_LINE), the VDP’s vertical offset only supports two granularities: per whole plane (VSCROLL_PLANE) or per two-tile columns (VSCROLL_COLUMN, called VSCROLL_2TILE in old versions of SGDK). There’s no “one vertical offset per line” table analogous to the horizontal one. Any effect that needs to vary the vertical scroll line by line — a relatively rare need, but one that shows up in some pseudo-perspective effects — can only be achieved by writing directly to VSRAM from an HInterrupt.
- **“Hard” screen splits.** A HUD that shouldn’t move while the rest of the screen scrolls, or a zone that uses a different plane configuration than the rest, is handled by reconfiguring the VDP exactly at the split line, something that by definition happens in the middle of the frame and doesn’t allow for a precomputed table.

The rest of this chapter builds an example combining both pieces: a fixed “sky” strip at the top and a “water” strip at the bottom, whose per-line offset is recalculated in real time, band by band, triggered by a real HInterrupt.

4. Full example: differential scroll by bands

The example divides the screen into bands 8 lines tall (to simplify the calculations, although HSCROLL_LINE would also allow single-line bands). Above a cutoff line, WATER_START, the background is shown with no additional offset. Below that line, each 8-line band receives a different horizontal offset, taken from a precomputed wave table, which also rotates by one step each frame to give a sense of motion: the classic water effect.

```
#include <genesis.h>

#define BAND_HEIGHT      8      // height of each band, in lines
#define WATER_START      144    // line where the water starts (multiple of
    ↪ BAND_HEIGHT)
#define WAVE_SIZE        32     // number of entries in the wave table

// Precomputed horizontal offset table (a low-amplitude wave).
// The values are in pixels; they could be generated with sin() at design time,
// but are left as constants to avoid depending on floating-point calculations.
static const s16 waveTable[WAVE_SIZE] =
{
    0,  6, 12, 17, 20, 22, 22, 20,
    17, 12,  6,  0, -6, -12, -17, -20,
   -22, -22, -20, -17, -12, -6,  0,  6,
    12, 17, 20, 22, 22, 20, 17, 12
};

static u16 rasterLine; // screen line about to be drawn
static u16 waveFrame;  // wave phase; advances one step per frame

// HInterrupt handler: runs once every BAND_HEIGHT lines.
// Must be as short as possible: no calls to heavy functions,
// no DMA, nothing that could take longer than the time available before the
// VDP starts drawing the next line.
HINTERRUPT_CALLBACK hintHandler(void)
```

```

{
    if (rasterLine >= WATER_START)
    {
        u16 index = (((rasterLine - WATER_START) >> 3) + waveFrame) & (WAVE_SIZE -
↪ 1);
        s16 offset = waveTable[index];
        s16 values[BAND_HEIGHT] = { offset, offset, offset, offset,
                                     offset, offset, offset, offset };

        VDP_setHorizontalScrollLine(BG_B, rasterLine, values, BAND_HEIGHT, CPU);
    }

    rasterLine += BAND_HEIGHT;
}

// Runs once per frame, during the vblank: resets the line counter
// for the frame that's starting and advances the wave phase.
void onVBlank(void)
{
    rasterLine = 0;
    waveFrame = (waveFrame + 1) & (WAVE_SIZE - 1);
}

int main(bool hardReset)
{
    // A background is assumed to already be loaded into BG_B, as explained in
    // chapter 7 and onward; the example focuses on the raster effect itself.

    VDP_setScrollingMode(HSCROLL_LINE, VSCROLL_PLANE);

    rasterLine = 0;
    waveFrame = 0;

    SYS_setVBlankCallback(onVBlank);
    SYS_setHIntCallback(hintHandler);
    VDP_setHIntCounter(BAND_HEIGHT - 1); // fires every 8 lines
    VDP_setHInterrupt(TRUE);

    while (TRUE)
    {
        // The rest of the game logic can go here, completely unrelated
        // to the raster effect: once configured, it runs on its own via
        // interrupts, without the main loop needing to deal with it.

        SYS_doVBlankProcess();
    }

    return 0;
}

```

A few of the example's decisions deserve a separate explanation:

- The line counter, `rasterLine`, is reset in the vblank callback (`onVBlank()`), and not at any other point in the main loop, because the vblank is the only moment you can be sure

corresponds exactly to the start of a new frame. If it were reset in the middle of the main loop, it could fall out of sync with the scan beam's actual position.

- The handler runs for **every** band on the screen, including the sky bands, but for those bands the `if` immediately discards the work. That's a bit of "wasted" CPU time on bands where nothing is needed, but it keeps the example simple; a more optimized version could reprogram `VDP_setHIntCounter()` inside the handler itself the first time it crosses `WATER_START`, so the interrupt stops firing entirely in the sky zone and only starts right where it's needed.
- The transfer method used inside the handler is CPU, never DMA, `DMA_QUEUE`, nor `DMA_QUEUE_COPY`. The next section explains why.

The visual result is what you'd expect from this kind of effect: the top sky strip stays fixed, with no offset at all, while below the cutoff line each 8-pixel band moves with a horizontal offset different from its neighbors, following the shape of the wave table. The progressive difference between adjacent bands is what creates the sense of a rippling surface, rather than blocks simply sliding each at its own speed with no relation to one another.

5. Cautions and limitations

The handler must be extremely fast. Between when an `HInterrupt` fires and when the VDP starts drawing the next line, there's very little margin: just the duration of the *horizontal blank*, the interval between the end of one visible line and the start of the next. With `BAND_HEIGHT = 8`, the example's handler has the width of eight full lines of margin before the next trigger, which is more than enough to write eight words into the scroll table. An effect configured to fire on **every line** (`VDP_setHIntCounter(0)`), by contrast, has only the margin of a single line, and the handler has to be trimmed down to the bare minimum: no loops, no calls to functions beyond what's strictly necessary, no calculations that aren't already resolved in a table.

Don't use DMA inside the handler. DMA, as explained in chapter 44, has its own setup cost and blocks the CPU until it finishes; triggering it from inside an `HInterrupt` adds that cost at exactly the moment with the least margin available, and can end up consuming several whole lines instead of a fraction of one. The same applies to `DMA_QUEUE` and `DMA_QUEUE_COPY`: queuing a transfer for "the next vblank" makes no sense inside a handler that runs in the middle of active drawing, still far from the next vblank. The only reasonable transfer method inside an `HInterrupt` is CPU, and in limited amounts: a few words, not a whole tileset.

Watch out for reentrancy. If the handler's work takes longer, for whatever reason, than the configured interval between triggers, the next trigger's moment can arrive before the previous one has finished. On the 68000, this doesn't necessarily produce a second, overlapping execution of the same handler, but it can cause the next trigger to be lost or delayed, with the visible result of a band that "jumps" a position or looks misaligned for one frame. The simplest way to avoid the problem isn't to program the handler to tolerate reentrancy, but to make sure, with margin to spare through careful measurement, that it never gets close to the available time limit.

Avoid leaving HInterrupts active in parts of the game that don't need them. When the screen using the raster effect ends — a level transition, a pause screen, a menu — it's best to turn off the interrupt with `VDP_setHInterrupt(FALSE)` rather than leaving it active with a handler that no longer makes sense for the current screen. Besides saving the CPU time it consumes, this avoids the risk of the old handler acting on a plane or scroll table that the new screen has already reconfigured for another purpose.

Variables shared between the handler and the rest of the program should stay simple. This chapter's example shares only two variables (`rasterLine`, `waveFrame`) between the `HIn-`

interrupt handler, the vblank callback, and potentially the rest of the game. The more shared variables there are, and the more complex the logic that updates them, the greater the risk that an interrupt occurs right in the middle of a half-finished update from the main code. Keeping that set of variables as small and simple as possible — integer counters, read-only tables — is, in practice, the simplest way to avoid that kind of intermittent bug.

Review exercises

1. Explain the difference between achieving a rippling water effect by uploading the full table via `VDP_setHorizontalScrollLine()` once per frame during the vblank, versus achieving it with an HInterrupt that recalculates each band in real time. In which cases is the second option specifically necessary?
2. Why can't the VDP's background color (`VDP_setBackgroundColor()`) be varied line by line with a precomputed table, unlike horizontal scroll? What mechanism is needed instead?
3. A teammate writes an HInterrupt handler that calls `PAL_setPalette()` with `DMA_QUEUE` to change palettes halfway down the screen. The effect doesn't work as expected, and on top of that, the game starts showing erratic behavior in other parts of the screen. Explain the two separate problems with that approach.
4. In the example from section 4, what would happen if `WATER_START` weren't a multiple of `BAND_HEIGHT`? Explain the specific problem it would cause with the alignment of the bands.
5. Why doesn't a plane's vertical scroll support a per-line granularity equivalent to `HSCROLL_LINE`, and what alternative is available for an effect that does need to vary the vertical offset line by line?

Chapter 64 — Saving Games - SRAM and Flash Save

The cartridge's battery-backed SRAM, the save declaration in the ROM header, SGDK's `sram.h` unit, and a modern battery-free alternative

Up until now, every time a game ended — whether by turning off the console or resetting the emulator — all the information that existed only in the 68000's working RAM disappeared: the level reached, the score, the state of every enemy. That's exactly what you'd expect from an ordinary variable, but it's a real problem for any game that aspires to be played across more than one session. This chapter tackles that concrete question: how to make a handful of bytes survive the console being turned off, using the same hardware mechanism that made cartridges like *Phantasy Star IV* or *Sonic 3* famous, and the layer SGDK provides on top of it.

It's a self-contained topic: it doesn't depend on the previous chapter of this Part X (dedicated to horizontal interrupts), but instead picks up ideas from further back in the manual, in particular the use of struct to group related data (Chapter 53) and the level catalog built as permanent data in Part IX.

Prerequisites

Before continuing, it helps to have:

- Familiarity with C structs for grouping data, as presented in Chapter 53 (“C structs applied to game entities”).
- Familiarity with the level catalog built as game data in Part IX (for example, an array or enum that identifies each level by an index), which this chapter revisits as the typical piece of data to persist between sessions.
- A general sense of how SGDK organizes a project and builds a ROM (Chapter 5), since this chapter modifies the ROM header.

1. What cartridge SRAM is, and why it exists

The 68000's working memory (the system RAM) loses its contents as soon as power is cut: it's volatile memory, and there's no way around that with the original Mega Drive hardware. For a game to remember anything from one session to the next, it needs a different kind of memory — one that keeps its contents even without external power.

The solution adopted by many cartridges of the era — not all of them, since it added manufacturing cost — was to include inside the cartridge itself an **SRAM** (static RAM) chip backed by a small lithium battery soldered to the board. While the console is powered on, that SRAM is read and written just like any other mapped memory region; when the console is turned

off, the battery keeps the chip powered with minimal consumption, and its contents persist. When the console is turned on again, the SRAM comes back with the same data it held the last time it was written.

That battery is also the cause of a problem well known to anyone who has played original cartridges decades after they were manufactured: once it runs out, the SRAM stops retaining data, and saved games are lost as soon as power is cut. It's a physical limitation of the mechanism, not something software can work around, and it's part of why section 6 of this chapter covers a modern alternative that doesn't depend on any battery.

Cartridge SRAM belongs to the cartridge, not the console: every cartridge with save support brings its own chip and its own battery. That matters because it means the available space and its exact behavior depend on how the cartridge's hardware is built, not on the Mega Drive itself.

2. Declaring SRAM usage in the ROM header

The Mega Drive's memory map reserves an entire region for a cartridge, if it has SRAM, to expose it there. But nothing requires a cartridge to have it: that's why the ROM includes, within its header (the metadata area at the start of the ROM already mentioned in passing in earlier chapters for other fields, such as the video region), a specific block declaring whether the game uses SRAM, in what address range, and of what type.

That declaration isn't a cosmetic detail: it's the information consulted both by real SRAM-equipped cartridge hardware and by modern emulators and flash carts to decide whether they should reserve persistent space for that particular game and at what address to expect the accesses. Without a correct declaration in the header, a write to the SRAM region may fail to persist, or may have no effect at all, even if the code correctly calls the save functions.

SGDK exposes this configuration as a project-level directive, with no need to hand-edit header bytes. The usual approach is to define it in the project's configuration file (or, in simpler projects, with a macro at the top of `main.c`):

```
// Enables the SRAM block in the ROM header,  
// declaring a 64 KB range of 8-bit SRAM.  
#define SRAM_SIZE 0x10000  
#define SRAM_TYPE 0 // 0 = normal SRAM (byte-wide), no automatic backup
```

Conceptually, what matters more than the exact macro name are the two pieces of information that declaration fixes: **how much SRAM** the cartridge has (up to 64 KB in the simplest variant, addressed as bytes) and whether that SRAM is **battery-backed** or not. An emulator that respects the header creates, the first time it runs that ROM, an associated save file (typically with a `.srm` extension) that reproduces the behavior of that battery: it persists between emulator runs just as the battery persists between power cycles on real hardware.

3. SGDK's `sram.h` unit

With SRAM already declared in the header, SGDK offers a dedicated unit, `sram.h`, for reading and writing it from game code without having to calculate memory addresses by hand or worry about the exact mapping the hardware uses to expose it.

Before any access, it must be explicitly enabled:

```
void SRAM_enable(void);
```

SRAM_enable() enables access to the cartridge's SRAM area; it should be called once, typically at the start of main(), before any read or write. There's also a counterpart to disable access once a save operation is done, leaving the bus in the proper state for the rest of the game:

```
void SRAM_disable(void);
```

With SRAM enabled, the basic read and write functions work over an offset within that region, not over an absolute memory address:

```
u8  SRAM_readByte(u32 offset);
u16 SRAM_readWord(u32 offset);
void SRAM_writeByte(u32 offset, u8 value);
void SRAM_writeWord(u32 offset, u16 value);
```

For blocks larger than a single byte or word — like the progress struct built in section 4 — a function that copies an entire array in one go is more convenient than iterating byte by byte from game code:

```
void SRAM_readAll(u8 *dest, u32 offset, u32 len);
void SRAM_writeAll(const u8 *src, u32 offset, u32 len);
```

All of these functions assume SRAM_enable() has already been called. An important detail of the real hardware, which SGDK handles internally but which is worth keeping in mind: on many cartridges, SRAM is mapped as 8-bit memory on a 16-bit bus, which means every “word” access actually occupies two address positions with only one useful byte in each. This is exactly the kind of detail that makes it worth relying on the sram.h functions instead of computing offsets by hand: the unit already resolves it consistently.

4. Saving and loading a progress structure

The most common use case isn't saving loose bytes, but a small structure holding the state a game needs to remember. Picking up the level catalog from Part IX, a reasonable example is saving the highest level reached and the highest score achieved:

```
typedef struct
{
    u16 highestLevel; // index of the highest level reached, within the level
    ↪ catalog
    u32 highScore;    // highest score recorded
    u8  checksum;     // simple integrity check, computed on save
} GameProgress;
```

The checksum field is the missing piece before trusting any data read back from SRAM. SRAM with a battery that hasn't run out is reliable, but a weak battery, SRAM that has never been written before (filled with an arbitrary factory value, not necessarily zeros), or, on a flash cart, a corrupted save file, are all real scenarios. Before using the data read back to decide which level to start on, it's worth computing a simple checksum over the rest of the structure and comparing it against the one saved alongside the data:

```
// Computes a simple checksum: the sum of every byte in progress,
// excluding the checksum field itself, truncated to 8 bits.
static u8 compute_checksum(const GameProgress *progress)
{
    const u8 *bytes = (const u8 *) progress;
    u16 sum = 0;
```

```

    for (u16 i = 0; i < sizeof(GameProgress) - sizeof(progress->checksum); i++)
        sum += bytes[i];

    return (u8) sum;
}

```

With that helper function, saving progress amounts to filling in the checksum and dumping the whole structure to SRAM:

```

#define SAVE_OFFSET 0

void save_progress(u16 highestLevel, u32 highScore)
{
    GameProgress progress;

    progress.highestLevel = highestLevel;
    progress.highScore = highScore;
    progress.checksum = 0; // filled in afterward, once the rest of the fields are
    ↪ set
    progress.checksum = compute_checksum(&progress);

    SRAM_enable();
    SRAM_writeAll((const u8 *) &progress, SAVE_OFFSET, sizeof(GameProgress));
    SRAM_disable();
}

```

Loading progress follows the reverse path, but it should never blindly trust what it reads: if the checksum doesn't match, the right move is to treat the SRAM as empty and return initial values, rather than starting the game with a corrupted highestLevel that could, in the worst case, push the level catalog out of range:

```

bool load_progress(GameProgress *out)
{
    GameProgress progress;

    SRAM_enable();
    SRAM_readAll((u8 *) &progress, SAVE_OFFSET, sizeof(GameProgress));
    SRAM_disable();

    u8 expected = compute_checksum(&progress);

    if (progress.checksum != expected)
        return FALSE; // missing or corrupted data: no valid saved game

    *out = progress;

    return TRUE;
}

```

The typical call, when the game starts, is then:

```

GameProgress progress;

if (load_progress(&progress))

```

```

{
    // There's a valid saved game: use progress.highestLevel
    // to jump directly to the corresponding level in the catalog.
}
else
{
    // No saved game (or it was corrupted): start
    // from the first level in the catalog with a score of zero.
    progress.highestLevel = 0;
    progress.highScore = 0;
}

```

Saving again — for example, every time the player completes a level or beats their high score — is simply a matter of calling `save_progress()` again with the updated values. No prior “erase” operation is needed: writing over the same `SAVE_OFFSET` directly replaces the previous contents.

This same scheme — struct with a checksum, `SRAM_writeAll()/SRAM_readAll()` over a fixed offset — scales without changes to a richer progress structure (multiple save slots, inventory, options settings): each additional piece of data is just one more field in the struct, and each independent save slot is simply a different `SAVE_OFFSET`, spaced apart by `sizeof(GameProgress)`.

In terms of memory, the cartridge ends up organized into two clearly separated regions: the ROM, read-only, where the game’s code and data live, and the SRAM region, mapped separately and backed by its battery. Within that SRAM region, `GameProgress` occupies a fixed block starting at `SAVE_OFFSET`: the struct’s three fields (`highestLevel`, `highScore`, and `checksum`) sit one after another, in the same order in which they were defined, together occupying `sizeof(GameProgress)` bytes.

5. Before reading: checking whether SRAM is available

Not every environment a ROM can run on has SRAM configured the same way: an emulator that hasn’t yet initialized its associated `.srm` file, or factory-fresh SRAM that has never been written, returns arbitrary bytes (often all `0xFF`) instead of zeros. This is exactly the case the checksum from the previous section is meant to catch: if nothing was ever saved, the initial read won’t match any valid checksum, and `load_progress()` returns `FALSE`, sending the game down the “no saved game” path instead of interpreting factory bytes as if they were a real level.

That’s why it’s worth resisting the temptation to skip the checksum check “since the SRAM is already declared in the header”: the declaration in the ROM header guarantees that the space exists and that an emulator or flash cart will persist it, but it guarantees absolutely nothing about whether what’s there at any given moment is actually a valid `GameProgress` written by this same game.

6. Flash save: the battery-free alternative

Everything above describes the historical mechanism, designed for a physical cartridge with a soldered battery. For those developing today and testing mainly on modern flash carts — devices like EverDrive, which load a ROM from an SD card and run it on real hardware without needing to manufacture a cartridge for every version — there’s a considerably simpler alternative to work with from the development side: **flash save**.

Instead of relying on a battery-backed SRAM chip, these flash carts offer their own persistent save mechanism based on flash memory (the same technology used in SD cards), managed by the flash cart itself rather than by a discrete chip on the cartridge board. From the point of view of SGDK and game code, the practical difference is usually minimal or nonexistent: the same `sram.h` unit, with the same size declaration in the ROM header, is the interface these devices also expect to find, so the same `save_progress()/load_progress()` code from section 4 works unchanged both on an emulator with simulated SRAM and on a flash cart with real flash-based saving.

The advantage for modern development is twofold: there's no need to manufacture or program a physical battery-backed cartridge to test saving on real hardware, and there's no risk of a battery running out between one testing session and the next. The downside is that it remains, ultimately, a feature of the flash cart device used for testing, not of the Mega Drive itself: a commercial cartridge actually manufactured for sale still needs, if it's going to offer persistent saving, to deal with the battery-backed SRAM mechanism (or some later variant, such as the non-volatile memory chips that replaced the battery in later generations of cartridges) from section 1 onward.

Review exercises

1. Explain why cartridge SRAM needs to be declared in the ROM header, and what would happen if a game writes data to that region without having correctly declared its size and type.
2. Why isn't it enough to directly read the fields of a `GameProgress` saved in SRAM and use them as-is, without checking a checksum first? Given a cartridge with a dead battery, describe what the game would see on startup if `load_progress()` didn't check the checksum.
3. A teammate proposes saving `highestLevel` and `highScore` as two independent calls to `SRAM_writeWord()/SRAM_writeByte()`, instead of building a struct `GameProgress` and using a block function. What concrete advantage does grouping everything into a struct saved in one shot with `SRAM_writeAll()` have instead?
4. From the point of view of the SGDK API, what do a cartridge with battery-backed SRAM and a modern flash cart with flash save have in common? What real difference persists between the two, beyond the game code?
5. Design the offset scheme needed to save three independent save slots using the same struct `GameProgress` from this chapter, so that each slot can be read and verified (with its own checksum) without affecting the other two.

Chapter 65 — Multitasking and Advanced SGDK 2.x Features

A brief look at recent SGDK additions that don't fit into any earlier chapter: cooperative tasks, dynamic memory, and resource compression

The previous chapters of this part covered specific, well-delimited topics: horizontal interrupts and their use for line effects, and SRAM handling for saved games. This chapter closes Part X with something different: a set of features that SGDK gradually added throughout its 2.x series (from version 2.00 onward) and that, being either more general or more recent, didn't have a natural place in any earlier chapter of the manual. There isn't a single unifying theme, but there is a common denominator: these are tools that solve problems that only show up once a project grows beyond a technical demo, and it's worth knowing about them, even just at an introductory level, before taking on a larger project of your own.

Prerequisites

- The main loop and `SYS_doVBlankProcess()`, introduced in Chapter 5.
- The concept of interrupts and the difference between code driven by hardware events and code that runs inside the main loop, revisited in the previous chapter on horizontal interrupts.
- A general sense of how SGDK organizes its resources in ROM (Chapter 39).
- Some familiarity with pointers and dynamic memory in C (`malloc/free` as a concept, even though SGDK doesn't use exactly those functions).

1. The problem that motivates cooperative multitasking

A good part of the code in a Mega Drive game is naturally written as a state machine: a variable indicates what state a character or sequence is in (walking, jumping, falling; or, in a cutscene, “showing text,” “waiting for the player to press a button,” “moving the camera”), and on every pass through the main loop a function checks that state and decides what to do. It's a pattern that works well and that has already been used implicitly in earlier chapters of this manual, but it becomes awkward to write by hand when a sequence has many chained steps that must happen one after another across several frames: a cutscene with dialogue, camera movements, and interspersed waits, for example, ends up turning into a state machine with dozens of cases, hard to read because the logical order of the sequence gets scattered across a switch and several control variables.

What in a normal desktop program would be solved with a separate thread — writing the sequence as a linear function, top to bottom, and letting the operating system handle splitting CPU time between that function and the rest of the program — isn't available in the

same way on the Mega Drive: there's no operating system with a thread scheduler, and the 68000 executes a single instruction stream at a time. Even so, SGDK's 2.x series introduced a mechanism that allows something similar within that constraint: a **cooperative task** system, designed for writing long sequences as linear code instead of as an explicit state machine.

2. What SGDK's cooperative multitasking is (and isn't)

It's worth clarifying up front what kind of "multitasking" this is, because the term can be misleading. This is not real parallel execution: the Mega Drive still has a single 68000 executing a single instruction at a time, just like in any other chapter of this manual. What SGDK offers is a **cooperative context-switching** mechanism: each "task" is a function with its own separately reserved stack, and SGDK's internal scheduler takes turns handing the processor to each task, saving and restoring the execution context (the 68000's registers and the stack pointer) when switching from one task to another. The difference from a hardware interrupt — like the ones seen in the previous chapter — is that this task switch isn't triggered by the hardware at an arbitrary instant; rather, it's decided by the task itself, voluntarily, by calling a function that hands control back to the scheduler. It's "cooperative" precisely for that reason: a task that never yields control blocks all the others, including the main loop, just as completely as an infinite loop without calls to `SYS_doVBlankProcess()` would.

This enables a coding style very different from the traditional state machine. A cutscene sequence that used to require a state variable and a switch with one case per step can be written, inside a task, as an ordinary function that proceeds from top to bottom, and that, at the points where it needs to "wait" — for an animation to finish, for a certain number of frames to pass, for the player to press a button — simply yields control and gets it back later, at the exact point where it had left off, once the wait condition is met.

The usual skeleton of this pattern, regardless of the exact name of each function, looks like this:

```
// Task that runs a multi-step intro sequence,  
// written as linear code instead of as a state machine.  
void introSequenceTask(void)  
{  
    showText("Welcome to the adventure");  
    waitFrames(60);           // yields control for 60 frames  
  
    moveCameraTo(200, 0);  
    waitCameraArrived();     // yields control until the camera arrives  
  
    showText("Press START to begin");  
    waitButtonPress(BUTTON_START);  
}
```

Each of the waiting functions (`waitFrames()`, `waitCameraArrived()`, `waitButtonPress()` in this example) is, internally, a loop that checks the condition and yields the turn back to the scheduler over and over while that condition isn't met, in a way similar to how the game's main loop yields its turn to the VDP by calling `SYS_doVBlankProcess()`. The difference from writing that same logic directly in the main loop is that, while the intro task is "waiting," the rest of the game — including the main loop itself, which can keep calling other tasks or continuing with its own logic — isn't blocked: the scheduler distributes time among all active tasks on every pass.

A task that performs a long computation without ever yielding control — for example, a loop with thousands of iterations and no intermediate call to the yield

mechanism — behaves exactly as badly as that same computation placed directly in the main loop without calling `SYS_doVBlankProcess()`: it blocks everything else for as long as it runs. Cooperative multitasking doesn't automatically distribute CPU time; it only offers an escape point that's more convenient to use than a manual state machine.

Each task also needs its own stack reserved in RAM in order to save its context when yielding control, and that RAM is a scarce resource on the Mega Drive (barely 64 KB in total). That's why the stack size for each task is usually an explicit parameter when creating it, and it's worth sizing it carefully: too small, and a sequence with nested function calls can silently overflow it and corrupt neighboring memory; too large, and several active tasks at once can exhaust the RAM available for the rest of the game. In practice, this mechanism is best reserved for sequences that genuinely benefit from being written as linear code — cutscenes, dialogue scripts, long boss attack patterns — rather than as a general replacement for frame-by-frame gameplay logic, which is still better handled with the main loop as used throughout the rest of this manual.

3. Improvements in dynamic memory handling

Another area where SGDK advanced throughout its 2.x series, without having had a dedicated chapter until now, is dynamic memory handling. SGDK has always offered `MEM_alloc()` and `MEM_free()` as equivalents of standard C's `malloc()/free()`, adapted to the Mega Drive's limited RAM, and it's the recommended way to allocate memory at runtime instead of using the standard C library functions directly.

Recent versions of this memory unit improved both the allocator's performance (less fragmentation when reusing freed blocks of similar size) and the tools for diagnosing memory issues during development: functions for finding out how much free memory remains available at a given moment, and for inspecting the heap's state, are particularly useful when debugging a crash that only appears after playing for several minutes — a typical symptom of a memory leak (code that allocates with `MEM_alloc()` and, along some code path, forgets to free it with `MEM_free()`).

```
// Before loading a new level's resources, it's worth  
// recording how much free memory remains, to detect leaks  
// by comparing this value from one playthrough to the next.  
u32 freeBefore = MEM_getFree();  
loadLevelResources();  
u32 freeAfter = MEM_getFree();
```

As a general rule of good practice, it's best to allocate with `MEM_alloc()` as few times as possible during active gameplay — ideally, all the dynamic memory a level needs is allocated once when it loads, and freed all at once when leaving it — rather than constantly allocating and freeing small blocks frame by frame, a pattern that fragments the heap over time even with a well-optimized allocator.

4. Resource compression and the resource compiler

SGDK's resource compiler, already introduced in Chapter 39 for converting images, tilesets, and maps into data embedded in the ROM, gained compression support for several resource types throughout the 2.x series. The underlying idea is simple: many graphics and sound resources have considerable internal redundancy (repeated areas of a tileset, silences in an audio track), and compressing them when building the ROM lets them take up less space on

the cartridge in exchange for a runtime decompression cost, which SGDK handles automatically with decompression functions provided by the library itself.

In practice, this is declared as an additional option in the resource file (the same `.res` format used since Chapter 39), specifying the desired compression method for a given resource instead of leaving it uncompressed. SGDK's own engine takes care of detecting, when loading that resource from ROM, that it's compressed, and of invoking the corresponding decompression routine before using it, transparently to code that already knew how to draw an image or play back a tileset without compression.

The practical result for the programmer is that a project can get much closer to a cartridge's space limit without having to manually reduce the quantity or quality of its graphics and sound resources, as long as the CPU time budget spent on decompression — relevant above all when decompressing a large resource during the loading of a new level — fits within what the game can afford at that moment, an analysis for which the same CPU load measurement tools presented in Chapter 49 come in handy.

Review exercises

1. Explain in your own words the difference between the cooperative multitasking SGDK offers and the real parallel execution of a multi-core operating system. Why does a task that never yields control end up blocking the entire game?
2. Rewrite the example sequence from Section 2 (`introSequenceTask`) as a traditional state machine (with a state variable and a switch). Compare the readability of both versions.
3. Why is the stack size assigned to each task an important parameter, and what can happen if it's underestimated?
4. What concrete advantage does compressing a game's graphics and sound resources offer when building the ROM, and what cost must be weighed in exchange? Which tool from an earlier chapter of this manual could be used to measure that cost in a real project?

Chapter 66 — Raylib as an Alternative for Porting Games to PC

What Raylib is, how it differs conceptually from SGDK, and why it can be the bridge to PC and other platforms

This chapter steps away from the Mega Drive, for once. The previous chapters taught how to program directly for the console's hardware, with all the particularities that implies: 16-color palettes, 8×8 tiles, a quantity of VRAM measured in a handful of kilobytes. This chapter introduces Raylib, a game development library for PC and other modern platforms, meant for anyone who—after having programmed for the Mega Drive with SGDK—wants to port their game to a more accessible platform, or simply program without the constraints of late-1980s hardware.

This is not an exhaustive technical chapter about Raylib: there's no need to cover its entire API to understand what it offers. The idea, rather, is to show the conceptual parallel between programming for the Mega Drive's fixed hardware and programming for a modern cross-platform library, and to verify, with a concrete code example, how much of a game's logic—structs, enums, state machines, simple physics—is actually independent of the target platform.

Prerequisites

Before continuing, it helps to have covered:

- A good part of this manual and a reasonable familiarity with SGDK, in order to appreciate the contrast this chapter presents.
- Chapter 30, “Palette Fades”: it is directly compared, later on, with the way Raylib achieves an equivalent effect.
- Chapter 31, “D-pad Controls”: the example in this chapter replicates, with Raylib, the same sprite movement via the joystick that was built there for the Mega Drive.

It is not necessary to install Raylib to read this chapter, though it is worth doing so to experiment with the example: it can be downloaded for free from its official site (raylib.com), with packages already prepared for Windows, Linux, and macOS.

1. Why consider a PC port

Almost anyone developing a new game for the Mega Drive, sooner or later, considers the possibility of also bringing it to PC or some modern console, simply because that makes it

accessible to far more people than those who still own, or emulate, the original hardware. Before settling on one, it's worth reviewing the available options.

One option is general-purpose game engines, such as Unreal Engine or Unity: engines designed for much more ambitious productions, whose power is disproportionate for a game in the style of the ones built in this manual—a platformer with 16-bit aesthetics. Another option, closer in spirit to programming directly for hardware, is to interact at a low level with the Windows API; it's an interesting path, but considerably more arduous than working with a library already designed for games. Raylib occupies a middle ground between those two extremes: it is not a game engine, but rather what its own documentation calls a “development library” (*a simple and easy-to-use library to enjoy videogames programming*), with a spirit quite similar to that of SGDK.

2. What Raylib is, and how similar it is to SGDK

Raylib is a free, cross-platform game development library, with its source code publicly available. Although it supports several programming languages through various bindings, the language that most of its community gathers around is C, the same one already used throughout this manual with SGDK. Anyone who has followed the previous chapters therefore doesn't need to learn a new language to experiment with Raylib, only a different library.

Raylib's official website is one of its strongest points: it includes a large number of already-solved examples—title screens, cameras, and many other common use cases—that can be used directly as a starting point, along with a reference sheet (*cheatsheet*) listing all the functions grouped by module, and a wiki with more extensive guides.

The most important conceptual difference between Raylib and SGDK has to do with how each one treats graphics. SGDK forces you to distinguish, at all times, whether something is a background or a sprite, because that distinction exists in the Mega Drive's hardware itself: a background is built from a tile map with its own VRAM and palette limits, and a sprite is a completely different hardware entity, with its own limits on size and on how many can appear on screen at once. Raylib makes no such distinction: both a background and a sprite are loaded the same way, as a generic 2D texture.

```
// In Raylib there is no structural difference between loading
// a background or a sprite: both are simply textures.
Texture2D background = LoadTexture("background.png");
Texture2D player_sprite = LoadTexture("player.png");
```

Going deeper. A good example to illustrate this difference is the screen fade-in, already covered in chapter 30. On the Mega Drive, dimming the screen to black means manipulating, color by color, the values stored in CRAM—the console's palette memory—as explained in that chapter via `PAL_fadeIn()`. In Raylib, the same effect is achieved by drawing a black rectangle the size of the entire screen, on top of everything else, and gradually varying its transparency until it becomes invisible:

```
// alpha starts at 1.0f (screen completely black)
// and is reduced frame by frame until it reaches 0.0f
DrawRectangle(0, 0, GetScreenWidth(), GetScreenHeight(),
    Fade(BLACK, alpha));
```

The visual result is equivalent to that of `PAL_fadeIn()`, but the path to achieve it is radically different: one depends on understanding and manipulating a specific, limited hardware resource (sixteen colors per palette, four palettes in total, 64 color slots in play); the other depends solely on a generic property—transparency, handled here through the `Fade()` func-

tion, which adjusts a color's alpha channel—that any modern graphics engine offers out of the box, with no hardware limits involved.

After programming for a good while for the Mega Drive's limited hardware, working with Raylib can feel, at first, almost like cheating. It's worth not taking that feeling too literally, though: Raylib's functions are all different from SGDK's, so there is a learning curve just like with any new tool. Just as with SGDK, Raylib doesn't hand you a finished game engine either: you still have to build your own, with your own game logic, your own systems, and your own code organization. What it does eliminate is much of the repetitive, uninteresting work (the *boilerplate*) that has to be written before getting to that actual game logic: opening a window, creating a main loop at a stable frame rate, loading an image, or playing a sound are each, in Raylib, a single function call.

3. The same movement, two platforms

The best way to see the parallel between SGDK and Raylib is to compare how each one solves the same problem: moving a character on screen based on what the player presses. Chapter 31 solved this for the Mega Drive with `JOY_readJoypad()` and the `BUTTON_*` constants:

```
static void handle_input(void)
{
    u16 value = JOY_readJoypad(JOY_1);

    if (value & BUTTON_RIGHT)
        player_x += 1;
    else if (value & BUTTON_LEFT)
        player_x -= 1;

    if (value & BUTTON_UP)
        player_y -= 1;
    else if (value & BUTTON_DOWN)
        player_y += 1;

    SPR_setPosition(player, player_x, player_y);
    SPR_update();
}
```

The equivalent version in Raylib needs no SGDK, ROM, or emulator: it's a complete PC program, compiled with any C compiler and run like any other executable on the operating system.

```
#include "raylib.h"

int main(void)
{
    const int screen_width = 320;
    const int screen_height = 240;

    InitWindow(screen_width, screen_height, "Movement example with Raylib");
    SetTargetFPS(60);

    // Player position, analogous to player_x/player_y in SGDK
    Vector2 player_pos = { 100.0f, 50.0f };
    Texture2D player_sprite = LoadTexture("player.png");
```

```

while (!WindowShouldClose())
{
    // --- Logic: input reading, equivalent to handle_input() ---
    if (IsKeyDown(KEY_RIGHT))
        player_pos.x += 1.0f;
    else if (IsKeyDown(KEY_LEFT))
        player_pos.x -= 1.0f;

    if (IsKeyDown(KEY_UP))
        player_pos.y -= 1.0f;
    else if (IsKeyDown(KEY_DOWN))
        player_pos.y += 1.0f;

    // --- Drawing: there is no separate SPR_update(), it is
    // drawn directly every frame inside Begin/EndDrawing ---
    BeginDrawing();
    ClearBackground(BLACK);
    DrawTextureV(player_sprite, player_pos, WHITE);
    EndDrawing();
}

UnloadTexture(player_sprite);
CloseWindow();

return 0;
}

```

The overall structure is the same in both cases: a position value that gets updated based on player input, and an if/else if per axis to allow diagonal movement without needing to chain all four directions together—exactly the same reasoning as in chapter 31. What changes is the source of the input—`JOY_readJoypad()` against the `BUTTON_*` constants in SGDK; `IsKeyDown()` against the `KEY_*` constants in Raylib—and the way the result is applied on screen: SGDK separates `SPR_setPosition()` from `SPR_update()`, while Raylib handles all the drawing inside a `BeginDrawing()/EndDrawing()` block on every pass of the main loop, which also takes care of clearing the previous frame with `ClearBackground()`. Raylib also supports gamepads via `IsGamepadButtonDown()`, which brings the parallel with reading a Mega Drive joystick even closer.

What’s interesting is to notice what *didn’t* change between one version and the other: the position value itself, the idea of checking one direction at a time, the separation between “read input” and “draw the result.” That part of the logic, the part that doesn’t depend on any hardware register or any platform-specific function, is essentially portable as-is between Mega Drive and PC. The same holds for more elaborate structures already seen in this manual: a struct that groups an enemy’s position, velocity, and state (chapter 53), an enum that identifies an enemy type or a game state (chapter 54 and chapter 57), or a function pointer used to resolve an enemy’s AI (chapter 56) are written in standard C, with no dependency on SGDK, and therefore compile just as well against Raylib. Porting a Mega Drive game to PC with Raylib largely consists of reusing that code almost unchanged, and replacing only the layer that interacts with the hardware—input reading, drawing, sound—with its Raylib equivalent.

4. Beyond PC: other platforms thanks to the community

Although Raylib was born mainly with PC in mind, its community has taken it further. On one hand, there are efforts to bring games made with Raylib to modern consoles, such as the Nintendo Switch. On the other hand, and more experimentally, various people in the community have been working on adapting Raylib to much older consoles, such as the Nintendo 64 and the Dreamcast.

At the time of writing this chapter, that work of adapting to older consoles hasn't yet reached the point of simply compiling and being done: additional work is needed to get there, and that work is under active development by the community. Since this is an ongoing external effort, it's worth checking the current state within the Raylib community before treating this description as final.



Figure 157: Raylib running on Dreamcast

One of the community's tests: Raylib running on Dreamcast, still in an experimental state.

Review exercises

1. Compare how SGDK and Raylib handle loading a background versus a sprite, and explain what specific limitation of the Mega Drive's hardware lies behind the distinction SGDK requires.
2. Explain how a fade to black is achieved in Raylib, and compare it with `PAL_fadeIn()` seen in chapter 30. What is gained and what is lost with each approach?
3. Take the `handle_input()` snippet from this chapter and add an explicit fourth diagonal direction (for example, moving faster if `KEY_RIGHT` and `KEY_UP` are pressed simultaneously). What, if anything, would change in the equivalent SGDK version from chapter 31?
4. Although Raylib removes many hardware constraints, you still need to build your own game engine. Why doesn't a library like Raylib offer that out of the box, unlike an engine such as Unity or Unreal Engine?

5. Review the data structures used in Part IX of this manual (entity structs, state enums, function pointers for AI). Which of them depend on some specific SGDK function, and which would compile, as-is, against Raylib?

Chapter 67 — Scope Note - Sega CD and 32X

Why this manual limits itself to the base Mega Drive hardware, and what paths exist for anyone who wants to continue on to those two peripherals

Everything developed throughout this manual — from the first program to the capstone project in Part IX — runs on the base Mega Drive hardware: the Motorola 68000 processor, the Z80 as sound coprocessor, the VDP, and the YM2612 and PSG chips described in Part I. Sega also released two peripherals that substantially expand that hardware: the Sega CD (known as the Mega-CD outside North America), which adds an optical disc drive, a second dedicated 68000 processor, and additional RAM; and the 32X, a cartridge that adds two SH-2 processors and its own graphics mode with greater color depth. Neither one is covered in this manual.

The reason is not that these are minor topics or of limited interest, but a matter of rigor: writing with the same depth and technical reliability maintained throughout the previous parts requires having reference material of the same caliber, verified with the same care as the SGDK 2.11 changelog or the `dma.h` documentation cited in Part VIII. Neither the Sega CD nor the 32X have that kind of backing within the material available for this project, and SGDK itself is designed first and foremost for the base Mega Drive hardware: support for these two peripherals, where it exists in the homebrew development tooling ecosystem, comes from separate kits and libraries, each with its own learning curve and its own set of hardware quirks.

Including a superficial chapter on either peripheral, without sufficient verification, would go against the standard upheld throughout this manual: when in doubt, it is better to flag a feature as pending verification — or, in this case, to leave an entire topic out of scope — than to offer information without the necessary reliability.

An open door, not a closed one. What you’ve learned here remains the necessary foundation for taking the leap to the Sega CD or the 32X down the line: applied C fundamentals, working with limited memory, organizing a project across multiple files, the struct patterns and data catalogs seen in the capstone project, and reading technical documentation as a working habit all carry over directly to any additional platform you might want to explore afterward. Anyone who reaches this point in the manual and wants to continue on to the Sega CD or the 32X will find, in today’s homebrew development scene, dedicated development kits and active communities ready to guide those first steps, in the same way this manual supported your first steps on the base hardware.

This manual continues, in the next part, with a final project that does carry a complete pipeline from start to finish on the base Mega Drive hardware, closing out the journey begun in the first chapter.

Review exercises

1. Name at least two hardware elements that the Sega CD adds on top of the base Mega Drive, and two that the 32X adds.
2. Explain, in your own words, why this manual prefers to leave a topic out of scope rather than treat it with less rigor than the rest of the chapters.
3. Of everything learned in the previous parts of this manual, pick three concrete concepts you consider most transferable to a different hardware platform, and justify your choice.

Chapter 68 — Final Project - BOINGMD, a Pong Clone from Start to Finish

A complete, playable Mega Drive game, built step by step with the tools presented throughout this manual

This manual began, in its first chapter, by describing the hardware of a console from 1988, and ended, in the previous part, by pointing out the limits of what SGDK does and does not cover. Between those two extremes, sixty-seven chapters of tools accumulated: data types and pointers, planes and palettes, sprites and animation, sound, collision physics, DMA and VRAM, and an entire project featuring enemies with their own artificial intelligence. This final chapter closes the manual with a different goal than Part IX: instead of leaving an open project as a base for further extension, it builds a small but complete game, playable from start to finish, to demonstrate that all those tools coexist without friction in a single real project.

The game is called BOINGMD, a clone of the classic Pong (1972) set in a simple aesthetic of a ball bouncing between two paddles, inspired by the same spirit as the “Boing” game often used as a first introductory project on other platforms. It is not a game with multiple levels or the complexity of the Part IX project: it is, deliberately, the bare minimum needed for a player to sit down, play a two-minute match, and feel that the game is complete, with a menu, a win condition, sound, and persistence of the high score between play sessions.

Prerequisites

This chapter assumes the entire manual up to this point. In particular, it directly relies on:

- Sprites, animation, and AABB collisions (Parts III and VII).
- Reading the joypad with `J0Y_setEventHandler()` (Part IV).
- Playing sound effects with `rescomp` (Parts V and VI).
- The state machine pattern with `enum` and `switch`, as applied in the chapter on game states in Part IX.
- Saving game data with SRAM or flash save, from the previous chapter, to preserve the high score between sessions.
- A simple HUD with `VDP_drawText()`, along the lines suggested at the close of Part IX.

1. The game design in one sentence

Two vertical paddles, one at each end of the screen, move up and down; a ball bounces between them and off the ceiling and floor; each time the ball crosses one end without the

corresponding paddle intercepting it, the player on the opposite side scores a point; whoever reaches a target score first wins the match. The player controls the left paddle with the D-pad; the right paddle is controlled by a simple artificial intelligence, which chases the ball's height with a deliberate margin of error so the match remains winnable.

Reducing the design to a single sentence, before writing a single line of code, is a discipline applicable to any project of your own: if the game cannot be summarized this way, it probably isn't defined well enough yet to start coding.

2. Project structure

Following the organizational criteria seen in the chapter on header files, the project is divided into small pieces, each with a clear responsibility:

```
boingmd/
├── src/
│   ├── main.c
│   ├── paddle.c
│   ├── ball.c
│   ├── game.c
│   └── save.c
├── inc/
│   ├── paddle.h
│   ├── ball.h
│   ├── game.h
│   └── save.h
└── res/
    ├── resources.res
    ├── sprites/
    └── sounds/
```

`game.h` holds the state machine and the shared variables (scores, current state); `paddle.c` and `ball.c` each encapsulate the logic for their own entity; and `save.c` isolates all interaction with SRAM, so the rest of the project only needs to call `load_high_score()` and `save_high_score()` without knowing the details of the low-level API.

3. The game's state machine

The entire lifecycle of a match is resolved with three states, following the same pattern that already organized the level transitions in the Part IX project:

```
// inc/game.h
#ifndef GAME_H
#define GAME_H

#include <genesis.h>

enum GameStates {
    STATE_MENU,
    STATE_PLAYING,
    STATE_GAME_OVER
};

#define WINNING_SCORE 5
```

```

extern u8 current_state;
extern u8 player_score;
extern u8 cpu_score;
extern u16 high_score;

```

```

void game_init(void);
void game_update(void);

```

```

#endif // GAME_H

```

```

// src/game.c

```

```

#include "game.h"
#include "paddle.h"
#include "ball.h"
#include "save.h"

```

```

u8 current_state = STATE_MENU;
u8 player_score = 0;
u8 cpu_score = 0;
u16 high_score = 0;

```

```

void game_init(void) {
    high_score = load_high_score();

    VDP_drawText("BOINGMD", 16, 10);
    VDP_drawText("PRESS START", 12, 13);
}

```

```

static void start_new_match(void) {
    player_score = 0;
    cpu_score = 0;
    VDP_clearTextArea(0, 0, 40, 28);

    paddle_reset();
    ball_reset();

    current_state = STATE_PLAYING;
}

```

```

static void end_match(void) {
    current_state = STATE_GAME_OVER;

    if (player_score > high_score) {
        high_score = player_score;
        save_high_score(high_score);
    }

    char buffer[20];
    sprintf(buffer, "HIGH SCORE: %d", high_score);
    VDP_drawText("GAME OVER", 11, 10);
    VDP_drawText(buffer, 14, 12);
    VDP_drawText("PRESS START", 12, 15);
}

```

```

}

void game_update(void) {
    switch (current_state) {
        case STATE_MENU:
            // Just waits for the Start button, handled in joyEvent()
            break;

        case STATE_PLAYING:
            paddle_update();
            ball_update();

            if (player_score >= WINNING_SCORE || cpu_score >= WINNING_SCORE) {
                end_match();
            }
            break;

        case STATE_GAME_OVER:
            // Just waits for the Start button, handled in joyEvent()
            break;
    }
}

void joyEvent(u16 joy, u16 changed, u16 state) {
    if (changed & state & BUTTON_START) {
        if (current_state == STATE_MENU || current_state == STATE_GAME_OVER) {
            start_new_match();
        }
    }
}

```

With just three states and a simple win condition, this machine is enough to give the game a sense of a complete match, with a clear beginning and end—something the Part IX project deliberately left unresolved.

4. The paddles: sprites, human control, and an AI with a margin of error

Each paddle is a vertical sprite with movement restricted to the Y axis, within a range that keeps it from leaving the screen:

```

// inc/paddle.h
#ifndef PADDLE_H
#define PADDLE_H

#include <genesis.h>

#define PADDLE_SPEED    FIX32(2)
#define PADDLE_MIN_Y    16
#define PADDLE_MAX_Y    192

extern Sprite *player_paddle;

```

```
extern Sprite *cpu_paddle;
extern fix32 player_y;
extern fix32 cpu_y;
```

```
void paddle_reset(void);
void paddle_update(void);
```

```
#endif // PADDLE_H
```

```
// src/paddle.c
```

```
#include "paddle.h"
#include "ball.h"
#include "resources.h"
```

```
Sprite *player_paddle;
Sprite *cpu_paddle;
fix32 player_y;
fix32 cpu_y;
```

```
void paddle_reset(void) {
    player_y = FIX32(104);
    cpu_y = FIX32(104);

    player_paddle = SPR_addSprite(&paddle_sprite, 16, 96,
        TILE_ATTR(PAL0, FALSE, FALSE, FALSE));
    cpu_paddle = SPR_addSprite(&paddle_sprite, 296, 96,
        TILE_ATTR(PAL0, FALSE, FALSE, FALSE));
}
```

```
static void move_player(void) {
    u16 joy_state = JOY_readJoypad(JOY_1);

    if (joy_state & BUTTON_UP)    player_y -= PADDLE_SPEED;
    if (joy_state & BUTTON_DOWN)  player_y += PADDLE_SPEED;

    if (player_y < FIX32(PADDLE_MIN_Y)) player_y = FIX32(PADDLE_MIN_Y);
    if (player_y > FIX32(PADDLE_MAX_Y)) player_y = FIX32(PADDLE_MAX_Y);

    SPR_setPosition(player_paddle, 16, F32_toInt(player_y));
}
```

```
// Chases the ball's height with a deliberate margin of error,
// so the AI isn't impossible to beat
```

```
static void move_cpu(void) {
    fix32 target_y = ball_get_y();
    fix32 error_margin = FIX32(6);

    if (cpu_y + error_margin < target_y)    cpu_y += PADDLE_SPEED;
    else if (cpu_y - error_margin > target_y) cpu_y -= PADDLE_SPEED;

    if (cpu_y < FIX32(PADDLE_MIN_Y)) cpu_y = FIX32(PADDLE_MIN_Y);
    if (cpu_y > FIX32(PADDLE_MAX_Y)) cpu_y = FIX32(PADDLE_MAX_Y);
}
```

```

    SPR_setPosition(cpu_paddle, 296, F32_toInt(cpu_y));
}

void paddle_update(void) {
    move_player();
    move_cpu();
}

```

The AI's margin of error (`error_margin`) is the most important piece of this section: without it, an AI chasing the ball with perfect precision would, in practice, be unbeatable. Adjusting that single value together with `PADDLE_SPEED` is the most direct way to calibrate the game's difficulty without touching any other part of the code. The underlying principle can be reused in any project of your own: isolate balance values in named constants, rather than scattering them as magic numbers throughout the logic.

5. The ball: movement, bouncing, and collision with the paddles

The ball moves in a straight line until it hits something, at which point it reverses the corresponding component of its velocity, applying the same kind of AABB collision seen in Part VII:

```

// src/ball.c
#include "ball.h"
#include "paddle.h"
#include "game.h"
#include "resources.h"
#include "sfx.h"

static Sprite *ball_sprite_obj;
static fix32 ball_x, ball_y;
static fix32 ball_vel_x, ball_vel_y;

void ball_reset(void) {
    ball_x = FIX32(160);
    ball_y = FIX32(112);
    ball_vel_x = FIX32(1.5);
    ball_vel_y = FIX32(1);

    ball_sprite_obj = SPR_addSprite(&ball_sprite, F32_toInt(ball_x),
                                     F32_toInt(ball_y), TILE_ATTR(PAL0, FALSE, FALSE, FALSE));
}

fix32 ball_get_y(void) {
    return ball_y;
}

static bool check_paddle_collision(fix32 paddle_x, fix32 paddle_y) {
    bool overlap_x = (ball_x + FIX32(8) > paddle_x) && (ball_x < paddle_x +
        ↪ FIX32(4));
    bool overlap_y = (ball_y + FIX32(8) > paddle_y) && (ball_y < paddle_y +
        ↪ FIX32(32));
}

```

```

    return overlap_x && overlap_y;
}

void ball_update(void) {
    ball_x += ball_vel_x;
    ball_y += ball_vel_y;

    // Bounce off the ceiling and floor
    if (ball_y <= FIX32(16) || ball_y >= FIX32(216)) {
        ball_vel_y = -ball_vel_y;
        SND_startPlay_PCM(&bounce_sfx, SOUND_PCM_CH_AUTO);
    }

    // Bounce off the player's paddle
    if (ball_vel_x < 0 && check_paddle_collision(FIX32(16), player_y)) {
        ball_vel_x = -ball_vel_x;
        SND_startPlay_PCM(&bounce_sfx, SOUND_PCM_CH_AUTO);
    }

    // Bounce off the AI's paddle
    if (ball_vel_x > 0 && check_paddle_collision(FIX32(296), cpu_y)) {
        ball_vel_x = -ball_vel_x;
        SND_startPlay_PCM(&bounce_sfx, SOUND_PCM_CH_AUTO);
    }

    // Point for the AI: the ball crossed the left edge
    if (ball_x < FIX32(0)) {
        cpu_score++;
        SND_startPlay_PCM(&score_sfx, SOUND_PCM_CH_AUTO);
        ball_reset();
    }

    // Point for the player: the ball crossed the right edge
    if (ball_x > FIX32(320)) {
        player_score++;
        SND_startPlay_PCM(&score_sfx, SOUND_PCM_CH_AUTO);
        ball_reset();
    }

    SPR_setPosition(ball_sprite_obj, F32_toInt(ball_x), F32_toInt(ball_y));
}

```

Every bounce—off the ceiling, the floor, or a paddle—triggers the same short sound effect (`bounce_sfx`), played with `SND_startPlay_PCM()` on a free channel indicated with `SOUND_PCM_CH_AUTO`, while scoring a point triggers a different effect (`score_sfx`), following the rescomp mechanism for declaring both resources already covered in Part VI:

```

// res/resources.res
SPRITE paddle_sprite "sprites/paddle.png" 4 32 FAST
SPRITE ball_sprite   "sprites/ball.png"   8 8  FAST

WAV bounce_sfx "sounds/bounce.wav" XGM
WAV score_sfx  "sounds/score.wav"  XGM

```

6. The HUD: on-screen score without additional sprites

With the player and the AI now scoring points, they still need to be shown on screen. Following the suggestion at the close of Part IX, this game's HUD is resolved entirely with text over a background plane, without any sprites of its own, thereby avoiding any conflict with `SPR_reset()` or other sprite operations in the game:

```
// Inside game_update(), called once per frame during STATE_PLAYING
static void update_hud(void) {
    char score_text[10];
    sprintf(score_text, "%d - %d", player_score, cpu_score);
    VDP_drawText(score_text, 18, 2);
}
```

Updating the HUD every frame, instead of only when the score changes, is needlessly costly when viewed with the same criteria from Part VIII on profiling: `update_hud()` is best called only inside `ball_update()`, right after incrementing `player_score` or `cpu_score`, and not on every pass of the main loop.

7. Saving the high score

`save.c` isolates all interaction with SRAM behind two simple functions, directly applying the save mechanism with integrity checking explained in the previous chapter:

```
// src/save.c
#include "save.h"

#define SAVE_MAGIC 0x42 // Control value to detect uninitialized SRAM

u16 load_high_score(void) {
    SRAM_enable();

    u8 magic = SRAM_readByte(0);
    u16 score = 0;

    if (magic == SAVE_MAGIC) {
        score = SRAM_readWord(1);
    }

    SRAM_disable();
    return score;
}

void save_high_score(u16 score) {
    SRAM_enable();
    SRAM_writeByte(0, SAVE_MAGIC);
    SRAM_writeWord(1, score);
    SRAM_disable();
}
```

If `SRAM_readByte(0)` does not return the expected control value, `load_high_score()` assumes the SRAM was never written by this game—for example, the first time it runs on a new console or a new emulator—and returns a high score of zero instead of a meaningless value read from uninitialized memory.

8. main.c: bringing all the pieces together

```
// src/main.c
#include <genesis.h>
#include "game.h"
#include "paddle.h"
#include "ball.h"

int main(bool hard_reset) {
    JOY_init();
    JOY_setEventHandler(joyEvent);
    SPR_init();

    game_init();

    while (TRUE) {
        game_update();
        SPR_update();
        SYS_doVBlankProcess();
    }

    return 0;
}
```

Compared to what it would have been to write the entire game inside a single file without the organization presented throughout this manual, `main.c` ends up being very short in this finished project: each responsibility—game state, paddles, ball, saving—lives in its own pair of files, and `main.c` is limited to initializing the hardware once and delegating the rest to the functions exposed by each module.

9. What's deliberately left out

BOINGMD is a playable game, but it does not aim to be an example of every technique in this manual: it does not use queued DMA because its graphical resources are minimal and it doesn't need it, it has no multiple levels and doesn't require the level catalog from Part IX, and its AI is deliberately simple because a fixed margin of error is enough for a Pong design. Each of these omissions is a design decision, not a technical limitation: the tools for solving more complex problems—queued DMA, data catalogs, function pointers for more elaborate AI—are already documented in earlier parts of this manual, ready to be applied the day a project of your own needs them.

Review exercises

1. Modify `error_margin` in `move_cpu()` and play several matches with different values. Describe how the perceived difficulty changes and at what point the AI stops being a reasonable challenge.
2. `check_paddle_collision()` uses the same paddle dimensions (4×32 pixels) for both players. What would you change in the code if you wanted the human player's paddle to be narrower than the AI's, as an additional difficulty mechanism?
3. Explain why `update_hud()` is called only when the score changes and not on every pass of the main loop, relating your answer to what was covered in Part VIII on profiling and

frame budget.

4. What would happen if `load_high_score()` did not check the `SAVE_MAGIC` value before reading the saved score? Describe a concrete scenario in which this check prevents an error visible to the player.
5. Propose a concrete extension for `BOINGMD` (for example, a second human player, a power-up that speeds up the ball, or a pause screen) and describe, in broad strokes, which files in the project you would need to touch to implement it.

Note on the Use of Audiovisual Material

This manual is distributed freely and free of charge, without any commercial purpose, and may be copied, modified, and redistributed by anyone who wishes to do so.

Several of the screenshots illustrating the chapters on graphics, sprites, sound, collision physics, and the action-game capstone project come from the video series *Pigsy's Retro Game Dev Tutorials*, credited and linked in the Acknowledgments section. These screenshots are included solely for educational purposes, in isolated fragments that illustrate concepts already explained in the text, without ever substituting for the original video content or implying that its author participated in, endorsed, or sponsored the creation of this manual.

Repeated attempts were made, through different channels, to obtain the author's express authorization to use this material prior to publishing this manual, without receiving a response as of this edition. This manual will continue attempting that contact, and the Acknowledgments section includes the official channels of *Pigsy's Retro Game Dev Tutorials* for anyone who wants to learn more about the original work, follow it, or support it directly.

If the author of that material, or anyone else whose work is reflected in this manual, believes that a particular screenshot should not be included, or would prefer a different form of attribution, they may request its removal or modification by writing to gustavovalenzuela.dev@gmail.com; the request will be honored and that part of the content will be withdrawn or adjusted in the manual's next revision.